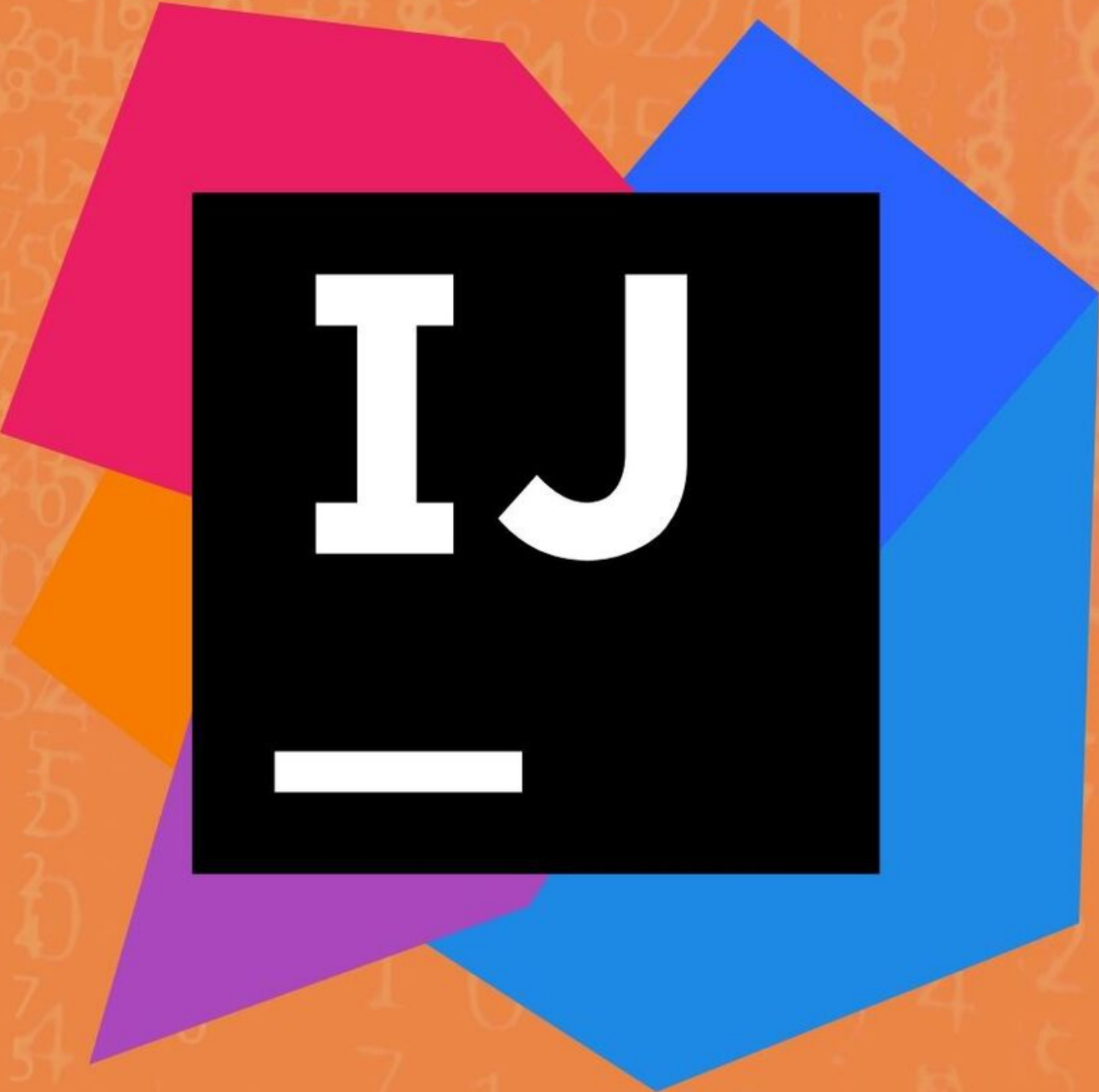


Quick & Easy Learning



Learn IntelliJ IDEA

The IntelliJ IDEA logo, which consists of a black square containing the white letters 'IJ' and a white horizontal bar below them. This square is set against a background of several overlapping, semi-transparent geometric shapes in magenta, blue, orange, and purple.

IJ
—

Copyright © All rights reserved

No part of this book may be reproduced, or stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without express written permission of the publisher.

TABLE OF CONTENT

[INTELLIJ IDEA OVERVIEW](#)

[INSTALL INTELLIJ IDEA](#)

[RUN INTELLIJ IDEA FOR THE FIRST TIME](#)

[REGISTER INTELLIJ IDEAUULTIMATE](#)

[UPDATE INTELLIJ IDEA](#)

[UNINSTALL INTELLIJ IDEA](#)

[OVERVIEW OF THE USER INTERFACE](#)

[CREATE YOUR FIRST JAVA APPLICATION](#)

[INTELLIJ IDEA KEYBOARD SHORTCUTS](#)

[MANAGE PLUGINS](#)

[WORK OFFLINE](#)

[ACCESSIBILITY](#)

[SUPPORT AND ASSISTANCE](#)

[INTELLIJ IDEA PRO TIPS](#)

[MIGRATE FROM ECLIPSE TO INTELLIJ IDEA](#)

[IMPORT A PROJECT FROM ECLIPSE](#)

[EXPORT AN INTELLIJ IDEA PROJECT TO ECLIPSE](#)

[NETBEANS](#)

[INTELLIJ IDEA VS NETBEANS TERMINOLOGY](#)

[CONFIGURING THE IDE](#)

[TOOL WINDOWS](#)

[ARRANGE TOOL WINDOWS](#)

[TOOL WINDOW VIEW MODES](#)

[SPEED SEARCH IN TOOL WINDOWS](#)

[MENUS AND TOOLBARS](#)

[USER INTERFACE THEMES](#)

[IDE VIEWING MODES](#)

[BACKGROUND IMAGE](#)

[TOUCH BAR SUPPORT](#)

[LINUX NATIVE MENU](#)

[CONFIGURING CODE STYLE](#)

[CONFIGURING LINE SEPARATORS](#)

[COPY CODE STYLE SETTINGS](#)

[COLORS AND FONTS](#)

[CONFIGURE KEYBOARD SHORTCUTS](#)

[SET FILE TYPE ASSOCIATIONS](#)

[ENCODING](#)

[SWITCH BETWEEN SCHEMES](#)

[SHARE IDE SETTINGS](#)

[ADVANCED CONFIGURATION](#)

[CHANGE THE BOOT JAVA RUNTIME OF THE IDE](#)

[INCREASE THE MEMORY HEAP OF THE IDE](#)

[MONITOR IDE PERFORMANCE](#)

[CONFIGURE PROJECTS](#)

[PROJECT SECURITY](#)

[PROJECTS](#)

[CREATE A NEW PROJECT](#)

[IMPORT A PROJECT](#)

[ADD ITEMS TO YOUR PROJECT](#)

[PROJECT SETTINGS](#)

[PROJECT STRUCTURE SETTINGS](#)

[SAVE PROJECTS AS TEMPLATES](#)

[OPEN, CLOSE, AND MOVE PROJECTS](#)

[INDEXING](#)

[SHARED INDEXES](#)

[MODULES](#)

[MODULE STRUCTURE SETTINGS](#)

[CONTENT ROOTS](#)

[MODULE DEPENDENCIES](#)

[ADD FRAMEWORKS \(FACETS\)](#)

[UNLOAD MODULES](#)

[SDKS](#)

[SUPPORTED JAVA VERSIONS AND FEATURES](#)

[LIBRARIES](#)

[FAVORITES](#)

[SCOPES AND FILE COLORS](#)

[SCOPE LANGUAGE SYNTAX REFERENCE](#)

[INVALIDATE CACHES](#)

[PATH VARIABLES](#)

[RESOURCE FILES](#)

[WRITE AND EDIT SOURCE CODE](#)

[EDITOR BASICS](#)

[MULTIPLE CARETS AND SELECTION RANGES](#)

[LIGHTEDIT MODE](#)

[SOURCE CODE NAVIGATION](#)

[SOURCE CODE HIERARCHY](#)

[SOURCE FILE STRUCTURE](#)

IntelliJ IDEA overview

IntelliJ IDEA is an **Integrated Development Environment (IDE)** for JVM languages designed to maximize developer productivity. It does the routine and repetitive tasks for you by providing clever code completion, static code analysis, and refactorings, and lets you focus on the bright side of software development, making it not only productive but also an enjoyable experience.

Multi-platform

IntelliJ IDEA is a cross-platform IDE that provides consistent experience on Windows, macOS, and Linux.

- See [Install IntelliJ IDEA](#) for OS-specific instructions.
- See [IntelliJ IDEA keyboard shortcuts](#) for instructions on how to choose the right keymap for your operating system, and learn the most useful shortcuts.

Supported languages

Development of modern applications involves using multiple languages, tools, frameworks, and technologies. IntelliJ IDEA is designed as an IDE for JVM languages but numerous [plugins](#) can extend it to provide a polyglot experience.

JVM languages

Use IntelliJ IDEA to develop applications in the following languages that can be compiled into the JVM bytecode, namely:

- [Java](#)
- [Kotlin](#)
- [Scala](#)
- [Groovy](#)

Other languages

Plugins bundled with IntelliJ IDEA and available out of the box add support for some of the most popular languages, namely:

- [Python](#) (full [PyCharm](#) functionality)
- [Ruby](#) (full [RubyMine](#) functionality)
- [PHP](#) (full [PhpStorm](#) functionality)
- [SQL](#) (full [DataGrip](#) functionality)
- [Go](#) (full [GoLand](#) functionality)
- [JavaScript](#) (full [WebStorm](#) functionality)
- [TypeScript](#) (full [WebStorm](#) functionality)
- [CoffeeScript](#) (full [WebStorm](#) functionality)
- [Thymeleaf](#)
- [JSON](#)
- [Markdown](#)
- [HTML and XHTML](#)
- [XML and XSL](#)
- [XPath and XSLT](#)
- [Velocity and FreeMarker](#)
- [Stylesheets](#) (CSS, Less, Sass)
- [Dart](#)
- [Erlang](#)

C/C++ are not officially supported in IntelliJ IDEA, but you can use [CLion](#).

You can browse the [JetBrains Marketplace](#) to find an official plugin that adds support for almost any language, framework or technology used today, or for third-party plugins. See [Manage plugins](#) for details on how to manage plugins in IntelliJ IDEA.

Do I need a language plugin for IntelliJ IDEA or a separate IDE?

IntelliJ IDEA Ultimate is a superset of most IntelliJ platform-based IDEs. If the bundled language plugins are enabled, it includes support for all technologies that are available within our more specific IDEs, such as [PyCharm](#), [WebStorm](#), [PHPStorm](#), and so on.

So, for example, if your application's codebase is mainly in Java, but it also uses Python scripts, we recommend using IntelliJ IDEA in combination with the bundled **Python** plugin. If your codebase is mainly in Python, [PyCharm](#) is the right IDE for you.

IntelliJ IDEA editions

IntelliJ IDEA comes in three editions:

- **IntelliJ IDEA Ultimate:** the commercial edition for JVM, web, and enterprise development. It includes all the features of the **Community** edition, plus adds support for [languages](#) that other IntelliJ platform-based IDEs focus on, as well as support for a variety of server-side and front-end frameworks, [application servers](#), integration with [database](#) and [profiling](#) tools, and more.
- **IntelliJ IDEA Community Edition:** the free edition based on open-source for JVM and Android development.
- **IntelliJ IDEA Edu:** the free edition with built-in lessons for learning Java, Kotlin, and Scala, interactive programming tasks and assignments, and special features for teachers to create their own courses and manage the educational process (see [IntelliJ IDEA Edu](#)).

See the [IntelliJ IDEA editions comparison matrix](#).

The Early Access program

IntelliJ IDEA Ultimate is available for free within the [Early Access Program \(EAP\)](#). EAP builds are published before the release of a stable product version, and you can download them to try out the new features before they are publicly available in return for your feedback. EAP builds are configured to collect feature usage statistics, and are a valuable source of information for our developers. You can also [report an issue](#) if you encounter any problems.

Release Candidate (RC) builds published before a release are also available for download, but require a paid license.

Preview builds published after the release of a stable version that are followed by an official update, also require a paid license and cannot be evaluated for free.

User Interface

IntelliJ IDEA provides an editor-centric environment. It follows your context and brings up the necessary tools automatically to help you minimize the risk of interrupting the developer's flow.

[Take a guided tour](#) around IntelliJ IDEA user interface.

Ergonomic design and customizable appearance

One of the best things about IntelliJ IDEA is its tunability. You can configure virtually anything: the IDE appearance, the layout of tool windows and toolbars, code highlighting, and more. There are also numerous ways you can fine-tune the editor and customize its behavior to speed up navigation and get rid of any extras that distract you from code.

- [Configure the colors and fonts](#) for your source code, console output, debugger information, search results, and more. You can choose from a number of predefined color schemes or [customize](#) a scheme to create a unique working environment.
- [Learn](#) how to configure the editor settings, including appearance, font, code formatting, and more.
- [Customize menus and toolbars](#) to spare the annoyance of looking for an action among a dozen buttons you never use.

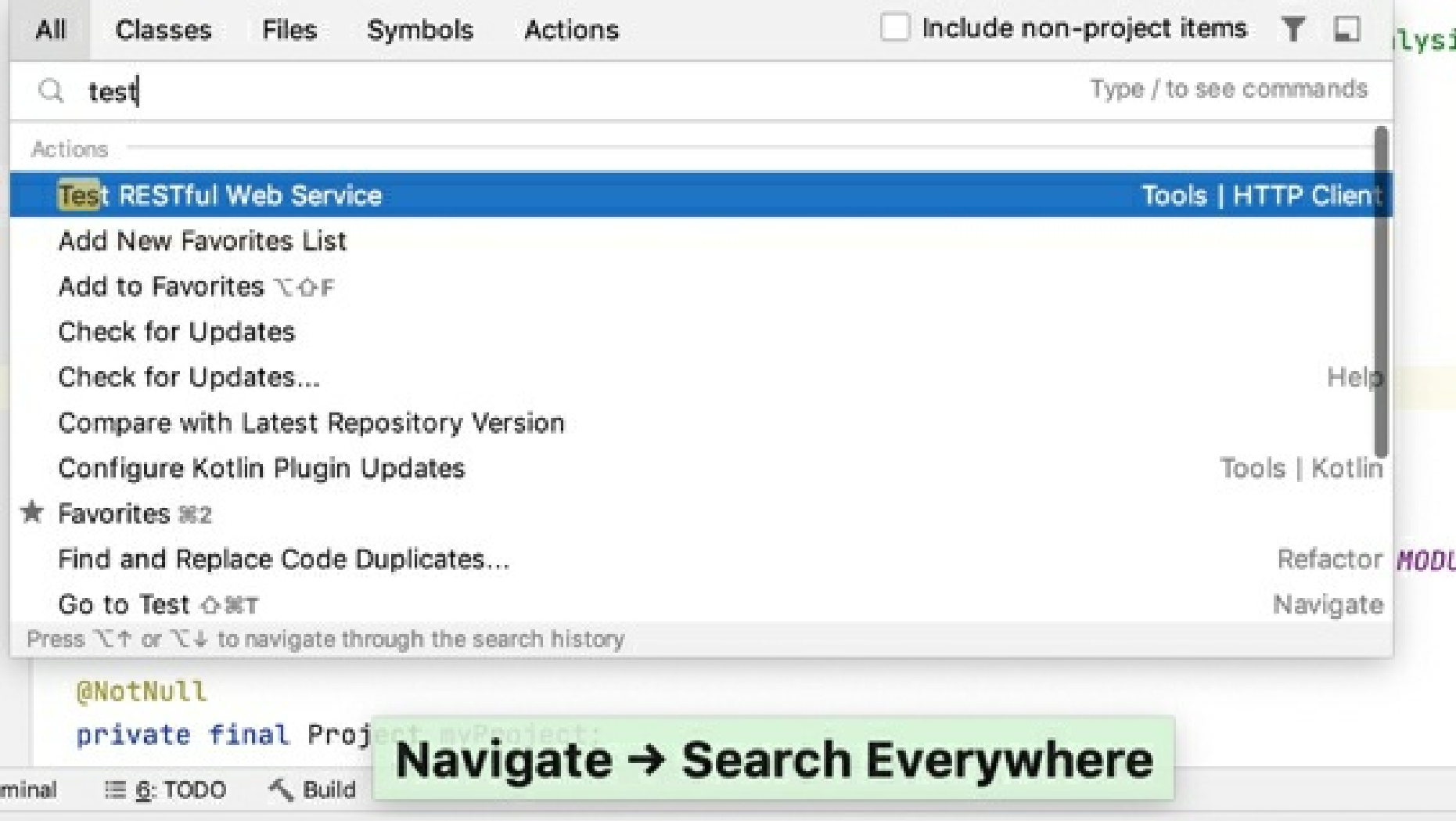
Shortcuts for everything

In IntelliJ IDEA, you have shortcuts for nearly every action, including selection and switching between the editor and various tool windows.

Use the [most useful shortcuts](#) to invoke frequent actions without switching your focus from the editor and [tune your keymap](#) to assign custom shortcuts for your favorite commands. Watch this video to learn about some of the most useful shortcuts:

Navigation and search

IntelliJ IDEA provides quick navigation not only inside source code files but also throughout the entire project. One of the most useful shortcuts that is worth remembering is double Shift that brings up the **Search Everywhere** dialog: start typing and IntelliJ IDEA will look for your search string among all files, classes, and symbols that belong to your project, and even among the IDE actions.



Gif

Here are some of the most useful navigation shortcuts:

Action	Shortcut
Search everywhere	Double Shift
Go to file	Ctrl+Shift+N
Go to class	Ctrl+N
Go to symbol	Ctrl+Alt+Shift+N
Go to declaration	Ctrl+B

See [Source code navigation](#) for more hints on how to navigate through the source code, and [learn](#) the most useful shortcuts that help you quickly switch between the editor and various tool windows, switch focus, jump to the **Navigation bar**, and so on.

Recent files and locations

Normally, you work with a small subset of files at a time and need to switch between them quickly. The **Recent Files** action is a real time-saver here. Press `Ctrl+E` to see a list of last accessed files. Note that

you can also use this action to open any tool window:

Recent Files

☐ Show changed only ⌘E

Project

Favorites

TODO

Structure

Version Control

Build

Database Changes

Database

Event Log

Gradle

Java Enterprise

Maven

Persistence

Spring

Terminal

DataTypeRendering.java

LocalHistory.java

PostfixCompletion.java

standard-java/.editorconfig

.../debugging

Apart from jumping to a recent file, you can also get quick access to *Recent Locations*- that is code snippets you last viewed or edited. Press `Ctrl+Shift+E` and you'll be able to jump to a particular line you modified lately:

Recent Locations

Show changed only ⬆⌘E

Library.java Library

23 }
24 static
25 }

HelloWorld.java

7 }

HelloWorld.java HelloWorld > main()

3 public class HelloWorld {
4 public static void main (String args[]){
5 System.out.println("Hello, World!");
6 }

File structure

Press `Ctrl+F12` to open the file structure popup that gives you an overview of all elements used in the current file and lets you jump to any of them:

DataFlowAnalysis.java

☐ Inherited members (⌘F12)

☐ Anonymous Classes (⌘I)

☐ Lambdas (⌘L)

⚙

▼ DataFlowAnalysis

m findCauseOfProblem(): void

▶ Other Data Flow Examples

▶ Private helpers

Alternatively, use the **Structure tool window** `Alt+7`

Find action

If you don't remember the shortcut or the menu path for an action you want to use, press `Ctrl+Shift+A` and start typing the action name:

AllClassesFilesSymbolsActions

Include disabled actions ⬆

import

Press ⌘↵ to assign a shortcut

Import from Sources...

Import into Subversion... VCS | Import into Version Control

Import Patches...

Import Project from Existing Sources...

Import Settings

Import Settings... File

Import Tests from File... Run

Import into Version Control

Import Profiler Results Run

Optimize Imports ^⌘O Code

Reimport All Gradle Projects

Press ⌘↑ or ⌘↓ to navigate through the search history

Coding assistance

Code completion

IntelliJ IDEA helps you speed up the coding process by providing context-aware code completion.

- **Basic completion** helps you complete the names of classes, methods, fields, and keywords within the visibility scope:

```
} catch (NumberFormatException nfe) {  
    if (nfe) {  
        p nfe  
        v conversionFactor  
    }  
}
```

^↓ and ^↑ will move caret down and up in the editor [Next Tip](#)

- **Smart completion** suggests the most relevant symbols applicable in the current context when IntelliJ IDEA can determine the appropriate type:

```
Calendar now = new GregorianCalendar(T)
```

Calendar.THURSDAY (= 5) (java.util)	int
Calendar.TUESDAY (= 3) (java.util)	int
Locale.TAIWAN (= TRADITIONAL_CHINESE) (java.util)	Locale
Locale.TRADITIONAL_CHINESE (java.util)	Locale
TimeZone.getTimeZone(String ID) (java.util)	TimeZone
TimeZone java.util	
Time java.sql	
TimeZone.getTimeZone(ZoneId zoneId) (java.util)	TimeZone
TimeZone.getDefault() (java.util)	TimeZone
Throwable java.lang	
Thread java.lang	
ThreadDeath java.lang	

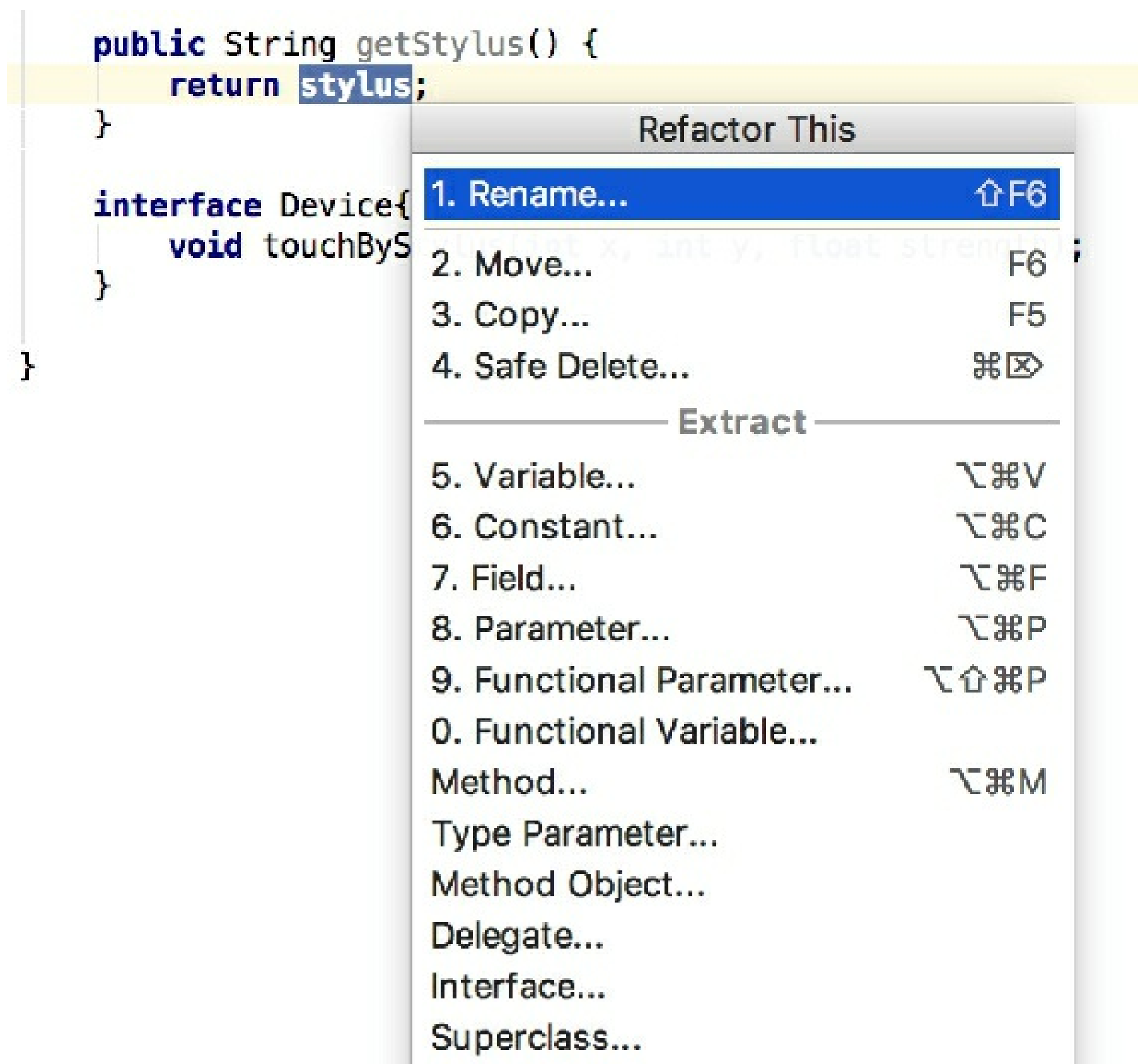
Press ^↑Space to show only variants that are suitable by type [Next Tip](#)

For more information on the different types of code completion available in IntelliJ IDEA with examples and productivity tips, see [Code completion](#).

Refactorings

IntelliJ IDEA offers a comprehensive set of automated code refactorings that lead to significant productivity gains. For example, when you rename a class, the IDE will update all references to this class throughout your project.

You don't even need to bother to select anything before you apply a refactoring. IntelliJ IDEA is smart enough to figure out which statement you're going to refactor, and only asks for confirmation if several choices are possible. Just press `Ctrl+Alt+Shift+T` to open a list of refactorings available in the current context:



See section [Refactoring code](#) for a full list of available refactorings with usages scenarios and the **before** and **after** examples.

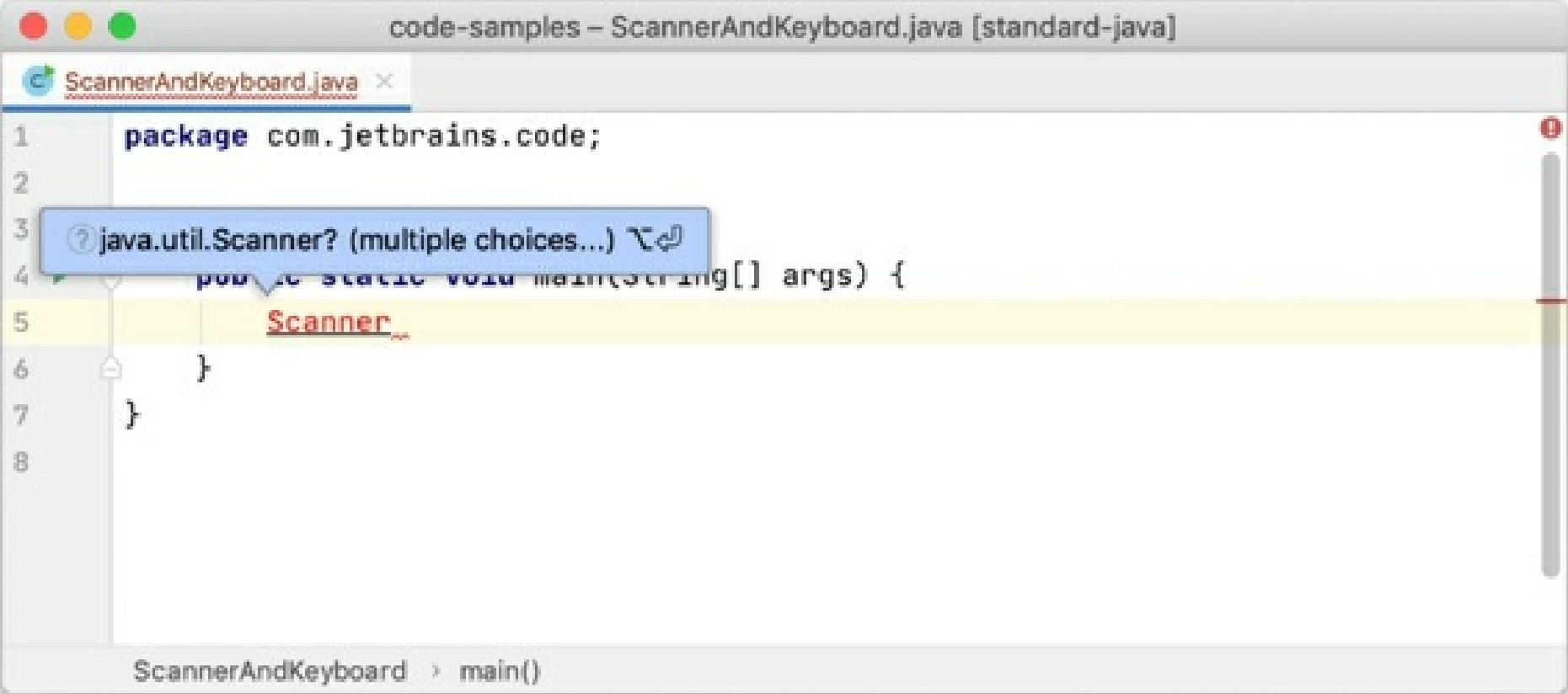
Learn some of the most useful refactoring shortcuts:

Action	Shortcut
Refactor this	Ctrl+Alt+Shift+T
Rename	Shift+F6
Extract variable	Ctrl+Alt+V
Extract field	Ctrl+Alt+F
Extract a constant	Ctrl+Alt+C
Extract a method	Ctrl+Alt+M

Extract a parameter	Ctrl+Alt+P
Inline	Ctrl+Alt+N
Copy	F5
Move	F6

Static code analysis

IntelliJ IDEA provides a set of inspections that are built-in static code analysis tools. They help you find potential bugs, locate dead code, detect performance issues, and improve the overall code structure. Inspections not only tell you where a problem is but also provide quick fixes that help you deal with it right away. Click the red bulb next to the highlighted code, or press Alt+Enter to choose a fix:



Gif

Apart from quick-fixes, IntelliJ IDEA also provides intention actions that help you apply automatic changes to code that is correct. For example, you can inject a language, add Java annotations, add JavaDoc, convert HTML or XML tags, and much more. To view a full list of intention actions, in the **Settings/Preferences** dialog Ctrl+Alt+S, go to **Editor | Intentions**.

Code generation

IntelliJ IDEA provides multiple ways to generate common code constructs and recurring elements, which helps you increase productivity by delegating routine tasks to the IDE. This includes generating code from predefined or custom code templates, generating wrappers, getters and setters, automatic pairing of characters, and more. Press Alt+Insert to open a popup with the available constructs you can generate from your caret position. See Generate code for more detail.

Integration with developer tools

Apart from providing smart navigation and coding assistance, IntelliJ IDEA integrates the essential developer tools and lets you debug, analyze, and version the code base of your applications from within the IDE.

Debugger

IntelliJ IDEA provides a built-in JVM debugger. It lets you get and analyze runtime information, which is useful for diagnosing issues and getting a deeper understanding of how a program operates. It enables you to:

- Suspend the program execution to examine its behavior using breakpoints. Multiple types of breakpoints, together with conditions and filters, allow you to specify the exact moment

when an application needs to be paused.

- Play with the program state by modifying variable values, evaluate expressions, and so on.
- Examine variable values, call stacks, thread states, and so on.
- Control the step-by-step execution of the program.

See [Tutorial: Debug your first Java application](#) to learn the basics of debugging and play with the debugger features in the IDE.

Profiler

IntelliJ IDEA provides the following built-in profiler tools that let you explore which methods consume most CPU time, thus helping you detect the most expensive methods and understand exactly how they behave:

- The [Async Profiler](#): a tool for Linux and macOS that lets you see how exactly memory and CPU resources are allocated in your application.
- The [Java Flight Recorder](#): a multi-platform tool that collects the information on events at a particular moment in time in a Java Virtual Machine when executing an application.

Terminal

IntelliJ IDEA includes a built-in [terminal](#) for working with a command-line shell from inside the IDE. For example, if you're used to executing Git commands from the command line, you can run them from the Terminal instead of invoking these actions from the menu.

The Terminal runs with your default system shell, but it also supports a number of other shells, such as `cmd.exe`, `bash`, `sh`, and so on.

Build tools

IntelliJ IDEA comes with a fully-functional [Gradle](#) and [Maven](#) integration that allows you to automate your building process, packaging, running tests, deployment, and other activities.

When you open an existing Gradle or Maven project or create a new one, IntelliJ IDEA detects and automatically downloads all the required repositories and plugins, so you virtually don't need to configure anything and can focus solely on the development process. You can

edit `build.gradle` and `pom.xml` files directly from the editor and configure the IDE to automatically sync all changes to the build configurations.

For instructions on how to work with Gradle and Maven projects in IntelliJ IDEA, see [Gradle](#) and [Maven](#).

Version Control

IntelliJ IDEA provides integration with the most popular version control tools: [Git](#), [Mercurial](#), [Perforce](#), and [Subversion](#).

You can [review the history](#) of your entire project or separate files, [compare file versions](#), [manage branches](#), and even [process GitHub pull requests](#) without leaving the IDE.

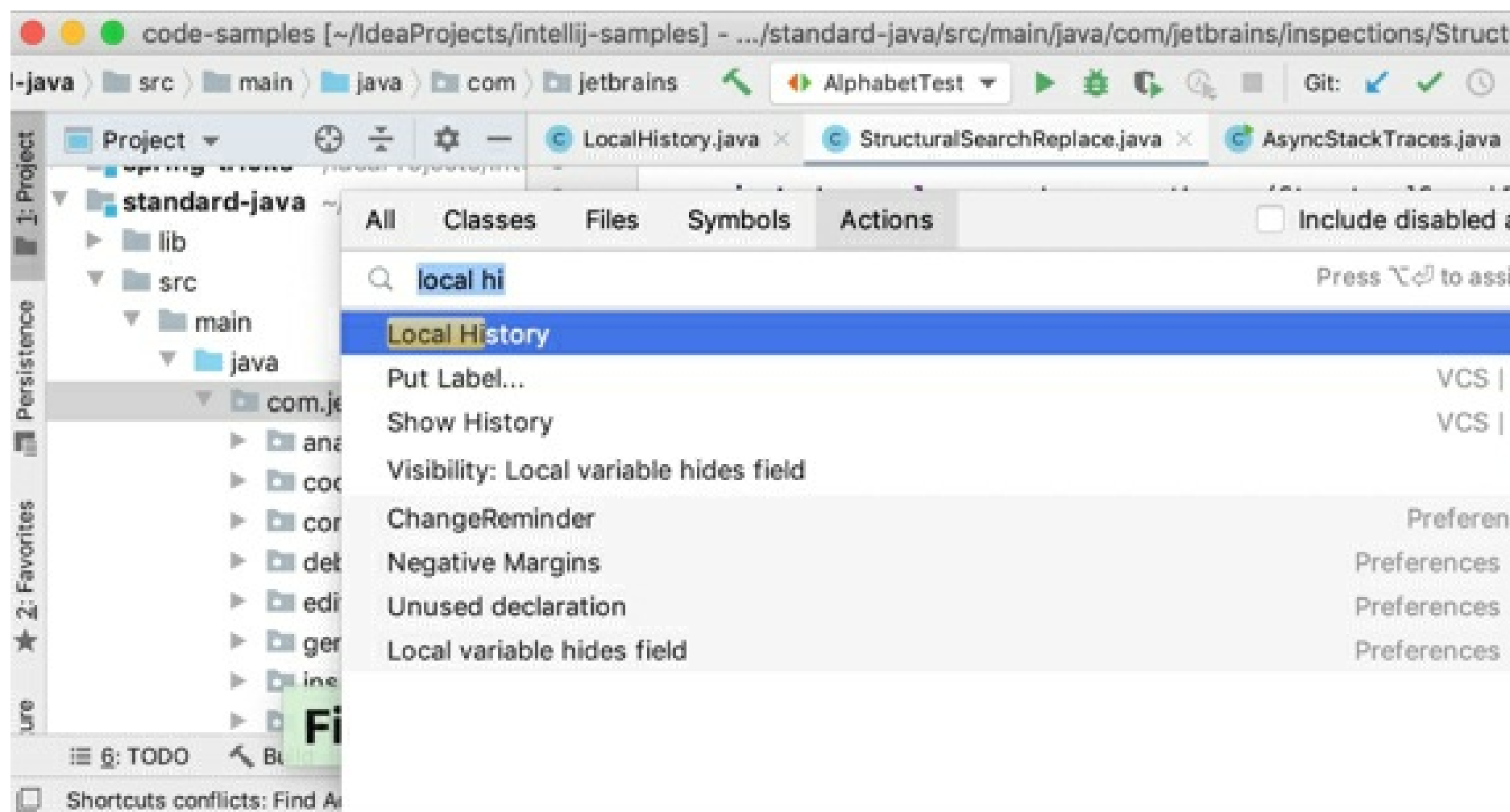
You can quickly access all VCS actions from the VCS operations popup `Alt+⌘`:

VCS Operations		
Git		
✓	1. Commit...	⌘K
	2. Commit File	
↶	3. Rollback...	⌘Z
🕒	4. Show History	
	5. Annotate	
↔	6. Compare with the Same Repository Version	
🔗	7. Branches...	
➦	8. Push...	⌘K
	9. Stash Changes...	
	0. UnStash Changes...	
Local History		
	Show History	
	Put Label...	

See [Version control](#) for instructions on how to configure integration with your VCS and perform the VCS-related operations.

Local History

Even if no version control is enabled for your project yet, you can still keep track of modifications to your project, and restore deleted files or separate changes with Local History. It acts as your personal version control system that automatically records your project's revisions triggered by various events as you edit code, run tests, deploy applications, and so on.



Install IntelliJ IDEA

IntelliJ IDEA is a cross-platform IDE that provides consistent experience on the Windows, macOS, and Linux operating systems.

IntelliJ IDEA is available in the following editions:

- **Community Edition** is free and open-source, licensed under Apache 2.0. It provides all the basic features for JVM and Android development.
- **IntelliJ IDEA Ultimate** is commercial, distributed with a 30-day trial period. It provides additional tools and features for web and enterprise development.

For more information, see the [comparison matrix](#).

System requirements

Requirement	Minimum	Recommended
RAM	2 GB of free RAM	8 GB of total system RAM
CPU	Any modern CPU	Multi-core CPU. IntelliJ IDEA supports multithreading for different operations and processes making it faster the more CPU cores it can use.
Disk space	2.5 GB and another 1 GB for caches	SSD drive with at least 5 GB of free space
Monitor resolution	1024x768	1920×1080
Operating system	Officially released 64-bit versions of the following: • Microsoft Windows 8 or later • macOS 10.13 or later • Any Linux distribution that supports Gnome, KDE, or Unity DE. Pre-release versions are not supported.	Latest 64-bit version of Windows, macOS, or Linux (for example, Debian, Ubuntu, or RHEL)

You do not need to install Java to run IntelliJ IDEA because JetBrains Runtime is bundled with the IDE (based on JRE 11). However, to develop Java applications, a standalone JDK is required.

Install using the Toolbox App

The [JetBrains Toolbox App](#) is the recommended tool to install JetBrains products. Use it to install and

manage different products or several versions of the same product, including [Early Access Program](#) (EAP) and Nightly releases, update and roll back when necessary, and easily remove any tool. The Toolbox App maintains a list of all your projects to quickly open any project in the right IDE and version.

Windows

macOS

Linux

Install the Toolbox App

1. Download the installer **.exe** from the [Toolbox App web page](#).
2. Run the installer and follow the wizard steps.

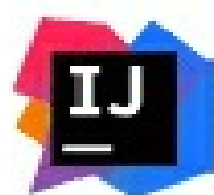
After you run the Toolbox App, click its icon in the notification area and select which product and version you want to install.



Projects

Tools

Installed

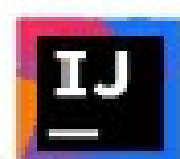


IntelliJ IDEA Ultimate

2018.2.7



▼ Available



IntelliJ IDEA Community

Capable and ergonomic

Install



IntelliJ IDEA Ultimate EAP

Experimental build
Runtime 11



IntelliJ IDEA Community EAP

Experimental build
Runtime 11



Android Studio

Everything you need to
develop Android



PyCharm Professional

Python IDE for professional



PyCharm Community

Python IDE for professional developers

Install



Install	2019.2 Nightly	192.1600
Install	2019.1 Nightly	191.6313
Install	2019.1 Beta3	191.6183.20
Install	2018.3.6 Preview	183.6156.4
Install	2018.3.5	183.5912.21
Install	2018.2.7	182.5107.41
Install	2018.1.7	181.5540.23
Install	2017.3.6	173.4674.60
Install	2017.2.7	172.4574.19
Install	2017.1.6	171.4694.73
Install	2016.3.8	163.15529.8



ENG

1:50 PM
3/18/2019



Log in to your JetBrains Account from the Toolbox App and it will automatically activate the available licenses for any IDE that you install.

Standalone installation

Install IntelliJ IDEA manually to manage the location of every instance and all the configuration files. For example, if you have a policy that requires specific install locations.

Windows

Windows (ZIP)

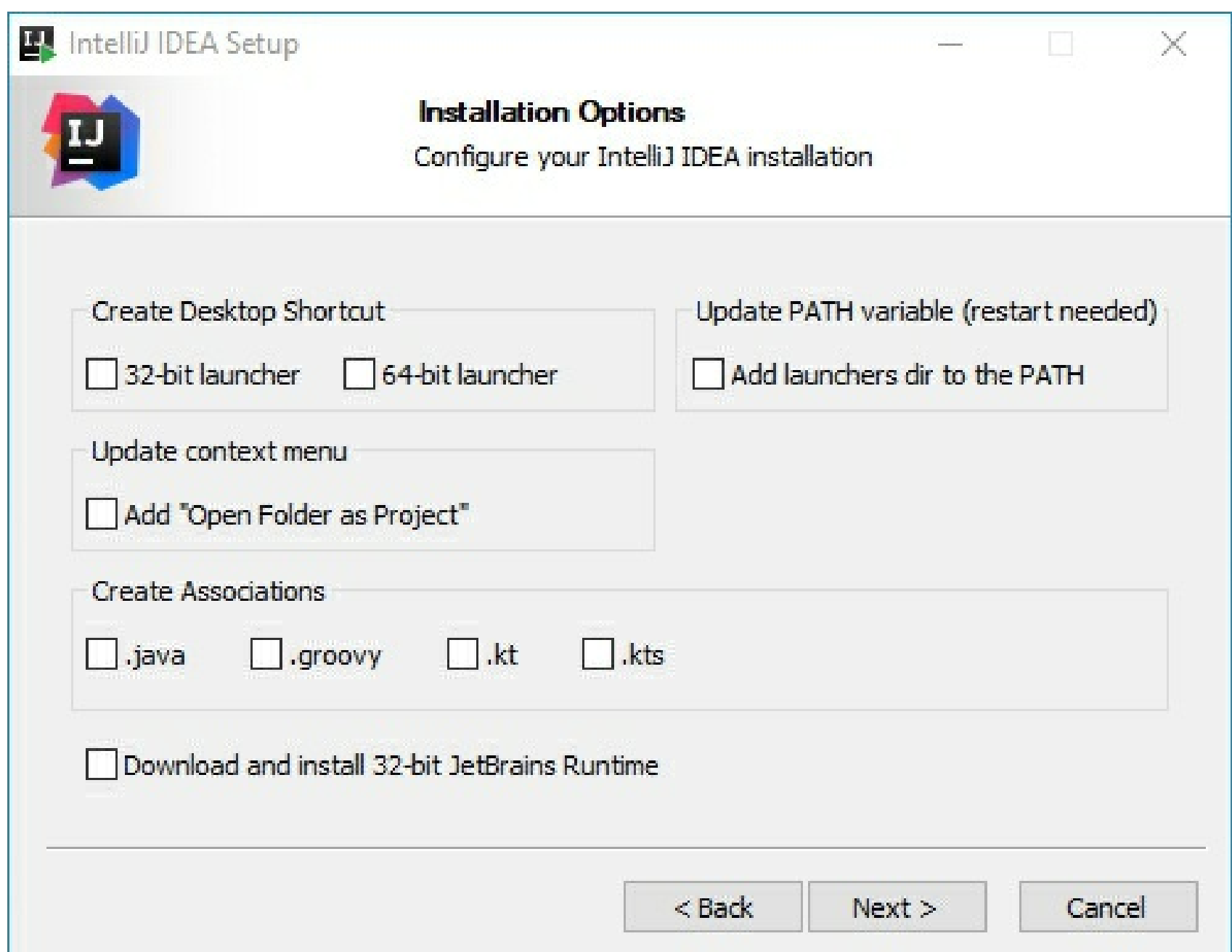
macOS

Linux

1. Download the installer .exe.
2. Run the installer and follow the wizard steps.

On the **Installation Options** step, you can configure the following:

- Create a desktop shortcut for the launcher relevant to your operating system.
- Add the directory with IntelliJ IDEA command-line launchers to the PATH environment variable to be able to run them from any working directory in the Command Prompt.
- Add an item **Open Folder as Project** to the system context menu (when you right-click a folder).
- Associate specific file extensions with IntelliJ IDEA to open them with a double-click.
- Install the 32-bit version of JetBrains Runtime if you are running a 32-bit Windows version.



To run IntelliJ IDEA, find it in the Windows **Start** menu or use the desktop shortcut. You can also run the launcher batch script or executable in the installation directory under **bin**.

When you run IntelliJ IDEA for the first time, some steps are required to complete the installation, customize your instance, and start working with the IDE.

For more information, see [Run IntelliJ IDEA for the first time](#).

For information about the location of the default IDE directories with user-specific files, see [Default IDE directories](#).

Silent installation on Windows

Silent installation is performed without any user interface. It can be used by network administrators to install IntelliJ IDEA on a number of machines and avoid interrupting other users.

To perform silent install, run the installer with the following switches:

- `/s`: Enable silent install
- `/CONFIG`: Specify the path to the [silent configuration file](#)
- `/D`: Specify the path to the installation directory

This parameter must be the last in the command line and it should not contain any quotes even if the path contains blank spaces.

For example:

```
>
ideaIU.exe /S /CONFIG=d:\temp\silent.config /D=d:\IDE\IntelliJ IDEA Ultimate
Copied!
```

To check for issues during the installation process, add the `/LOG` switch with the log file path and name between the `/s` and `/D` parameters. The installer will generate the specified log file. For example:

```
>
ideaIU.exe /S /CONFIG=d:\temp\silent.config /LOG=d:\JetBrains\IDEA\install.log /D=d:\IDE\IntelliJ IDEA Ultimate
Copied!
```

Silent configuration file

You can download the default silent configuration file for IntelliJ IDEA

at <https://download.jetbrains.com/idea/silent.config>

The silent configuration file defines the options for installing IntelliJ IDEA. With the default options, silent installation is performed only for the current user: `mode=user`. If you want to install IntelliJ IDEA for all users, change the value of the installation mode option to `mode=admin` and run the installer as an administrator.

The default silent configuration file is unique for each JetBrains product. You can modify it to enable or disable various installation options as necessary.

It is possible to perform silent installation without the configuration file. In this case, omit the `/CONFIG` switch and run the installer as an administrator. Without the silent configuration file, the installer will ignore all additional options: it will not create desktop shortcuts, add associations, or update the `PATH` variable. However, it will still create a shortcut in the **Start** menu under **JetBrains**.

Install as a snap package on Linux

You can install IntelliJ IDEA as a self-contained [snap](#) package. Since snaps update automatically, your IntelliJ IDEA installation will always be up to date.

To use snaps, install and run the `snaptd` service as described in the [installation guide](#).

On Ubuntu 16.04 LTS and later, this service is pre-installed.

IntelliJ IDEA is distributed via two channels:

- The *stable* channel includes only stable versions. To install the latest stable release of IntelliJ IDEA, run the following command:

```
IntelliJ IDEA Ultimate
IntelliJ IDEA Edu
Community Edition

$
sudo snap install intellij-idea-ultimate --classic
Copied!
```

The `--classic` option is required because the IntelliJ IDEA snap requires full access to the system,

like a traditionally packaged application.

- The *edge* channel includes EAP builds. To install the latest EAP build of IntelliJ IDEA, run the following command:

IntelliJ IDEA Ultimate

IntelliJ IDEA Edu

Community Edition

\$

```
sudo snap install intellij-idea-ultimate --classic --edge
```

Copied!

When the snap is installed, you can launch it by running the `intellij-idea-community`, `intellij-idea-ultimate`, or `intellij-idea-educational` command.

To list all installed snaps, you can run `sudo snap list`. For information about other snap commands, see the [Snapcraft documentation](#).

Run IntelliJ IDEA for the first time

You can use the Toolbox App to run any JetBrains product. In case of a standalone installation, running IntelliJ IDEA depends on the operating system:

Windows

macOS

Linux

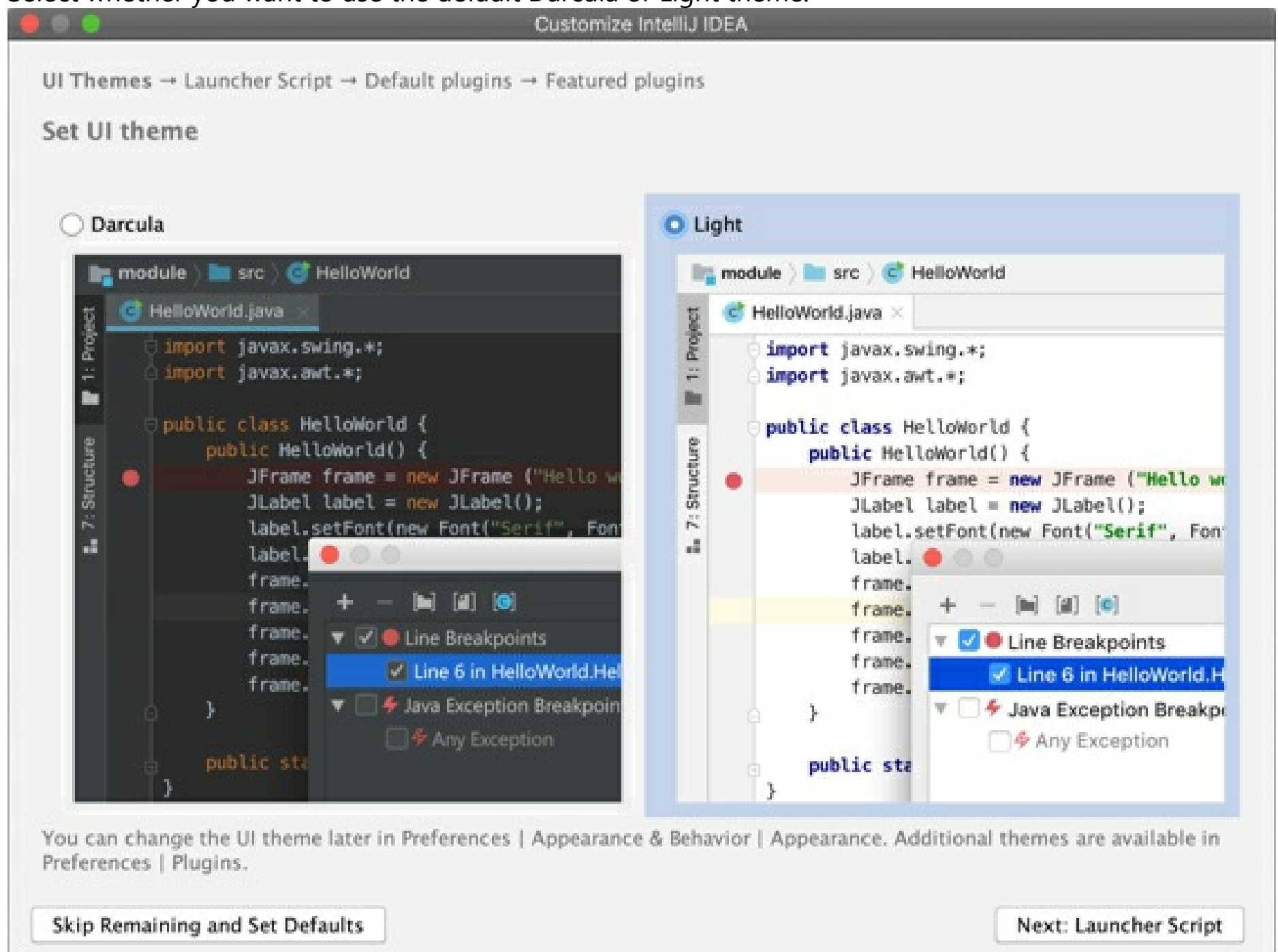
To run IntelliJ IDEA, find it in the Windows **Start** menu or use the desktop shortcut. You can also run the launcher batch script or executable in the installation directory under **bin**.

For information about running IntelliJ IDEA from the command line, see [Command-line interface](#).

When you run IntelliJ IDEA for the first time, some steps are required to complete the installation, customize your instance, and start working with the IDE.

Select the user interface theme

Select whether you want to use the default Darcula or Light theme.



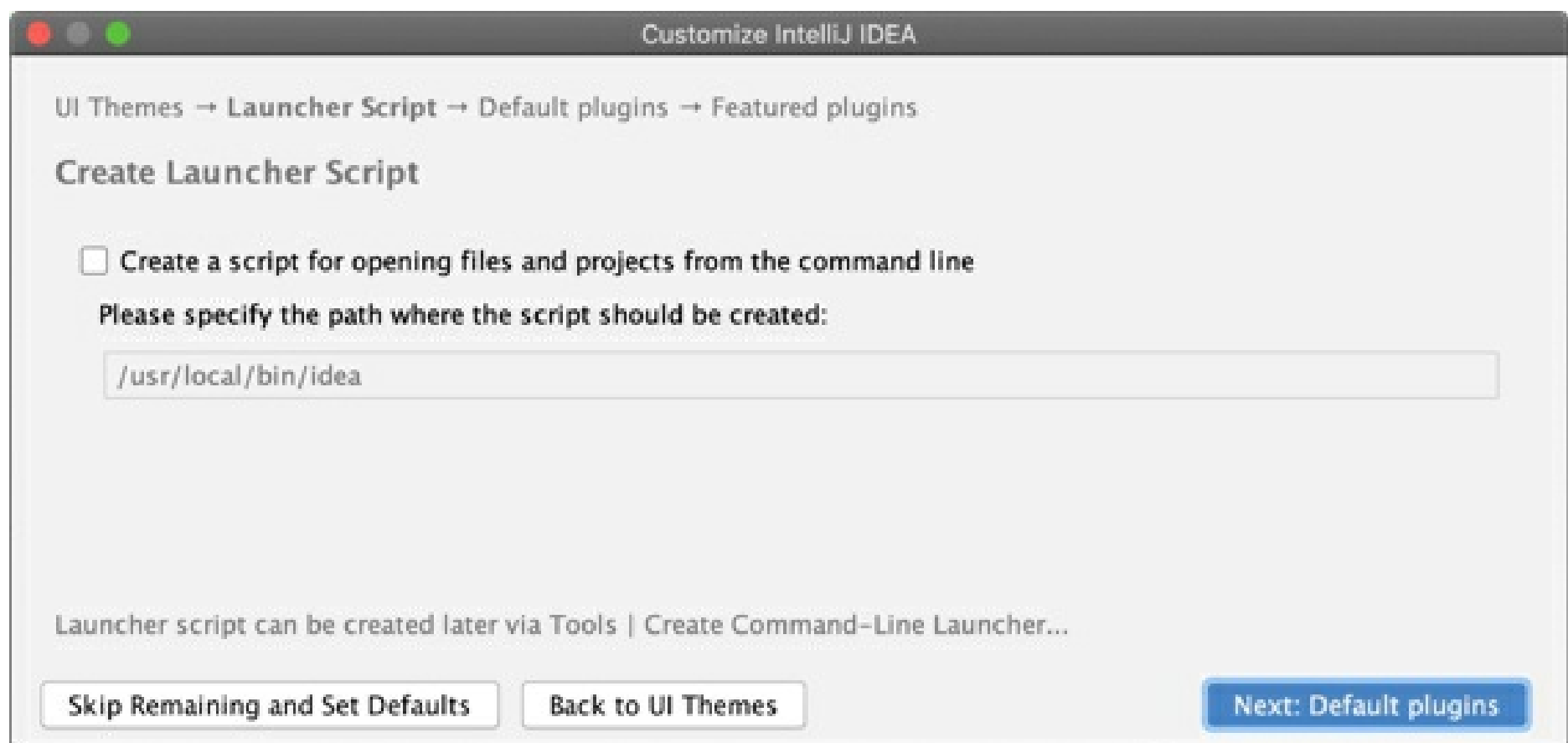
For more information, see [User interface themes](#).

Create launcher script

For **macOS** and **Linux** users, IntelliJ IDEA suggests to create a script for opening projects and files from the command line.

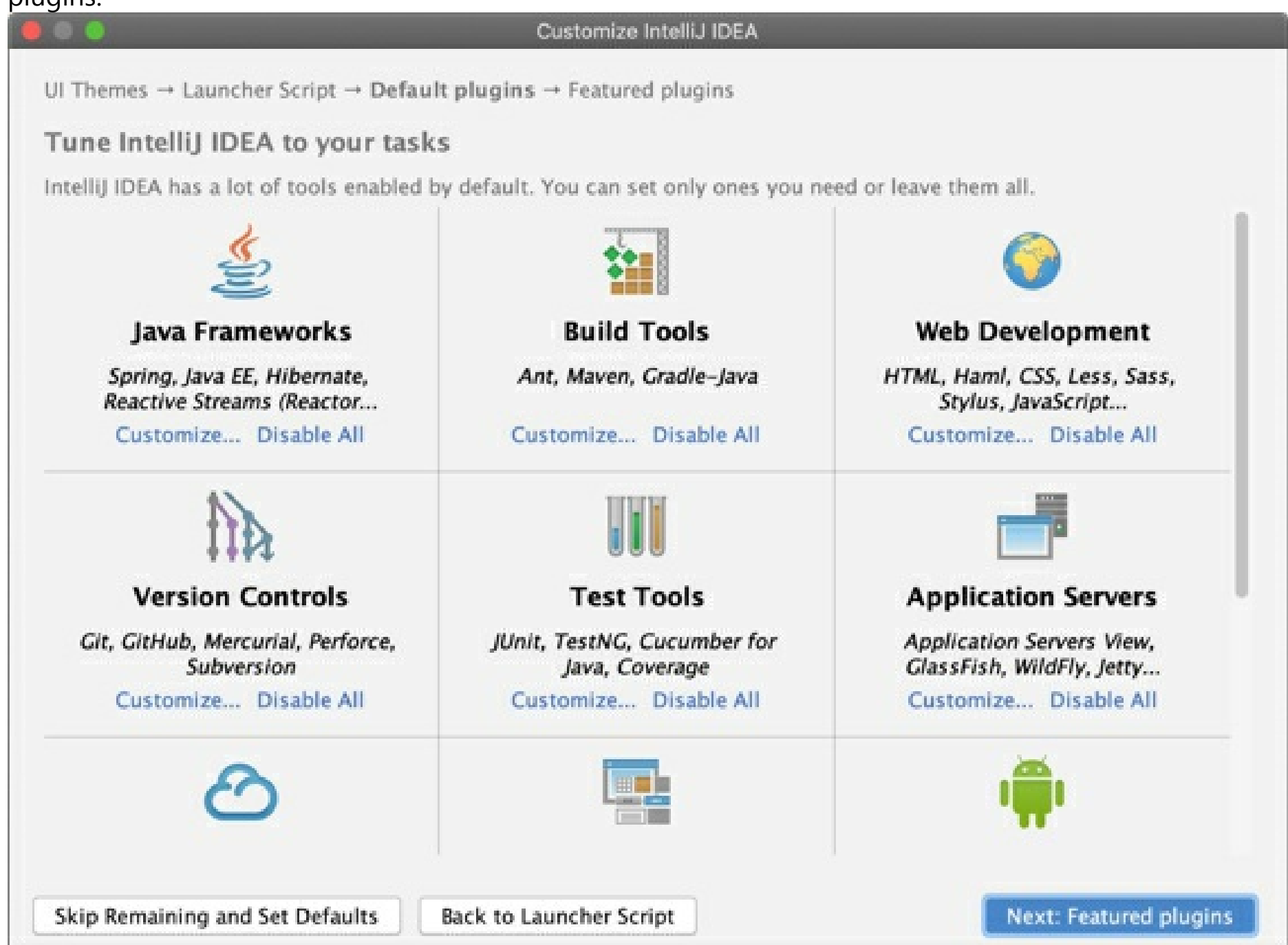
Windows users can run the executable file in the installation directory.

For more information, see [Command-line interface](#).



Disable unnecessary plugins

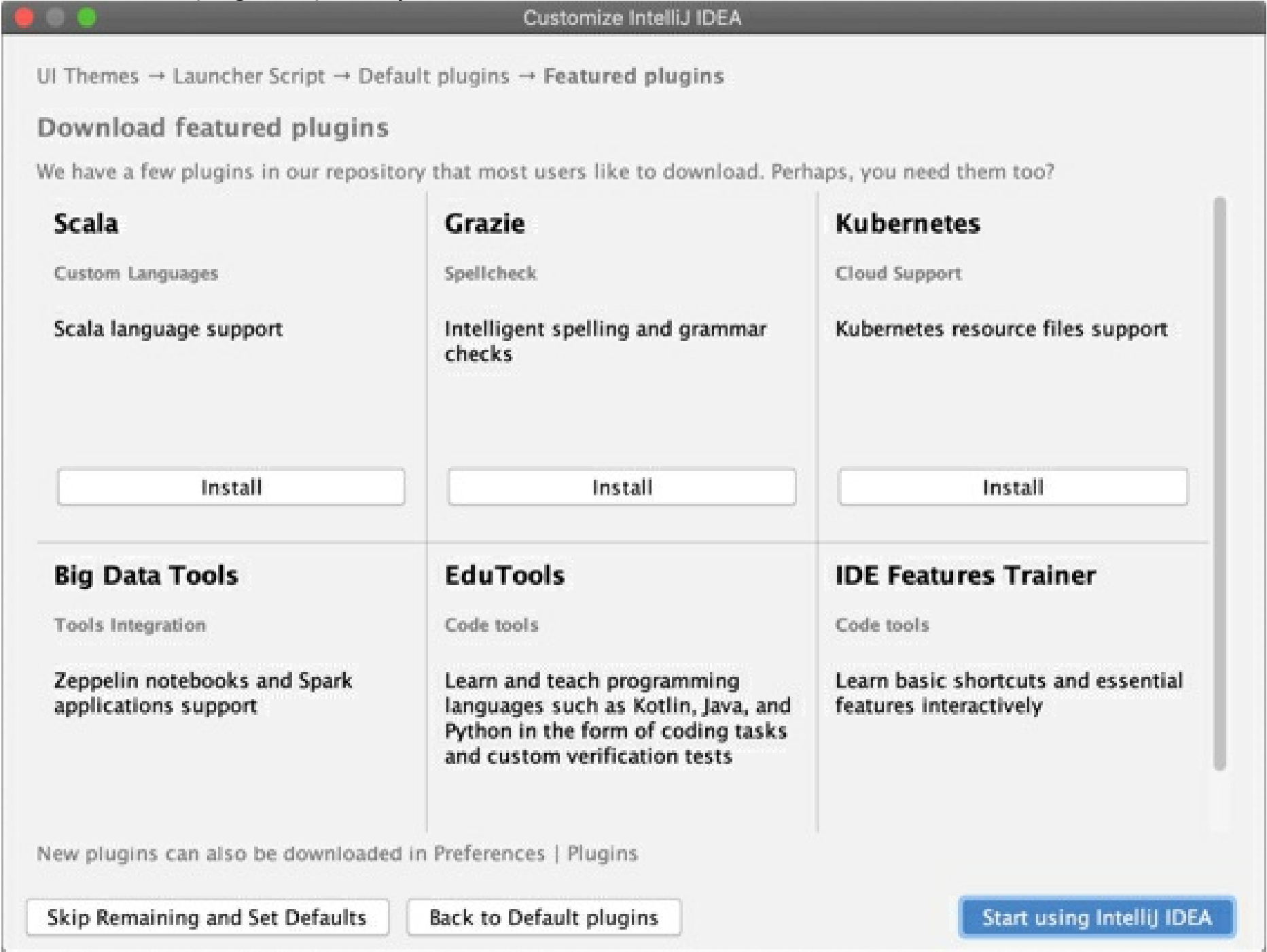
IntelliJ IDEA includes plugins that provide integration with different version control systems and application servers, add support for various frameworks and development technologies, and so on. To increase performance, you can disable plugins that you do not need. If necessary, you can enable them later in the **Settings/Preferences** dialog `Ctrl+Alt+S` under **Plugins**. For more information, see Manage plugins.



You can click the **Disable All** link for each group of plugins to disable them all, or **Customize** to disable individual plugins in the group.

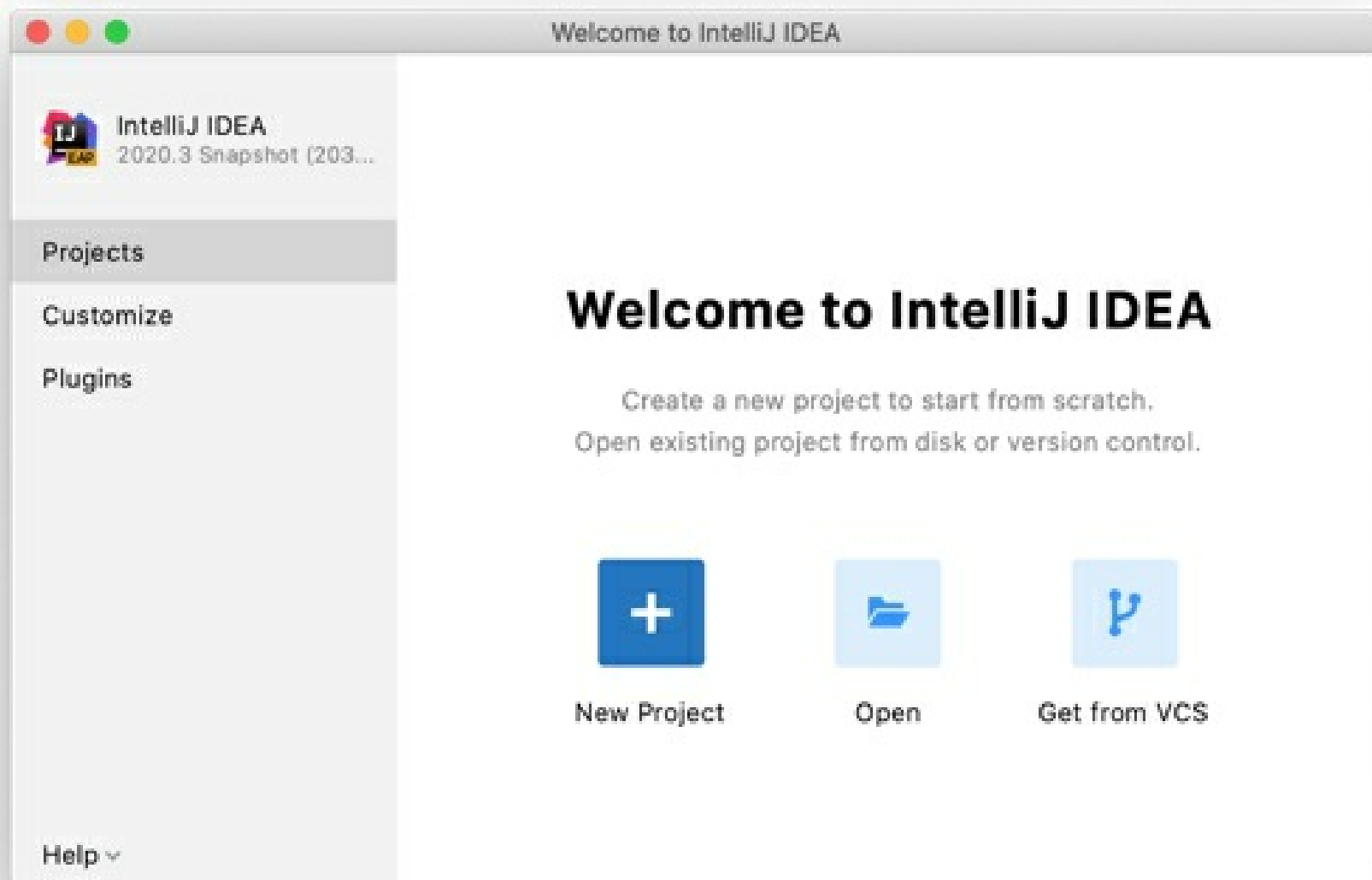
Download and install additional plugins

If necessary, click **Plugins** in the left-hand pane and download and install additional plugins from the IntelliJ IDEA plugins repository.



Start a project in IntelliJ IDEA

When you click **Start using IntelliJ IDEA**, it will show the Welcome screen.



In the **Welcome to IntelliJ IDEA** dialog, you can do the following:

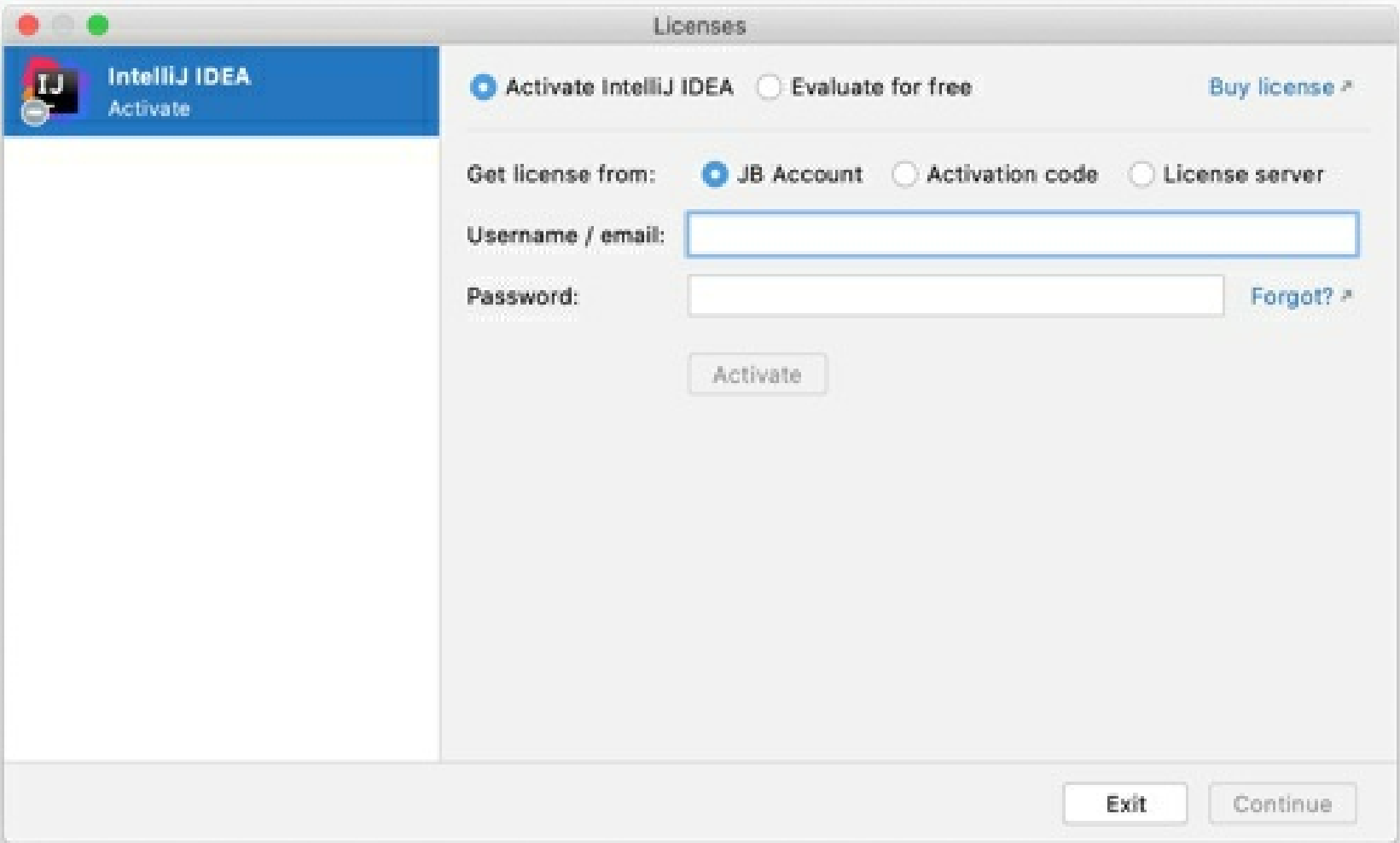
- Create a new project
- Open a project
- Get a project from a version control system

Register IntelliJ IDEA^{Ultimate}

You can evaluate IntelliJ IDEA Ultimate for up to 30 days. After that, buy and register a license to continue using the product.

1. Do one of the following to open the **Licenses** dialog:
- From the main menu, select **Help | Register**

◦ On the **Welcome** screen, click **Help | Manage License**



2. Select how you want to register IntelliJ IDEA or a plugin that requires a license:

Option	Description
JB Account	Register using the <u>JetBrains Account</u> . If you are using two-factor authentication for your JetBrains Account, specify the generated app password instead of the main JetBrains Account password.

	Two-factor authentication is switched on View Recovery Codes Disable 2FA If your JetBrains product isn't prompting you to enter a one-time password when you sign in, you can use the App Password instead of your regular password. The App Password can only be used if two-factor authentication turned on. Get App Password For more information, see What is JetBrains Account?
Activation code	Register using an activation code. You can get an activation code when you purchase a license for the corresponding product.
License server	Register using the Floating License Server. When performing silent install or managing IntelliJ IDEA installations on multiple machines, you can set the <code>JETBRAINS_LICENSE_SERVER</code> environment variable to point the installation to the Floating License Server URL. Alternatively, you can set the Floating License Server URL by adding the <code>-DJETBRAINS_LICENSE_SERVER</code> JVM option.

- IntelliJ IDEA detects the system proxy URL during initial startup and uses it for connecting to the JetBrains Account and Floating License Server. To override the URL of the system proxy, add the `-Djba.http.proxy` [JVM option](#). Specify the proxy URL as the host address and optional port number: `proxy-host[:proxy-port]`. For example: `-Djba.http.proxy=http://my-proxy.com:4321`.
- If you want to disable proxy detection entirely and always connect directly, set the property to `-Djba.http.proxy=direct`.

1.

Update IntelliJ IDEA

File | Settings | Appearance & Behavior | System Settings | Updates for Windows and Linux

IntelliJ IDEA | Preferences | Appearance & Behavior | System Settings | Updates for macOS

Ctrl+Alt+S

By default, IntelliJ IDEA is configured to check for updates automatically and notify you when a new version is available. Updates are usually *patch-based*: they are applied to the existing installation and only require you to restart the IDE. However, sometimes patch updates are not available, and a new version of IntelliJ IDEA must be installed.

When IntelliJ IDEA updates to a new major release, it opens the **What's New in IntelliJ IDEA** tab in the editor with information about the changes, improvements, and fixes. To open this tab manually, select **Help | What's New in IntelliJ IDEA**.

If IntelliJ IDEA does not have HTTP access outside your local network, it will not be able to check for updates and apply patches. In this case, you have to download new versions of the IDE and install them manually as described in Standalone installation.

Toolbox App

If you installed IntelliJ IDEA using the Toolbox App, it will suggest you to update the IDE when a new version is available.

Automatically update all managed tools

1. Open the Toolbox App and click  in the top right corner.
2. In the **Toolbox App Settings** dialog, expand **Tools** and select **Update all tools automatically**.

If you disable this option, you will need to click **Update** next to any instance when a newer version comes out.

You can also configure the update policy for every managed IDE instance separately.

Configure the update policy for a specific instance

1. Open the Toolbox App, click  next to the relevant IDE instance, and select **Settings**.
2. In the instance settings dialog, expand **Updates**.
3. If necessary, select to keep using the current major version to make sure that the Toolbox App does not update the current IDE instance to a newer major version when it comes out.
4. If you did not choose to automatically update all tools, you can select to automatically update this particular IDE instance.
5. Select the update channel to use for this IDE instance:
 - **Update to Release**: Update only to stable releases that are recommended for production.
 - **To Release and Early Access Program**: Includes updates to beta releases, release candidates, and EAP builds, which are not recommended for production and include feature previews.

Standalone instance

If you installed IntelliJ IDEA manually, the standalone IDE instance will manage its own updates. It will notify you when a new version is available. You can choose to update the current instance, download and install the new version as a separate instance, postpone the notification, or ignore the update entirely.



Configure the update policy

To manage the IntelliJ IDEA update policy, open **Settings/Preferences** `Ctrl+Alt+S` and select **Appearance & Behavior | System Settings | Updates**.

If the IDE instance is managed by the Toolbox App, these settings will affect only plugin updates.

The **Updates** page contains the following settings:

Item	Description
Check IDE updates for	Select whether you want IntelliJ IDEA to check for updates automatically and choose an update channel. • Early Access Program: Provides all updates, including major version EAP builds and minor version Preview builds. This channel is not recommended for production development. IntelliJ IDEA can be updated only to a minor Preview version, but not to a major EAP build. For example, you can update IntelliJ IDEA 2021.1.1 to 2021.1.2, but not to 2021.2 EAP. The 2021.2 EAP version in this case will be installed as an additional instance. EAP versions can be updated to both newer EAP and stable IntelliJ IDEA versions. If an EAP version is updated to a stable version at some point, the name of the original installation directory does not change. • Beta Releases or Public Previews: Includes stable releases, release candidates, and beta releases. Some updates in this channel may contain minor bugs and feature previews. • Stable Releases: Includes only stable releases that are recommended for production. You can choose the update channel only if you are using a <i>stable version</i>. For EAP builds, the channel is always set to Early Access Program.

Check for plugin updates	Select whether you want IntelliJ IDEA to check for new versions of plugins automatically.
Check for Updates	Check for updates immediately. Alternatively, from the main menu, select Help Check for Updates on Windows or Linux, or IntelliJ IDEA Check for Updates on macOS.
Manage ignored updates	Show the list of updates that were ignored. These updates will not be suggested until you remove them from the list of ignored updates. If you remove several updates from the ignored list, only the most recent will be offered for download when you check for updates.
Show What's New in the editor after an IDE update	Open a tab with information about new features and improvements after a major IDE update.

Snap package

If you installed IntelliJ IDEA as a snap package, it will manage updates automatically. All snaps are updated automatically in the background every day. You can also get the latest version of all snaps manually at any time by running the following command:

```
$ sudo snap refresh
Copied!
```

Or if you want to update only the IntelliJ IDEA snap:

IntelliJ IDEA Ultimate

IntelliJ IDEA Edu

Community Edition

```
$ sudo snap refresh intellij-idea-ultimate
```

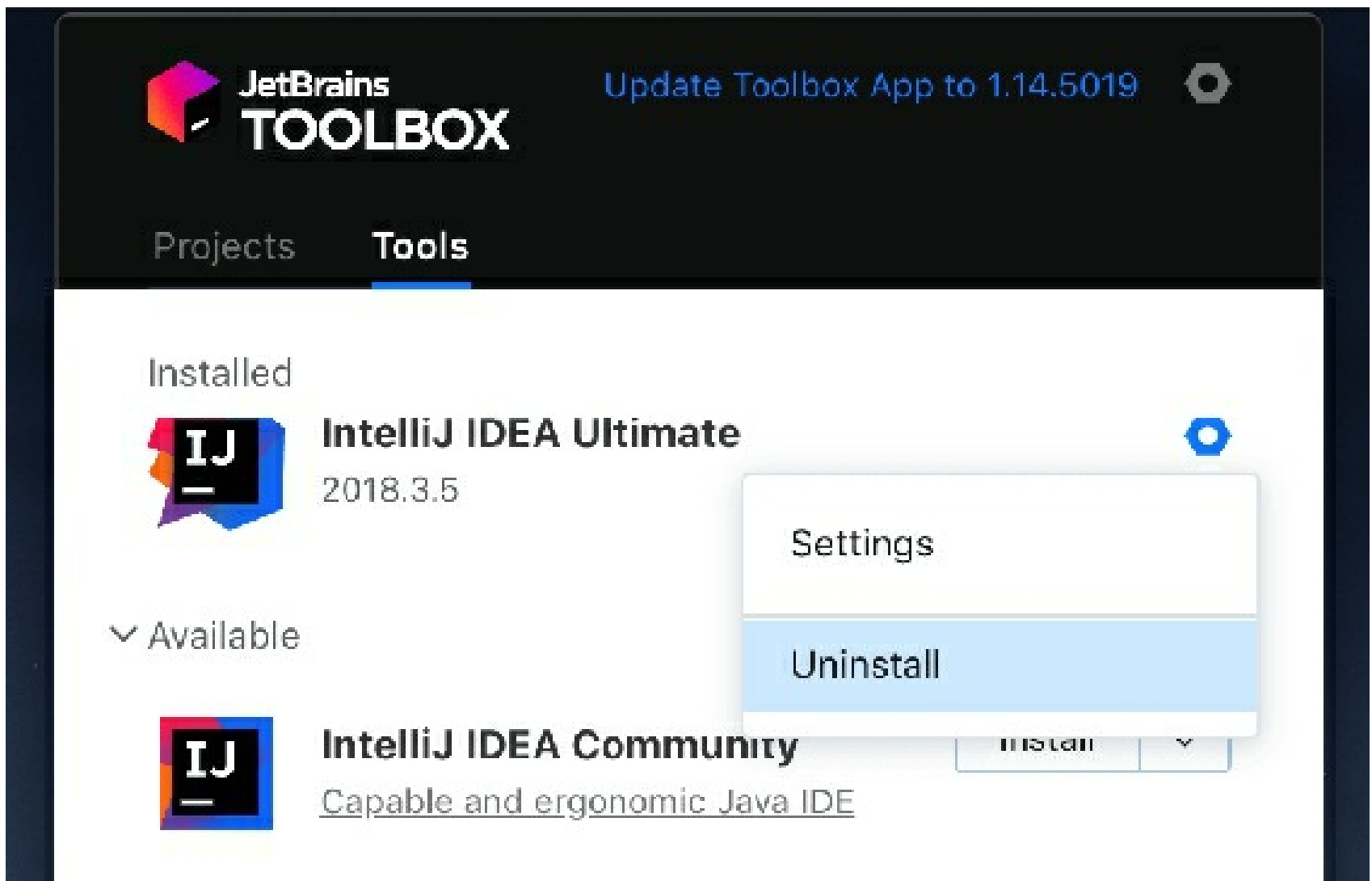

Uninstall IntelliJ IDEA

The proper way to remove IntelliJ IDEA depends on the method you used to install it.

Uninstall using the Toolbox App

If you installed IntelliJ IDEA using the Toolbox App, do the following:

- Open the Toolbox App, click the screw nut icon for the necessary instance, and select **Uninstall**.



Uninstall a standalone instance

If you are running a standalone IntelliJ IDEA instance, the IDE configuration and system directories are preserved when you remove your instance in case you want to keep your settings for later or to use them with another instance, another version, or another IDE. You can remove those directories if you are sure you won't need them.

Windows

macOS

Linux

1. Open the **Apps & Features** section in the Windows **Settings** dialog, select the IntelliJ IDEA app and click **Uninstall**.

Depending on your version of Windows, the procedure for uninstalling programs may be different. You can also manually run **Uninstall.exe** in the installation directory under **/bin**.

2. Remove the following directories:

Syntax

%APPDATA%\JetBrains\<product><version>

%LOCALAPPDATA%\JetBrains\<product><version>

Example

C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJdea2021.1

C:\Users\JohnS\AppData\Local\JetBrains\IntelliJ\Idea2021.1

The default IDE directories changed starting from IntelliJ IDEA 2020.1. If you had a previous version, new installations will import configuration from the old directories. For information about the location of the default directories in previous IDE versions, see the corresponding help version, for example: <https://www.jetbrains.com/help/idea/2019.3/tuning-the-ide.html#default-dirs>.

Uninstall silently on Windows

If you installed IntelliJ IDEA silently, you can run the uninstaller with the `/s` switch as an administrator. The uninstaller is located in the installation directory under **bin**.

Run **cmd** (Windows Command Prompt) as administrator, change to the IntelliJ IDEA installation directory, and run the following:

```
>
bin\uninstall.exe /S
Copied!
```

Uninstall the snap package on Linux

If you installed IntelliJ IDEA as a snap package, use the following command to remove it:

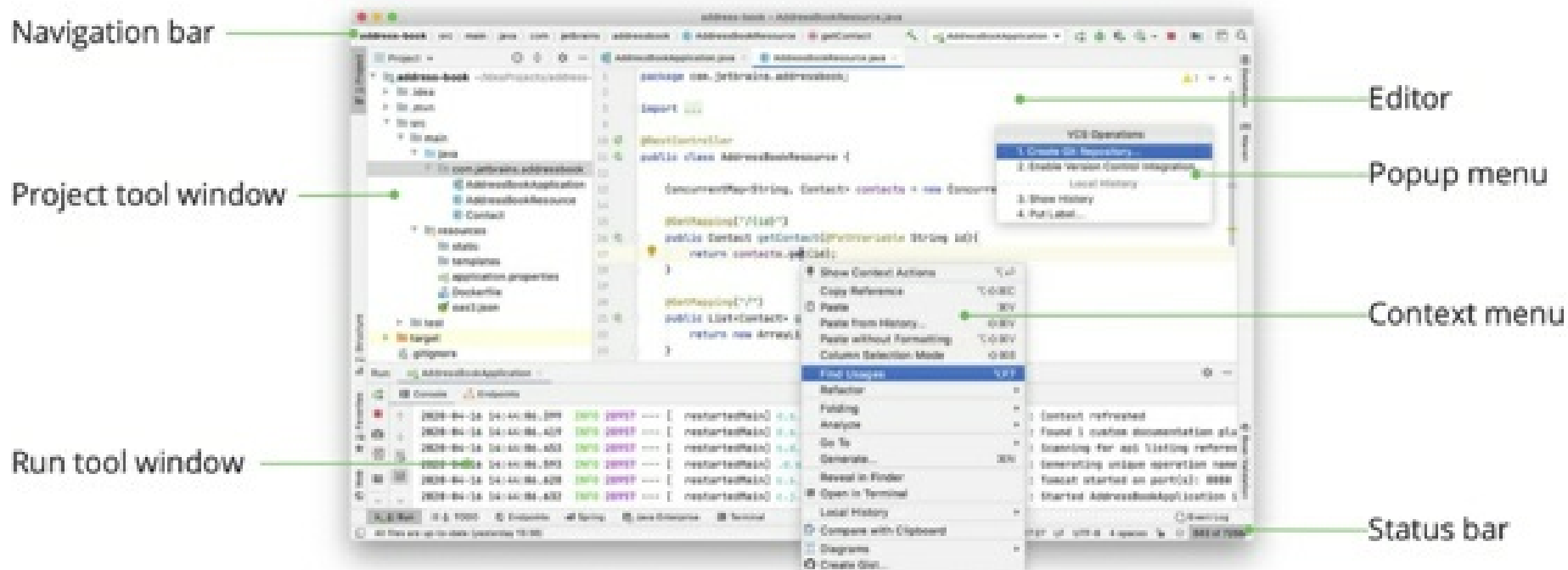
IntelliJ IDEA Ultimate

Community Edition

```
$
sudo snap remove intellij-idea-ultimate
```

Overview of the user interface

When you open a project in IntelliJ IDEA, the default user interface looks as follows:



Depending on the set of plugins, IntelliJ IDEA edition, and configuration settings, your IDE may look and behave differently.

Editor

Focus: Escape

Use the editor to read, write, and explore your source code.

Navigation bar

Focus: Alt+Home

Show/hide: **View | Appearance | Navigation Bar**

The navigation bar at the top is a quick alternative to the Project tool window **Workspace** tool window where you can navigate the structure of your project and open files for editing.

If VCS integration is enabled, items in the navigation bar are highlighted according to VCS file status colors.

Use the buttons to the right of the navigation bar to build, run, and debug your application, access your project structure settings, and perform basic version control operations (if the version control integration is configured). It also contains buttons to **Run Anything** (press Ctrl twice) and **Search Everywhere** (press Shift twice).

The main toolbar with buttons for opening and saving files, undo and redo actions is hidden by default.

To show it, select **View | Appearance | Toolbar**.

Status bar

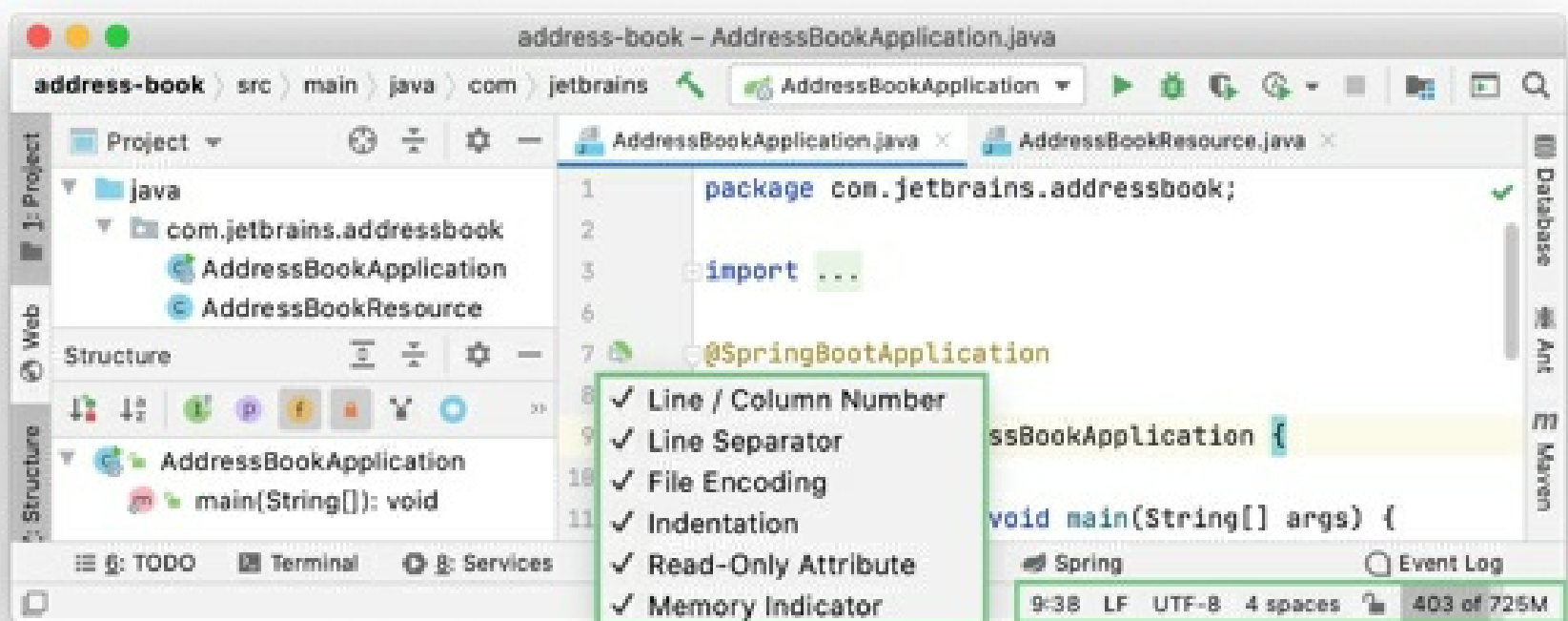
Show/hide: **View | Appearance | Status Bar**

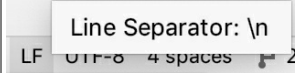
The left part of the status bar at the bottom of the main window shows the most recent event messages and descriptions of actions when you hover over them with the mouse pointer. Click a message in the status bar to open it in the **Event Log**. Right-click the message in the status bar and select **Copy** to paste the message text when you are searching for a solution to a problem or need to add it to a support ticket or to the IntelliJ IDEA issue tracker.

Use the quick access button or to switch between tool windows and hide the tool window bars.

The status bar also shows the progress of background tasks. You can click to show the **Background Tasks** manager.

The right part of the status bar contains widgets that indicate the overall project and IDE status and provide access to various settings. Depending on the set of plugins and configuration settings, the set of widgets can change. Right-click the status bar to select the widgets that you want to show or hide.



Widget	Description
52:11	Shows the line and column number of the current caret position in the editor. Click the numbers to move the caret to a specific line and column. If you select a code fragment in the editor, IntelliJ IDEA also shows the number of characters and line breaks in the selected fragment.
	Shows the line endings used to break lines in the current file. Click this widget to change the line separators.
UTF-8	Shows the encoding used to view the current file. Click the widget to use another encoding.
Column	Indicates that the column selection mode is enabled for the current editor tab. You can press Alt+Shift+Insert to toggle it.
	Click to lock the file from editing (set it to read-only) or unlock it if you want to edit the file.
Git: master	If version control integration is enabled, this widget shows the current VCS branch. Click it to manage VCS branches.
2 spaces	Shows the indent style used in the current file. Click to configure the tab and indent settings for the current file type or disable indent detection in the current project.
	Shows the amount of memory that IntelliJ IDEA consumes out of the total

amount of heap memory. For more information, see [Increase the memory heap of the IDE](#).

Tool windows

Show/hide: **View | Tool Windows**

Tool windows provide functionality that supplements editing code. For example, the Project tool window shows you the structure of your project, and the Run tool window displays the output of your application when you run it.

By default, tool windows are docked to the sides and bottom of the main window. You can arrange them as necessary, undock, resize, hide, and so on. Right-click the title of the tool window or click in the title for its arrangement options.

You can assign shortcuts to quickly access the tool windows that you frequently use. Some of them have shortcuts by default. For example, to open the Project tool window, press `Alt+1`, and to open the Terminal tool window, press `Alt+F12`. To jump from the editor to the last active tool window, press `F12`.

Context menus

You can right-click various elements of the interface to see the actions available in the current context.

For example, right-click a file in the Project tool window for actions related to that file, or right-click in the editor to see actions that apply to the current code fragment.

Most of these actions can also be performed from the main menu at the top of the screen or the main window. Actions with shortcuts show the shortcut next to the action name.

Popup menus

Popup menus provide quick access for actions related to the current context. Here are some useful popup menus and their shortcuts:

- `Alt+Insert` opens the **Generate** popup for generating boilerplate code based on the context.
- `Ctrl+Alt+Shift+T` opens the **Refactor This** popup with a list of contextually available refactorings.
- `Alt+Insert` in the Project tool window opens the **New** popup for adding new files and directories to your project.
- `Alt+`` opens the **VCS Operations** popup with contextually available actions for your version control system.

You can create custom popup menus using quick lists of actions that you often use.

Main window

The main window contains all the information for a single IntelliJ IDEA project. You can open multiple projects in multiple windows. By default, the window header contains the name of the project and the name of the currently open file. If there are multiple modules, it will also show the name of the relevant module.

Show full paths in the header

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, open **Appearance & Behavior | Appearance** and select the **Always show full paths in window header** checkbox.

This will show the path to the project and to the current file.

Create your first Java application

In this tutorial, you will learn how to create, run, and package a simple Java application that prints `Hello, World!` to the system output. Along the way, you will get familiar with IntelliJ IDEA features for boosting your productivity as a developer: coding assistance and supplementary tools.

Prepare a project

Create a new Java project

In IntelliJ IDEA, a project helps you organize your source code, tests, libraries that you use, build instructions, and your personal settings in a single unit.

1. Launch IntelliJ IDEA.

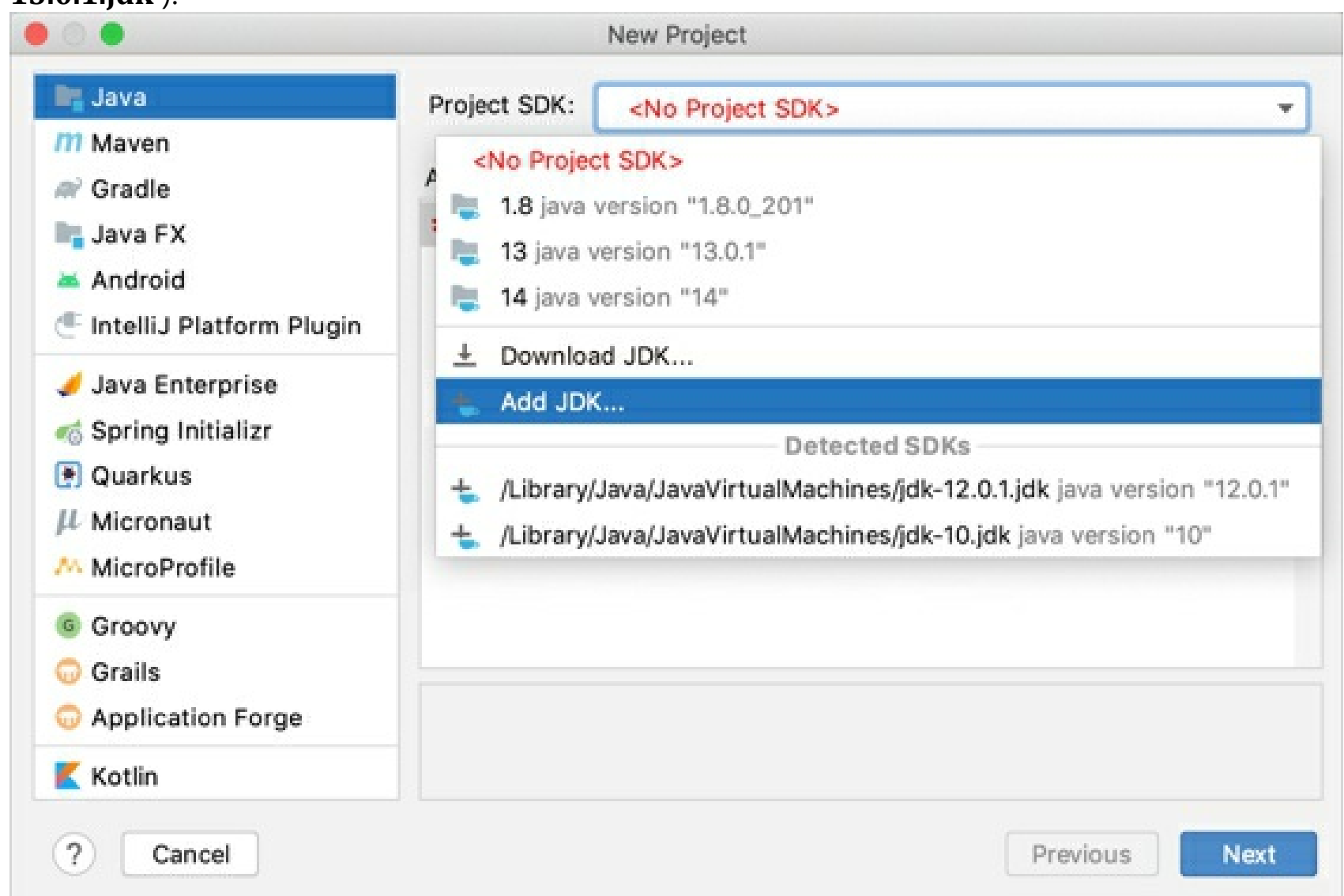
If the Welcome screen opens, click **New Project**.

Otherwise, from the main menu, select **File | New Project**.

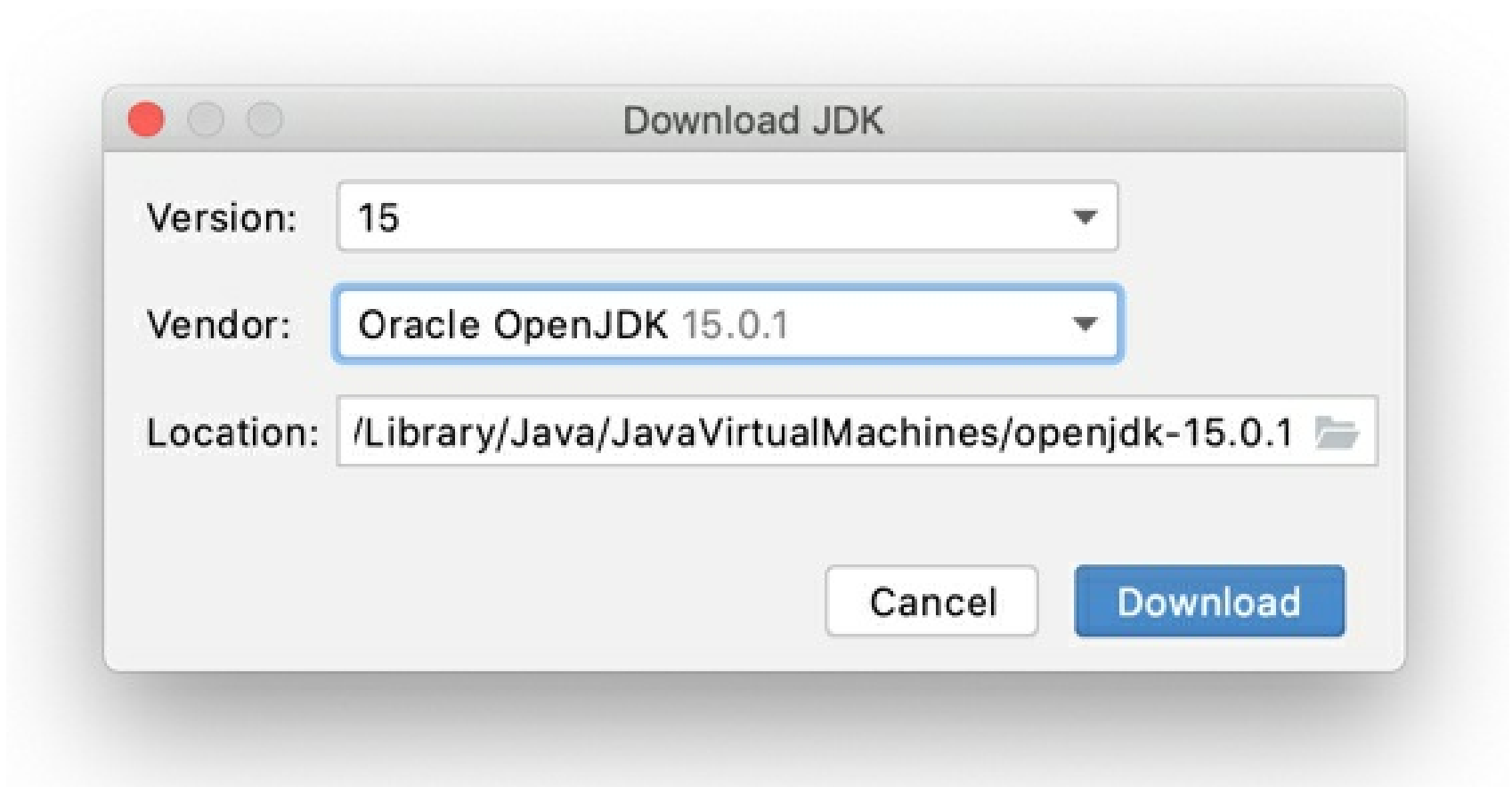
2. In the **New Project** wizard, select **Java** from the list on the left.
3. To develop Java applications in IntelliJ IDEA, you need the Java SDK (JDK).

If the necessary JDK is already defined in IntelliJ IDEA, select it from the **Project SDK** list.

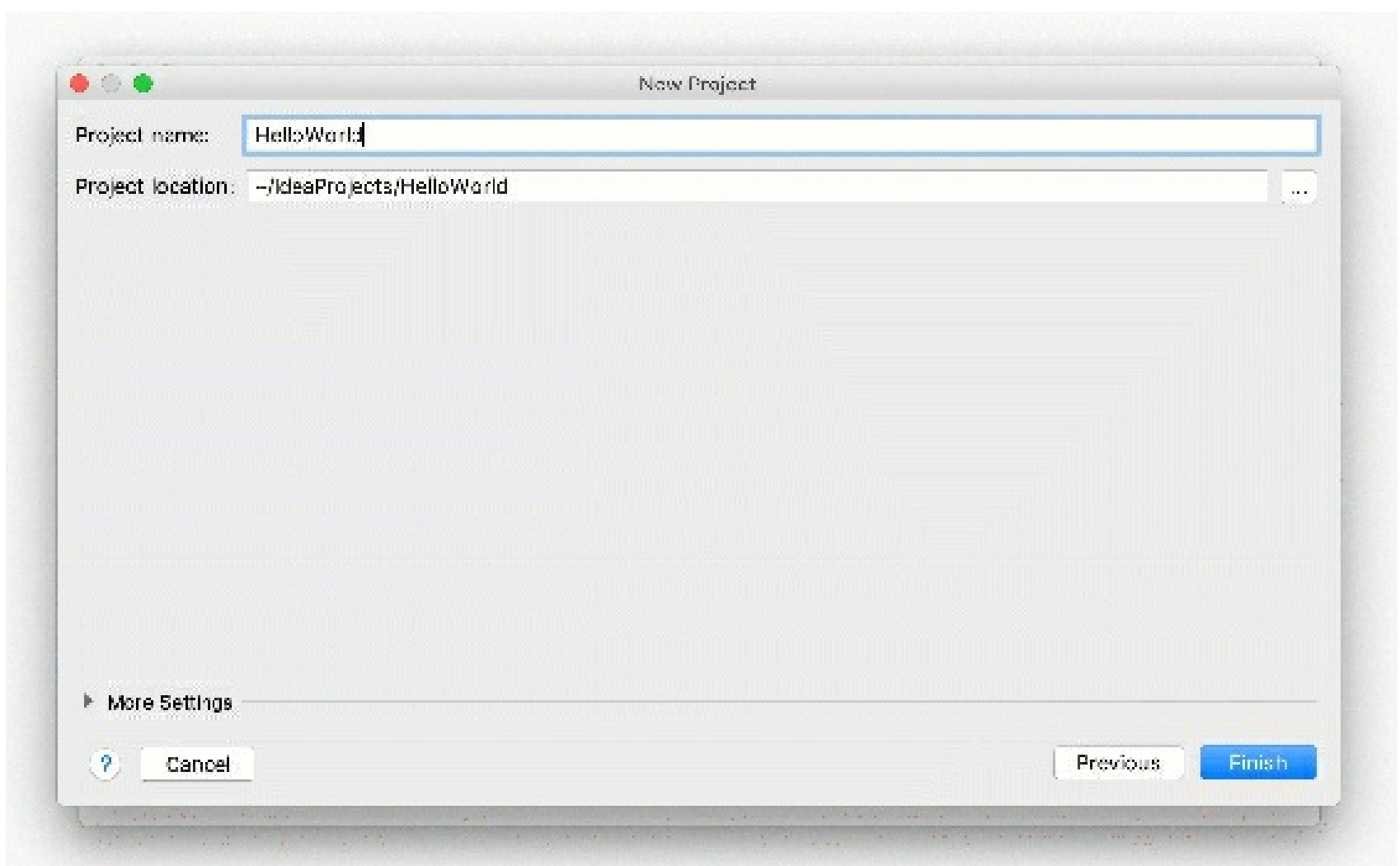
If the JDK is installed on your computer, but not defined in the IDE, select **Add JDK** and specify the path to the JDK home directory (for example, `/Library/Java/JavaVirtualMachines/jdk-13.0.1.jdk`).



If you don't have the necessary JDK on your computer, select **Download JDK**. In the next dialog, specify the JDK vendor (for example, OpenJDK), version, change the installation path if required, and click **Download**.



4. We're not going to use any additional libraries or frameworks for this tutorial, so click **Next**.
5. Don't create a project from the template. In this tutorial, we're going to do everything from scratch, so click **Next**.
6. Name the project, for example: HelloWorld.
7. If necessary, change the default project location and click **Finish**.



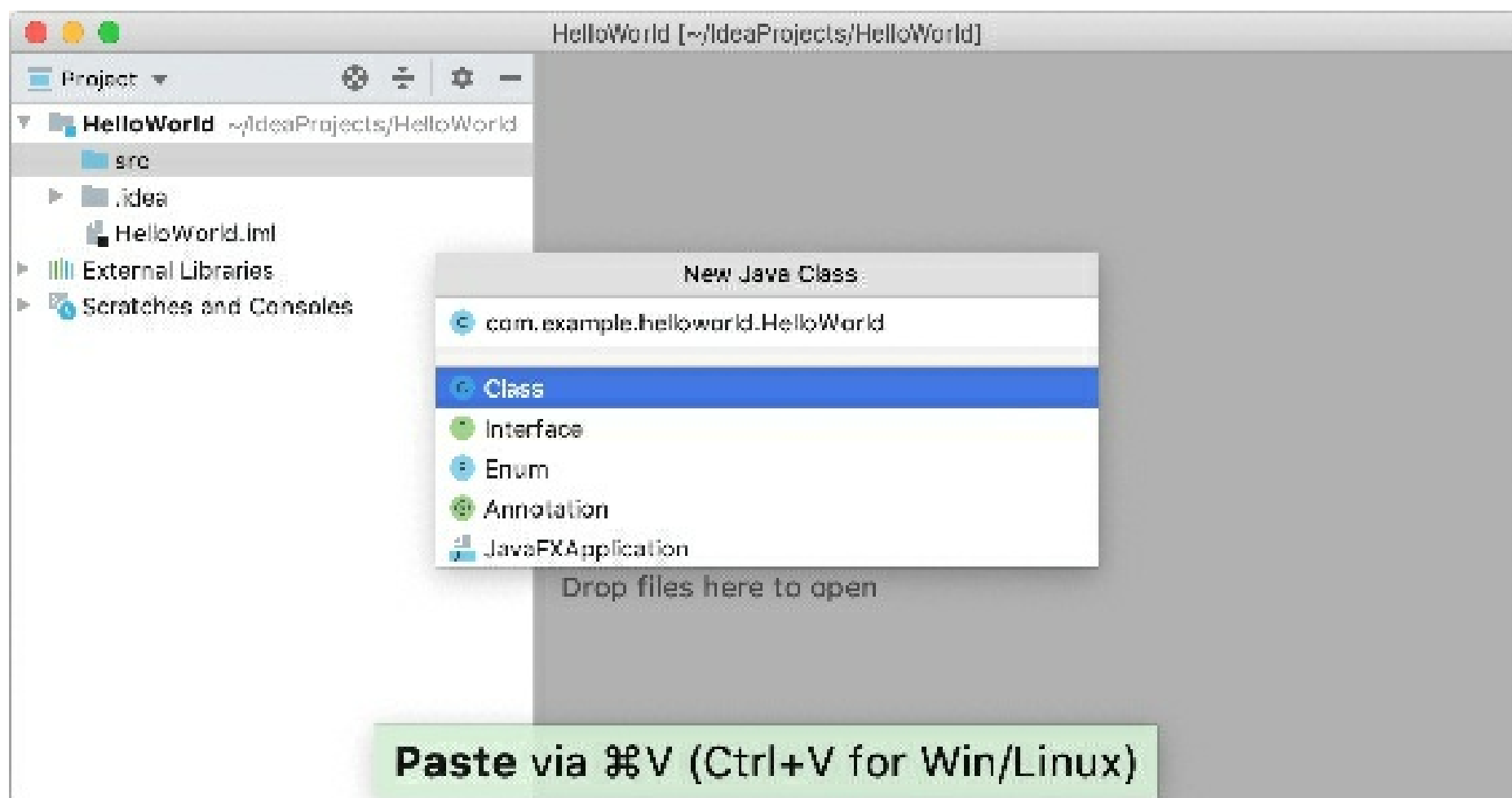
Gif

Create a package and a class

Packages are used for grouping together classes that belong to the same category or provide similar functionality, for structuring and organizing large applications with hundreds of classes.

1. In the **Project** tool window, select the **src** folder, press **Alt+Insert**, and select **Java Class**.
2. In the **Name** field, type `com.example.helloworld.HelloWorld` and click **OK**.

IntelliJ IDEA creates the `com.example.helloworld` package and the `HelloWorld` class.



Gif

Together with the file, IntelliJ IDEA has automatically generated some contents for your class. In this case, the IDE has inserted the package statement and the class declaration.

This is done by means of file templates. Depending on the type of the file that you create, the IDE inserts initial code and formatting that is expected to be in all files of that type. For more information on how to use and configure templates, refer to [File templates](#).

The **Project** tool window `Alt+1` displays the structure of your application and helps you browse the project. In Java, there's a naming convention that you should follow when you name packages and classes.

Write the code

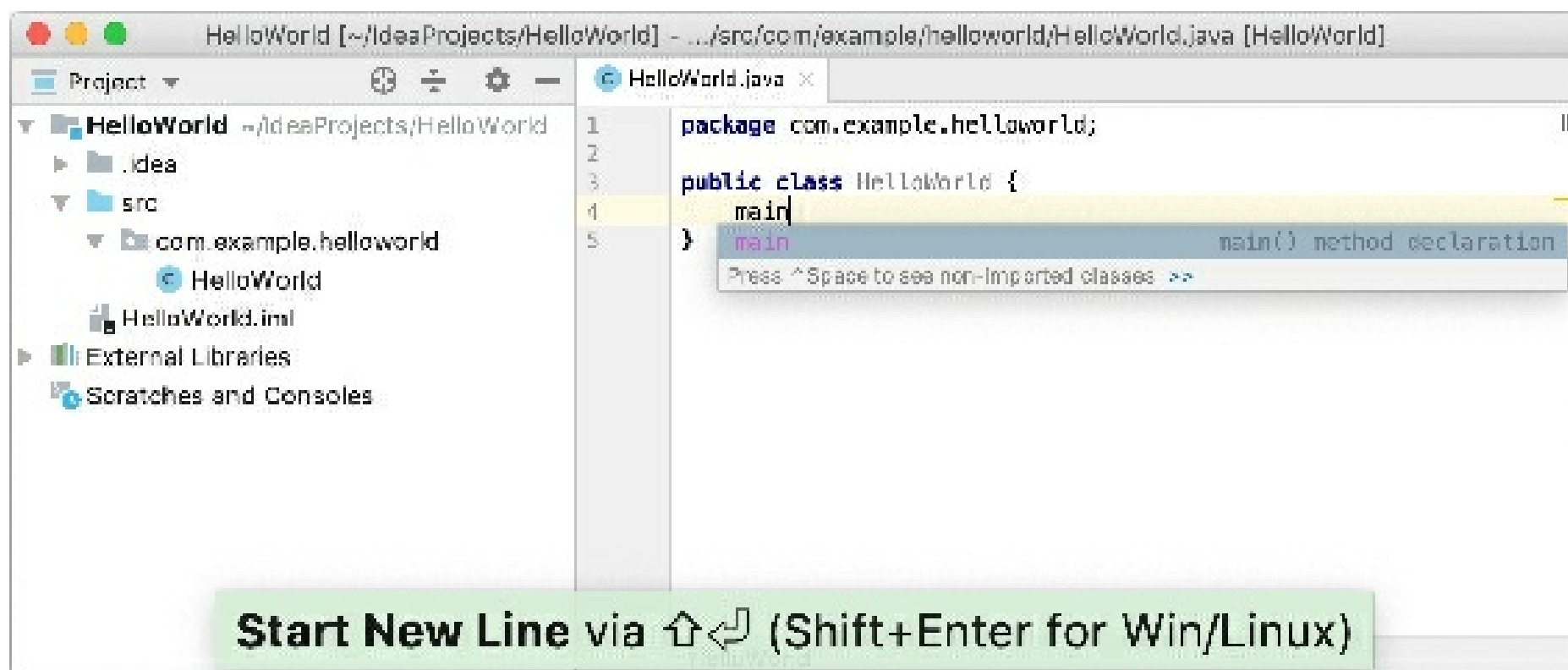
Add the `main()` method using live templates

1. Place the caret at the class declaration string after the opening bracket `{` and press `Shift+Enter`.

In contrast to `Enter`, `Shift+Enter` starts a new line without breaking the current one.

2. Type `main` and select the template that inserts the `main()` method declaration.

As you type, IntelliJ IDEA suggests various constructs that can be used in the current context. You can see the list of available live templates using `Ctrl+J`.



Gif

Live templates are code snippets that you can insert into your code. `main` is one of such snippets. Usually, live templates contain blocks of code that you use most often. Using them can save you some time as you don't have to type the same code over and over again.

For more information on where to find predefined live templates and how to create your own, refer to Live templates.

You can also add the statement using the `sout` live template.

Call the `println()` method using code completion

After the `main()` method declaration, IntelliJ IDEA automatically places the caret at the next line. Let's call a method that prints some text to the standard system output.

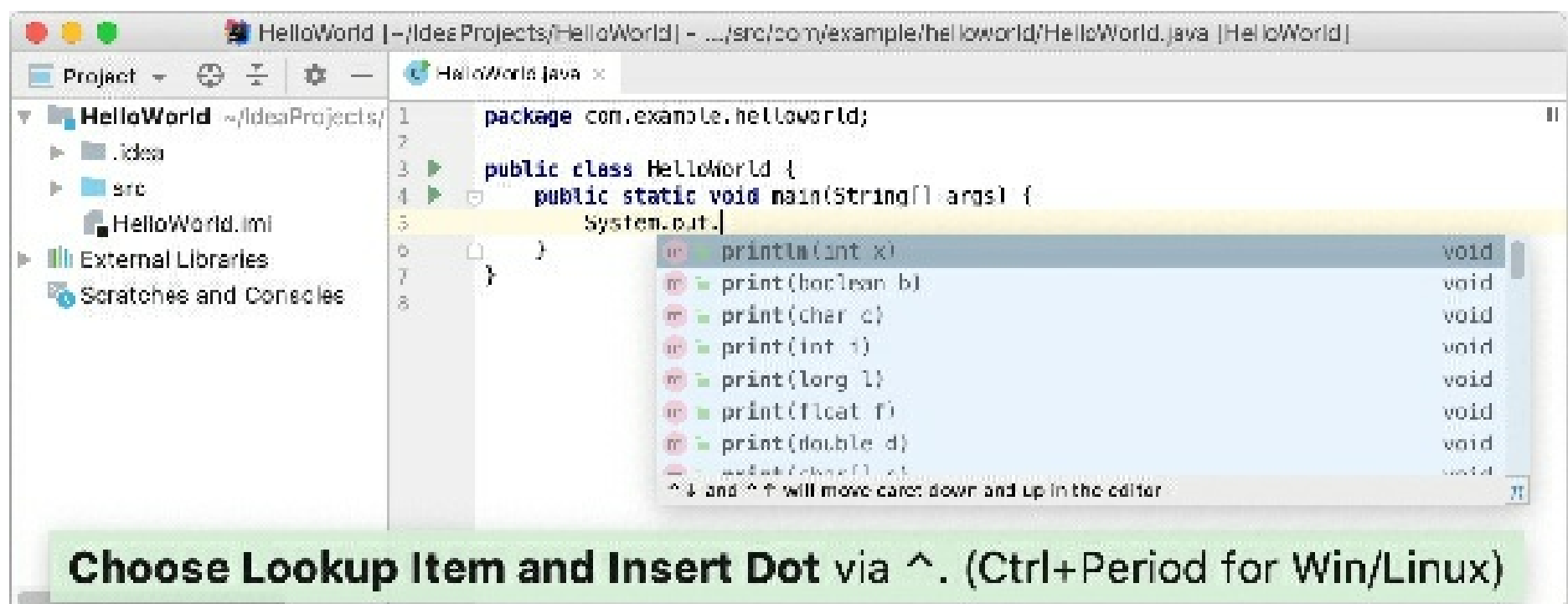
1. Type `sy` and select the `System` class from the list of code completion suggestions (it's from the standard `java.lang` package).

Press `Ctrl+.` to insert the selection with a trailing comma.

2. Type `o`, select `out`, and press `Ctrl+.` again.
3. Type `p`, select the **`println(String x)`** method, and press `Enter`.

IntelliJ IDEA shows you the types of parameters that can be used in the current context. This information is for your reference.

4. Type `"`. The second quotation mark is inserted automatically, and the caret is placed between the quotation marks. Type `Hello, World!`



Gif

Basic code completion analyses the context around the current caret position and provides suggestions as you type. You can open the completion list manually by pressing **Ctrl+Space**. For information on different completion modes, refer to [Code completion](#).

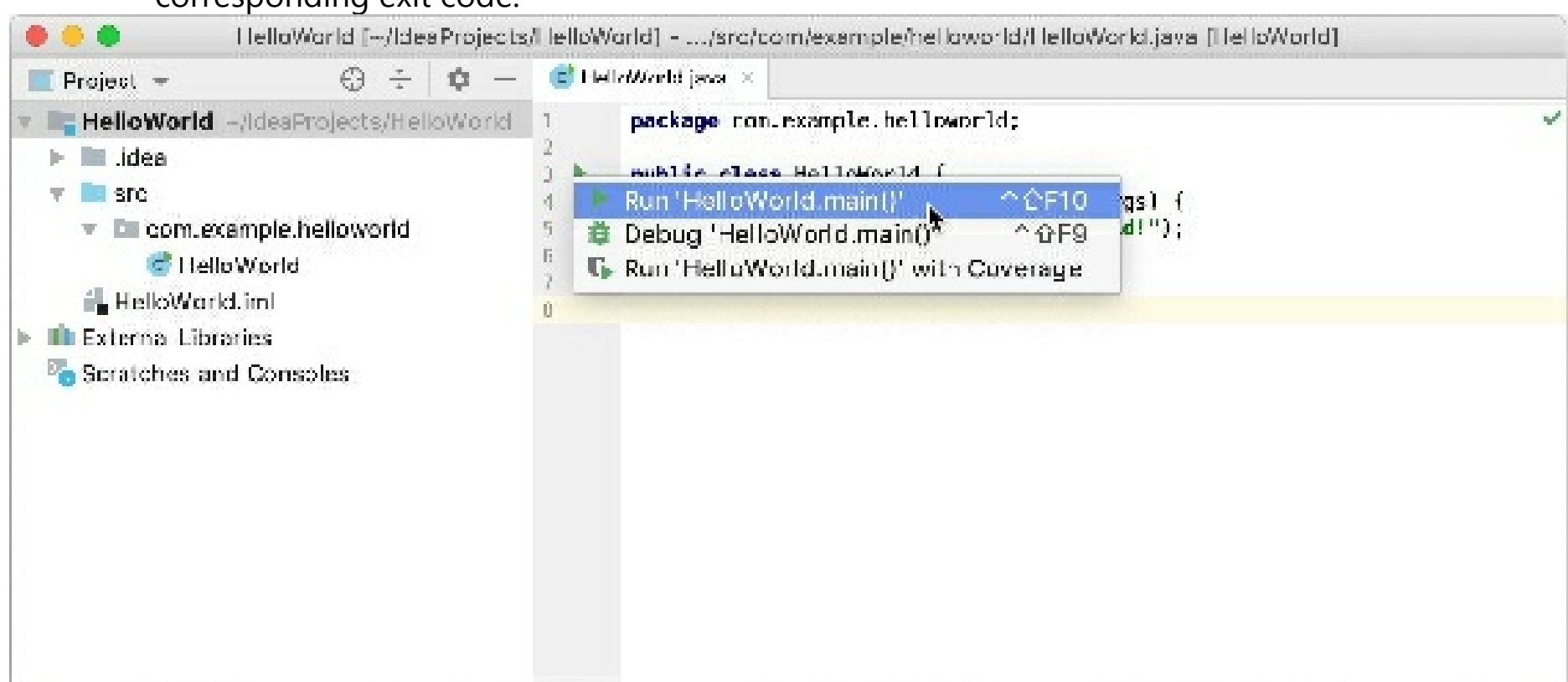
Build and run the application

Valid Java classes can be compiled into bytecode. You can compile and run classes with the `main()` method right from the editor using the green arrow icon in the gutter.

1. Click in the gutter and select **Run 'HelloWorld.main()'** in the popup. The IDE starts compiling your code.
2. When the compilation is complete, the **Run** tool window opens at the bottom of the screen.

The first line shows the command that IntelliJ IDEA used to run the compiled class. The second line shows the program output: `Hello, World!`. And the last line shows the exit code `0`, which indicates that it exited successfully.

If your code is not correct, and the IDE can't compile it, the **Run** tool window will display the corresponding exit code.



Gif

When you click **Run**, IntelliJ IDEA creates a special run configuration that performs a series of actions. First, it builds your application. On this stage, `javac` compiles your source code into JVM bytecode.

Once javac finishes compilation, it places the compiled bytecode to the **out** directory, which is highlighted with yellow in the **Project** tool window.

After that, the JVM runs the bytecode.

Automatically created run configurations are temporary, but you can modify and save them.

If you want to reopen the **Run** tool window, press `Alt+4`.

IntelliJ IDEA automatically analyzes the file that is currently opened in the editor and searches for different types of problems: from syntax errors to typos. The **Inspections** widget at the top-right corner of the editor allows you to quickly see all the detected problems and look at each problem in detail. For more information, refer to Current file.

Package the application in a JAR

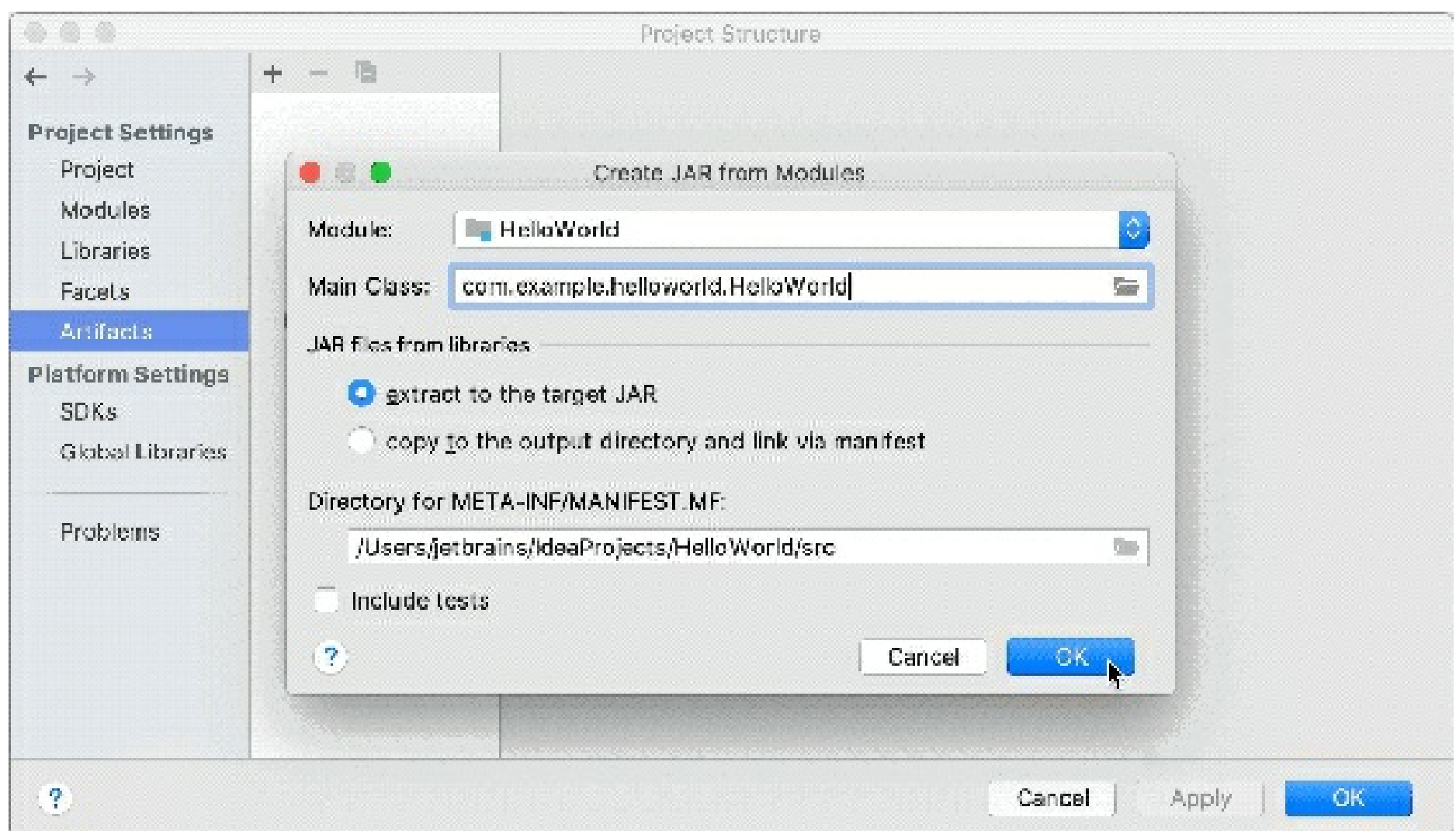
When the code is ready, you can package your application in a Java archive (JAR) so that you can share it with other developers. A built Java archive is called an *artifact*.

Create an artifact configuration for the JAR

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Artifacts**.
2. Click **+**, point to **JAR** and select **From modules with dependencies**.
3. To the right of the **Main Class** field, click **+** and select **HelloWorld (com.example.helloworld)** in the dialog that opens.

IntelliJ IDEA creates the artifact configuration and shows its settings in the right-hand part of the **Project Structure** dialog.

4. Apply the changes and close the dialog.

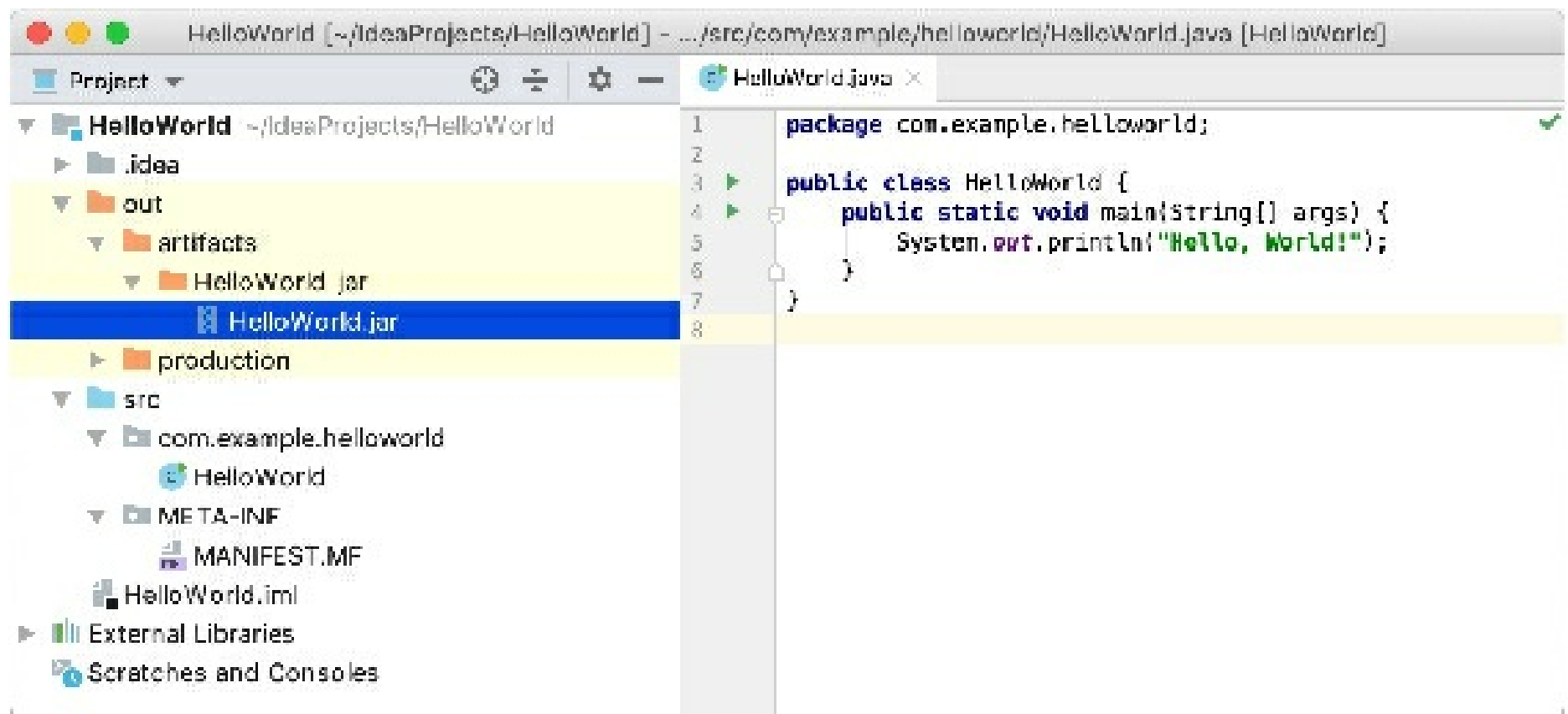


Gif

Build the JAR artifact

1. From the main menu, select **Build | Build Artifacts**.
2. Point to **HelloWorld:jar** and select **Build**.

If you now look at the **out/artifacts** folder, you'll find your JAR there.



Run the packaged application

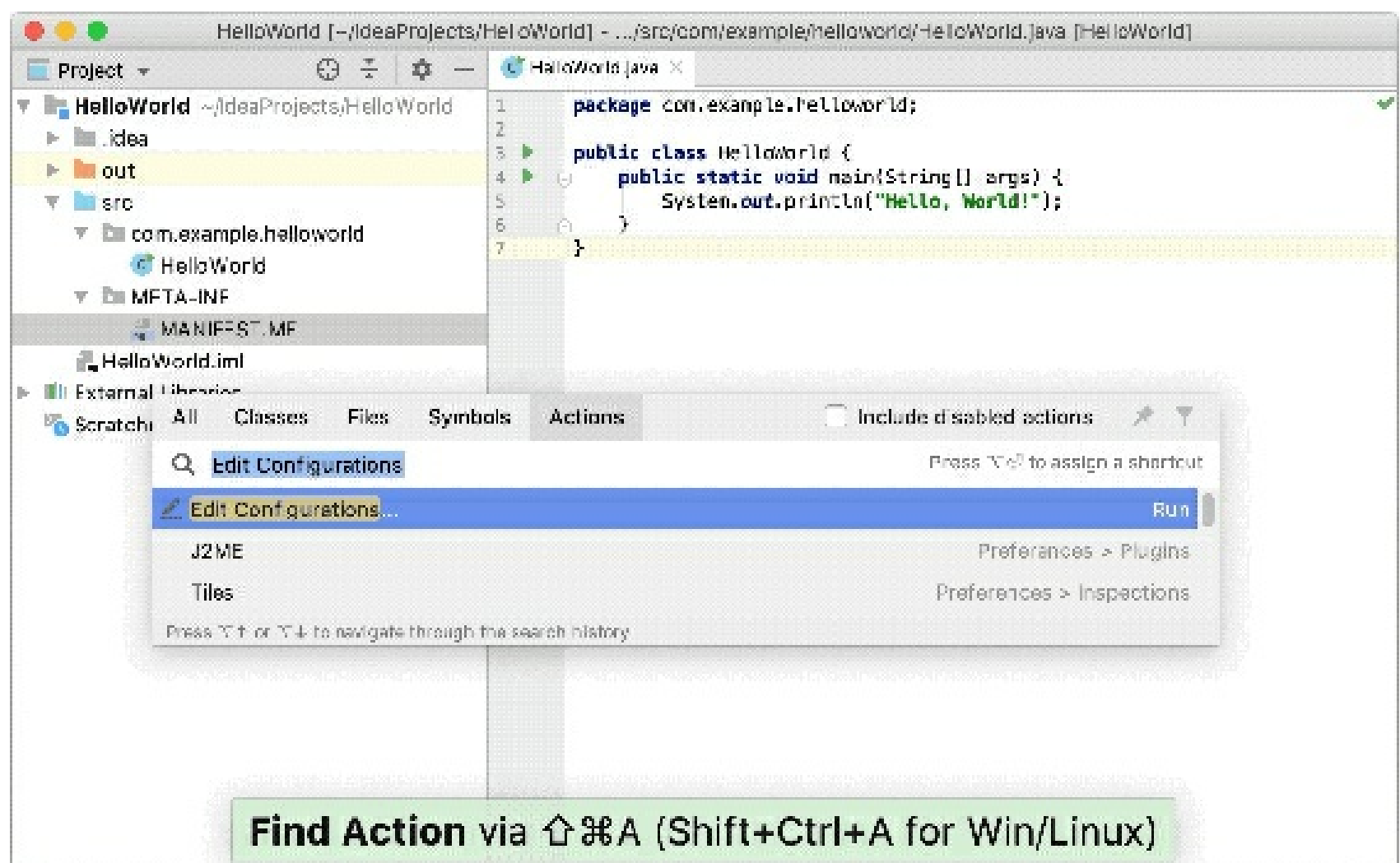
To make sure that the JAR artifact is created correctly, you can run it.

Use **Find Action** `Ctrl+Shift+A` to search for actions and settings across the entire IDE.

Create a run configuration for the packaged application

To run a Java application packaged in a JAR, IntelliJ IDEA allows you to create a dedicated run configuration.

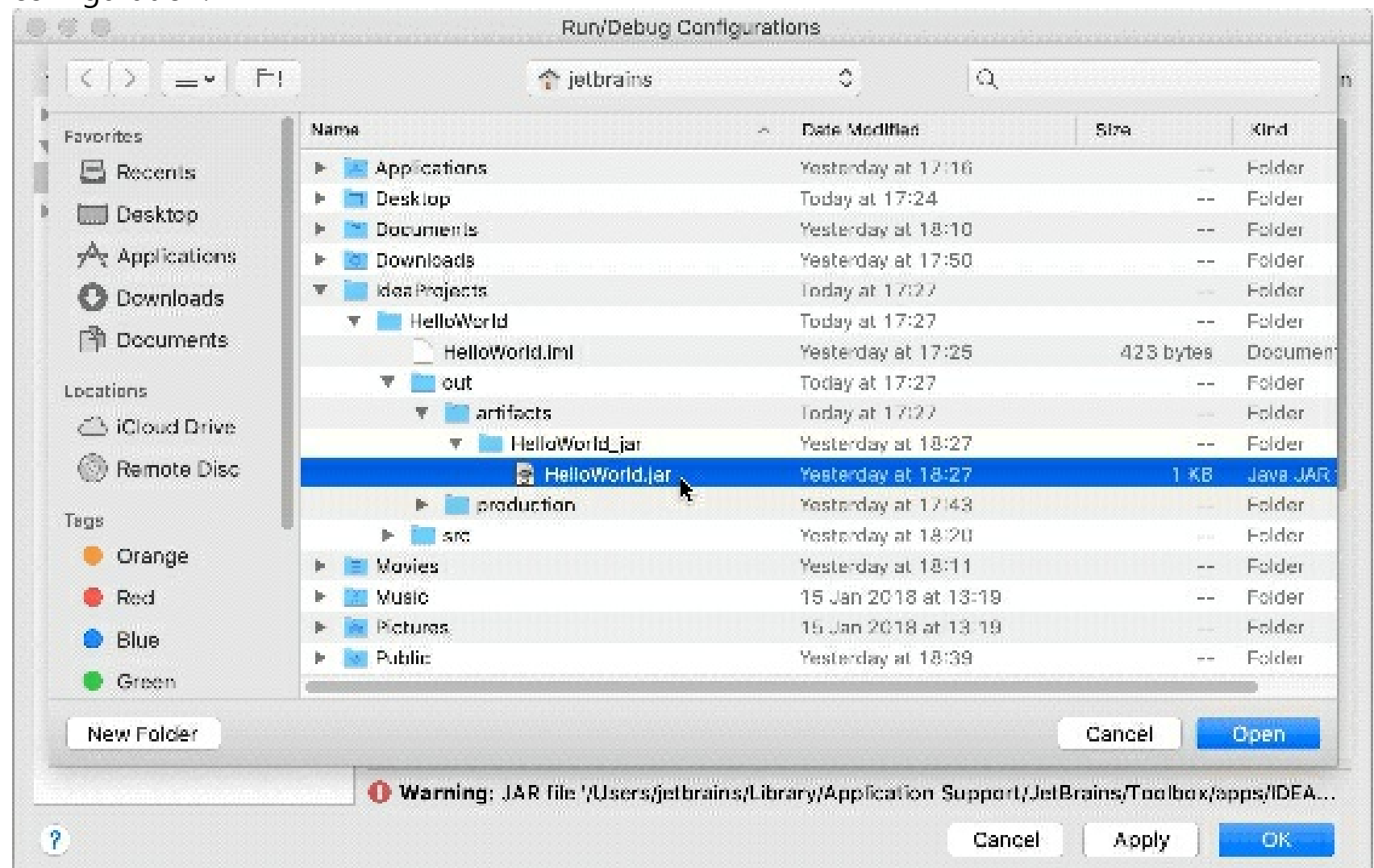
1. Press `Ctrl+Shift+A`, find and run the **Edit Configurations** action.
2. In the **Run/Debug Configurations** dialog, click and select **JAR Application**.
3. Name the new configuration: `HelloWorldJar`.



Gif

4. In the **Path to JAR** field, click and specify the path to the JAR file on your computer.
5. Under **Before launch**, click , select **Build Artifacts | HelloWorld.jar** in the dialog that opens.

Doing this means that the **HelloWorld.jar** is built automatically every time you execute this run configuration.



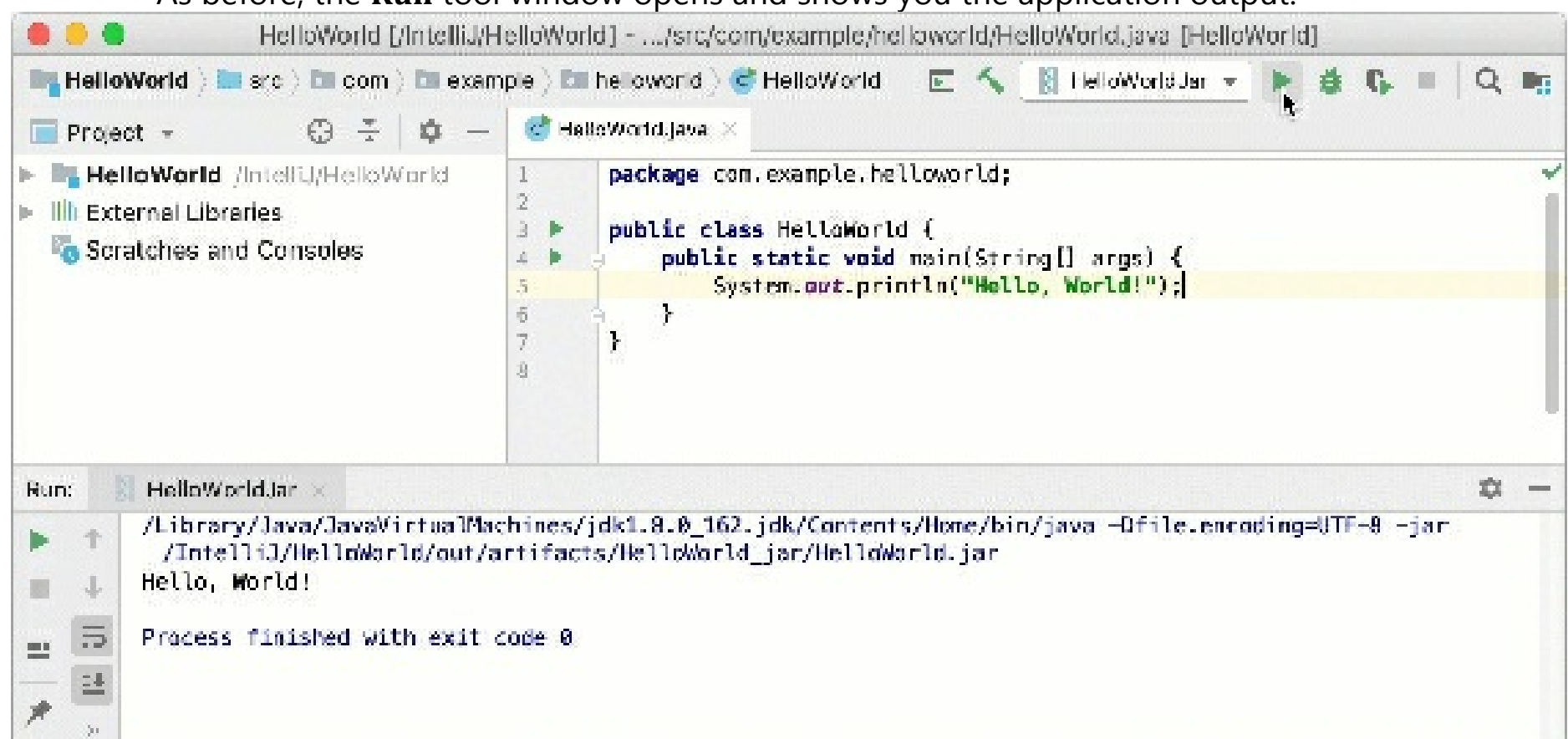
Gif

Run configurations allow you to define how you want to run your application, with which arguments and options. You can have multiple run configurations for the same application, each with its own settings.

Execute the run configuration

- On the toolbar, select the HelloWorldJar configuration and click to the right of the run configuration selector. Alternatively, press Shift+F10 if you prefer shortcuts.

As before, the **Run** tool window opens and shows you the application output.



Gif

The process has exited successfully, which means that the application is packaged correctly.

IntelliJ IDEA keyboard shortcuts

IntelliJ IDEA has keyboard shortcuts for most of its commands related to editing, navigation, refactoring, debugging, and other tasks. Memorizing these hotkeys can help you stay more productive by keeping your hands on the keyboard.

If your keyboard does not have an English layout, IntelliJ IDEA may not detect all of the shortcuts correctly.

The following table lists some of the most useful shortcuts to learn:

Shortcut	Action
Double Shift	Search Everywhere Quickly find any file, action, symbol, tool window, or setting in IntelliJ IDEA, in your project, and in the current Git repository.
Ctrl+Shift+A	Find Action Find a command and execute it, open a tool window, or search for a setting.
Alt+Enter	Show Context Actions Quick-fixes for highlighted errors and warnings, intention actions for improving and optimizing your code.
F2 Shift+F2	Navigate between code issues Jump to the next or previous highlighted error.
Ctrl+E	View recent files Select a recently opened file from the list.
Ctrl+Shift+Enter	Complete Current Statement Insert any necessary trailing symbols and put the caret where you can start typing the next statement.
Ctrl+Alt+L	Reformat Code Reformat the whole file or the selected fragment according to the current code style settings.
Ctrl+Alt+Shift+T	Invoke refactoring Refactor the element under the caret, for example, safe delete, copy, move, rename, and so on.
Ctrl+W Ctrl+Shift+W	Extend or shrink selection Increase or decrease the scope of selection according to specific code constructs.
Ctrl+/ Ctrl+Shift+/	Add/remove line or block comment Comment out a line or block of code.

Ctrl+B	Go To Declaration Navigate to the initial declaration of the instantiated class, called method, or field.
Alt+F7	Find Usages Show all places where a code element is used across your project.
Alt+1	Focus the Project tool window
Escape	Focus the editor

If you are using one of the predefined keymaps for your OS, you can print the default keymap reference card and keep it on your desk to consult it if necessary. This cheat sheet is also available under **Help | Keymap Reference**.

Choose the right keymap

To view the keymap configuration, open the **Settings/Preferences** dialog `Ctrl+Alt+S` and select **Keymap**. Enable function keys and check for possible conflicts with global OS shortcuts.

- *Use a predefined keymap*

IntelliJ IDEA automatically suggests a predefined keymap based on your environment. Make sure that it matches the OS you are using or select the one that matches shortcuts from another IDE or editor you are used to (for example, Eclipse or NetBeans). When consulting this page and other pages in IntelliJ IDEA documentation, you can see keyboard shortcuts for the keymap that you use in the IDE — choose it using the selector at the top of the page.

- *Tune your keymap*

You can modify a copy of any predefined keymap to assign your own shortcuts for commands that you use frequently.

- *Import custom keymap*

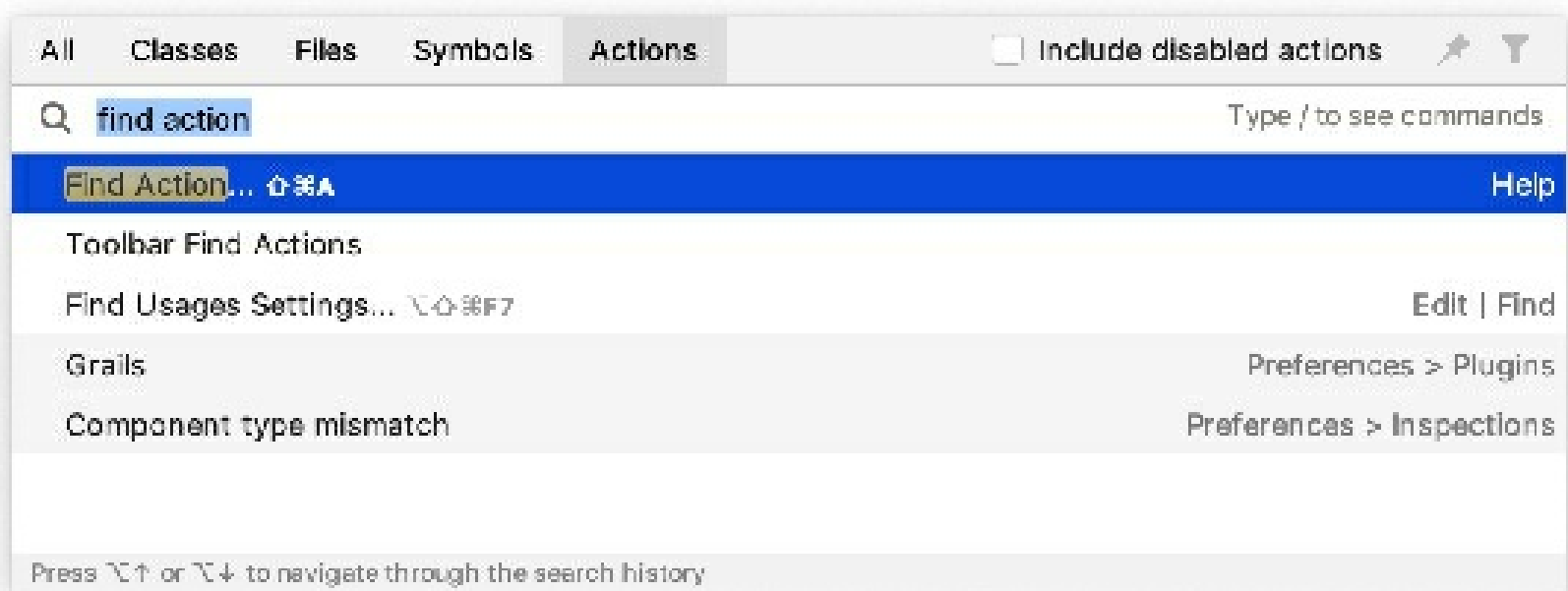
If you have a customized keymap that you are used to, you can transfer it to your installation. Besides the default set of keymaps, you can add more as plugins (such as, keymaps for GNOME and KDE): open the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Plugins** and search for *keymap* in the Marketplace. If your keymap stopped working after an update, it is likely that the keymap is not available by default in the new version of IntelliJ IDEA. Find this keymap as a plugin and install it on the Plugins page as described in Manage plugins.

Learn shortcuts as you work

IntelliJ IDEA provides several possibilities to learn shortcuts:

- Find Action is the most important command that enables you to search for commands and settings across all menus and tools.

Press `Ctrl+Shift+A` and start typing to get a list of suggested actions. Then select the necessary action and press `Enter` to execute it.



- Key Promoter X is a plugin that shows a popup notification with the corresponding keyboard shortcut whenever a command is executed using the mouse. It also suggests creating a shortcut for commands that are executed frequently.
- If you are using one of the predefined keymaps for your OS, you can print the default keymap reference card and keep it on your desk to consult it if necessary. This cheat sheet is also available under **Help | Keymap Reference**.
- To print a non-default or customized keymap, use the Keymap exporter plugin.

If an action has a keyboard shortcut associated with it, the shortcut is displayed near the name of the action. To add a shortcut for an action that you use frequently (or if you want to change an existing shortcut), select it and press `Alt+Enter`.

Use advanced features

You can further improve your productivity with the following useful features:

- **Quick Lists**

If there is a group of actions that you often use, create a quick list to access them using a custom shortcut. For example, you can try using the following predefined quick lists:

- **Refactor this** `Ctrl+Alt+Shift+T`
- **VCS Operations** `Alt+``

- **Smart Keys**

IntelliJ IDEA provides a lot of typing assistance features, such as automatically adding paired tags and quotes, and detecting *CamelHump* words.

- **Speed search**

When the focus is on a tool window with a tree, list, or table, start typing to see matching items.

- **Press twice**

Many actions in IntelliJ IDEA provide more results when you execute them multiple times. For example, when you invoke basic code completion with `Ctrl+Space` on a part of a field, parameter, or variable declaration, it suggests names depending on the item type within the current scope. If you invoke it again, it will include classes available through module dependencies. When invoked for the third time in a row, the list of suggestions will include the whole project.

- **Resize tool windows**

You can adjust the size of tool windows without a mouse:

- To resize a vertical tool window, use `Ctrl+Shift+Left` and `Ctrl+Shift+Right`
- To resize a horizontal tool window, use `Ctrl+Shift+Up` and `Ctrl+Shift+Down`

Manage plugins

Plugins extend the core functionality of IntelliJ IDEA. For example, by installing plugins, you can get the following features:

- integration with version control systems, Docker, Kubernetes, and other technologies
- coding assistance support for various languages and frameworks
- shortcut hints, live previews, file watchers, and so on
- coding exercises that can help you to learn a new programming language

The following video gives a short overview of the plugin subsystem.

Open plugin settings

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Plugins**.



Use the **Marketplace** tab to browse and install plugins from the JetBrains Plugin Repository or from a custom plugin repository.

Use the **Installed** tab to browse installed plugins, enable, disable, update, or remove them. Disabling unnecessary plugins can increase performance.

Most plugins can be used with any JetBrains product. Some are limited only to commercial products. There are also plugins that require a separate license.

If a plugin depends on some other plugin, IntelliJ IDEA will notify you about the dependencies. If your project depends on certain plugins, add them to the list of required plugins.

If existing plugins do not provide some functionality that you need, you can create your own plugin for IntelliJ IDEA. For more information, see [Develop your own plugins](#).

By default, IntelliJ IDEA includes a number of bundled plugins. You can disable bundled plugins, but they cannot be removed. You can install additional plugins from the plugin repository or from a local archive file (ZIP or JAR).

Install plugin from repository

1. Press `Ctrl+Alt+S` to open IDE settings and select **Plugins**.
2. Find the plugin in the **Marketplace** and click **Install**.

To install a specific version, go to the plugin page in the JetBrains Plugin Repository, download and install it as described in [Install plugin from disk](#). For example, you can do it if the most recent version of the plugin is broken.

Install plugin from disk

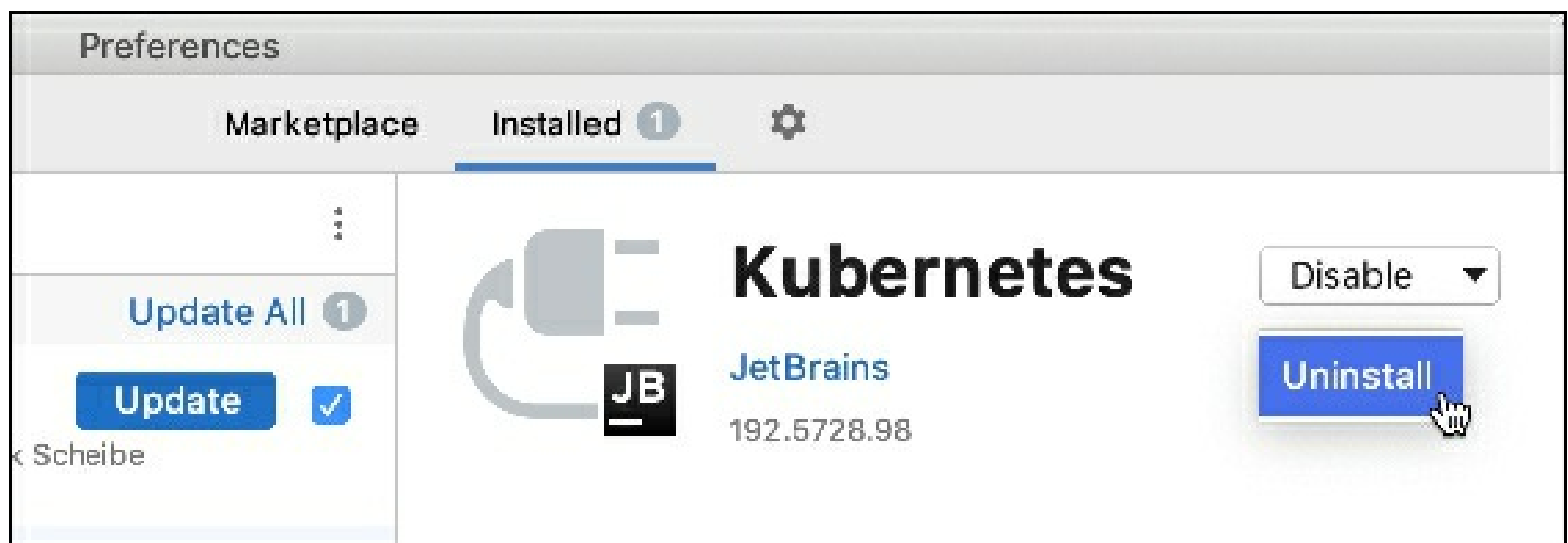
After you download the plugin archive (ZIP or JAR), do the following:

1. Press **Ctrl+Alt+S** to open IDE settings and select **Plugins**.
2. On the **Plugins** page, click  and then click **Install Plugin from Disk**.
3. Select the plugin archive file and click **OK**.
4. Click **OK** to apply the changes and restart the IDE if prompted.

Remove plugin

You cannot remove bundled plugins.

1. In the **Settings/Preferences** dialog **Ctrl+Alt+S**, select **Plugins**.
2. Open the **Installed** tab and find the plugin that you want to remove.
3. Click  next to the **Disable/ Enable** button and select **Uninstall** from the drop-down menu.



If you need to remove a plugin without launching IntelliJ IDEA, you can delete it manually from the plugin directory.

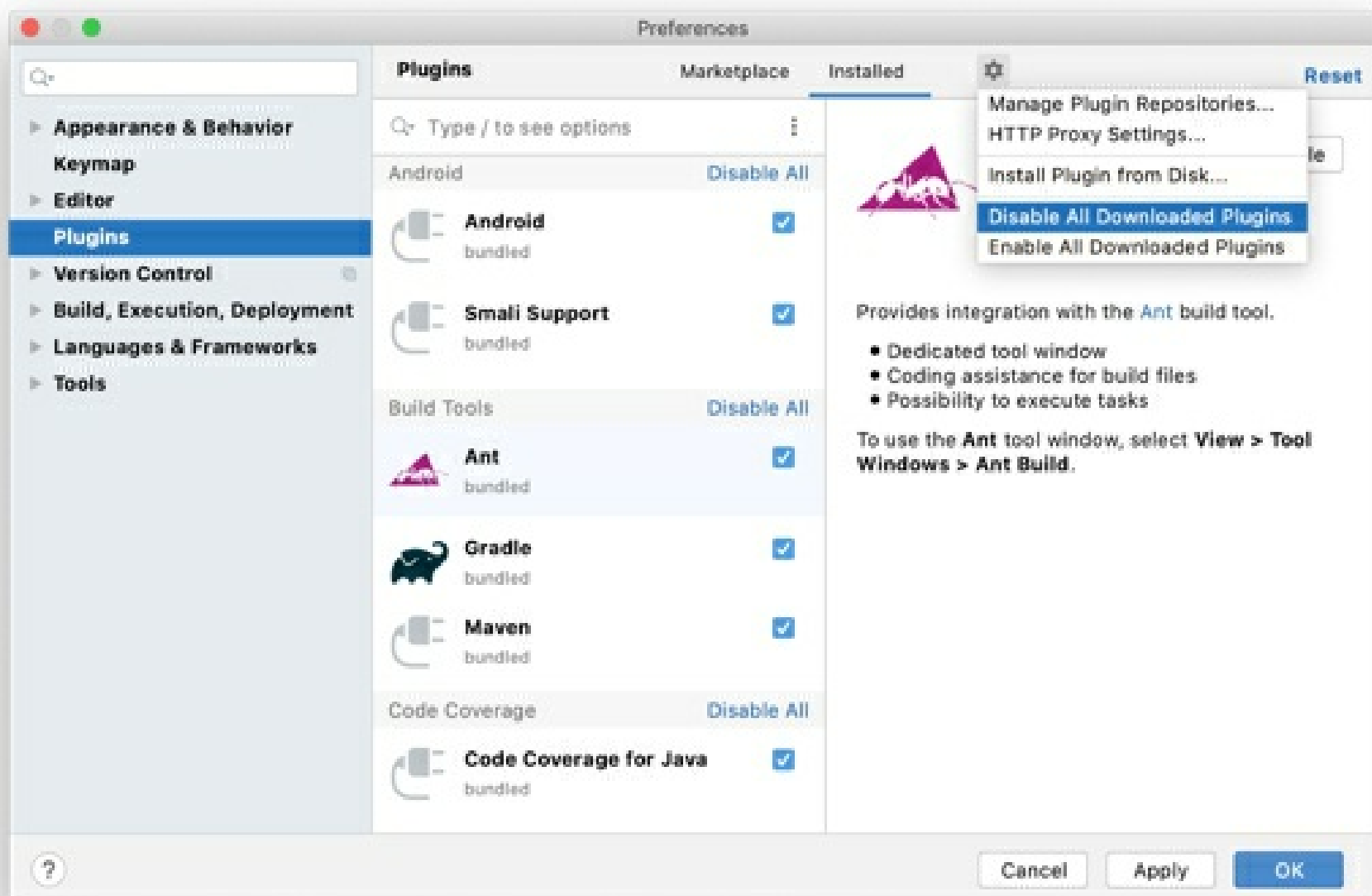
Disable plugin

You can disable a plugin without removing it if you do not need the corresponding functionality.

1. In the **Settings/Preferences** dialog **Ctrl+Alt+S**, select **Plugins**.
2. Open the **Installed** tab, find and select the plugin that you want to disable.
3. Click **Disable**. The button will change to **Enable**.

Alternatively, you can use the checkboxes in the list of plugins or the **Disable All** buttons for plugin categories.

You can disable or enable all manually installed plugins at once (non-bundled) in the menu under .



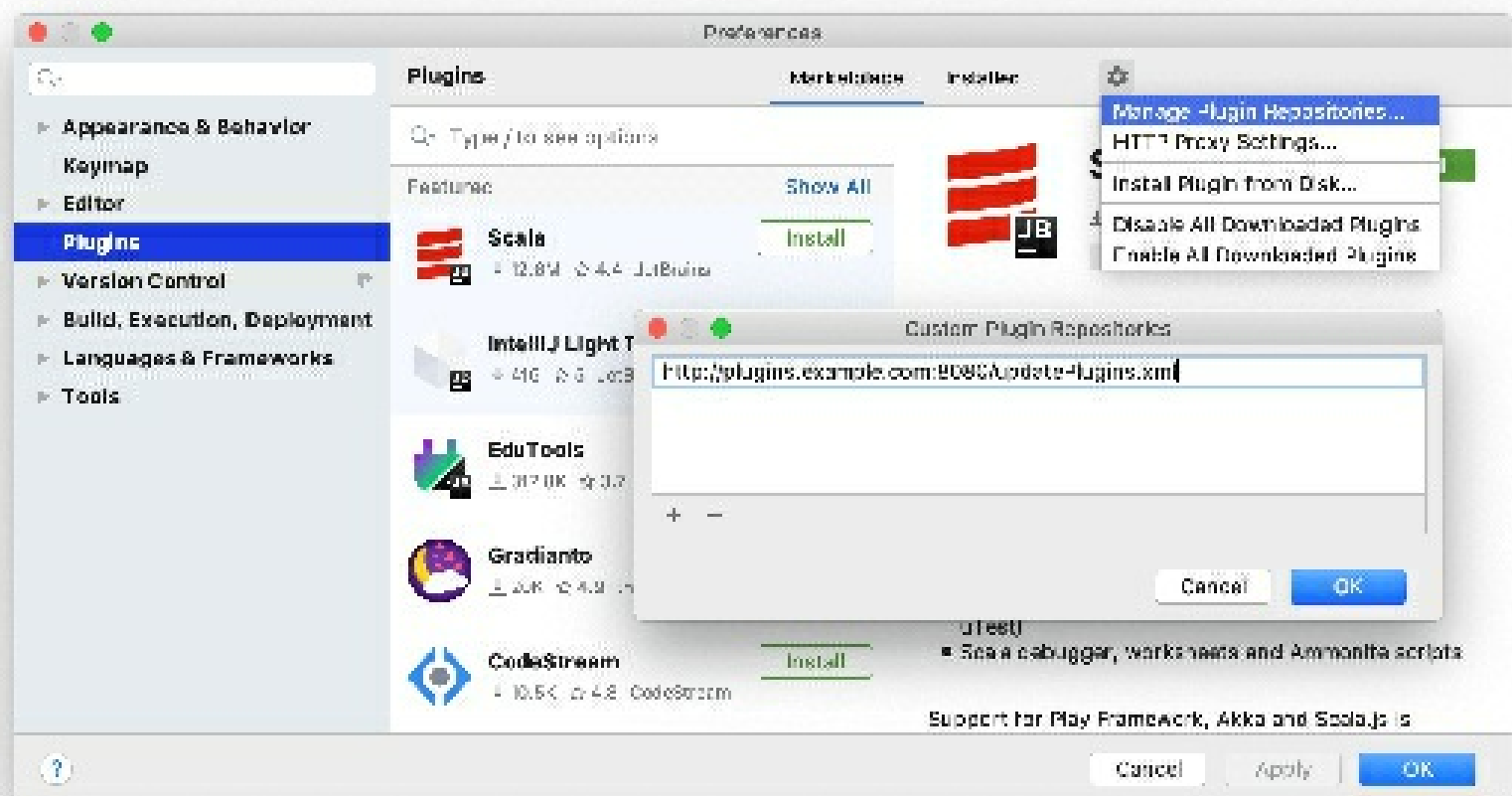
Custom plugin repositories

By default, IntelliJ IDEA is configured to use plugins from the JetBrains Plugin Repository. This is where all the community plugins are hosted, and you are free to host your plugins there. However, if you develop plugins for internal use only, you can set up a custom plugin repository for them.

For information about setting up a custom plugin repository, see the IntelliJ Platform SDK documentation.

Once you set up your plugin repository, add it to IntelliJ IDEA:

1. Press **Ctrl+Alt+S** to open IDE settings and select **Plugins**.
2. On the **Plugins** page, click  and then click **Manage Plugin Repositories**.
3. In the **Custom Plugin Repositories** dialog, click  and specify your repository URL. It must point to the location of the **updatePlugins.xml** file. The file can be on the same server as your custom plugins, or on a dedicated one.



4. Click **OK** in the **Custom Plugin Repositories** dialog to save the list of plugin repositories.
5. Click **OK** in the **Settings/Preferences** dialog to apply the changes.

To browse the custom plugin repository, type `repository:` followed by the URL of the repository in the **Marketplace** tab of the **Plugins** page. For example:

`repository:http://plugins.example.com:8080/updatePlugins.xml myPlugin`
Copied!

Alternatively, you can replace the default JetBrains Plugin Repository with your custom repository URL. This can be helpful if you want only your custom repository plugins to be available from IntelliJ IDEA. To do this, edit the platform properties or VM options file as described below. For more information, see [Advanced configuration](#).

Use a custom plugin repository

1. From the main menu, select **Help | Edit Custom Properties**.
2. Add the `idea.plugins.host` property to the platform properties file. For example:

```
idea.plugins.host="http://plugins.example.com:8080/"
```

Copied!

To add multiple URLs, separate them with semicolons ;.

Make sure that there is no `plugins.jetbrains.com` URL.

3. Restart IntelliJ IDEA.

If you replace the default plugin repository with a custom one, the search field on the **Marketplace** tab of the **Plugins** dialog will browse only the plugins in your custom repository.

Required plugins

A project may require plugins that provide support for certain technologies or frameworks. You can add such plugins to the list of *required plugins* for the current project, so that IntelliJ IDEA will verify that the plugins are installed and enabled. It will notify you if you forget about some plugin, or someone on your team is not aware about the dependency as they work on the project.

Add a required plugin for your current project

1. Make sure that the required plugin is installed.
2. Press `Ctrl+Alt+S` to open IDE settings and select **Build, Execution, Deployment | Required**

Plugins.

3. On the **Required Plugins** page, click  and select the plugin. Optionally, specify the minimum and maximum version of the plugin.

To specify the required version of IntelliJ IDEA itself, add **IDE Core** to the list of required plugins. After the required plugin is added, when you open the project in IntelliJ IDEA, it will notify you if the plugin is disabled, not installed, or requires an update.

Click the link in the notification message to quickly enable, install, or update the required plugin.

Develop your own plugins

You can use any edition of IntelliJ IDEA to develop plugins. It provides an open API, a dedicated SDK, module, and run/debug configurations to help you.

The recommended workflow is to use Gradle. The old workflow using the internal IntelliJ IDEA build system is also supported. For more information, see the IntelliJ Platform SDK Developer Guide.

Productivity tips

Filter and sort search results

- Type a forward slash / in the search string to see options for filtering and sorting search results. For example, you can add the following options to your search string to list only language-related plugins and sort them by the number of downloads:

`/tag:Languages /sortBy:downloads`

Work offline

A lot of features in IntelliJ IDEA require access to the internet. If you are working offline (for example, in an isolated environment), there are some aspects that you should keep in mind.

Updates

By default, IntelliJ IDEA is configured to check for updates automatically and notify you when a new version is available. Updates are usually *patch-based*: they are applied to the existing installation and only require you to restart the IDE. However, sometimes patch updates are not available, and a new version of IntelliJ IDEA must be installed.

If IntelliJ IDEA does not have HTTP access outside your local network, it will not be able to check for updates and apply patches. In this case, you have to download new versions of the IDE and install them manually as described in Standalone installation.

Without internet access, you can't install IntelliJ IDEA using the Toolbox App and snaps.

For more information, see [Update IntelliJ IDEA](#).

Plugins

Usually, plugins are installed from the JetBrains Plugin Repository. However, you can set up a custom plugin repository in your local network and configure IntelliJ IDEA to use it for installing and updating plugins.

Alternatively, you can download and manually install plugins from disk.

License activation

You can evaluate IntelliJ IDEA Ultimate for up to 30 days. After that, buy and register a license to continue using the product.

If IntelliJ IDEA does not have HTTP access outside your local network, you will not be able to use the JetBrains Account for signing in. However, you can generate an offline activation code that will be valid during your subscription term.

If your organization has at least 50 active subscriptions or licenses of JetBrains products, you can use the Floating License Server to activate IntelliJ IDEA instances within your company network. Keep in mind that the License Server itself requires internet access for connecting to the JetBrains Account.

For more information, see [Register IntelliJ IDEA](#).

Code inspections

Some code inspections verify external resources. For example, the **Non-existent web resource** inspection highlights dead links. If you don't have internet access, these inspections will not work and dead links will not be highlighted.

For more information, see [Code inspections](#).

External documentation

External documentation opens the necessary information in a web browser so that you can navigate to related symbols and keep the information for further reference at the same time. However, if you don't have an internet connection, online documentation is not accessible. In this case, you can download it and open it by means of the Quick Documentation popup.

Access SDK documentation offline

1. Download the documentation package of the necessary version.

The documentation package is normally distributed in a ZIP archive that you need to unpack once it is downloaded.

For example, you can download the official Java SE Development Kit 14.0.1 Documentation and unzip it.

2. In the **Project Structure** dialog `Ctrl+Alt+Shift+S`, select **SDKs**.
3. Select the necessary JDK version if you have several JDKs configured, and open

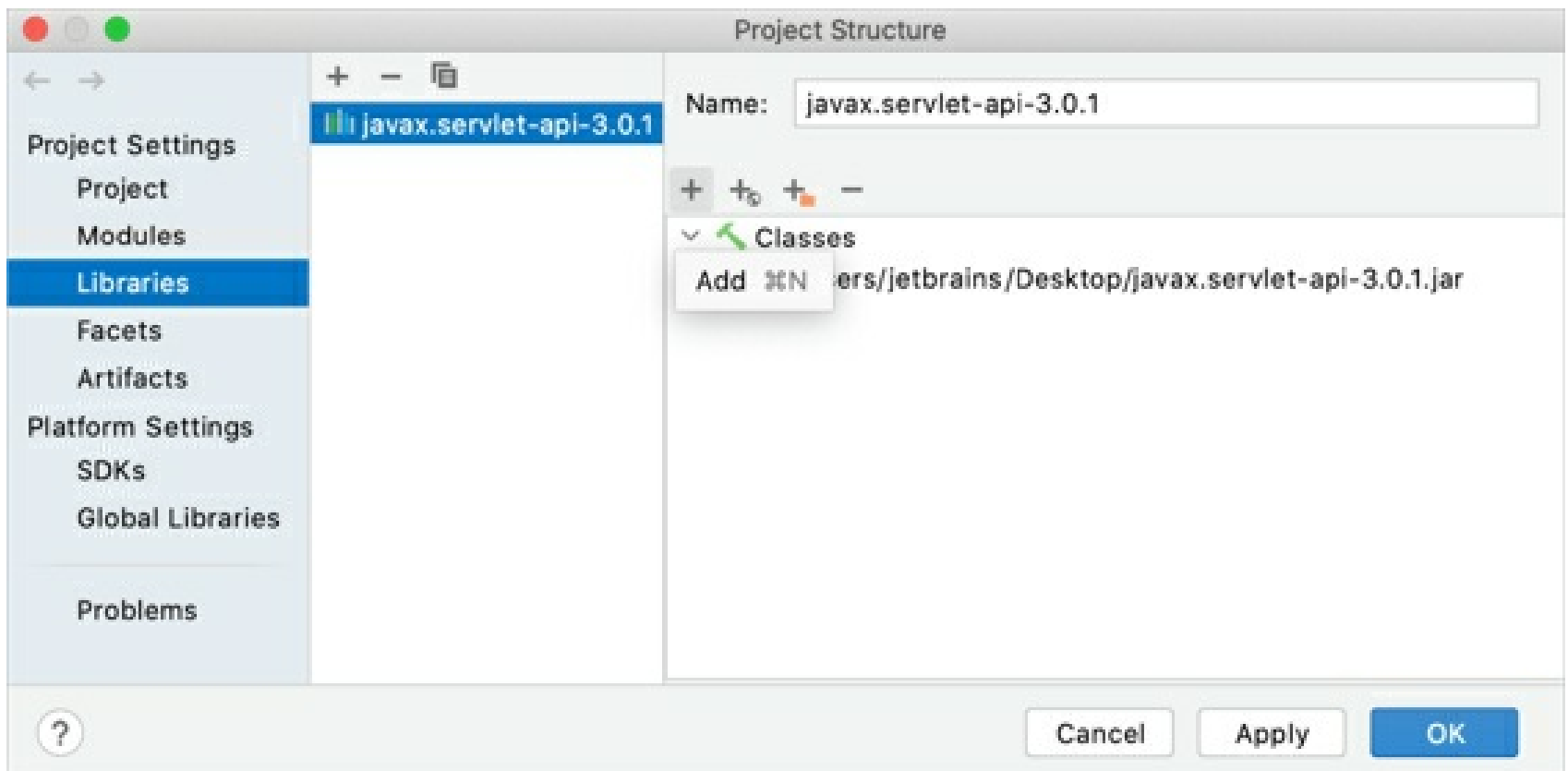
the **Documentation Path** tab on the right.

- Click the icon and specify the directory with the downloaded documentation package (for example, `C:\Users\jetbrains\Desktop\docs\api`).
- Apply the changes and close the dialog.

Access library documentation offline

You can add downloaded documentation to your project to be able to access it offline.

- From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Libraries**.
- Select the library for which you want to add the documentation and click **+** in the right section of the dialog.



- In the dialog that opens, select the file with the documentation and click **Open**.
- Apply the changes and close the dialog.

When the documentation is downloaded and configured, hover the mouse over the necessary symbol in the editor or place the caret at the symbol, and press `Ctrl+Q` (**View | Quick Documentation**).

For more information, see [Quick documentation](#).

Version control systems

Most likely, your source code is under some sort of version control system (VCS). If a remote repository is not in your local network, and there is no internet access, IntelliJ IDEA will not be able to communicate with the VCS. For example, if you are using Git, you will be able to commit your changes but won't be able to push them to the remote repository or pull updates from it.

For more information about VCS integration, see [Version control](#).

Tasks and issue trackers

You can set up a connection with an issue tracker to work with tasks and bugs assigned to you directly from IntelliJ IDEA. For example, you can connect to YouTrack, Jira, GitHub, and so on.

If the issue tracker server is not in your local network, and there is no internet access, IntelliJ IDEA will not be able to sync your issues. In this case, you will be able to work only with local tasks that you create yourself.

For more information, see [Tasks and contexts](#).

Maven dependencies

By default, Maven connects to remote repositories and checks for updates on every launch. Resolving Maven dependencies can require downloading new artifacts. You can switch to offline mode if you want Maven to use only those resources that are available locally.

Switch Maven to offline mode

- In the **Maven** tool window, click .

This will append the `--offline` option to all Maven commands that IntelliJ IDEA runs. It will also report any items that cannot be found in the local repository.

Gradle dependencies

By default, Gradle connects to remote repositories and checks for updates on every launch. Resolving Gradle dependencies can require downloading new artifacts. You can switch to offline mode if you want Gradle to use only those resources that are available locally.

Switch Gradle to offline mode

- In the **Gradle** tool window, click .

This will append the `--offline` option to all Gradle commands that IntelliJ IDEA runs. It will also report any items that cannot be found in the local repository.

Usage statistics

When you run IntelliJ IDEA for the first time, you are prompted whether to send anonymous data on the features and plugins you use, your hardware and software configuration, file types, number of files per project, and so on. This does not include any personal or sensitive data, such as parts of your source code or file names. This information is collected in accordance with the JetBrains Privacy Policy and is used to help improve the products and overall experience.

Even if you enable anonymous usage statistics, it will not be sent if there is no HTTP access outside your local network. Also, you can disable this feature entirely if you agreed at first and then changed your mind later.

Disable sending usage statistics

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | System Settings | Data Sharing**.
2. Clear the **Send usage statistics** checkbox.

Accessibility

IntelliJ IDEA lets you enable various accessibility features to accommodate your needs. You can use a screen reader or adjust font size, colors, and the behavior of certain UI elements to make the process of working with IntelliJ IDEA easier.

Set up a screen reader

Currently, IntelliJ IDEA fully supports screen readers for IntelliJ IDEA on Windows.

For *macOS*, switch on the VoiceOver and install and set up IntelliJ IDEA. However, for a full screen readers' support, we recommend Windows.

Enable a screen reader for Windows

1. Download and enable your preferred screen reader. Check the following recommended screen readers:
 - NVDA: use the NVDA 2015 or later. If you are using a 32-bit version of NVDA, you must install 32-bit JRE on your machine, as this version of NVDA requires **C:\Windows\SysWOW64\WindowsAccessBridge-32.DLL** to work with IntelliJ IDEA. If NVDA cannot locate this file, the NVDA Event Log window displays a message.
 - JAWS: download the version you need and restart your computer to enable the JAWS screen reader.
2. Ensure you have installed Java Access Bridge and the proper Java version for your screen reader, as follows:
 - To enable Java Access Bridge, open the command prompt and type **[JRE_HOME]\bin\jabswitch -enable**, where **[JRE_HOME]** is the directory of the JRE on your machine. For Java version 1.8, Java Access Bridge is part of JDK and you don't need to download it separately. Use control panel to enable the Java Access Bridge.
 - If your screen reader is 32-bit, install the *32-bit JRE version 1.7* or later. If your screen reader is 64-bit, install the *64-bit JRE version 1.7* or later.

Your computer may have multiple versions of some important components of Java Access Bridge and they may not be compatible across versions. You need to verify that your Java Access Bridge configuration is correct.

If your screen reader is 32-bit, ensure that **C:\Windows\SysWOW64\WindowsAccessBridge-32.DLL** is present and has a version number of 7.x.x.x or later. The file's description should read "Java(TM) Platform SE 7".

Install and set up IntelliJ IDEA

1. Download and install IntelliJ IDEA.

For Windows and macOS, when IntelliJ IDEA detects a screen reader on the first launch, it displays a dialog where you can enable a screen reader for IntelliJ IDEA.

2. To enable screen reader support before the initial launch of IntelliJ IDEA, do the following:
 - Open the **configuration** directory that contains personal settings, such as, keymaps, color schemes, and so on.

Syntax

%APPDATA%\JetBrains\<product><version>

Example

C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJdea2021.1

- Create a file called **idea.properties**.
 - Add the `ide.support.screenreaders.enabled=true` property to the file you have created.
3. Launch IntelliJ IDEA. The **Support screen readers** option located in **Settings / Preferences | Appearance & Behavior | Appearance** will be enabled.

Customize the IDE

You can customize the IDE depending on your accessibility needs.

Adjust colors for red-green color vision deficiency

You can adjust colors if you have red-green color vision deficiency. In this case, code fragments such as errors that are usually highlighted in red or strings that are usually green, will change the color (red will change to orange, green will change to blue). The color of the progress bar in the test runner will also get adjusted so it can be easily recognized.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | Appearance**.
2. From the options on the right, select the **Adjust colors for red-green vision deficiency (protanopia, deuteranopia)** option and click **OK** to save your changes.

Check the following example with the **Before** image that has *String* highlighted in green color and *Errors* highlighted in red and the **After** image where the colors are adjusted:

Before

After

```
public class apples {  
    public static void main(String[] args){  
        variety varietyObject = new varietyBuilder().setName("String").createVariety();  
        varietyObject.saying();  
    }  
    Errors  
}
```

Add the contrast color for scrollbars

You can make scrollbars in the editor more visible.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | Appearance**.
2. From the options on the right, under **Accessibility** section, select **Use contrast scrollbars**.

To configure the editor vertical scrollbar color and opacity, use the color scheme settings.

Configure colors for code elements, editor, scrollbar, hyperlinks, and so on

You can adjust colors for code elements, errors, elements of the editor, and tool windows. You can also configure a color for the vertical scrollbar in the editor.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme | General**.
2. From the options list on the right, select an element for which you want to adjust the color. For example, you can select **Code** and adjust colors for injected language fragments or matched braces, and so on. Click **OK** to save the changes.

You can also adjust colors for the debugger, consoles and other parts of the IDE: select the appropriate node in the list of options located in **Settings/Preferences | Editor | Color Scheme**.

Override the default UI fonts

You can override the default fonts of the UI elements.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | Appearance**.
2. From the options on the right, select **Use custom font**. From the list of fonts, select the one you need and in the **Size** field, specify the font's size.

Click **OK** to save the changes.

Resize tool windows

You can resize the actual tool windows vertically or horizontally using shortcuts.

1. To resize vertically up or down, press `Ctrl+Shift+Up` OR `Ctrl+Shift+Down`.
2. To resize horizontally left or right, press `Ctrl+Shift+Left` OR `Ctrl+Shift+Right`.

Adjust text size in the editor

You can change font and a size of the text in the editor.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | General**.
2. From the options on the right, select the **Change font size (Zoom) with Ctrl+Mouse Wheel** option to quickly change the text size (turning the mouse wheel) while you are working in the editor.
3. If you need to specify the exact font size, select **Editor | Font**.
4. From the options on the right, specify the font, its size, line spacing, and other available options. Click **OK** to save changes.

Customize shortcuts

You can configure custom shortcuts to actions that you frequently use.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Keymap**.
2. From the list of options on the right such as menus, actions, and tools, select the action you need.
3. Right-click the selected item and from the context menu, select an action you want to perform such as **Add keyboard shortcut**, **Add mouse shortcut**, or **Add abbreviation**.
4. In the dialog that opens, specify a shortcut. If you need, select the **Second stroke** option and specify an additional key for the shortcut. Click **OK** to save the changes.

Click **OK** with the mouse. If you press `Enter` IntelliJ IDEA will consider it a shortcut.

You can ignore a conflict and assign a shortcut to several actions. However, it is strongly recommended that you avoid binding two actions with the same shortcut, because the priority of these actions is not defined.

Customize smart keys behavior

You can configure the behavior of smart keys.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | General | Smart keys**.
2. From the options on the right, select or clear the smart keys options, for example, you can clear the **Insert paired brackets** or the **Insert paired quotes** option that automatically inserts a closing bracket or a quote since it might not be useful when you use a screen reader. Click **OK** to save changes.

Disable automatic code completion

You can disable automatic code completion to avoid auto-inserting code elements when you work in the editor with a screen reader.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | General | Code completion**.
2. Clear the **Type-Matching Completion** option. If you need, clear **Basic Completion** to disable basic completion as well.

Customize code folding

You can control the code folding behavior and specify what should or should not be folded.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | General | Code folding**.
2. From the options on the right, select what should be collapsed by default.

Customize code style

You can configure spaces, tabs and indents.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | Code Style | [Language]**.
2. From the options on the right, click **Tabs and Indents** to configure tabs or **Spaces** to configure where and how to use spaces.
3. Click **OK** to save the changes.

Read gutter icons and line numbers in the editor

You can configure a screen reader to read line numbers, VCS annotations, debugger, and other icons that are located in the left gutter of your editor.

Make sure that the **Support screen readers (requires restart)** option is selected in **Settings/Preferences | Appearance Behavior | Appearance**.

1. Open your file in the editor.
2. Press `Alt + Shift + 6` and `F` simultaneously to focus on the gutter. IntelliJ IDEA starts reading from the line where your caret is currently located.
3. Use the *Up* and *Down* arrow keys to move between lines. If you need to move to the next or previous gutter element in the line, use the *Right* and *Left* arrow keys respectively.

While the focus is in the gutter, the screen reader can read the gutter icon tooltip if it is available.

To access a tooltip, press the double shortcut `Alt+Shift+6, T`. To browse through the tooltip's content (symbol by symbol), use the *Right* and *Left* arrow keys.

4. Press `Escape` to switch the focus back to the editor.

Set a high contrast color theme

You can set a high contrast interface theme to work in IntelliJ IDEA. The interface theme defines the appearance of windows, dialogs, and controls.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Appearance & Behavior | Appearance**.
2. In the *UI Options* area, from the **Theme** list, select **High Contrast**, and click **OK** to apply changes.

Set a high contrast color scheme

You can set a high contrast color scheme for your editor. IntelliJ IDEA uses color schemes to help you define the preferred colors and fonts in the editor.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | Color Scheme**.
2. On the **Color Scheme** page, from the **Scheme** list, select **High Contrast**.
3. Click **OK** to apply your changes.

You can check Editor basics, IntelliJ IDEA keyboard shortcuts, and Overview of the user interface to familiarize yourself with other useful shortcuts.

Support and assistance

The most important source of information about IntelliJ IDEA is this online help. To access it from IntelliJ IDEA, do one of the following:

- From the main menu, select **Help | Help**.
- Press F1 on the keyboard.
- Click in a dialog or a tool window.

If you do not have internet access to view the online help, you can use the IntelliJ IDEA Help plugin, which serves the help pages via the built-in web server for offline use.

The offline help plugin is updated when a new major version is released. Changes that are added to online help during the release cycle may not be available in the offline help.

Contact support

If you can't find the information you need in the online help, you can contact the JetBrains support team.

- Browse the IntelliJ IDEA Knowledge Base.
- Create a topic in the community forum.
- From the main menu, select **Help | Contact Support** to create a direct request for the support team.

Report bugs

If you encounter a bug or would like to suggest a new feature, use the IntelliJ IDEA issue tracker.

- From the main menu, select **Help | Submit a Bug Report**.

Before submitting, it is a good idea to search the issue tracker for similar reports and feature requests to avoid duplicates. If you find a similar ticket, you can add a comment to it or vote for it to bring more attention of the development team.

You can copy a message in the status bar to the clipboard and paste it when you search for solutions to some problems or add it to your issue in the IntelliJ IDEA issue tracker. Right-click the status bar and select **Copy**.

If you need to attach the IntelliJ IDEA log file, locate it using the relevant **Help** menu item:

- **Help | Show Log in Explorer** (for Windows and Linux)
- **Help | Show Log in Finder** (for macOS)

Share feedback

You can use the feedback form to tell us what you like or don't like about JetBrains products.

- From the main menu, select **Help | Submit Feedback**.

Learn more

There are several ways to learn more about IntelliJ IDEA:

- *Tips of the day* show up every time you start IntelliJ IDEA and provide useful hints. To suppress them, you can clear the **Show Tips on Startup** checkbox.

To open the **Tip of the Day** dialog, select **Help | Tip of the Day** from the main menu.

- The **Productivity Guide** displays a list of useful features with statistics and tips.

To open it, select **Help | Productivity Guide** from the main menu.

To discover new features, sort the list by group and note rarely used features in the same group as the ones you use more frequently. Click the feature to see its usage description.

- The **IDE Features Trainer** is a tool for learning the most common and useful actions. It is a sequence of lessons that interactively guide new users through the shortcuts with real-world examples.

To use it, select **Help | IDE Features Trainer** from the main menu.

IntelliJ IDEA pro tips

This guide targets IntelliJ IDEA users who are already familiar with its basic features and would like to learn more. If you're relatively new to IntelliJ IDEA, we recommend that you read the [Discover IntelliJ IDEA guide](#) before delving into this one.

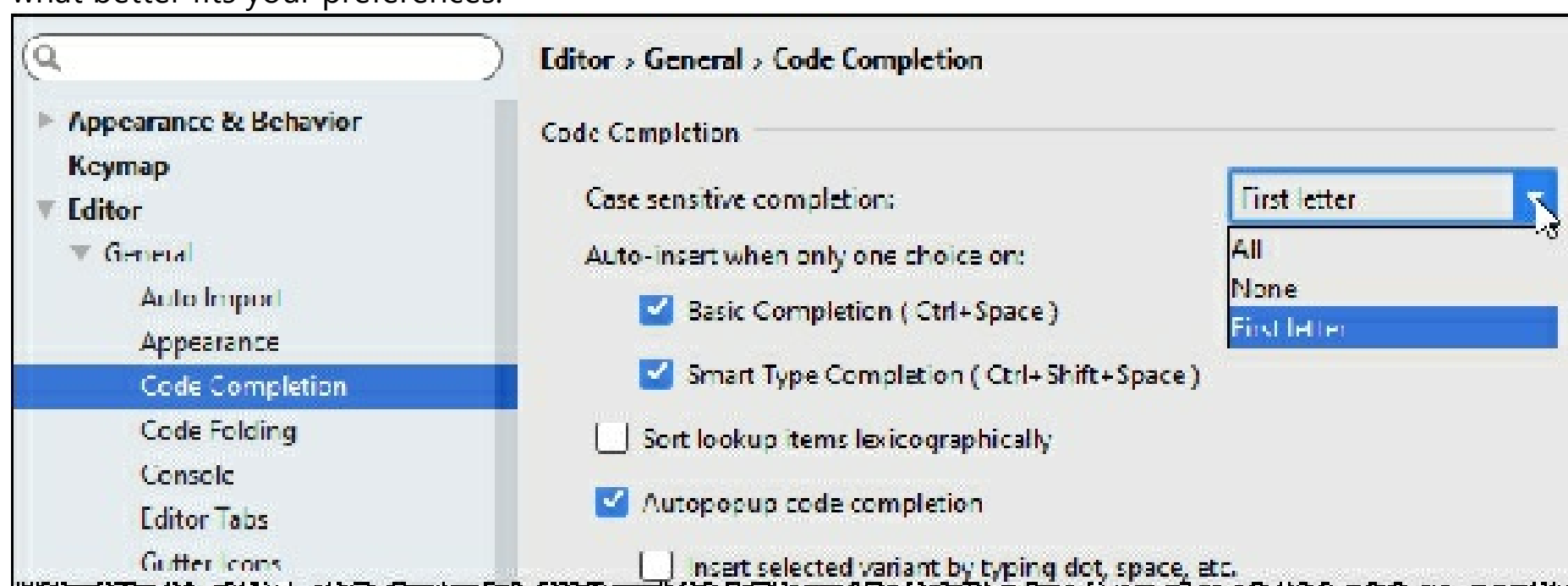
Coding assistance

Type info

If you want more information about a symbol at caret, for example, where it comes from or what its type is, the [Quick Documentation](#) is your friend. Press `Ctrl+Q` to invoke it and you will see a popup with these details. If you don't need the full info, then use the Type Info action instead: it only shows the type of the selected expression, but doesn't take up that much of screen space.

Code completion case sensitivity

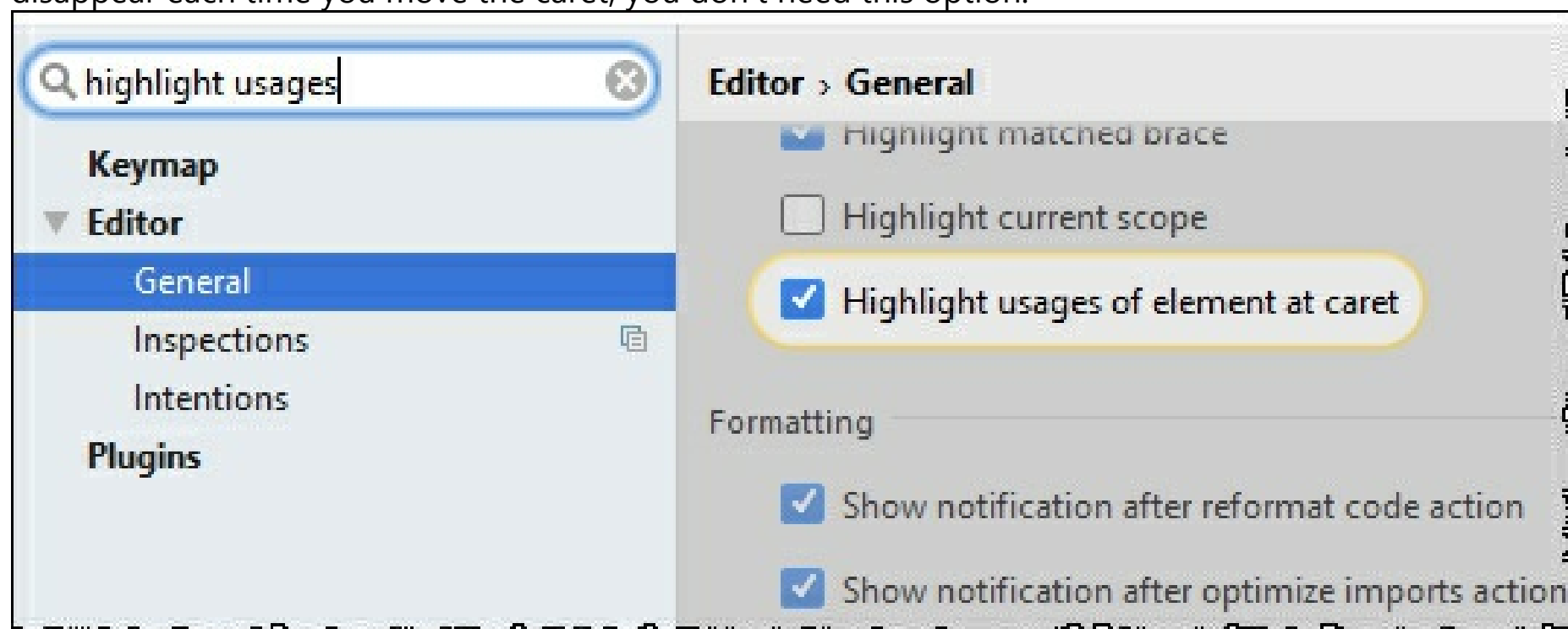
By default IntelliJ IDEA code completion case sensitivity only affects the first letter you type. This strategy can be changed in the **Settings/Preferences** dialog `Ctrl+Alt+S`, **Editor | General | Code Completion**, so you can make to either make the IDE sensitive to all letters or make it insensitive to the case at all, based on what better fits your preferences.



Hot tip: Here you can also turn off the **Autopopup code completion** option. This makes sense if you want the code completion popup to show up only when you explicitly call it.

Disable highlighting usages of element at caret

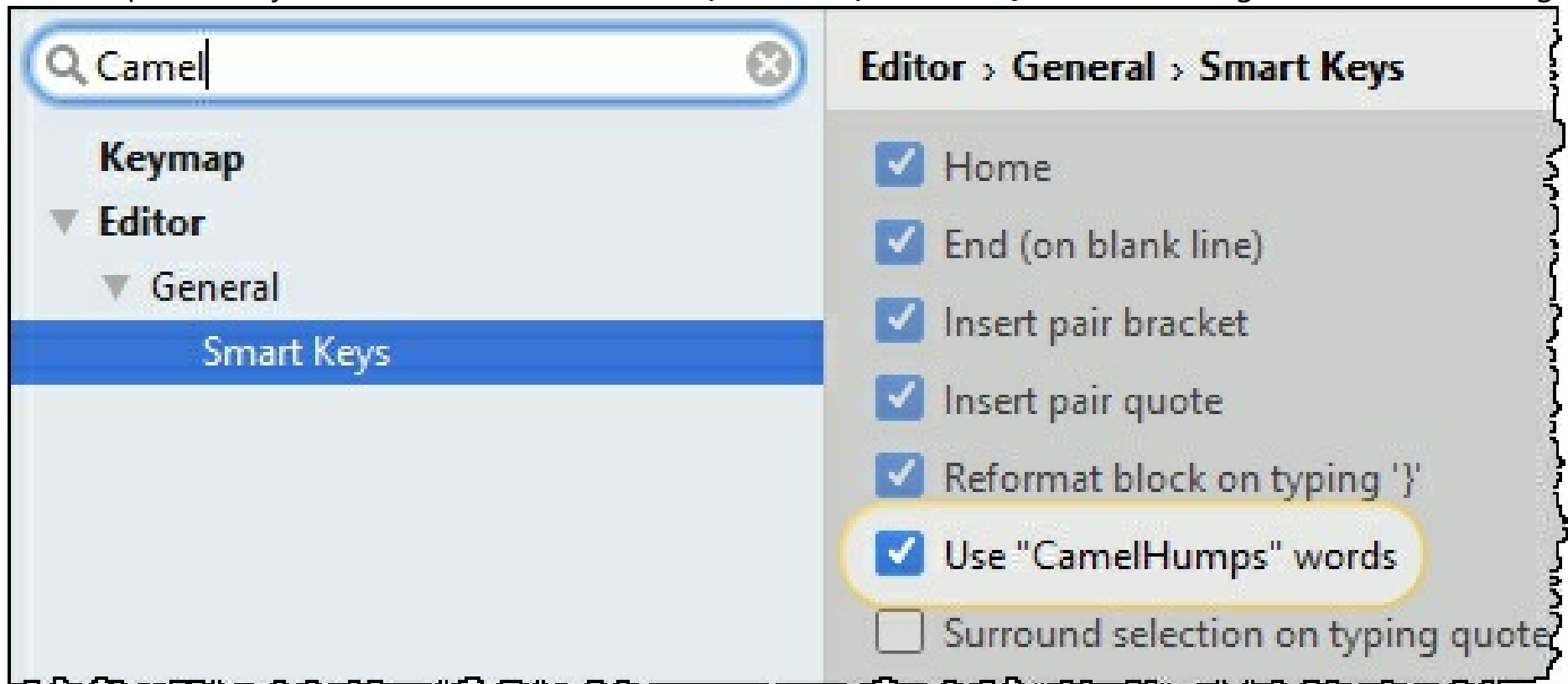
Talking about the defaults you may want to change after learning IntelliJ IDEA better, we can't miss the **Highlight usages of element at caret** option in the **Editor | General** page of the Settings/Preferences dialog. If you know the `Ctrl+Shift+F7` shortcut and don't like the highlighting in the editor to appear and disappear each time you move the caret, you don't need this option.



CamelHumps

By default, when you select anything in the editor, IntelliJ IDEA isn't sensitive to the case of the words. If

you prefer to select words according to CamelCase, for example, instead of selecting the whole word, select a part of it, you can enable this in **Editor | General | Smart Keys** of the Settings/Preferences dialog.



Hippie completion

IntelliJ IDEA provides Basic completion via `Ctrl+Space`, Smart type-matching completion via `Ctrl+Shift+Space`, and Statement completion via `Ctrl+Shift+Enter`. All these features are based on the actual understanding of the code structure. However, sometimes you may need a more trivial, yet flexible logic that would suggest the words used earlier in the current file or even project regardless to their context. This feature is called Hippie completion and is available via `Alt+.`

```
reportArray(((PatternSet) robj).getExcludePatterns(getProject()));
System.out.println();
System.out.println("Include patterns: ");
reportArray(((PatternSet) robj).getIncludePatterns(getProject()));
getExcludePatterns|

static String[] convertMetersToInches(String metersToConvertString) {
    String[] conversionResult = new String[2];
    Double conversionFactor = 39.37;
    String a = new String( original: "");
    try {
        double metersToConvertDouble = Double.parseDouble(metersToConvertString);
        dou
        double
        v metersToConvertDouble double
        Press ^ to choose the selected (or first) suggestion and insert a dot afterwards Next Tip
```

Refactorings

Undo refactorings

With IntelliJ IDEA you don't need to worry about consequences when refactoring code, because you can always undo anything by invoking Undo via the convenient `Ctrl+Z` shortcut.

Extract string fragments

IntelliJ IDEA is capable of refactoring not only executable code, but also string literals. Select any fragment of a string, call *Extract* variable/constant/field/parameter to extract it as a constant and replace its usages throughout the code.

Type migration

When you refactor, you usually rename symbols, or extract and move statements in the code. However, there's more to refactoring than just that. For example, Type migration (available via `Ctrl+Shift+F6`) lets you change the type for a variable, field, parameter or a method's return value (`int` → `String`, `int` → `Long`, etc), update the dependent code, and resolve possible conflicts.

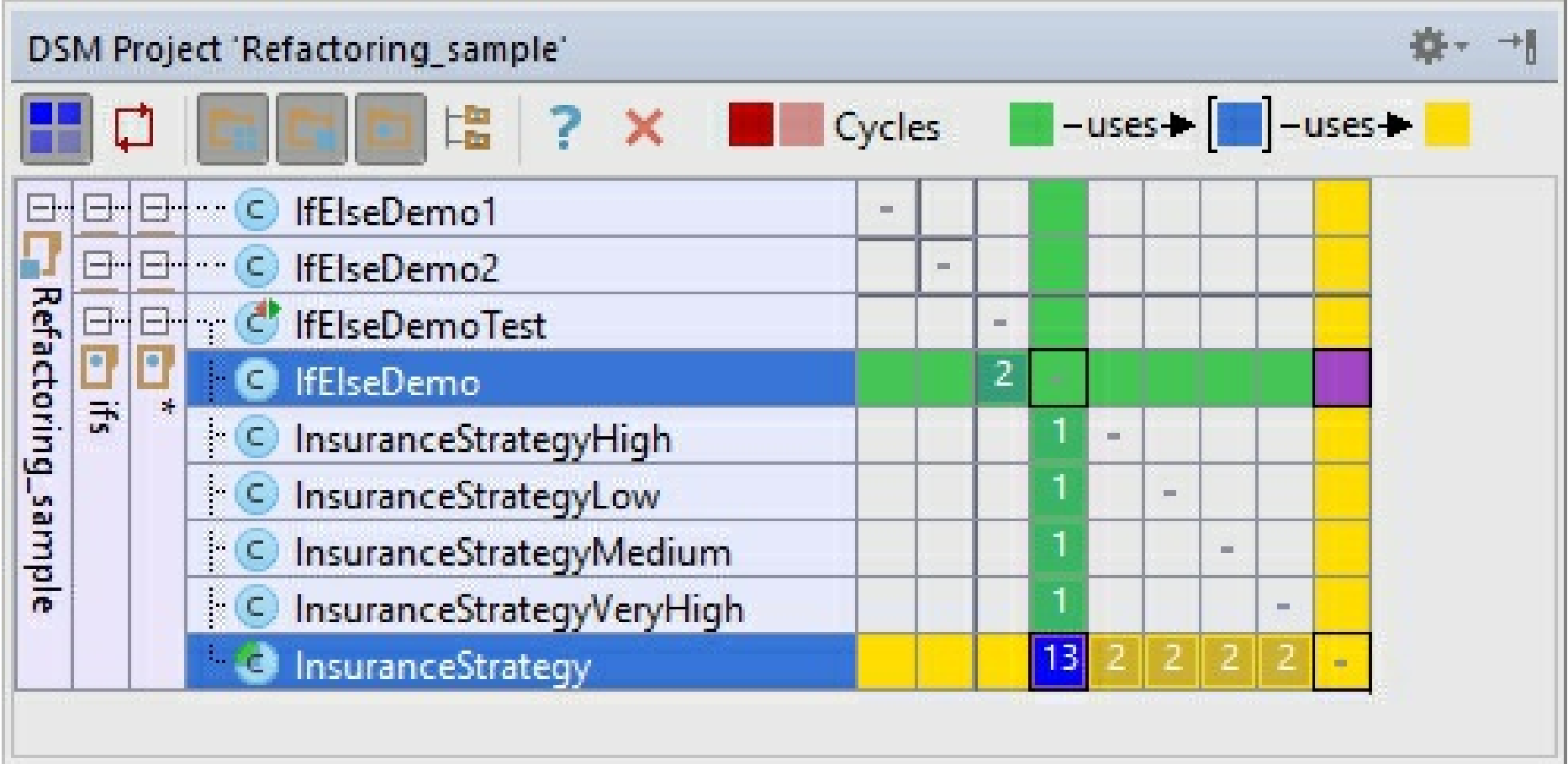
Invert boolean

If IntelliJ IDEA can automate type migration, why not do the same with semantics? To invert all usages of a boolean symbol, just use the Invert Boolean refactoring.

Code analysis

Dependency structure matrix

IntelliJ IDEA lets you analyze how tightly components in your code depend on each other, and you need to keep an eye on that because when there's too much dependencies, it's likely to cause various problems. Dependency Structure Matrix action (available via the **Analyze** menu) will help you visualize and explore dependencies between modules, packages and classes.

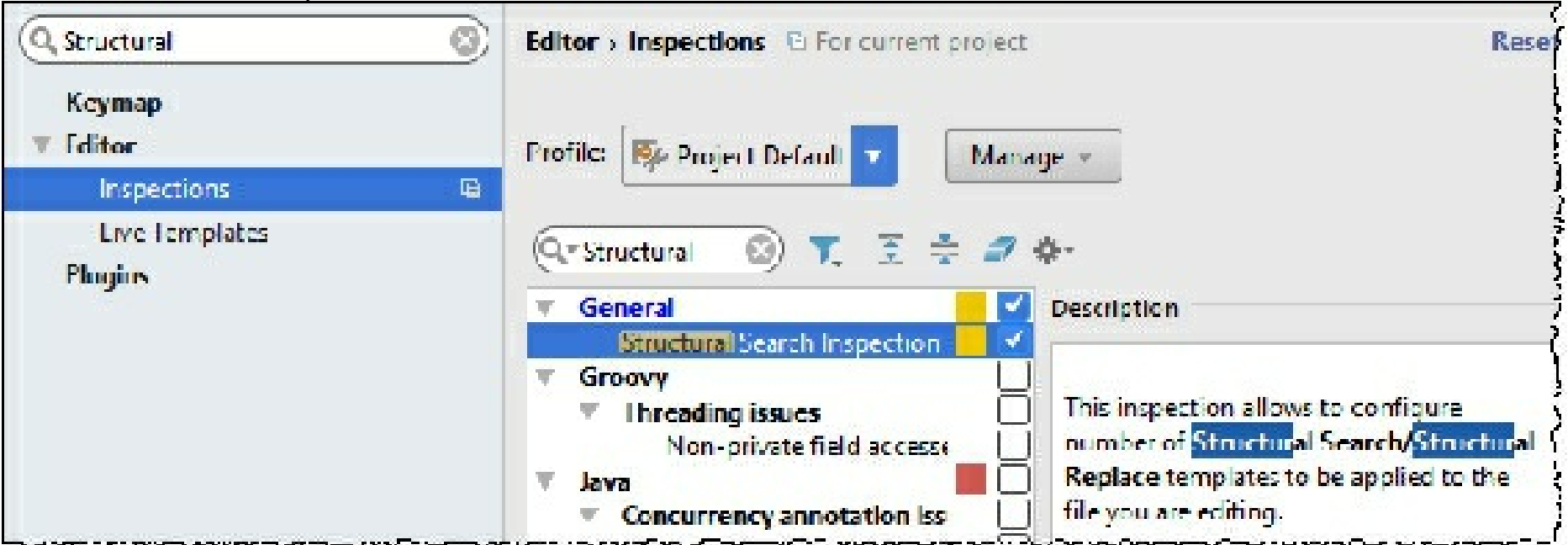


Despite its complex looks, it's a very easy-to-use tool. Just select a class or package and see where it's used and what it uses.

Structural search and replace

Structural Search and Replace, or SSR, is quite powerful (after you learn to use it right), and can be used for static code analysis and refactoring automation. In a nutshell, it lets you search for specific patterns in your code and replace them with parametrized templates. For that, it's equipped with its own language for defining code patterns that is described in more detail in this article.

To access this feature, use the **Edit | Find | Search/Replace Structurally...** If you want to create your templates or patterns, go to **Settings/Preferences** dialog, click the page Editor | Inspections, and enable Structural Search Inspection under the General node:



User interface

Disable breadcrumbs and tag tree highlighting

If you work with lots of HTML and XML and would like to avoid unnecessary distraction, you may want to disable breadcrumbs and tag tree highlighting in Editor | General | Appearance.

Disable unnecessary gutter icons

Gutter, the leftmost editor column, typically displays useful information related to the code you're

editing. If you feel that sometimes it's just too much, you can configure what you want to see in the **Settings/Preferences** dialog `Ctrl+Alt+S`: [Editor](#) | [General](#) | [Gutter Icons](#).

Disable intention light bulb

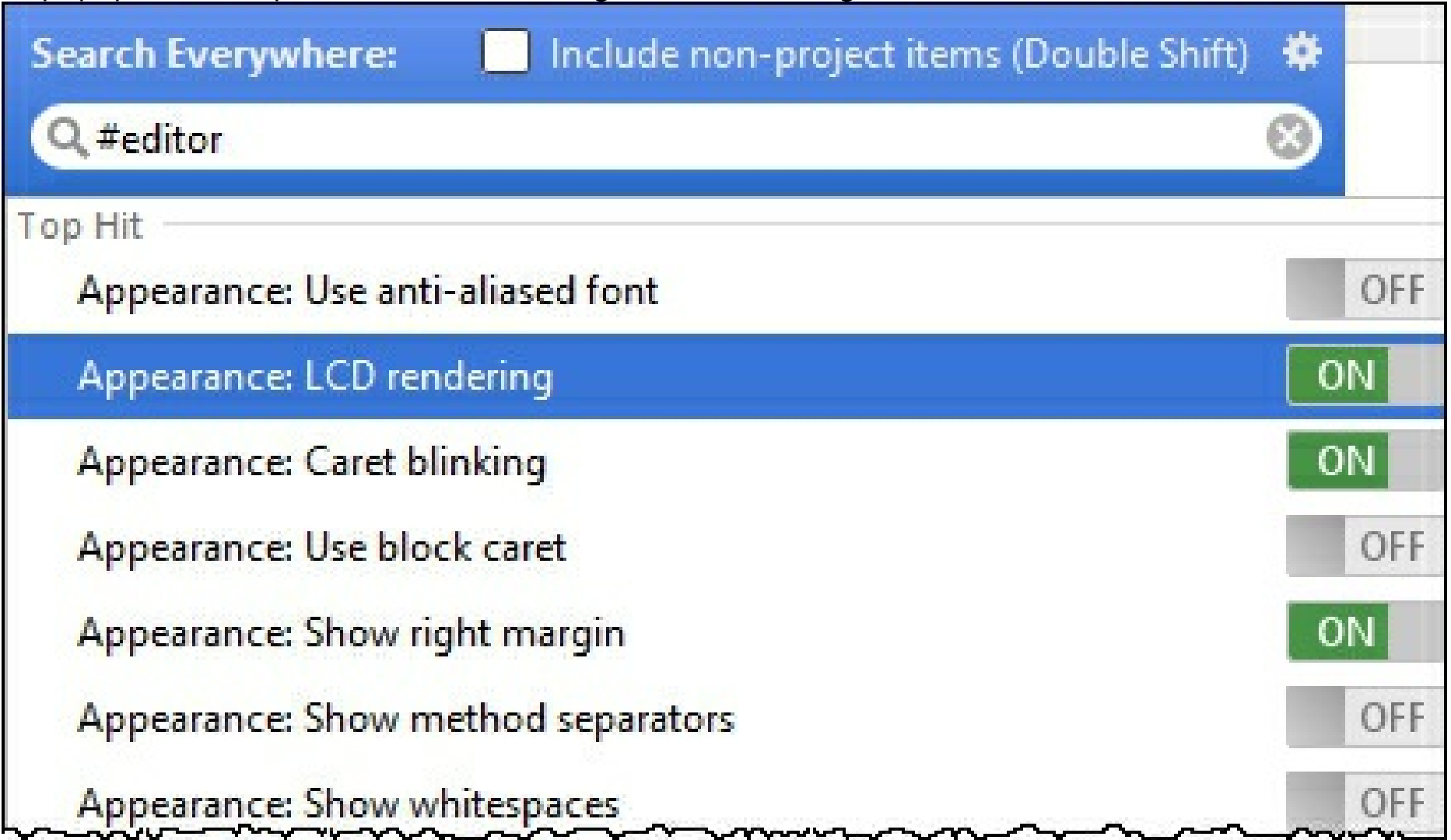
One more thing that might be annoying is the intention bulb that appears in the editor every time there is an intention available at the caret. Disabling that is a little bit more difficult: you need to manually edit your **<IntelliJ IDEA preferences folder>/options/editor.xml**, and add the following line:

```
<option name="SHOW_INTENTION_BULB" value="false" />
```

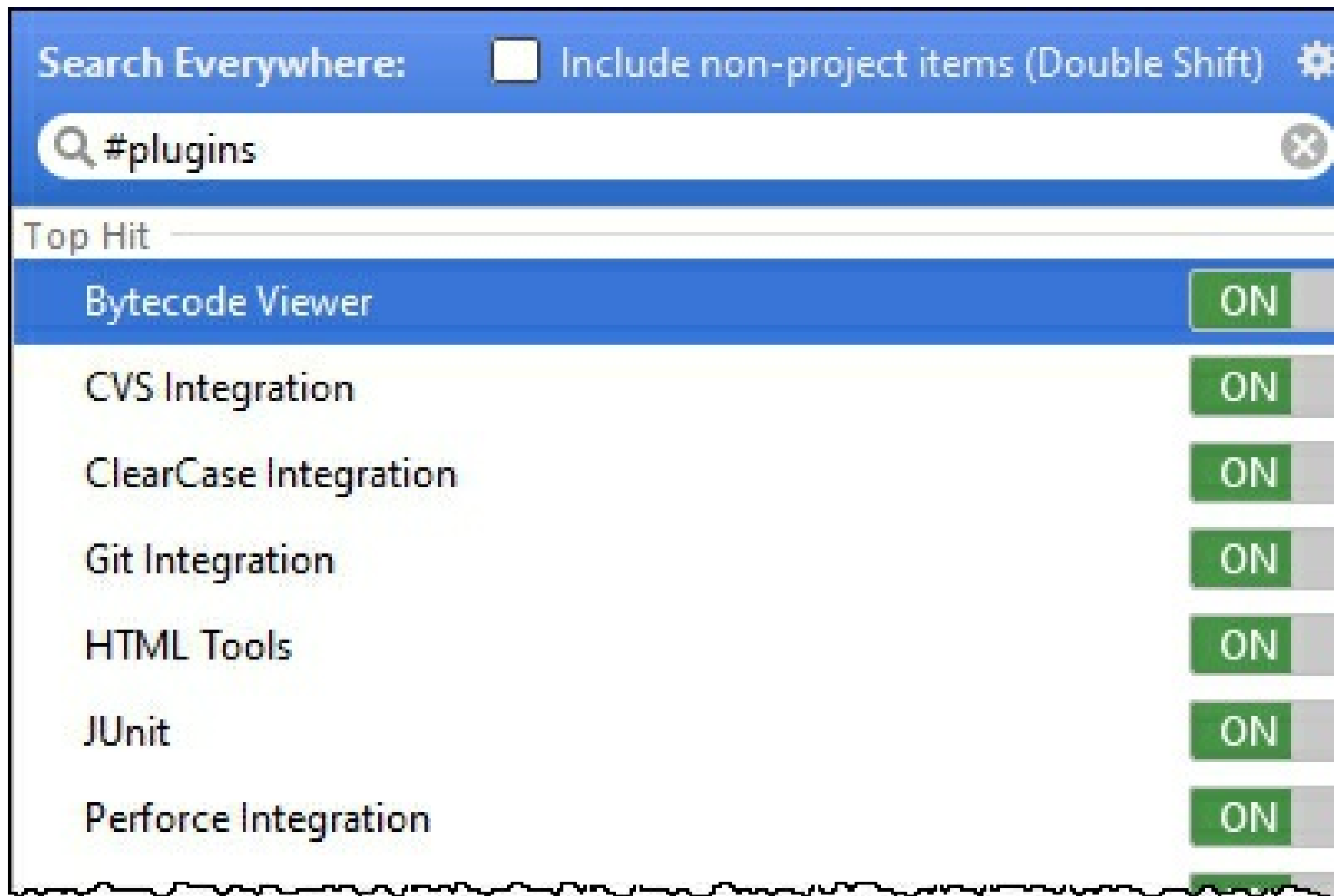
Copied!

Search everywhere

With Search Everywhere you can find arbitrary text fragments literally everywhere: in the code, libraries, parts of the UI, settings (by prepending the settings name with #), or even action names. If you're using this feature a lot, it's worth knowing that you can access IntelliJ IDEA settings by just pressing `Enter` right in its popup. For example, here we're accessing the editor settings:

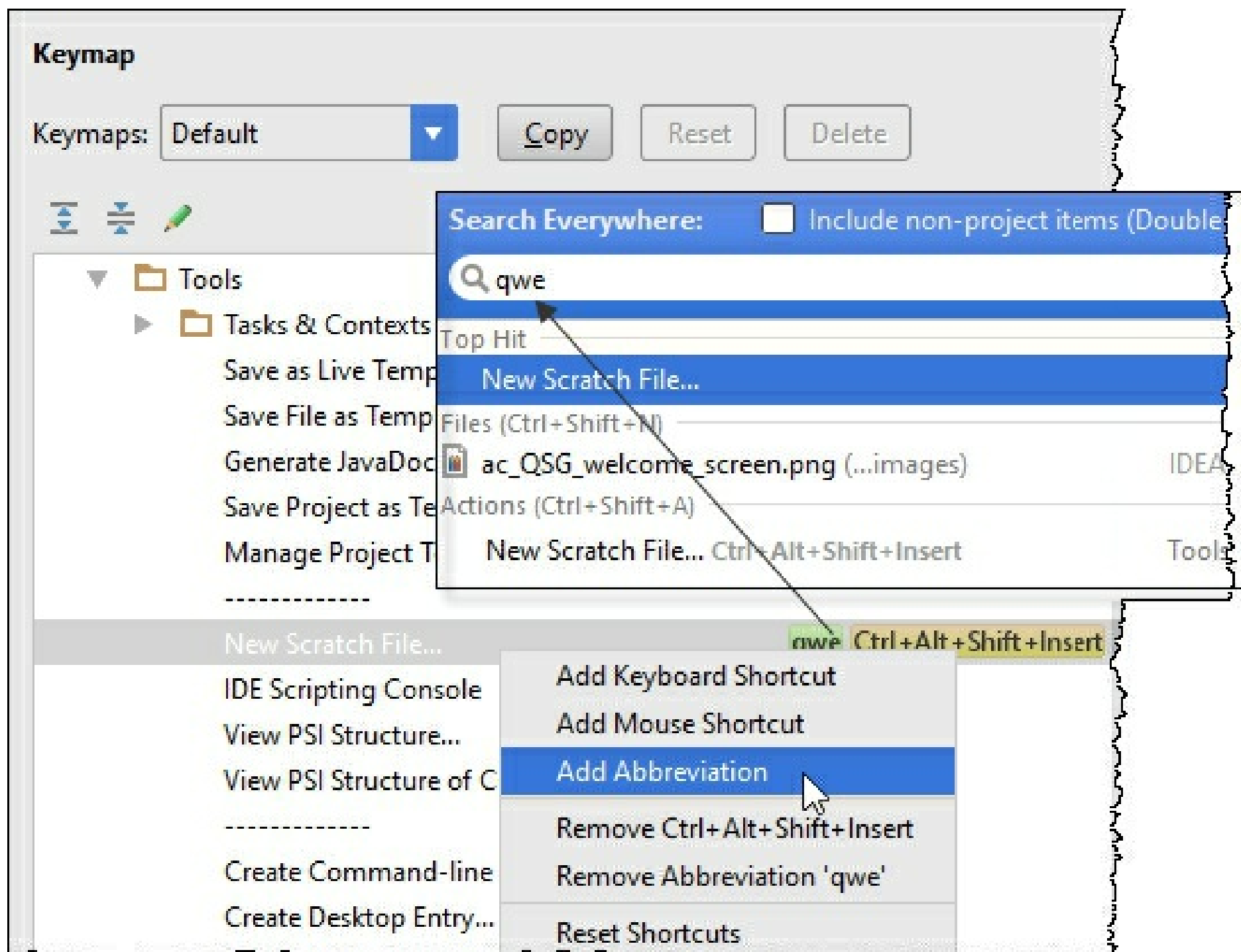


If you start your search query with #plugins, you'll be able to turn them on and off:



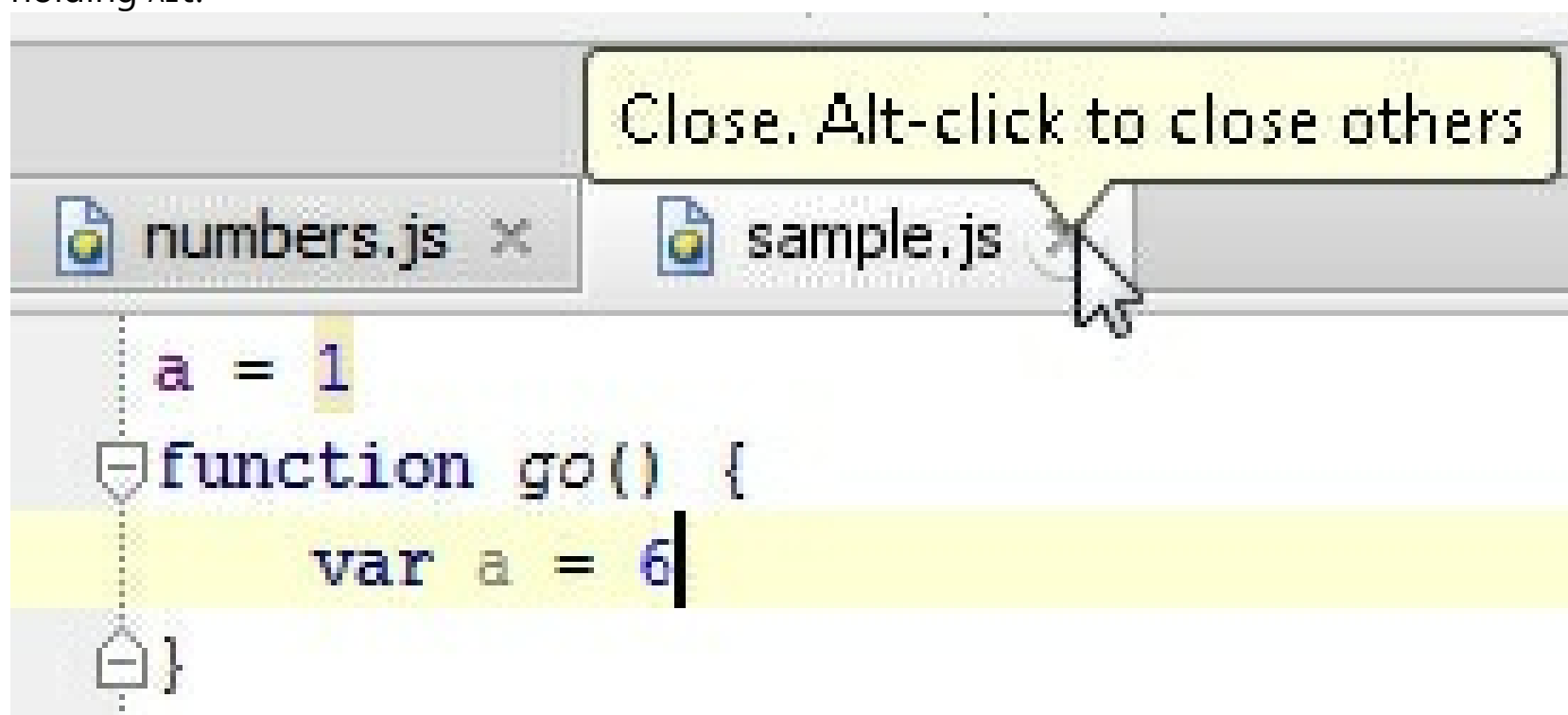
Other tags include #appearance, #system, #inspections, #registry, #intentions, #templates, and #vcs.

Another interesting fact is that Search Everywhere supports abbreviations. You can use [Keymap page](#) of the Settings/Preferences dialog to assign a short text to any action, and then have this action called from Search Everywhere by entering this text:



Hide editor tabs

When you need to close all editor tabs except the current one, click the close icon on the current tab holding **Alt**:



If you don't want to see the editor tabs at all, go to the [Editor Tabs](#) page of the Editor Settings/Preferences and under the **Placement** drop-down select **None**.

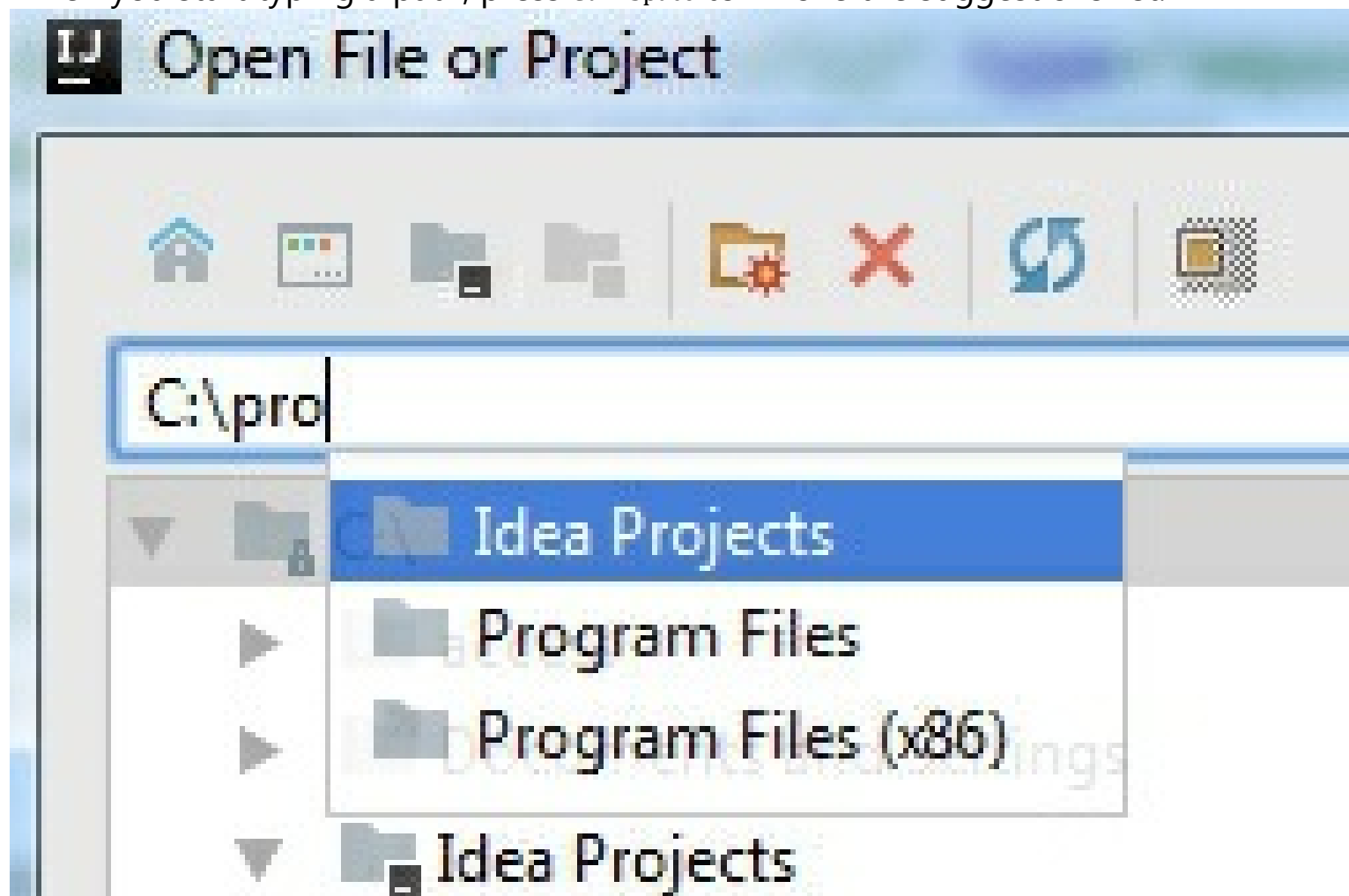
Open file in new window

A feature that is not that easy to find, yet comes in handy, is opening a file in a new window by selecting it in the [Project tool window](#) and clicking **Shift+Enter**.

Use path completion

Path completion helps you speed up the selection of files, folders, and so on. This is useful when adding a new SDK in the **Project Structure** dialog, or specifying an application server home directory.

When you start typing a path, press `Ctrl+Space` to invoke the suggestions list:



Add stop and resume buttons to the toolbar

It might be convenient to add Stop and Resume buttons to the toolbar of the Navigation Bar. You can do it via the [Appearance and Behavior | Menus and Toolbars](#) page of the Settings/Preferences dialog. If you prefer to use the mouse rather than keyboard shortcuts, this way you won't need to open the Debug tool window to manage your current debugging session.



Editor

Compare with clipboard

IntelliJ IDEA has a built-in Diff viewers for code, jar files, revisions and even images. To invoke it, select any pair of files and press `Ctrl+D`.

If you have selected a single file, the IDE will prompt you to select the one to compare to. To quickly compare active editor with Clipboard, choose **View | Compare with Clipboard**.

Paste from history

Speaking of Clipboard, IntelliJ IDEA keeps track of everything you put there. Anytime you want to paste one of the previously copied items, press `Ctrl+Shift+V`.

Multiple selections

Multiple selection is a relatively new, very powerful editor feature, which lets you quickly select and edit multiple (adjacent or not) pieces of code at once.

In a nutshell, here's what happens. You either start with pressing `Alt+J` (and then IntelliJ IDEA selects a symbol at caret), or you can just select something as you normally would.

Then, press `Alt+J` and IntelliJ IDEA will search the current file forward until it finds a matching piece of text, which it adds to the selection. You can press `Alt+J` again to go forward, or `Alt+Shift+J` to go back, but note that when search reaches the end of file, it will start over from the beginning of the file.


```

<procedure title="To select multiple words" alternative-title="Using m
    <step...>
</procedure>
<anchor name="column selection"/>
<procedure title="To toggle between the line and the column selection
<anchor name="altdrag"/>
<procedure title="To make selection in the Column Selection Mode"

```

After selection is complete, you can start editing all the fragments as if they were one.

Hot tip: One more way to clone caret is to press `Ctrl` twice, and then move the caret up or down with arrows or with the mouse.

Emmet

In case you didn't know, Emmet is a great way to write HTML, XML and CSS code. IntelliJ IDEA supports it out of the box: write an Emmet expression and press `Tab` to expand it.

Use the Emmet preview action (which is available via Find Action or Search Everywhere — so make sure to assign it to a handy shortcut) to see a preview of the resulting code.

htmlbody

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    table#users>tr.user>td
</body>
</html>

```

```

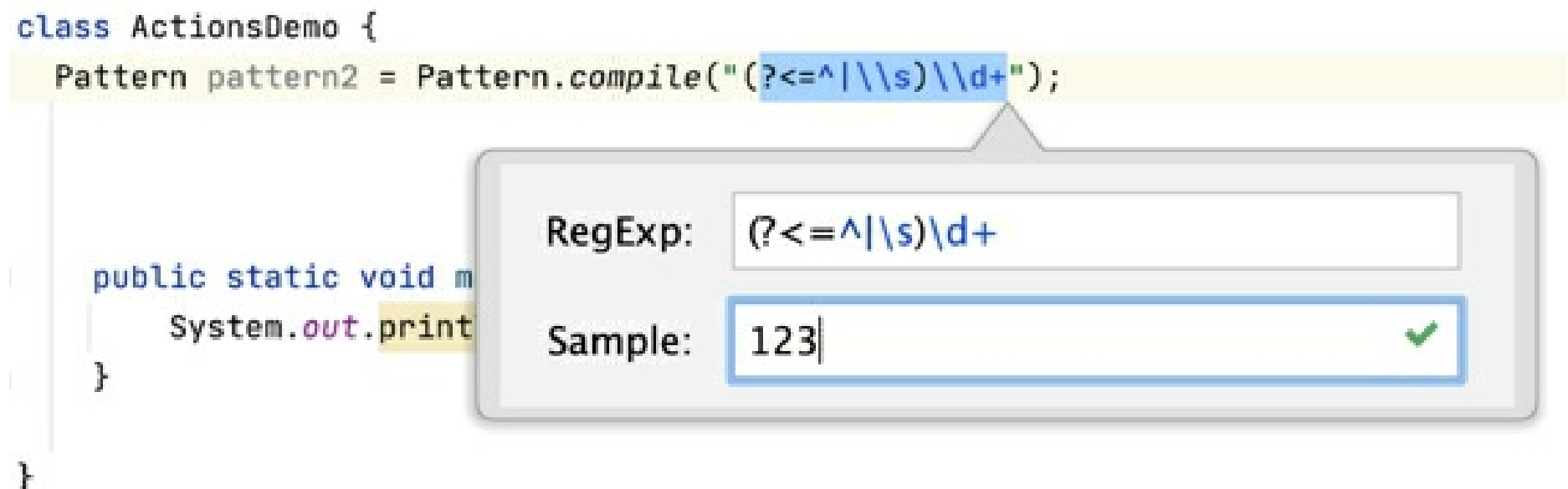
<table id="users">
    <tr class="user">
        <td></td>
    </tr>
</table>

```

Regex

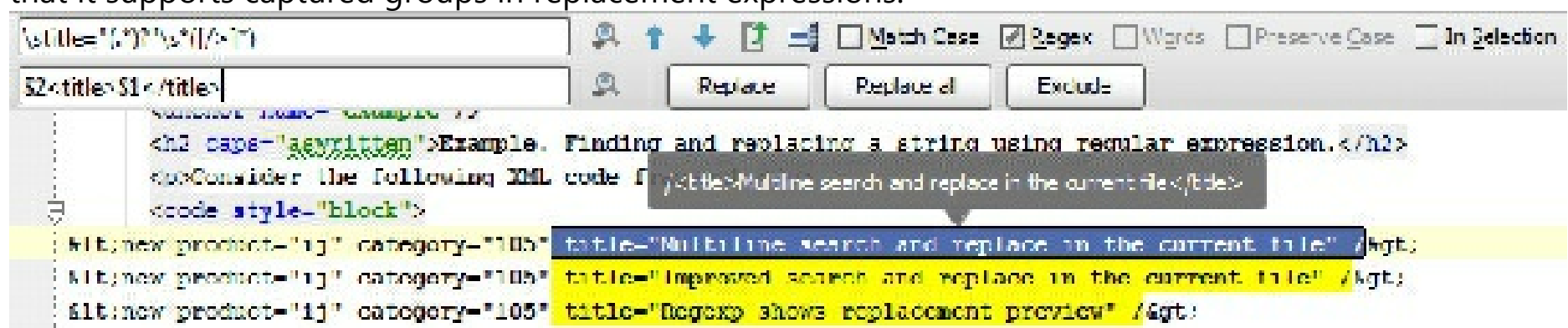
Regular expressions are powerful and widely used, but sometimes it's just too hard to write them

properly. IntelliJ IDEA will help you check any Regex in your code: just place caret at it and press Alt+Enter to use the [Check Regex](#) intention:



Find and replace with Regex groups

Another place where IntelliJ IDEA helps with Regex is the [Find and Replace](#) feature. It's worth knowing that it supports captured groups in replacement expressions.



Bytecode viewer

Sometimes seeing the actual bytecode your program generates is very [insightful](#).

In IntelliJ IDEA you can always do that via **View | Show Bytecode**.

Version control

Amend changes

In the [Commit Changes](#) dialog IntelliJ IDEA offers to perform a variety of operations. One of them is **Amend commit**, which is useful when you want to change your last commit and join your current change to it.

Shelves and patches

Shelves is an IDE feature similar to [Git Stash](#), but that works for all VCS: it helps when you need to pause your current work and pull something from the repository to fix it asap, and then resume working on whatever it was you were working on. This feature takes care of locally changed files without committing them, so no more lost changes or hastily made merge commits.

Refer to the page [Git-Stash](#) and to the section [Stashing and Unstashing](#) for more detail.

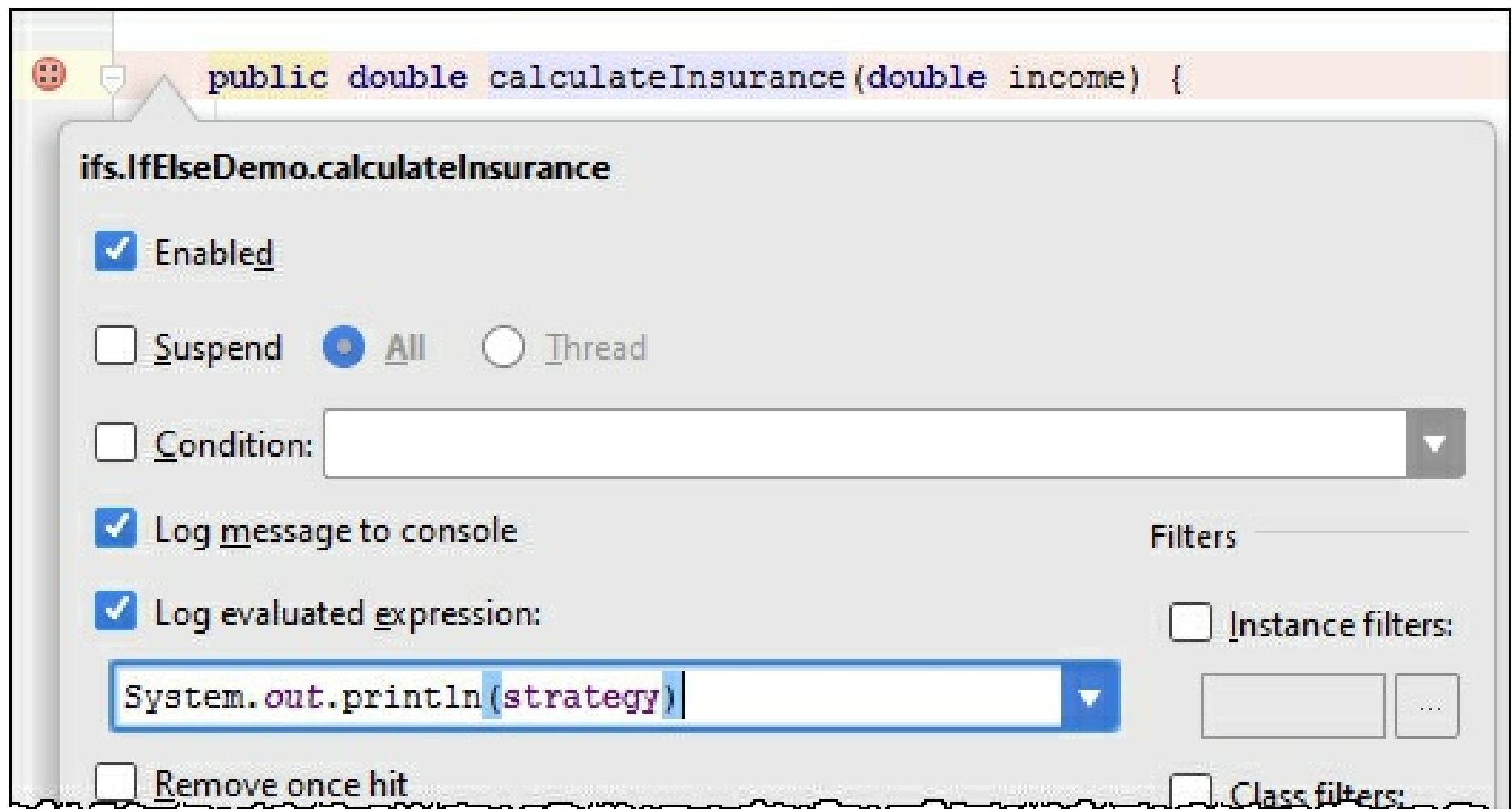
Patches allow you to save a set of changes to a text file that can be transferred via email (or any other ancient medium), and then applied to the code. It's super helpful when you really need to commit something after your plane crash landed on a desert island, or you somehow else got yourself in a situation without reliable broadband connection.

Refer to the section [Using Patches](#) for more detail.

Debugging

Action, or method breakpoints

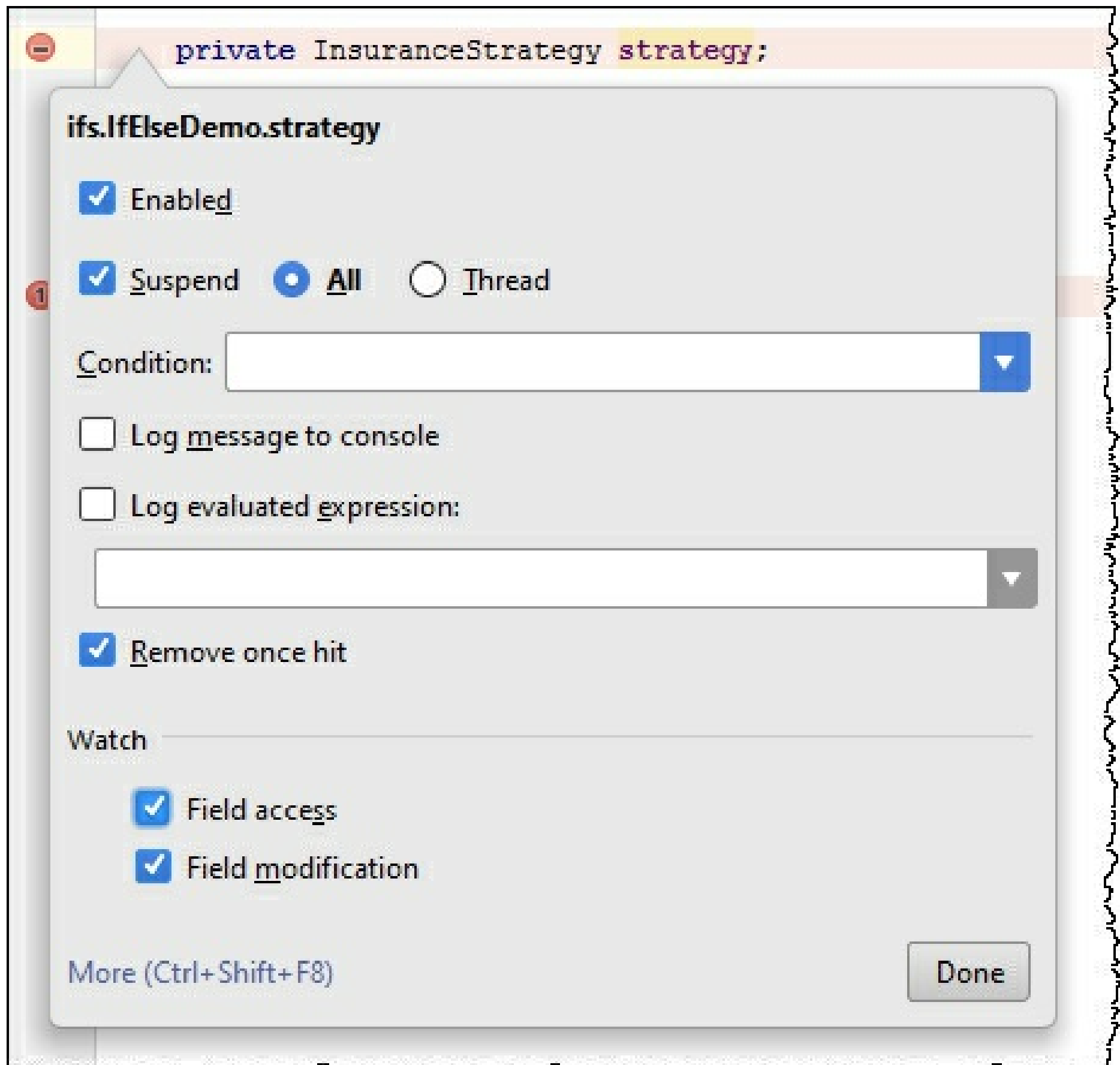
Sometimes you may want to evaluate something at a particular line of code without actually making a stop. You can do that by using a [Method breakpoint](#). To create one, just click the gutter holding `Shift`.



This way you can print any expression to the output without changing the code. This is especially useful when you debug libraries or a remote application.

Field breakpoints or field watchpoints

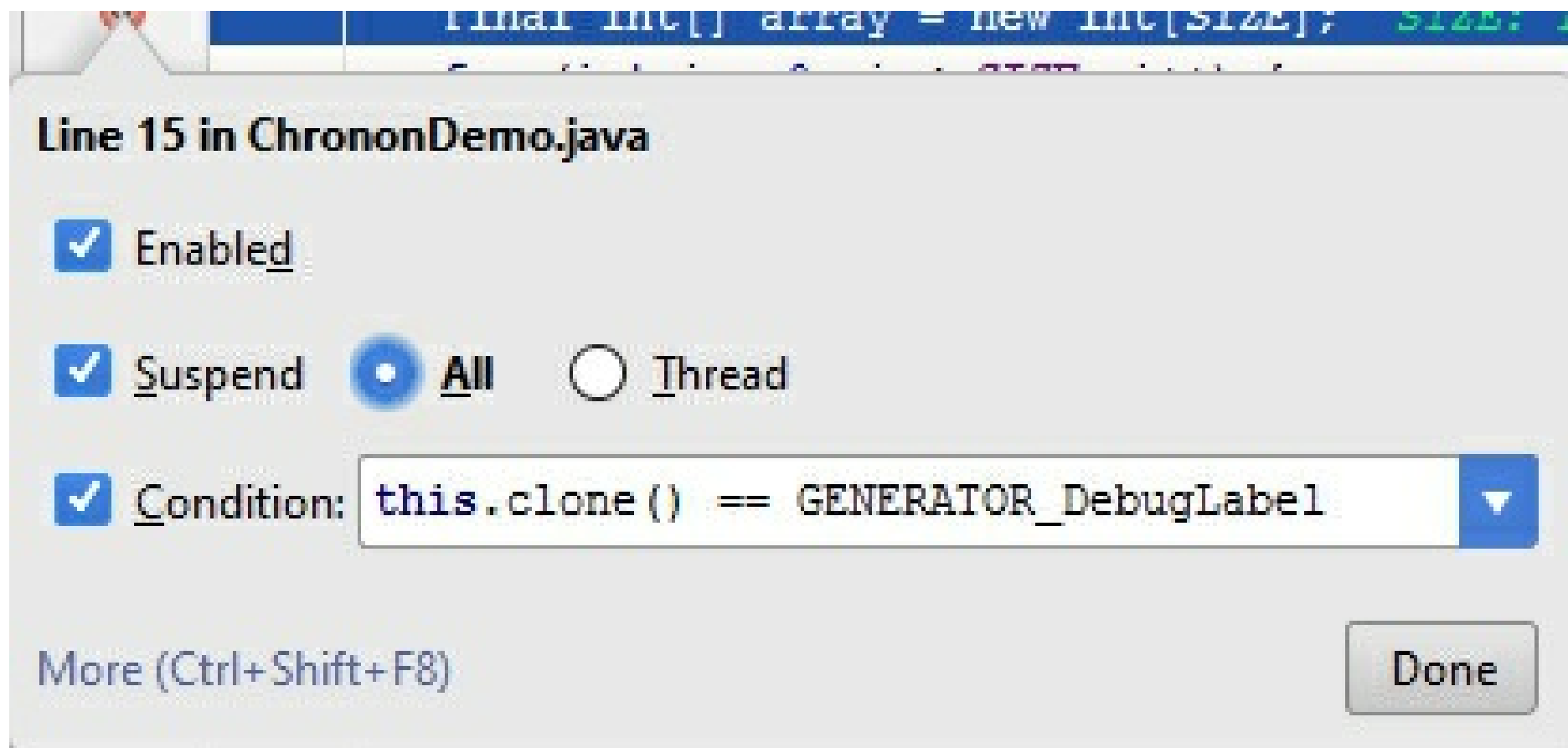
In addition to the action breakpoints mentioned above, you can also use Field watchpoints. This breakpoint will stop execution when a field associated with it is accessed. To create field watchpoints, just click the gutter holding Alt (Ctrl+Cmd for macOS).



Object markers

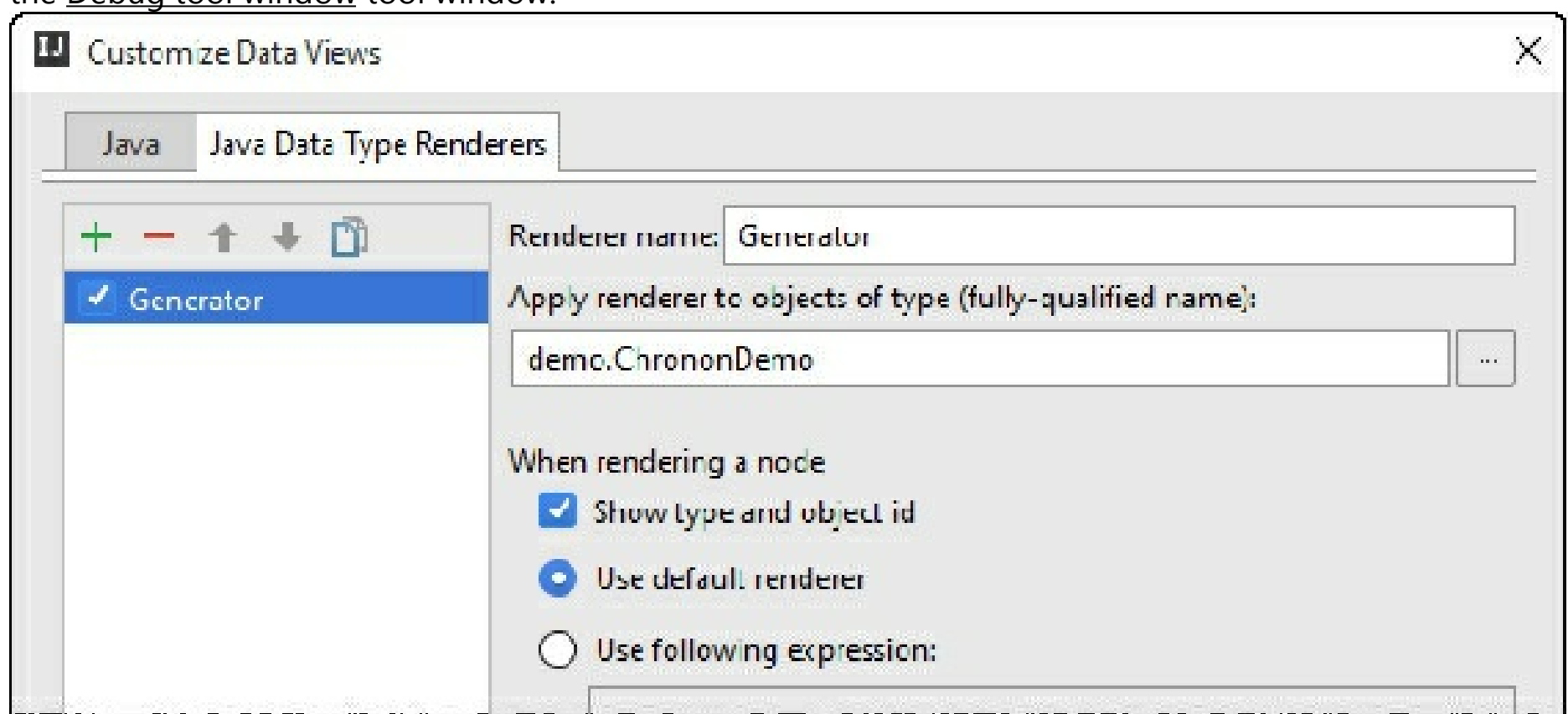
When you're debugging an application, IntelliJ IDEA lets you mark particular instances of arbitrary objects with colored labels for easier identification via the [Mark Object](#) action (available in the Evaluate Expression, [Variables](#) or views. [Watches](#) views.)

And if you have any instance marked with a label, you can use it in the condition expression as well:



Custom data renderers

[Evaluate Expression](#), [Variables](#), [Watches](#) and [inline debugger](#) all use a standard way to render variable values, mostly based on `toString` implementation of classes. Not everyone knows that you can define your own custom renderers for any class. For that, select **Customize Data Views** from the context menu in the [Debug tool window](#) tool window.



This is especially useful when some of the classes in the libraries you're using do not provide a meaningful `toString` implementation—so you can define it yourself outside of the library.

Drop frame

In case you want to “go back in time” while debugging you can do it via the Drop Frame action. This is a great help if you mistakenly stepped too far. This will not revert the global state of your application but at least will get you back by stack of frames.

Force return

The way around, if you want to jump to the future, and force the return from the current method without executing any more instructions from it, use the **Force Return** action (to invoke it, press `Ctrl+Shift+A` and type the action name). If the method returns a value, you'll have to specify it.

DCEVM

Sometimes when you're making quick changes to code, you want to immediately see how they will behave in a working application. Unfortunately, the Java HotSwap VM has lots of limitations: you can't, say, add a new method or a field to a class and perform the hot swapping; the only thing you can actually

change during the hot swapping is the method bodies.

Refer to the sections [Reload modified classes](#) and [Alter the program's execution flow](#) for details.

Luckily, there is a way to amend this situation with the new open-source project Dynamic Code Evolution VM, a modification of Java HotSwap VM with unlimited support for reloading classes at runtime.

Using it in IntelliJ IDEA is easy with the dedicated [plugin](#). When you enable the plugin, the IDE will offer you to download DCEVM JRE for your environment. Then you'll have to choose it in the list of alternative JREs.

Update application

If you are running your application on an application server (Tomcat, JBoss, and so on), can you reload changed classes and resources using the Update application action via `Ctrl+F10`.

Refer to the section [Update applications on application servers](#) for details.

Tools

External tools

IntelliJ IDEA has many developer tools integrated and working out of the box. If a tool you need is not integrated, but you'd like to use it via a shortcut, go to **Settings/Preferences | Tools | [External Tools](#)**, and configure how to run this tool. Then you'll be able to run this tool via the **Tools | External Tools** main menu.

Migrate from Eclipse to IntelliJ IDEA

Switching from *Eclipse* to *IntelliJ IDEA*, especially if you've been using *Eclipse* for a long time, requires understanding some fundamental differences between the two IDEs, including their user interfaces, compilation methods, shortcuts, project configuration and other aspects.

User Interface

No workspace

The first thing you'll notice when launching *IntelliJ IDEA* is that it has no *workspace* concept. This means that you can work with only one project at a time. While in *Eclipse* you normally have a set of projects that may depend on each other, in *IntelliJ IDEA* you have a single project that consists of a set of modules.

If you have several unrelated projects, you can open them in separate windows.

If you still want to have several unrelated projects opened in one window, as a workaround you can configure them as modules.

IntelliJ IDEA vs Eclipse terminology

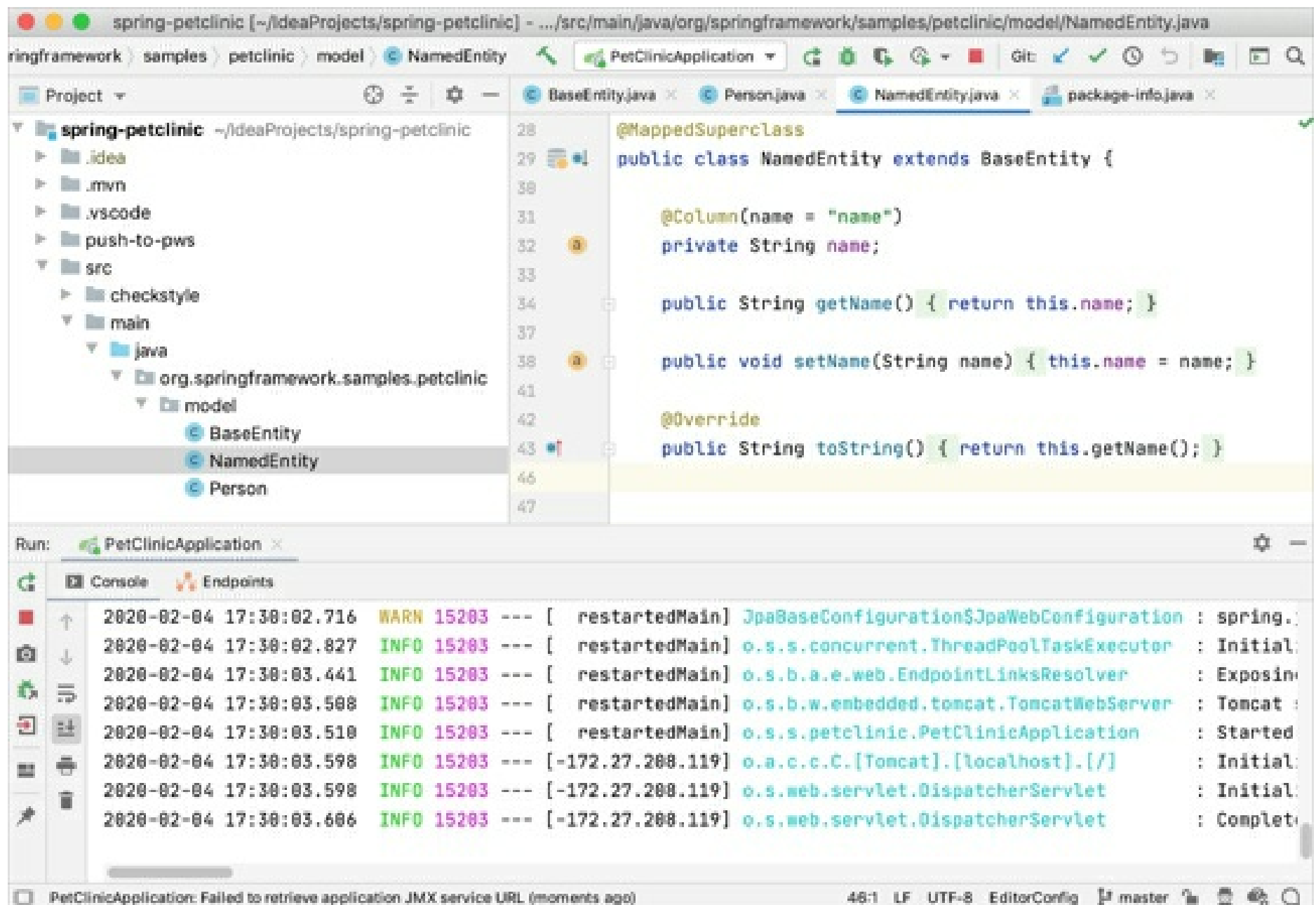
The table below compares the terms in *Eclipse* and *IntelliJ IDEA*:

Eclipse	IntelliJ IDEA
Workspace	Project
Project	Module
Facet	Facet
Library	Library
JRE	SDK
Classpath variable	Path variable

No perspectives

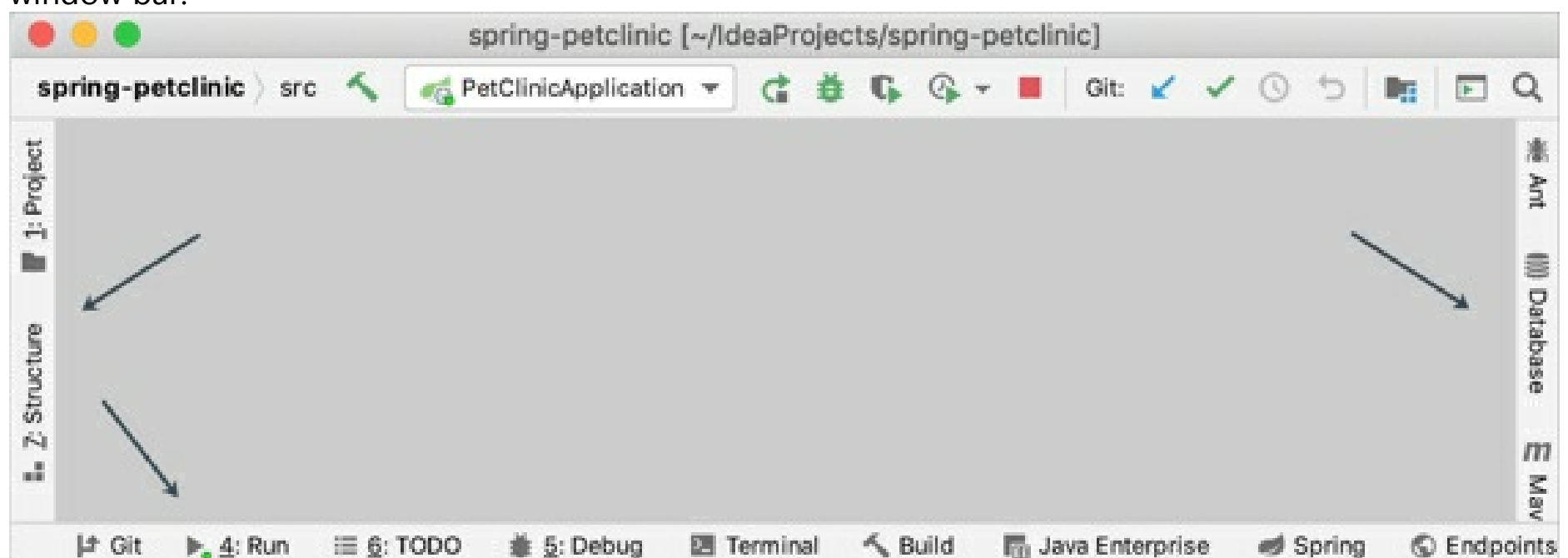
The second big surprise when you switch to *IntelliJ IDEA* is that it has no *perspectives*.

It means that you don't need to switch between different workspace layouts manually to perform different tasks. The IDE follows your context and brings up the relevant tools automatically.

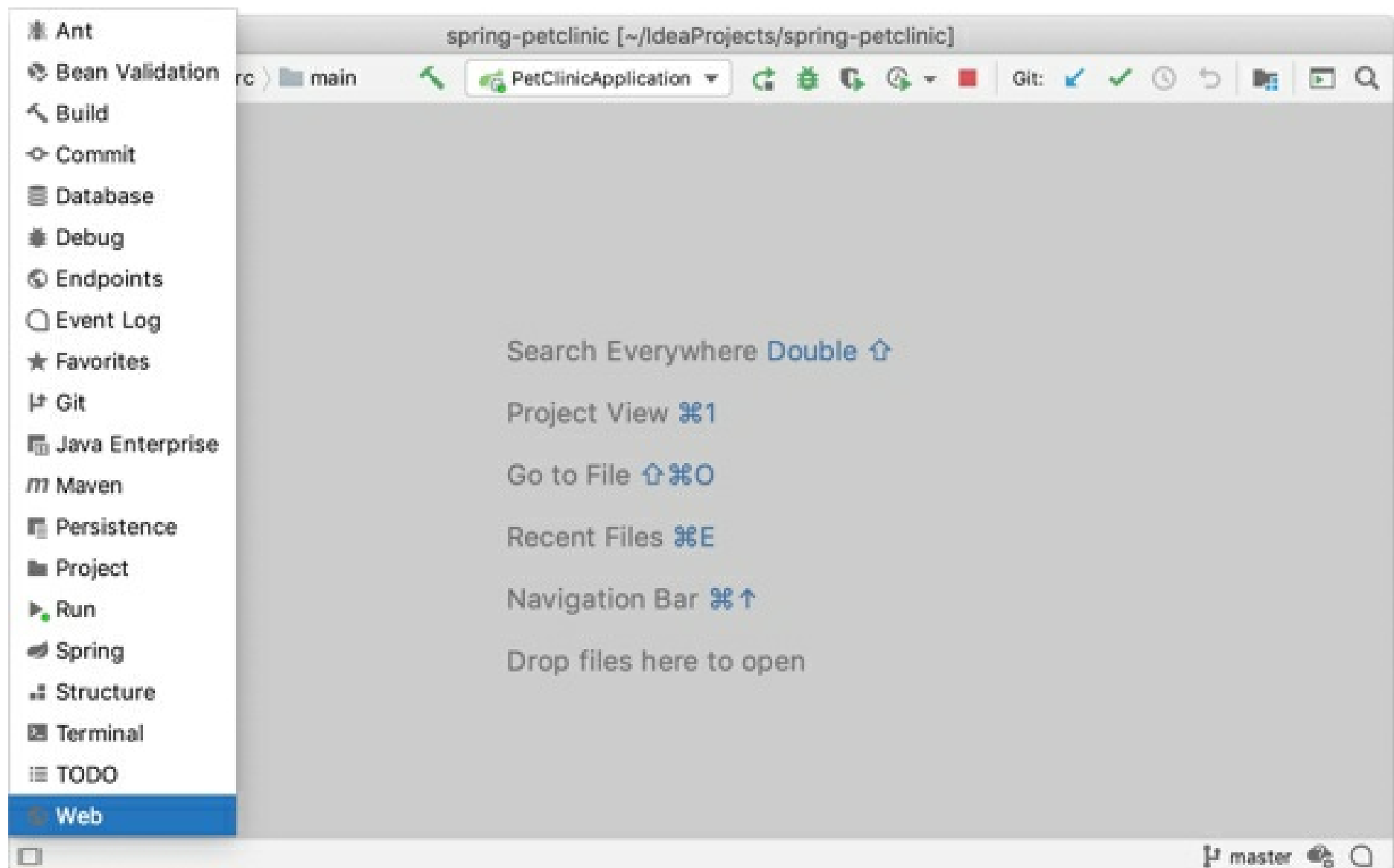


Tool windows

Just like in *Eclipse*, in *IntelliJ IDEA* you also have tool windows. To open a tool window, click it in the tool window bar:



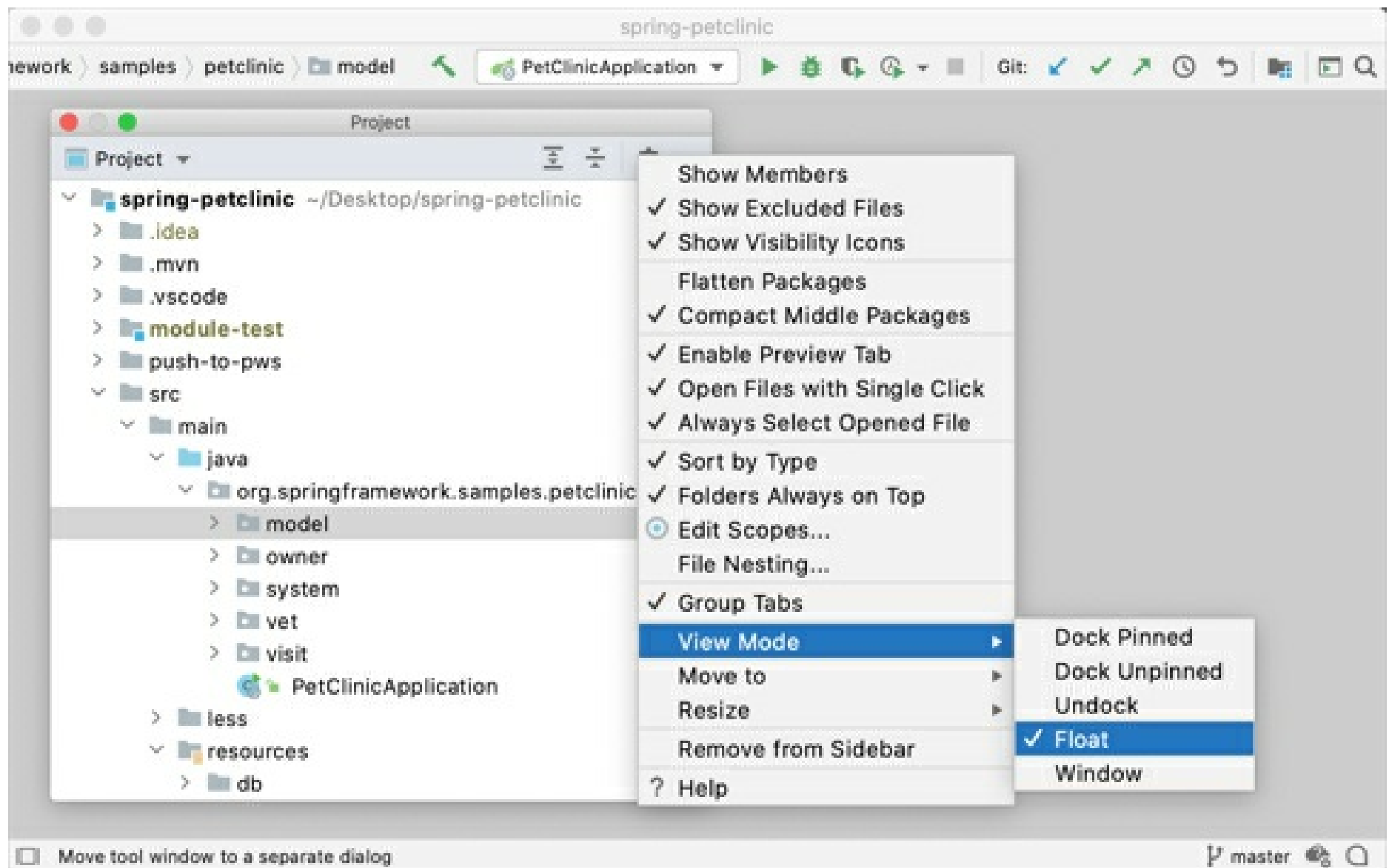
If the tool window bar is hidden, you can open any tool window by hovering over the corresponding icon in the bottom left corner:



If you want to make the tool window bar visible for a moment, you can press `Alt` twice and hold it. If you don't want to use the mouse, you can always switch to any toolbar by pressing the shortcut assigned to it. The most important shortcuts to remember are:

- **Project:** `Alt+1`
- **Commit:** `Alt+9`
- **Terminal:** `Alt+F12`

Another thing about tool windows is that you can drag, pin, unpin, attach and detach them:



To help store/restore the tool windows layout, there are two useful commands:

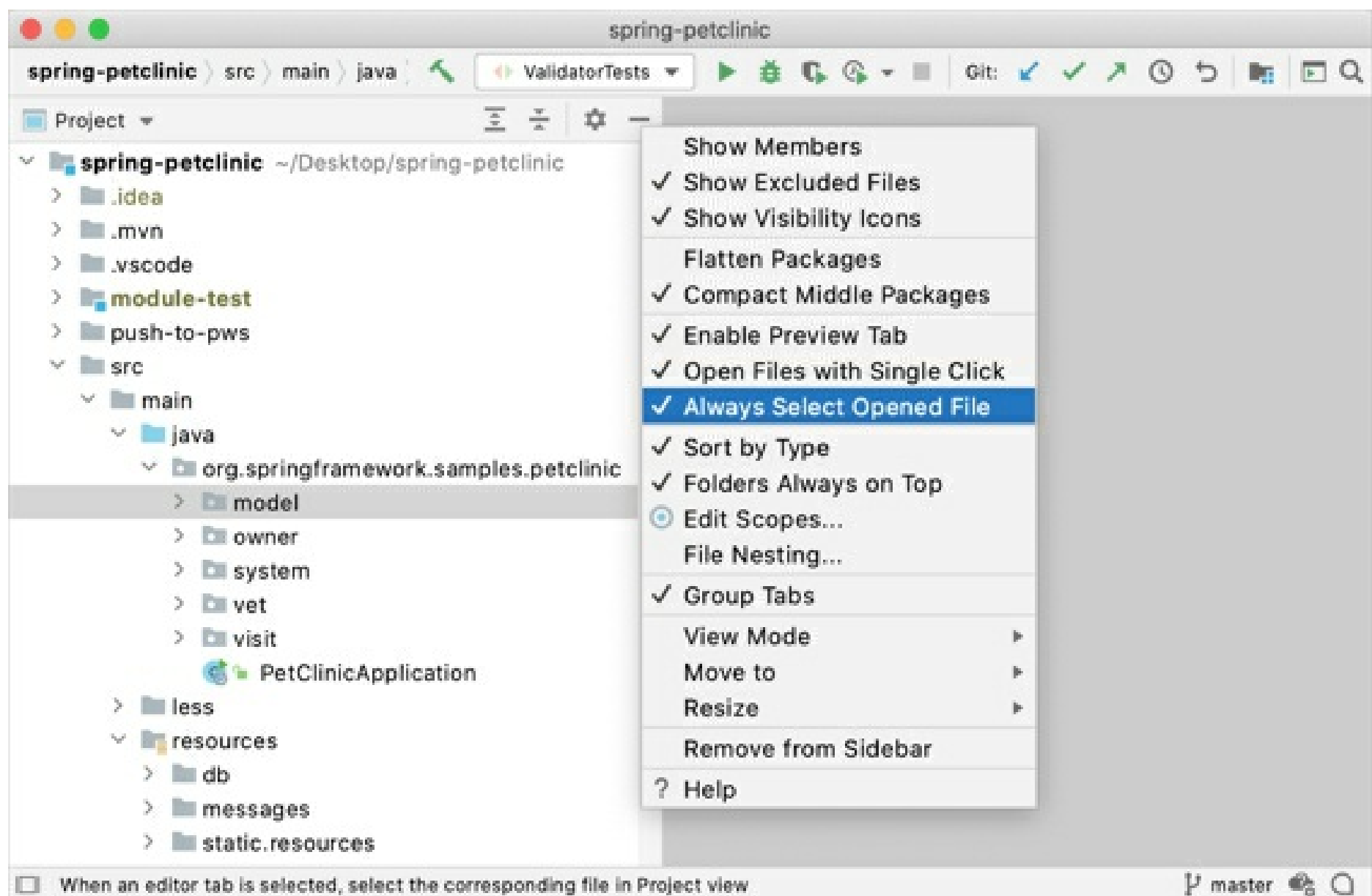
- **Window | Store Current Layout as Default**
- **Window | Restore Default Layout** (also available via `Shift+F12`)

Multiple windows

Windows management in *IntelliJ IDEA* is slightly different from *Eclipse*. You can't open several windows with one project, but you can detach any number of editor tabs into separate windows.

Always select opened files

By default, *IntelliJ IDEA* doesn't change the selection in the Project tool window when you switch between editor tabs. However, you can enable it in the tool window settings:

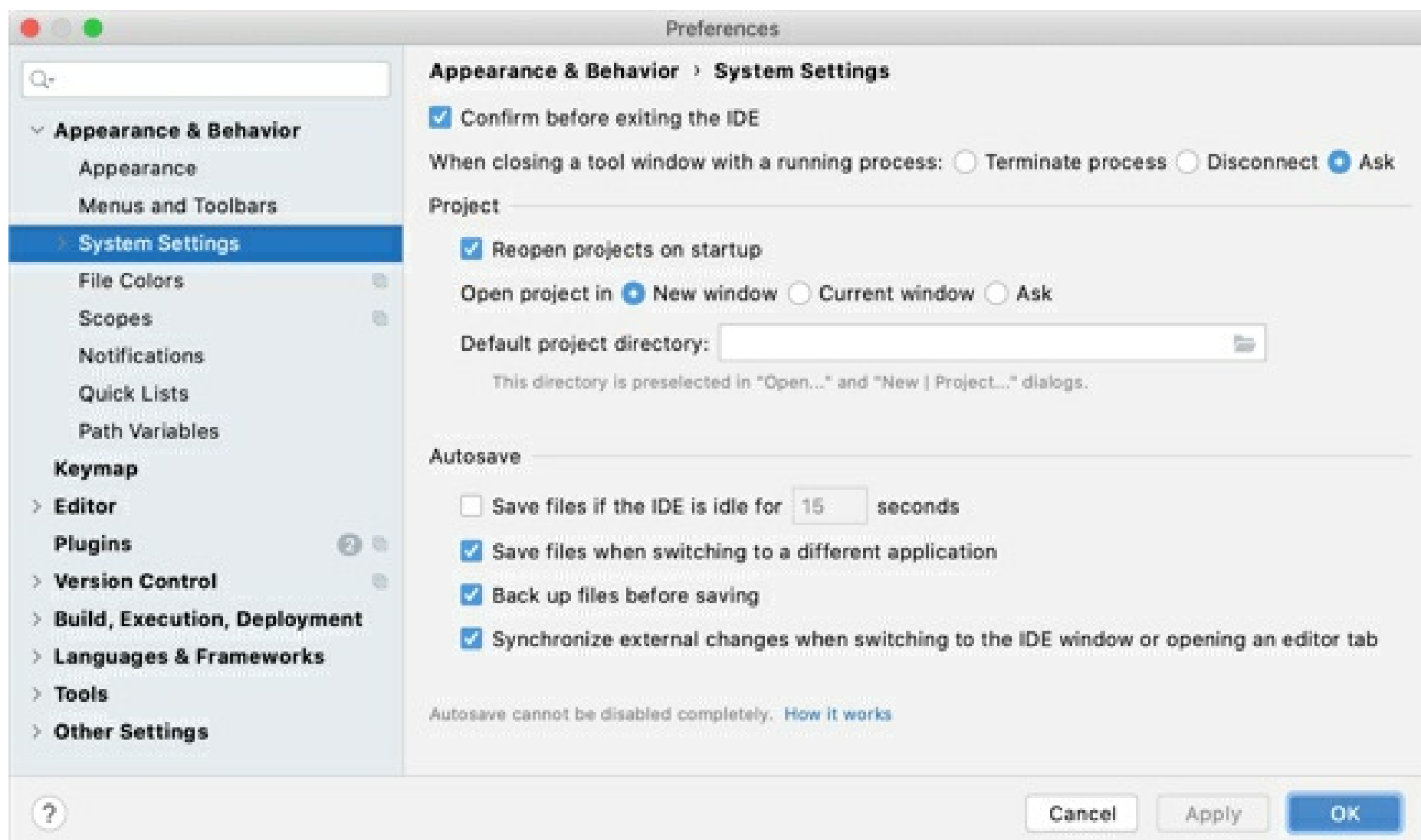


General workflows

No 'save' button

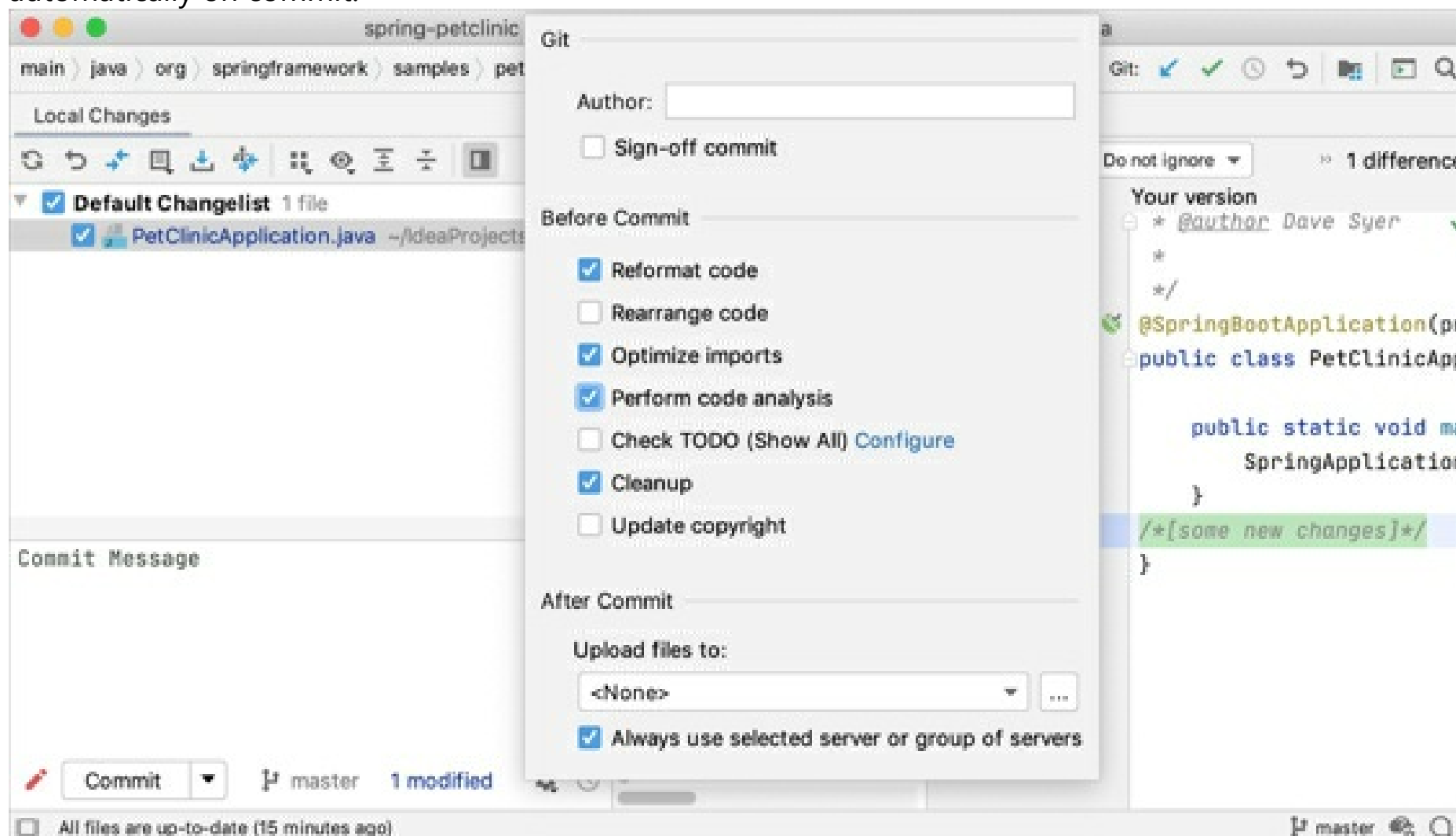
Time for some really shocking news: *IntelliJ IDEA* has no **Save** button. Since in *IntelliJ IDEA* you can undo refactorings and revert changes from Local History, it makes no sense to ask you to save your changes every time.

Still, it's worth knowing that physical saving to disk is triggered by certain events, including compilation, closing a file, switching focus out of the IDE, and so on. You can change this behavior via **Settings/Preferences | Appearance & Behavior | System Settings**:



No save actions

One of the features you may miss in *IntelliJ IDEA* as an Eclipse user is **save actions**: the actions triggered automatically on save, such as reformatting code, organizing imports, adding missing annotations and the **final** modifier, and so on. Instead, IntelliJ IDEA offers you to run the corresponding actions automatically on commit:



Or manually:

- **Code | Reformat Code** `Ctrl+Alt+L`
- **Code | Optimize Imports** `Ctrl+Alt+O`
- **Analyze | Cleanup**

Compilation

Auto-compilation

If you want to mimic the **Eclipse** behavior, you can invoke the **Build Project** action `Ctrl+F9`- it will save the changed files and compile them. For your convenience, you can even reassign the `Ctrl+S` shortcut to the **Build Project** action.

The screenshot shows the 'Preferences' dialog in IntelliJ IDEA, specifically the 'Build, Execution, Deployment' > 'Compiler' tab. The left sidebar lists various settings categories, with 'Compiler' selected. The main panel is titled 'For current project' and contains the following settings:

- Resource patterns:** A text field containing 'ava;!*.form;!*.class;!*.groovy;!*.scala;!*.flex;!*.kt;!*.clj;!*.a...' with a help icon. Below it, a note explains the use of semicolons, negation, and wildcards.
- Clear output directory on rebuild:** Checked.
- Add runtime assertions for notnull-annotated methods and parameters:** Checked, with a 'Configure annotations...' button.
- Automatically show first error in editor:** Checked.
- Display notification on build completion:** Checked.
- Build project automatically:** Unchecked, with a note '(only works while not running / debugging)'.
- Compile independent modules in parallel:** Unchecked, with a note '(may require larger heap size)'.
- Rebuild module on dependency change:** Checked.
- Shared build process heap size (Mbytes):** A text field with the value '700'.
- Shared build process VM options:** An empty text field.
- User-local build process heap size (Mbytes) (overrides Shared size):** An empty text field.
- User-local build process VM options (overrides Shared options):** An empty text field.

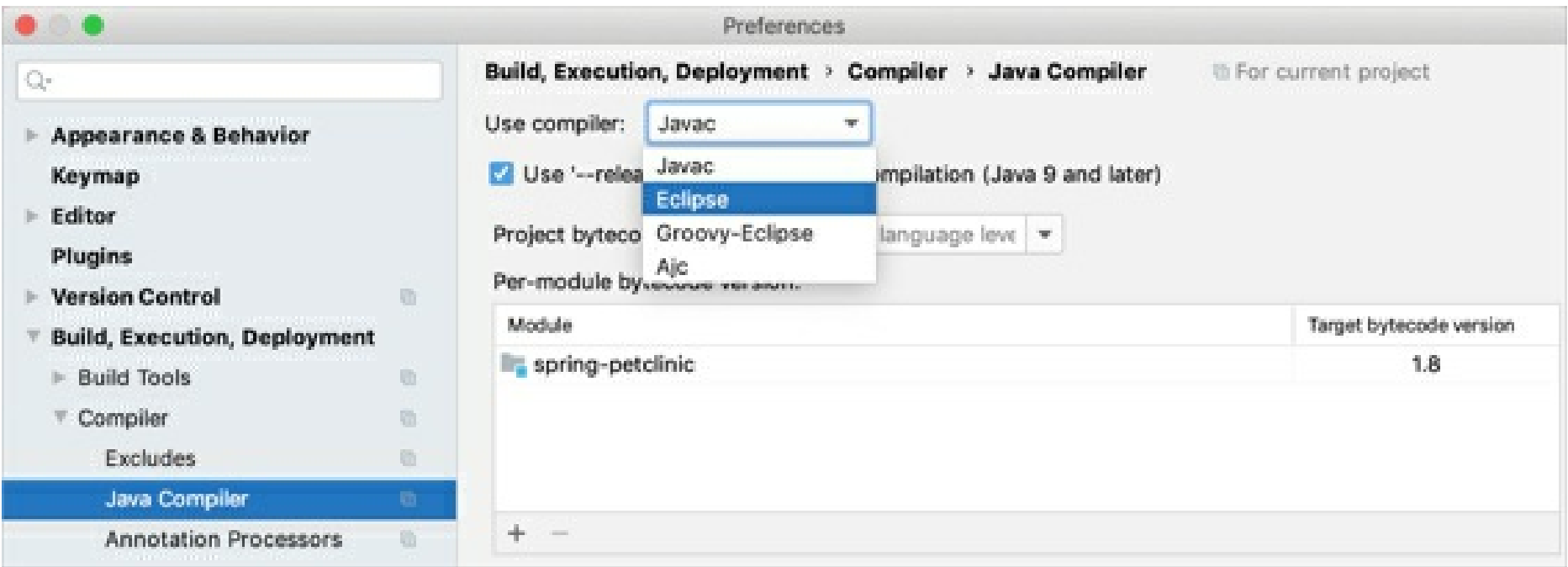
At the bottom, a 'WARNING!' section states: 'If option 'Clear output directory on rebuild' is enabled, the entire contents of directories where generated sources are stored WILL BE CLEARED on rebuild.' The bottom of the dialog has 'Cancel', 'Apply', and 'OK' buttons.

This is why, even if the **Build project automatically** option is enabled, *IntelliJ IDEA* doesn't perform automatic compilation if at least one application is running: it will reload classes in the application implicitly. In this case you can call **Build | Build Project** `Ctrl+F9`.

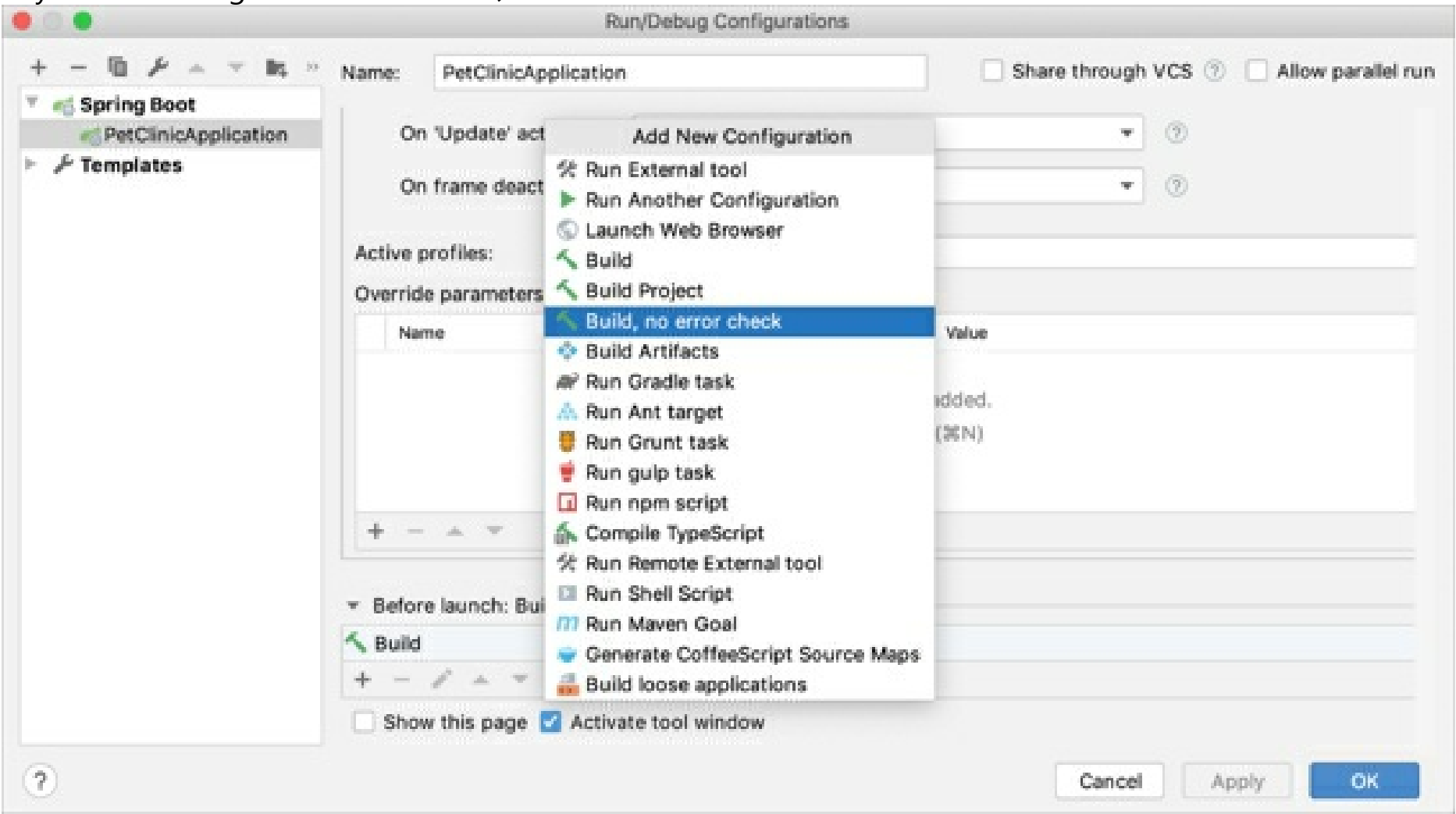
Problems tool window

Eclipse compiler

While *Eclipse* uses its own compiler, *IntelliJ IDEA* uses the *javac* compiler bundled with the project JDK. If you must use the *Eclipse* compiler, navigate to **Settings/Preferences | Build, Execution, Deployment | Compiler | Java Compiler** and select it as shown below:



The biggest difference between the *Eclipse* and *javac* compilers is that the *Eclipse* compiler is more tolerant to errors, and sometimes lets you run code that doesn't compile. In situations when you need to run code with compilation errors in *IntelliJ IDEA*, replace the **Build** option in your run configuration with **Build, no error check**:



Shortcuts

IntelliJ IDEA shortcuts are completely different from those in *Eclipse*. The table below shows how the top *Eclipse* actions (and their shortcuts) are mapped to *IntelliJ IDEA* (you may want to print it out to always have it handy). If you choose a keymap specific to your operating system (**Default** for Windows/Linux or **macOS** for macOS), there may be conflicts between shortcuts used in *IntelliJ IDEA* and your OS. To avoid such conflicts, we recommend tweaking your OS shortcut settings (refer to Keymap for more details).

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Code completion	Ctrl+Space	Basic completion	Ctrl+Space

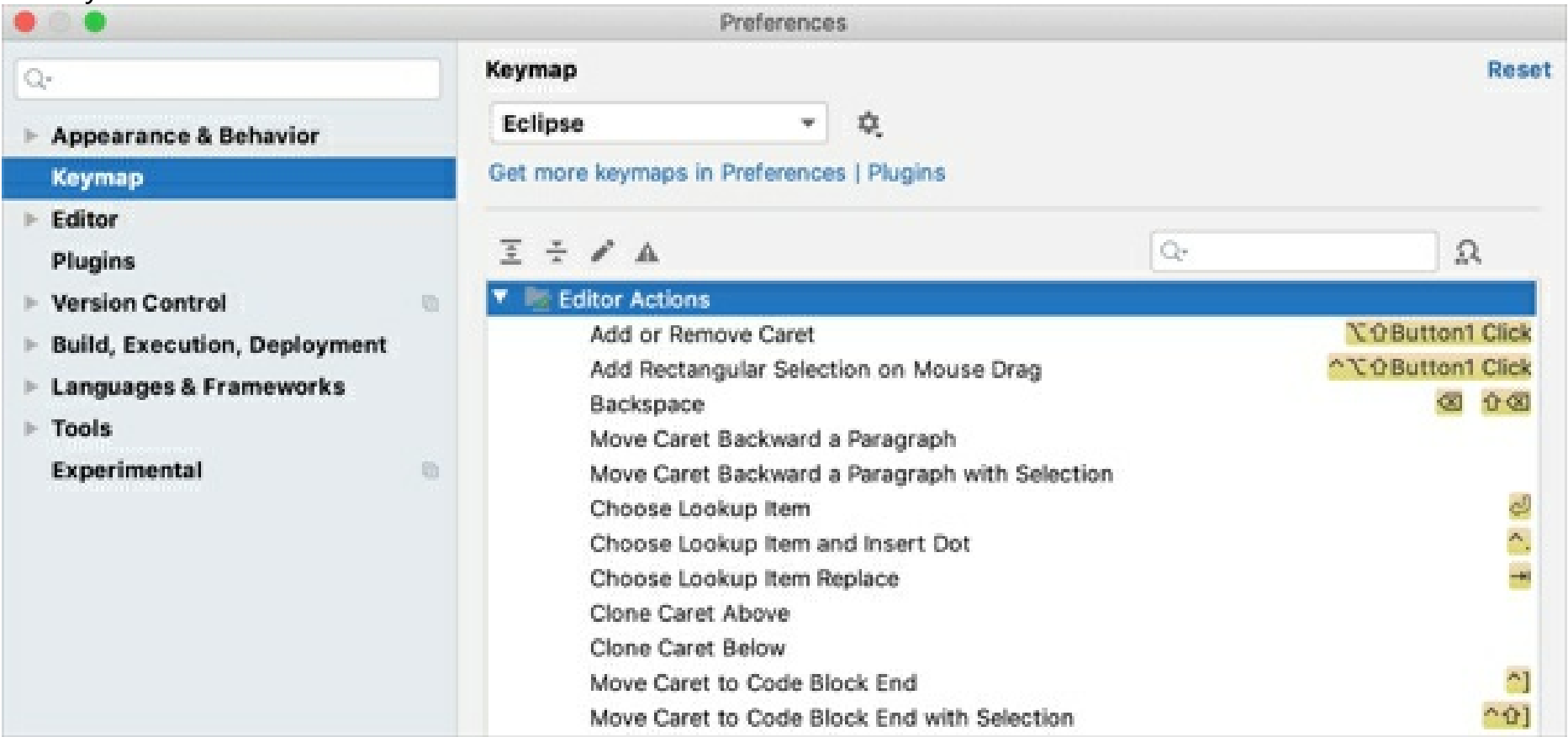
-	-	Type-matching completion	Ctrl+Shift+Space
-	-	Statement completion	Ctrl+Shift+Enter
Quick access	Ctrl+3	Search everywhere	Double Shift
Maximize active view or editor	Ctrl+M	Hide all tool windows	Ctrl+Shift+F12
Open type	Ctrl+Shift+T	Navigate to class	Ctrl+N
Open resource	Ctrl+Shift+R	Navigate to file	Ctrl+Shift+N
-	-	Navigate to symbol	Ctrl+Alt+Shift+N
Next view	Ctrl+F7	-	-
-	-	Recent files	Ctrl+E
-	-	Switcher	Ctrl+Tab
Quick outline	Ctrl+O	File structure	Ctrl+F12
Move lines	Alt+Up/Down	Move lines	Alt+Shift+Up/ Alt+Shift+Down
Delete lines	Ctrl+D	Delete lines	Ctrl+Y
Quick fix	Ctrl+1	Show intention action	Alt+Enter
Quick switch editor	Ctrl+E	Switcher	Ctrl+Tab
-	-	Recent files	Ctrl+E

Quick hierarchy	Ctrl+T	Navigate to type hierarchy	Ctrl+H
-	-	Navigate to method hierarchy	Ctrl+Shift+H
-	-	Show UML popup	Ctrl+Alt+U
Last edit location	Ctrl+Q	Last edit location	Ctrl+Shift+Backspace
Next editor	Ctrl+F6	Select next tab	Alt+Right
Run	Ctrl+Shift+F11	Run	Shift+F10
Debug	Ctrl+F11	Debug	Shift+F9
Correct indentation	Ctrl+I	Auto-indent lines	Ctrl+Alt+I
Format	Ctrl+Shift+F	Reformat code	Ctrl+Alt+L
Surround with	Ctrl+Alt+Z	Surround with	Ctrl+Alt+T
-	-	Surround with live template	Ctrl+Alt+J
Open declaration	F3	Navigate to declaration	Ctrl+B
-	-	Quick definition	Ctrl+Shift+I
Open type hierarchy	F4	Navigate to type hierarchy	Ctrl+H
-	-	Show UML popup	Ctrl+Alt+U

References in workspace	Ctrl+Shift+G	Find usages	Alt+F7
-	-	Show usages	Ctrl+Alt+F7
-	-	Find usages settings	Ctrl+Alt+Shift+F7
Open search dialog	Ctrl+H	Find in Files	Ctrl+Shift+F
Occurrences in file	Alt+Ctrl+U	Highlight usages in file	Ctrl+Shift+F7
Copy lines	Ctrl+Alt+Down	Duplicate lines	Ctrl+D
Extract local variable	Ctrl+Alt+L	Extract variable	Ctrl+Alt+V
Assign to field	Ctrl+2/ Ctrl+F	Extract field	Ctrl+Alt+F
Show refactor quick menu	Ctrl+Alt+T	Refactor this	Ctrl+Alt+Shift+T
Rename	Ctrl+Alt+R	Rename	Shift+F6
Go to line	Ctrl+L	Navigate to line	Ctrl+G
Structured selection	Alt+Shift+Up/ Alt+Shift+Down	Select word at caret	Ctrl+W/ Ctrl+Shift+W
Find next	Ctrl+]	Find next	F3
Show in	Ctrl+Alt+W	Select in	Alt+F1
Back	Ctrl+[	Back	Ctrl+Alt+Left
Forward	Ctrl+]	Forward	Ctrl+Alt+Right

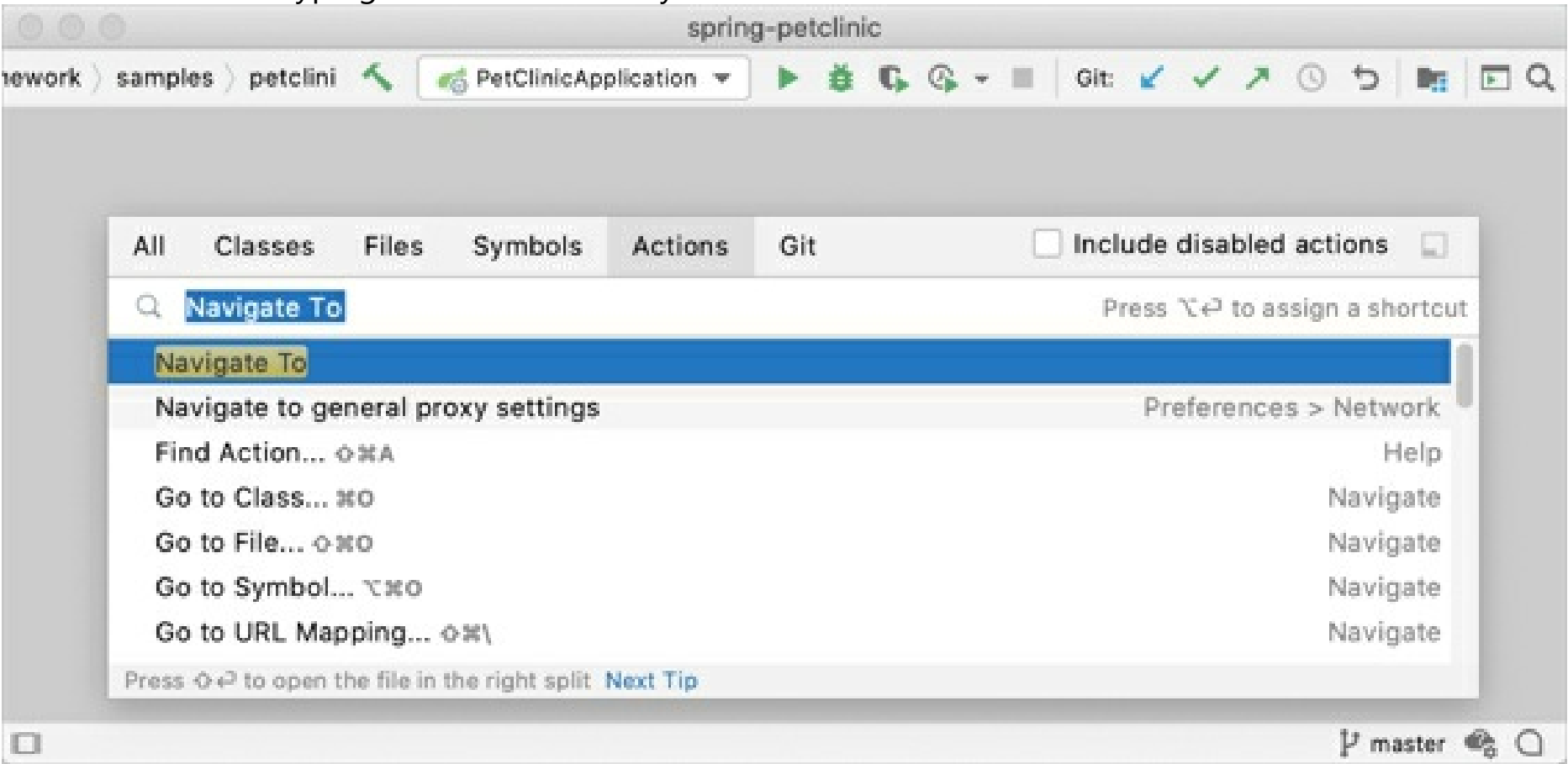
Eclipse keymap

For *Eclipse* users who prefer not to learn new shortcuts, *IntelliJ IDEA* provides the *Eclipse* keymap which closely mimics its shortcuts:



Find action

When you don't know the shortcut for some action, try using the **Find action** feature available via `Ctrl+Shift+A`. Start typing to find an action by its name, see its shortcut, or call it:

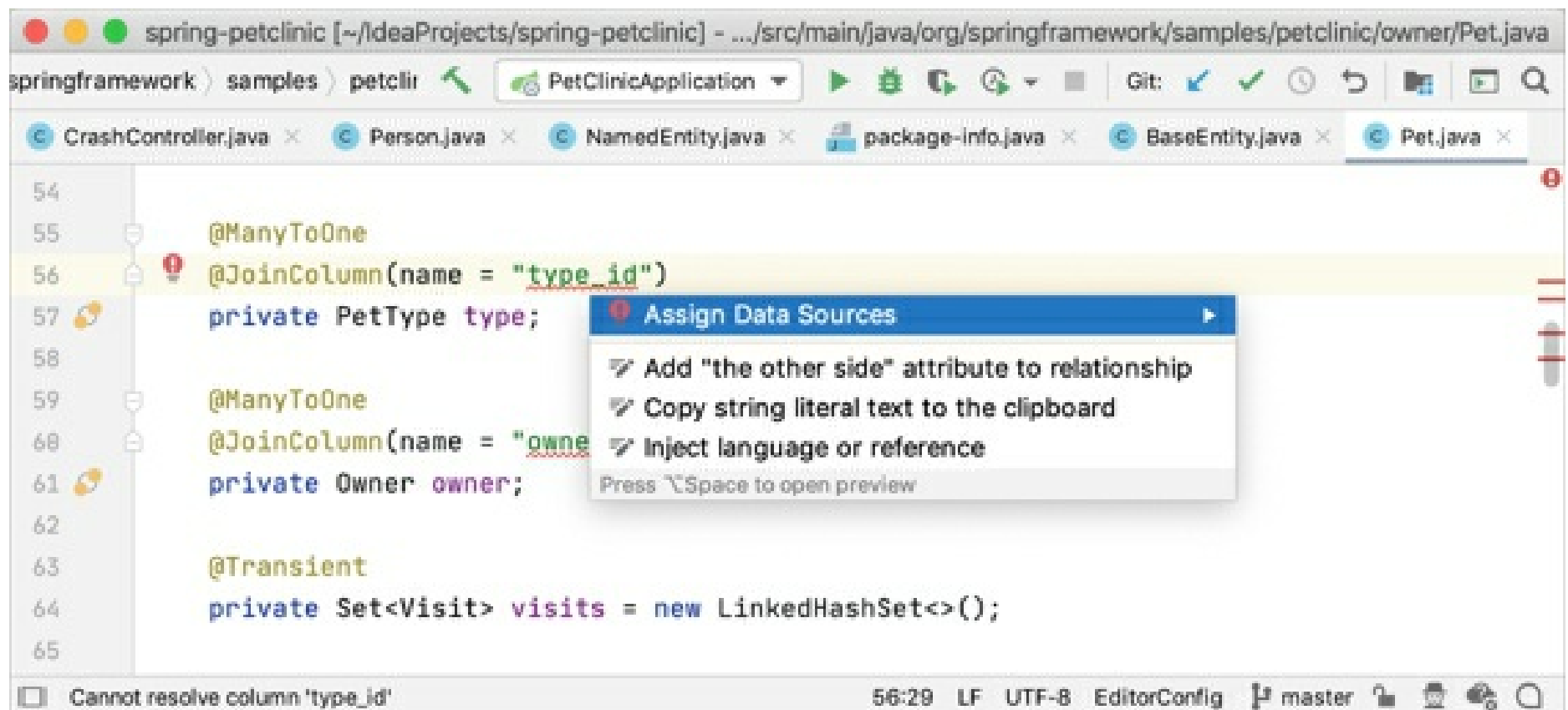


Coding assistance

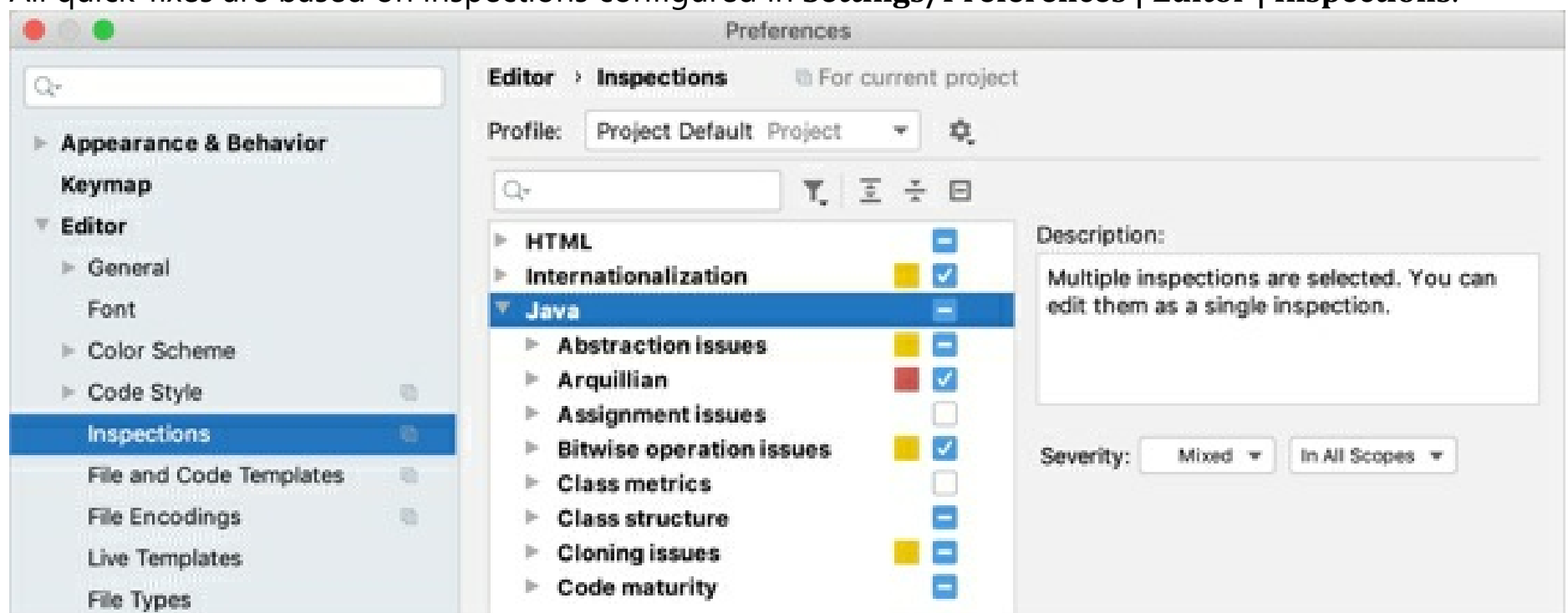
Both *Eclipse* and *IntelliJ IDEA* provide coding assistance features, such as code completion, code generation, quick-fixes, live templates, etc.

Quick-fixes

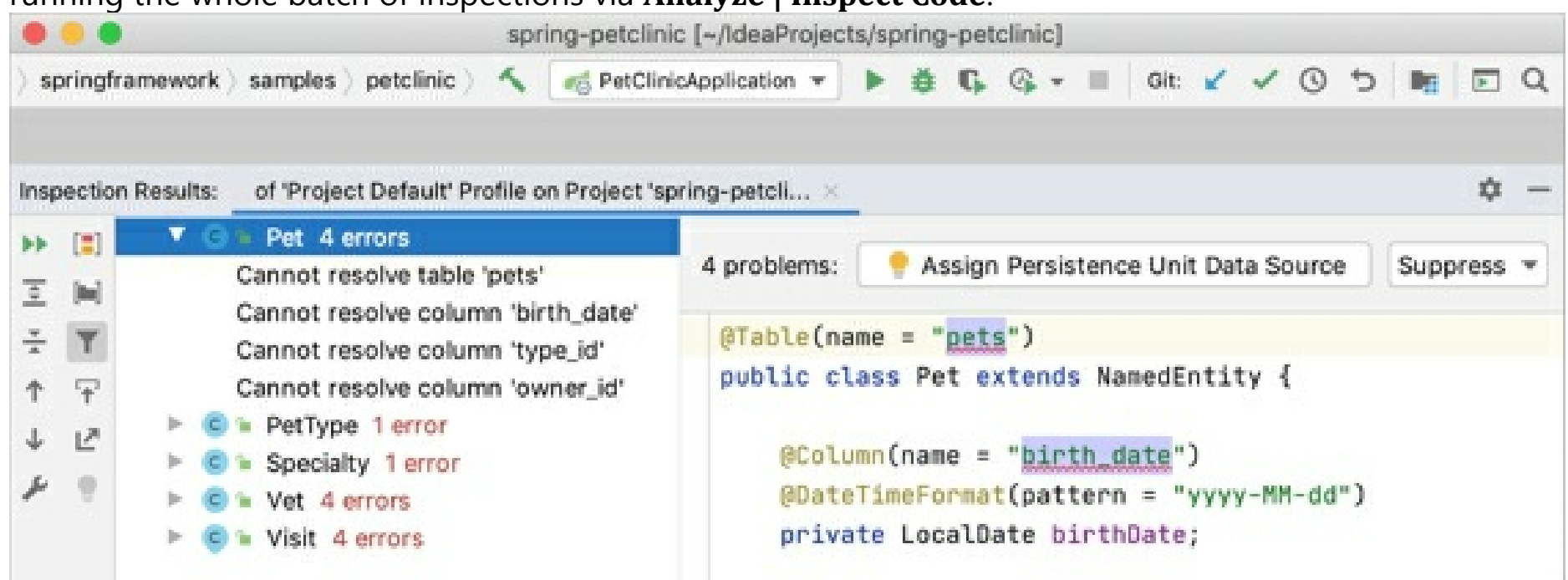
To apply a quick-fix in *IntelliJ IDEA*, press `Alt+Enter`:



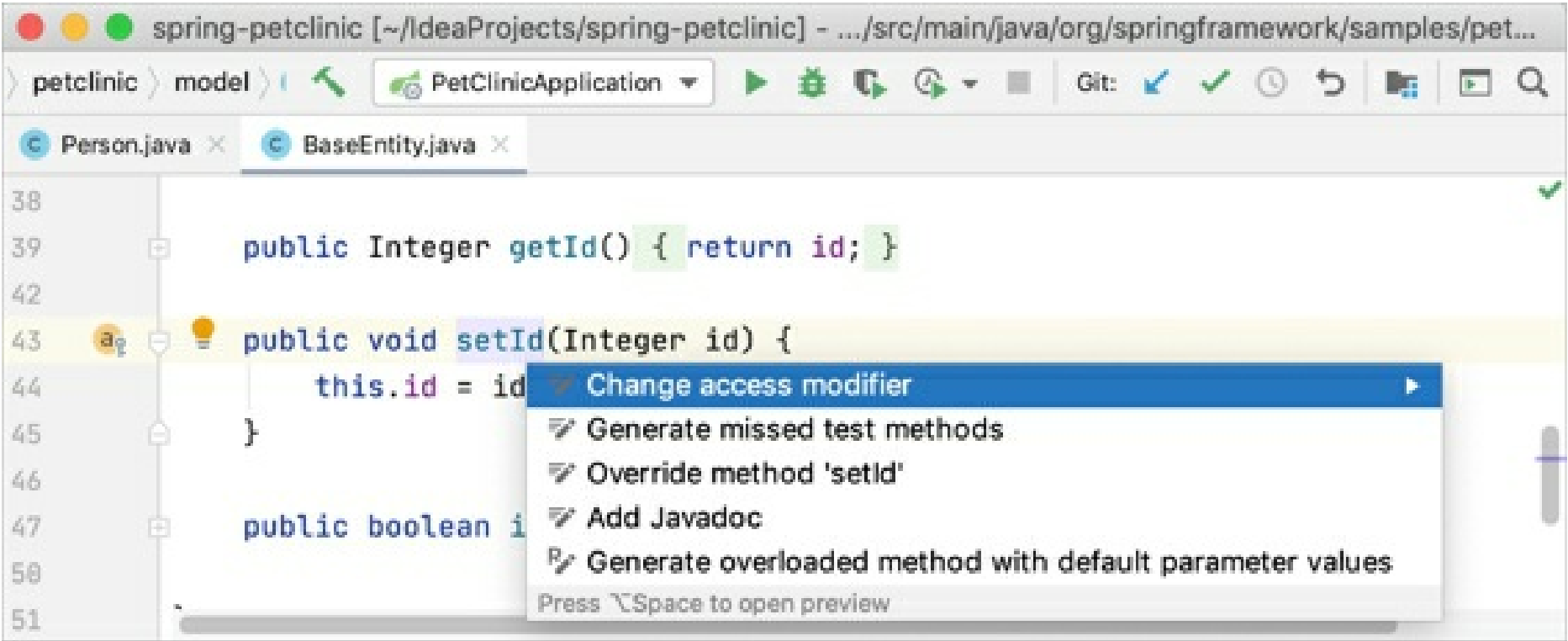
All quick-fixes are based on inspections configured in **Settings/Preferences | Editor | Inspections**:



If you want to apply a quick-fix to several places at once (i.e. to a whole folder, module or even a project), you can do it by running the corresponding inspection via **Analyze | Run Inspection By Name** or by running the whole batch of inspections via **Analyze | Inspect Code**:



Apart from outright problems, *IntelliJ IDEA* also recognizes code constructs that can be improved or optimized via the so-called *intentions* (also available with **Alt+Enter**):



Eclipse

IntelliJ IDEA

Action

Shortcut

Action

Shortcut

Quick fix

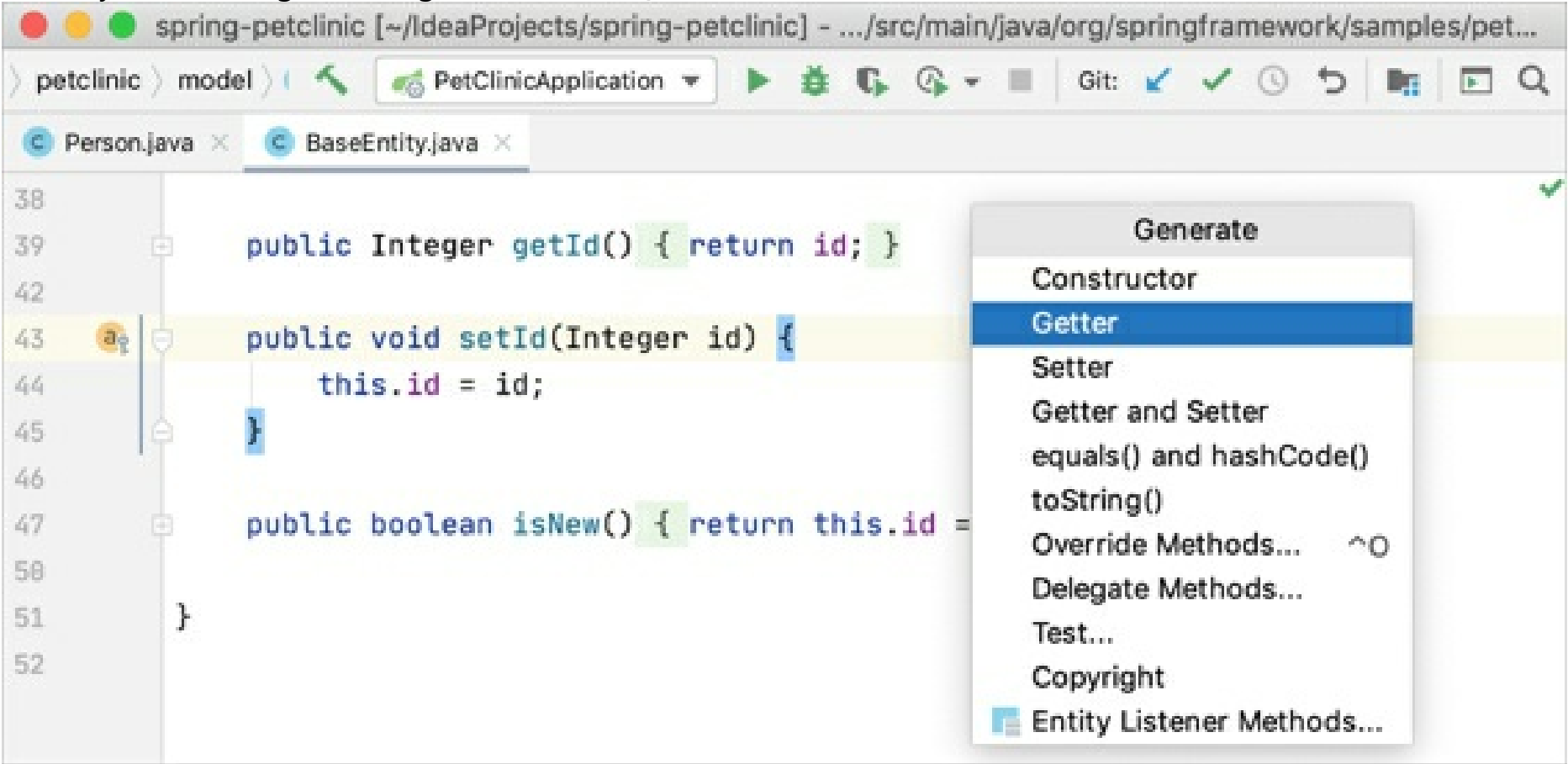
Ctrl+1

Show intention action

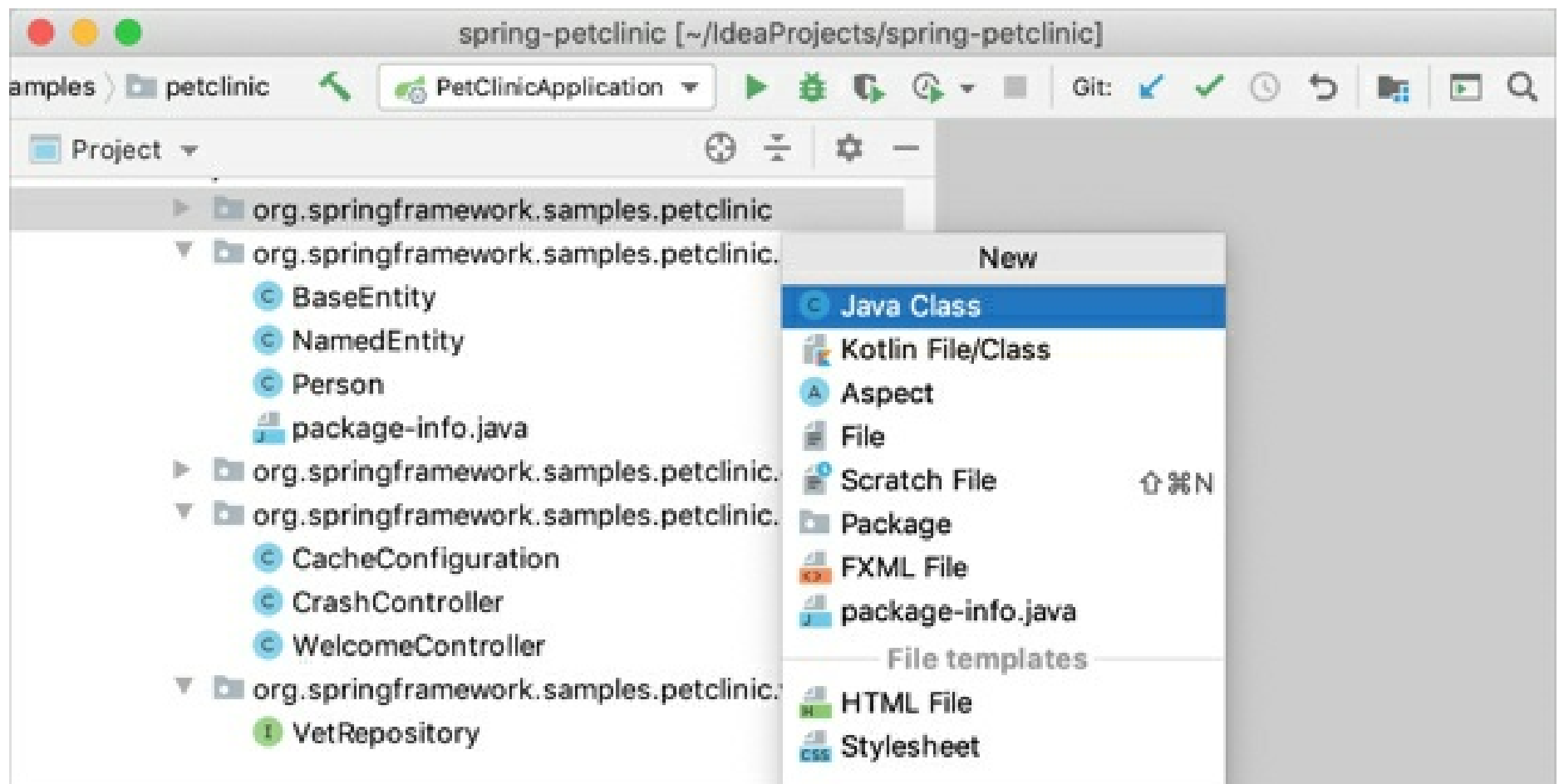
Alt+Enter

Generating code

The key action for generating code is **Code | Generate**, available via Alt+Insert:



This action is context-sensitive and is available not only within the editor, but also in the Project tool window and the Navigation bar:



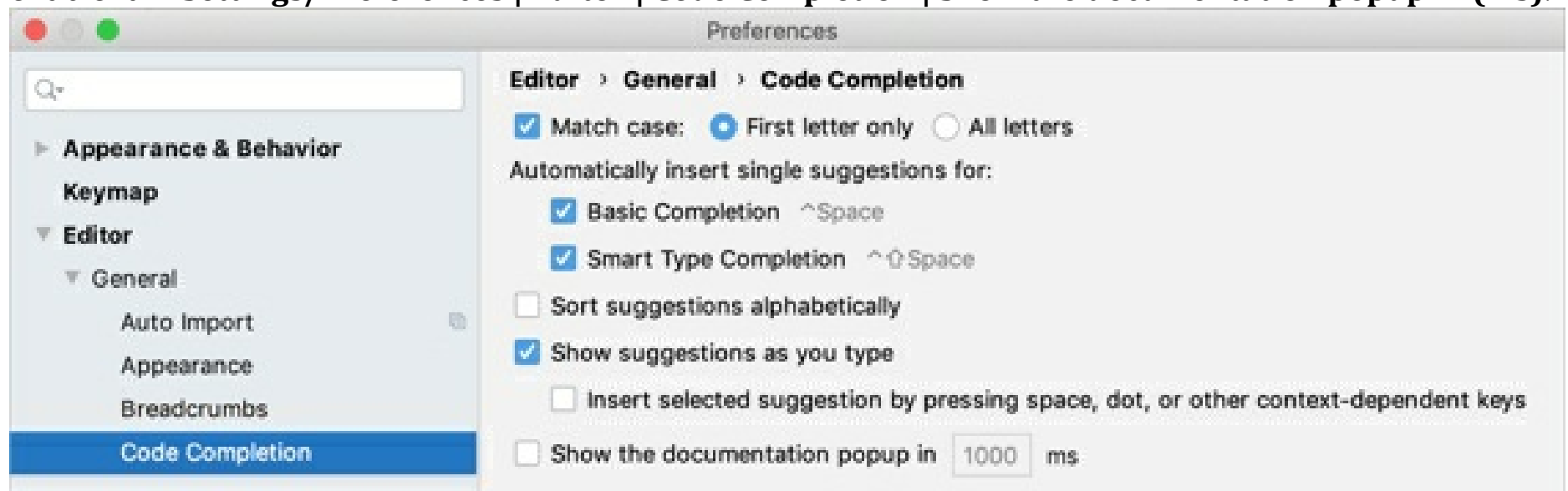
Code completion

IntelliJ IDEA provides several different types of code completion, which include:

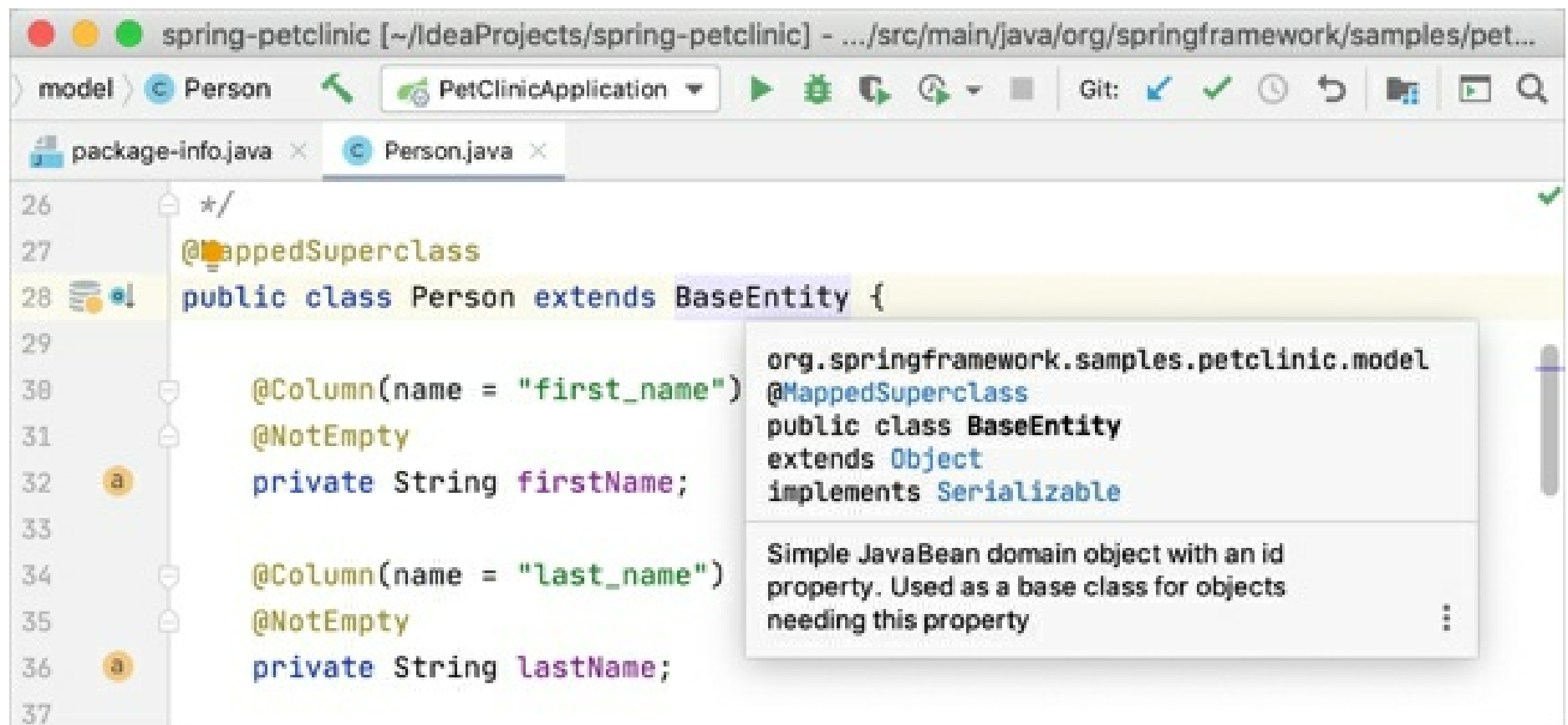
- Basic completion
- Second basic completion
- Type-matching completion
- Type-matching completion
- Statement completion

To learn more about the differences between these completion types, refer to [Top 20 Features of Code Completion in IntelliJ IDEA](#).

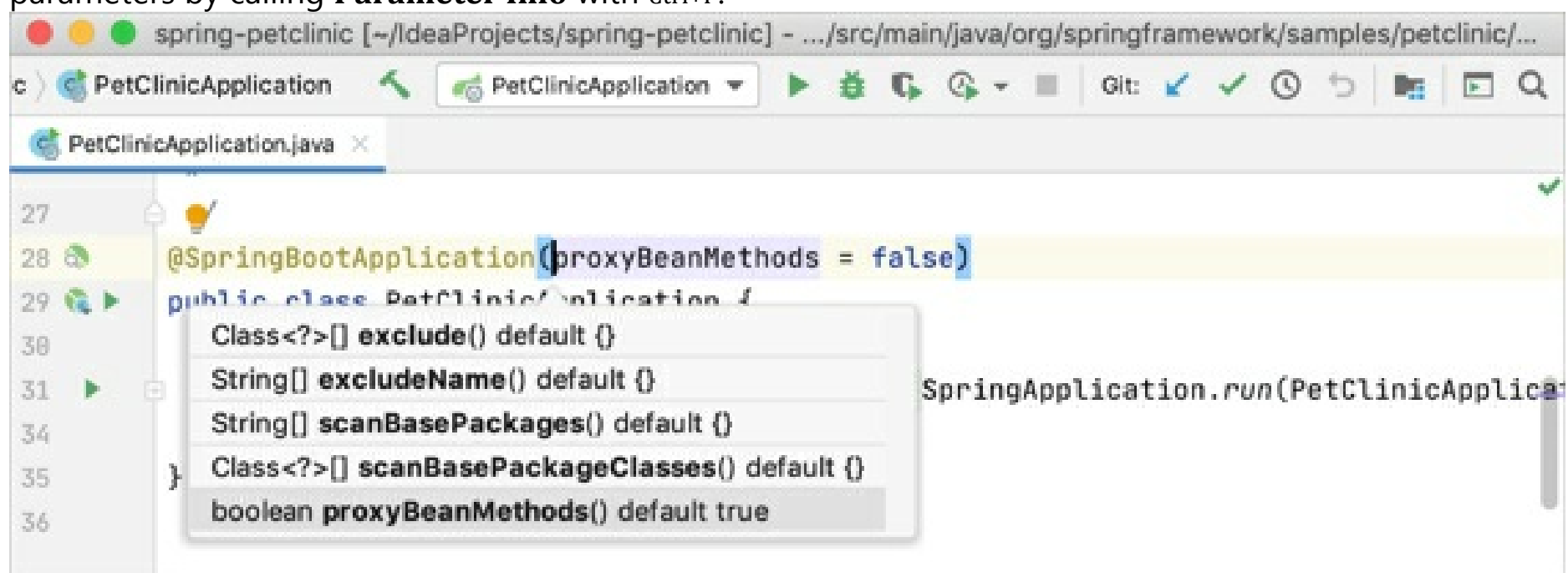
By default, *IntelliJ IDEA* doesn't show the **Documentation** popup for the selected item, but you can enable it in **Settings/Preferences | Editor | Code Completion | Show the documentation popup in (ms)**:



If you don't want to enable this option, you can manually invoke this popup by pressing `Ctrl+Q` when you need it:



When the caret is within the brackets of a method or a constructor, you can get the info about the parameters by calling **Parameter Info** with **Ctrl+P**:



Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Code completion	Ctrl+Space	Basic completion	Ctrl+Space
-	-	Smart completion	Ctrl+Shift+Space
-	-	Statement completion	Ctrl+Shift+Enter

Templates

You may be used to typing `main` in the editor and then calling code completion to have it transformed into a main method definition. However, *IntelliJ IDEA* templates are a little different:

Template	Eclipse	IntelliJ IDEA
----------	---------	---------------

Define a main method	main	psvm
Iterate over an array	for	itar
Iterate over a collection	for	itco
Iterate over a list	for	itli
Iterate over an iterable using foreach syntax	foreach	iter
Print to System.out	sysout	sout
Print to System.err	syserr	serr
Define a static field	static_final	psf

The list of available templates can be found in **Settings/Preferences | Editor | Live Templates**. There you can also add your own templates or modify any existing ones.

While *IntelliJ IDEA* suggests templates in code completion results, you can quickly expand any template without using code completion simply by pressing `Tab`.

Postfix templates

In addition to 'regular' templates, *IntelliJ IDEA* offers the so-called *postfix* templates. They are useful when you want to apply a template to an expression you've already typed. For instance, type a variable name, add `.ifn` and press `Tab`. *IntelliJ IDEA* will turn your expression into a `if (...==null){...}` statement.

To see a complete list of available postfix templates, go to **Settings/Preferences | Editor | General | Postfix Completion**.

Surround with live template

The *surround with* templates is another addition that works similarly to *live templates* but can be applied to the selected code with `Ctrl+Alt+J`.

To define your own *surround with* template, go to **Settings/Preferences | Editor | Live Templates** and use `$SELECTION$` within the template text:

```
$LOCK$.readLock().lock();
try {
    $SELECTION$
} finally {
    $LOCK$.readLock().unlock();
}
Copied!
```

Navigation

The table below roughly maps the navigation actions available in *Eclipse* with those in *IntelliJ IDEA*:

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut

Quick access	Ctrl+3	Search everywhere	Shift x 2
Open type	Ctrl+Shift+T	Navigate to class	Ctrl+N
Open resource	Ctrl+Shift+R	Navigate to file	Ctrl+Shift+N
-	-	Navigate to symbol	Ctrl+Alt+Shift+N
Quick switch editor	Ctrl+E	Switcher	Ctrl+Tab
-	-	Recent files	Ctrl+E
Open declaration	F3	Navigate to declaration	Ctrl+B
Open type hierarchy	F4	Navigate to type hierarchy	Ctrl+H
-	-	Show UML popup	Ctrl+Alt+U
Quick outline	Ctrl+O	File structure	Ctrl+F12
Back	Ctrl+[	Back	Ctrl+Alt+Left
Forward	Ctrl+]	Forward	Ctrl+Alt+Right

Later, when you get used to these navigation options and need more, refer to [Top 20 Navigation Features in IntelliJ IDEA](#).

Refactorings

The following table maps the shortcuts for the most common refactorings in *Eclipse* with those in *IntelliJ IDEA*:

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Extract local variable	Ctrl+Alt+L	Extract variable	Ctrl+Alt+V
Assign to field	Ctrl+2	Extract field	Ctrl+Alt+F

Show refactor quick menu	Alt+Shift+T	Refactor this	Ctrl+Alt+Shift+T
Rename	Ctrl+Alt+R	Rename	Shift+F6

To learn more about many additional refactorings that *IntelliJ IDEA* offers, refer to [Top 20 Refactoring Features in IntelliJ IDEA](#)

Undo

Sometimes, refactorings may affect a lot of files in a project. *IntelliJ IDEA* not only takes care of applying changes safely, but also lets you revert them. To undo the last refactoring, switch the focus to the Project tool window and press `Ctrl+Z`.

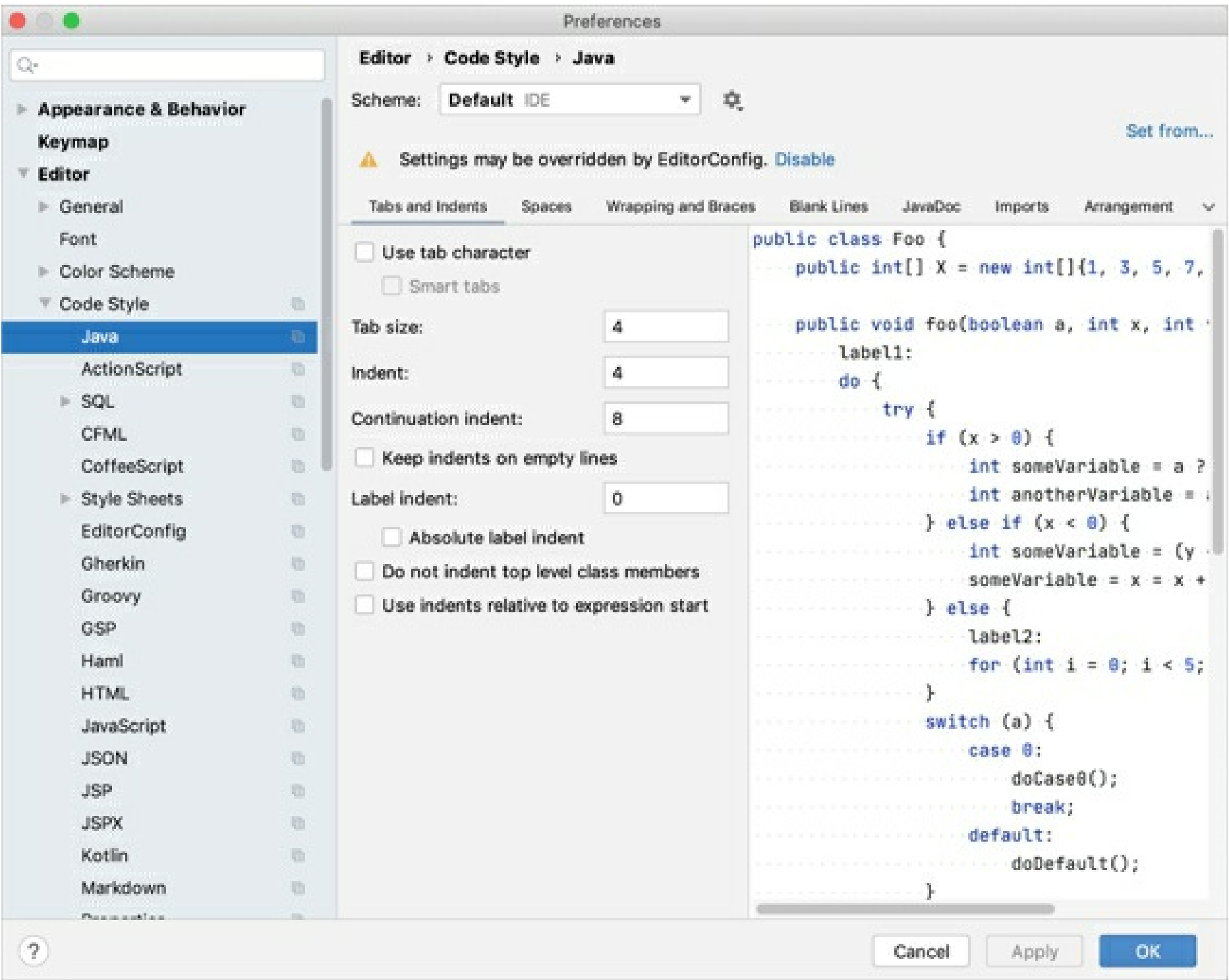
Search

Below is a map of the most common search actions and shortcuts:

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Open search dialog	Ctrl+H	Find in Files	Ctrl+Shift+F
References in workspace	Ctrl+Shift+G	Find usages	Alt+F7
-	-	Show usages	Ctrl+Alt+F7
-	-	Find usages settings	Ctrl+Alt+Shift+F7
Occurrences in file	Alt+Ctrl+U	Highlight usages in file	Ctrl+F7

Code formatting

IntelliJ IDEA code formatting rules (available via **Settings/Preferences | Editor | Code Style**) are similar to those in *Eclipse*, with some minor differences. You may want to take note of the fact that the **Use tab character** option is disabled by default, the **Indent size** may be different, etc.



If you would like to import your *Eclipse* formatter settings, go to **Settings/Preferences | Editor | Code Style | Java**, click **Import Scheme** and select the exported *Eclipse* formatter settings (an XML file). Note that if the Eclipse formatter settings cannot be imported, the following error message shows up: *The input file is not a valid Eclipse XML profile*.

Note that there may be some discrepancies between the code style settings in *IntelliJ IDEA* and *Eclipse*. For example, you cannot tell *IntelliJ IDEA* to put space after (but not before). If you want *IntelliJ IDEA* to use the *Eclipse* formatter, consider installing the Eclipse code formatter plugin.

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Format	Ctrl+Shift+F	Reformat code	Ctrl+Alt+L

Running and reloading changes

Similarly to *Eclipse*, *IntelliJ IDEA* also has Run/debug configurations dialog that you can access either from the main toolbar, or the main menu. Compare the related shortcuts:

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut

Run	Ctrl+Shift+F11	Run	Shift+F10
Debug	Ctrl+F11	Debug	Shift+F9
-	-	Make	Ctrl+F9
-	-	Update application	Ctrl+F10

As mentioned before, by default *IntelliJ IDEA* doesn't compile changed files automatically (unless you configure it to do so). That means the IDE doesn't reload changes automatically. To reload changed classes, call the **Build** action explicitly via `Ctrl+F9`. If your application is running on a server, in addition to reloading you can use the **Update application** action via `Ctrl+F10`:

Debugging

The debuggers in *Eclipse* and *IntelliJ IDEA* are similar but use different shortcuts:

Eclipse		IntelliJ IDEA	
Action	Shortcut	Action	Shortcut
Step into	F5	Step into	F7
-	-	Smart step into	Shift+F7
Step over	F6	Step over	F8
Step out	F7	Step out	Shift+F8
Resume	F8	Resume	F9
Toggle breakpoint	Ctrl+Shift+B	Toggle breakpoint	Ctrl+F8
-	-	Evaluate expression	Alt+F8

Working with Application Servers (Tomcat/TomEE, Glassfish, WebLogic, WebSphere)^{Ultimate}

Deploying to application servers in *IntelliJ IDEA* is more or less similar to what you are probably used to in *Eclipse*. To deploy your application to a server:

1. Configure your artifacts via **Project Structure | Artifacts** (done automatically for *Maven* and *Gradle* projects).

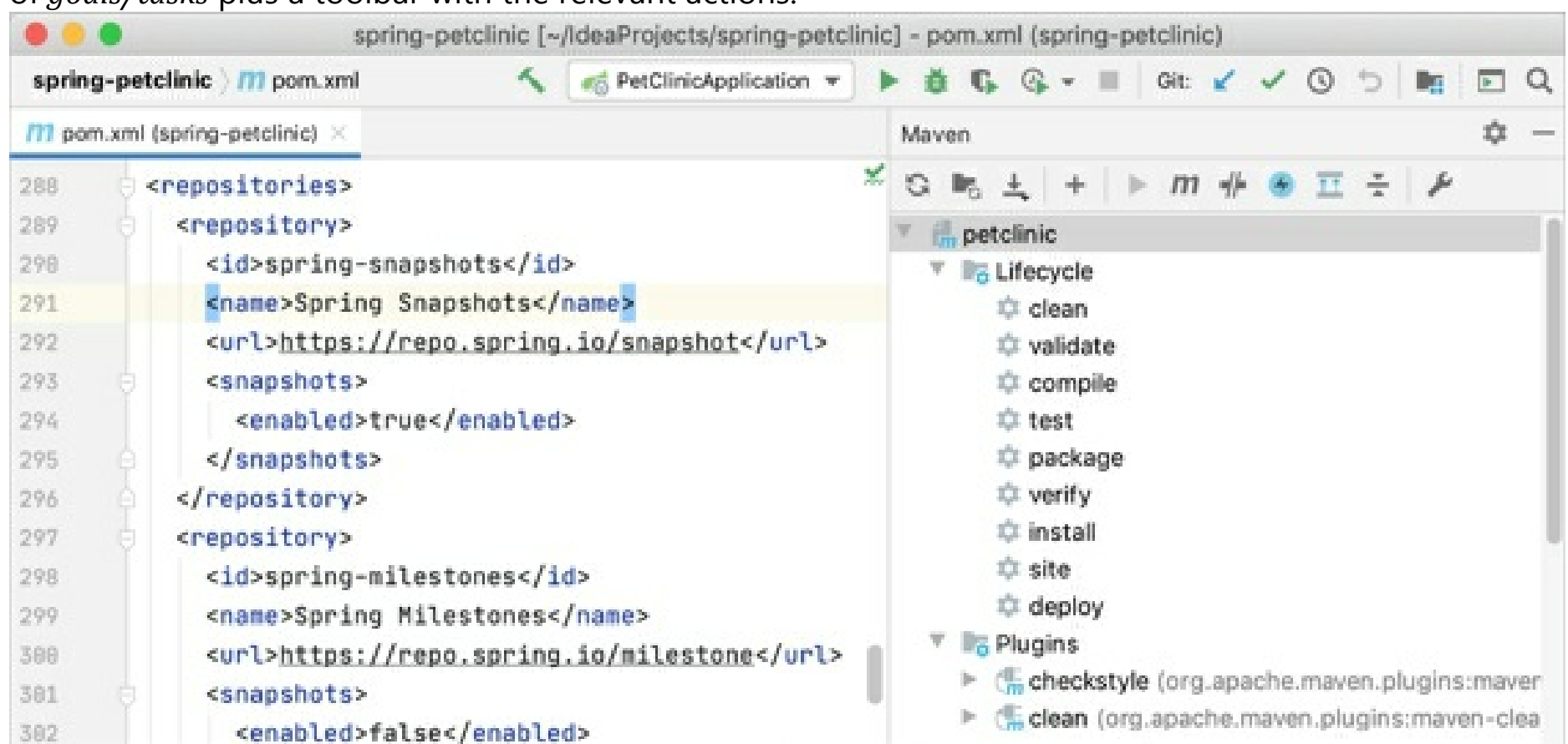
2. Configure an application server via **Settings/Preferences | Application Servers**.
3. Create a run configuration, and then specify the artifacts to deploy and the server to deploy to.

You can always tell the IDE to build/rebuild your artifacts once they have been configured via **Build | Build Artifacts**.

Working with Build Tools (Maven/Gradle)

IntelliJ IDEA doesn't provide visual forms for editing *Maven/Gradle* configuration files. Once you've imported/created your *Maven/Gradle* project, you are free to edit its **pom.xml/build.gradle** files directly in the editor. Later, you can tell *IntelliJ IDEA* to synchronize the project model with the changed files on demand, or automatically import changes to the new build files. Any changes to the underlying build configuration will eventually need to be synced with the project model in *IntelliJ IDEA*.

For operations specific to *Maven/Gradle*, *IntelliJ IDEA* provides the Maven Project tool window and the Gradle tool window. Apart from your project structure, these tool windows provide a list of *goals/tasks* plus a toolbar with the relevant actions.

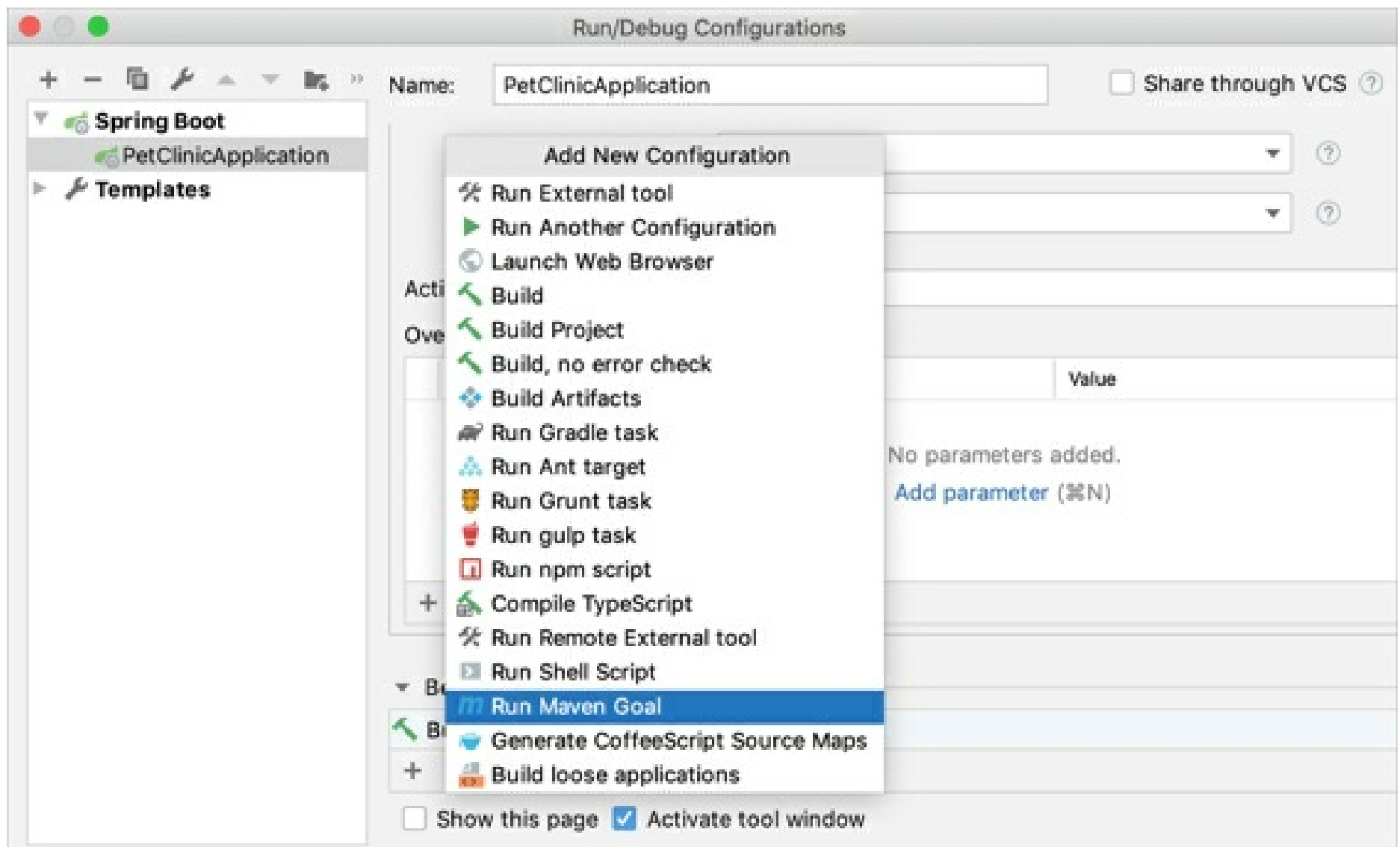


For manual synchronization, use the corresponding action on the *Maven/Gradle* tool window toolbar: .

Running goals/tasks

Use the **Maven/Gradle** tool window to run any project *goal/task*. When you do, *IntelliJ IDEA* creates the corresponding run configuration which you can reuse later to run the *goal/task* quickly.

It's worth mentioning that any *goal/task* can be attached to be run before a *Run Configuration*. This may be useful when your *goal/task* generates specific files needed by the application.



Both the *Maven* and *Gradle* tool windows provide the **Run Task** action. It runs a *Maven/Gradle* command similarly to how you'd run it using the console.

Configuring artifacts

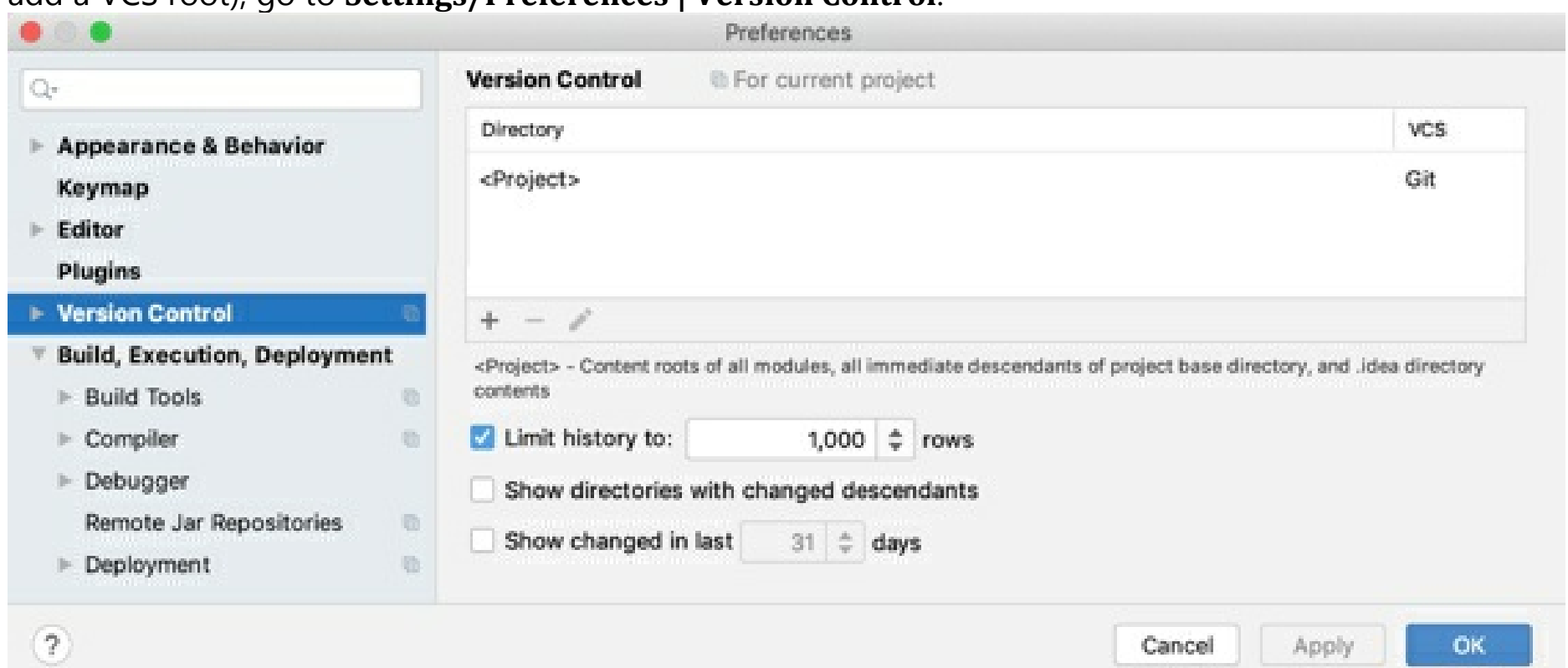
If you have *WAR artifacts* configured in your **pom.xml/build.gradle** file, *IntelliJ IDEA* automatically configures the corresponding artifacts in **Project Structure | Artifacts**.

Note that when you compile your project or build an artifact, *IntelliJ IDEA* uses its own build process which may be faster, but is not guaranteed to be 100% accurate. If you notice inconsistent results when compiling your project with **Build** in *IntelliJ IDEA*, try using a *Maven goal* or a *Gradle task* instead.

Working with VCS (Git, Mercurial, Subversion, Perforce)

Configuring VCS roots

When you open a project located under a VCS root, *IntelliJ IDEA* automatically detects it and suggests adding this root to the project settings. To change version control-related project settings (or manually add a VCS root), go to **Settings/Preferences | Version Control**:

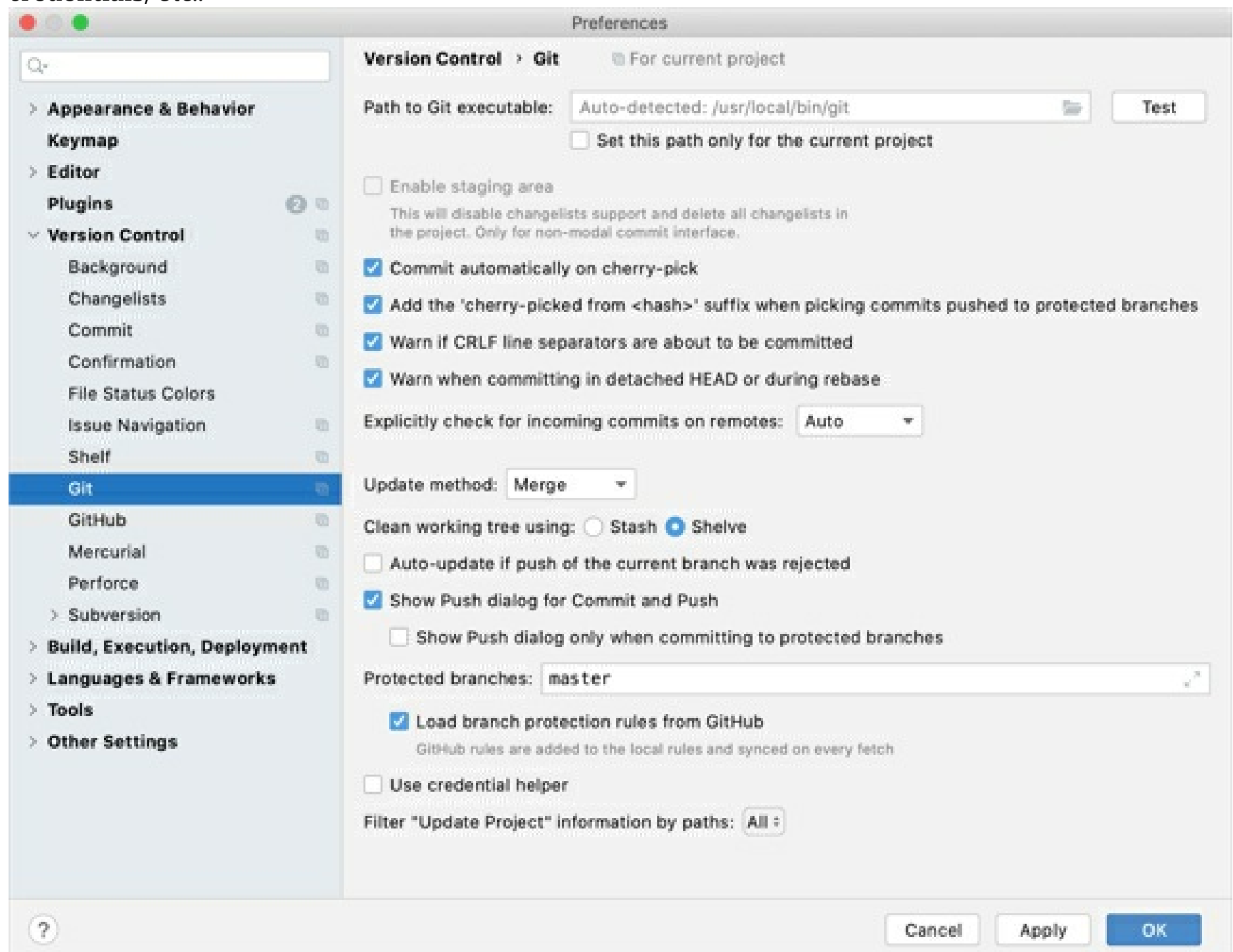


IntelliJ IDEA works perfectly with multi-repository projects. Just map your project directories to VCS, and

the IDE will take care of the rest. For Git and Mercurial, the IDE will even offer you synchronized branch control, so that you can perform branch operations on multiple repositories simultaneously (for more details, see Manage Git branches).

Editing VCS settings

Every VCS may require specific settings, for example, **Path to Git executable**, **GitHub/Perforce credentials**, etc.:



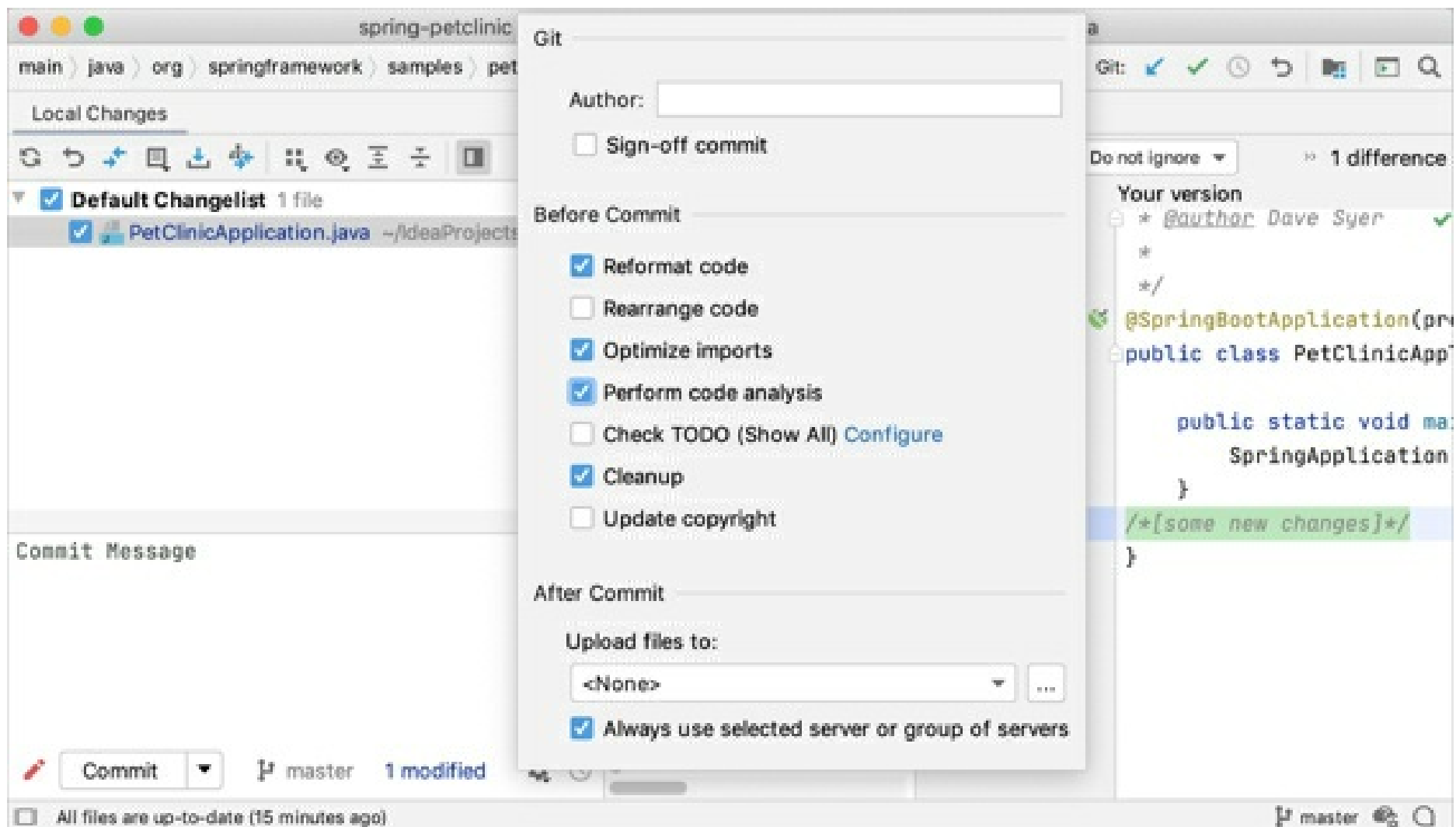
Once you've configured the VCS settings, you'll see the **Version Control** tool window **Alt+9**.

Checking projects out

To check out a project from a VCS, click **Get from Version Control** on the Welcome Screen, or in the main **VCS** menu.

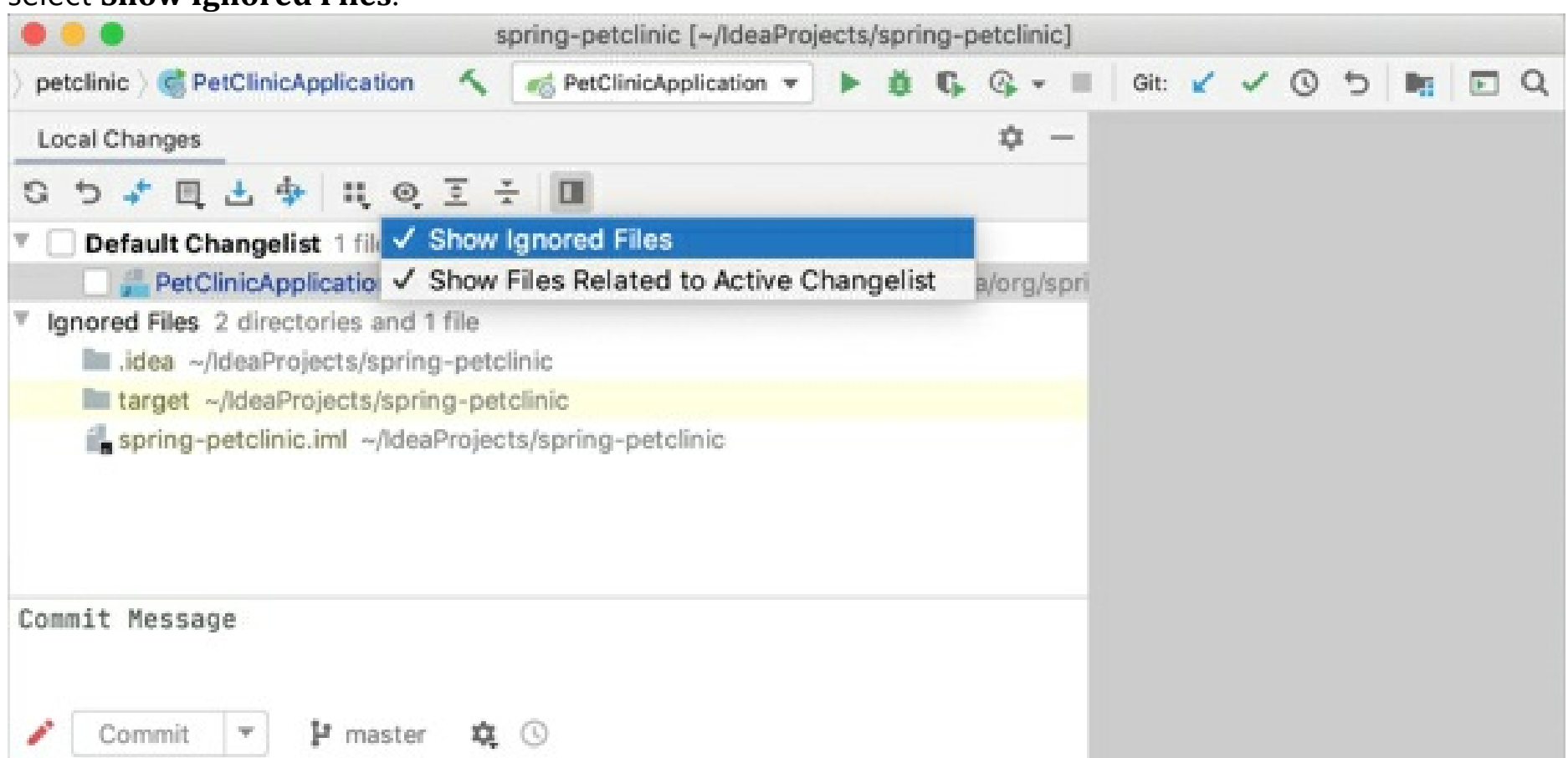
Working with local changes

The **Local Changes** view shows your local changes: both *staged* and *unstaged*. To simplify managing changes, all changes are organized into *changelists*. Any changes made to source files are automatically included into the active changelist. You can create new changelists, delete the existing ones (except for the **Default** changelist), and move files between changelists.



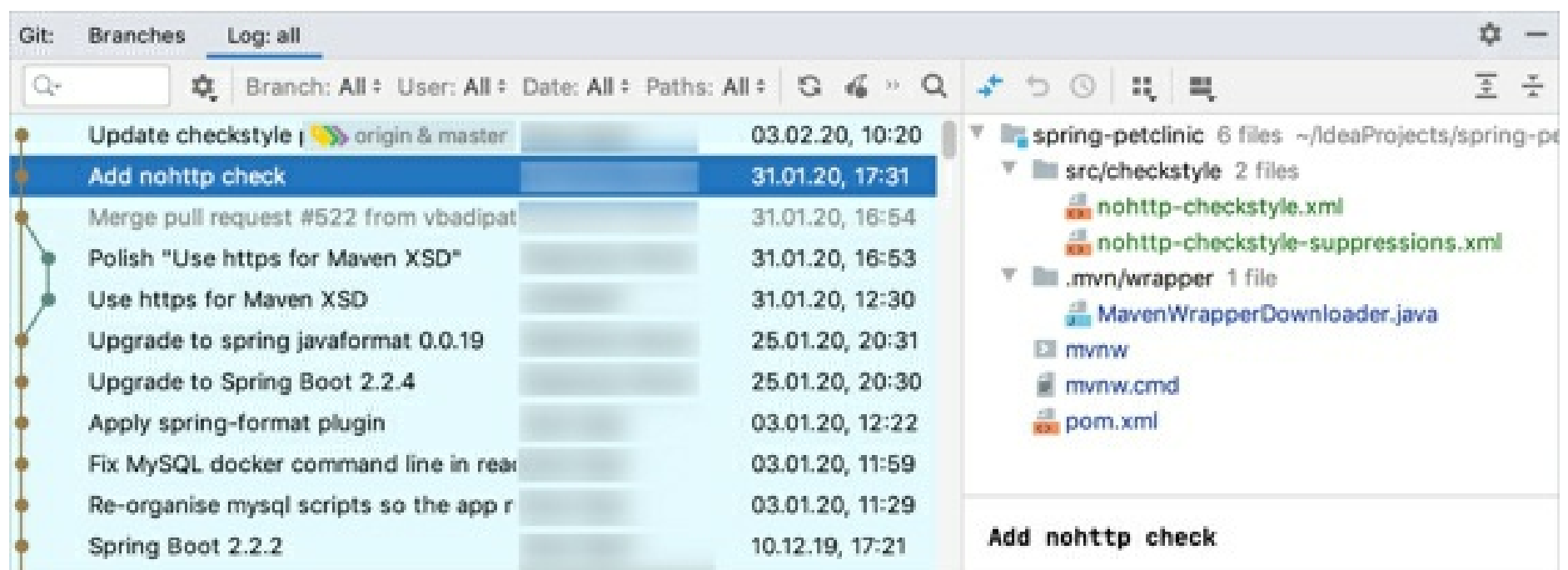
Right-click the unversioned file or folder you want to ignore in the **Local Changes** tab of the **Version Control** tool window **Alt+9** or in **Project** tool window and select **Git | Add to .gitignore** or **Git | Add to .git/info/exclude**.

If you want ignored files to be also displayed in the **Local Changes** view, click on the toolbar and select **Show Ignored Files**.



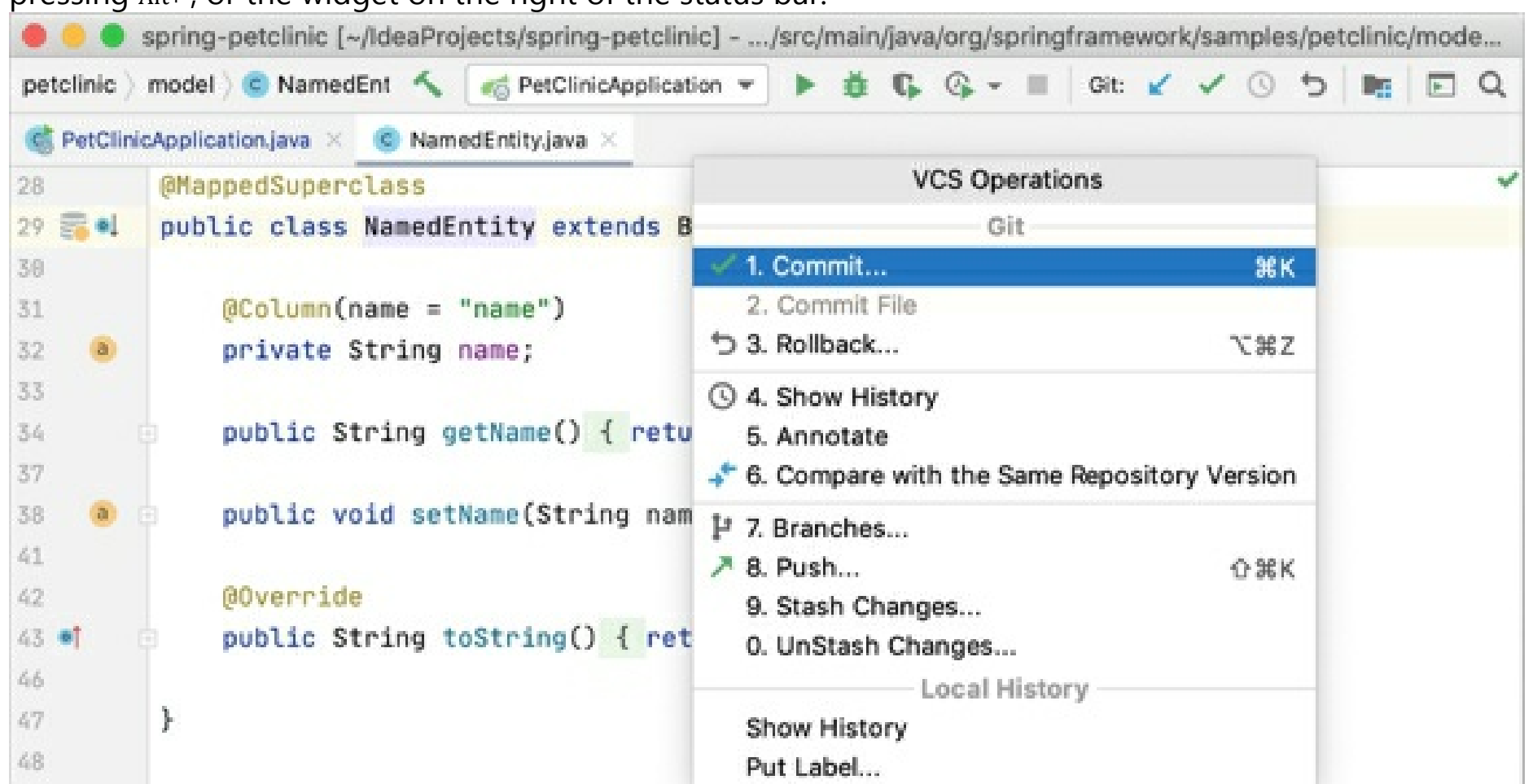
Working with history

The **Log** tab of the **Git** tool window lets you see and search through the history of commits. You can sort and filter commits by the repository, branch, user, date, folder, or even a phrase in the description. You can find a particular commit, or just browse through the history and the branch tree:



Working with branches

IntelliJ IDEA lets you create, switch, merge, compare and delete branches. For these operations, either use **Branches** from the main or context **VCS** menu, or the **VCS operations popup** (you can invoke it by pressing **Alt+`**), or the widget on the right of the status bar:



All VCS operations are available from the VCS main menu:

Action	Shortcut
Version Control tool window	Alt+9
VCS operations popup	Alt+`
Commit changes	Ctrl+K
Update project	Ctrl+T

Importing an Eclipse project to IntelliJ IDEA

Despite these differences in terms and the UI, you can import either an *Eclipse* workspace or a single *Eclipse* project. To do this, click **Open** on the **Welcome Screen**, or select **File | Open** in the main menu.

If your project uses a build tool such as Maven or Gradle, we recommend selecting the associated build file **pom.xml** or **build.gradle** when importing the project. For more information on how to import a project, refer to [Import a project from Eclipse](#).

If you'd like to import your existing run configurations from *Eclipse*, consider using this third-party plugin.

Configuring PHP development environment

What configuration is needed before start?

A lot of IntelliJ IDEA features are available without any configuration right after you launch it. Still, to take full advantage of running your PHP application, you need to configure a PHP interpreter and a server.

If you plan to launch the application locally, you need a PHP engine installed and registered in IntelliJ IDEA, as well as a Web server installed, configured, and integrated with IntelliJ IDEA. You can install these components separately or use an AMP package. For more details about initial environment configuration, refer to [Configure PHP development environment](#).

If you are going to run and debug an application directly on a remote host, the only thing you need is register access to this host in IntelliJ IDEA to enable synchronization.

How do I start with deployment to a remote host?

If you've checked out your project from the remote host, the deployment server is already configured. Otherwise, you will need to get it configured (it can be FTP/SFTP/FTPS server or mounted/local folder) on the Deployment page of the **Settings/Preferences** dialog. The Remote host tool window is available on the right-hand side of the IntelliJ IDEA window, which can be handy for browsing through your remote server and performing various actions.

See [Deploy your application](#) for details.

How do I start debugging?

IntelliJ IDEA comes with support for both Xdebug and Zend Debugger for debugging and profiling. There is a zero-configuration debugging workflow available, which means that to start debugging you only need to:

- Click **Start Listening for PHP Debugging Connections** on the toolbar of the IDE.
- Place a breakpoint in code by clicking in the editor gutter next to the line.
- Start debugging in the browser using a plugin or browser bookmarklets.

Import a project from Eclipse

Import a project to IntelliJ IDEA

1. Launch IntelliJ IDEA.

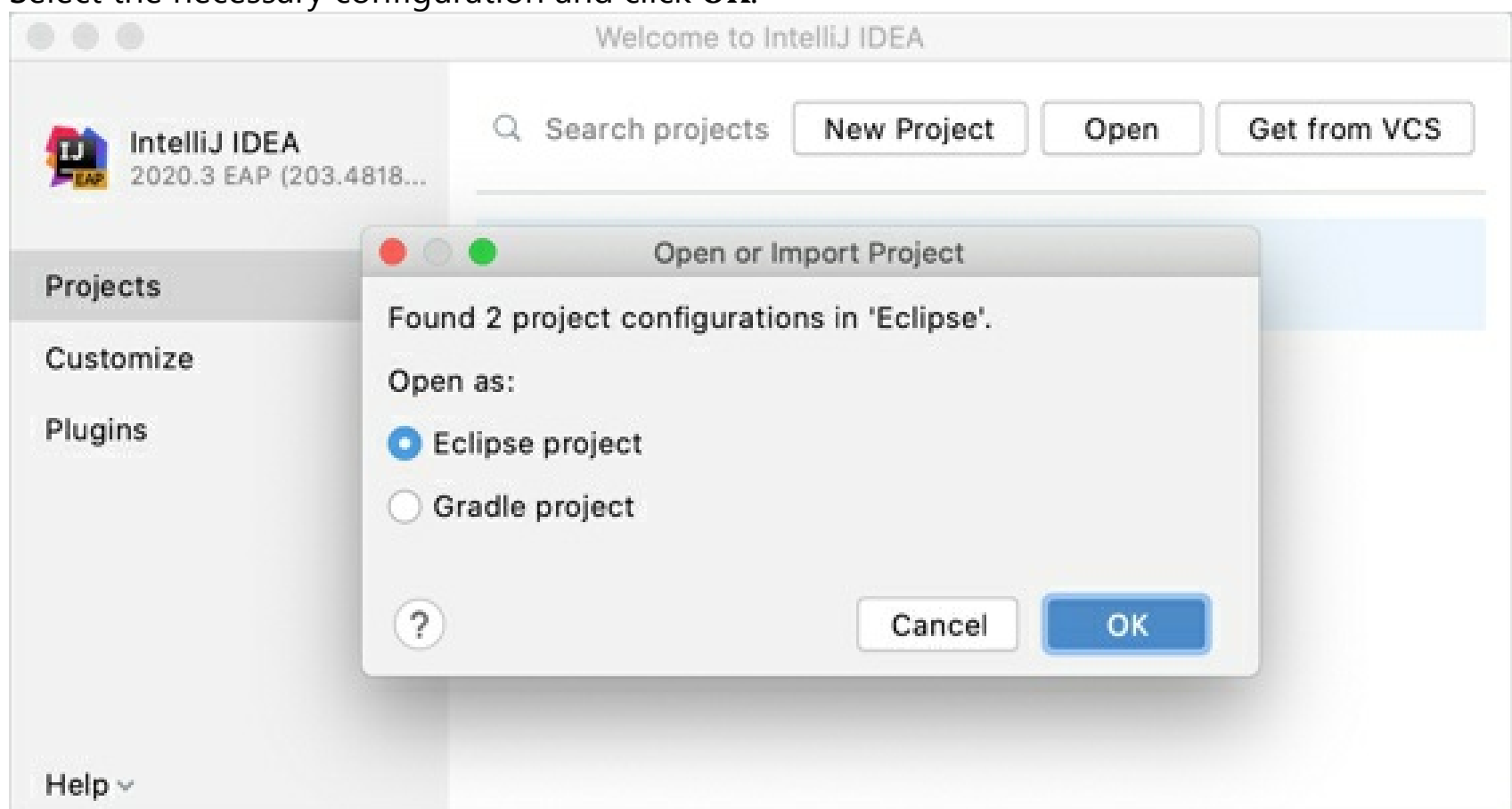
If the Welcome screen opens, click **Open**.

Otherwise, from the main menu, select **File | Open**.

2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. When you import or clone a project for the first time, IntelliJ IDEA analyzes it. If the IDE detects more than one configuration (for example, Eclipse and Gradle), it prompts you to select which configuration you want to use.

If the project that you are importing uses a build tool, such as Maven or Gradle, we recommend that you select the build tool configuration.

Select the necessary configuration and click **OK**.



The IDE pre-configures the project according to your choice. For example, if you select **Gradle project**, IntelliJ IDEA executes its build scripts, loads dependencies, and so on.

4. If you have been working with another project, select whether you want to open the new project in a new dialog or in the current one.

Import a project with settings

Use this type of import if your project comes from an external model and you want to import it as a whole. In this case, your Eclipse project will be migrated to IntelliJ IDEA.

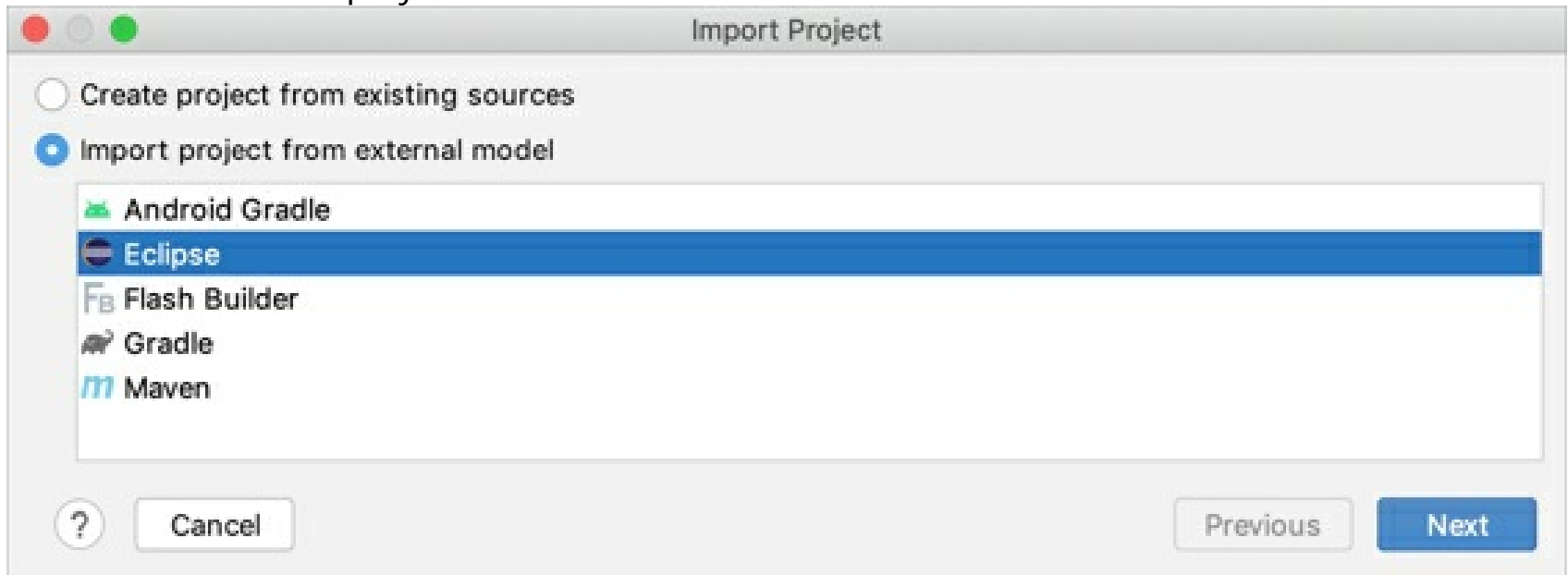
1. Launch IntelliJ IDEA.

If the Welcome screen opens, press **Ctrl+Shift+A**, type **project from existing sources**, and click the **Import project from existing sources** action in the popup.

Otherwise, from the main menu, select **File | New | Project from Existing Sources**.

2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. Select **Import project from external model | Eclipse** and click **Next**.

If the project that you are importing uses a build tool, we recommend that you import it as a Maven or a Gradle project.



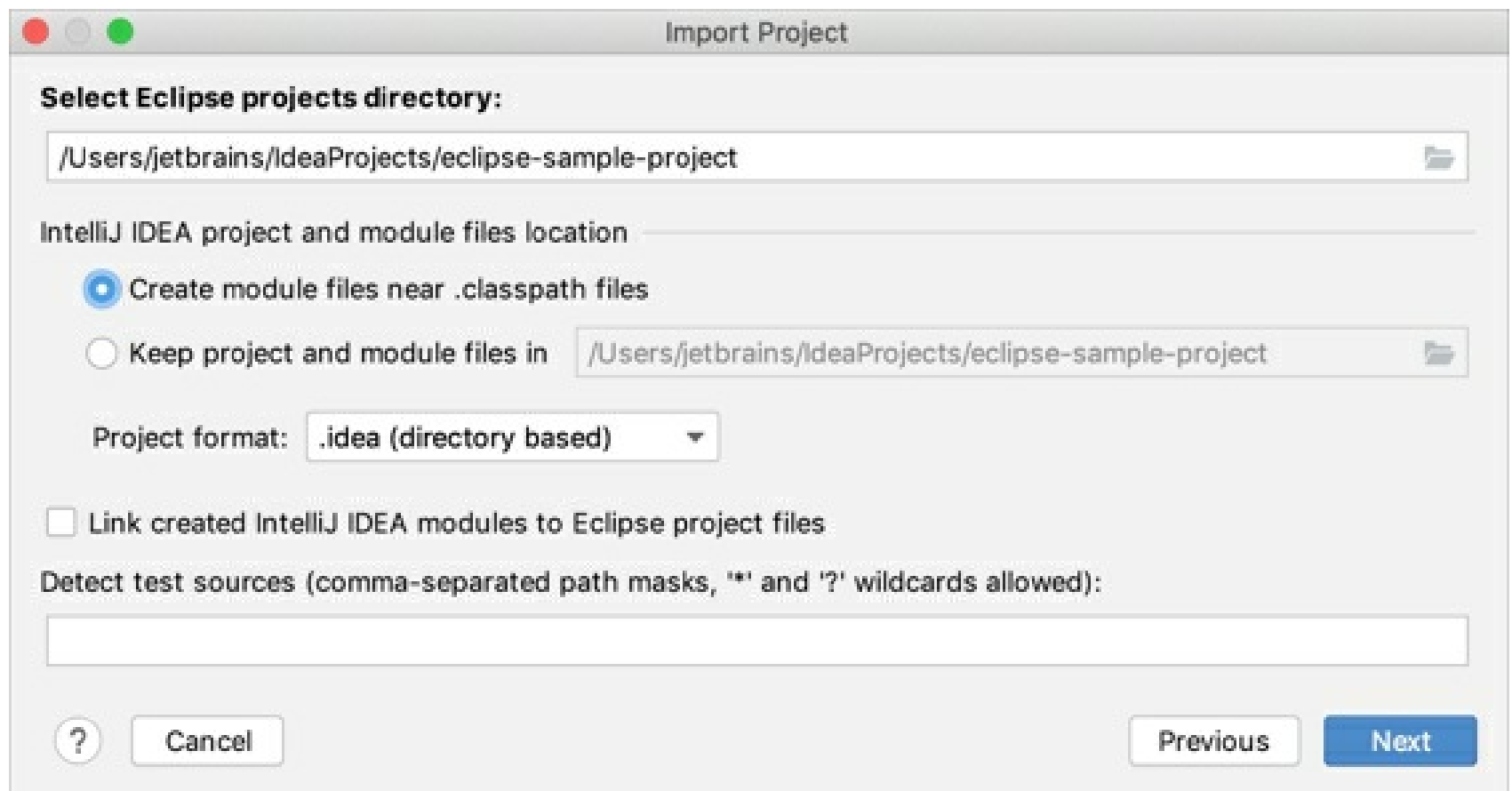
4. Configure the project:

- **Select Eclipse projects directory:** the path to the Eclipse workspace that contains projects that you want to import.
- **Create module files near .classpath files:** create a new IntelliJ IDEA module per each Eclipse project the respective project directories. The IntelliJ IDEA project in the specified format will be created in the root of the Eclipse workspace, or Eclipse project directory.
- **Keep project and module files in:** if you want to import your project to IntelliJ IDEA without modifying the original Eclipse project, specify the folder in which the IDE will create the **.iml** file (module file) and the **.idea** directory with configuration files.

If you leave the default path, IntelliJ IDEA creates the **.iml** file and **.idea** directory in the Eclipse project folder.

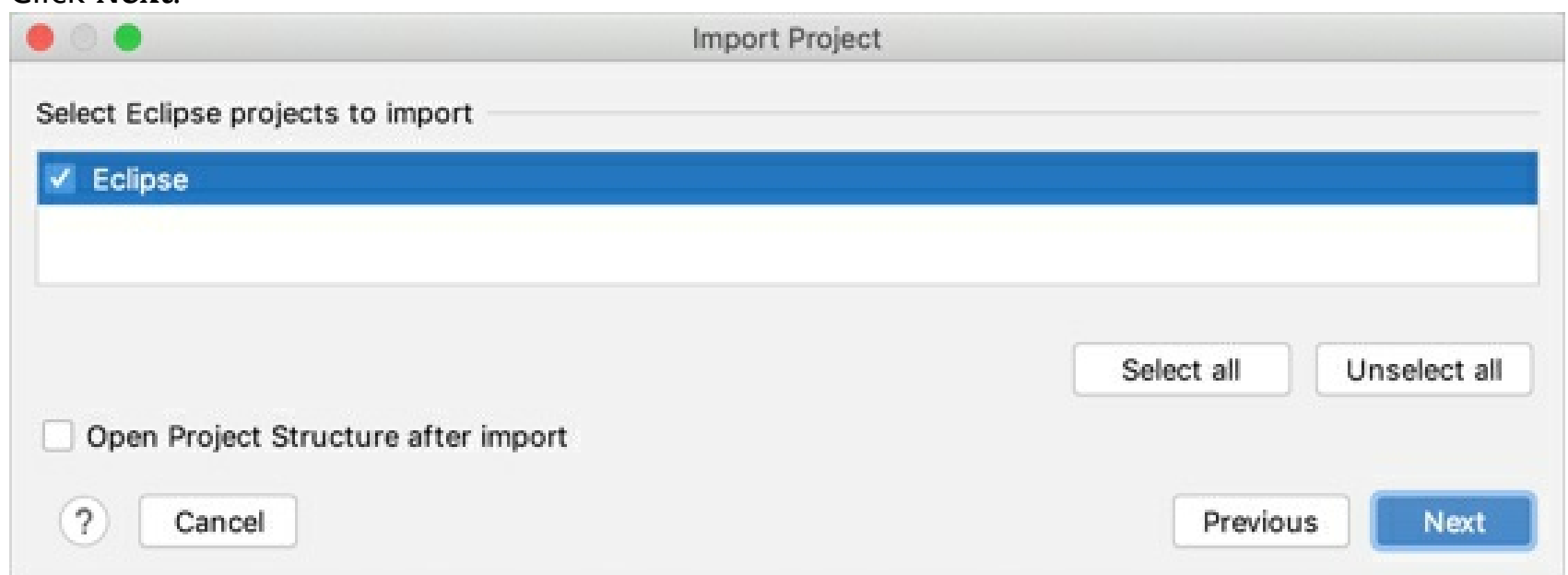
- **Project file format:** it's recommended that you use the directory-based format.
- **Link created IntelliJ IDEA modules to Eclipse project files:** automatically keep the Eclipse projects and IntelliJ IDEA modules synchronized.
- **Detect test sources:** specify the list of folders in which test sources are stored. Refer to the Compiler section for the wildcards syntax.

Click **Next**.



5. Review the projects detected in the selected Eclipse workspace and select the ones that you want to import to IntelliJ IDEA. Each Eclipse project will be converted into an IntelliJ IDEA module.

Click **Next**.



6. Select the code style configuration that you want to use in the project and click **Next**.
7. Specify the SDK that you want to use.

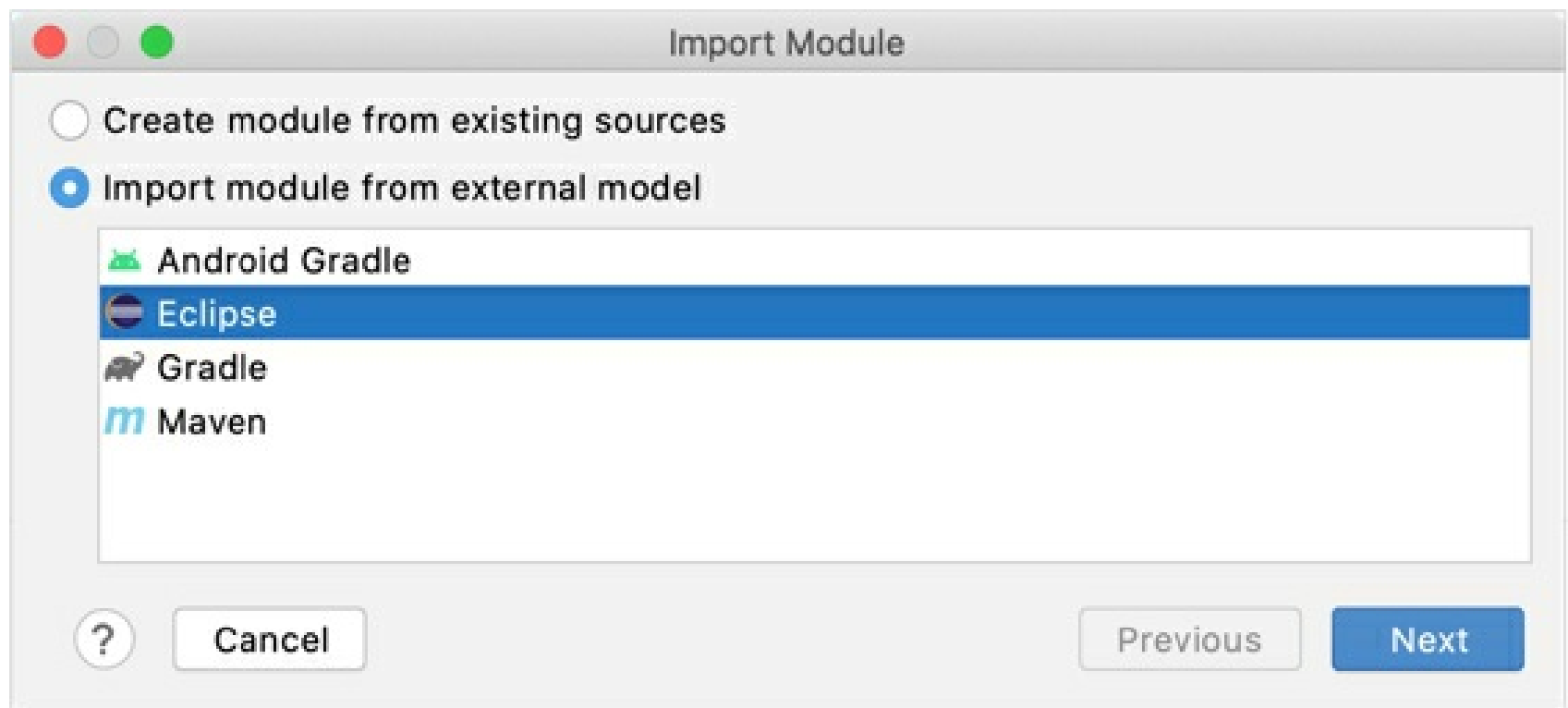
If the necessary SDK is already defined in IntelliJ IDEA, select it from the list on the left. Otherwise, click **+** and add a new SDK.

8. Click **Finish**.

Import an Eclipse project as a module

1. From the main menu, select **File | New | Module from Existing Sources**.
2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. Select **Import module from external model | Eclipse** and click **Next**.

If the project that you are importing uses a build tool, we recommend that you import it as a Maven or a Gradle module.



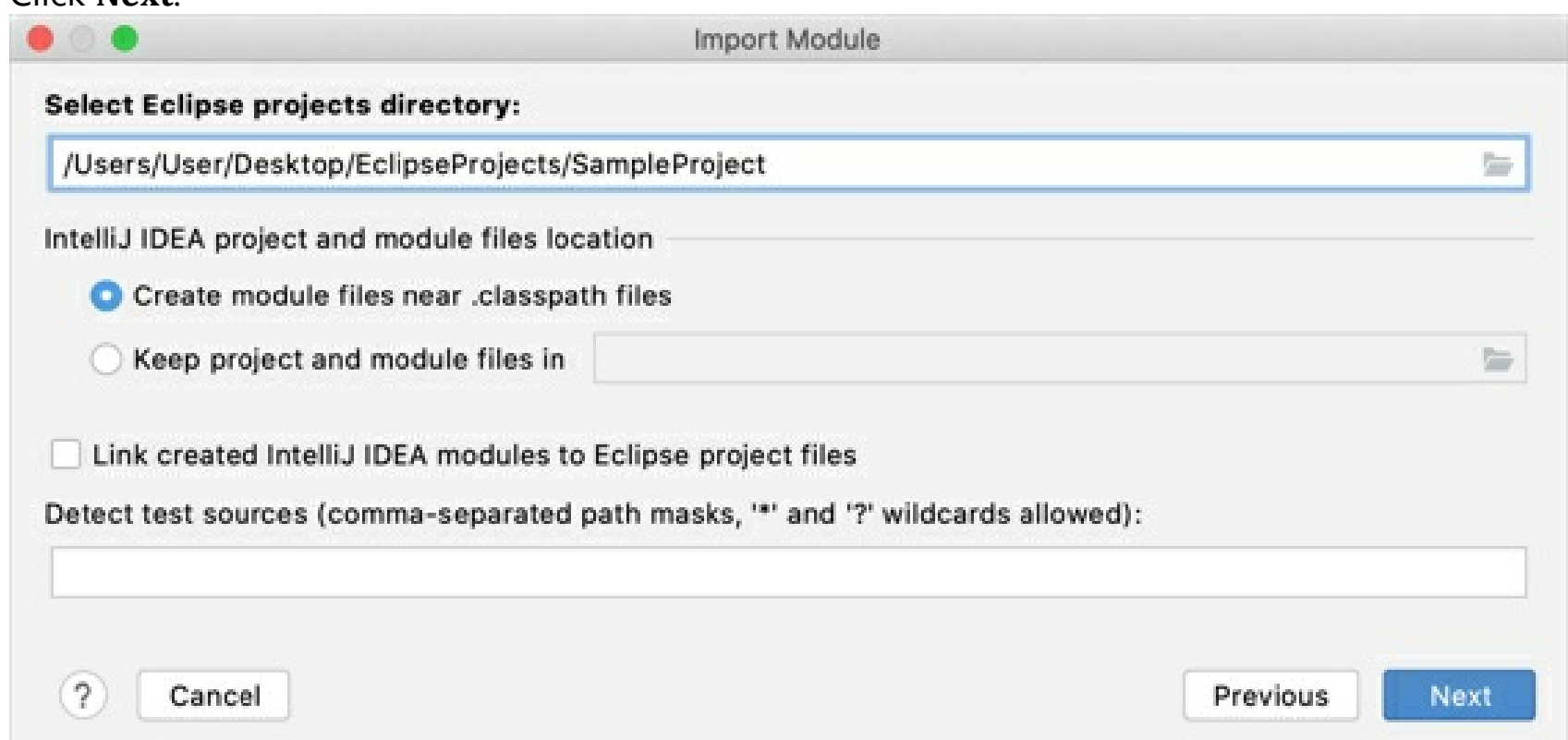
4. Configure the module:

- **Select Eclipse projects directory:** the path to the Eclipse workspace that contains projects that you want to import.
- **Create module files near .classpath files:** create a new IntelliJ IDEA module per each Eclipse project the respective project directories. The IntelliJ IDEA project in the specified format will be created in the root of the Eclipse workspace, or Eclipse project directory.
- **Keep project and module files in:** if you want to import your project to IntelliJ IDEA without modifying the original Eclipse project, specify the folder in which the IDE will create the **.iml** file (module file) and the **.idea** directory with configuration files.

If you leave the default path, IntelliJ IDEA creates the **.iml** file and **.idea** directory in the Eclipse project folder.

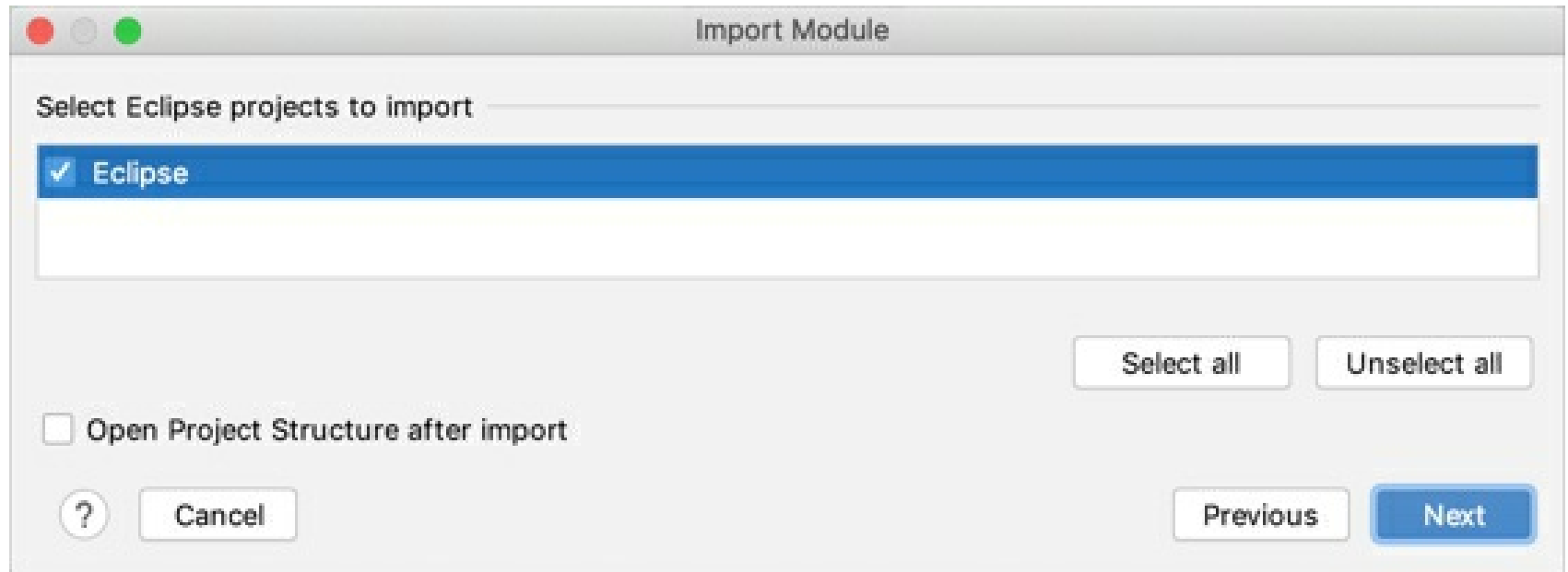
- **Link created IntelliJ IDEA modules to Eclipse project files:** automatically keep the Eclipse projects and IntelliJ IDEA modules synchronized.
- **Detect test sources:** specify the list of folders in which test sources are stored. Refer to the Compiler section for the wildcards syntax.

Click **Next**.



5. Review the projects detected in the selected Eclipse workspace and select the ones that you want to import to IntelliJ IDEA. Each Eclipse project will be converted into an IntelliJ IDEA module.

Click **Next**.



6. Select the code style configuration that you want to use in the project and click **Next**.
7. Click **Finish**.

The project will appear in the **Project** tool window as a module.

Export an IntelliJ IDEA project to Eclipse

Before you start exporting a project, make sure that the Eclipse Integration plugin is enabled.

At present, migration from IntelliJ IDEA to Eclipse is subject to certain limitations:

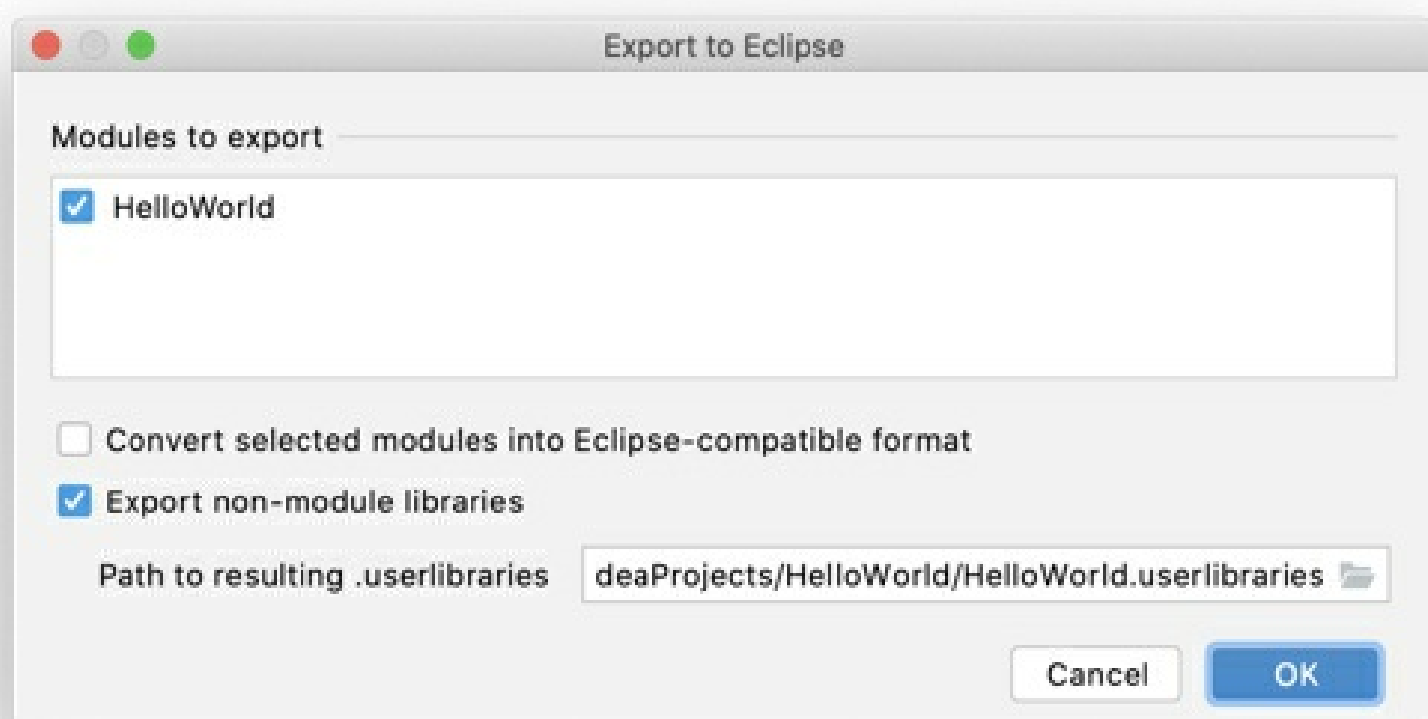
- IntelliJ IDEA modules with multiple content roots cannot not be migrated.
- External sources in Eclipse are not migrated.
- Only Java modules are created automatically during the export. However, any module can be converted to the Eclipse compatible format manually.
- The default workspace JRE is not converted to/from the project SDK.

Export a project to Eclipse

When you export an IntelliJ IDEA project to Eclipse, it results in creating Eclipse project files **.project** and **.classpath** for each module file **.iml** in the module directory that contains the content root.

1. From the main menu, select **File | Export | Project to Eclipse**.

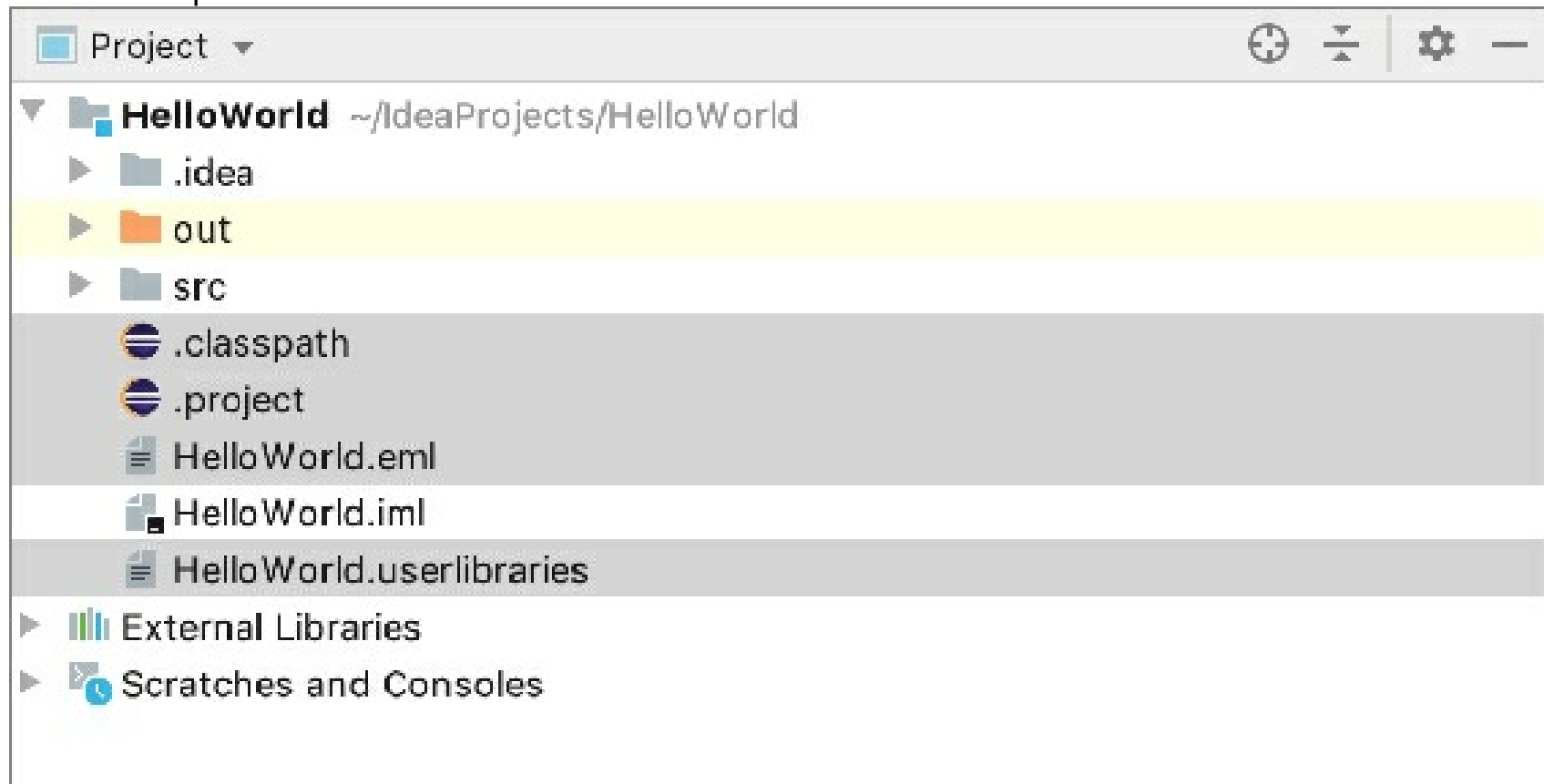
The **Export to Eclipse** dialog displays the list of modules that have not been converted and switched to use the Eclipse format yet (the modules that have the IntelliJ IDEA module format **.iml**).



2. Select the modules you want to export.
3. Select the suggested options if necessary:
 - **Convert selected modules into Eclipse-compatible format:** convert the modules selected in the **Modules to export** to the Eclipse-compatible format.
 - **Export non-module libraries:** IntelliJ IDEA creates a User Library configuration file for Eclipse ***.userlibraries**, containing definitions of all external libraries used in the project.
 - **Path to resulting .userlibraries:** the path to the generated ***.userlibraries** file. This field is available when the **Export non-module libraries** option is selected.
4. Click **OK**.

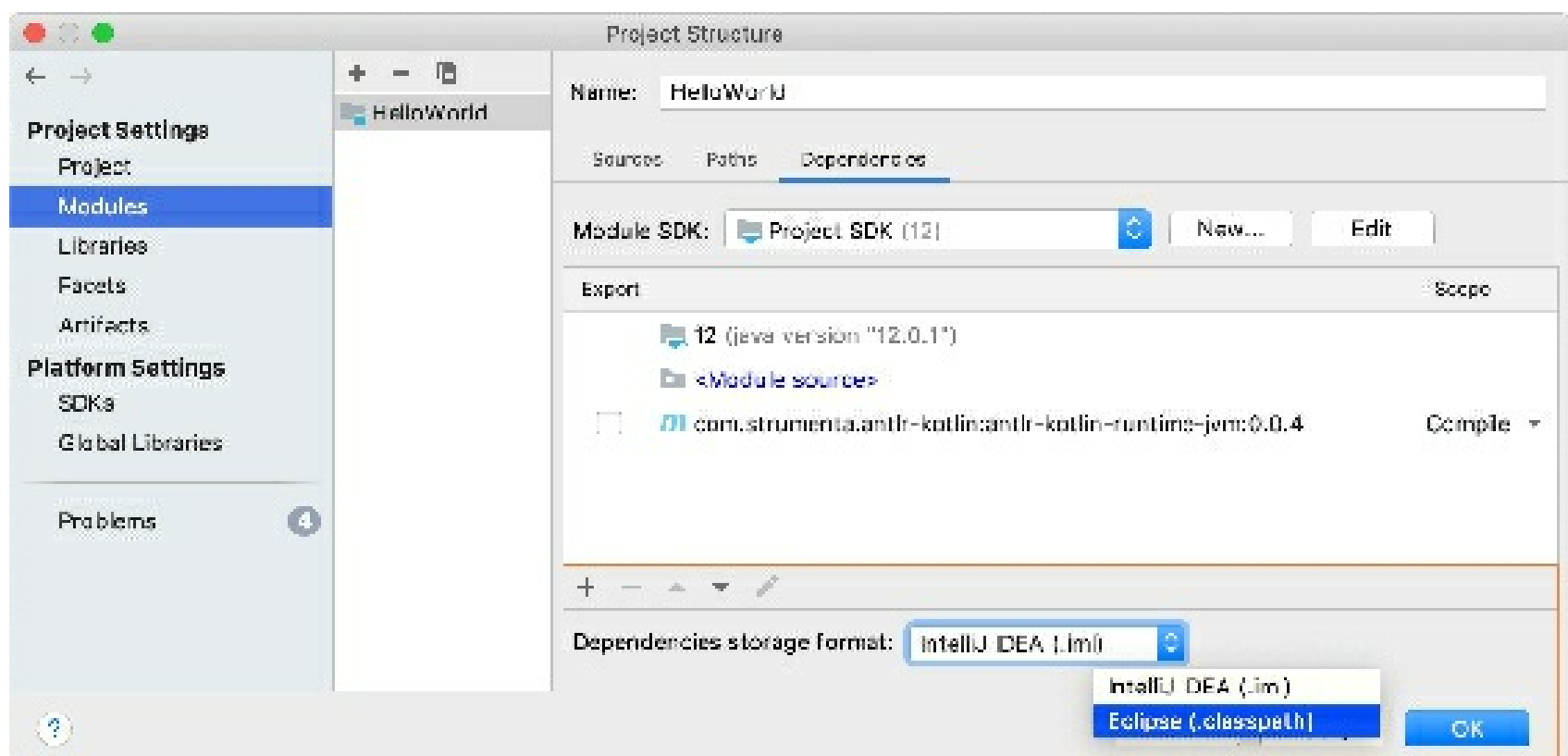
The project is exported to the same directory. The Project tool window displays the newly

created Eclipse files.



Convert a module to the Eclipse-compatible format

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` or click on the toolbar.
2. Select the module you want to convert and switch to the **Dependencies** tab.
3. From the **Dependencies storage format** list, select **Eclipse (.classpath)**.
4. Apply the changes and close the dialog.



NetBeans

Working with projects

How do I open a NetBeans project in IntelliJ IDEA?

Use **File | New | Project from Existing Sources** and select your NetBeans project directory. When the **Import Project** wizard opens, select the **Create project from existing sources** option and then follow the instructions of the wizard. IntelliJ IDEA will add the necessary definition files (the **.idea** directory) to your project directory. The NetBeans **.nbproject** directory and **build.xml** will remain untouched, and you'll be able to use IntelliJ IDEA along with NetBeans. During the import IntelliJ IDEA will fix missing libraries, add facets for different Web frameworks and create a run configuration.

If you are using Maven with NetBeans, and you want to import a Maven project into IntelliJ IDEA, select **File | Open** and then select your project's **pom.xml**. You'll still need to configure a run configuration, however, all project dependencies will get resolved.

What's the difference between projects and modules?

IntelliJ IDEA creates a project for an entire code base and a module for each of its individual components. So, IntelliJ IDEA module is more like a NetBeans project. The following table maps the most important NetBeans concepts to IntelliJ IDEA ones.

NetBeans	IntelliJ IDEA
Project	Module
Global library	Global library
Project library	Module library
Project dependency	Module dependency

Is there a directory-based project format in IntelliJ IDEA?

Yes, there is a **.idea** directory where project definition XML files are stored. For more information, see [Projects](#).

How do I change the JDK for my project?

1. Open the **Project Structure** dialog (**File | Project Structure** or **Ctrl+Alt+Shift+S**).
2. Under **Platform Settings**, select **SDKs**.
3. Click **+**, select **JDK** and specify the JDK installation directory.
4. Click **Apply**.
5. Under **Project Settings**, select **Project**.
6. Under **Project SDK**, select the JDK from the list.
7. Click **OK**.

For more information, see [SDKs](#).

How do I add a library to my project?

1. Open the **Project Structure** dialog (**File | Project Structure** or **Ctrl+Alt+Shift+S**).
2. Under **Project Settings**, select **Libraries**.
3. Click **+**, select **Java** and specify the library location.

4. Select the modules in which this library will be used.
5. Click **OK**.

For more information, see Working with libraries.

How do I configure a Web framework for my project?

In IntelliJ IDEA Ultimate (there's no corresponding functionality in the Community Edition):

1. Open the **Project** tool window (for example **View | Tool Windows | Project**).
2. Right-click the necessary module and select **Add Framework Support**. (Framework support is enabled at a module level.)
3. In the **Add Frameworks Support** dialog that opens, select the frameworks to be supported, specify the associated settings and click **OK**.

For more information, see Add frameworks (facets) and for example Jakarta Server Faces (JSF).

The Run button is disabled. How do I run my application?

The Run button is disabled because there are no run configurations in your project.

If you have a Java class with a `main()` method, open the corresponding file in the editor, right-click the editing area and select **Run '<FileName>.main()'**. As a result, the necessary run configuration will be created automatically and then executed.

You can create run configurations yourself: in the main menu, select **Run | Edit Configurations |**. Click to add a new configuration and choose how you want to run your application.

How do I generate an Ant build script for my project?

Select **Build | Generate Ant Build**. For more information, see Generate Ant Build file.

How can I open several projects in IntelliJ IDEA simultaneously?

It's possible to work with multiple projects simultaneously using IntelliJ IDEA. To achieve this, you only need to open a project, while another one is already opened, and choose **Add to currently opened projects**.

How do I close a project?

Select **File | Close Project**. You can also use **File | Exit** to close all open projects and quit IntelliJ IDEA.

Where is the Options dialog?

In IntelliJ IDEA, the **Settings** dialog is used for similar purposes. To open this dialog, press `Ctrl+Alt+S`.

There is also the **Project Structure** dialog (`Ctrl+Alt+Shift+S`) which lets you manage JDKs, libraries, module dependencies, and so on.

For more information, see Settings / Preferences dialog and Project Structure dialog.

How do I start with VCS integration?

The most popular Version Control Systems including Git, Subversion, Mercurial, Perforce, and more are supported by IntelliJ IDEA. VCS integration for your project can be configured in on the Version Control page of the **Settings / Preferences** dialog. See Version control fore details.

Working with the code editor

Can I use the NetBeans key bindings in IntelliJ IDEA?

Yes, you can.

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Keymap** under **Appearance and Behavior**.
2. In the right-hand part of the dialog, next to **Keymaps**, select **NetBeans 6.5** from the list.

How does code completion in IntelliJ IDEA work?

The code completion suggestion list appears automatically after you type one or two letters. To narrow down this list, use:

- `Ctrl+Space`. The list is reduced to keywords and the names of classes, methods, and fields available in the current context. Note that the list changes when you press `Ctrl+Space` for the second or third time.
- `Ctrl+Shift+Space`. Only the types appropriate for the current context are shown.

For more information, see Code completion.

Is local history in IntelliJ IDEA any different from that in NetBeans?

Local history in IntelliJ IDEA, generally, is more detailed. Whatever you do with a directory, file, class, method or field, or a code block is reflected in your local history. The local history also includes VCS operations.

For more information, see Local History.

Are there any special code analysis features in IntelliJ IDEA?

IntelliJ IDEA can analyze dependencies, data flows and stack traces, find duplicates and evaluate code quality. Just have a look at the options in the **Analyze** menu.

Can I enable 'mark occurrences' in IntelliJ IDEA?

You can. The corresponding option in IntelliJ IDEA is called **Highlight usages of element at caret**. This option is enabled by default.

Just in case:

1. Open the **Settings** dialog `Ctrl+Alt+S`.
2. In the **Editor** category, select **General**.
3. In the right-hand part of the dialog, under **Highlight on Caret Movement**, select the **Highlight usages of element at caret** checkbox.
4. Click **OK**.

Can I enable 'compile on save' in IntelliJ IDEA?

You can.

To enable automatic compilation on every save (or autosave), turn on the **Build project automatically** option in the **Settings** dialog:

1. Open the **Settings** dialog `Ctrl+Alt+S`.
2. In the **Build, Execution, Deployment** category, select **Compiler**.
3. In the right-hand part of the dialog, select the **Build project automatically** checkbox.
4. Click **OK**.

Note that by default, IntelliJ IDEA saves changed files automatically, so you don't need to use `Ctrl+S` as frequently as in other IDEs.

Can I enable 'deploy on save' in IntelliJ IDEA?

There is no such option in IntelliJ IDEA settings, however, you can get similar result by choosing an appropriate application update option in the corresponding run configuration.

For more information, see Update applications on application servers.

(The corresponding functionality is available only in IntelliJ IDEA Ultimate. The Community Edition doesn't provide integration with application servers.)

Using plugins

Can I use NetBeans plugins in IntelliJ IDEA?

Unfortunately not. However, a lot of functionality implemented as plugins for NetBeans is available in IntelliJ IDEA "out of the box". Besides, there's a lot of plugins for IntelliJ IDEA, so you can always find an IntelliJ IDEA plugin with the functionality similar to that of your favorite NetBeans plugin.

How do I find the plugin that I need?

All the functions related to working with plugins are on the **Plugins** page of the **Settings** dialog `Ctrl+Alt+S`. You can look for, download, install and update the plugins as well as enable and disable them.

For more information, see Plugins and Manage plugins.

How do I install the plugin that I have available on my computer?

1. Open the **Settings** dialog `Ctrl+Alt+S`.
2. In the left-hand pane, select **Plugins**.
3. In the lower part of the **Plugins** page, click **Install plugin from disk**.
4. In the dialog that opens, select the plugin file (normally, a JAR or ZIP).

5. Click **OK**.
6. If asked, restart IntelliJ IDEA.

I'd like to write a plugin for IntelliJ IDEA. Are there any instructions?

Yes, have a look at:

- [IntelliJ Platform SDK Developer Guide](#).
- [Information for Plugin Developers on the IntelliJ IDEA Plugins page](#)
- [Open API and Plugin Development](#)

Is it possible to build NetBeans RCP applications with IntelliJ IDEA?

It is possible, however you won't get the same kind of support you would in the case of NetBeans (wizards, menu actions, and so on). Have a look at [Using IntelliJ IDEA for NetBeans Platform Development](#).

Configuring PHP development environment

What configuration is needed before start?

A lot of IntelliJ IDEA features are available without any configuration right after you launch it. Still, to take full advantage of running your PHP application, you need to configure a PHP interpreter and a server.

If you plan to launch the application locally, you need a PHP engine installed and registered in IntelliJ IDEA, as well as a Web server installed, configured, and integrated with IntelliJ IDEA. You can install these components separately or use an AMP package. For more details about initial environment configuration, refer to [Configure PHP development environment](#).

If you are going to run and debug an application directly on a remote host, the only thing you need is register access to this host in IntelliJ IDEA to enable synchronization.

How do I start with deployment to a remote host?

If you've checked out your project from the remote host, the deployment server is already configured. Otherwise, you will need to get it configured (it can be FTP/SFTP/FTPS server or mounted/local folder) on the Deployment page of the **Settings/Preferences** dialog. The Remote host tool window is available on the right-hand side of the IntelliJ IDEA window, which can be handy for browsing through your remote server and performing various actions.

See [Deploy your application](#) for details.

How do I start debugging?

IntelliJ IDEA comes with support for both Xdebug and Zend Debugger for debugging and profiling. There is a zero-configuration debugging workflow available, which means that to start debugging you only need to:

- Click **Start Listening for PHP Debugging Connections** on the toolbar of the IDE.
- Place a breakpoint in code by clicking in the editor gutter next to the line.
- Start debugging in the browser using a plugin or browser bookmarklets.

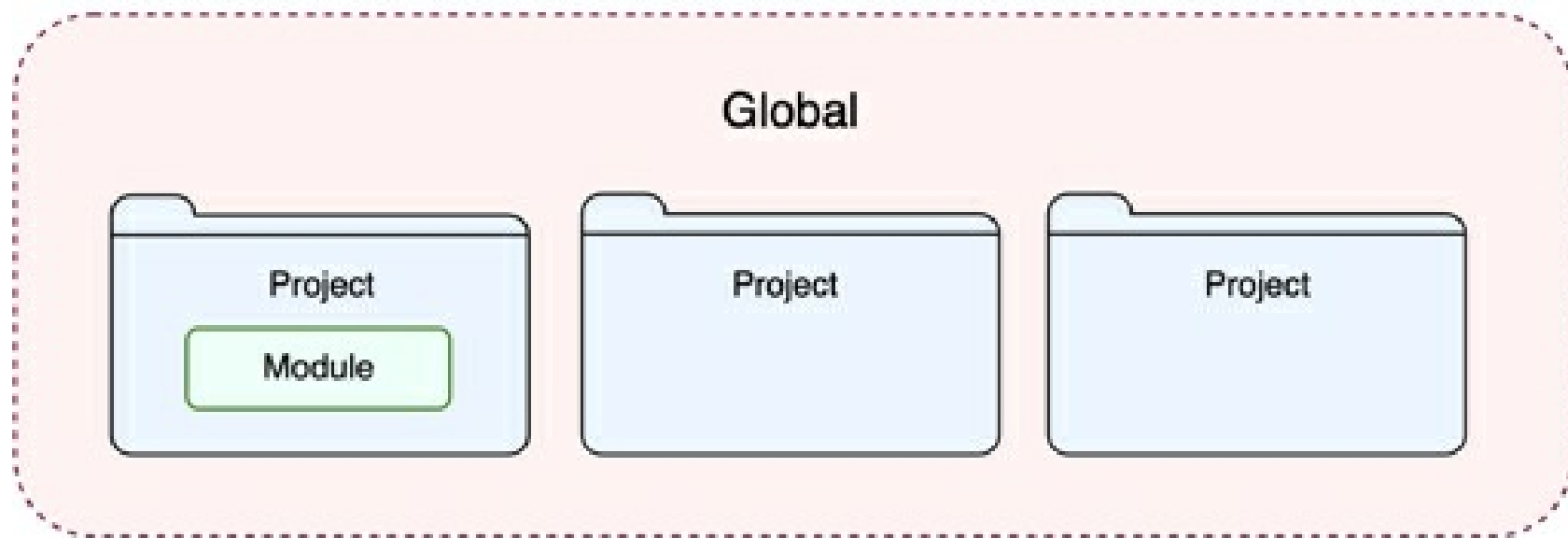
IntelliJ IDEA vs NetBeans terminology

NetBeans and IntelliJ IDEA use different names for similar entities. It is important to understand the difference between the terminologies of both IDEs. In this section you can find a table of mappings between the NetBeans and IntelliJ IDEA terms.

NetBeans	IntelliJ IDEA
Project	Module
Project-specific JDK	Module SDK
Global library	Global library
Project library	Module library
Project dependency	Module dependency

Configuring the IDE

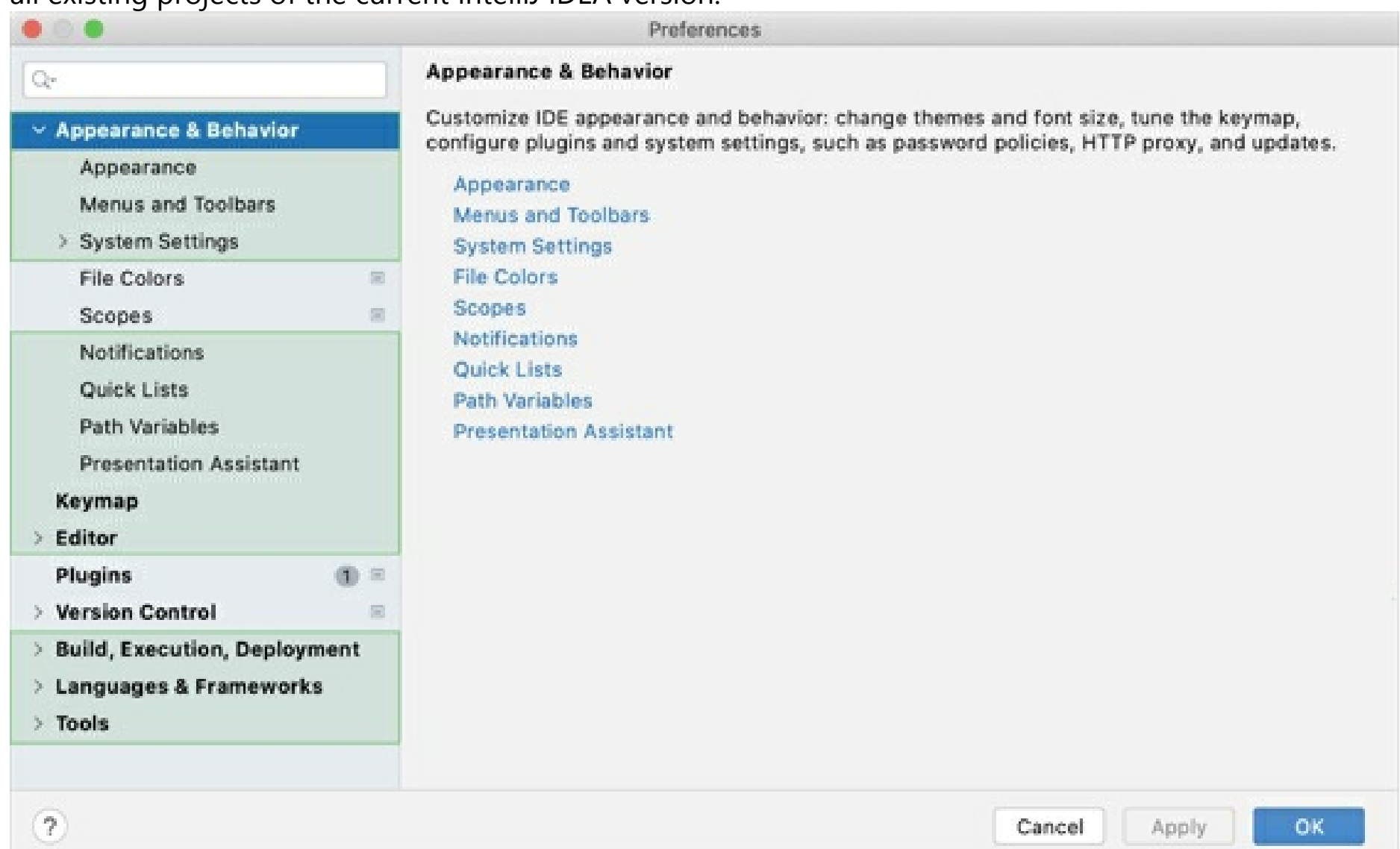
IntelliJ IDEA allows you to configure the settings on several levels: the module level, the project level, and globally.



Global settings apply to all projects of a specific installation, or version, of IntelliJ IDEA. Such settings include IDE appearance (themes, color schemes, menus and toolbars), notification settings, the set of the installed and enabled plugins, debugger settings, code completion, and so on.

To configure your IDE, select **IntelliJ IDEA | Preferences** for macOS or **File | Settings** for Windows and Linux. Alternatively, press `Ctrl+Alt+S` or click on the toolbar.

Settings that are NOT marked with the  icon in the **Settings/Preferences** dialog are global and apply to all existing projects of the current IntelliJ IDEA version.



Restore IDE settings

In IntelliJ IDEA 2020.1, the configuration paths have been changed. That is why, the information in this section is valid for IntelliJ IDEA of version 2020.1 and later. If you use a previous version of IntelliJ IDEA, add the version number to the page URL in order to access the corresponding documentation. For example, if you use IntelliJ IDEA 2019.3, type www.jetbrains.com/help/idea/2019.3/configuring-project-and-ide-settings.html.

When you restore the default IDE settings, IntelliJ IDEA backs up your configuration to a directory. You

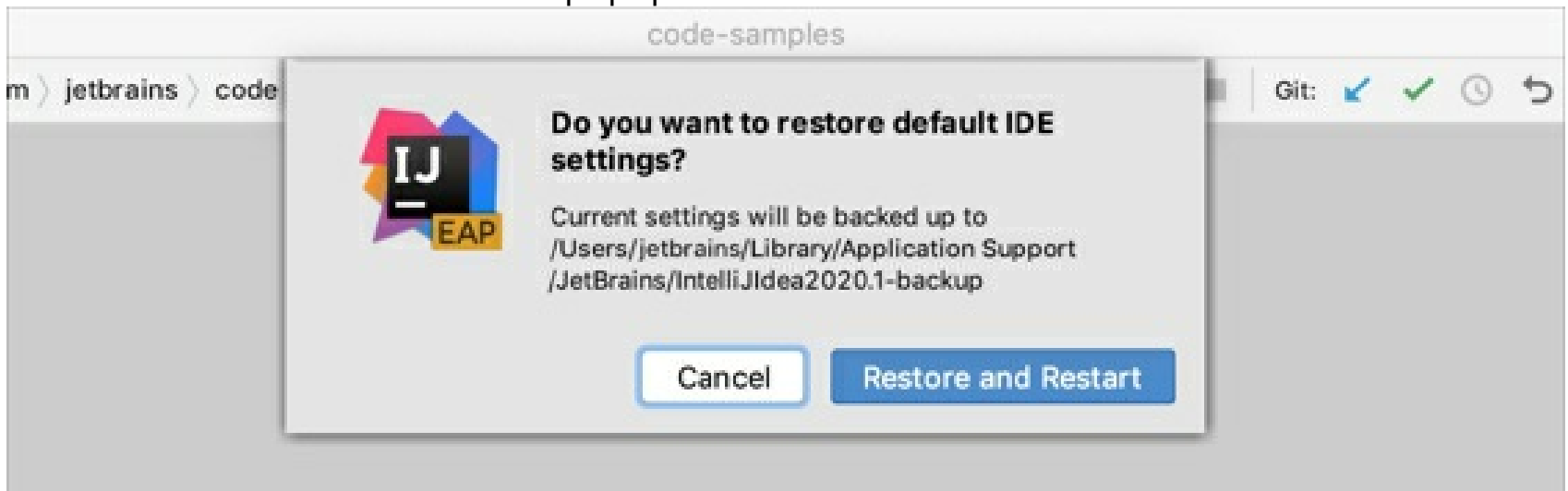
can always restore your settings from this backup.

Back up your settings and restore the defaults

1. From the main menu, select **File | Manage IDE Settings | Restore Default Settings**.

Alternatively, press Shift twice and type `Restore default settings`.

IntelliJ IDEA shows a confirmation popup:



2. Click **Restore and Restart**. The IDE will be restarted with the default configuration.

When IntelliJ IDEA restores the default IDE settings, it creates a backup directory with your configuration in:

Windows

macOS

Linux

Syntax

`%APPDATA%\JetBrains\<product><version>-backup`

Example

`C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJ2021.1-backup`

Apply the IDE settings from a backup

1. From the main menu, select **File | Manage IDE Settings | Import Settings**.
2. In the dialog that opens, specify the path to the backup directory and click **Open**.

IntelliJ IDEA shows a confirmation popup. Note, that after you apply the settings from the backup, *these settings will be overwritten* with your current IDE configuration.

Apart from the backup configuration directory, you can select the configuration directory from another IntelliJ IDEA version or a **.zip** file with the previously exported settings.

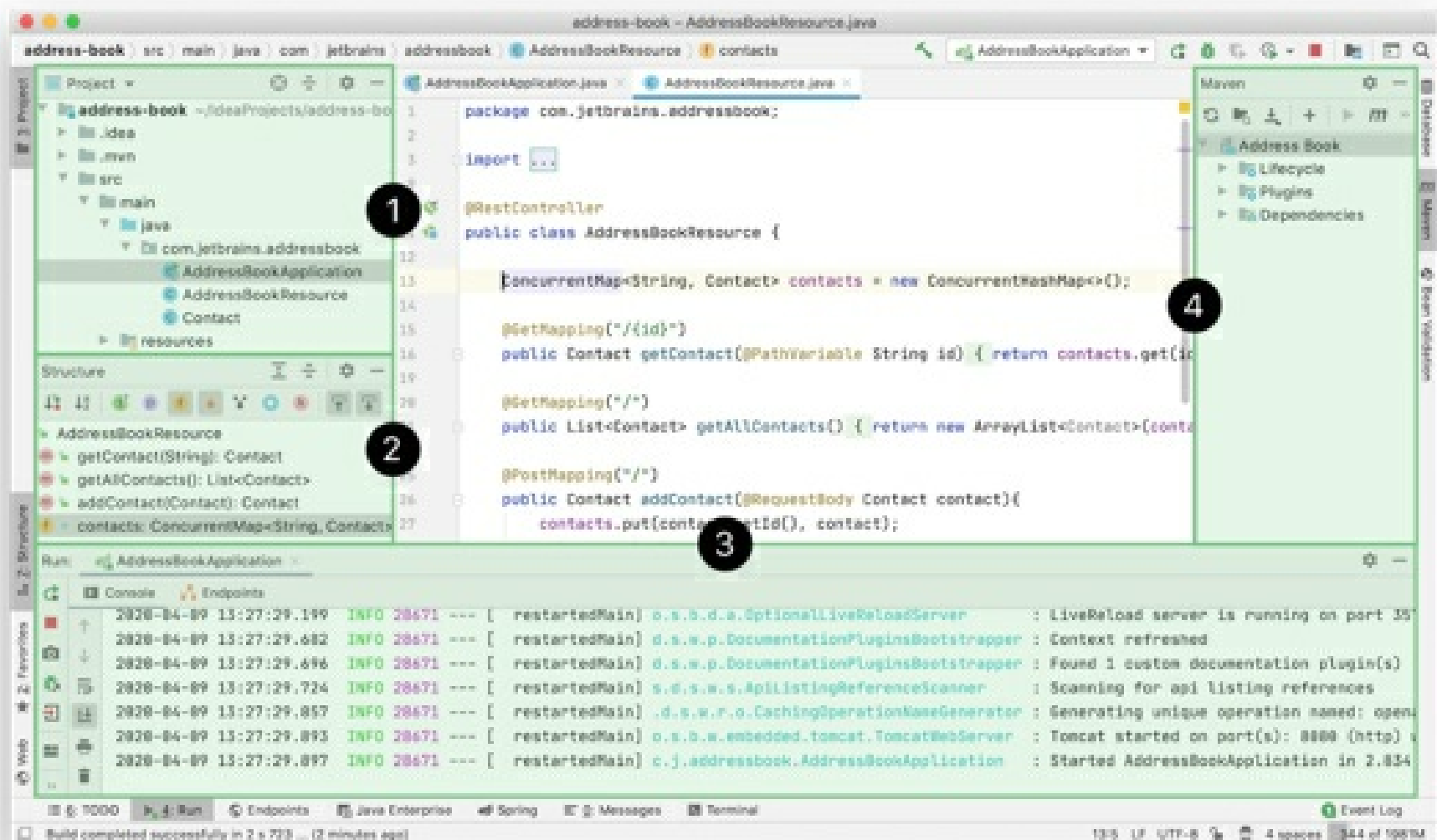
3. Click **Restart** to apply the settings from the backup and restart the IDE.

Tool windows

View | Tool Windows

Tool windows provide access to useful development tasks: viewing your project structure, running and debugging your application, integration with version control systems and other external tools, code analysis, search, and navigation, and so on. By default, tool windows are attached to the bottom and sides of the main window. However, you can rearrange and even detach them to use as separate windows, for example, on another monitor.

The following screenshot shows several common tool windows occupying space around the editor:



1. **Project** tool window
2. **Structure** tool window
3. **Run** tool window
4. **Maven** tool window

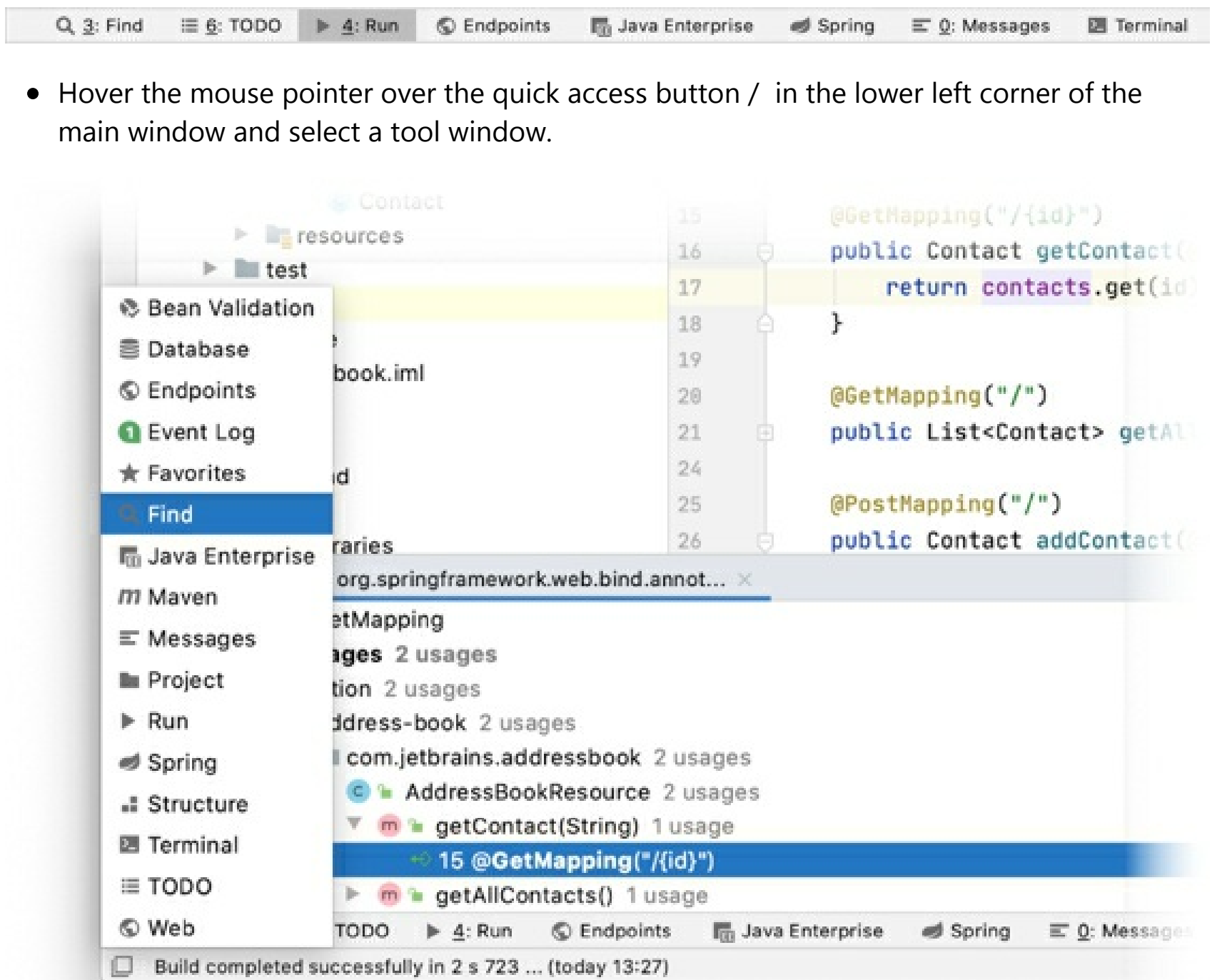
Some tool windows are always available (for example, **Project** and **Structure**), some are activated when a specific plugin is enabled or your project has a specific structure (for example, **Maven**, **Gradle**, and **Spring**), and some appear only when you perform a certain action (for example, **Run**, **Debug**, and **Find**).

For example, the **Run** tool window appears only after you run your code. It is accessible from the main menu **View | Tool Windows | Run** or using the shortcut **Alt+4** as long as there is some output in the console and you don't close all the tabs in it.

Open a tool window

To show or hide a tool window, do one of the following:

- From the main menu, select a tool window under **View | Tool Windows**.
- Use the corresponding shortcut, for example, **Alt+1** to open the Project tool window. If there is no shortcut for a tool window, you can assign it as described in Configure keyboard shortcuts.
- Click the corresponding tool window button on the tool window bar.



- Hover the mouse pointer over the quick access button / in the lower left corner of the main window and select a tool window.

Hide the active tool window

- Press Shift+Escape or select **Window | Active Tool Window | Hide Active Tool Window** from the main menu. Press Shift+Escape again to show the hidden tool window.

Hide all tool windows

This can help you focus on the editor.

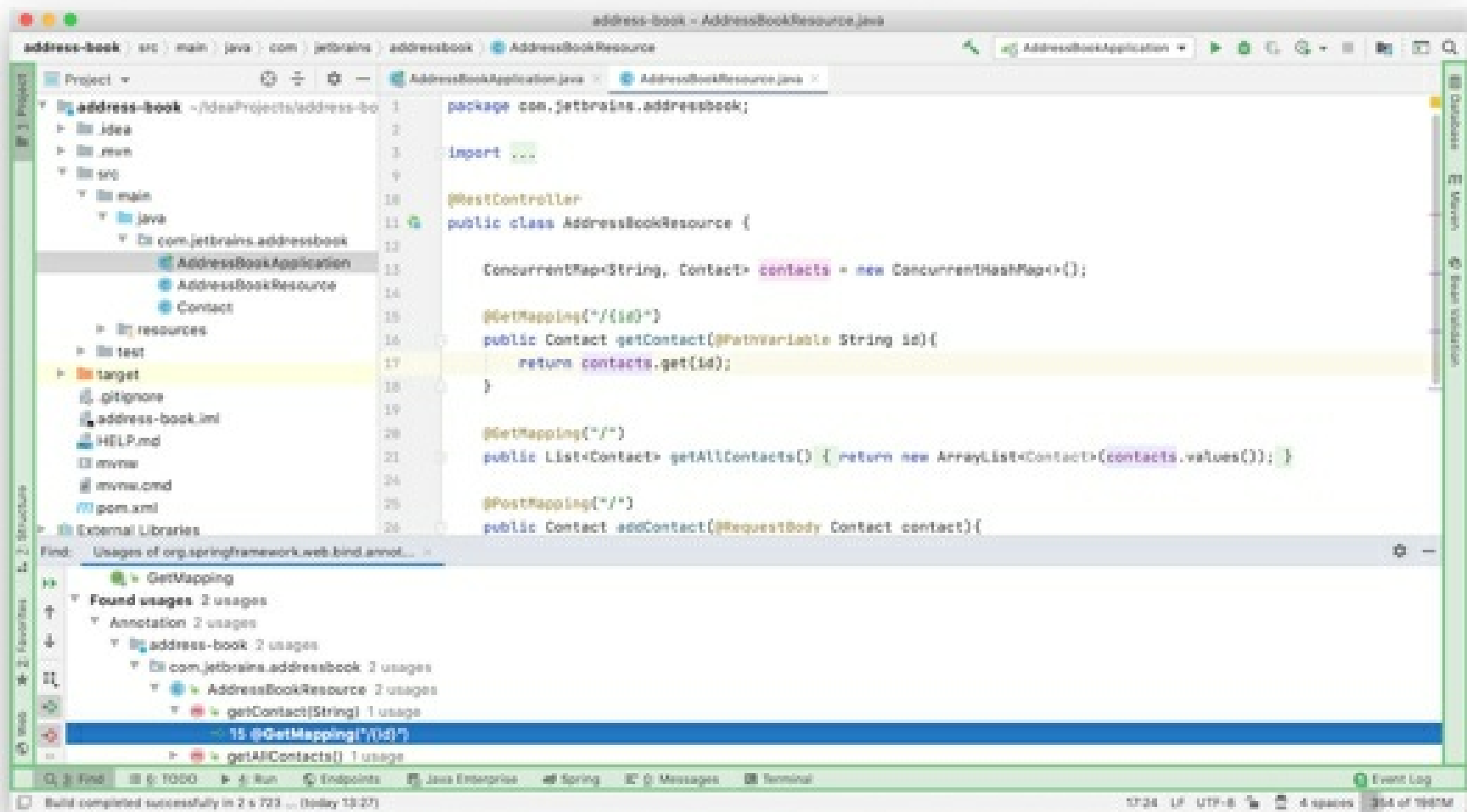
- Press Ctrl+Shift+F12 or select **Window | Active Tool Window | Hide All Windows** from the main menu. Press Ctrl+Shift+F12 again to show the hidden tool windows.

Navigate between the editor and a tool window

- To change focus from a tool window to the editor, press `Escape`.
- To change focus from the editor back to the last active tool window, press `F12` or select **Window | Active Tool Window | Jump to Last Tool Window** from the main menu.

Tool window bars and buttons

Tool window bars on the edges of the main window contain buttons to show and hide tool windows. Right-click a tool window button to open its context menu, where you can change the viewing mode and move the tool window. You can also drag tool window buttons to rearrange tool windows on the bars.



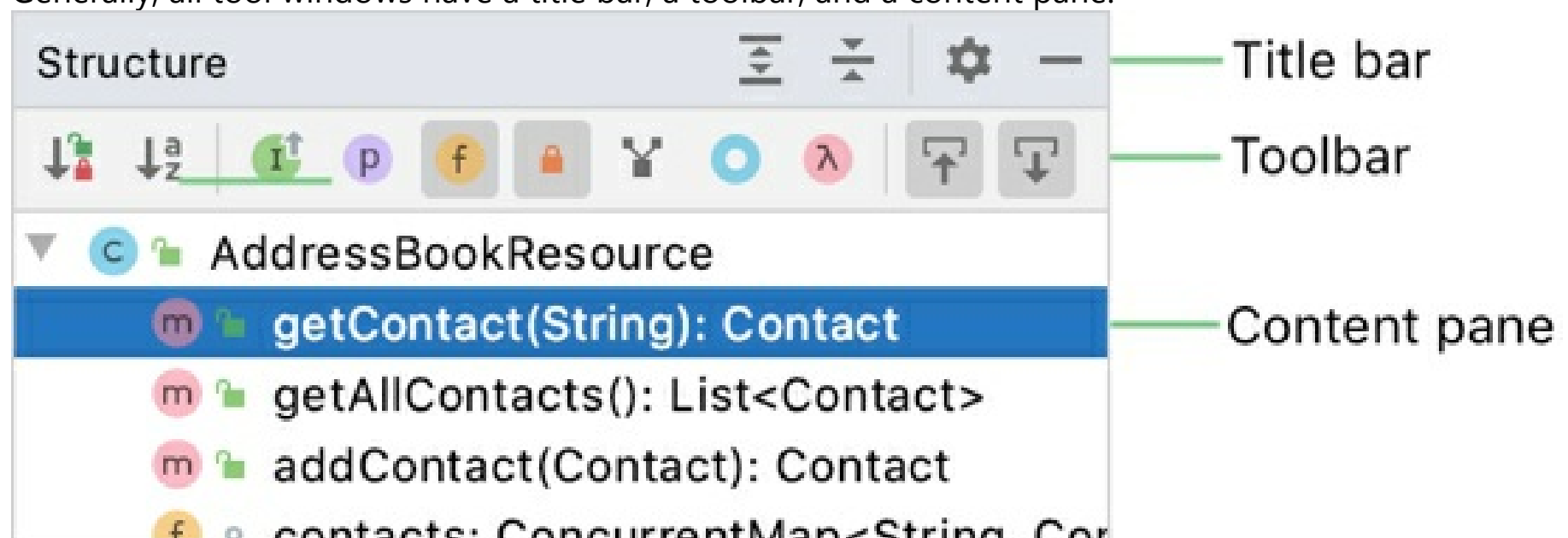
Show or hide tool window bars

- Click the quick access button in the lower left corner of the main window to hide the tool window bars. The button changes to and you can click it to show the tool window bars.
- Alternatively, select or clear **Tool Window Bars** from the main menu under **View | Appearance**.

When the tool window bars are hidden, you can double-press and hold **Alt** to show hidden tool window bars.

Tool window components

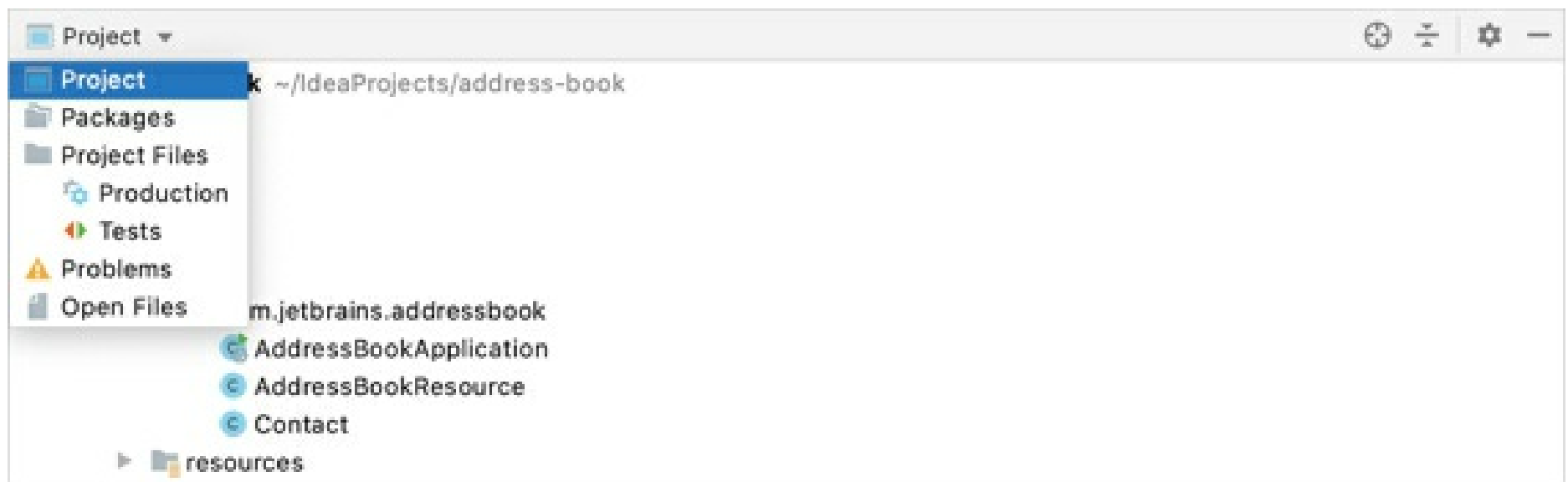
Generally, all tool windows have a title bar, a toolbar, and a content pane.



Some tool windows are also separated using tabs or a dropdown selector in the title bar, based on the functionality that it covers. Select **Window | Active Tool Window | Group Tabs** to show tabs. Disable this option to show a dropdown menu.

Group Tabs disabled

Group Tabs enabled



The title bar contains the tool window options menu for changing the viewing mode and the position of the tool window. You can also access these options by right-clicking the title bar or the tool window button. Some tool windows can have other options in this menu, depending on the functionality (for example, to sort, filter, and group items listed in a tool window).

Click to hide the tool window and use other buttons that may be on the title bar, for example:

- and to expand and collapse the contents of the tool window
- to locate and select the file from the editor in the tool window

Actions from the tool window toolbar are usually also available in the main menu and context menus. Some of them can also be executed with a default shortcut. You can assign shortcuts for actions as described in [Configure keyboard shortcuts](#).

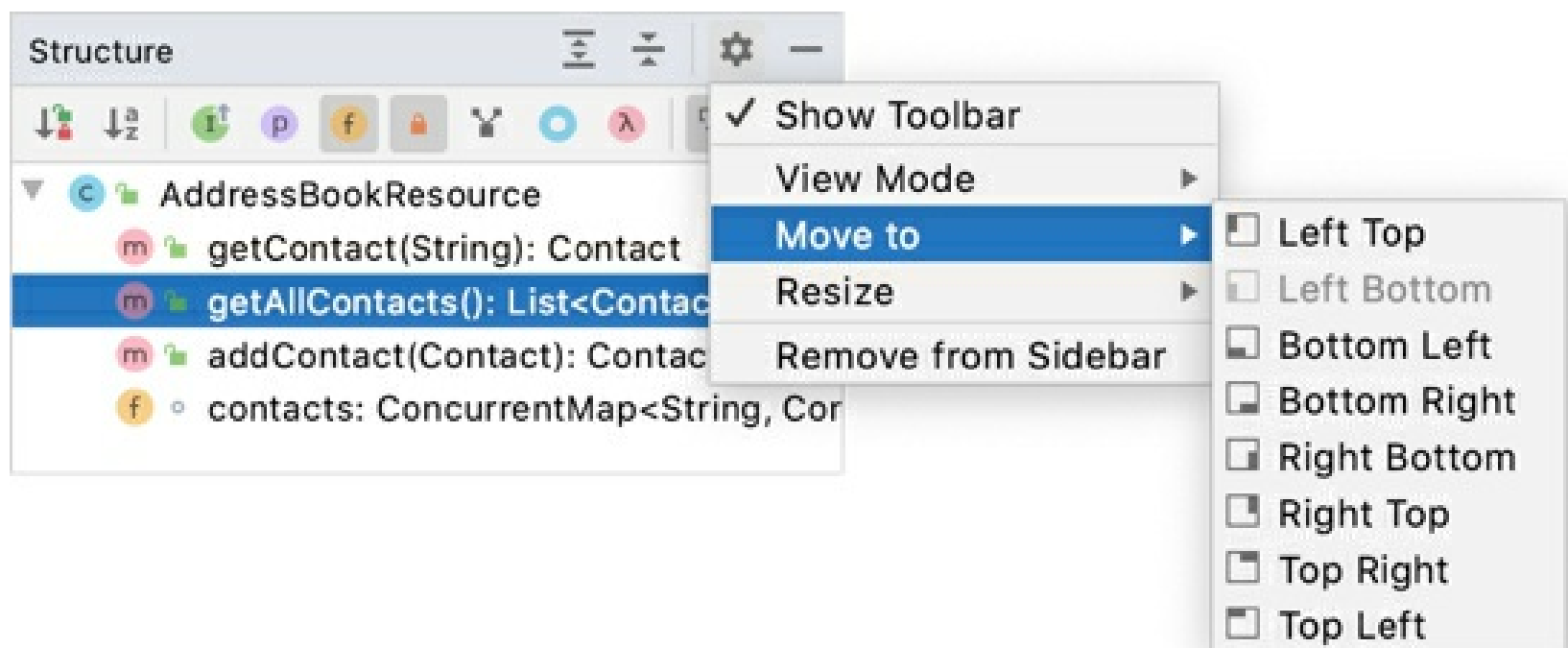
For all tool windows that display tree-like structures (for example, Project tool window) you can display vertical lines that mark indent levels in tree views and help you better understand the hierarchy of the components in your project. To display these lines, enable **Show tree indent guides** on the **Appearance and Behavior | Appearance** page of the **Settings/Preferences** [Ctrl+Alt+S](#).

Arrange tool windows

By default, tool windows are attached to the edges of the main window. You can detach them to use as separate windows, as described in Tool window view modes.

Move a tool window

- Click and drag the tool window button on the tool window bar.
- Alternatively, you can click the tool window options menu or right-click the tool window title bar and select where to attach the tool window under **Move to**.



Resize a tool window

- Click and drag the border of a tool window.

To resize the active tool window in steps, select the corresponding actions under **Window | Active Tool Window | Resize** from the main menu that are by default assigned to the following shortcuts: Ctrl+Shift+Left, Ctrl+Shift+Right, Ctrl+Shift+Up, Ctrl+Shift+Down.

To stretch the tool window to the maximum width or height, press Ctrl+Shift+Quote or select **Window | Active Tool Window | Resize | Maximize Tool Window** from the main menu.

Restore the default arrangement of tool windows

- From the main menu, select **Window | Restore Default Layout** or press Shift+F12.

Save the arrangement of tool windows

- From the main menu, select **Window | Store Current Layout as Default**.

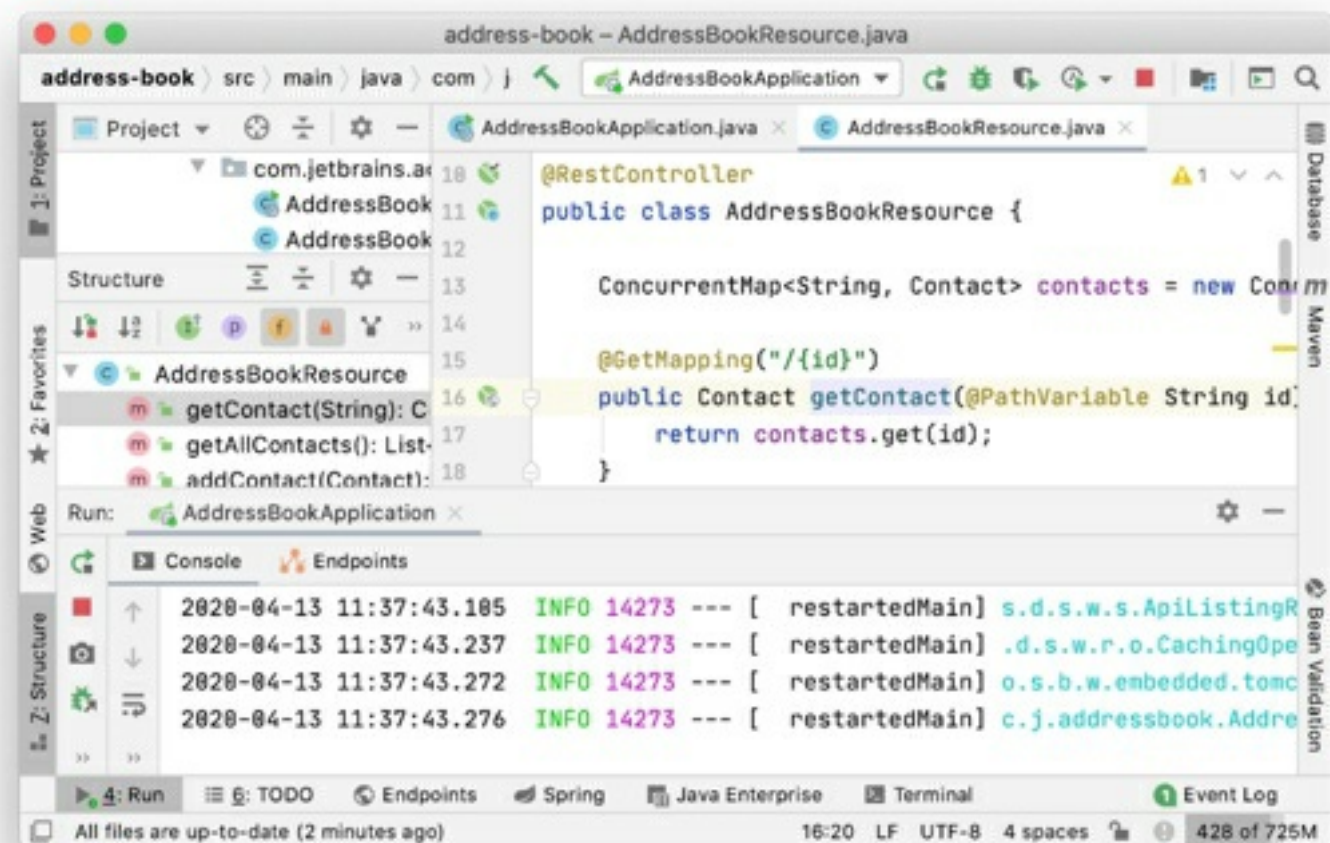
Optimize for wide-screen monitors

IntelliJ IDEA provides several options to optimize the positioning of tool windows on wide-screen monitors.

1. In the **Settings/Preferences** dialog Ctrl+Alt+S, select **Appearance & Behavior | Appearance**.
2. Under **Tool Windows**, configure the following:
 - **Widescreen tool window layout**: Maximize the height of vertical tool windows by limiting the width of horizontal tool windows.

Widescreen layout disabled

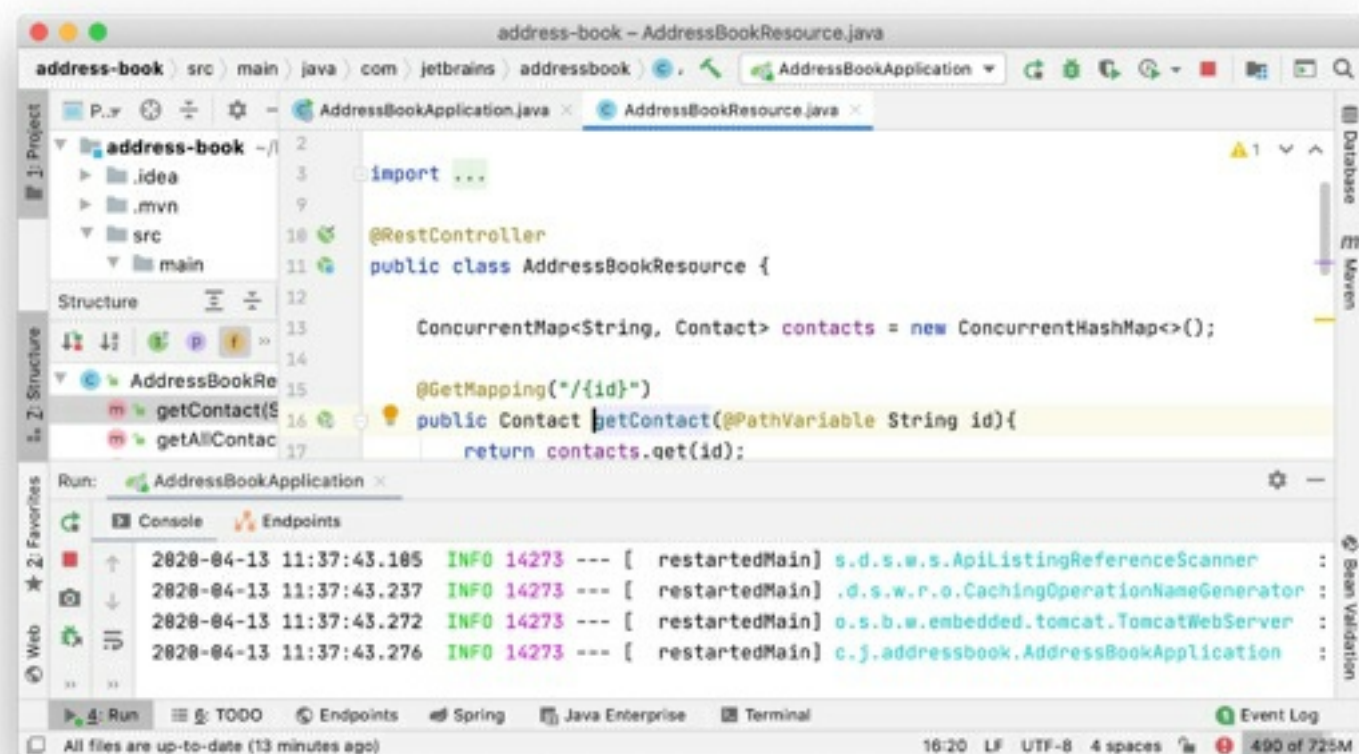
Widescreen layout enabled



- **Side-by-side layout on the left** and **Side-by-side layout on the right**: Display vertical tool windows that are attached to the top and bottom edges in two columns instead of stacked on top of each other.

Side-by-side layout disabled

Side-by-side layout enabled



3. Click **OK** to apply the changes.

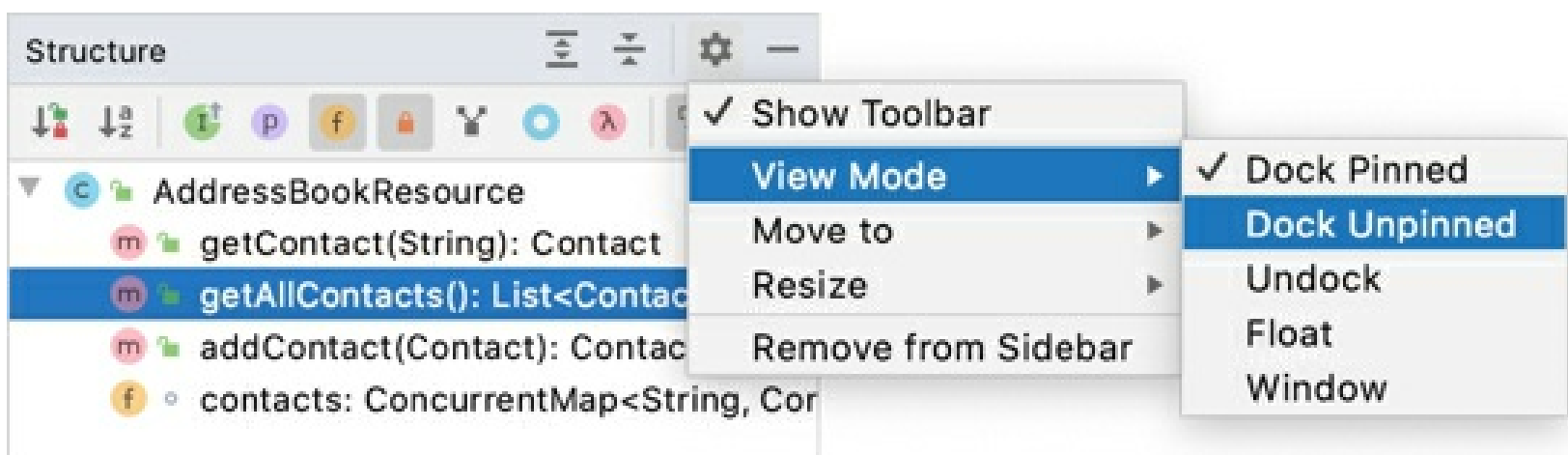
Tool window view modes

Window | Active Tool Window | View Mode

By default, tool windows are attached to the edges of the main window and are always visible. You can change the *view mode* of a specific tool window, for example, to make it visible only when active or to detach it from the tool window bar.

Change the view mode of a tool window

- From the main menu, select **Window | Active Tool Window | View Mode** and then choose the view mode.
- Alternatively, on the title bar of a tool window, click , select **View Mode** and then choose the view mode.



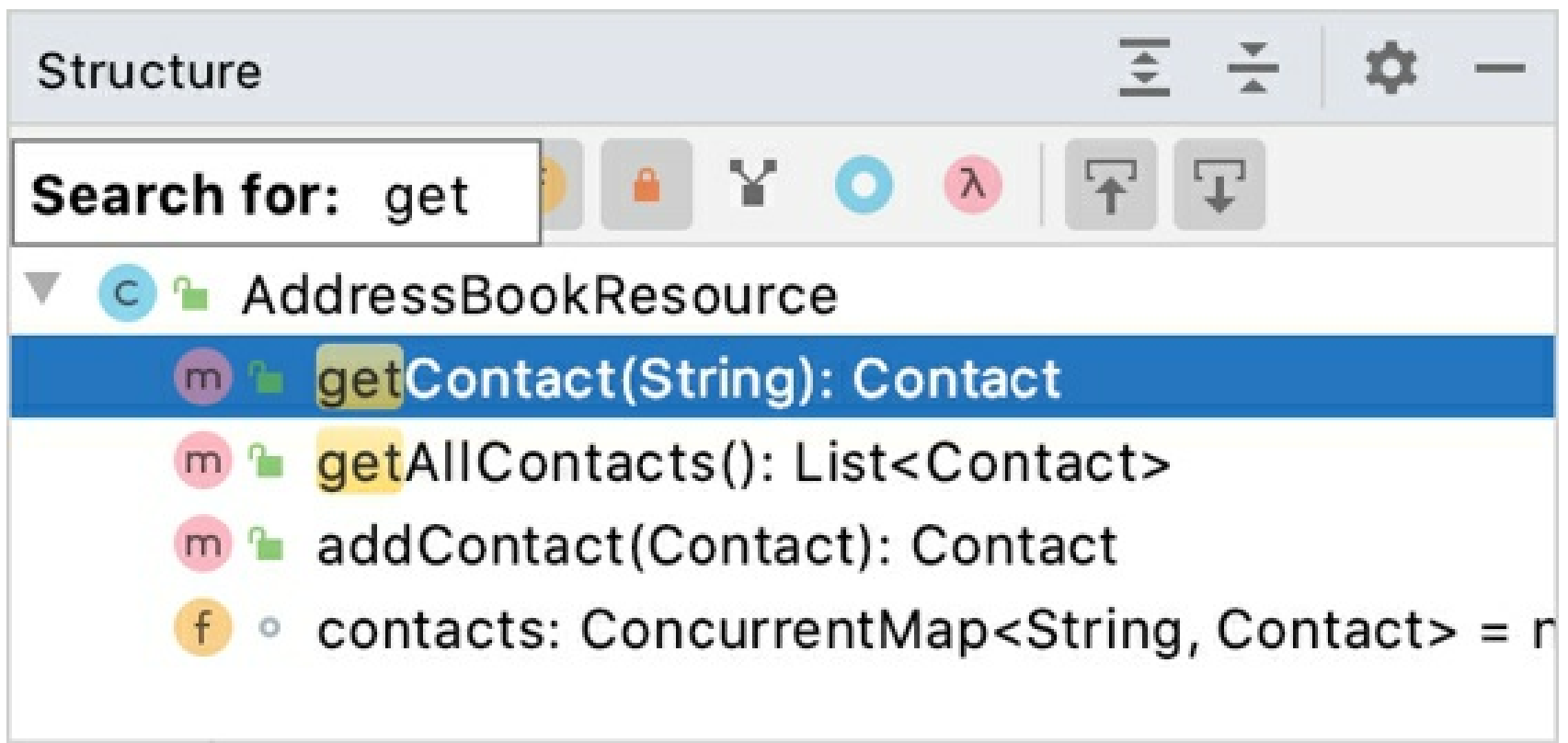
The following tool window view modes are available:

- **Dock pinned:** This is the default view mode when the tool window is attached to the tool window bar and is always visible along with the editor and other pinned tool windows.
- **Dock unpinned:** The tool window is attached to the tool window bar but is visible only when it is active. It does not obstruct the editor or other tool windows.
- **Undock:** The tool window is attached to the tool window bar and covers a part of the editor or other tool windows when active. It is not visible when another tool window is active.
- **Float:** The tool window is detached from the tool window bar, floating on top of the main window. It is visible only together with the main project window.
- **Window:** The tool window acts as a separate application window. You can view it independently from the main project window and even move it to a different monitor or desktop.

Speed search in tool windows

Speed search helps you quickly find an item in a tool window: a file or a folder in the **Project** tool window, a member in the **Structure** tool window, a changelist in the **Commit** tool window `Alt+0`, an item in the **TODO** list, and so on.

1. Select a tool window, a tree, a list, or a popup.
2. Start typing the item name, for example, the name of a file, class, or field. As you type, the **Search for** field appears over the tool window showing the entered characters, and the selection moves to the first item that matches the specified string. The matching part of the string is highlighted.



3. If several items match the pattern, use the `Up` and `Down` keys to move between them. Press `Enter` to open the selected item. Press `Escape` to hide the **Search for** field.

Speed search works only for expanded nodes. It will not highlight matching items in collapsed nodes.

Menus and toolbars

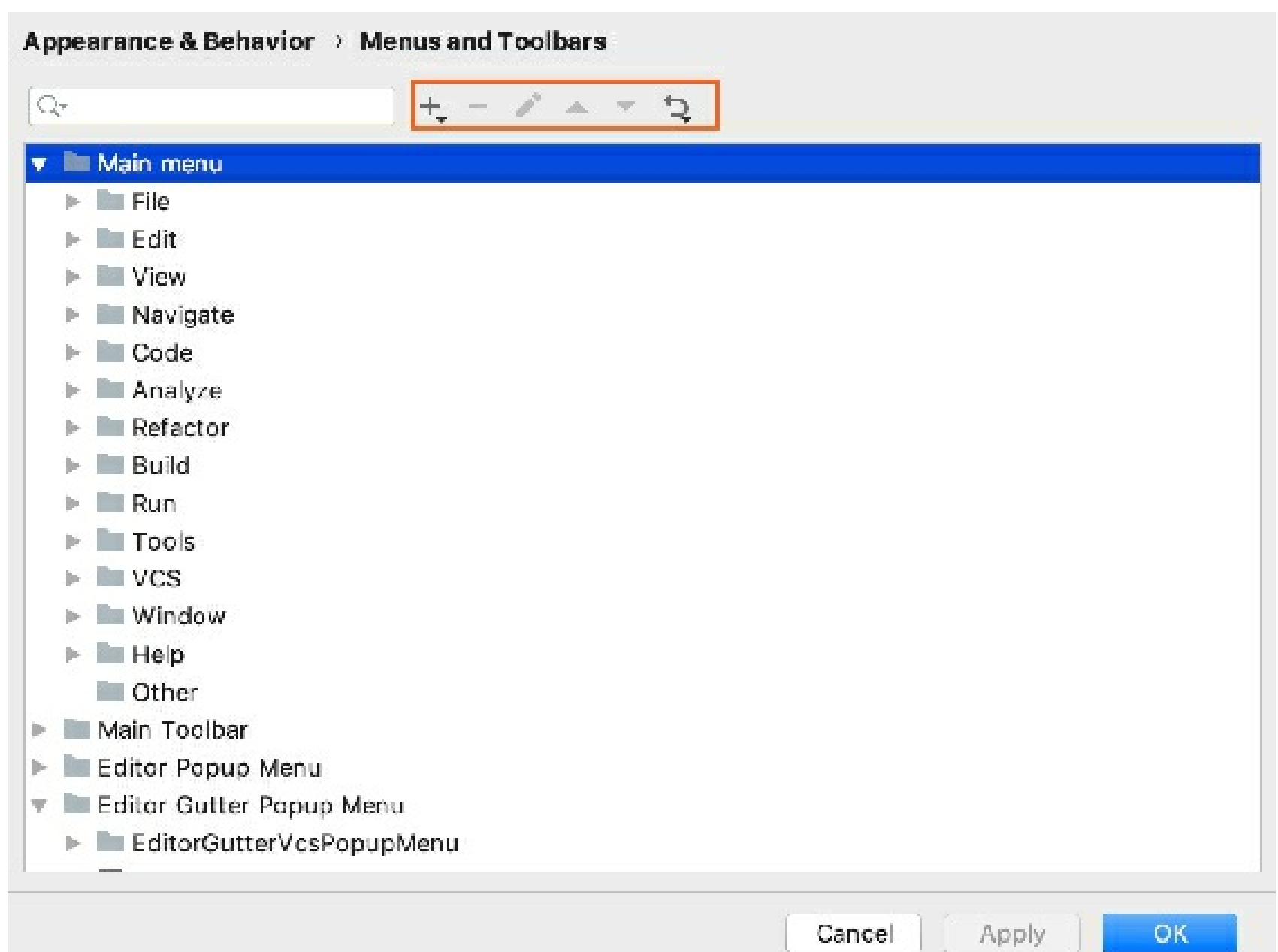
As you work with the IDE, you perform some actions more often than the others. To maximize your productivity, learn the default shortcuts for your favorite actions or assign shortcuts for them. You can also customize the menus and toolbars to only contain the actions that you need, regroup them, and configure their icons.

For example, if you work with a Java project, you might want to remove other framework files that you do not use from the **File | New** menu to keep that part of the menu shorter and leave only the needed options.

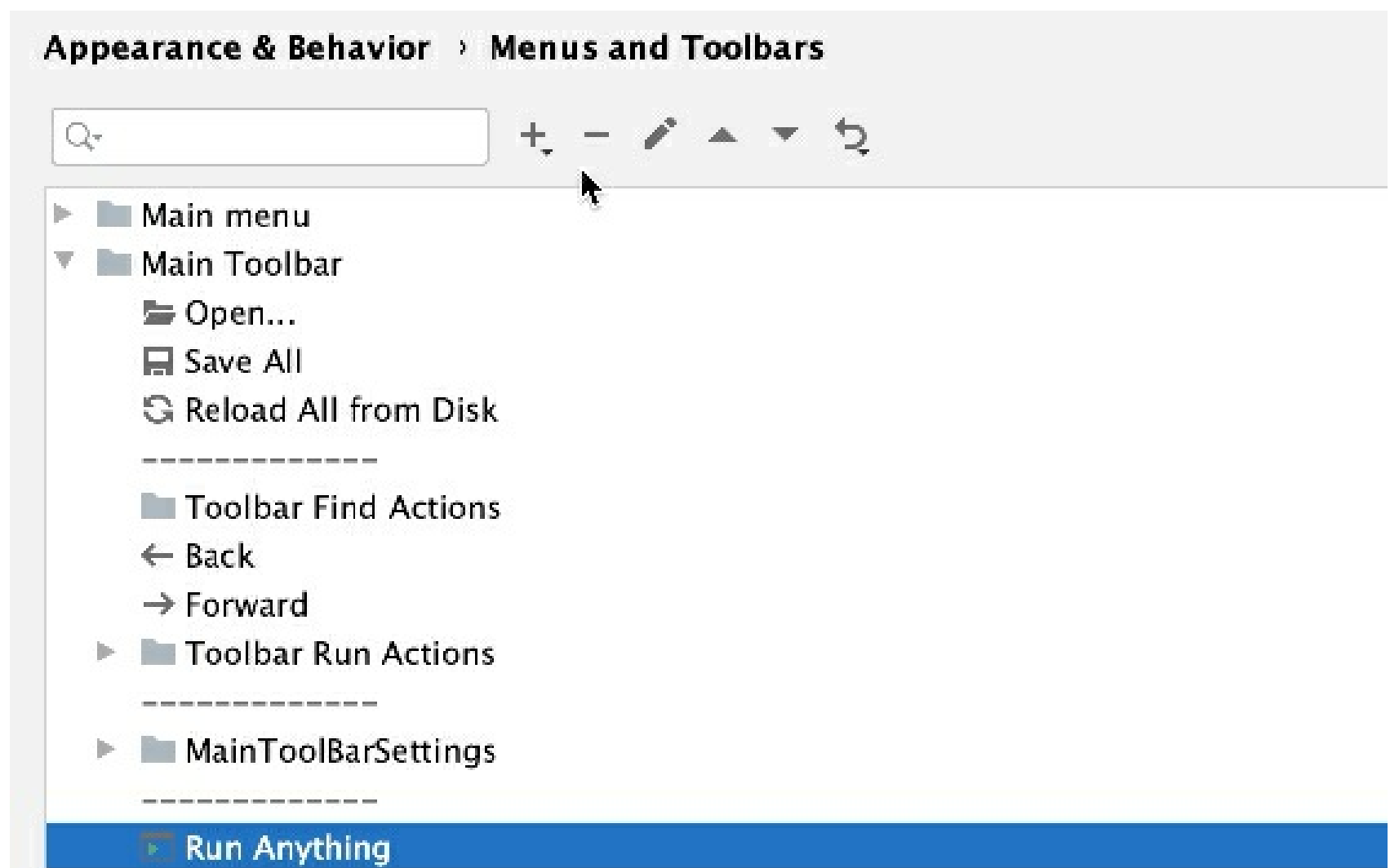
To quickly access your favorite files, directories, bookmarks, and breakpoints, use the **Favorites** tool window. For more information, see Favorites.

Customize menus and toolbars

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | Menus and Toolbars**.



2. In the list of available menus and toolbars, expand the node you want to customize and select the desired item.
 - Click to add an action or a separator under the selected item.
 - Click to remove the selected item.
 - Click to add or change the icon for the selected action. You can use only PNG or SVG files as icons.
 - Click or to move the selected item up or down.
 - Click to restore the selected action or all actions to default settings.



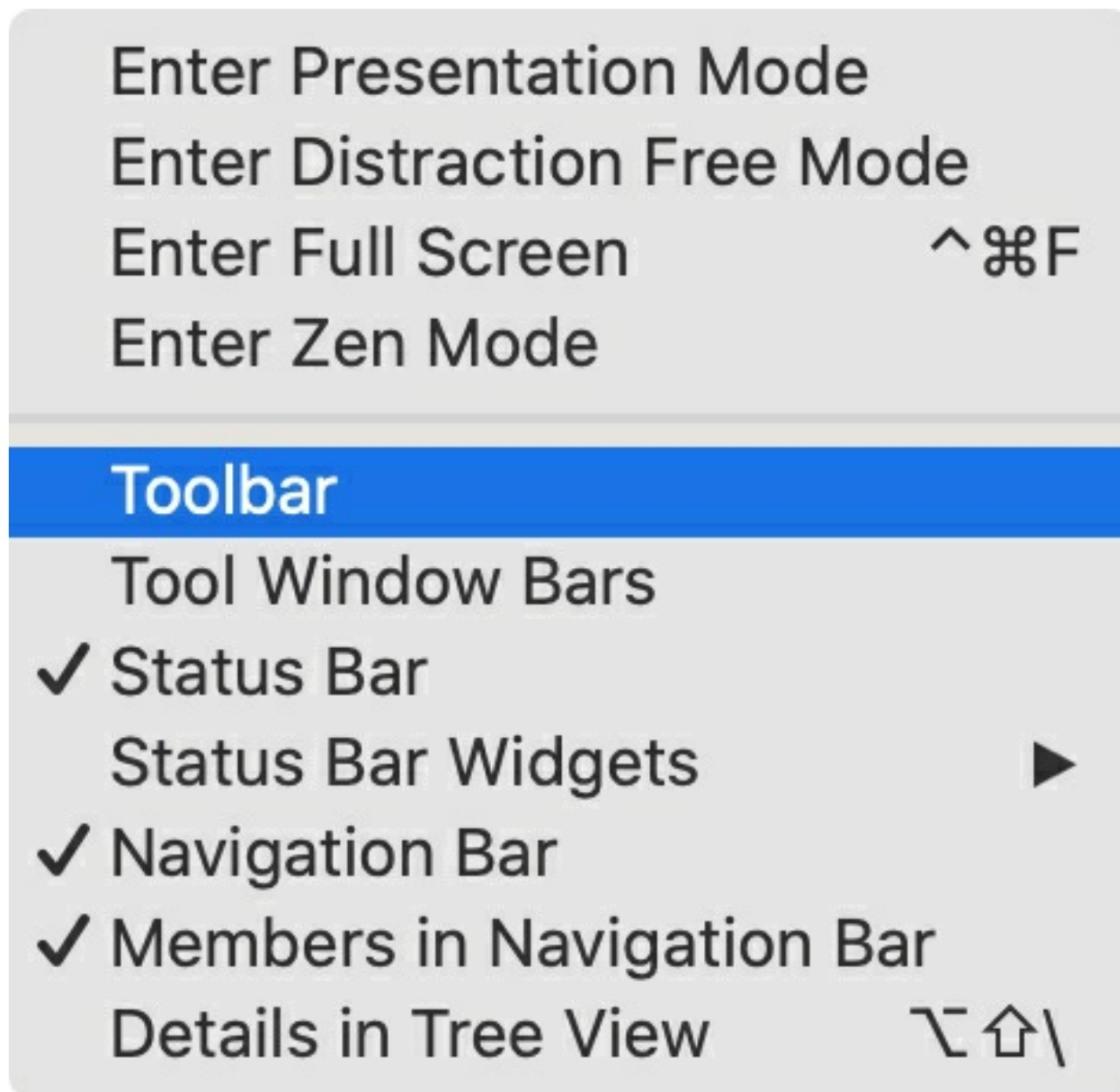
Gif

3. Click **OK** to save your changes.

Show and hide UI elements

If you have a small monitor, you can hide some of the UI elements that you never use. These elements are described in Overview of the user interface.

- From the main menu, select **View | Appearance** and enable or disable the necessary elements.



- **Toolbar:** Located at the top of the window, this is the main toolbar with buttons for opening files, undo and redo actions.
- **Tool Window Bars:** Located on the edges of the window, these bars contain buttons for showing, hiding, and arranging the tool windows. See Tool window bars and buttons.
- **Status Bar:** Located at the bottom of the window, it shows event messages, indicates the overall project and IDE status, and provides quick access to some settings via widgets. See Status bar.
- **Status Bar Widgets:** Located in the right part of the status bar. You can also right-click the status bar to show and hide widgets.
- **Navigation Bar:** Located at the top of the window, where you can navigate the directories and files of your project with `Alt+Home` as an alternative to the **Project** tool window. See Navigation bar.
- **Members in Navigation Bar:** Show the fields and methods in the navigation bar.
- **Main Menu:** On Windows and Linux, hide the standard menu bar of the application window with **File**, **Edit**, **View**, and other menus. You can't hide it on macOS or on Linux if you are using the Linux native menu.

If you hide the main menu, you can still access it with the corresponding action: press `Ctrl+Shift+A` and search for *main menu*.

- **Details in Tree View:** Show the last modification date and size of files in the **Project** tool window

Quick lists with your favorite actions

A *quick list* is a popup that contains a custom set of IntelliJ IDEA actions. Think of it as a custom menu or toolbar, for which you can assign a shortcut for quick access. You can create as many quick lists as necessary. Each action in a quick list is identified by a number from 0 through 9.

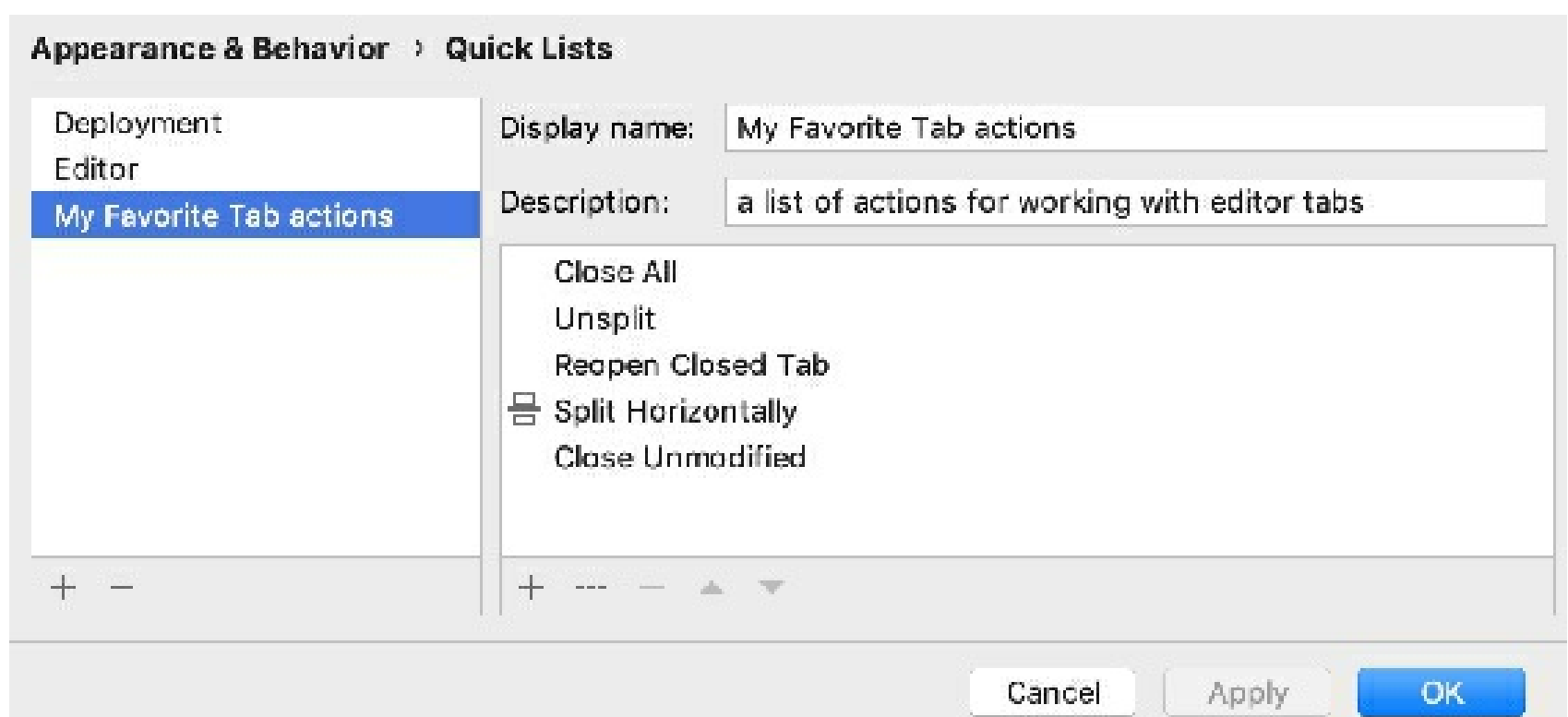
There are a number of predefined quick lists, but note that they are not configurable:

- **Refactor this** `Ctrl+Alt+Shift+T`
- **VCS Operations** `Alt+``

For example, you can create a list of actions that do not have shortcuts assigned to them and refer to that list by one shortcut followed by the number associated with a specific action. You can also create a quick list of your favorite Gradle tasks or a list of different run configurations that you can invoke with one shortcut.

Create a quick list

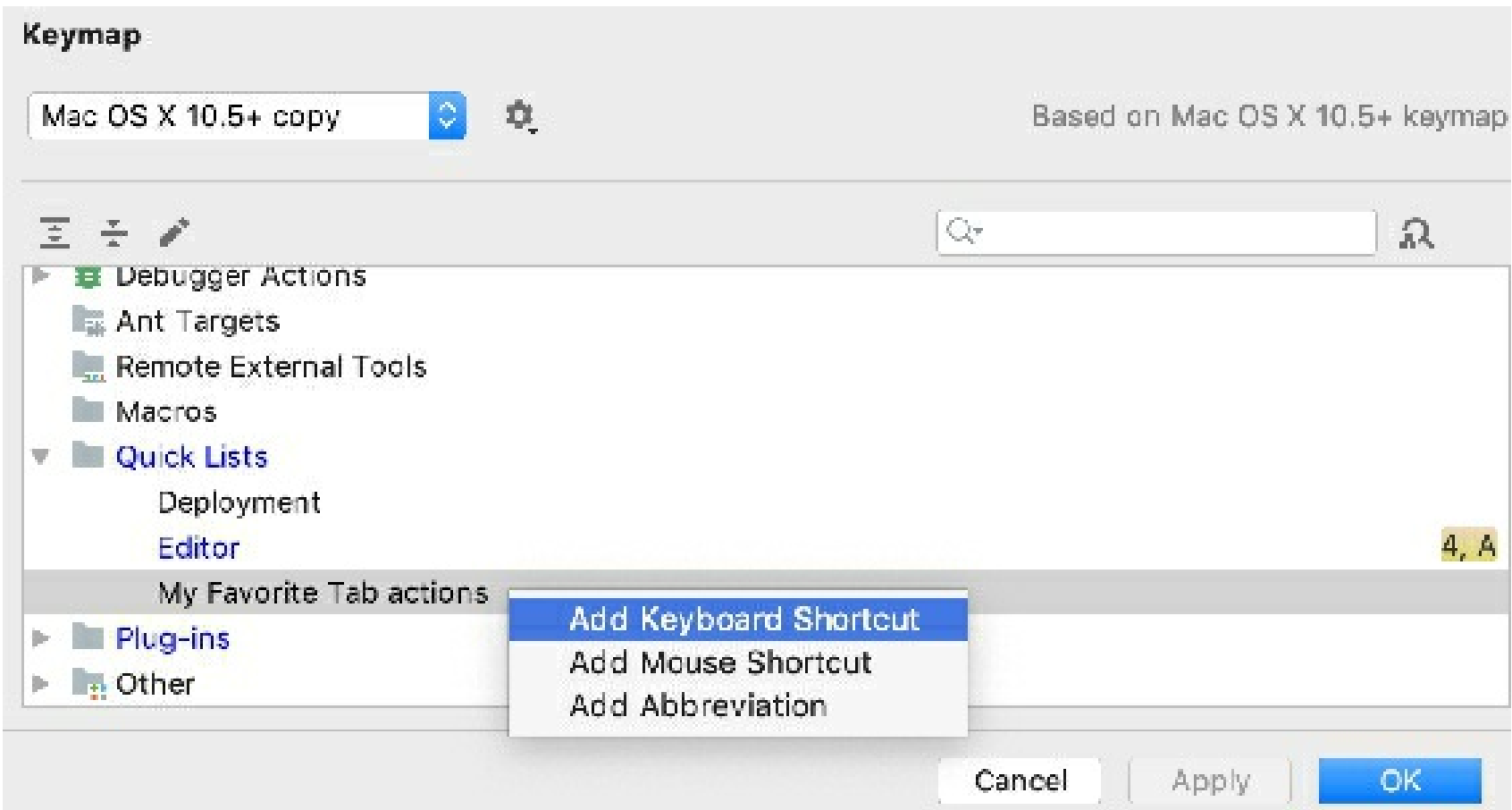
1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | Quick Lists**.
2. Click  or press `Alt+Insert` on the left pane to create a new quick list.
3. In the **Display name** field, specify the name of the quick list. Optionally, provide the quick list description.
4. On the right pane, add and arrange the actions on the quick list:
 - Click  to add an action to the list.
 - Click  to add a separator line.
 - Click  to remove the selected action from the list.
 - Click  or  to move the selected item up or down.



5. Click **OK** to save the changes.

Assign a shortcut to a quick list

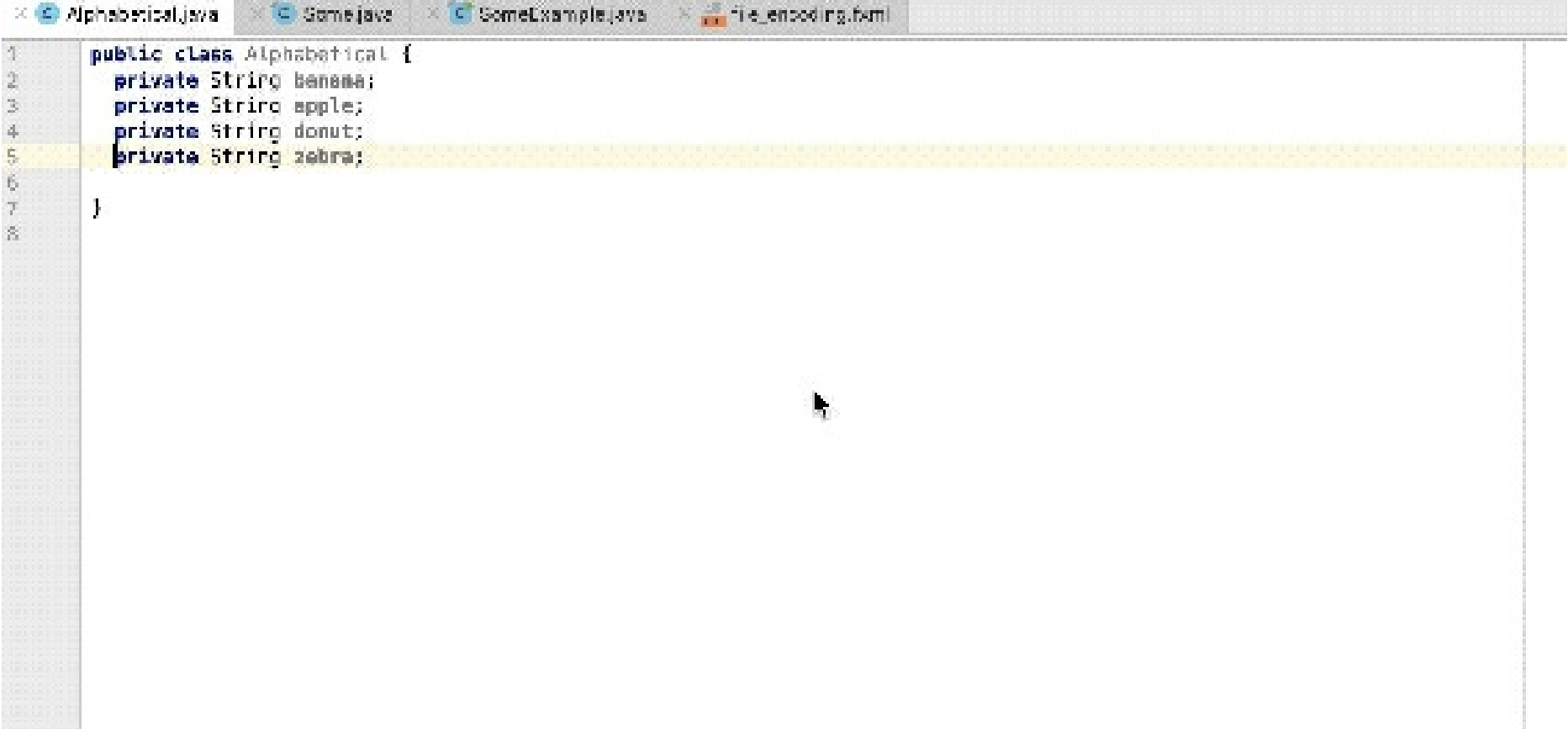
1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Keymap**.
2. Expand the **Quick Lists** node and select your quick list. Add a keyboard shortcut and save the changes.



For more information, see [Configure keyboard shortcuts](#).

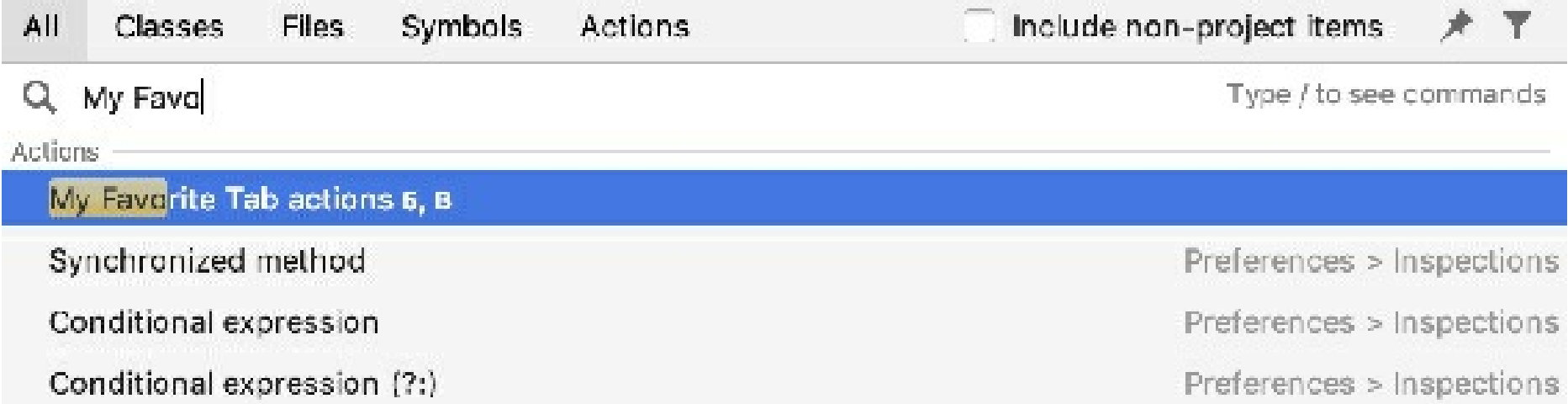
3. Click **OK** to save the changes.

In the editor, access the quick list by the associated shortcut.



Gif

If you don't remember the shortcut, you can search for your quick list by its name. Press `shift` twice and type the name of the quick list.



User interface themes

The interface theme defines the appearance of windows, dialogs, buttons, and all visual elements of the user interface. By default, IntelliJ IDEA uses the Darcula theme, unless you changed it during the first run. The *interface theme* is not the same as the color scheme, which defines the colors, fonts, and syntax-highlight for various text resources: the source code, search results, and so on.

Change the UI theme

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | Appearance**.
2. Select the UI theme from the **Theme** list:
 - **IntelliJ Light**: Traditional light theme for IntelliJ-based IDEs
 - **macOS Light** or **Windows 10 Light**: OS-specific light theme available as a bundled plugin
 - **Darcula**: Default dark theme
 - **High contrast**: Theme designed for users with color vision deficiency



Select **Sync with OS** to let IntelliJ IDEA detect the current system settings and use the default dark or light theme accordingly.

You can install a custom theme from the JetBrains Plugin Repository as described in Managing plugins. It is also possible to create your own UI themes for IntelliJ IDEA and customize the built-in themes. For more information, see IntelliJ Platform SDK Documentation.

Productivity tips

Use the quick switcher

1. Press `Ctrl+`` to execute the **View | Quick Switch Scheme** action.
2. In the **Switch** popup, select **Theme**, and then select the required interface theme.

Create shortcuts

- You can map the **Theme** action to your preferred key combination as described in Configure keyboard shortcuts.

IDE viewing modes

IntelliJ IDEA provides special viewing modes for specific usage patterns. For example, if you need to focus on the code or present your code to an audience.

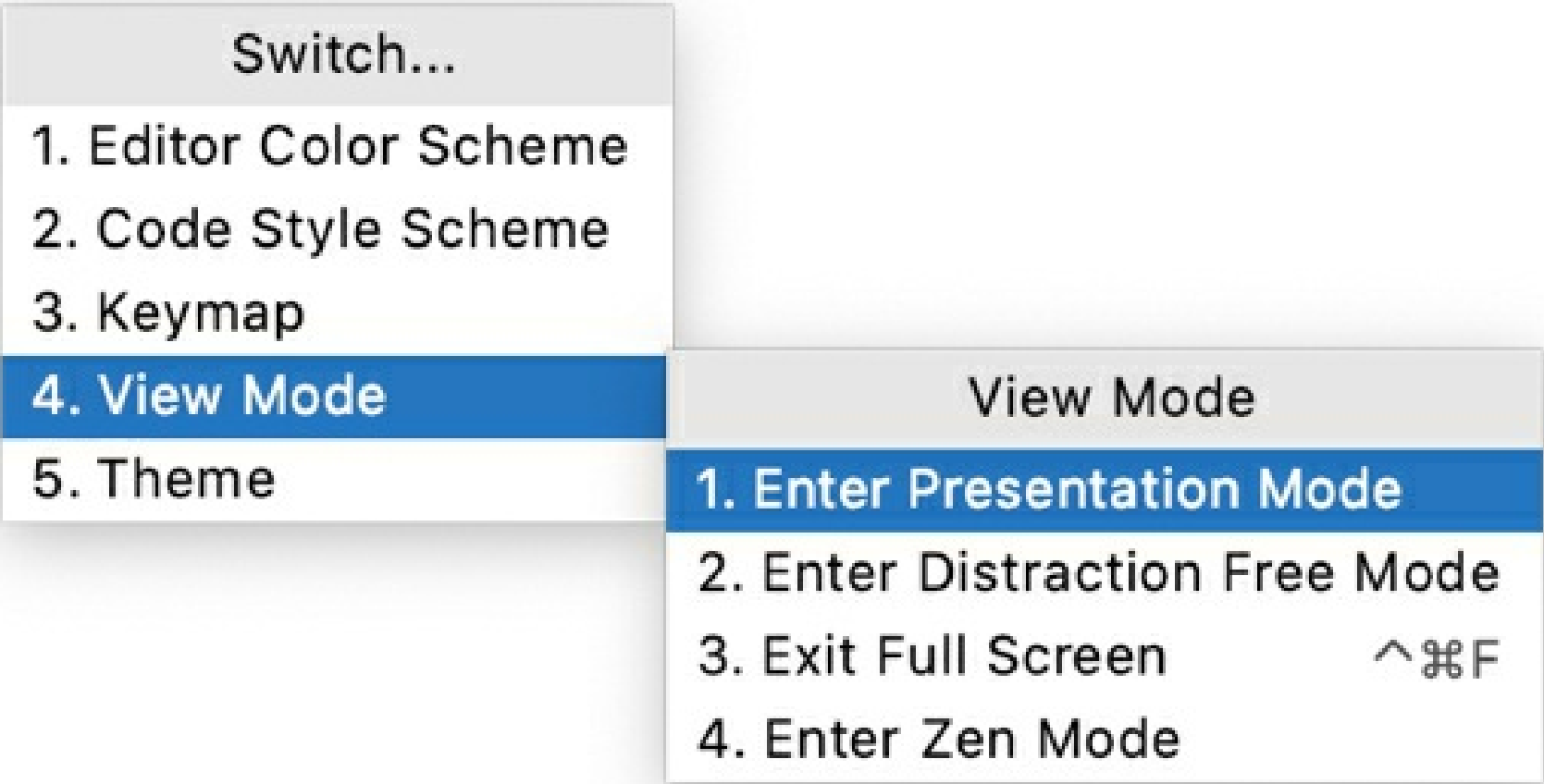
You can enter and exit viewing modes using actions under **View | Appearance** in the main menu.

- In **Full Screen** mode N/A, IntelliJ IDEA expands the main window to occupy the entire screen. On macOS, all operating system controls are hidden, but you can access the main menu if you hover the mouse pointer over the top of the screen.
- In **Distraction-free** mode, the editor occupies the entire main window with the source code centered. All other elements of the UI are hidden (tool windows, toolbars, and editor tabs) to help you focus on the source code of the current file. You can still use shortcuts to open tool windows, navigate, and perform other actions.
- In **Zen** mode, IntelliJ IDEA combines the **Full Screen** and **Distraction-free** modes, so the main window expands leaving only the editor with the source code for you to focus on programming.
- In **Presentation** mode, IntelliJ IDEA expands the editor to occupy the entire screen and increases the font size to make it easier for your audience to see what you are doing. Other elements of the UI are hidden, but you can bring them up with corresponding shortcuts or using the main menu if you hover the mouse pointer over the top of the screen.

Productivity tips

Use the quick switcher

1. Press `Ctrl+`` to execute the **View | Quick Switch Scheme** action.
2. In the **Switch** popup, select **View Mode**, and then select the required viewing mode.



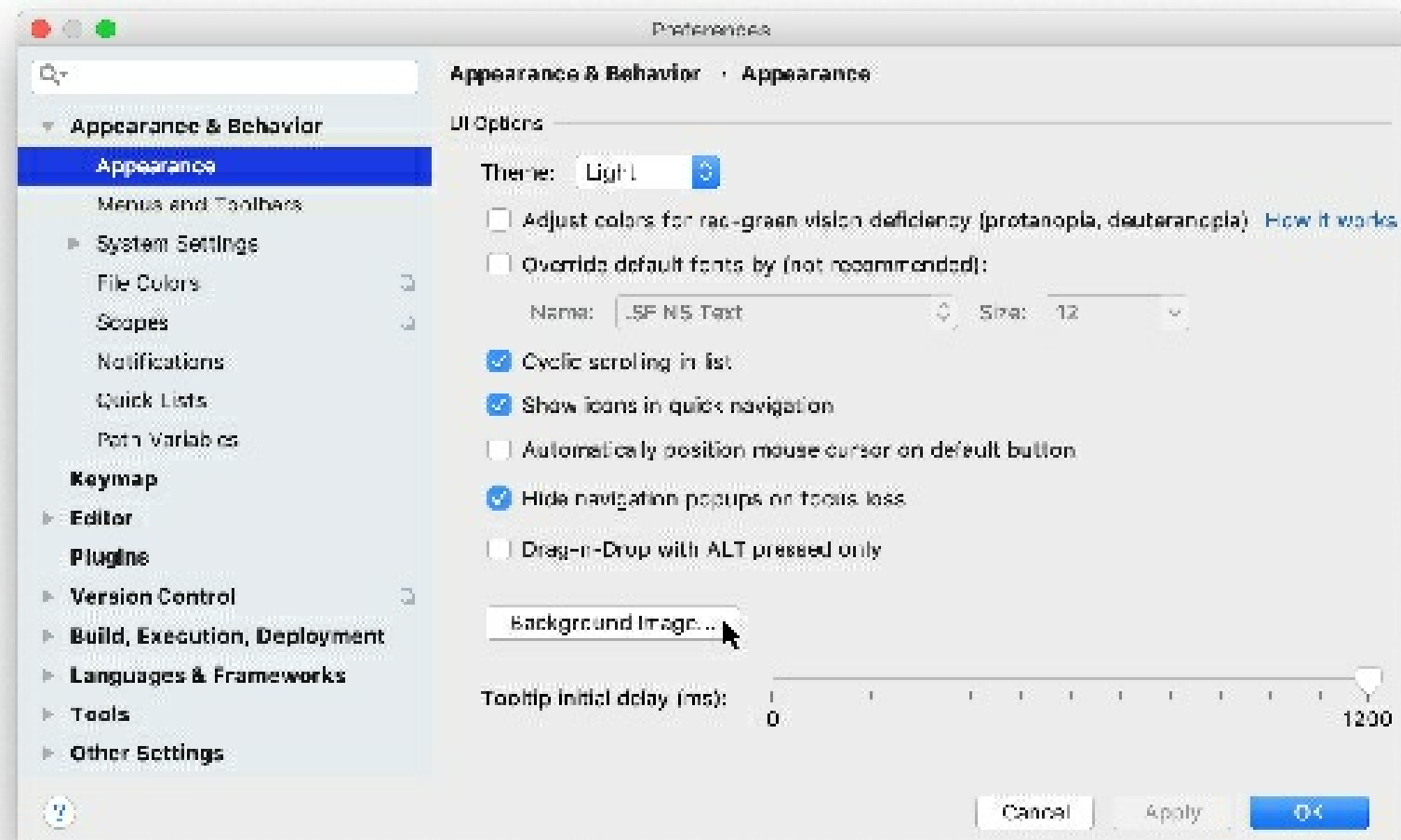
Create shortcuts

- You can map actions that toggle viewing modes to your preferred key combinations as described in [Configure keyboard shortcuts](#).

Background image

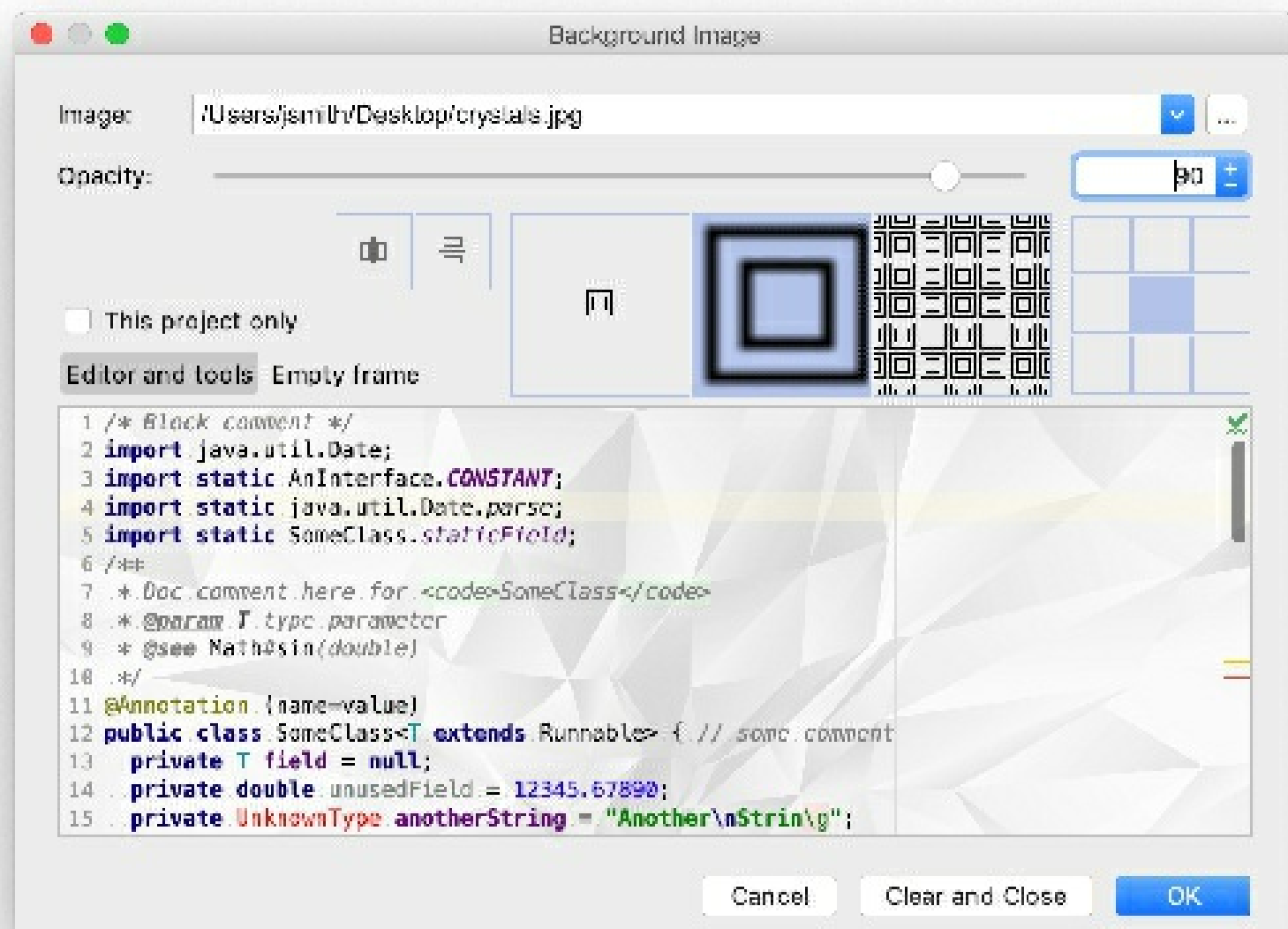
You can set any image as a custom background for the editor and all tool windows in IntelliJ IDEA.

1. Open the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | Appearance**, and click the **Background Image** button.



2. In the **Background Image** dialog, specify the image you want to use as the background, its opacity, filling and placement options. If necessary, mirror the image vertically or horizontally. You can set separate images for the editor and tool windows, and for the empty frame (when no files are opened in the editor).

You can also specify a URL of the desired image.



If you want to set the image only for the current project, select the **This project only** checkbox.

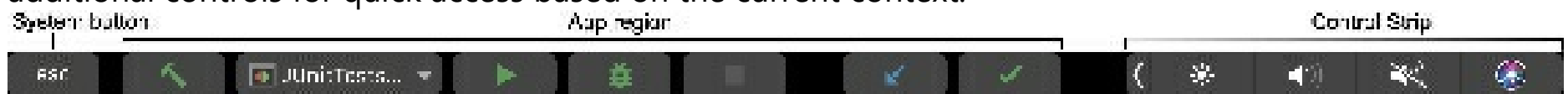
3. Click **OK** to apply the changes.

To remove the image and use the default background for your selected theme, click **Clear and Close**.

If you open an image in the editor or add it to the **Project** tool window, you can right-click the image and click **Set Background Image** in the context menu. You can also use the *Find Action* command Ctrl+Shift+A to execute the **Set Background Image** action.

Touch bar support

The Touch Bar is located above the keyboard on supported Apple MacBook Pro models. It provides additional controls for quick access based on the current context.



The *Control Strip* in the right part of the Touch Bar includes controls for system-level tasks that are usually accessed with function keys on classic keyboards. The *app region* to the left of the Control Strip contains controls that are specific to IntelliJ IDEA and the current context. One more *system button* is located in the left part of the Touch Bar; this is usually the Escape key, but it can be something else depending on the context.

IntelliJ IDEA provides the following contexts:

- *Default context* is used most of the time.

It includes controls for running, building, and debugging the application with the ability to quickly select or create a new run/debug configuration. It also provides VCS controls for updating your project and committing changes, which can be replaced in some contexts (for example, with the refresh action when focus is on the Maven tool window or the Gradle tool window).



For more controls, you can use modifier keys: Ctrl, Alt, Shift, and ⌘ .

- *Debugger context* is used when focus is on the Debug tool window.

It includes controls to stop, pause, resume the debugger, as well as stepping and evaluating expressions.



For more controls, hold down the Alt key.

- When focus is on a dialog, confirmation controls are displayed (for example, **Cancel**, **Apply**, **OK**, or other relevant buttons).



- When you start typing inside a popup with a list of actions, the actions are filtered according to what you type (for example, in the **Project** tool window, when you press Alt+Insert, you can filter the types of files you would like to create). When the popup is active, the touch bar contains the same list of items, and it is filtered accordingly as you type.



Customize the Touch Bar

You can configure the controls displayed on the Touch Bar in the default and debugger context.

1. In the **Settings/Preferences** dialog $\text{Ctrl}+\text{Alt}+\text{S}$, go to **Appearance & Behavior | Menus and Toolbars**.
2. Expand the **Touch Bar** node and configure the controls for corresponding contexts and modifier keys.

The **Touch Bar** node is available only if you are using an Apple MacBook Pro with the Touch Bar.

3. Apply changes when finished.

To show the function keys (F1, F2, and so on) on the Touch Bar, hold down the `Fn` key. You can also make function keys display permanently for selected applications, as described in the following Apple support article.

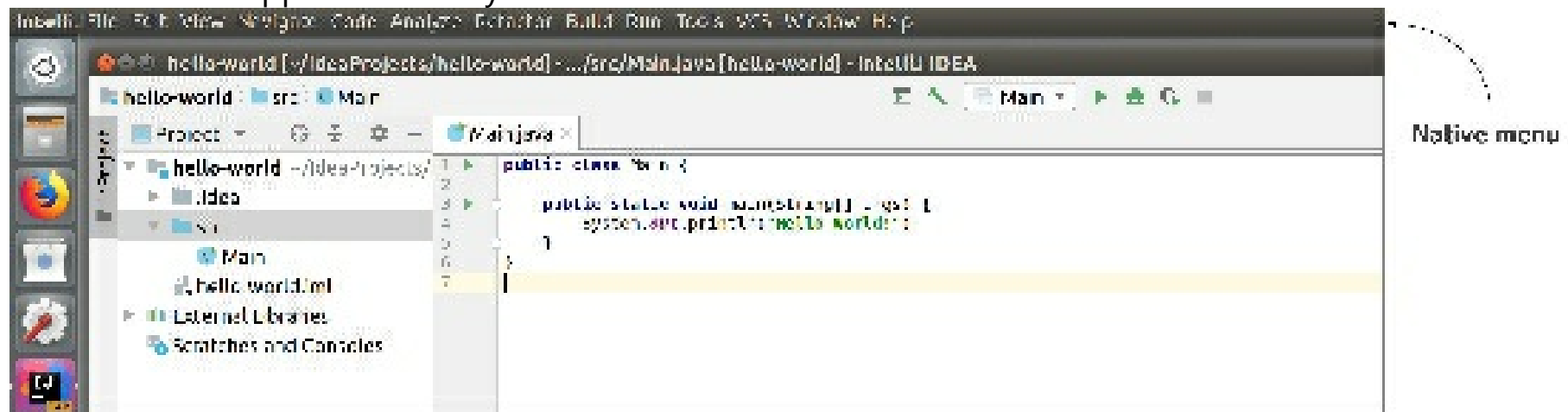
IntelliJ IDEA provides an option to always show function keys without the need to change system settings:

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Keymap**.
2. Select **Show F1, F2, etc. keys on the Touch Bar** at the bottom.

Linux native menu

This is an experimental feature in IntelliJ IDEA.

In IntelliJ IDEA for Linux, you can use the global main menu similar to macOS – a horizontal menu attached to the top of the screen. By default, this menu is enabled, although not all Linux desktop environments support it natively.



Disable the native menu

1. Press **Ctrl+Shift+A** to open the **Find Action** dialog, then type `Experimental features`, and press Enter.
2. Clear the checkbox next to the **linux.native.menu** item, apply the changes, and close the dialog.
3. Restart IntelliJ IDEA.

Troubleshoot

The native menu might not display on some desktop environments. If you have followed the steps described on this page, but the menu doesn't appear, install JavaFX Runtime for Plugins from the JetBrains plugin repository.

Configuring code style

If certain coding guidelines exist in a company, one has to follow these guidelines when creating source code. IntelliJ IDEA helps you maintain the required code style.

Code styles are defined at the *project level* and at the *IDE level* (global).

- At the project level, settings are grouped under the **Project** scheme, which is predefined and is marked in bold. The **Project** style scheme is applied to the current project only.

You can copy the **Project** scheme to the IDE level, using the **Copy to IDE** command.

- At the IDE level, settings are grouped under the predefined **Default** scheme (marked in bold), and any other scheme created by the user by the **Duplicate** command (marked as plain text). Global settings are used when the user doesn't want to keep code style settings with the project and share them.

You can copy the IDE scheme to the current project, using the **Copy to Project** command.

Configure code style for a language

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Code Style**.
2. Select the language-specific settings page.
3. Choose the code style **Scheme** to be used as a base for your custom code style for the selected language.
4. Browse through the tabs of the selected language page, and configure code style preferences for it.

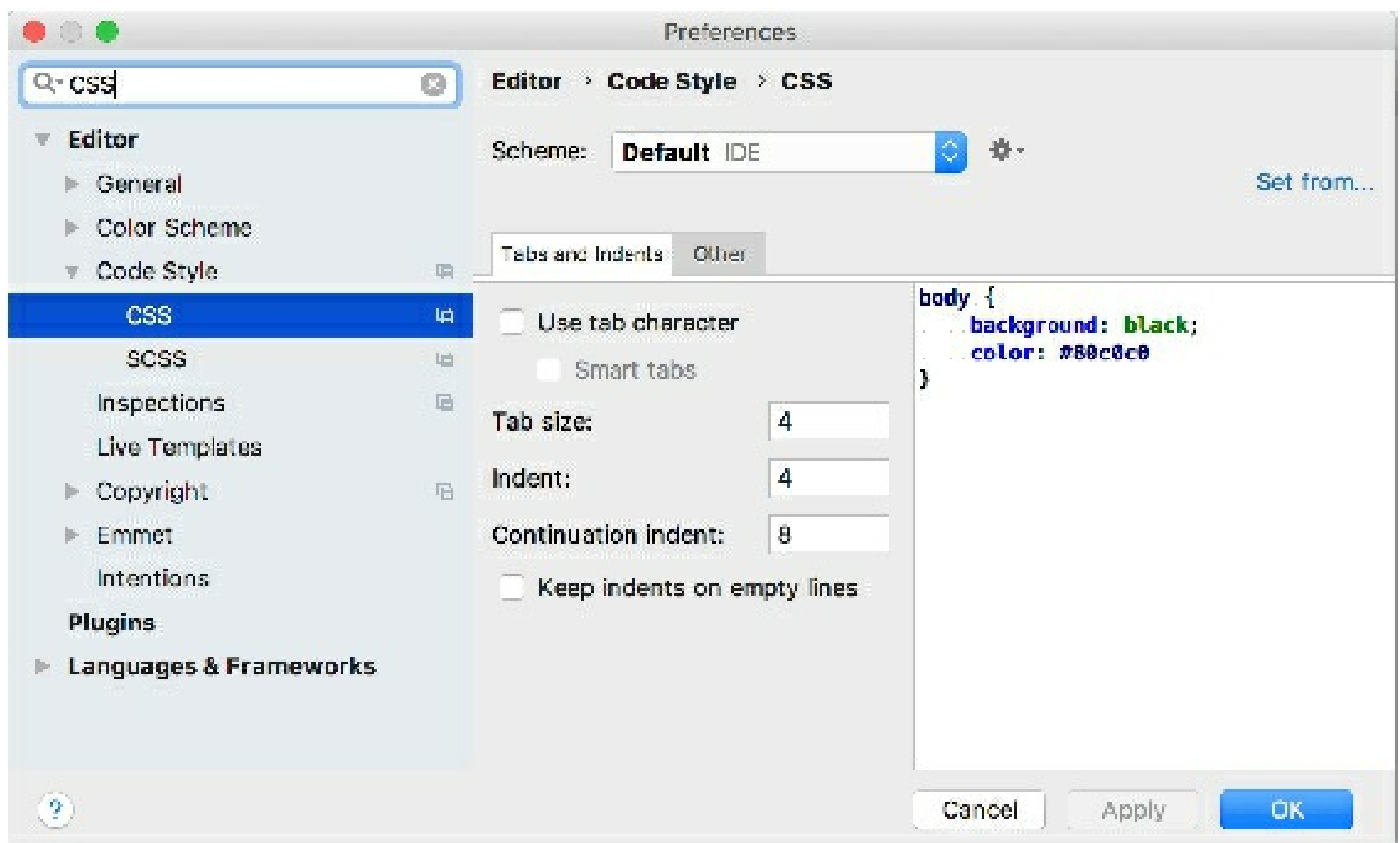
Copy code style settings from other languages

For most of the supported languages, you can copy code style settings from other languages or frameworks.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Code Style**.
2. Select the language-specific settings page.
3. Click **Set From** in the upper-right corner.

The link is shown for those languages only, where defining settings on the base of other languages is applicable.

4. From the list that appears, select the language to copy the code style from.



Apply framework-specific pre-configured coding standards

For PHP files, you can have framework-specific pre-configured coding standards applied.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Code Style**.
2. Select a language-specific page.
3. Click the **Set from** link, choose **Predefined**, then choose the relevant pre-configured standard.

Manage code style on a directory level with EditorConfig

To use EditorConfig, make sure the *EditorConfig* plugin is enabled on the **Settings/Preferences | Plugins** page, tab **Installed**, see Managing plugins for details.

IntelliJ IDEA allows you to manage all code style settings for each individual set of files with EditorConfig support (enabled by default in the **Settings/Preferences** dialog **Ctrl+Alt+S**). All you need to do is place an **.editorconfig** file in the root directory containing the files whose code style you want to define. You can have as many **.editorconfig** files within a project as needed, so you can specify different styles for different modules.

All options from the **.editorconfig** file are applied to the directory where it resides as well as all of its sub-directories on top of the current project code style. If anything is not defined in **.editorconfig**, it's taken from the project settings.

All options in the **.editorconfig** file are divided into the following categories:

- Standard options such as `indent_size`, `indent_style`, and so on. These options do not have any domain specific prefixes.
- Generic IntelliJ options that have the `ij_` prefix and are applicable to all languages:

- `ij_visual_guides`
- `ij_formatter_off_tag`
- `ij_formatter_on_tag`
- `ij_formatter_tags_enabled`
- `ij_wrap_on_typing`
- `ij_continuation_indent_size`
- `ij_smart_tabs`

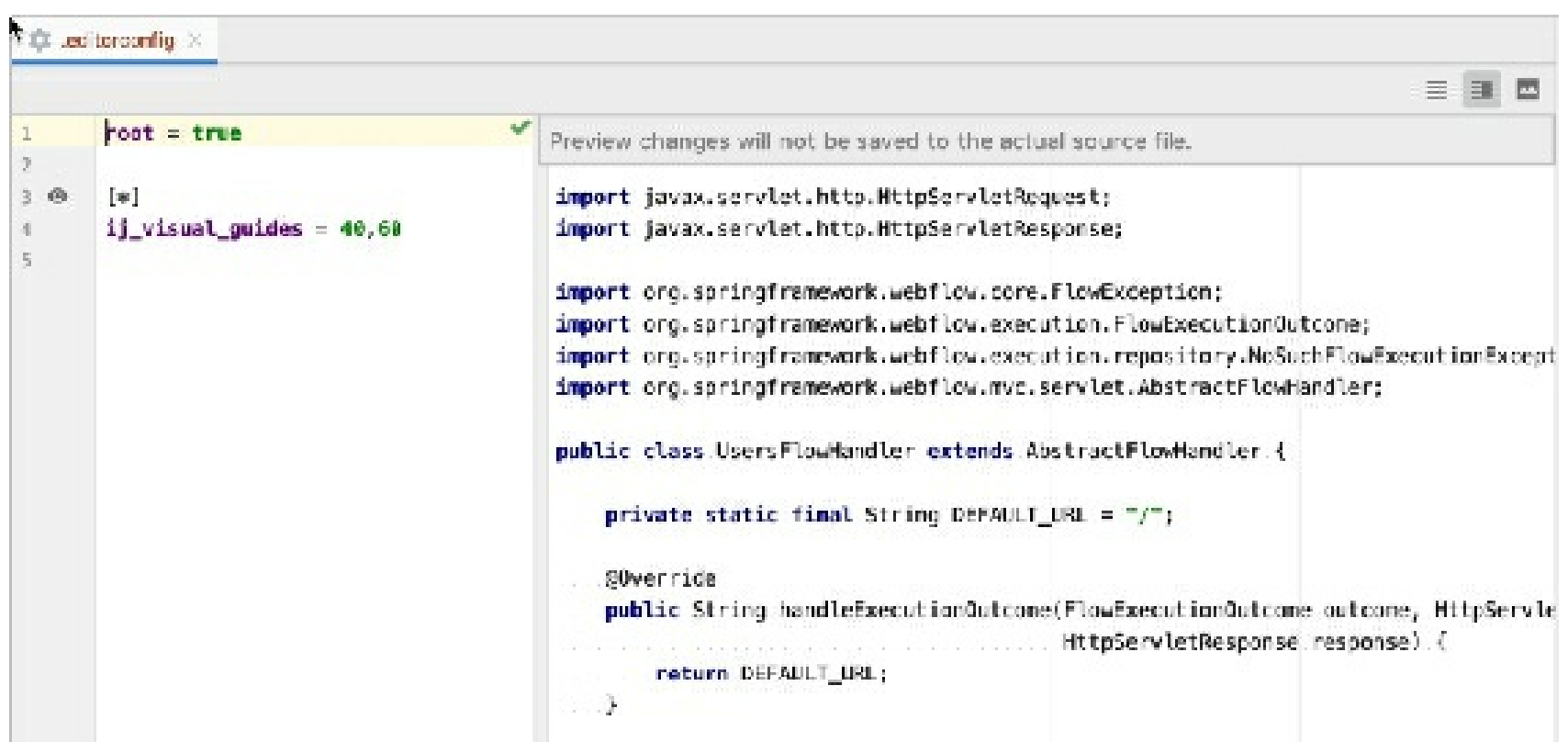
- Common IntelliJ options supported by many (but not all) languages. They start with the `ij_any` prefix, for example, `ij_any_brace_style`.

- IntelliJ language-specific options starting with the `ij_<lang>_` prefix where `<lang>` is the language domain ID (normally a low-case language name), for example, `ij_java_blank_lines_after_imports`.

The same options can be defined as a common option and a language-specific option, for example, `ij_<...>_brace_style`. Language-specific options have higher priority over common or generic options.

Add an `.editorconfig` file

1. In the **Project** view, right-click a source directory containing the files whose code style you want to define and choose **New | EditorConfig** from the context menu.
2. Select the properties that you want to define so that IntelliJ IDEA creates stubs for them, or leave all checkboxes empty to add the required properties manually.
3. To preview how changes to your code style settings will impact the actual source files, click in the gutter of the `.editorconfig` file and select a source file affected by it. The preview will open on the right.



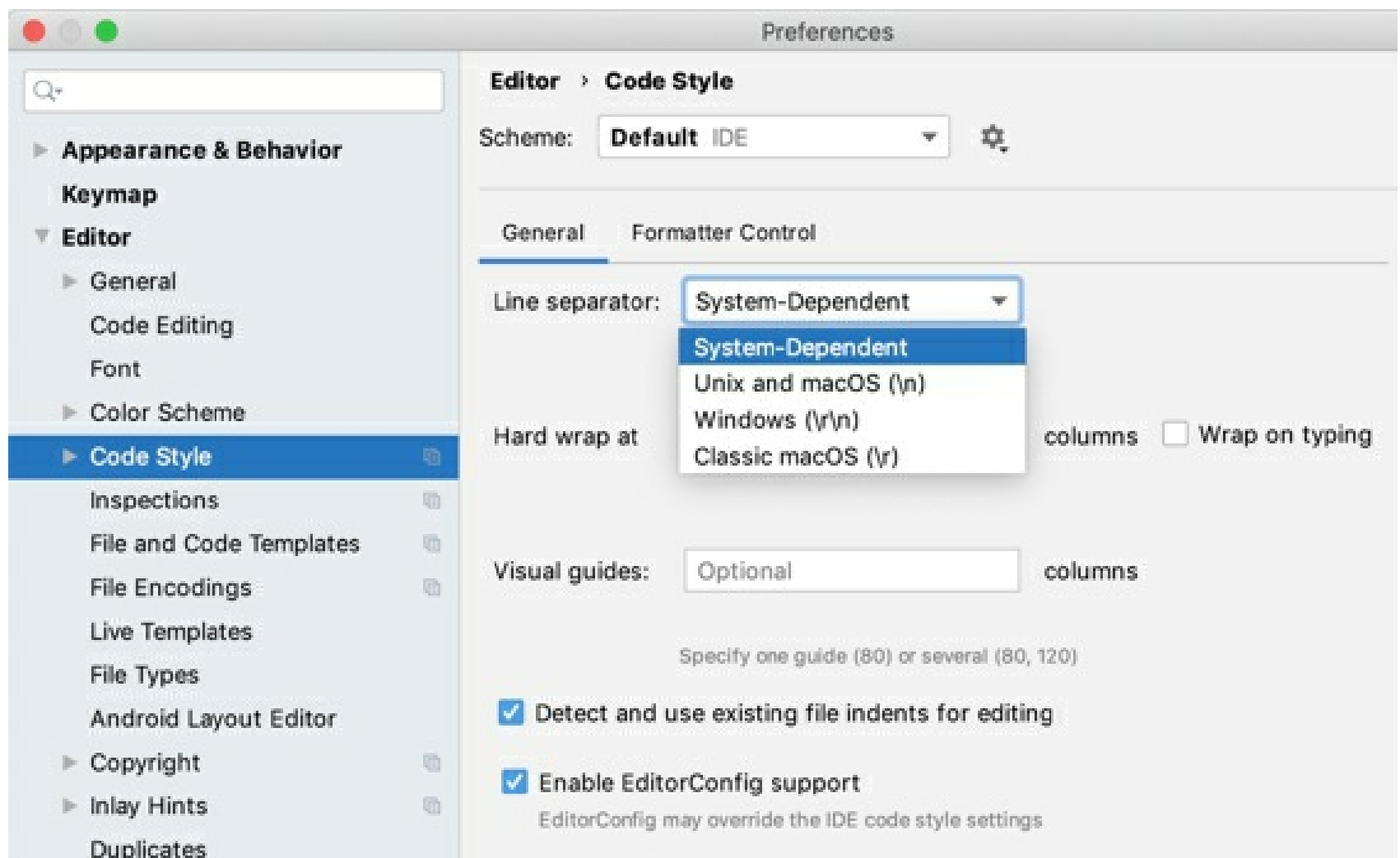
You can make changes in the preview pane to try and test how your configuration changes are reflected without worrying about making unwanted changes to the source code: all these changes are discarded when you close the `.editorconfig` file.

Configuring Line Separators

With IntelliJ IDEA, you can set up line separators (line endings) for newly created files, and change line separator style for existing files.

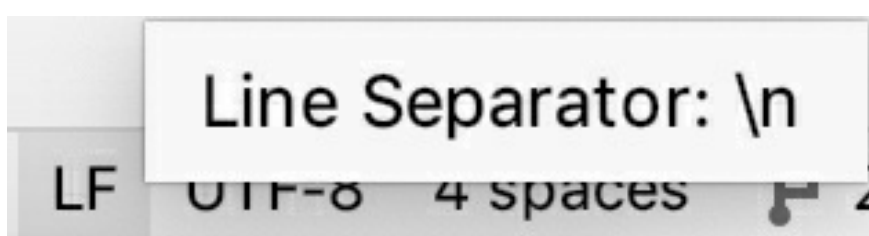
Set up line separators for new files

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Code Style**.
2. To configure line separators for new projects, go to **File | New Projects Settings | Settings/Preferences for New Projects | Editor | Code Style**.
3. From the **Line separator** list, select the line separator style you want to apply.



Check line separator style for the current file

- The line separator style applied to the current file is indicated in the status bar, for example:



Change line separator for a file currently opened in the editor

- From the main menu, choose **File | File Properties | Line Separators** and choose a line ending style from the list.

Change line separator for a file or directory selected in the Project view

1. Select a file or directory in the Project tool window **Alt+1**.

Note that if a directory is selected, the line ending style applies to all nested files recursively.

2. From the main menu, choose **File | File Properties | Line Separators**, and then select a line ending style from the list.

Productivity tips

- Use multiple selection in the Project view.

- Changing line separator is reflected in the Local history of a file.
- Run the inspection 'Inconsistent line separators' to find out, which files use line separator different from project's default.

Copy code style settings

You can define the code styles that differ from the pre-defined ones. These code style schemes are stored in XML files, in the **config/codestyles** folder under the user home directory.

You can use the created copy for modifying code styles, and for export.

If you select a code style scheme other than **Project**, then this code style will be saved for a project. Thus you can assign a global (IDE) code style for each project.

Create a copy of code style settings

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Code Style**, select the desired scheme from the list, and click .
2. Select one of the following options:

- **Copy to IDE:** select this option to store the selected scheme in the global level.

IntelliJ IDEA saves the new code style with the specified name to the **config/codestyles/<code_style_name>.xml** file under the IntelliJ IDEA home directory.

- **Copy to Project:** select this option to store the selected scheme in the project level.

The selected code style is saved to: **.idea/codeStyles/codeStyleConfig.xml**.

- **Duplicate:** select this option to make a copy of the selected scheme and store it in the same level.

3. In the **Scheme** field, type the name of the new scheme and press `Enter` to save the changes.
4. Apply changes and close the dialog.

IntelliJ IDEA lets you modify existing names of code style schemes, export or import code style settings.

Manage a code style scheme

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Code Style**, select the desired scheme from the list, and click .
2. From the list, select one of the following options:

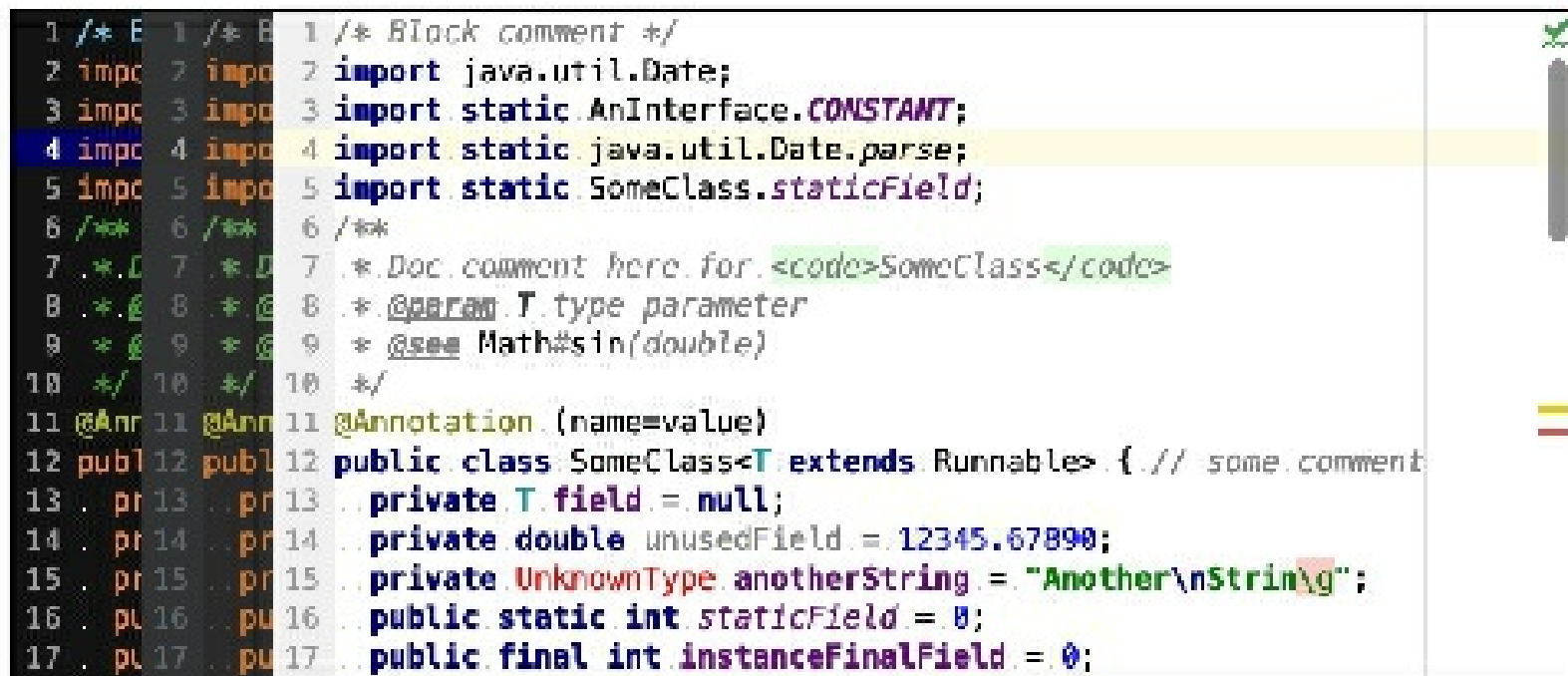
- **Rename:** change the name of the selected scheme.
- **Export:** export your code style settings to the desired location.
- **Import:** import IntelliJ IDEA XML code style settings, JSCS config file, or Eclipse XML Profile.

3. In the **Scheme** field, type the name of the new scheme and press `Enter` to save the changes.
4. Apply changes and close the dialog.

Colors and fonts

As a developer, you work with a lot of text resources: the source code in the editor, search results, debugger information, console input and output, and so on. Colors and font styles are used to format this text and help you better understand it at a glance.

IntelliJ IDEA uses *color schemes* that define the preferred colors and fonts.

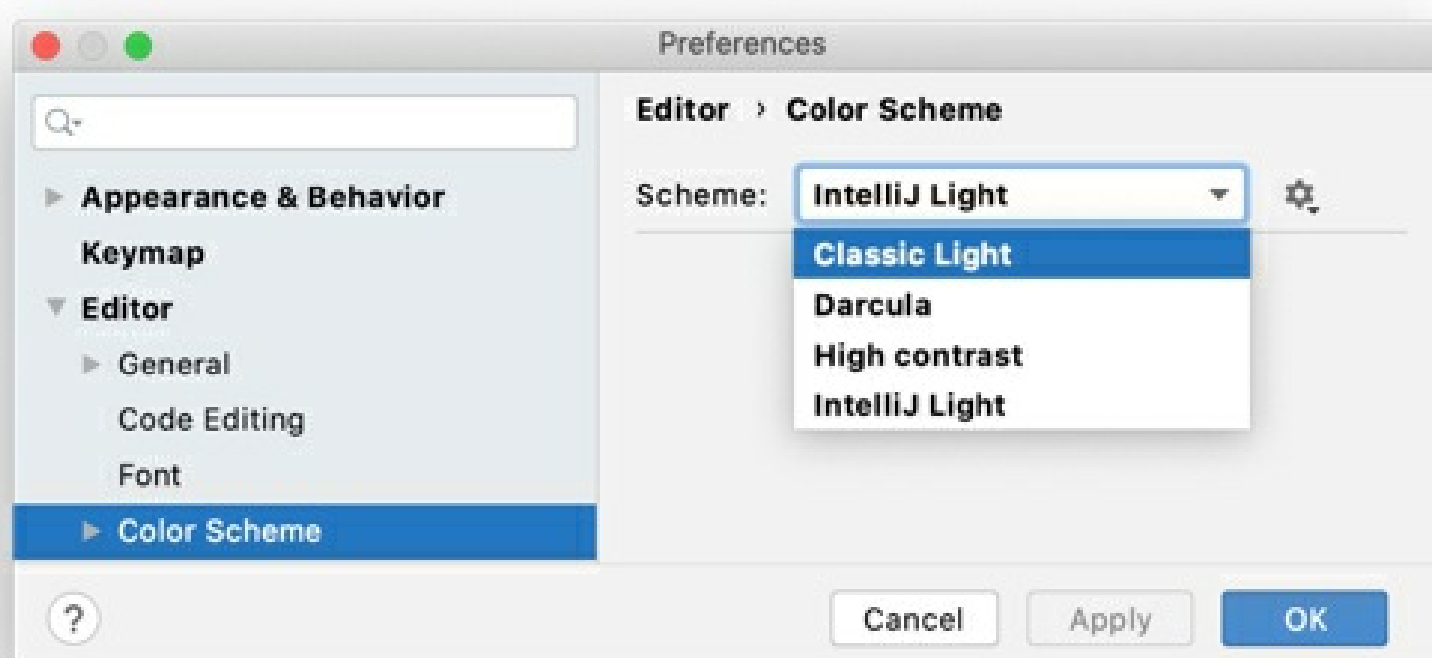


A *color scheme* is not the same as the interface theme, which defines the appearance of windows, dialogs, and controls.

You can use a predefined color scheme or customize it to your liking. It is also possible to share schemes.

Select a color scheme

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme**.
2. Use the **Scheme** list to select a color scheme.



By default, there are the following predefined color schemes:

- **Classic Light**: designed for the *macOS Light* and *Windows 10 Light* interface themes
- **Darcula**: designed for the *Darcula* interface theme

- **High contrast:** designed for the *High contrast* interface theme (recommended for users with sight deficiency)
- **IntelliJ Light:** designed for the *IntelliJ Light* interface theme

If you install a plugin with a color scheme, that scheme will be added to the list of predefined schemes. For more information, see [Share color schemes](#).

Customize a color scheme

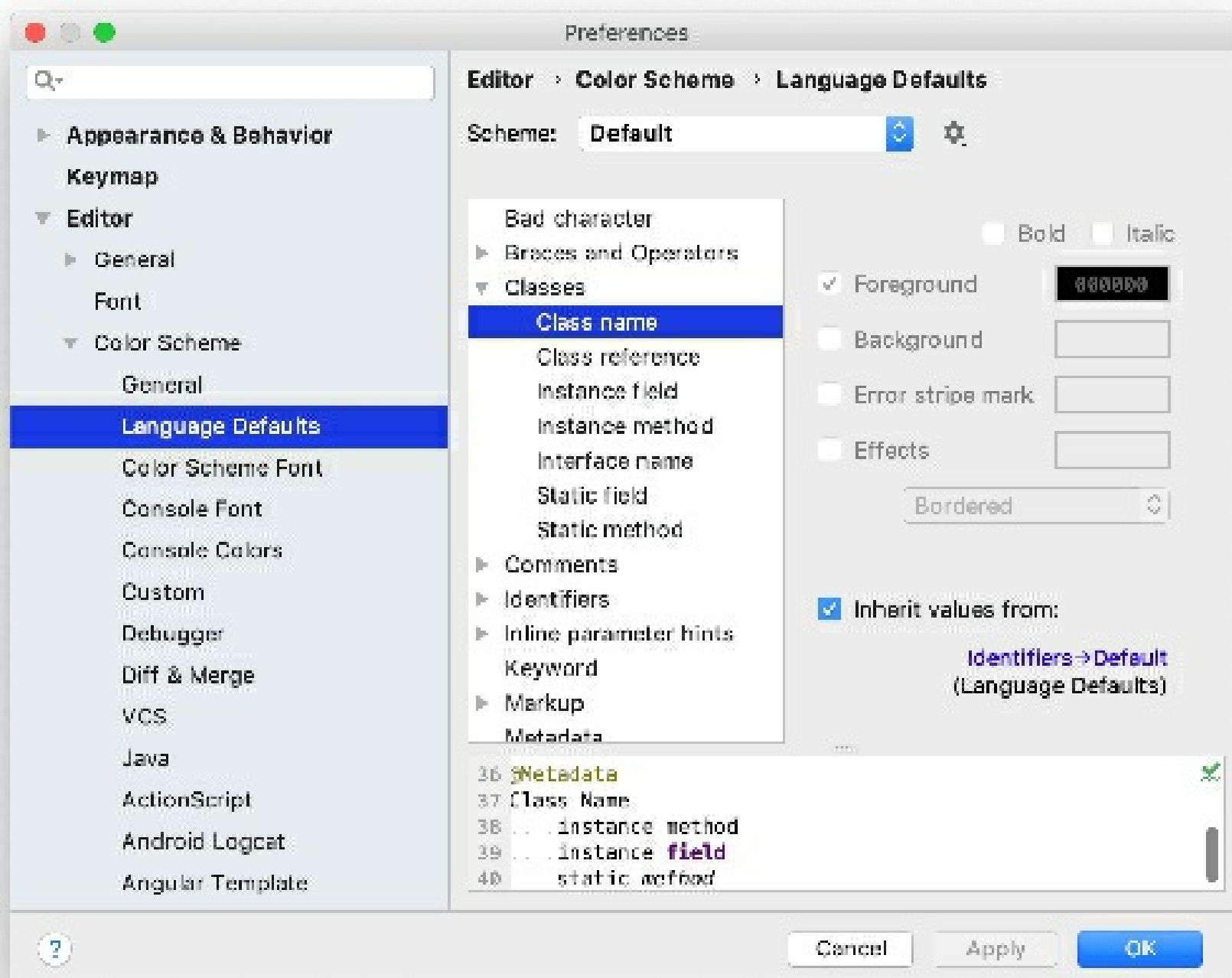
You can customize a predefined color scheme, but it is recommended to create a duplicate for your custom color and font settings:

Duplicate a color scheme

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme**.
2. Select a color scheme, click **⌵**, and then click **Duplicate**.
3. (Optional) To rename your custom scheme, click **⌵** and select **Rename**.

Predefined color schemes are listed in bold font. If you customize a predefined color scheme, it will be displayed in blue. To restore a predefined color scheme to default settings, click **⌵** and select **Restore Defaults**. You cannot remove predefined color schemes.

To define color and font settings, open the **Editor | Color Scheme** page of the **Settings/Preferences** **Ctrl+Alt+S**. The settings under **Editor | Color Scheme** are separated into sections. For example, the **General** section defines basic editor colors, such as the gutter, line numbers, errors, warnings, popups, hints, and so on. The **Language Defaults** section contains common syntax highlighting settings, which are applied to all supported programming languages by default. In most cases, it is sufficient to configure **Language Defaults** and make adjustments for specific languages if necessary. To change inherited color settings for an element, clear the **Inherit values from** checkbox.

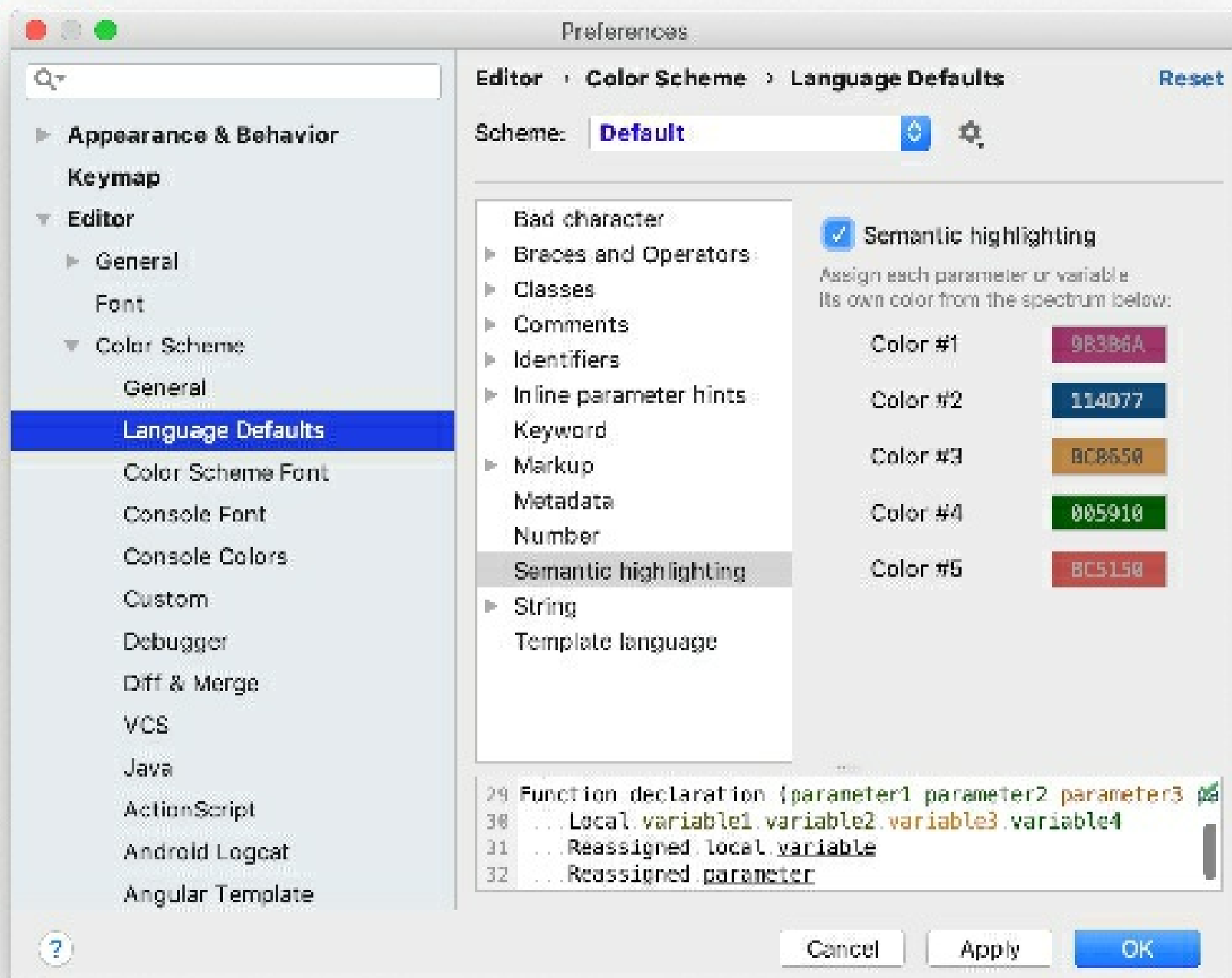


Semantic highlighting

By default, the color scheme defines syntax highlighting for reserved words and other symbols in your source code: operators, keywords, suggestions, string literals, and so on. If you have a function or method with many parameters and local variables, it may be hard to distinguish them from one another at a glance. You can use *semantic highlighting* to assign a different color to each parameter and local variable.

Enable semantic highlighting

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme | Language Defaults | Semantic highlighting**.
2. Select the **Semantic highlighting** checkbox and customize the color ranges if necessary.



This will enable semantic highlighting for all languages that inherit this setting from **Language Defaults**. To enable it for a specific language instead (for example, Java) go to the **Editor | Color Scheme | Java | Semantic highlighting** page of the **Settings/Preferences** `Ctrl+Alt+S`, clear the **Inherit values from** checkbox, and select the **Semantic highlighting** checkbox.

Share color schemes

If you are used to a specific color scheme, you can export it from one installation and import it to another one. You can also share color schemes with other developers. If necessary, you can import your favorite color settings from Eclipse.

Export a color scheme as XML

IntelliJ IDEA can save your color scheme settings as an XML file with the **.icls** extension. You can then import the file to another installation.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | Color Scheme**.
2. From the **Scheme** list, select a color scheme, click , then click **Export** and select **IntelliJ IDEA color scheme (.icls)**.
3. Specify the name and location of the file and save it.

Export a color scheme as a plugin

The plugin can be uploaded to the plugin repository for others to install. This format has several benefits over an XML file, including metadata, feedback, download statistics, and versioning (when you upload a new version of the plugin, users will be notified about it).

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | Color Scheme**.

2. From the **Scheme** list, select a color scheme, click , then click **Export** and select **Color scheme plugin .jar**.
3. In the **Create Color Scheme Plugin** dialog, specify the version details and vendor information. Then click **OK**.

When you install a plugin with a color scheme, that scheme will be added to the list of predefined schemes.

Import a color scheme

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme**.
2. From the **Scheme** list, select a color scheme, click , then click **Import Scheme**.

Fonts

To customize the default font, open the **Editor | Font** page of the **Settings/Preferences** **Ctrl+Alt+S**. This font is used and inherited in all color schemes by default.

Customize the color scheme font

You can set a different font for your current scheme.

This is not recommended if you are planning to share your scheme or use it on other platforms, which may not support the selected font. In such cases, use the default global font settings.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme | Color Scheme Font**.
2. Select the **Use color scheme font instead of the default** checkbox.

Customize the console font

By default, text in the console uses the same font as the color scheme. To use a different font in the console:

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | Color Scheme | Console Font**.
2. Select the **Use console font instead of the default** checkbox.

Productivity tips

See the color scheme settings for the current symbol

- Put the caret at the necessary symbol, press **Ctrl+Shift+A**, find the **Jump to Colors and Fonts** action, and execute it.

This will open the relevant color scheme settings for the symbol under the caret.

See which fonts are currently used in the editor

- Press **Ctrl+Shift+A**, find the **Show Fonts Used by Editor** action, and execute it.

This will open the **Fonts Used in Editor** dialog with a list of fonts.

Both the **Jump to Colors and Fonts** and the **Show Fonts Used by Editor** actions do not have a default shortcut. To assign a shortcut for an action, select it in the **Find Action** popup and press **Alt+Enter**.

Configure keyboard shortcuts

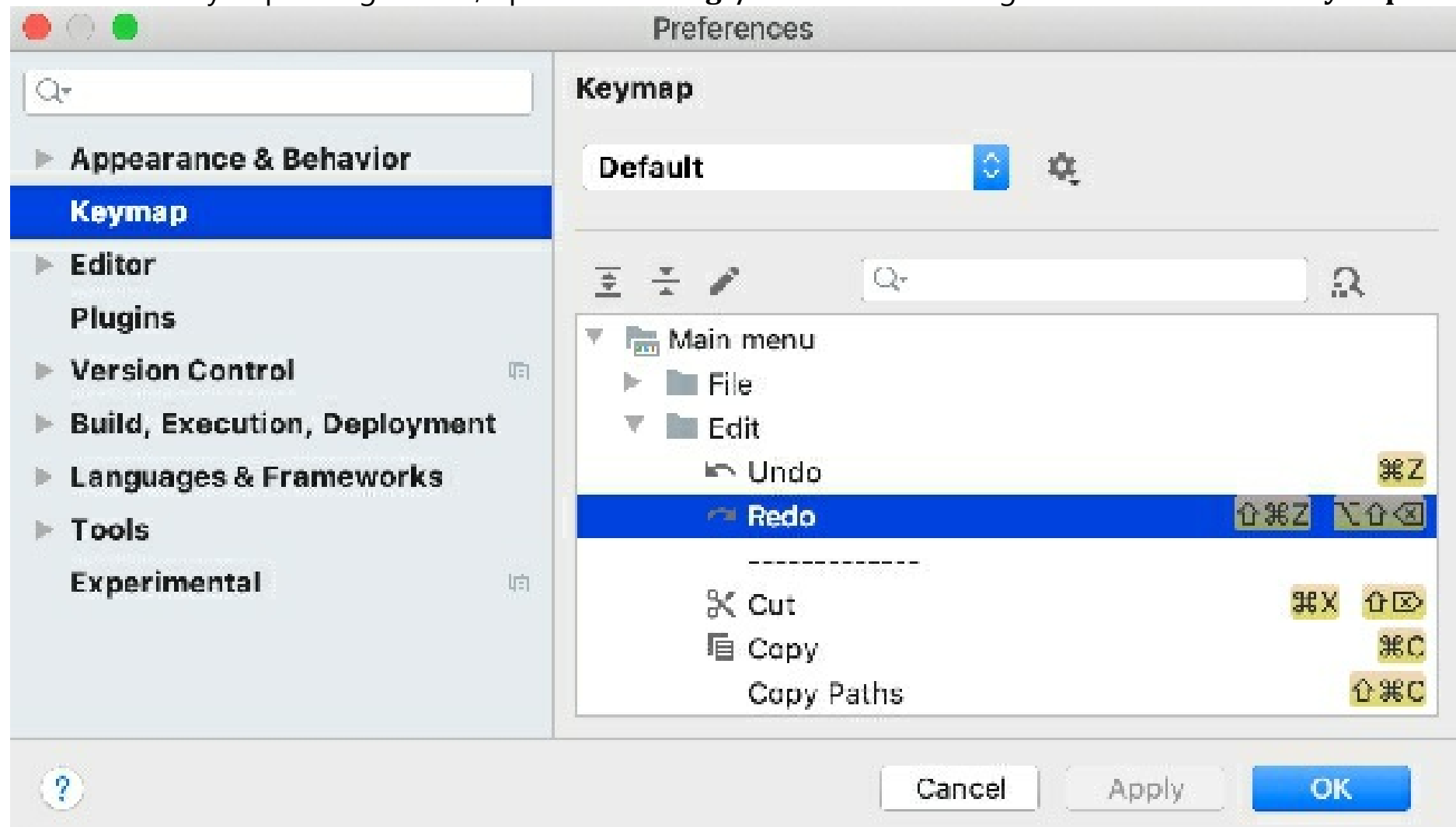
File | Settings | Keymap for Windows and Linux

IntelliJ IDEA | Preferences | Keymap for macOS

Ctrl+Alt+S

IntelliJ IDEA includes several predefined keymaps and lets you customize frequently used shortcuts.

To view the keymap configuration, open the **Settings/Preferences** dialog Ctrl+Alt+S and select **Keymap**.



IntelliJ IDEA automatically suggests a predefined keymap based on your environment. Make sure that it matches the OS you are using or select the one that matches shortcuts from another IDE or editor you are used to (for example, Eclipse or NetBeans).

You cannot change predefined keymaps. When you modify any shortcut, IntelliJ IDEA creates a copy of the currently selected keymap, which you can configure. Click to duplicate the selected keymap, rename, remove, or restore it to default values.

A custom keymap is not a full copy of its parent keymap. It inherits unmodified shortcuts from the parent keymap and defines only those that were changed. For information about the keymap files, see Location of user-defined keymaps.

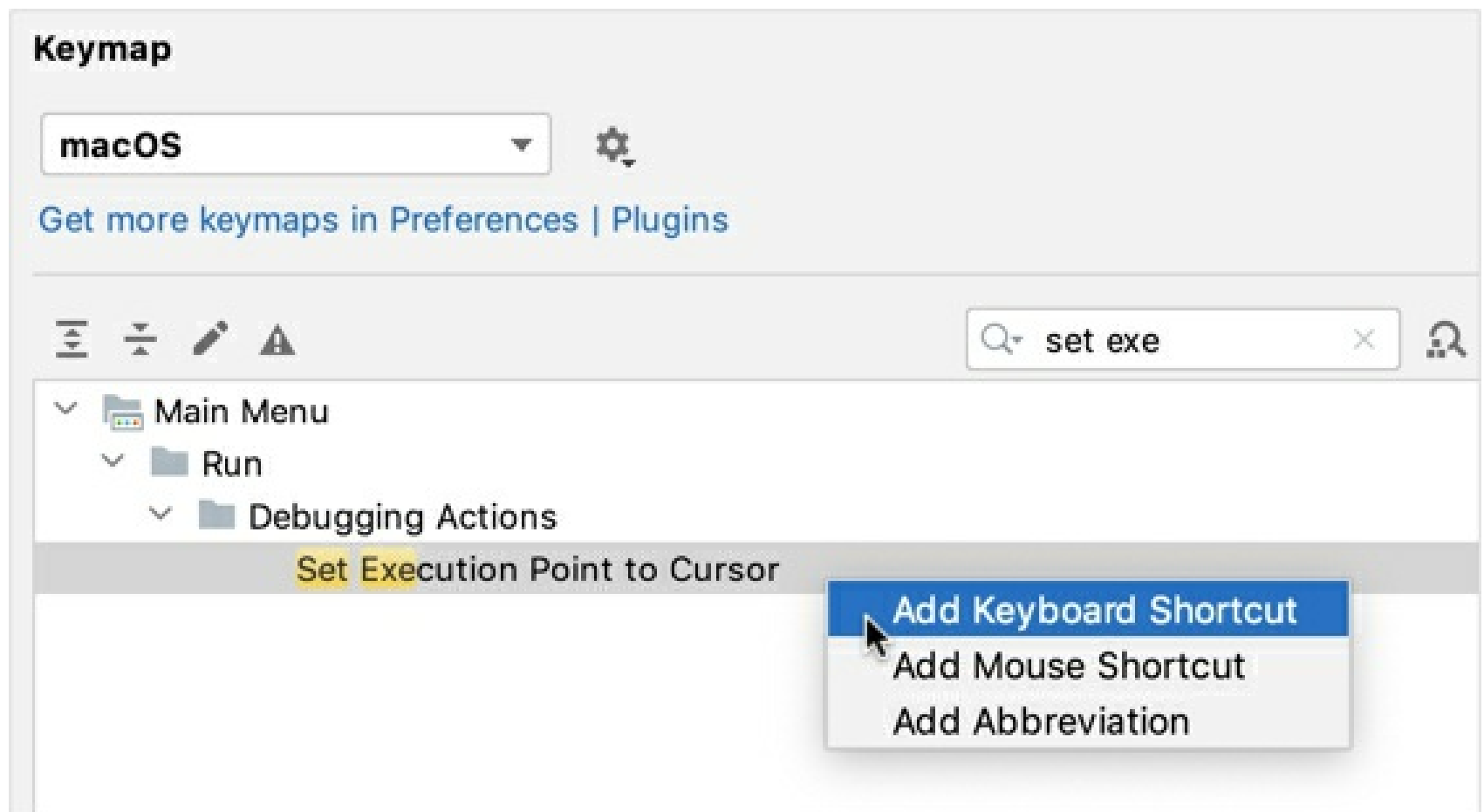
The keymap itself is a list of actions with corresponding keyboard and mouse shortcuts, and abbreviations. To find an action by name, type it in the search field. If you know the shortcut of an action, click and press the key combination in the **Find Shortcut** dialog.

When consulting this page and other pages in IntelliJ IDEA documentation, you can see keyboard shortcuts for the keymap that you use in the IDE — choose it using the selector at the top of the page.

To view the keymap reference as PDF, choose **Help | Keymap Reference** from the main menu.

Add a keyboard shortcut

1. On the **Keymap** page of the **Settings/Preferences** dialog Ctrl+Alt+S, right-click an action and select **Add Keyboard Shortcut**.



2. In the **Keyboard Shortcut** dialog, press the necessary key combination.

If you want to use `Enter`, `Escape`, or `Tab`, click and select the necessary key or combination. Pressing them in the **Keyboard Shortcut** dialog will result in the actual action, such as closing the dialog.

3. If necessary, select the **Second stroke** checkbox to define a complex shortcut with two sequential key combinations.
4. Click **OK** to save the shortcut.

The key combination that you press is displayed in the **Keyboard Shortcut** dialog, as well as a warning if it conflicts with existing shortcuts.

Add a mouse shortcut

1. On the **Keymap** page of the **Settings/Preferences** dialog `Ctrl+Alt+S`, right-click an action and select **Add Mouse Shortcut**.
2. In the **Mouse Shortcut** dialog, move the mouse pointer to the central area and click or scroll as necessary.
3. Click **OK** to save the shortcut.

The performed mouse manipulations are displayed in the **Mouse Shortcut** dialog, as well as a warning if it conflicts with existing shortcuts.

Add an abbreviation

An abbreviation can be used to quickly find an action without a shortcut. For example, you can press `Ctrl+Shift+A` and type the name of the **Jump to Colors and Fonts** action to quickly modify the color and font settings of the element under the current caret position. If you assign an abbreviation for this action (like **JCF**), you can then type it instead of the full action name.

1. On the **Keymap** page of the **Settings/Preferences** dialog `Ctrl+Alt+S`, right-click an action and select **Add Abbreviation**.
2. In the **Abbreviation** dialog, type the desired abbreviation and click **OK**.

Reset action shortcuts to default

If you changed, added, or removed a shortcut for an action, you can reset it to the initial configuration.

- On the **Keymap** page of the **Settings/Preferences** dialog `Ctrl+Alt+S`, right-click an action and

select **Reset Shortcuts**.

Location of user-defined keymaps

All user-defined keymaps are stored in separate configuration files under the **keymaps** subdirectory in the IntelliJ IDEA configuration directory:

Syntax

```
%APPDATA%\JetBrains\<product><version>\keymaps
```

Example

```
C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJ IDEA 2021.1\keymaps
```

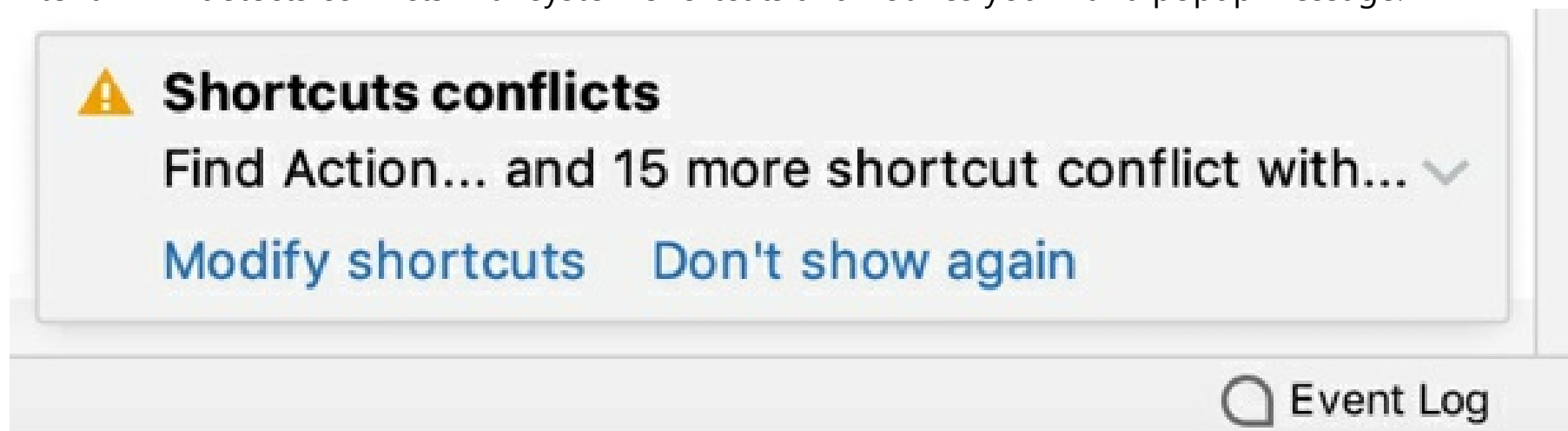
By default, this directory contains only the default keymaps. When you modify one of them, IntelliJ IDEA actually creates a child keymap file that contains only the differences relative to the parent keymap. For example, if you modified the default Windows keymap, your custom keymap will be its child. The file will contain only the shortcuts that you added or modified, while all other shortcuts of your custom keymap will be the same as the default Windows keymap.

You can share your custom keymaps with team members or between your IDE instances. Copy the corresponding keymap file and put it in the **keymaps** directory on another IntelliJ IDEA installation. Then select the copied keymap on the **Keymap** settings page.

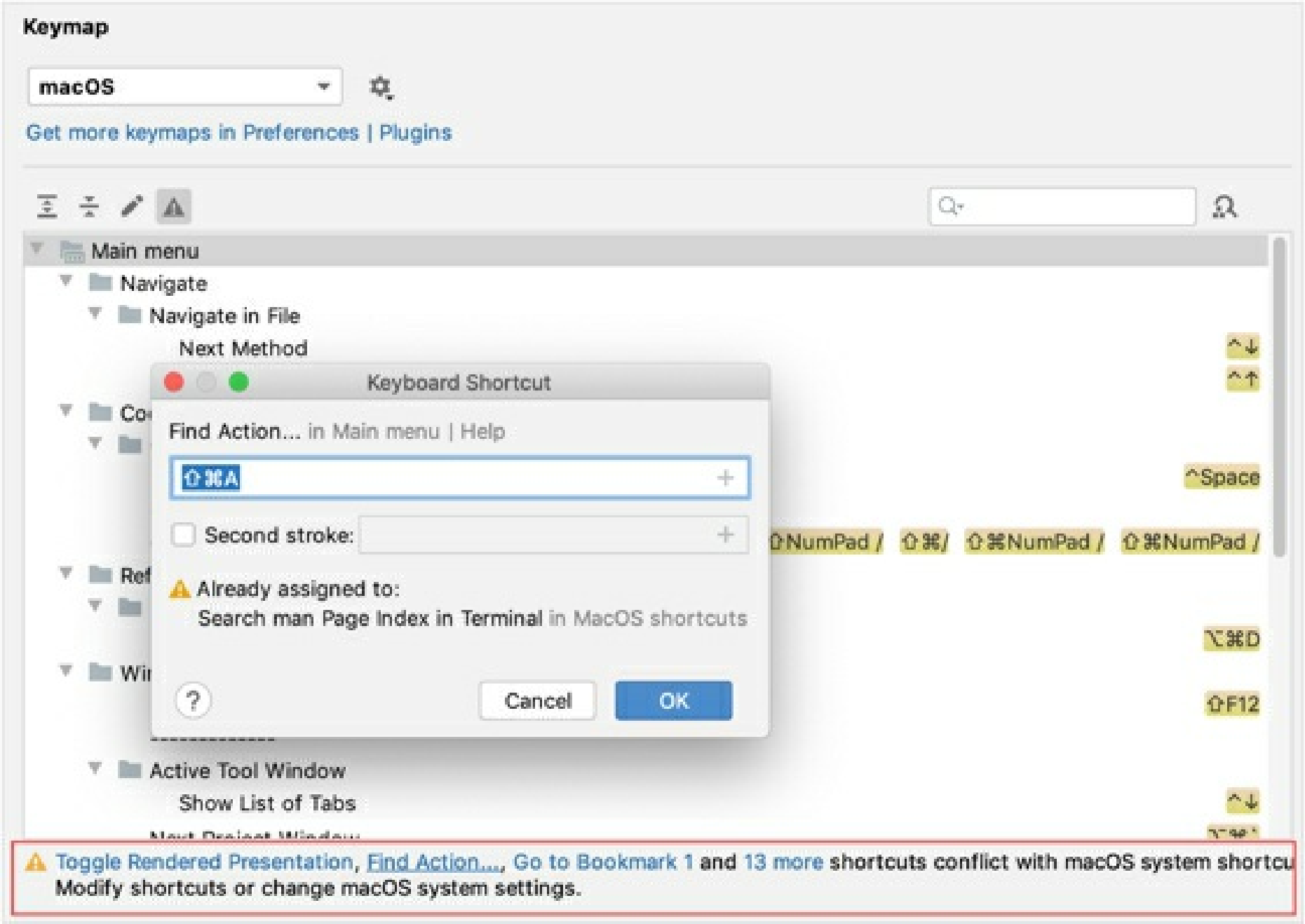
Conflicts with global OS shortcuts

Predefined keymaps do not cover every possible platform, version, and configuration. Some shortcuts can conflict with global system actions and shortcuts for third-party software. To fix these conflicts, you can reassign or disable the conflicting shortcut.

IntelliJ IDEA detects conflicts with system shortcuts and notifies you with a popup message:



Click **Modify shortcuts** to open the **Keymap** settings dialog where you can make the necessary adjustments:



Here are a few examples of possible system shortcut conflicts with the default keymap in IntelliJ IDEA. Make sure that function keys are enabled on your system.

macOS

Ubuntu

Shortcut	System action	IntelliJ IDEA action
^Space	Select the previous input source	Basic code completion
⇧⌘A	Search man Page Index in Terminal	Find Action

Set file type associations

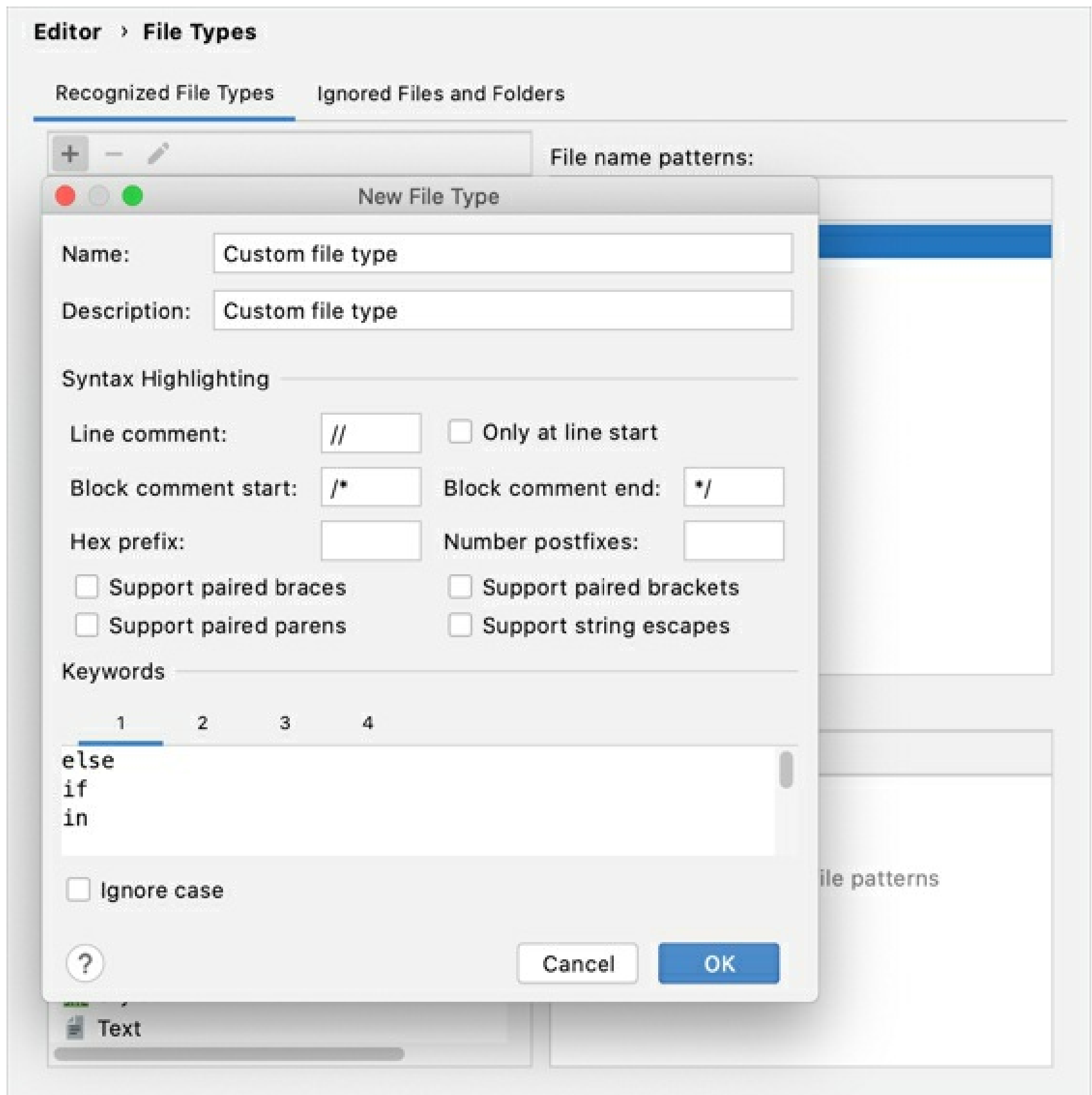
IntelliJ IDEA recognizes a default set of file types. Such files are parsed and highlighted according to the syntax of the corresponding languages.

If you are working with a file type that IntelliJ IDEA does not recognize (for example, if it's a proprietary in-house developed file type) you can also create a custom file type.

You can configure how the IDE will parse the files by defining highlighting schemes for keywords, comments, numbers, and so on. You can also associate each file type with an extension to help the IDE identify the files of the custom formats.

Create a new file type

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | File Types**.
2. In the **Recognized File Types** section, click **+**, specify the name of the new type, and provide a description.
3. In the **Syntax Highlighting** section, configure case sensitivity, brace matching settings, and specify ways of defining comments:
 - **Line comment**: specify characters that indicate the beginning of a single-line comment.
 - **Only at line start**: characters that indicate the beginning of a line comment are recognized as a comment if they are located in the beginning of a line.
 - **Block comment start, Block comment end**: specify characters that indicate the beginning and the end of a block comment.
 - **Hex prefix**: specify characters that indicate that the subsequent value is a hexadecimal number (for example, `0x`).
 - **Number postfixes**: specify characters that indicate which numeric system or unit is used. A postfix is a trailing string of characters (for example, `e-3`, `kg`).
 - **Support paired braces, Support paired brackets, Support paired parens, Support string escapes**: select these checkboxes to highlight paired braces, brackets, parentheses, and string escapes.
4. In the **Keywords** section, you can specify up to four lists of keywords. Keywords of each list will be highlighted differently in the editor and will be auto-completed.
5. The **Ignore case** checkbox indicates whether the language in files of the custom format is case-sensitive.



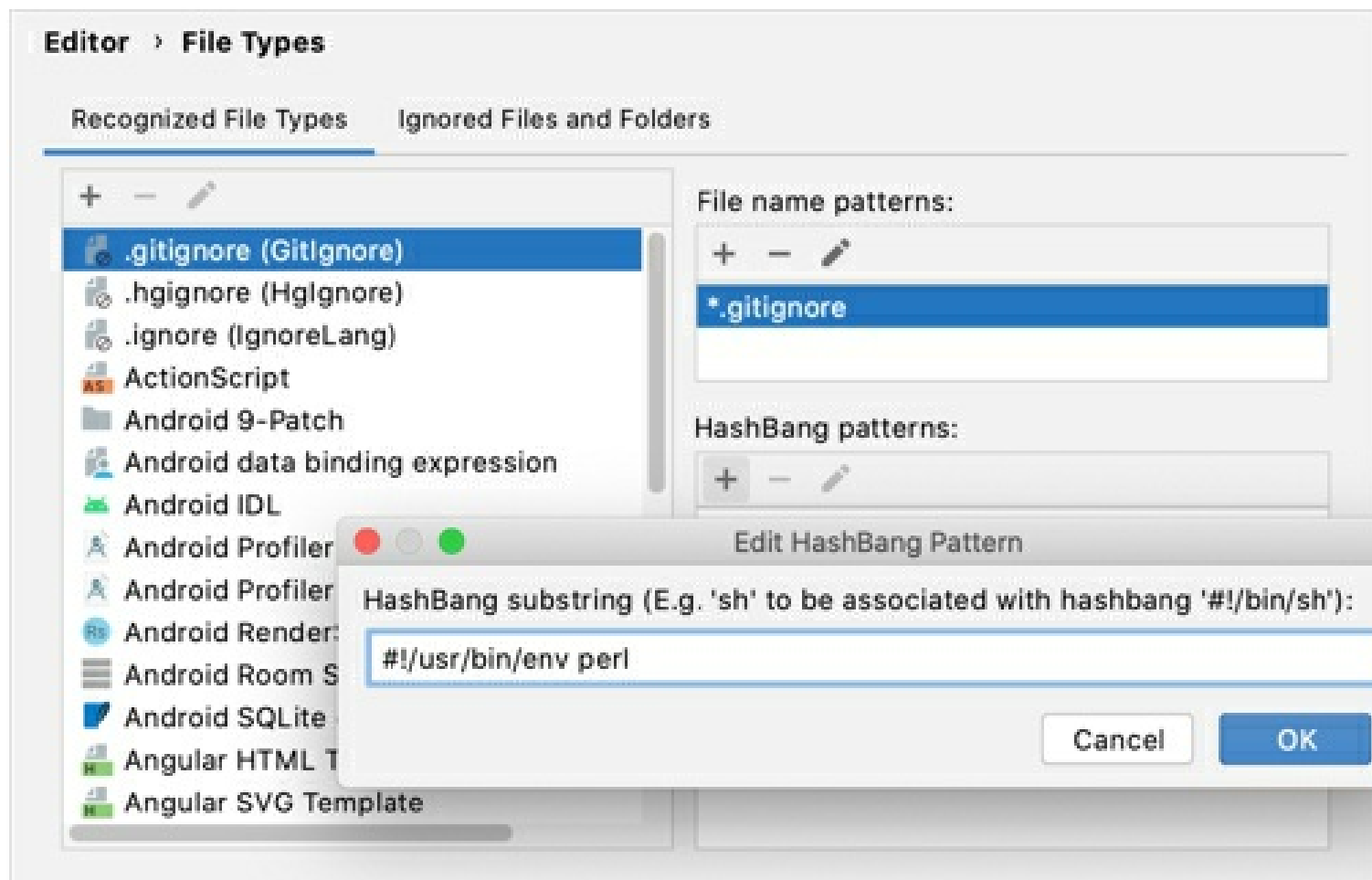
Each set of keywords has its own highlighting that you can modify. To do so, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Editor | Color Scheme | Custom**, and edit the **Keyword 1**, **Keyword 2**, **Keyword 3**, and **Keyword 4** properties.

In IntelliJ IDEA, recognized files are not always supplied with extensive support. For example, **.php** files are recognized in the IntelliJ IDEA Community Edition and marked with the corresponding icon, although this edition does not provide PHP development support.

Configure shebang commands for file types

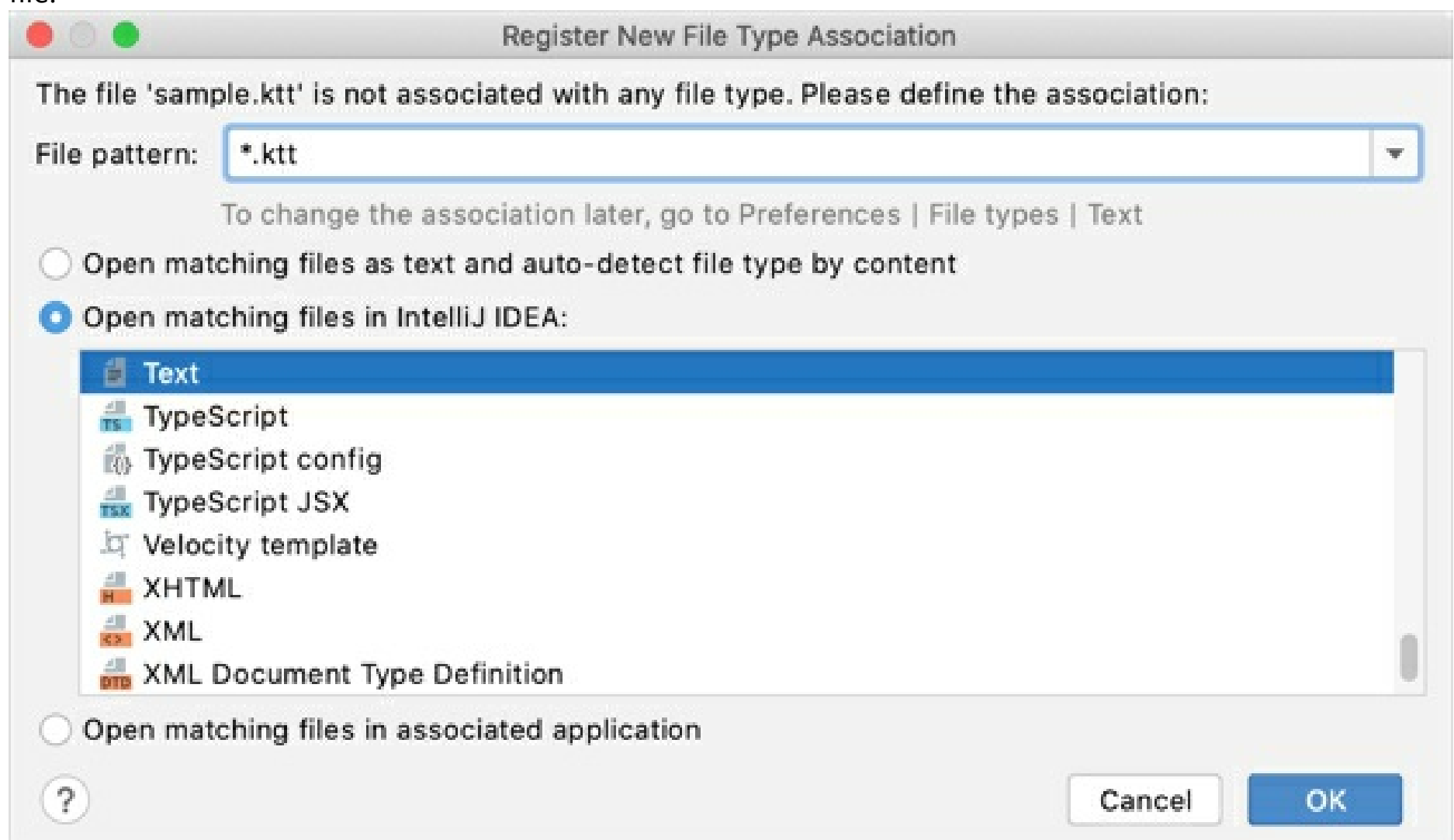
IntelliJ IDEA can recognize file types by the path specified on the shebang line. A shebang is a combination of characters in a script file followed by a path to the interpreter program that should execute this script. It starts with `#!` and it's always located on the first line of a script file.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | File Types**.
2. From the **Recognized File Types** list, select the file type for which you want to configure a command.
3. In the **HashBang patterns** area, click **(Add HashBang Pattern)**.
4. In the dialog that opens, specify the path to the interpreter program and click **OK**.
5. Apply the changes and close the dialog.



Register a new file type association

If IntelliJ IDEA cannot identify the type of the file that you are trying to open or create, it shows you the **Register New File Type Association** dialog in which you can choose the way you want to process this file.



If the dialog doesn't appear automatically, right-click the necessary file in the **Project** tool window and select **Associate with File Type** from the context menu. Alternatively, select the file in the **Project** tool window and go to **File | File Properties | Associate with File Type**.

In the **Register New File Type Association** dialog, select the necessary options:

1. From the **File pattern** list, select whether you want to specify a type for the current file (**file.extension**) or for all files with this extension (***.extension**).
2. Select one of the following options:

- **Open matching files as text and auto-detect file type by content:** open the file without an extension as a text file and identify its type by the content, for example, by the shebang line.
- **Open matching files in IntelliJ IDEA:** associate the file with one of the existing file types. You can change this association later in the settings.
- **Open matching files in associated application:** open the file in the default application. For example, PDF files are opened in the default PDF viewer. The default application is configured in your system systems.

You can find all extensions associated with external applications in the **Files opened in associated applications** list in **Settings/ Preferences | Editor | File Types**.

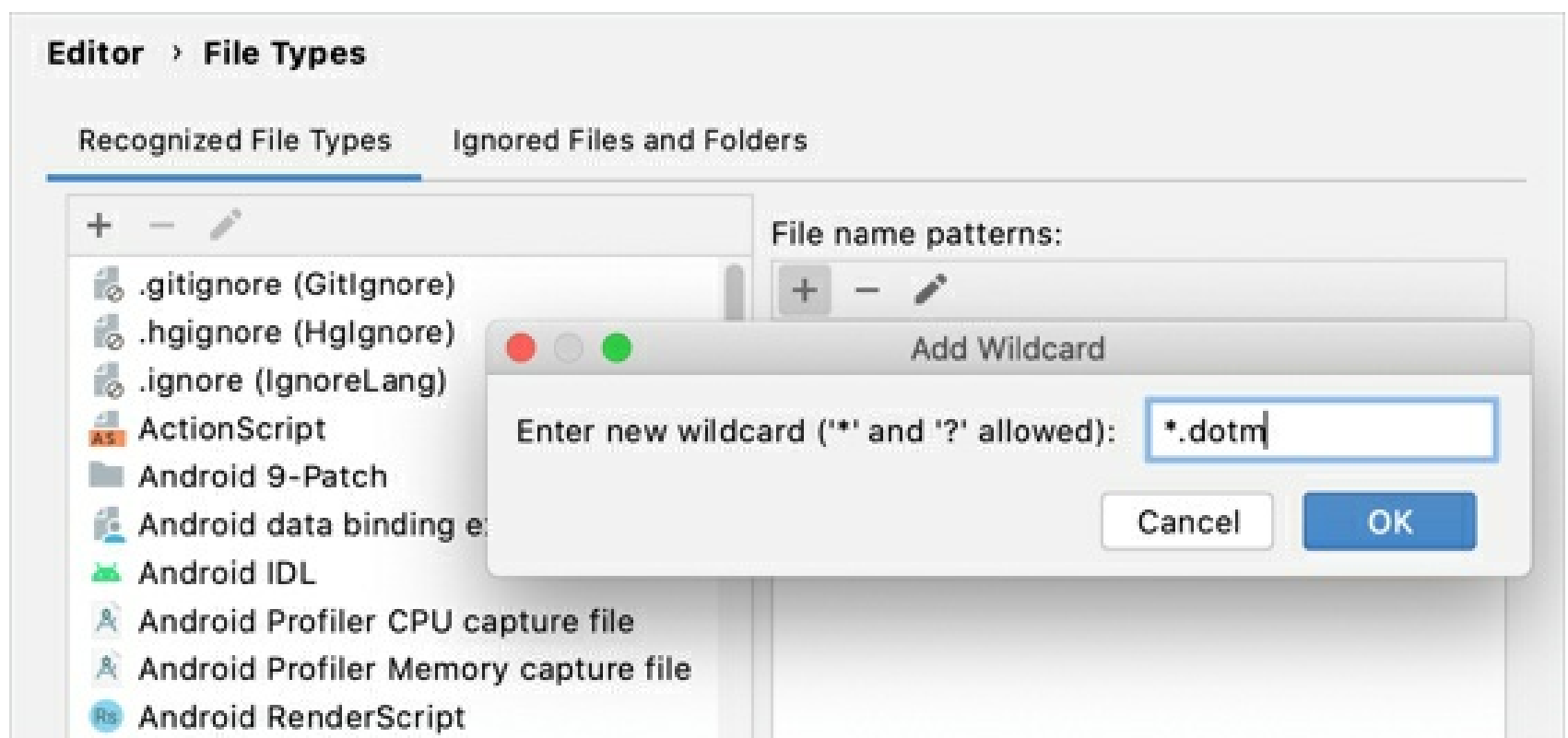
There, you can also associate file types with IntelliJ IDEA to open the selected types of files in the IDE by default.

3. Click **OK** to apply the settings.

Change a file type association

You can associate a file type with another extension, a file, or a group of files or remove an association.

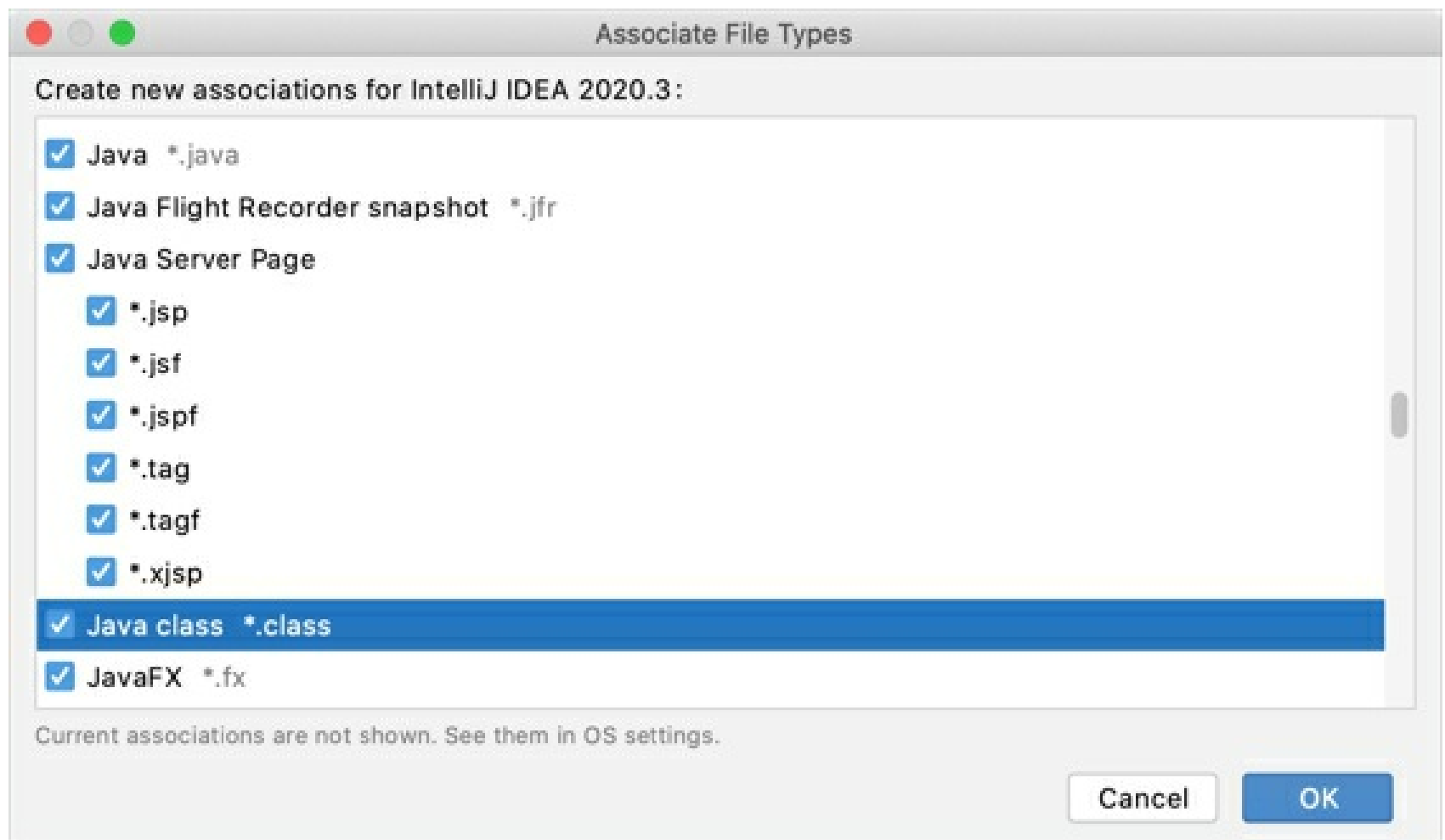
1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | File Types**.
2. From the **Recognized File Types** list, select the file type that you want to associate with another extension or a group of files.
3. Use the **File name patterns** section to make the necessary changes. You can add a new pattern (**+**), remove an existing one (**-**), or modify an existing pattern (**✎**).



Open files in IntelliJ IDEA by default

You can make IntelliJ IDEA the default application for opening specific file types. The IDE remembers these associations and automatically applies them to the next IntelliJ IDEA instance when you upgrade to a newer major version.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | File Types**.
2. Click the **Associate File Types with IntelliJ IDEA** button and select the file extensions you want to open with the IDE in the dialog that opens.



3. Click **OK** and close the dialog.

If you're using macOS, restart your computer to apply the changes.

Ignore files and folders

In IntelliJ IDEA, there's a list of files and folders that are completely excluded from any kind of processing. Out of the box, this list includes temporary files, service files related to version control systems, and so on:

.pyc;.pyo;*.rbc;*.yarb;*~;.DS_Store;git;hg;svn;CVS;_pycache_;_svn;vssver.scc;vssver2.scc;
Copied!

Modify the list of ignored files and folders

1. Press **Ctrl+Alt+S** to open IDE settings and select **Editor | File Types**.
2. Switch to the **Ignored Files and Folders** tab.

You can add a new extension (), remove an existing one (), or modify an existing extension ().

3. Apply the changes and close the dialog.

Editor > File Types

Recognized File Types

Ignored Files and Folders

+

*.рус

*.pyo

*._rbc

```
*.yarb
```

*

.DS Store

gil

hg

..SVN

CV9

pycache

_SVFI

vssver.scc

vssver2.scc

Encoding

To display and edit files correctly, IntelliJ IDEA needs to know which encoding to use. In general, source code files are mostly in UTF-8. This is the recommended encoding unless you have some other requirements.

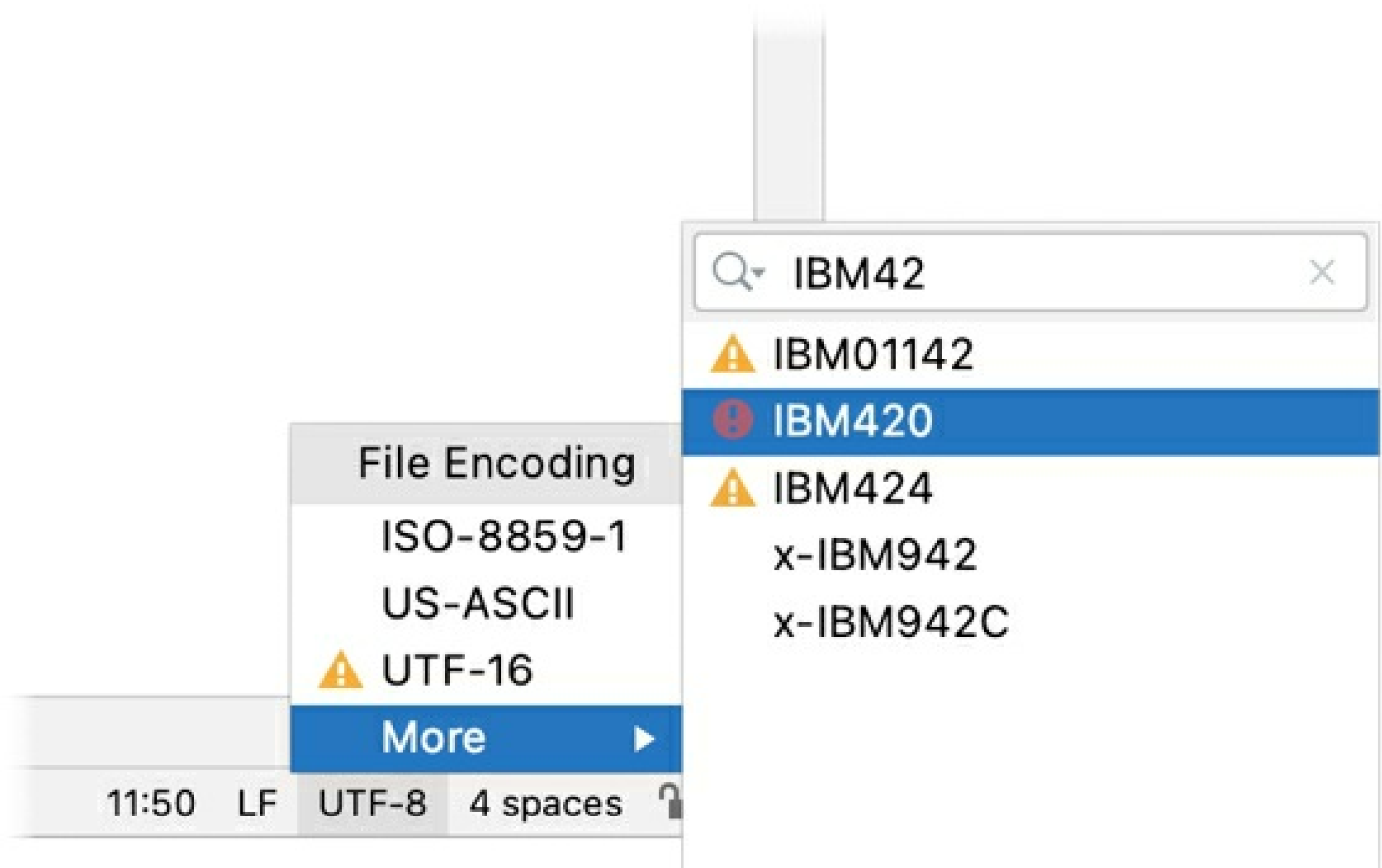
To determine the encoding of a file, IntelliJ IDEA uses the following steps:

- If the *byte order mark* (BOM) is present, IntelliJ IDEA will use the corresponding Unicode encoding regardless of all other settings. For more information, see [Byte order mark](#).
- If the file declares the encoding explicitly, IntelliJ IDEA will use the specified encoding. For example, this can apply to XML, HTML, and JSP files. The explicit declaration also overrides all other settings, but you can change it in the editor.
- If there is no BOM and no explicit encoding declaration in the file, IntelliJ IDEA will use the encoding configured for the file or directory in the file encoding settings. If encoding is not configured for the file or directory, IntelliJ IDEA will use the encoding of the parent directory. If the parent directory encoding is also not configured, IntelliJ IDEA will fall back to the **Project Encoding**, and if there is no project, to **Global Encoding**.

Change the encoding used to view a file

If IntelliJ IDEA displays characters in a file incorrectly, it probably couldn't detect the file encoding. In this case, you need to specify the correct encoding to use for viewing and editing this file.

- With the file open in the editor, either select **File | File Properties | File Encoding** from the main menu or click the **File Encoding** widget on the status bar, and select the correct encoding of the file.



The list of encodings is rather large. You can use speed search to quickly find the correct encoding: start typing when the popup is open.

Encodings marked with  or  might change the file contents. In this case, IntelliJ IDEA opens a dialog where you can choose what to do with the file:

- **Reload:** load the file in the editor from disk and apply encoding changes to the editor only. You will see the content changes related to the chosen encoding, but

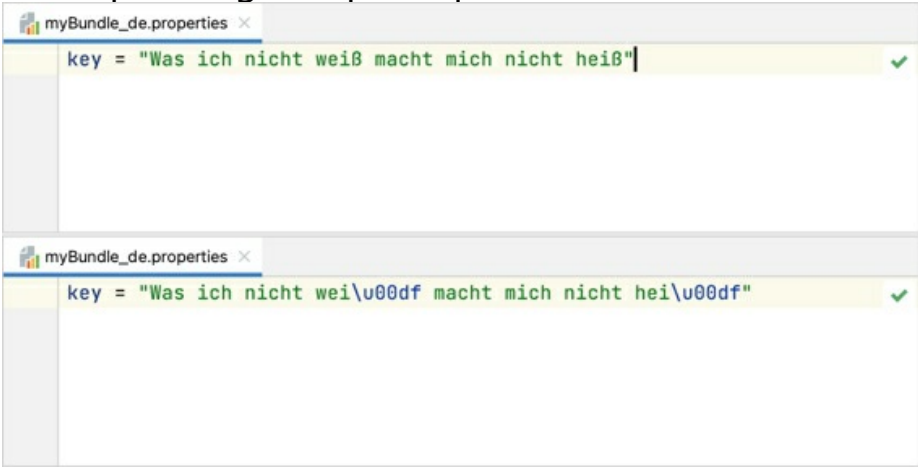
- the actual file will not change.
- **Convert:** overwrite the file with the chosen encoding.

This will add an association for the file to the file encoding settings. IntelliJ IDEA will use the specified encoding to view and edit this file.

Configure file encoding settings

In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Editor | File Encodings**.

IntelliJ IDEA uses these settings to view and edit files for which it was not able to detect the encoding and also uses the specified encodings for new files.

Global Encoding	Select the encoding to use when other encoding options don't apply. For example, IntelliJ IDEA will use this encoding for files that are not part of any project or when you check out sources from a version control system.
Project Encoding	Select the encoding to use for files that are not listed in the table.
Path	Specify the path to the files or directories for which you want to configure the encoding.
Encoding	Select the encoding to use for the specified files and directories. If this selector is disabled, the file probably has a BOM or declares the encoding explicitly. In this case, you can't configure the encoding to use for this file. The encoding selected for a directory applies to all files and subdirectories within it.
Default encoding for properties files	Select the encoding for properties files in your project. The standard Java API is designed to use the ISO 8859-1 encoding for properties files. You can use escape sequences for characters that are not defined by this encoding. Alternatively, you can define the default encoding for properties files on the project level and use a different API that can read properties files in the encoding you have defined.
Transparent native-to-ascii conversion	Show national characters (those not defined in ISO 8859-1) in place of the corresponding escape sequences.  The first screenshot shows a line in a properties file: <code>key = "Was ich nicht weiß macht mich nicht heiß"</code>. The second screenshot shows the same line with escape sequences: <code>key = "Was ich nicht wei\u00df macht mich nicht hei\u00df"</code>.
Create UTF-8 files	Select how IntelliJ IDEA should create UTF-8 files: • with BOM • without BOM • with BOM on Window and without BOM otherwise

By default, IntelliJ IDEA creates UTF-8 files without the BOM because some software is not compatible with the BOM, and it may be a problem when interpreting scripts. However, in some cases, you may want to have the BOM in your UTF-8 files. To add or remove the BOM from all UTF-8 files in your project, right-click the name of your project in the **Project** tool window and select either **Add BOM** or **Remove BOM**.

Select console output encoding

By default, IntelliJ IDEA uses the system encoding to view console output.

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Editor | General | Console**.
2. Select the default encoding from the **Default Encoding** list.
3. Click **OK** to apply the changes.

Switch between schemes

You can quickly switch between various color schemes, keyboard layouts, and look-and-feels without actually invoking the corresponding page of the Settings dialog.

1. Press `Ctrl+`` or choose **View | Quick Switch Scheme** from the main menu.
2. In the popup that opens, select the desired scheme (Color Scheme, Code Style, and so on).



3. In the suggestion list, click the desired option.



Share IDE settings

IntelliJ IDEA lets you share your IDE settings between different instances of the product, or among your team members. This helps you recreate a comfy working environment if you are working from different computers and spare the annoyance of things looking or behaving differently from what you are used to, or enforce the same standards throughout your team.

For information on how to share settings related to specific projects, see [Share project settings through VCS](#).

You can share your IDE settings by using one of the following:

- **IDE Settings Sync**: it utilizes the JetBrains server, so no additional configuration is required. Note that synced settings are linked to your JetBrains Account, so they will not be available to other team members, and are only useful to share settings between different IDE instances used by you.

The settings you can sync include: IDE themes, keymaps, color schemes, system settings, UI settings, menus and toolbars settings, project view settings, editor settings, code completion settings, parameter name hints, live templates, code styles, and the list of enabled and disabled plugins.

- **settings repository**: it allows you to sync any configurable components (except for the list of enabled and disabled plugins), but requires setting up a Git repository with the settings you want to share.

This option is useful if you want to implement the same settings among your team-members.

- **exporting the settings** you want to share as a ZIP archive and then importing them to a different IDE installation. You can export you code style settings, Git settings, including registered GitHub accounts, the Debugger settings, Registry keys, look and feel, and more.

Share settings through **Settings Sync**^{Ultimate}

Make sure that the **IDE Settings Sync** plugin is enabled in the **Settings/Preferences** dialog `Ctrl+Alt+S`, under **Plugins**.

If you have enabled a settings repository, you cannot share your settings through **Settings Sync**.

Sync settings between IDE instances

1. On the computer with the IDE instance containing the settings you want to share, sign in to either of the following:
 - **Your IDE**: from the main menu choose **Help | Register**, choose to activate your license with the JetBrains Account and enter your credentials.
 - **Toolbox App**: click the gear icon  in the top right corner of the application, select **Settings** and click **Log in**. Note that by signing in to Toolbox App, you automatically sign in to all JetBrains products that you run.
2. In the bottom-right corner of the IntelliJ IDEA window, click the gear icon and select **Enable Sync**. Alternatively, select **File | Manage IDE Settings | Sync Settings to JetBrains Account** from the main menu. In the dialog that opens, click the **Enable Settings Sync** button. Your local settings will be exported to the JetBrains repository linked to your account.
3. If you want to automatically sync the list of all enabled and disabled plugins, select the **Sync plugins silently** option. For instructions on how to sync plugins manually if it is disabled, refer to [Sync plugins](#).
4. On a different computer where you want these settings to be applied, click the gear button,

and select **Enable Sync**. Alternatively, select **File | Manage IDE Settings | Sync Settings to JetBrains Account** from the main menu. In the dialog that opens, click **Get Settings from Account** to import the settings from the repository.

If you want to override the repository with your local settings, click **Keep and Sync Local Settings**.

Your local settings will be automatically synchronized with the settings stored in the repository each time you run a different IDE instance (or activate it after more than one hour of inactivity), or when any of these settings has been modified and this change has been applied.

Sync plugins

When you install or uninstall plugins, or change their state (enabled/disabled), you can apply these changes to all your IDE installations.

If you want to automatically sync plugins across IDE instances, select the **Sync plugins silently** option when you enable settings synchronization.

Sync plugins manually

1. In the bottom-right corner of the IntelliJ IDEA window, click the gear icon and select **Sync Plugins**.
2. A dialog opens showing a list of all plugins that were modified since the last sync. Click the arrow button next to each plugin and choose either to modify the plugin's state, apply the repository state to all installations, skip this change locally, or skip it across all IDE instances.
3. After you've selected which action to take for each plugin, click **Apply Changes**.

Share settings through a settings repository

This functionality is unavailable if you have enabled Settings Sync.

Make sure that the **Settings Repository** plugin is enabled in the **Settings/Preferences** dialog `Ctrl+Alt+S`, under **Plugins**.

Configure a settings repository

1. Create a Git repository on any hosting service, such as GitHub.

If you select to use GitHub, we recommend creating a private repository that's not visible to everyone.

2. On the computer where the IntelliJ IDEA instance whose settings you want to share is installed, select **File | Manage IDE Settings | Settings Repository** from the main menu. Specify the URL of the repository you've created and click **Overwrite Remote**.
3. On each computer where you want your settings to be applied, select **File | Manage IDE Settings | Settings Repository** from the main menu. Specify the URL of the repository you've created, and click **Overwrite Local**.

You can click **Merge** if you want the repository to keep a combination of the remote settings and your local settings. If any conflicts are detected, a dialog will be displayed where you can resolve these conflicts.

If you want to overwrite the remote settings with your local settings, click **Overwrite Remote**.

Your local settings will be automatically synchronized with the settings stored in the repository each time you perform an **Update Project** or a **Push** operation, or when you close your project or exit IntelliJ IDEA.

On the first sync, you will be prompted to specify a username and password. It is recommended to use an access token for GitHub authentication. If, for some reason, you want to use a username and password instead of an access token, or your Git hosting provider doesn't support it, it is recommended to configure the Git credentials helper.

The macOS Keychain is supported, which means you can share your credentials between all IntelliJ Platform-based products (you will be prompted to grant access if the original IDE is different from the

requestor IDE).

If you want to disable automatic settings synchronization, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Tools | Settings Repository** and disable the **Auto Sync** option. You will be able to update your settings manually by choosing **VCS | Sync Settings** from the main menu.

Share more settings through additional read-only repositories

Apart from the **Settings Repository**, you can configure any number of additional repositories containing any types of settings you want to share, including live templates, file templates, schemes, deployment options, and so on.

These repositories are referred to as **read-only sources**, as they cannot be overwritten or merged, just used as a source of settings as is.

Synchronization with the settings from read-only sources is performed in the same way as for the **Settings Repository**.

Configure read-only repositories

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Tools | Settings Repository**.
2. Click  and add the URL of the GitHub repository that contains the settings you want to share.

Export your settings

This functionality is unavailable if you have a settings repository configured or if you have enabled Settings Sync.

Export settings to a ZIP archive

1. Choose **File | Manage IDE Settings | Export Settings** from the main menu.
2. Select the settings you want to export and specify the path to the target archive.

Import settings from a ZIP archive

1. Choose **File | Manage IDE Settings | Import Settings** from the main menu.
2. Select the ZIP archive that contains your settings in the dialog that opens.
3. Select the settings you want to apply in the **Select Components to Import** dialog that opens and click **OK**.

Advanced configuration

Besides the standard options available, IntelliJ IDEA enables you to perform low-level configuration of the underlying platform and the Java runtime.

This may lead to unexpected problems and make your IntelliJ IDEA installation inoperable if you are not sure what you are doing. Contact JetBrains Support for instructions regarding the options and values that might help you with whatever issue you are trying to solve.

JVM options

IntelliJ IDEA runs on the Java Virtual Machine (JVM), which has various options that control its performance. The default options used to run IntelliJ IDEA are specified in the following file:

<IDE_HOME>\bin\idea64.exe.vmoptions (for the default 64-bit JVM)

<IDE_HOME>\bin\idea.exe.vmoptions (for the optional 32-bit JVM)

Do not change JVM options in the default file, because it is replaced when IntelliJ IDEA is updated.

Moreover, in case of macOS, editing this file violates the application signature.

Configure JVM options

Do one of the following to create a copy of the default file with JVM options in the configuration directory that will override the original file:

- On the **Help** menu, click **Edit Custom VM Options**.
- If you do not have any project open, on the Welcome screen, click **Configure** and then **Edit Custom VM Options**.
- If you cannot start IntelliJ IDEA, manually copy the default file with JVM options to the IntelliJ IDEA configuration directory.

If you do not have write access to the IntelliJ IDEA configuration directory, you can add the `IDEA_VM_OPTIONS` environment variable to specify the location of the file with your preferred JVM options. This file will override both the original default file and the copy located in the IntelliJ IDEA configuration directory.

If you are using the Toolbox App, it manages the installation and configuration directory and lets you configure JVM options for every IDE instance. Open the Toolbox App, click the screw nut icon for the necessary instance, and select **Settings**.

Common options

The default values of the JVM options should be optimal in most cases. The following are the most commonly modified ones:

Option	Description
-Xmx	Limits the maximum memory heap size that the JVM can allocate for running IntelliJ IDEA. Default value depends on the platform. If you are experiencing slowdowns, you may want to increase this value, for example, to set the value to 2048 megabytes, change this option to <code>-Xmx2048m</code> .
-Xms	Specifies the initial memory allocated by the JVM for running IntelliJ IDEA. Default value depends on the platform. It is usually set to about half of the maximum allowed memory (<code>-Xmx</code>), for example, <code>-Xms1024m</code> .
-XX:NewRatio	Specifies the ratio between the size of the young and old generation of the heap. In most cases, a ratio between 2 and 4 is recommended. This will set the size of the young generation to be 1/2 to 1/4 of the old generation correspondingly, which is good when you are often working on one project and

	only a few files at a time. However, if you are constantly opening new files and switching between several projects, you may need to increase the young generation. In this case, try setting <code>-XX:NewRatio=1</code> , which will make the young generation as large as the old generation, allowing objects to remain in the young generation for longer.
--	---

For more information about available JVM options, see the `java` reference for Windows or macOS/Linux.

Platform properties

IntelliJ IDEA enables you to customize various platform-specific properties, such as the path to user-installed plugins and the maximum supported file size. The default properties used to run IntelliJ IDEA are specified in the following file:

Windows

macOS

Linux

<IDE_HOME>\bin\idea.properties

Do not change platform properties in the default file, because it is replaced when IntelliJ IDEA is updated. Moreover, in case of macOS, editing this file violates the application signature.

Configure platform properties:

Do one of the following to create an empty **idea.properties** file in the configuration directory that will override the values from the original file:

- From the **Help** menu, select **Edit Custom Properties**.
- If you do not have any project open, on the Welcome screen, click **Configure** and then select **Edit Custom Properties**.
- If you cannot start IntelliJ IDEA, manually create an empty **idea.properties** file in the IntelliJ IDEA configuration directory.

If you do not have write access to the IntelliJ IDEA configuration directory, you can add the `IDEA_PROPERTIES` environment variable to specify the location of the **idea.properties** file. The properties in this file will override the corresponding properties in both the original default file and the one located in the IntelliJ IDEA configuration directory.

Common properties

Users often change the following properties:

- The default IDE directories may need to be moved, for example, if the user profile drive runs out of space or it is located on a slow disk, if the home directory is encrypted (slowing down the IDE) or located on a network drive, if you want to create a portable installation or exclude caches from home directory backups, and so on.

Property	Path to
<code>idea.config.path</code>	Configuration directory
<code>idea.system.path</code>	System directory
<code>idea.plugins.path</code>	Plugins directory

idea.log.path	Logs directory
---------------	----------------

- Specify paths with forward slashes /, including Windows paths (for example, **C:/idea/system**).
- You can insert properties as variables. For example, use `${user.home}` (standard Java system property) to specify paths relative to the user's home directory:
 - `idea.config.path=${user.home}/MyIdeaConfiguration`
 - Copied!
- Limits that can affect performance:

Property	Description
idea.max.content.load.filesize	Maximum size of files (in kilobytes) that IntelliJ IDEA is able to open. Working with large files can affect editor performance and increase memory consumption. The default value is 20000.
idea.max.intellisense.filesize	Maximum size of files (in kilobytes) for which IntelliJ IDEA provides coding assistance. Coding assistance for large files can affect editor performance and increase memory consumption. The default value is 2500.
idea.cycle.buffer	Maximum size of the console cyclic buffer (in kilobytes). If the console output size exceeds this value, the oldest lines are deleted. To disable the cyclic buffer, set <code>idea.cycle.buffer.size=disabled</code> .
idea.max.vcs.loaded.size.kb	Maximum size (in kilobytes) that IntelliJ IDEA loads for showing past file contents when comparing changes. The default value is 20480.

IntelliJ IDEA provides a number of other properties that define interaction with the environment (window managers, launchers, file system, and so on). Most of them act like hidden settings (in the sense that they are not evidently exposed), which you may need to enable or disable in certain cases. Change these properties only if advised by JetBrains Support.

Default IDE directories

By default, IntelliJ IDEA stores user-specific files (configuration, caches, plugins, logs, and so on) in the user's home directory. However, you can change the location for storing those files, if necessary. The default IDE directories changed starting from IntelliJ IDEA 2020.1. If you had a previous version, new installations will import configuration from the old directories. For information about the location of the default directories in previous IDE versions, see the corresponding help version, for example: <https://www.jetbrains.com/help/idea/2019.3/tuning-the-ide.html#default-dirs>.

Configuration directory

The IntelliJ IDEA configuration directory contains user-defined IDE settings, such as keymaps, color schemes, custom VM options, platform properties, and so on.

Syntax

`%APPDATA%\JetBrains\<product><version>`

Example

```
C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJIdea2021.1
```

You can change the location of the IntelliJ IDEA configuration directory using the `idea.config.path` property.

To share your personal IDE settings, copy the files from the configuration directory to the corresponding folders on another IntelliJ IDEA installation. Make sure that IntelliJ IDEA is not running to avoid erasing the copied files when you shut down the IDE. Depending on which settings you modified, the IntelliJ IDEA configuration directory can contain the following subfolders:

Directory	User settings
codestyles	Code style schemes
colors	Customized editor color and font schemes
fileTemplates	User-defined file templates which pertain to the entire IntelliJ IDEA workspace
filetypes	User-defined file types
inspection	Code inspection profiles
keymaps	Customized keyboard shortcuts
options	Various options, for example, feature usage statistics and macros
scratches	Scratch files and buffers
templates	User-defined live templates
tools	Configuration files for user-defined external tools
shelf	Shelved changes

If your settings are synchronized using the Settings Repository plugin, these subfolders are located under **settingsRepository** in the configuration directory.

If your settings are synchronized through the IDE Settings Sync plugin, these subfolders are located under **jba_config** in the configuration directory.

System directory

The IntelliJ IDEA system directory contains caches and local history files.

Windows

macOS

Linux

Syntax

`%LOCALAPPDATA%\JetBrains\<product><version>`

Example

`C:\Users\JohnS\AppData\Local\JetBrains\IntelliJdea2021.1`

You can change the location of the IntelliJ IDEA system directory using the `idea.system.path` property.

Plugins directory

The IntelliJ IDEA plugins directory contains user-installed plugins.

Windows

macOS

Linux

Syntax

`%APPDATA%\JetBrains\<product><version>\plugins`

Example

`C:\Users\JohnS\AppData\Roaming\JetBrains\IntelliJdea2021.1\plugins`

You can change the location of the IntelliJ IDEA plugins directory using the `idea.plugins.path` property.

Logs directory

The IntelliJ IDEA logs directory contains product logs and thread dumps.

Windows

macOS

Linux

Syntax

`%LOCALAPPDATA%\JetBrains\<product><version>\log`

Example

`C:\Users\JohnS\AppData\Local\JetBrains\IntelliJdea2021.1\log`

You can change the location of the IntelliJ IDEA logs directory using the `idea.log.path` property.

You can open the location of the logs directory using the corresponding **Help** menu action: **Show Log in Explorer** on Windows, **Show Log in Finder** on macOS.

Change the boot Java runtime of the IDE

As a Java application, IntelliJ IDEA includes JetBrains Runtime (based on OpenJDK 11), which is used by default. It is recommended to run IntelliJ IDEA using JetBrains Runtime, which fixes various known OpenJDK and Oracle JDK bugs, and provides better performance and stability. However, in some cases you may be required to use another Java runtime or a specific version of JetBrains Runtime.

Changing the runtime may cause unexpected problems. Do not change it unless you were specifically asked to do so by JetBrains support.

Switch the Java runtime used to run IntelliJ IDEA

1. From the main menu, select **Help | Find Action** or press `Ctrl+Shift+A`.
2. Find and select the **Choose Boot Java Runtime for the IDE** action.
3. Select the desired runtime and click **OK**.

If necessary, you can change the location where IntelliJ IDEA will download the selected runtime.

4. Wait for IntelliJ IDEA to restart with the new runtime.

When you open the **Choose Boot Runtime for the IDE** dialog for the first time, it may take a while to load the list of JetBrains Runtime builds from the server.

To use a different Java runtime available on your computer, select **Add Custom Runtime** under **Advanced**. IntelliJ IDEA lists all the JDKs and JREs that it was able to detect. Alternatively, click **Add JDK** to specify the location of the desired Java home directory.

To reset back to the default runtime that the IDE initially used, click **Use Default Runtime**.

The path to the selected runtime is stored in the **idea.jdk** or **idea64.jdk** file in the IntelliJ IDEA configuration directory. If there are problems with the selected runtime, you can delete this file to revert to the default runtime.

You can also override the runtime used for IntelliJ IDEA by adding the `IDEA_JDK` environment variable with the path to the desired JDK home directory.

Increase the memory heap of the IDE

Help | Change Memory Settings

The Java Virtual Machine (JVM) running IntelliJ IDEA allocates some predefined amount of memory. The default value depends on the platform. If you are experiencing slowdowns, you may want to increase the memory heap.

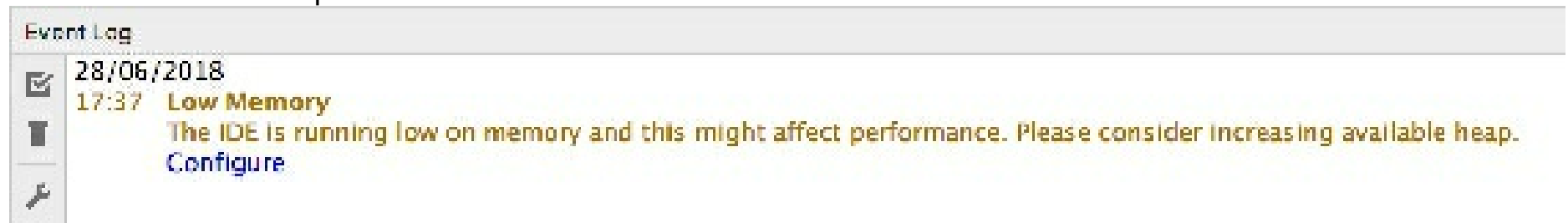
If you want to configure the heap size for the build process that compiles your code, open **Settings/Preferences** `Ctrl+Alt+S`, select **Build, Execution, Deployment | Compiler**, and specify the necessary amount of memory in the **Build process heap size** field.

1. From the main menu, select **Help | Change Memory Settings**.
2. Set the necessary amount of memory that you want to allocate and click **Save and Restart**.

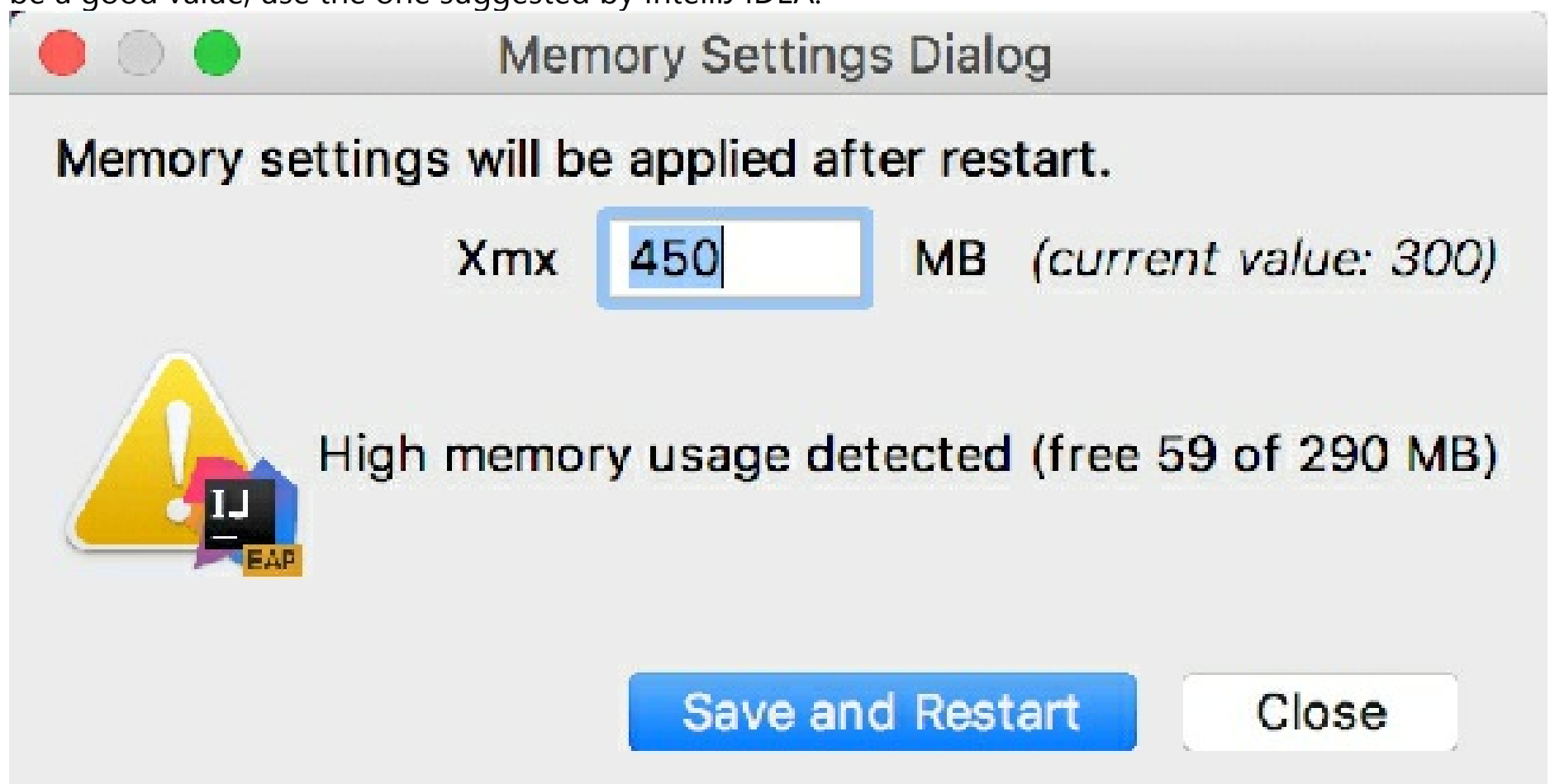
This action changes the value of the `-xmx` option used by the JVM and restarts IntelliJ IDEA with the new setting.

The **Change Memory Settings** action is available starting from IntelliJ IDEA version 2019.2. For previous versions or if the IDE crashes, you can change the value of the `-xmx` option manually as described in JVM options.

IntelliJ IDEA also warns you if the amount of free heap memory after a garbage collection is less than 5% of the maximum heap size:



Click **Configure** to increase the amount of memory allocated by the JVM. If you are not sure what would be a good value, use the one suggested by IntelliJ IDEA.



Click **Save and Restart** and wait for IntelliJ IDEA to restart with the new memory heap setting.

Enable the memory indicator

IntelliJ IDEA can show you the amount of used memory in the status bar. Use it to judge how much memory to allocate.

- Right-click the status bar and select **Memory Indicator**.

Toolbox App

If you are using the Toolbox App, you can change the maximum allocated heap size for a specific IDE instance without starting it.

1. Open the Toolbox App, click  next to the relevant IDE instance, and select **Settings**.
2. In the instance settings dialog, expand **Configuration** and specify the heap size in the **Maximum heap size** field.

If the IDE instance is currently running, the new settings will take effect only after you restart it.

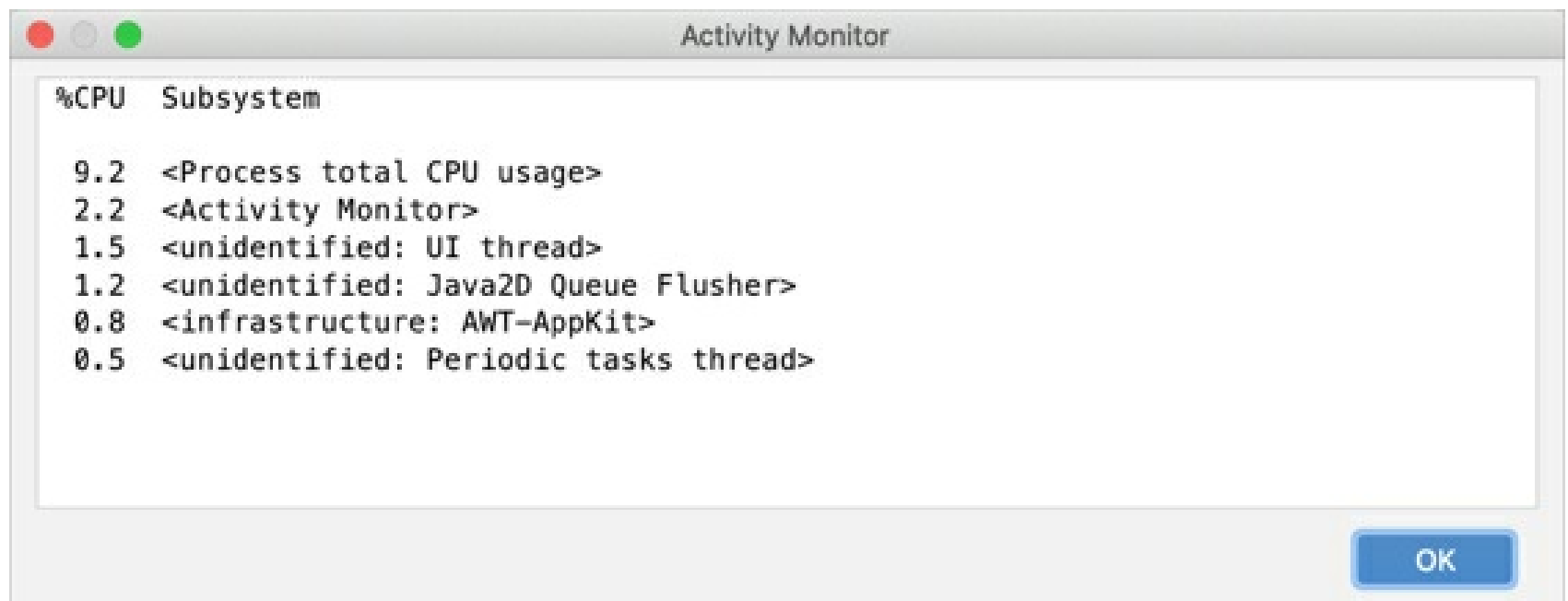
If you are using a standalone instance not managed by the Toolbox App, and you can't start it, it is possible to manually change the `-Xmx` option that controls the amount of allocated memory. Create a copy of the default JVM options file and change the value of the `-Xmx` option in it.

Monitor IDE performance

Activity Monitor is an experimental feature in IntelliJ IDEA.

In case of performance issues, you can use **Activity Monitor** to track the percentage of CPU consumed by various subsystems and plugins.

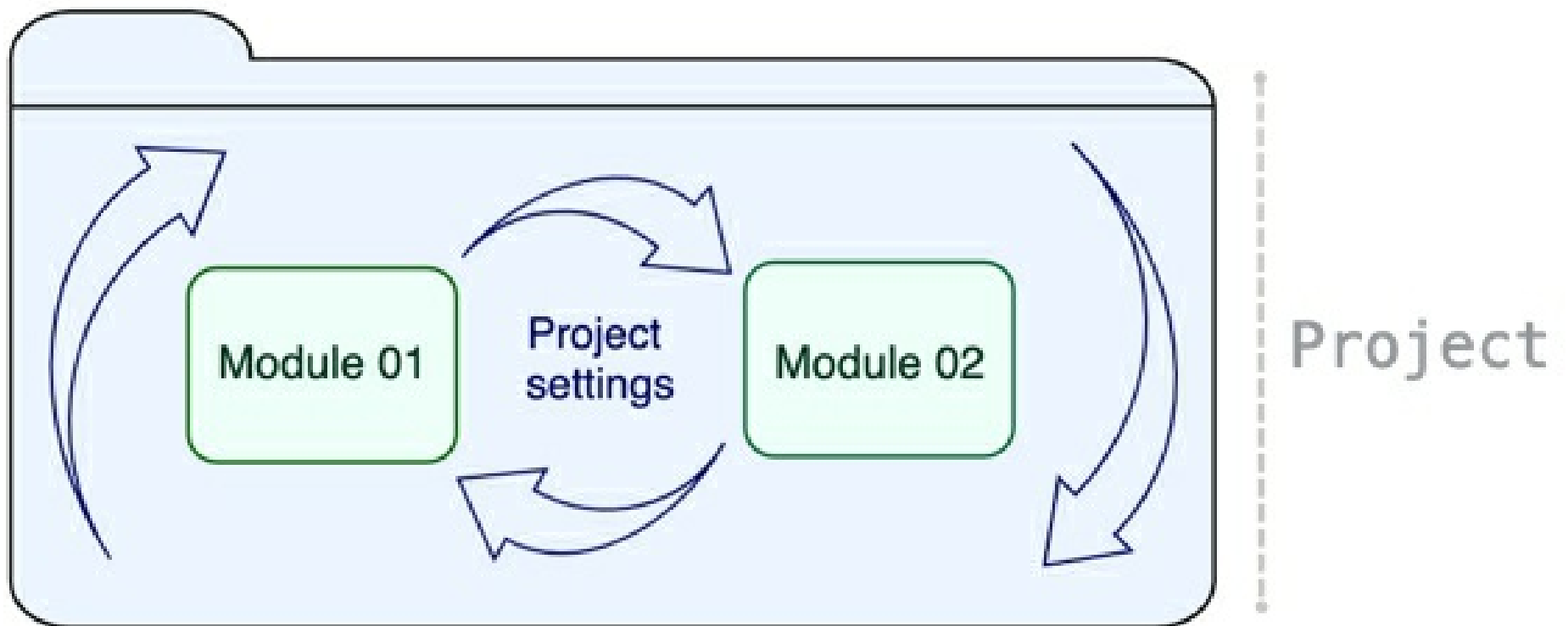
1. From the main menu, select **Help | Diagnostic Tools | Activity Monitor**.
2. The **Activity Monitor** window lists all subsystems and plugins that are consuming CPU at the moment and arranges them by how much %CPU they are using:



If you notice increased CPU consumption, and no indexing or background processes are running at this moment, we recommend that you contact our technical support for assistance. Background processes are always displayed in the status bar at the bottom of the IDE window.

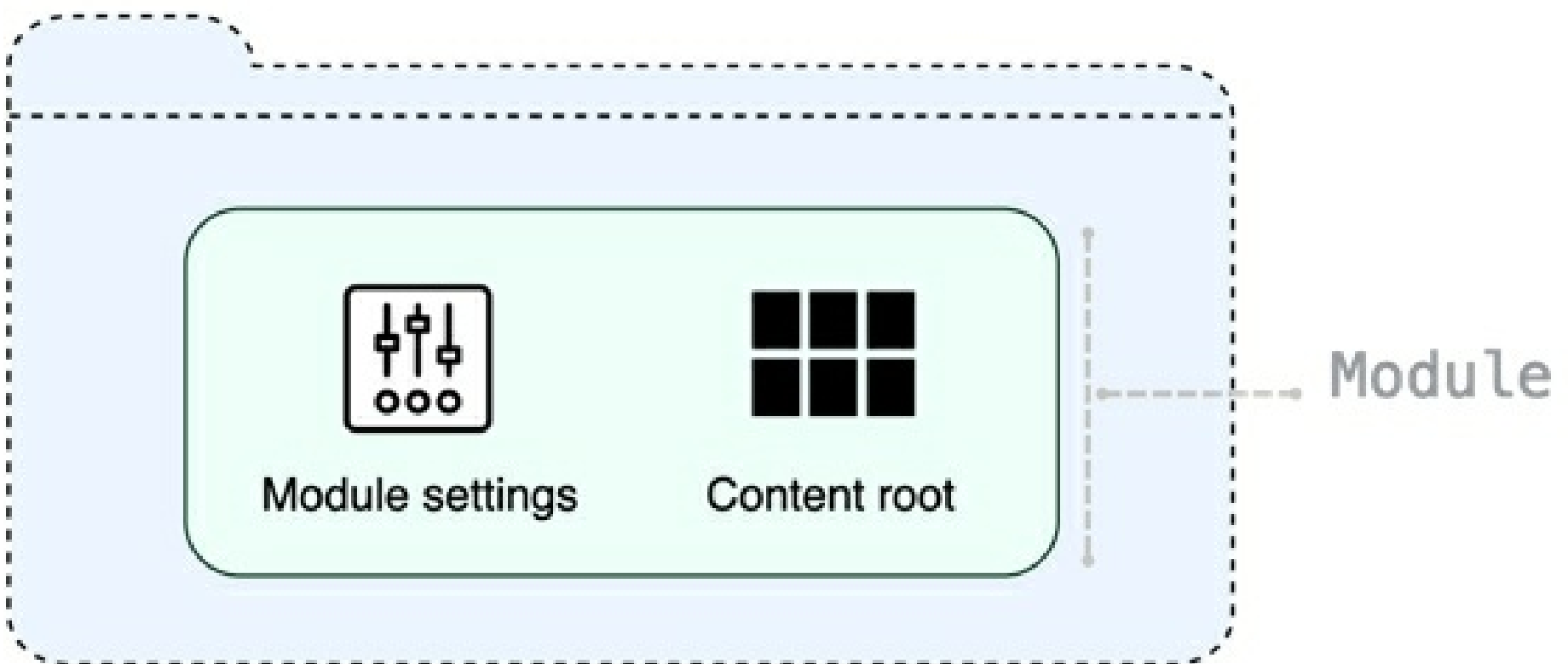
Configure projects

Development in IntelliJ IDEA starts with a project. Projects help you organize your code and resources in a single unit that is easy to store and share. In simple words, a project is a directory that keeps everything that makes up your application. A typical project normally has a set of settings and one or several modules.



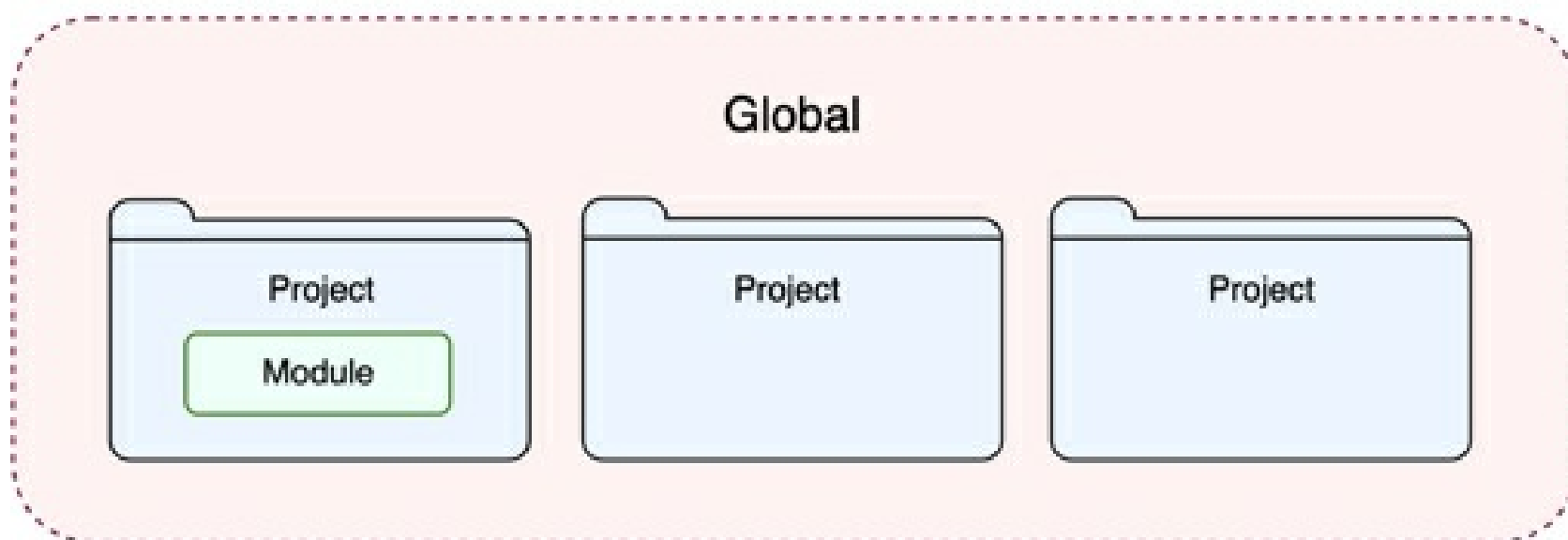
A project in IntelliJ IDEA is a shell that keeps modules together, provides dependencies between them, and stores their shared configuration.

A module is composed of the **.iml** file that keeps internal representation of module settings and the so-called *content root*, which stores your source code, resources, tests, and so on.



Project, module, and global settings

There are 3 types of settings in IntelliJ IDEA: module, project, and global settings.



Module settings

These settings apply only to one module and are stored in the **.iml** file. A module can have an SDK and a language level that are different from those configured for a project, and their own libraries. They can also carry a specific technology or a framework.

For more information, refer to [Module structure settings](#).

Project settings

These settings apply only to the current project. They are stored together with other project files in the **.idea** directory in the **.xml** format. For example, projects keep VCS settings, SDKs, code style and spellchecker settings, compiler output, libraries that are available for all modules within a project.

For more information, refer to [Project settings](#) and [Project structure settings](#).

Global settings

Global settings apply to all projects of a specific installation of IntelliJ IDEA. Such settings include IDE appearance (for example, themes and color schemes), the set of installed and enabled plugins, debugger settings, global inspection profiles, and much more.

Project security

To prevent potential security risks, IntelliJ IDEA lets you decide how to open a project if you're not sure about its source. IntelliJ IDEA warns you about tasks or configurations that will be executed during the opening process, and lets you configure sources that you can trust.

Gradle and Maven projects security

For sbt and BSP projects, the same security measures are applied.

When you open a project, such as Gradle or Maven, IntelliJ IDEA executes its build scripts during the loading process that may potentially contain the untrusted code.

Open a project for the first time

When you try to open a Gradle or a Maven project from an unknown source for the first time, IntelliJ IDEA displays a warning and lets you decide how to proceed.



You can select one of the following actions:

- **Preview in Safe Mode:** in this case IntelliJ IDEA opens a project in a "preview mode" meaning you can browse the project's sources, but it might be unsafe to execute any tasks or goals, build, or run your project.

IntelliJ IDEA displays a notification on top of the editor area, and you can click the **Trust project** link and load your project at any time.

- **Trust Project:** in this case, IntelliJ IDEA opens and loads a project. That means build scripts are executed, project's plugins are resolved, dependencies are added, and so on.
- **Don't Open:** in this case IntelliJ IDEA cancels the action.

To trust the source from which you are trying to open a project, select the **Trust projects in** option. The next time you open a project from this directory, it will be opened and loaded automatically.

Open an existing project

If a project you are planning to open was created on a different machine and contains the **.idea** directory, IntelliJ IDEA opens your project in the IDE automatically as if you chose the Preview in Safe Mode action. IntelliJ IDEA doesn't execute build scripts, resolve project's plugins, or add any dependencies. However, you still can browse the project's sources and open them in the editor.

If you try to execute any Maven goals or Gradle tasks through its dedicated tool window or through the **Run Anything** window, IntelliJ IDEA will display a notification suggesting you to trust and load the project before executing anything.

IntelliJ IDEA also displays an editor notification stating that the project is untrusted.



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/m
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.4.2</version>
  </parent>

  <groupId>org.springframework.samples</groupId>
  <artifactId>spring-petclinic-microservices</artifactId>
```

If you trust the source, click **Trust project** and load it.



Trust Maven Project?

If you don't trust the source, stay in safe mode.

Loading, running or building a Maven project may execute potentially malicious code from its build scripts.

Stay in Safe ModeTrust Project

In this case, IntelliJ IDEA loads the project, resolves plugins, adds the necessary dependencies, and so on. You can also add the source to the trusted locations, so the next time you open your project, IntelliJ IDEA will trust it implicitly.

Startup tasks

When you open a project created on a different machine, it might contain some scripts or tasks that are executed during the opening process. If such tasks are found, IntelliJ IDEA displays a notification suggesting that the code you are about to execute might be harmful.

You can review what tasks will be executed and modify the settings.

Review the startup tasks

1. In the **Settings/Preferences** dialog **Ctrl+Alt+S**, go to **Tools | Startup Tasks**.
2. On the **Startup Tasks** settings page, you can review and modify the startup tasks.

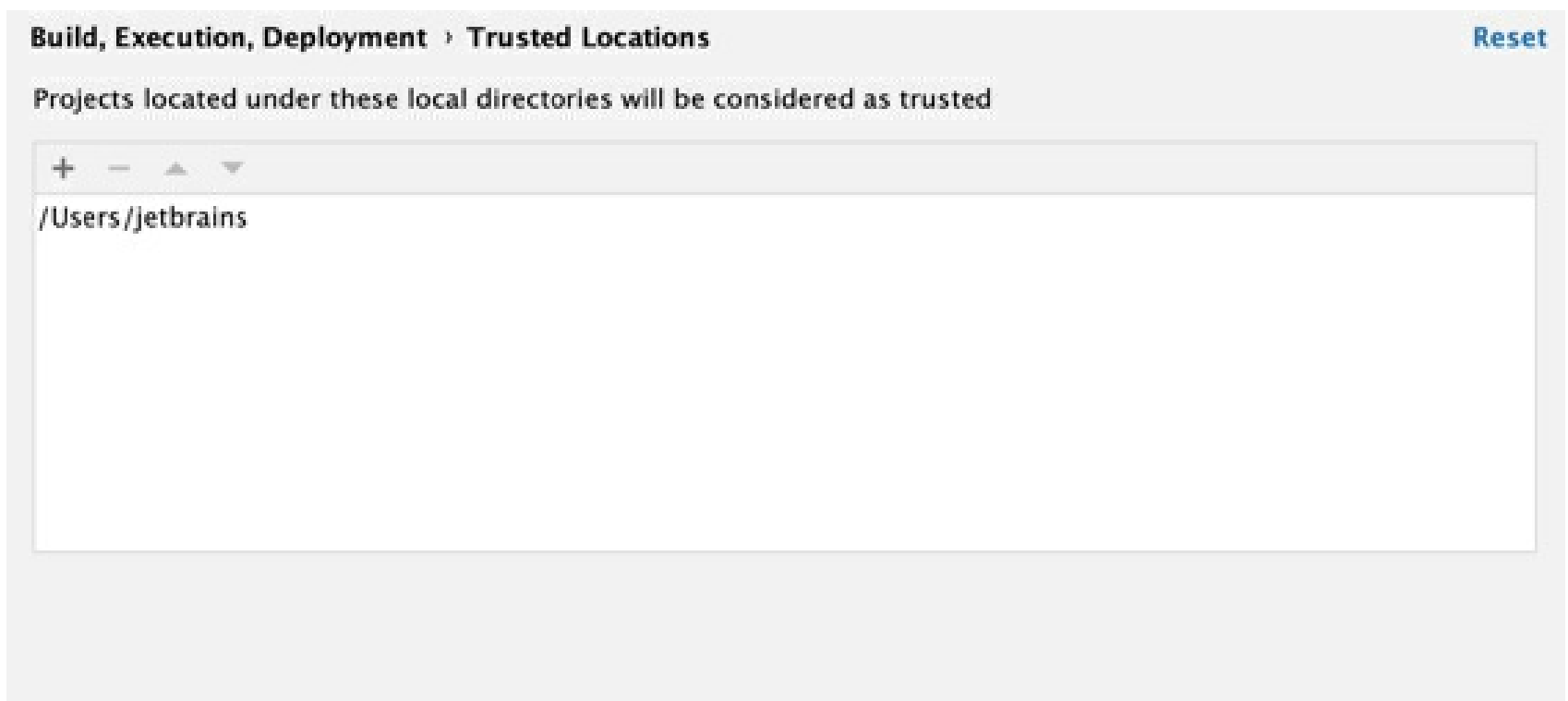
Trusted locations

You can configure what sources IntelliJ IDEA should consider safe and load such projects automatically during the opening process.

You can add your home directory to the trusted locations to disable IntelliJ IDEA's warnings about untrusted projects.

Configure trusted locations

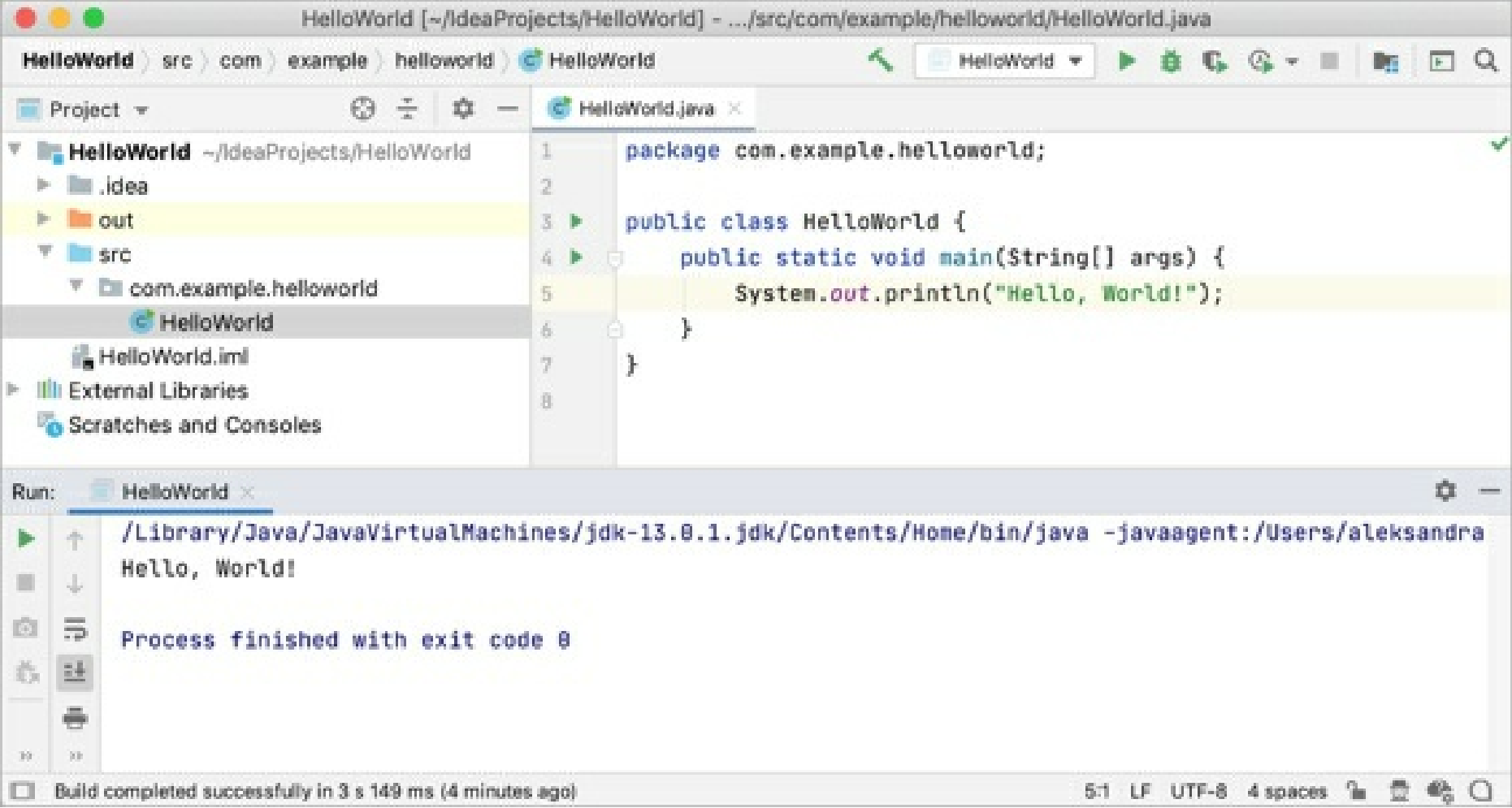
1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Build, Execution, Deployment | Trusted Locations**.
2. On the **Trusted Locations** settings page, configure the local directories where the projects you consider trusted reside. Click **OK** to save the changes.



The next time you open a project from one of those locations, IntelliJ IDEA will automatically load the project.

Projects

In IntelliJ IDEA, a project helps you to organize your source code, tests, libraries that you use, build instructions, and your personal settings in a single unit.



To quickly find any action, setting, file, or symbol in a project, use the Search Everywhere feature: press Shift twice and type your query in the popup.

Project formats

In IntelliJ IDEA, there are two types of formats in which a project's configuration can be stored — the file-based format and the directory-based format.

File-based format

The file-based format was the only one available in older versions of IntelliJ IDEA; now it is outdated. Projects in this format contain several files: the **.ipr**, **.iws**, and **.iml** files. Generally, we don't recommend using this format unless you need to open projects in different file managers by clicking the **.ipr** file, or unless you need to store multiple projects in one directory.

Name	^	Date Modified	Size	Kind
 file-based-sample.iml		Today, 12:12	423 bytes	Document
 file-based-sample.ipr		Today, 12:12	668 bytes	IntelliJ IDEA Project File
 file-based-sample.iws		Today, 12:13	2 KB	Document
 src		Today, 12:12	--	Folder

Directory-based format

For the directory-based format, the IDE creates the **.iml** file and the **.idea** directory that keeps project settings. It's the default format for projects in IntelliJ IDEA at this moment.

This format was introduced after the file-based format. Its main advantage is that it's adjusted to store project files in Version Control Systems: the project data is split over multiple files, and merge conflicts are less likely. For more information on how to share projects in different formats, refer to How to manage projects under Version Control Systems

On macOS and Linux, the **.idea** directory is hidden by default. To display the directory, allow showing hidden files in your system settings.

Name	^	Date Modified	Size	Kind
▼ .idea		Today at 12:45	--	Folder
.gitignore		Today at 12:45	176 bytes	TextEdit
misc.xml		Today at 12:45	271 bytes	XML Document
modules.xml		Today at 12:45	272 bytes	XML Document
workspace.xml		Today at 12:45	2 KB	XML Document
dir-based-sample.iml		Today at 12:45	423 bytes	Document
▶ src		Today at 12:45	--	Folder

Change the project format to directory-based

- From the main menu, select **File | Manage IDE Settings | Save as Directory-Based Format**.

Change the IML file location for Go projects in IntelliJ IDEA

Ensure that you installed and enabled the Go plugin.

When you create a Go project in IntelliJ IDEA, you can specify a directory for the **.iml** file. The default location is the root directory of the project.

Change the IML file location for new Go projects

- To create a new Go project, navigate to **File | New | Project**.
- In the **New Project** wizard dialog, select **Go** and click **Next**.
- Click **More Settings**.
- In the **Module file location** field, add **/.idea** to the existing path.

The screenshot shows the 'New Project' dialog in IntelliJ IDEA. The 'Project name' field contains 'ImIInIdeaDirectory'. The 'Project location' field contains '~/IdeaProjects/ImIInIdeaDirectory'. Under the 'More Settings' section, the 'Module name' is 'ImIInIdeaDirectory', the 'Content root' is '/Users/jetbrains/IdeaProjects/ImIInIdeaDirectory', and the 'Module file location' is '/Users/jetbrains/IdeaProjects/ImIInIdeaDirectory/.idea'. The 'Project format' is set to '.idea (directory based)'. At the bottom, there are buttons for '?', 'Cancel', 'Previous', and 'Finish'.

If you want to change the IML file location for existing Go projects in IntelliJ IDEA, you need to modify the **modules.xml** file in the **.idea** directory.

Change the IML file location for existing Go projects

- In the **Project** tool window, navigate to the **.idea** folder of the project.
- Add **.idea/** to **fileurl** and **filepath** attributes of the IML file location.
- Move the IML file to the **.idea** directory.



Share project settings through VCS

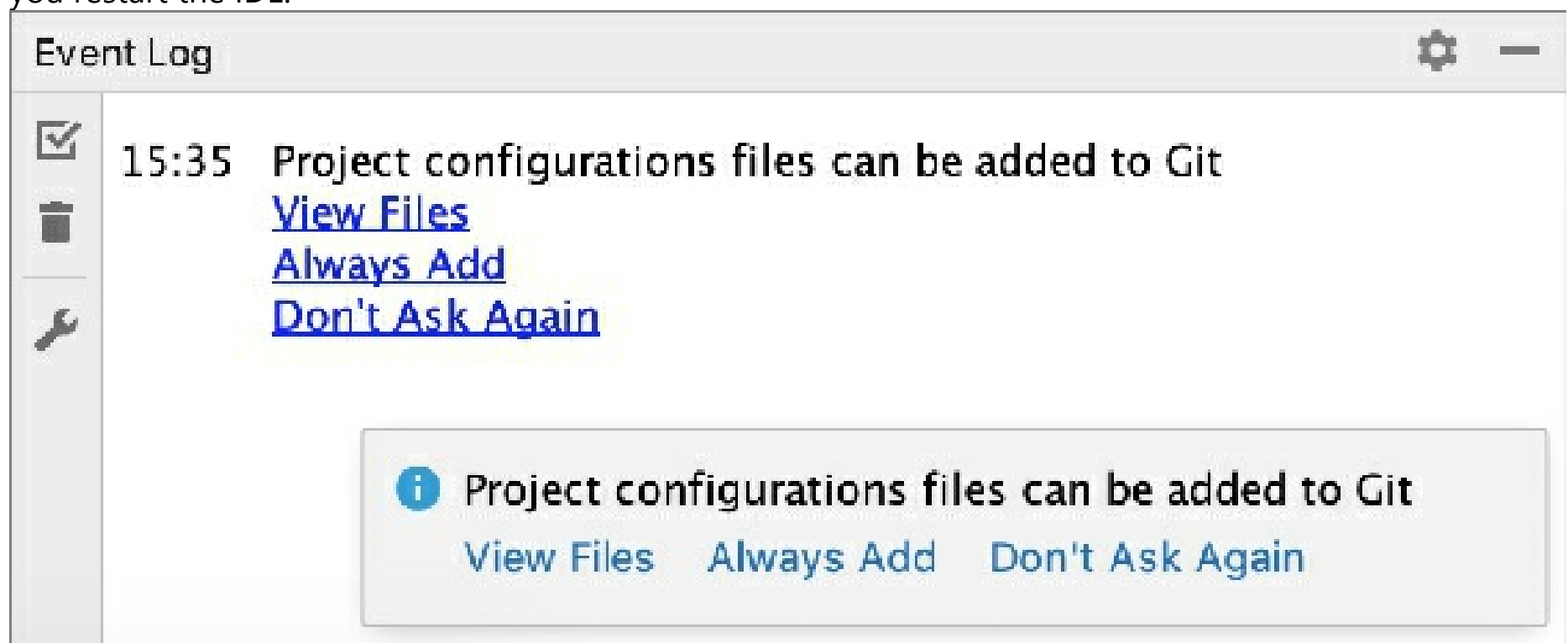
This information is valid for Git and Mercurial. If you use another version control system, refer to How to manage projects under Version Control Systems for information on how to share projects manually.

Project settings are stored in the project directory as a set of XML files under the **.idea** folder. This folder contains both user-specific settings that shouldn't be placed under version control and project settings that are normally shared among developers working in a team, for example, the code style configuration. When you place a project under version control, your personal settings are automatically ignored. IntelliJ IDEA moves **workspace.xml**— the file with your personal settings — to the list of *ignored* files to avoid conflicts with other developers' settings.

Configuration files are processed according to your choice. Once you modify the project settings, and a new configuration file is created, the IDE shows a notification at the bottom of the screen prompting you to select how you want to treat configuration files in this project:

- **View files:** view the list of created configuration files and select, which of them you want to place under version control. After that, the selected files will be scheduled for addition to VCS.
- **Always Add:** silently schedule all configuration files created in the **.idea** directory for addition to VCS (applies only to the current project).
- **Don't Ask Again:** never schedule configuration files for addition to VCS; they will have the *unversioned* status until you manually add them to VCS (applies only to the current project).

If you close the notification without selecting any option, it will appear again after a new configuration file is created. The new file will also go to the list that will be there until you select one of the options even if you restart the IDE.



If the **misc.xml** or the **.ipr** file is already under version control, project configuration files are silently scheduled for addition to VCS.

List of non-shareable configuration files

Copy global settings to the project level

Global (IDE) settings are stored separately from projects. That is why, these settings are not shared through version control together with the project.

Some settings, however, can be copied to the project level. For example, you can create a copy of your code style configuration, inspection profiles, the list of classes and packages excluded from code completion and auto-import. If you do so, the IDE creates the corresponding configuration files in the **.idea** directory that you can share together with the project through VCS.

IntelliJ IDEA also provides several ways of sharing settings between different IDE instances. See [Share IDE settings](#) for details.

Create a new project

This section describes the functionality available out of the box. If you are using a framework plugin, refer to the corresponding documentation section.

1. Launch IntelliJ IDEA.

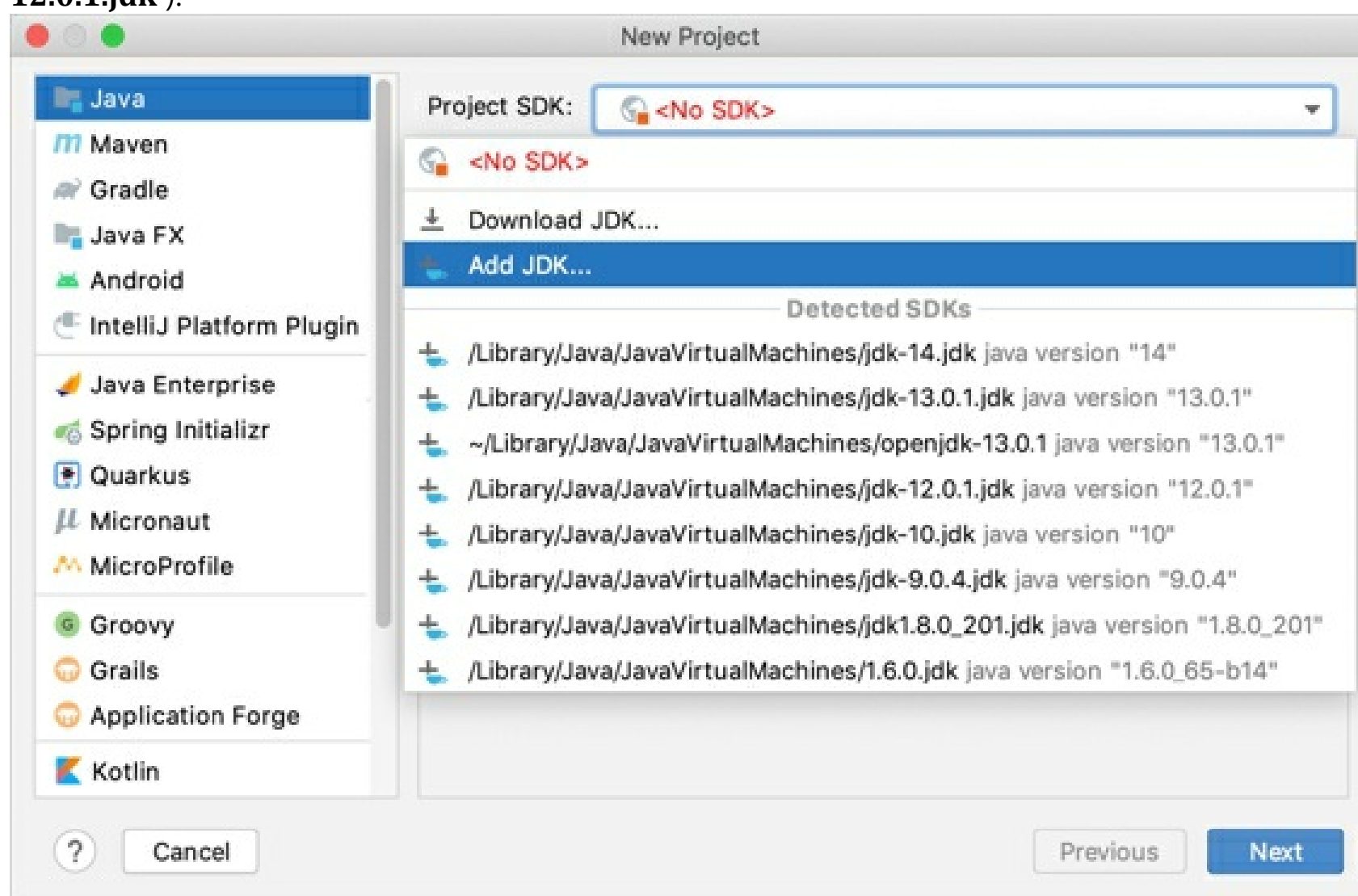
If the Welcome screen opens, click **New Project**.

Otherwise, from the main menu, select **File | New | Project**.

2. From the list on the left, select the framework that you want to use in your application.
3. If suggested, configure the project SDK. To develop Java-based applications, you need a JDK (Java Development Kit).

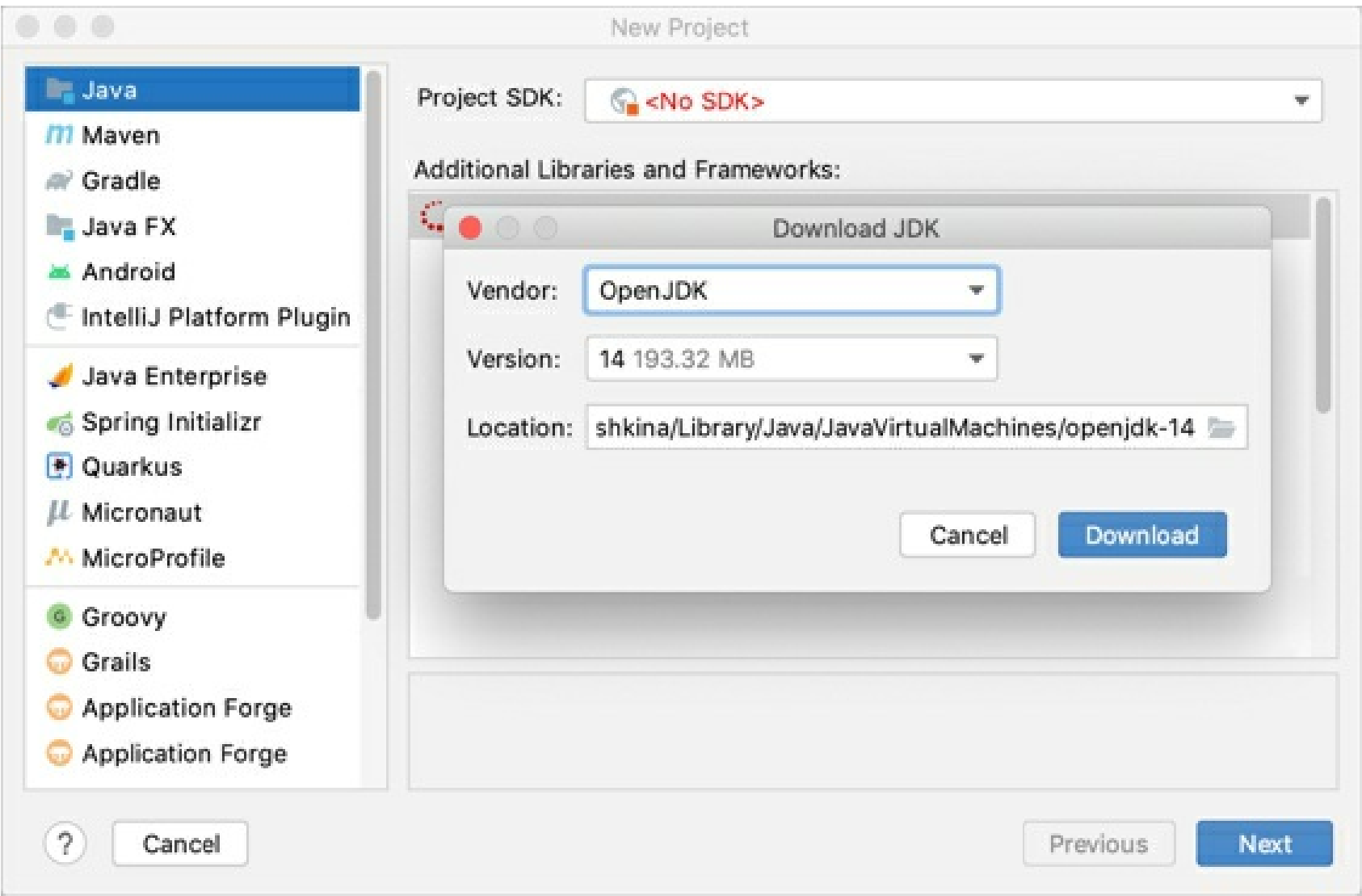
If the necessary JDK is already defined in IntelliJ IDEA, select it from the **Project SDK** list.

If the JDK is installed on your computer, but not defined in the IDE, select **Add JDK** and specify the path to the JDK home directory (for example, `/Library/Java/JavaVirtualMachines/jdk-12.0.1.jdk`).



If you don't have the necessary JDK on your computer, select **Download JDK**. In the next dialog, specify the JDK vendor, version, and change the installation path if required.

Some frameworks require their own SDKs in addition to the JDK, for example, Android or Grails.



4. Other options differ depending on the framework that you have selected. Refer to the list below for the detailed description of options for each framework.

Java

Step 1

Additional Libraries and Frameworks	Select the technologies, frameworks and languages that you want your project to support, and specify the associated settings. For more information, refer to Additional libraries and frameworks .
--	---

Step 2

Create project from template	Create a simple Java application that includes a class with <code>main()</code> method.
-------------------------------------	---

Step 3

Project name	Specify the project name.
Project location	Specify the path to the directory in which you want to create the project. By default, the IDE creates a directory with the same name as the project.
Module name	Specify the module name. By default, the project name is used. For more information on modules and the difference between projects and modules, refer to Configure projects and Modules .

Content root	Specify the path to the module <u>content root folder</u> (the folder that stores your source code). It's recommended that you use the default path.
Module file location	Specify the path to the folder where the .iml module file should be created. By default, this file is created in the module content root folder (recommended).
Project format	It's recommended that you use the <u>directory-based format</u> .

Maven
Step 1

Create from archetype	Select this checkbox if you want to use a predefined project template for your project. You can configure your own archetype by clicking Add Archetype .
-----------------------	---

Step 2

Name	Specify the project name.
Location	Specify the path to the directory in which you want to create the project. By default, the IDE creates a directory with the same name as the project.
GroupId	Specify a package of a new project.
ArtifactId	Specify the name of your project.
Version	Specify a version of a new project. By default, this field is specified automatically.

Step 3 (Create from archetype)

Maven home directory	Select a bundled Maven version that is available or the result of resolved system variables such as MAVEN_HOME or MAVEN2_HOME. You can also specify your own Maven version that is installed on your machine.
User settings file	Specify the file that contains user-specific configuration for Maven in the text field. If you need to specify another file, select the Override checkbox, click , and select another file.
Local repository	By default, this field shows the path to the local directory under the user home that stores downloads and contains the temporary build artifacts that you have not yet released. If you need to specify another directory, select the Override checkbox, click , and select another directory.

Properties	By default, the columns in this area display system properties that are passed to Maven for creating a project from the archetype. You can add, remove, and edit these properties.
-------------------	--

Import a project

Open a project (simple import)

This option imports the selected project to IntelliJ IDEA as is (opens it). If you want to set custom settings while importing the project (for example, select another SDK or choose the libraries that you want to import), refer to Create a project from existing sources.

1. Launch IntelliJ IDEA.

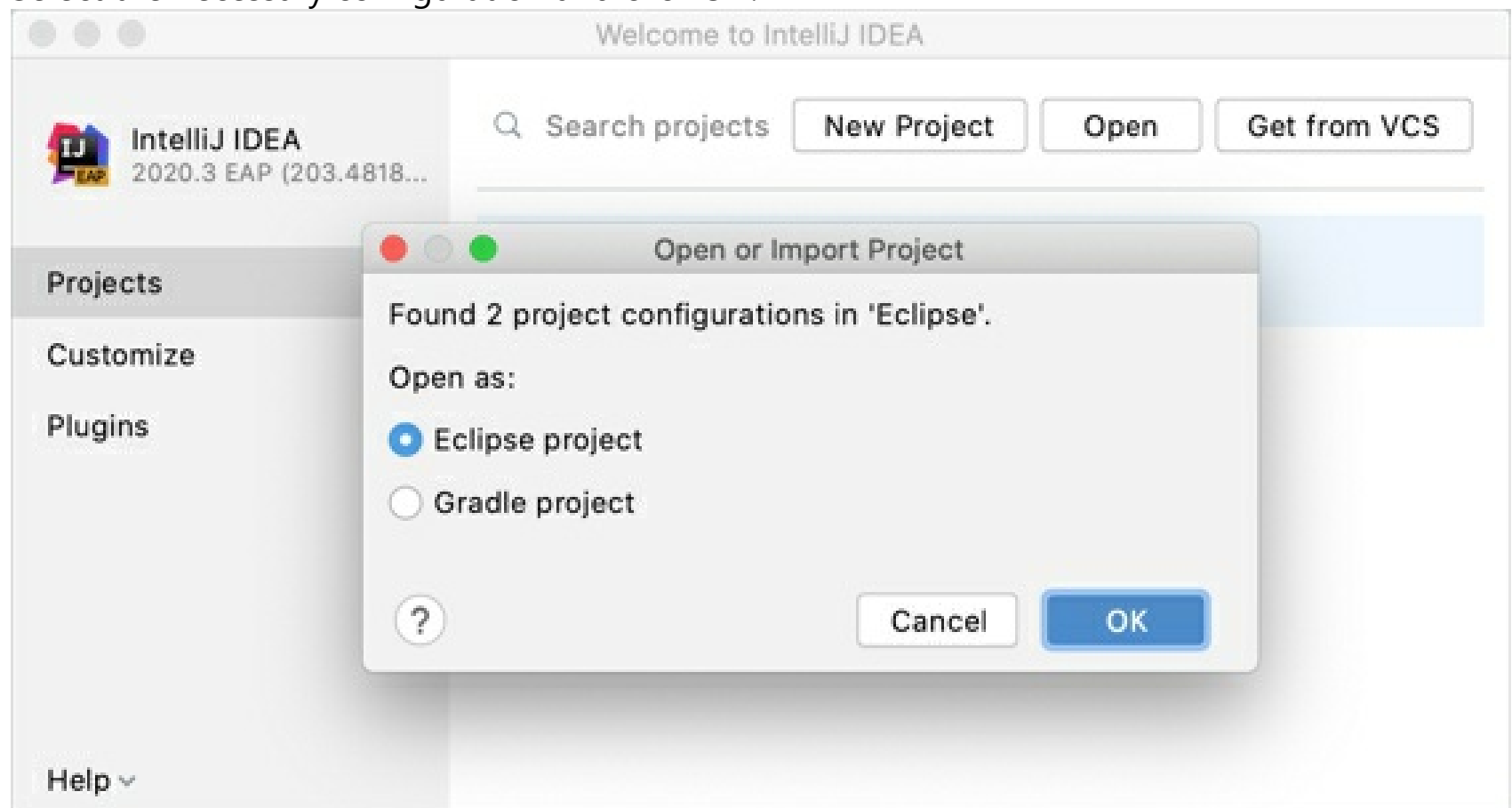
If the Welcome screen opens, click **Open**.

Otherwise, from the main menu, select **File | Open**.

2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. When you import or clone a project for the first time, IntelliJ IDEA analyzes it. If the IDE detects more than one configuration (for example, Eclipse and Gradle), it prompts you to select which configuration you want to use.

If the project that you are importing uses a build tool, such as Maven or Gradle, we recommend that you select the build tool configuration.

Select the necessary configuration and click **OK**.



The IDE pre-configures the project according to your choice. For example, if you select **Gradle project**, IntelliJ IDEA executes its build scripts, loads dependencies, and so on.

4. If you have been working with another project, select whether you want to open the new project in a new dialog or in the current one.

For instructions on how to get a project from version control, refer to Check out a project from a remote host (clone).

Import a project with settings

This section describes the functionality that is available out of the box. If you are using a framework plugin, refer to the corresponding documentation section.

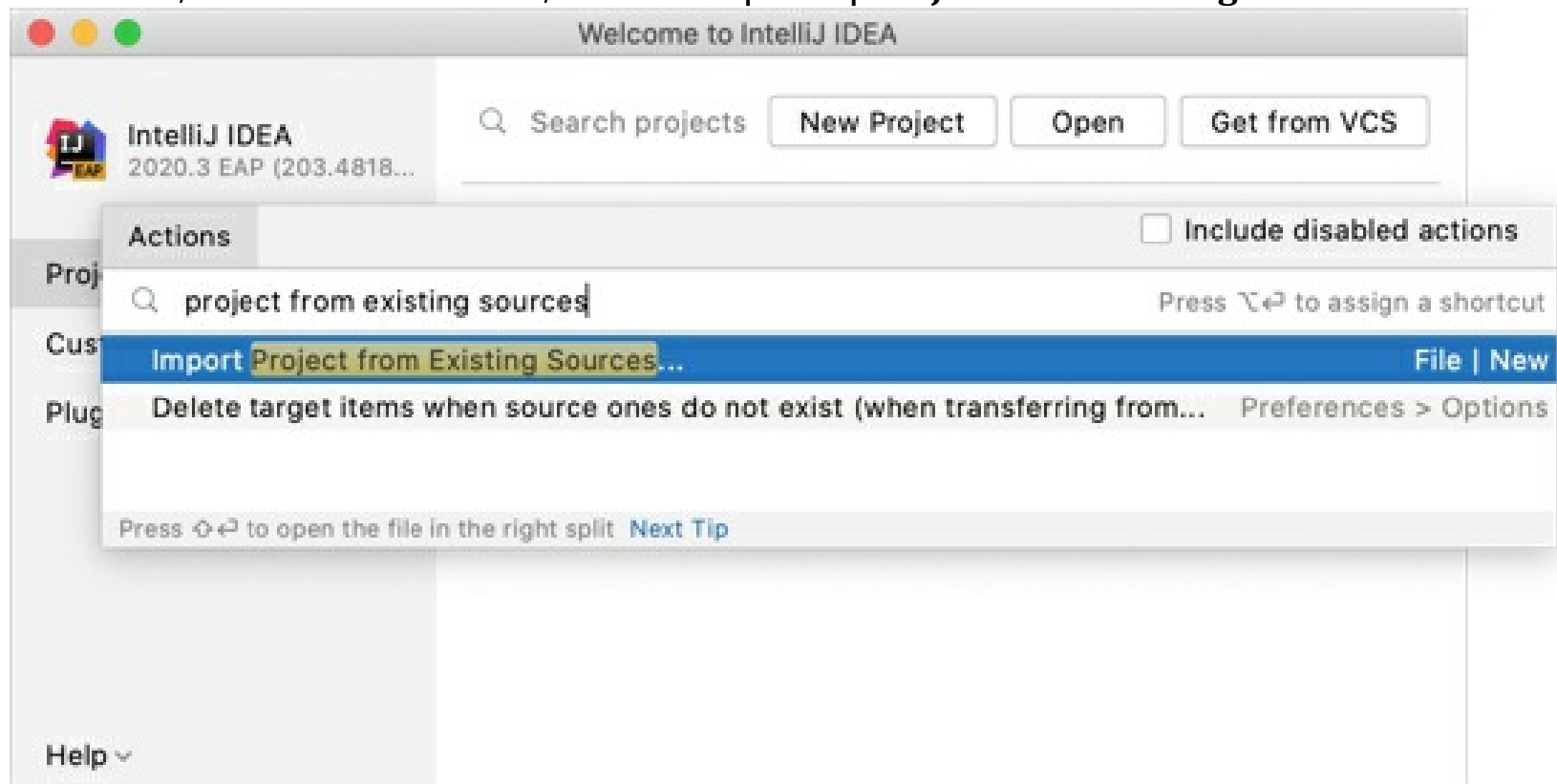
Import a project from an external model

Use this type of import if your project comes from an external model and you want to import it as a whole. In this case, IntelliJ IDEA interprets the project files (for example, your Eclipse project will be migrated to IntelliJ IDEA).

1. Launch IntelliJ IDEA.

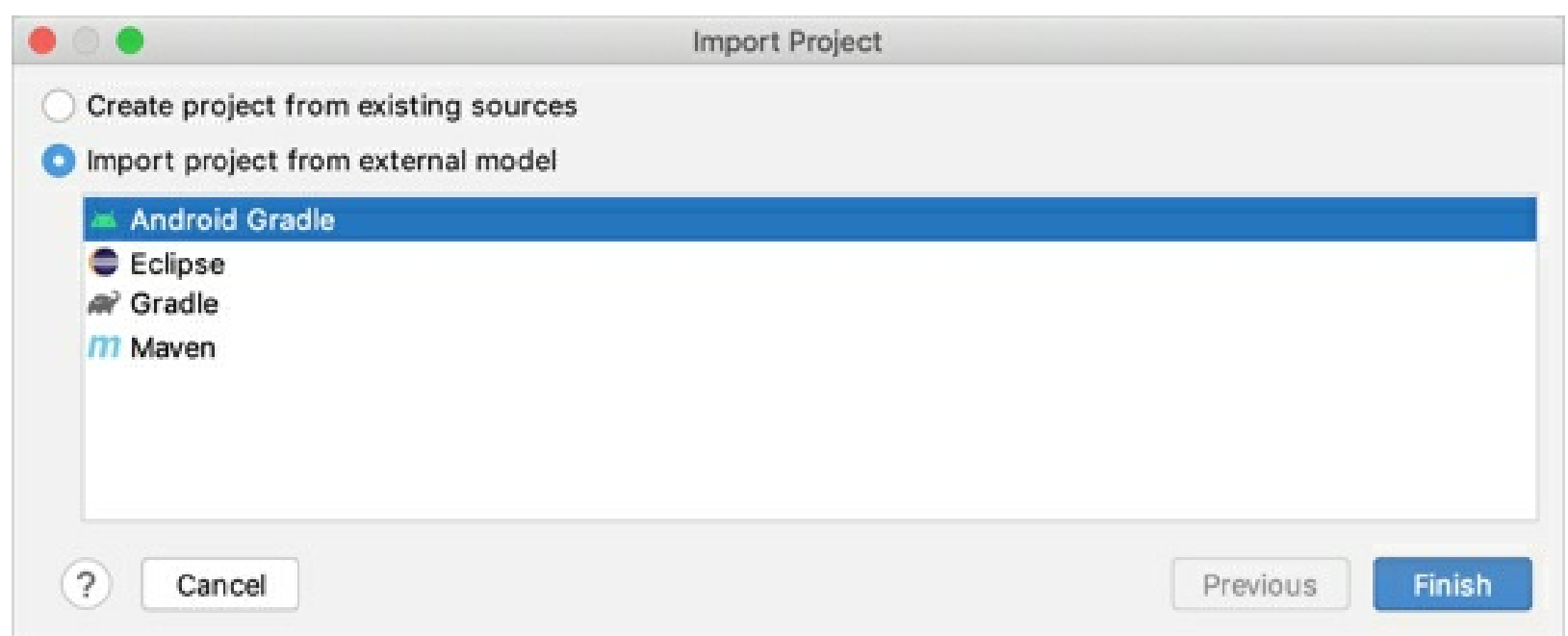
If the Welcome screen opens, press **Ctrl+Shift+A**, type **project from existing sources**, and click the **Import project from existing sources** action in the popup.

Otherwise, from the main menu, select **File | New | Project from Existing Sources**.



2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. Select the external model that your project uses:
 - Eclipse
 - Maven
 - Gradle or Android Gradle: select the necessary build tool and click **Finish**.

For Maven and Gradle projects, the IDE configures the settings automatically. You will be able to adjust them after the project is imported.



4.

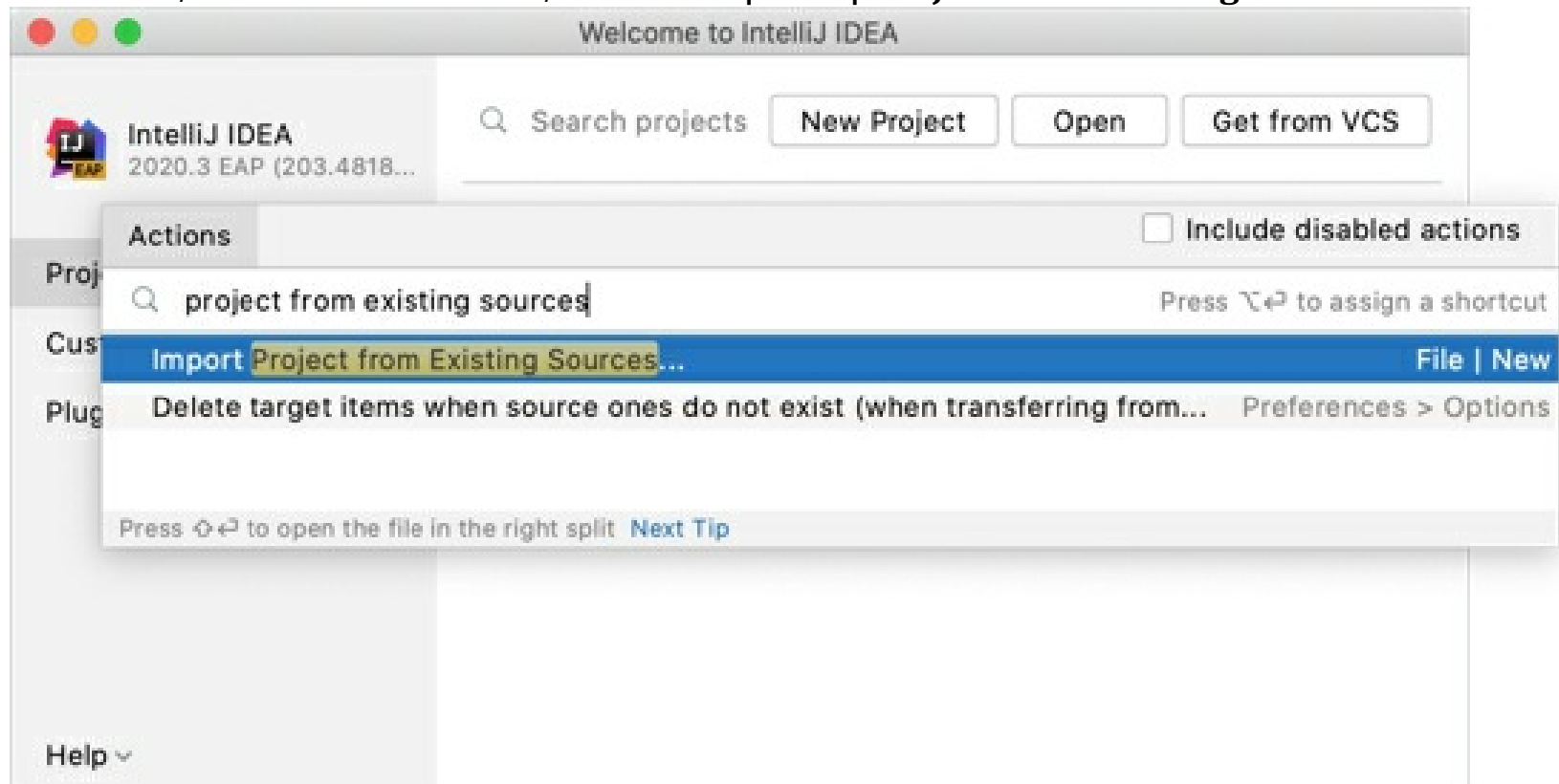
Create a project from existing sources

Use this type of import to create an IntelliJ IDEA project over the existing source code that is not necessarily an exported project.

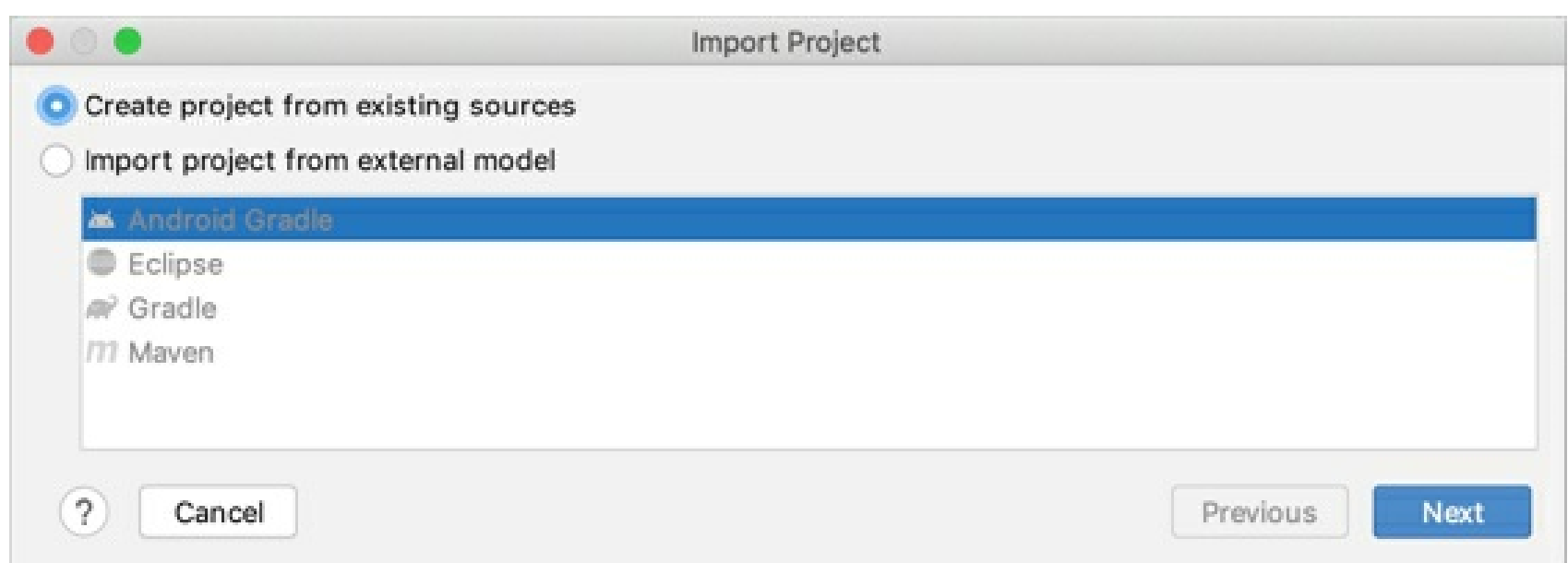
1. Launch IntelliJ IDEA.

If the Welcome screen opens, press **Ctrl+Shift+A**, type **project from existing sources**, and click the **Import project from existing sources** action in the popup.

Otherwise, from the main menu, select **File | New | Project from Existing Sources**.



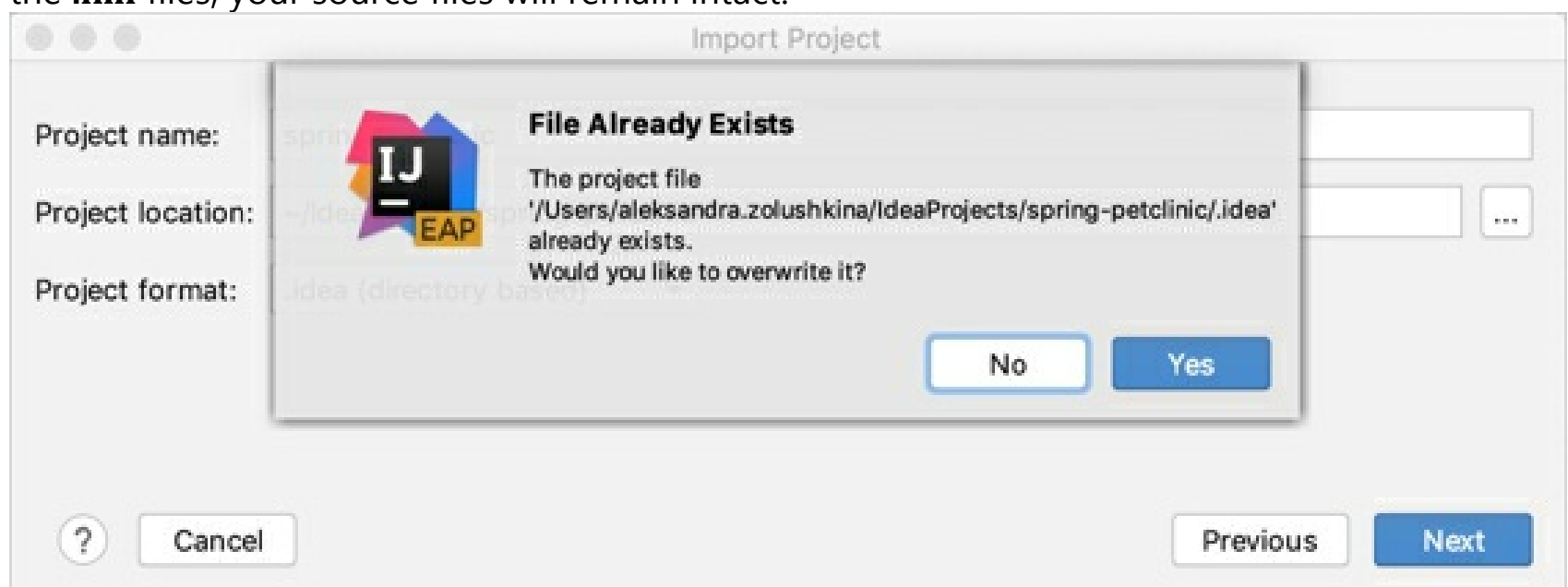
2. In the dialog that opens, select the directory in which your sources, libraries, and other assets are located and click **Open**.
3. Select the **Create project from existing sources** option and click **Next**.



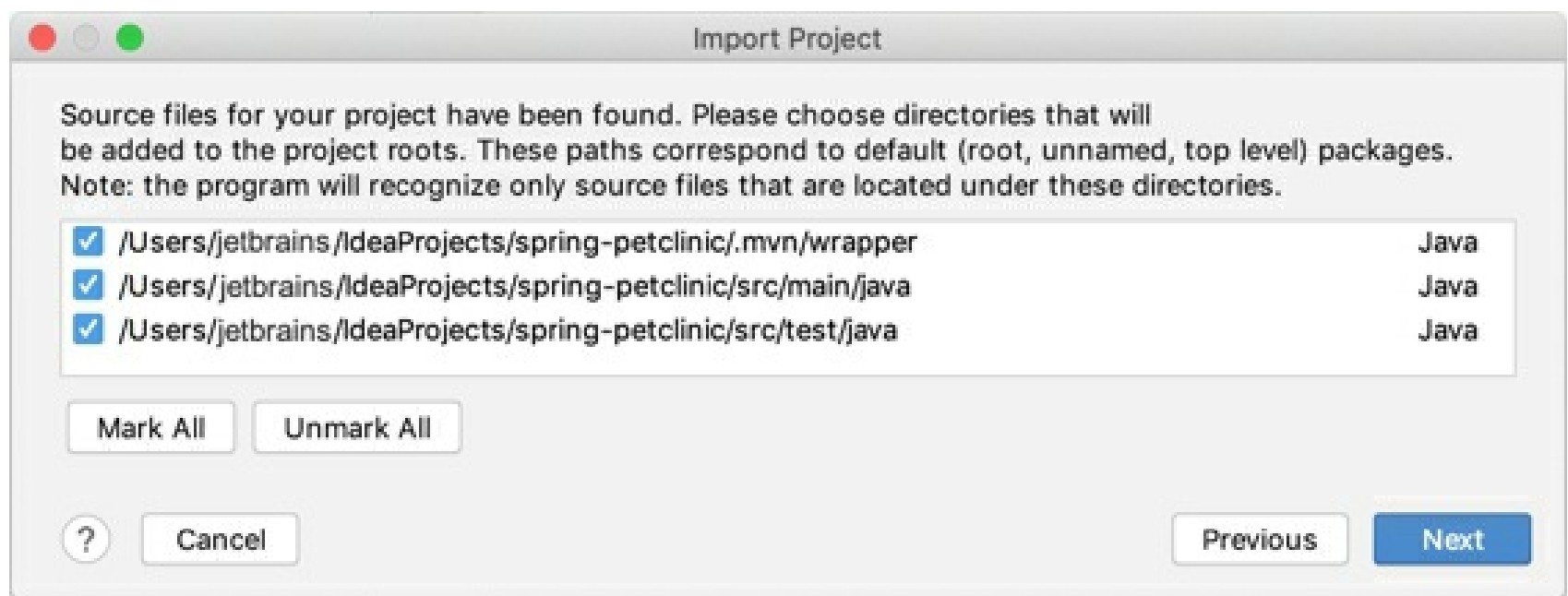
4. Specify the name and location and select a format for the new project. It's recommended that you use the directory-based format.

Click **Next**.

If you are importing the project to the same directory, the IDE asks you whether you want to overwrite it. If you click **Yes**, IntelliJ IDEA will overwrite the files in **.idea** directory and the **.iml** files, your source files will remain intact.



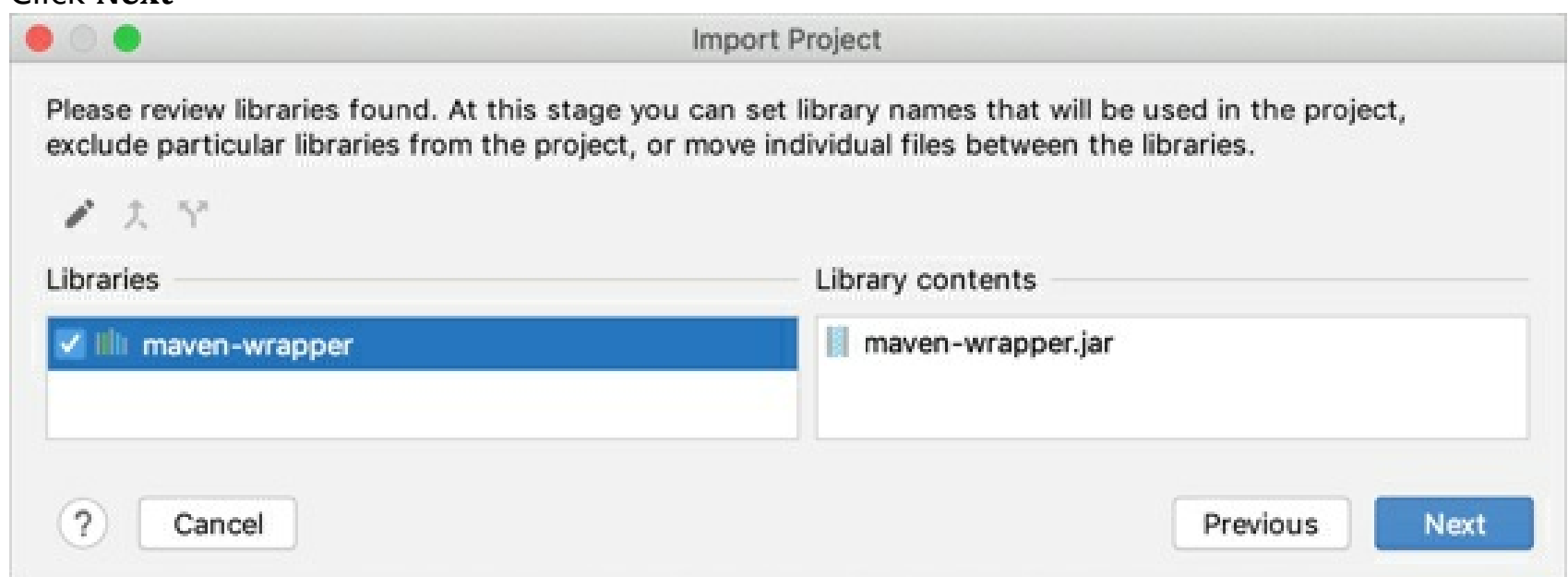
5. Select the directories that you want to use as source root directories (folders with your source code) and click **Next**.



6. Select the libraries that you want to add to the new project.

You can join several selected libraries or archives into a new library by clicking  or split the selected library into two by clicking .

Click **Next**



7. Review module structure: select the modules that you want to include in your project.

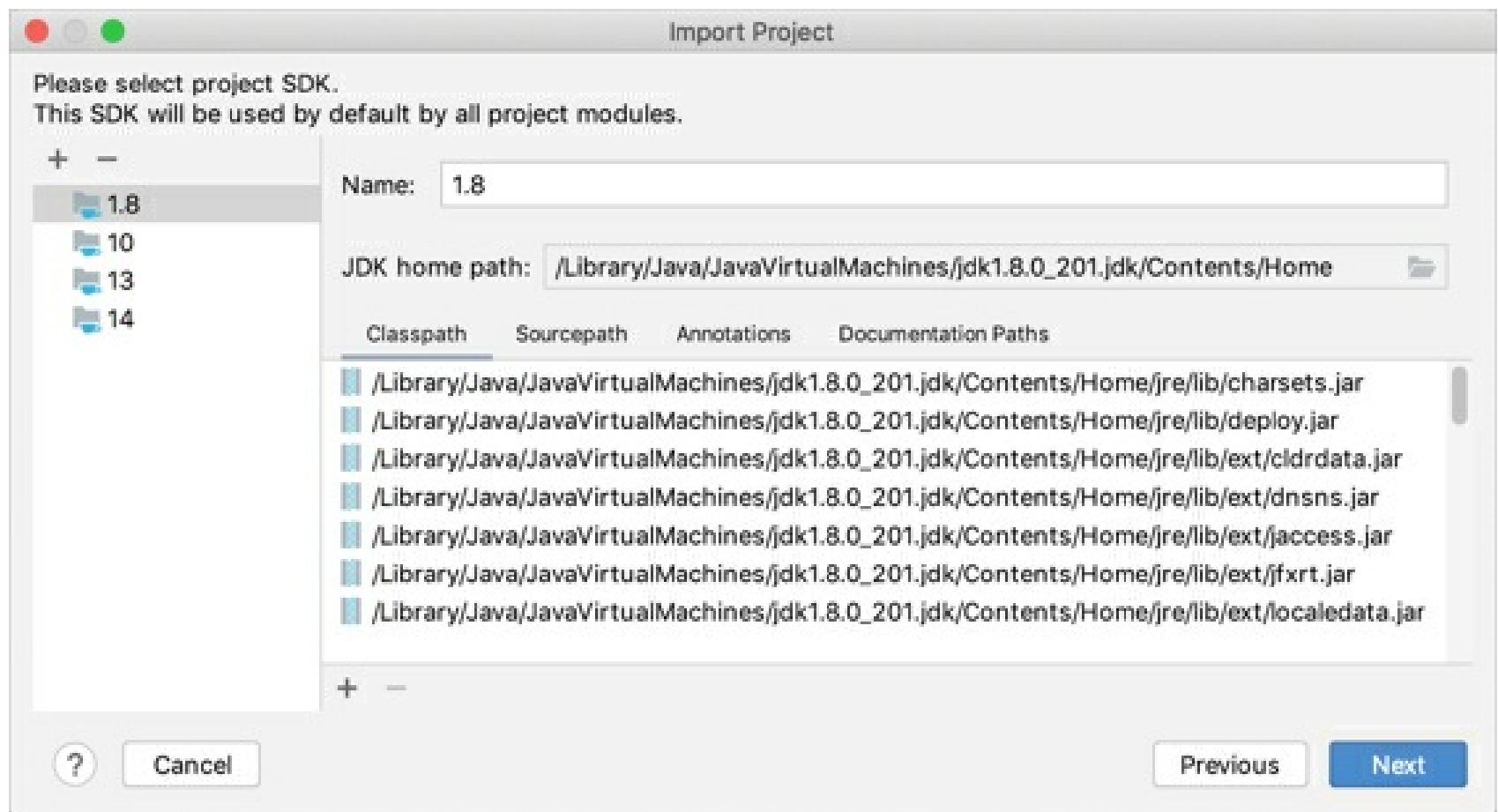
You can merge several modules into one by clicking  or split the selected module into two by clicking .

Click **Next**

8. Specify the SDK that you want to use.

If the necessary SDK is already defined in IntelliJ IDEA, select it from the list on the left. Otherwise, click  and add a new SDK.

Click **Next**.



9. Enable support for the detected frameworks and technologies: select checkboxes next to the necessary items.

You can also specify how the files-indicators should be grouped: by type (by framework) or by directory (by location).

10. Click **Finish**.

Add items to your project

Once you have created a project, you can start adding new items: create directories and packages, add new classes, import resources, and extend your project by adding more modules.

Create new items

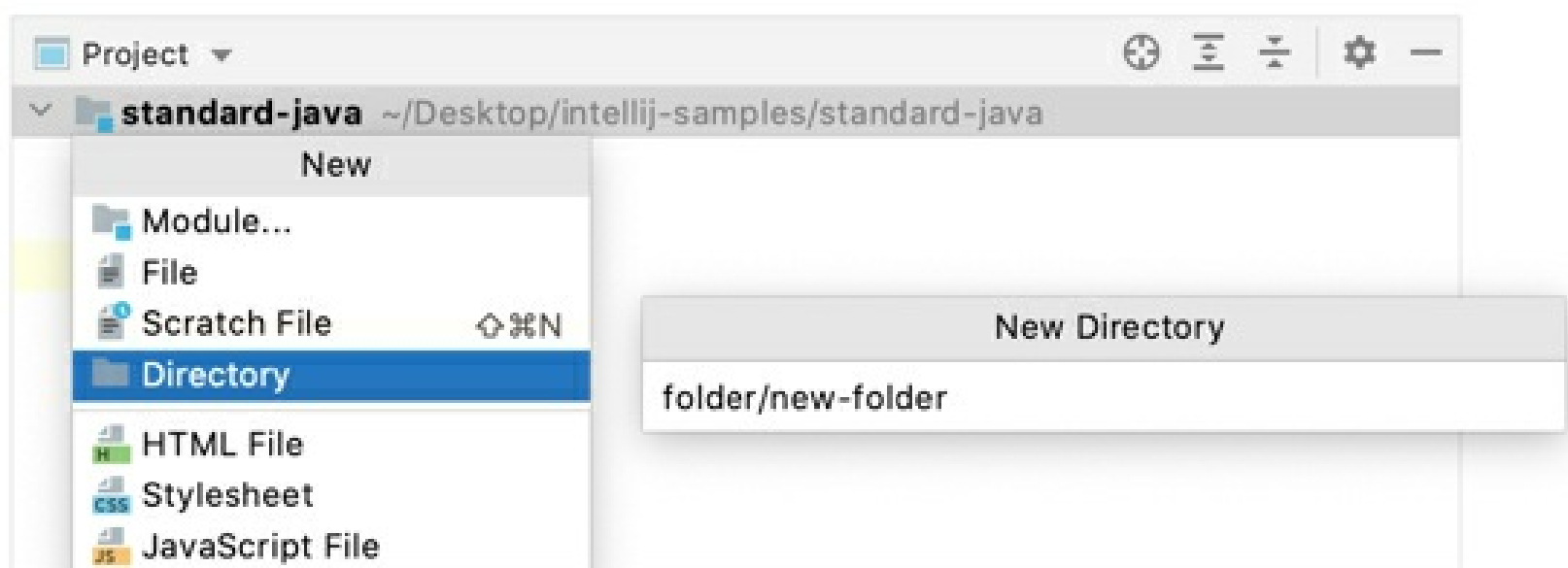
Create a new directory

1. In the **Project** tool window (Alt+1), right-click the node in which you want to create a new directory and select **New | Directory**.

Alternatively, select the node, press Alt+Insert, and click **Directory**.

2. Name the new directory and press Enter.

If you want to create several nested directories, specify their names separated with slashes, for example: **folder/new-folder**.



Create a new package

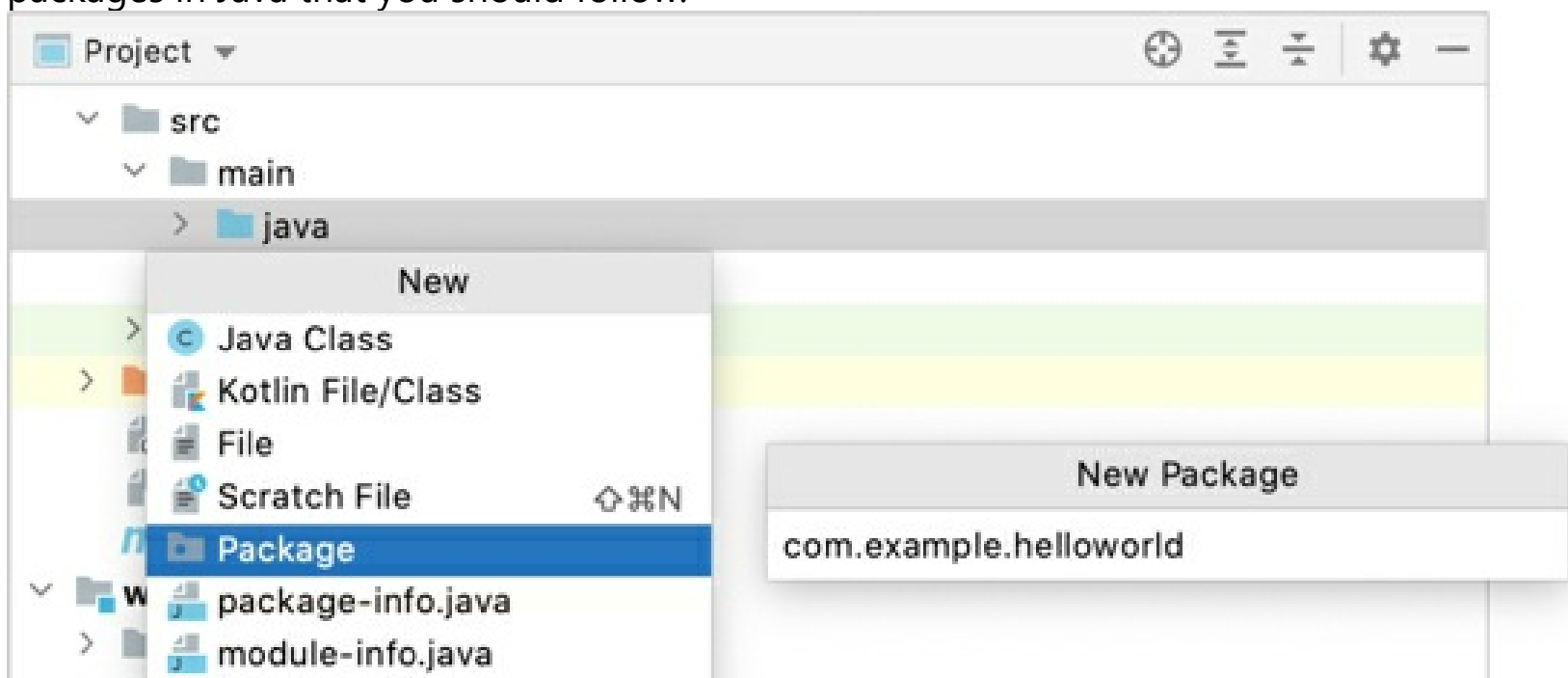
Packages in Java are used for grouping classes that belong to the same category or provide similar functionality, for structuring and organizing large applications with hundreds of classes.

1. In the **Project** tool window (Alt+1), right-click the node within the Sources Root or Test Sources Root in which you want to create a new package, and click **New | Package**.

Alternatively, select the node, press Alt+Insert, and click **Package**.

2. Name the new package and press Enter.

Write package names in lowercase letters. There are some other naming conventions for packages in Java that you should follow.



Create a new empty file

1. In the **Project** tool window (Alt+1), right-click the node in which you want to create a new file and click **New | File**.

Alternatively, select the node, press Alt+Insert, and click **File**.

2. Name the new file and specify its extension, for example: **File.js**, and press Enter.

If the extension you have specified is not associated with any of the file types recognized by IntelliJ IDEA, the Register New File Type Association dialog is displayed. In this dialog, you can associate the extension with one of the recognized file types.

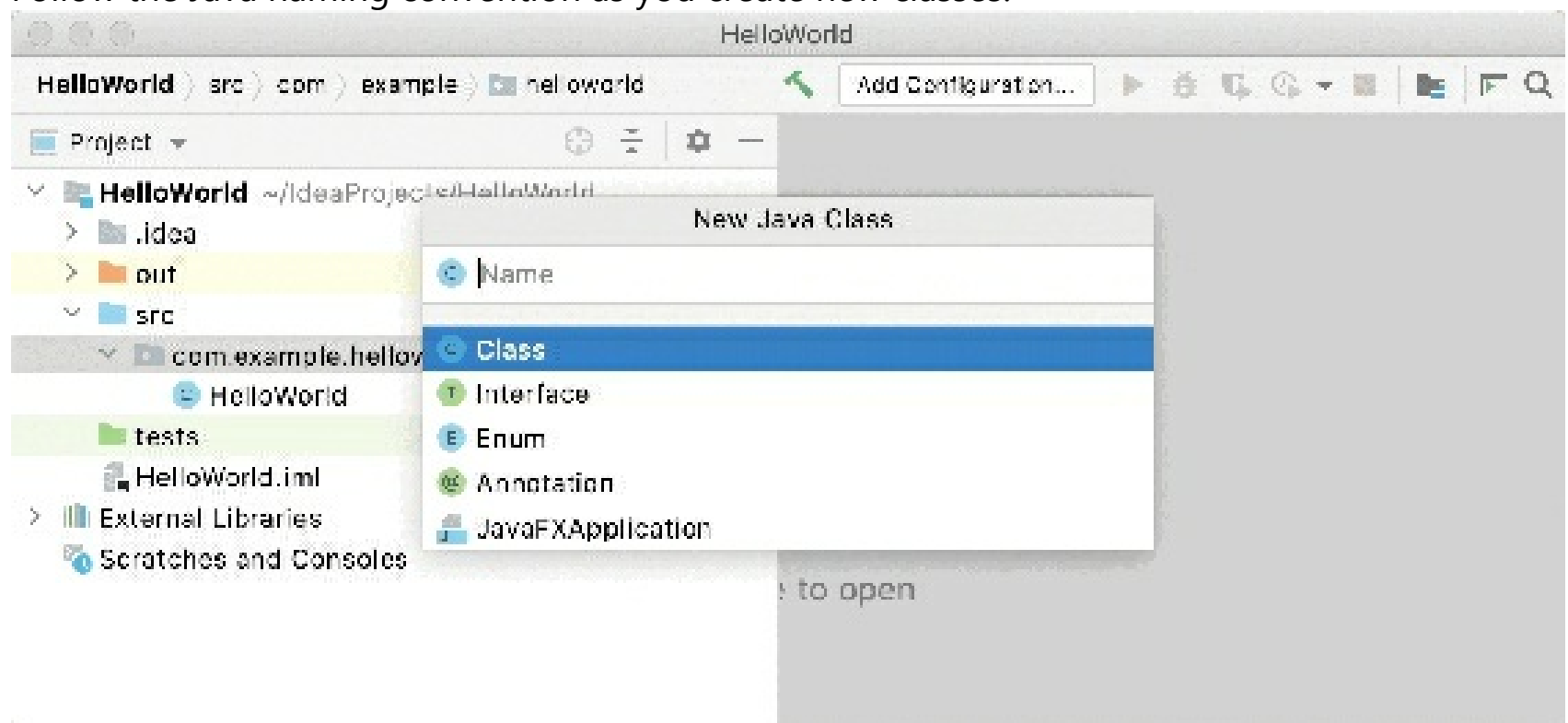
Create a new Java class

1. In the **Project** tool window (Alt+1), right-click the node in which you want to create a new class and select **New | Java Class**.

Alternatively, select the node, press Alt+Insert, and select **Java Class**.

2. Name the new class and press Enter.

Follow the Java naming convention as you create new classes.



Gif

Together with the file, IntelliJ IDEA automatically generates the class declaration.

This is done by means of file templates. Depending on the type of the file that you create, the IDE inserts initial code and formatting that is expected to be in all files of that type. For more information on how to use and configure templates, refer to File templates.

You can create a class together with a package. To do so, press Alt+Insert in the **Project** tool window, select **Java Class**, and specify the fully qualified name of the class, for example: `com.example.helloworld.HelloWorld`. For more information, refer to Create a package and a class.

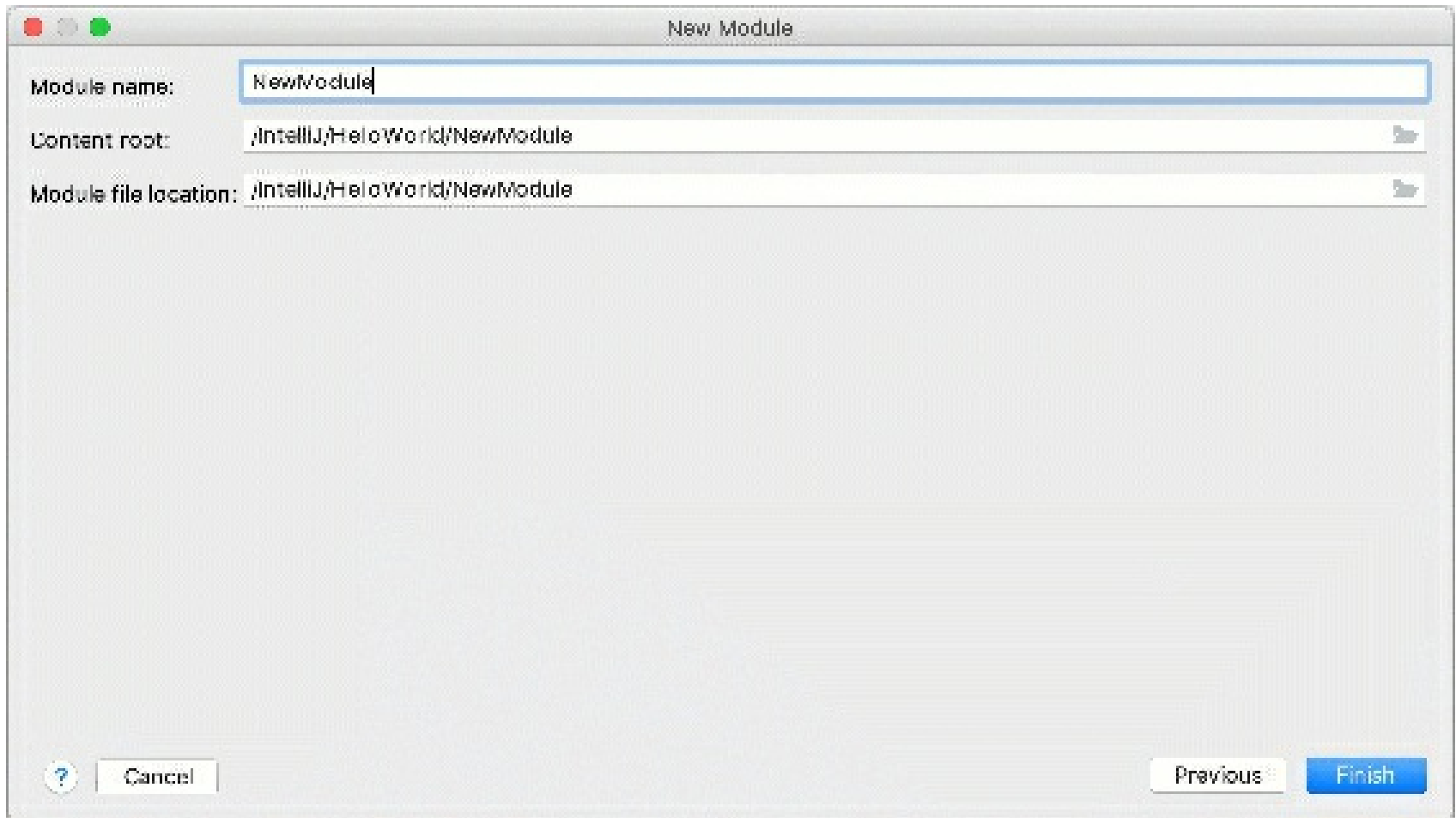
Create a new module

Modules allow you to combine several technologies and frameworks in one application. In IntelliJ IDEA, you can create several modules in one project and each of them can be responsible for its own framework.

1. Right-click the top-level directory in the **Project** tool window and select **New | Module**. The **New Module** wizard opens.
2. From the list on the left, select a module type.
3. In the right-hand part of the dialog, select an SDK that you want to use from the **Module SDK** list. You can use the project SDK or specify a new one.
4. In the **Additional Libraries and Frameworks** section, select additional assets that you

want to use in this module.

- On the next step, name the module and specify the location of the content root and the **.iml** file. You can place them within or outside of the project.
- Click **Finish**.



Gif

For more information on modules in IntelliJ IDEA, refer to Modules.

Import items

Import files

You can import files to your project using any of the following ways:

- Drag the file from your system file manager to the necessary node in the **Project** tool window.
- Copy the file in the system file manager by pressing **Ctrl+C** and then paste in to the necessary node in the IDE **Project** tool window by pressing **Ctrl+V**.
- Manually move the file to the project folder in your system file manager.

Example: Import an image

Images belong to resource files. They should be stored in a dedicated folder – Resources Root. If you don't have this folder in your project, create a new directory, right-click it in the **Project** tool window, and select **Mark Directory as | Resources Root**.

- Copy the file in the file manager and then paste in to the folder with resource files in the IDE **Project** tool window.
- In the dialog that opens, edit the filename and the target location if necessary. Click **OK**.
- Right-click the pasted image in the **Project** tool window and select **Copy | Path From Source Root**.
- In the class in which you want to use the image, place the caret at the necessary line and press **Ctrl+V** to paste the path to the image.

Run the class to make sure that the image is inserted correctly.

Import an existing module

You can import a module to your project by adding the **.iml** file from another project:

- From the main menu, select **File | New | Module from Existing Sources**.

2. In the dialog that opens, specify the path the **.iml** file of the module that you want to import, and click **Open**.

By doing so, you are attaching another module to the project without physically moving any files. If you don't need the modules to be located in one folder, the module import is finished, and you can start working with the project normally.

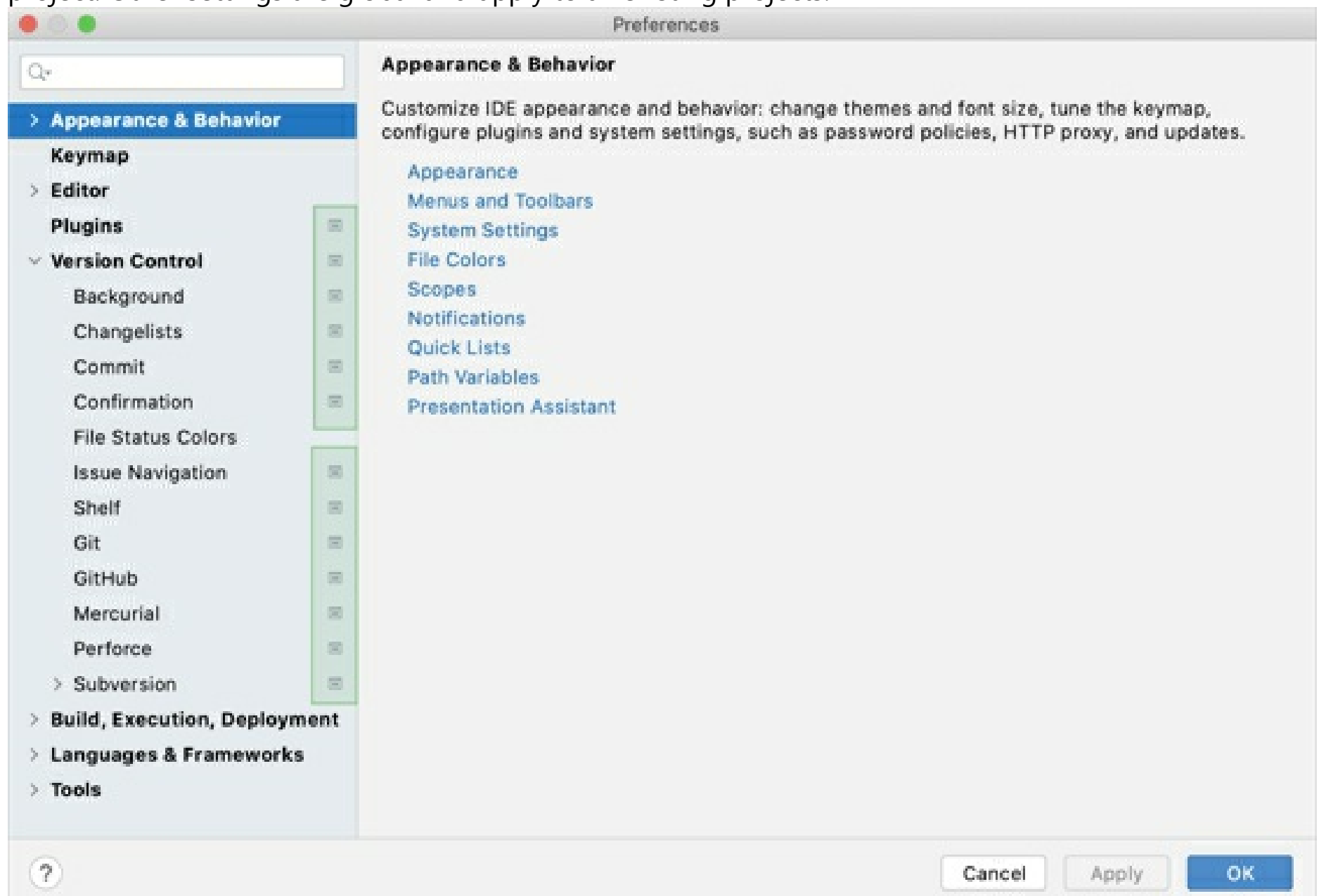
If you want the modules in the same folder, in the **Project** tool window, drag the imported module to the top-level directory. In this case, the contents of the imported module will be physically transferred to your project's folder.

Project settings

Project settings apply to the current project only. They are stored together with other project files in the **.idea** directory in the **.xml** format. For example, projects keep VCS settings, code style spellchecker settings, the list of language injections, and so on. These settings are placed under version control automatically together with your application code when you send it to your VCS.

To configure project settings, select **IntelliJ IDEA | Preferences** for macOS (Ctrl+Alt+S) or **File | Settings** for Windows and Linux.

In the **Settings/Preferences** dialog, settings that are marked with the  icon apply only to the current project. Other settings are global and apply to all existing projects.



If you want to share project settings between already existing projects, you can use the Settings Repository or the Settings Sync plugin. You can also export the settings to a ZIP archive and import it later to other IDE instances.

For the detailed description of every option in the settings, refer to Settings / Preferences dialog.

Default settings for new projects

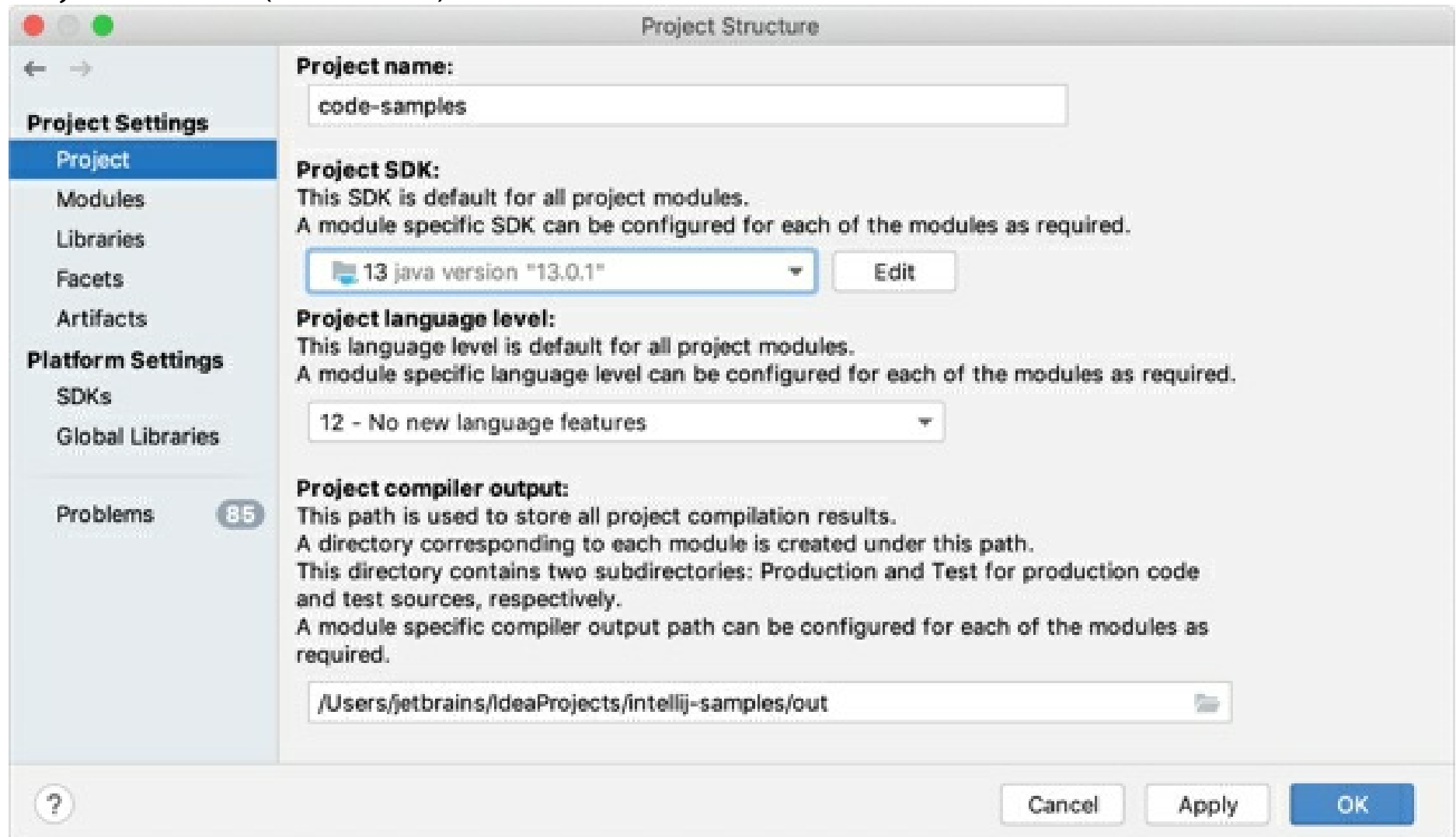
You can configure project settings not only for the current project, but for all projects that you will create later. This means that you can set the new default settings for your projects.

- From the main menu, select **File | New Projects Settings | Settings/Preferences for New Projects**.

Project structure settings

Project structure settings are stored together with other project files in the **.idea** directory in the **.xml** format. These settings include SDKs, project compiler output paths, and libraries that are available for all modules within a project.

To change the project structure settings, click on the toolbar and select **Project Structure** or select **File | Project Structure** (Ctrl+Alt+Shift+S) from the main menu.



Project SDK

An SDK is a collection of tools that you need to develop an application for a specific software framework. If the necessary SDK is installed on your computer, but not defined in the IDE, select **Add SDK | 'SDK name'**, and specify the path to the SDK home directory.

To develop Java-based applications, you need a JDK (Java Development Kit). For the detailed instructions on how to set up the project JDK, refer to Set up the project JDK.

To view or edit the name and contents of the selected SDK, click **Edit**. For more information on SDKs and how to work with them, refer to SDKs.

For Gradle projects, refer to Gradle JVM selection and for Maven projects, refer to Change the JDK version in a Maven project

Project language level

Language level defines coding assistance features that the editor provides. Language level can differ from your project SDK. For example, you can use the JDK 9 and set the language level to 8. This makes the bytecode compatible with Java 8, while inspections make sure you don't use constructs from Java 9.

Language level also affects compilation. If you don't manually configure the target bytecode version for your compiler (**Settings/Preferences | Build, Execution, Deployment | Compiler | Java Compiler**), it will be the same as the project language level.

For each module, you can configure its own language level.

In some cases, you can select the `Preview` language level that allows you to use preview features as described in Java Language Specification. Refer to Preview features policy for additional information about preview features support in IntelliJ IDEA.

Project compiler output

Compiler output path is the path to the directory in which IntelliJ IDEA stores the compilation results.

Click the  icon to select the output directory. In this directory, the IDE creates two sub-directories:

- **production** for production code.
- **test** for test sources.

In these subdirectories, individual output directories will be created for each of your modules. The output paths may be redefined at the module level.

Project libraries

Project-level libraries are available for all modules with a project. To configure a project library, in the **Project Structure** dialog, click **Libraries**. For more information, refer to Libraries.

Default structure for new projects

You can configure project structure settings not only for the current project, but for all projects that you will be creating later. This means that you can set the new default structure for your projects.

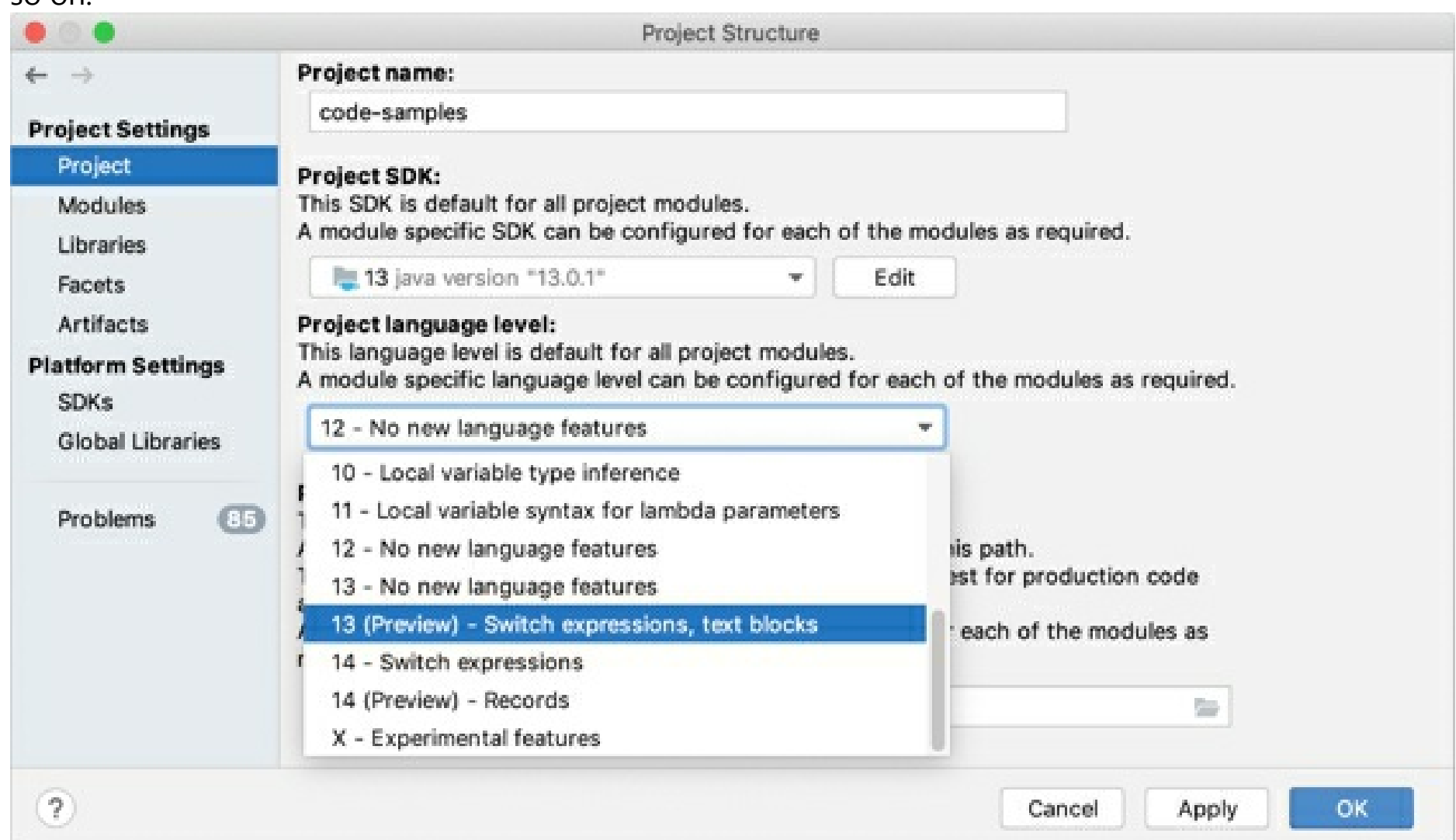
- From the main menu, select **File | New Projects Settings | Structure for New Projects**.

If you want to share project settings between already existing projects, you can use the Settings Repository or the Settings Sync plugin. You can also export the settings to a ZIP archive and import it later to other IDE instances.

Preview features policy

IntelliJ IDEA is committed to supporting the preview features from the latest Java release and, if possible, the preview features of whatever release is coming next.

For example, the IntelliJ IDEA v2019.2 supports Java 12 and 13 preview features. Note that v2019.3 drops the support for Java 12 preview features as IntelliJ IDEA 2019.3 is released after the release of Java 13, and so on.



Save projects as templates

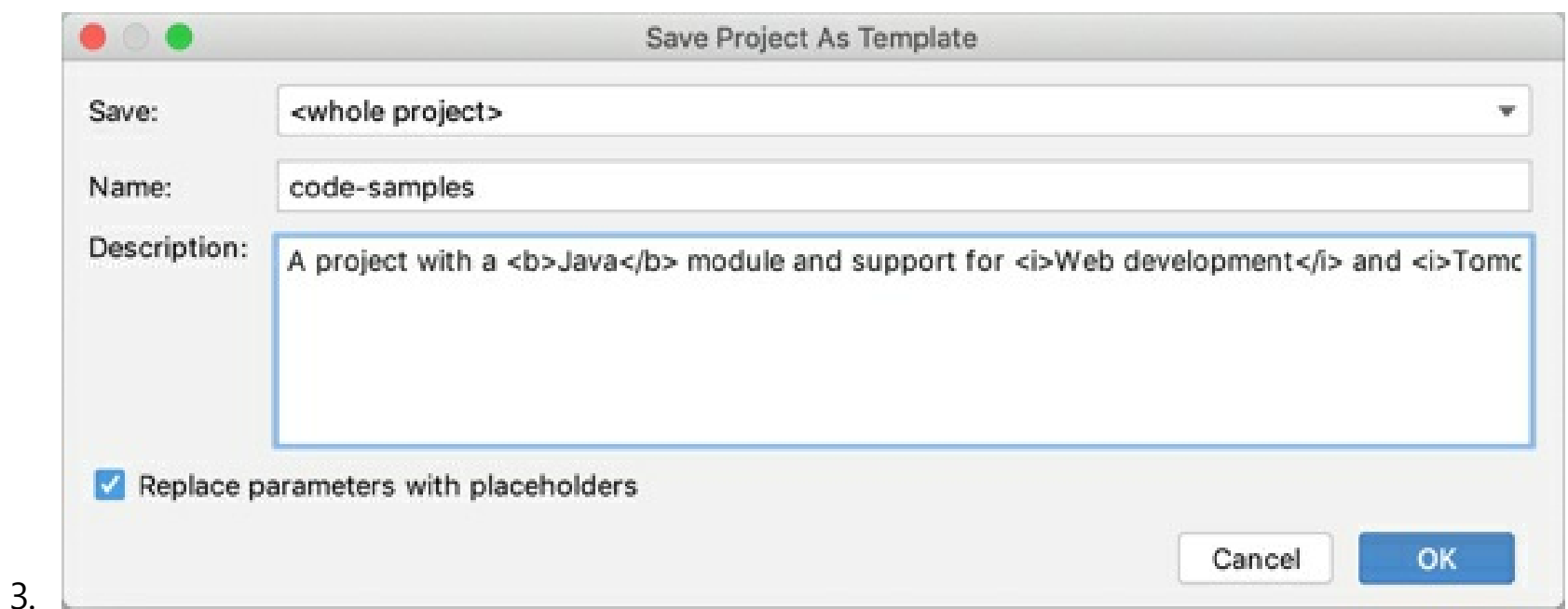
If you have a project that you want to reproduce when creating new projects, you can save it as a custom project template. The IDE recreates the project tree with files and folders, build configurations, libraries, SDKs, and language level settings.

IntelliJ IDEA saves all project settings from the **.idea** folder to the template. So if you want your new projects to have some predefined run/debug configurations, select the **Store as project file** checkbox in these configurations before you save the template.

Save a project as a template

1. From the main menu, select **Tools | Save Project as Template**.
2. In the dialog that opens, name the template and configure the options:
 - **Save:** if the project contains more than one module, select whether you want to create a template from the whole project or from one of the modules. Otherwise, this option is not available.
 - **Description:** you can use the `<b>` and `</b>`, and `<i>` and `</i>` tags for formatting.
 - **Replace parameters with placeholders** (recommended): IntelliJ IDEA has a number of built-in project template options. If you select this checkbox, you will be able to configure these options (for example, specify the base Java package or another application server) when creating a new template-based project or module.

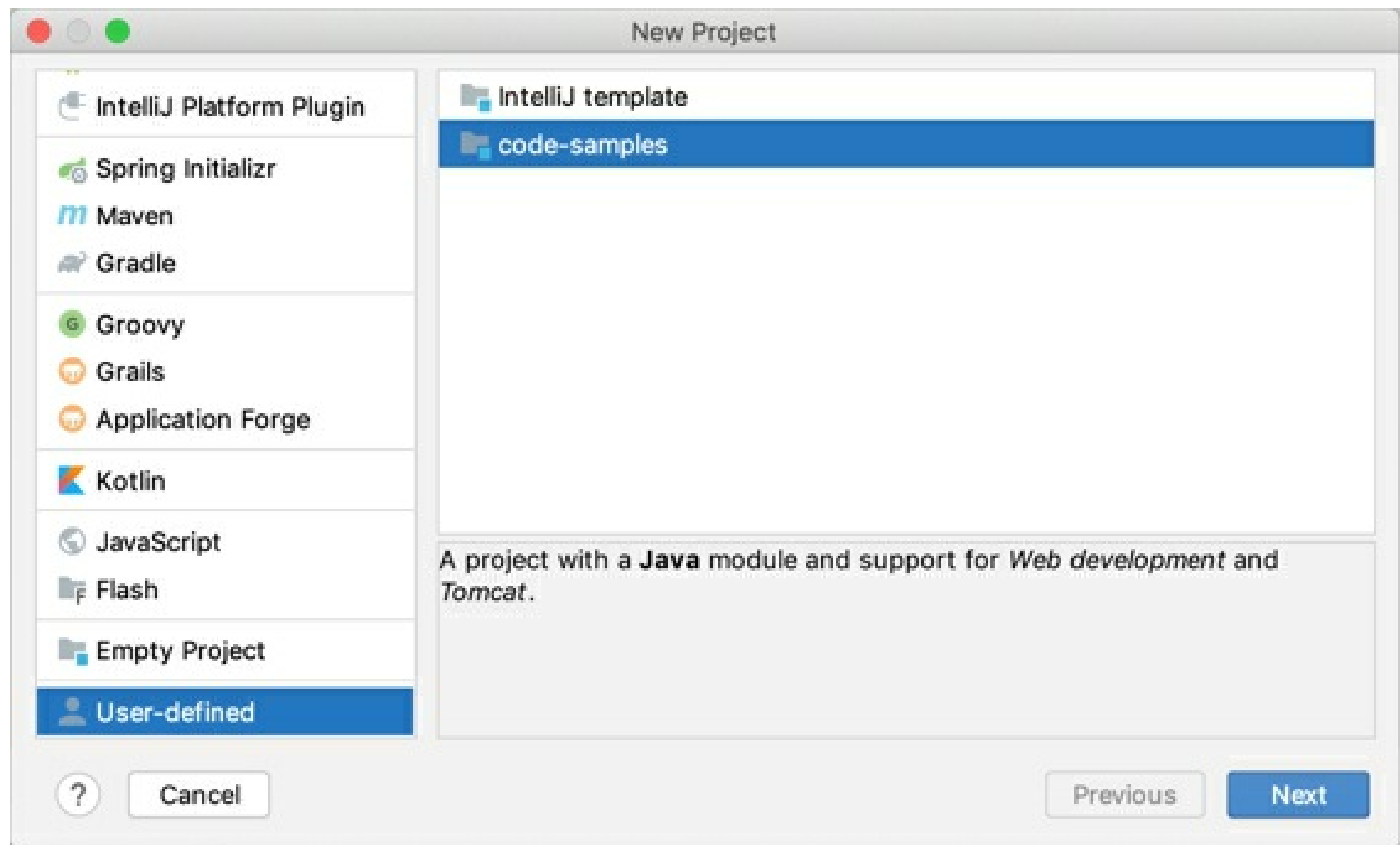
This option doesn't allow you to add your custom parameters to a project.



Templates are saved to the **projectTemplates** folder in the IDE configuration directory.

Create a project from a template

1. Click **New Project** on the **Welcome** screen or select **File | New Project** from the main menu.
2. In the dialog that opens, click the **User-defined** node and select the required template.



Delete project templates

1. From the main menu, select **Tools | Manage Project Templates**.
2. Select the template that you want to remove and click .

Open, close, and move projects

Open and reopen projects

Open a project

- From the main menu, select **File** and click **Open** or **Open Recent** if you have worked with the project lately.

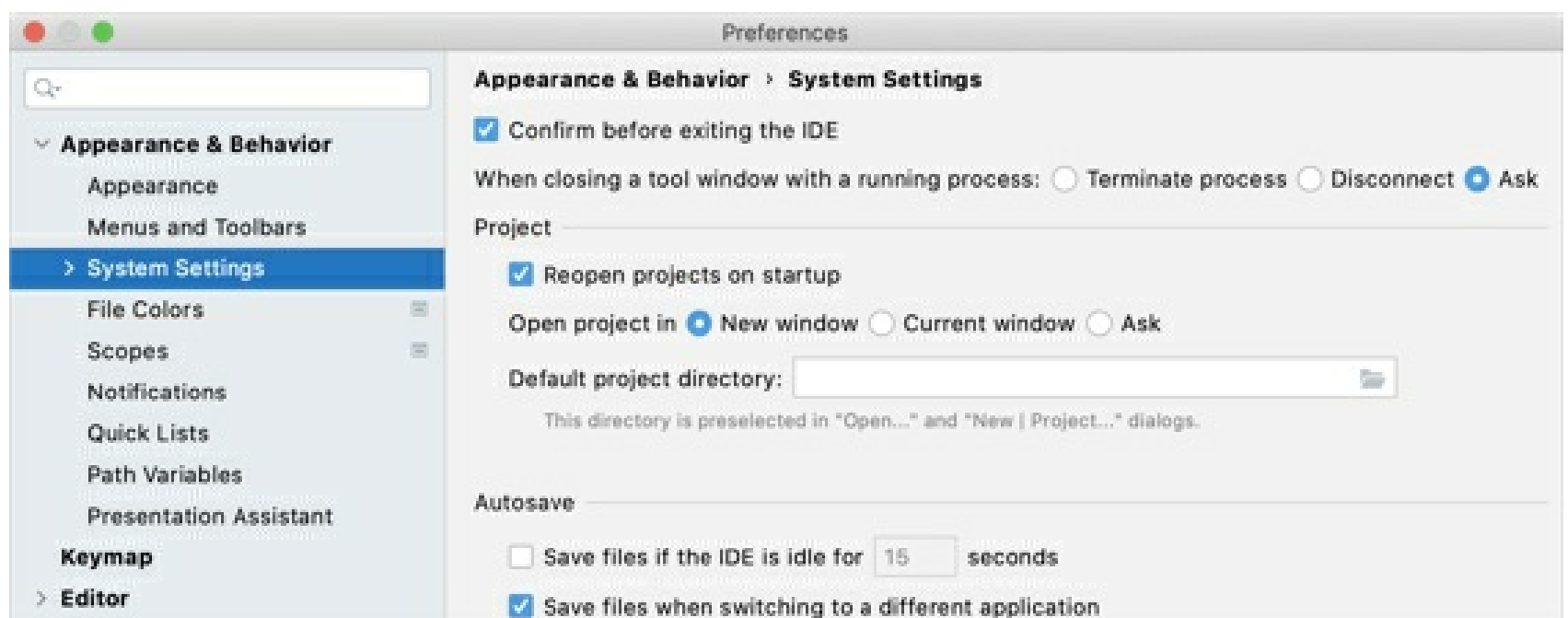
Alternatively, you can open projects from the command line.

If you are opening a project for the first time, and this project has several configurations (for example Eclipse and Maven), the IDE will ask you which configuration to load. Refer to Open a project (simple import) for more information on how to open such a project correctly.

Open projects in a new or the same window

By default, when you launch the second of any subsequent project, the IDE asks you how you want to open it: in a new window or in the same window. You can configure the default behavior for such cases in the settings:

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | System Settings**.
2. In the **Project** section, select the necessary **Open project in** option:
 - **New window**: open every project in a separate window.
 - **Current window**: close the current project and open the new one in the same window.
 - **Ask**: show a dialog with actions to choose from.



3. Apply the changes and close the dialog.

Always reopen projects

If you quit the IDE having multiple opened projects, they all will be reopened the next time you launch IntelliJ IDEA. If you don't want to automatically reopen your projects, you can change this behavior in the settings:

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | System Settings**.
2. In the **Project** section, clear the **Reopen projects on startup** checkbox.
3. Apply the changes and close the dialog.

Switch between projects

If you have several opened projects at the same time, you can switch between them using the following options:

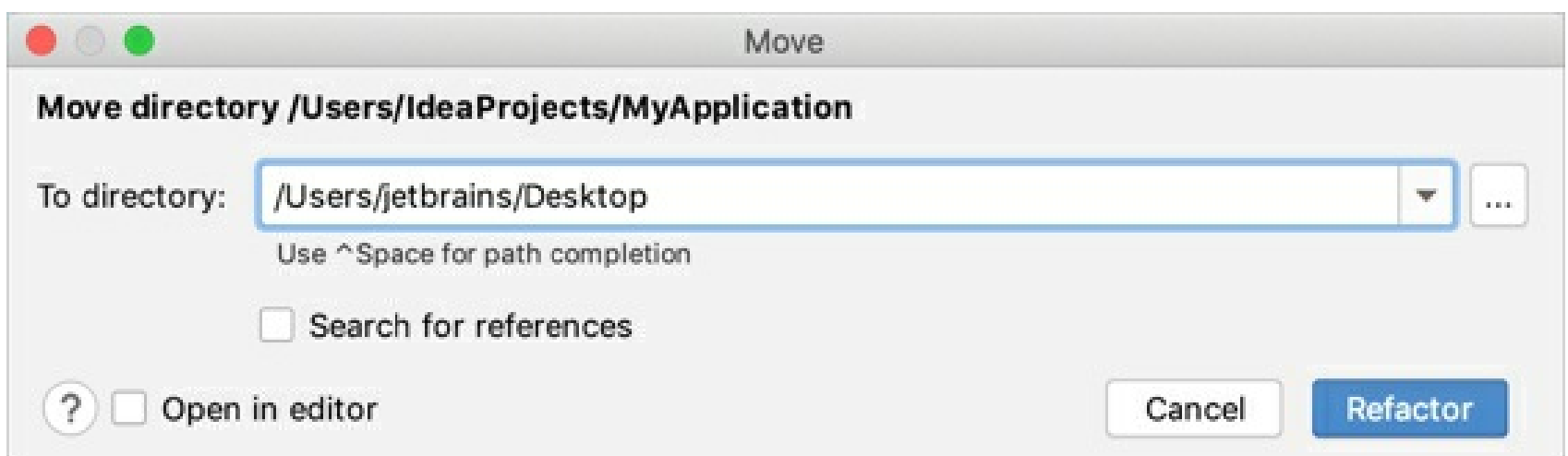
- Switch to the next project window: `Ctrl+Alt+] (Window | Next Project Window)`
- Switch to the previous project window: `Ctrl+Alt+[ (Window | Previous Project Window)`

Alternatively, open the **Window** menu and select the project to which you want to switch.

Change project location

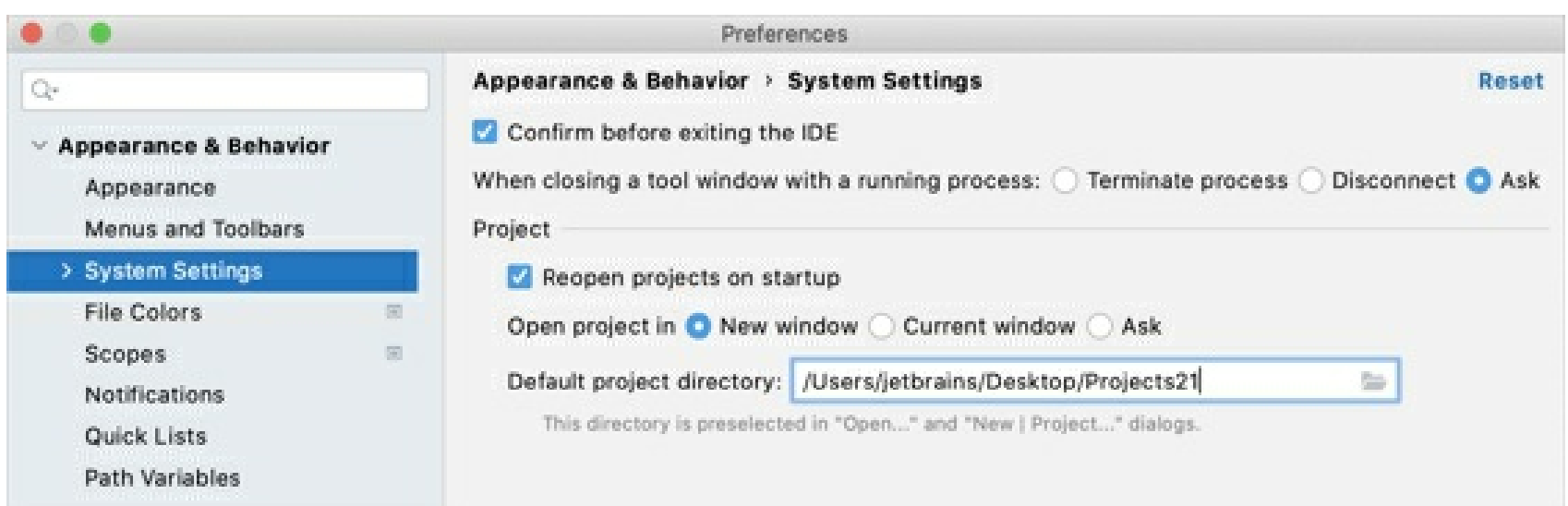
Move a project to another location

1. In the **Project** tool window `Alt+1`, right-click the root directory of your project and select **Refactor | Move directory** (`F6`).
2. In the dialog that opens, specify a new location for the project and click **Refactor**.



Change the default location for projects

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | System Settings**.
2. In the **Default project directory** field, specify the path to the folder in which you want to store your projects.
3. Apply the changes and close the dialog.



Close projects

If you need to close only one project, you can either close the project window or select **File | Close Project** from the main menu.

If you work with multiple projects, use the following actions to close many projects at once:

Close all projects

- From the main menu, select **File | Close All Projects**.

This action closes all projects that are currently opened in IntelliJ IDEA.

Close all but the current project

- From the main menu, select **File | Close Other Projects**.

This action closes all opened projects except the current one.

Indexing

To force reindex a project, use Invalidate caches.

Indexing in IntelliJ IDEA is responsible for the core features of the IDE: code completion, inspections, finding usages, navigation, syntax highlighting, and refactorings.

It starts when you open your project, switch between branches, after you load or unload plugins, and after large external file updates. For example, this can happen if multiple files in your project are created or generated after you build your project.



Indexing examines the code of your project to create a virtual map of classes, methods, objects, and other code elements that make up your application. This is necessary to provide the coding assistance functionality, search, and navigation instantaneously. After indexing, the IDE is aware of your code. That is why, actions like finding usages or smart completion are performed immediately.

While indexing is in progress, the above-mentioned coding assistance features are unavailable or partially available. Nevertheless, you can still work with the IDE: you can type code, work with VCS features, configure settings, and perform other code unrelated actions.

The amount of time required for indexing vary depending on your project: the more complex your project is, the more files it comprises, the more time it takes to index it. You can decrease the indexing time by excluding files and folders and by unloading modules.

Note that if indexing is already in progress, you cannot speed it up. Wait for the process to finish and then you can temporarily simplify your project. The next time, indexing will finish sooner.

There are several ways of making indexing faster:

- Use shared indexes
- Exclude files and folders
- Unload modules

Exclude files and folders

Marking dynamically generated files as excluded can speed up the indexing and overall IDE performance. For example, it's recommended that you exclude compilation output folders. Excluded files remain a part of a project, but are ignored by code completion, navigation, indexing, and inspections.

To exclude a file, right-click it in the **Project** tool window and select **Mark as Plain Text**. Plain text files are marked with the  icon.

To exclude a folder, right-click it in the **Project** tool window and select **Mark Directory as | Excluded**. Excluded folders are marked with the  icon.

You can also Exclude files and folders by name patterns.

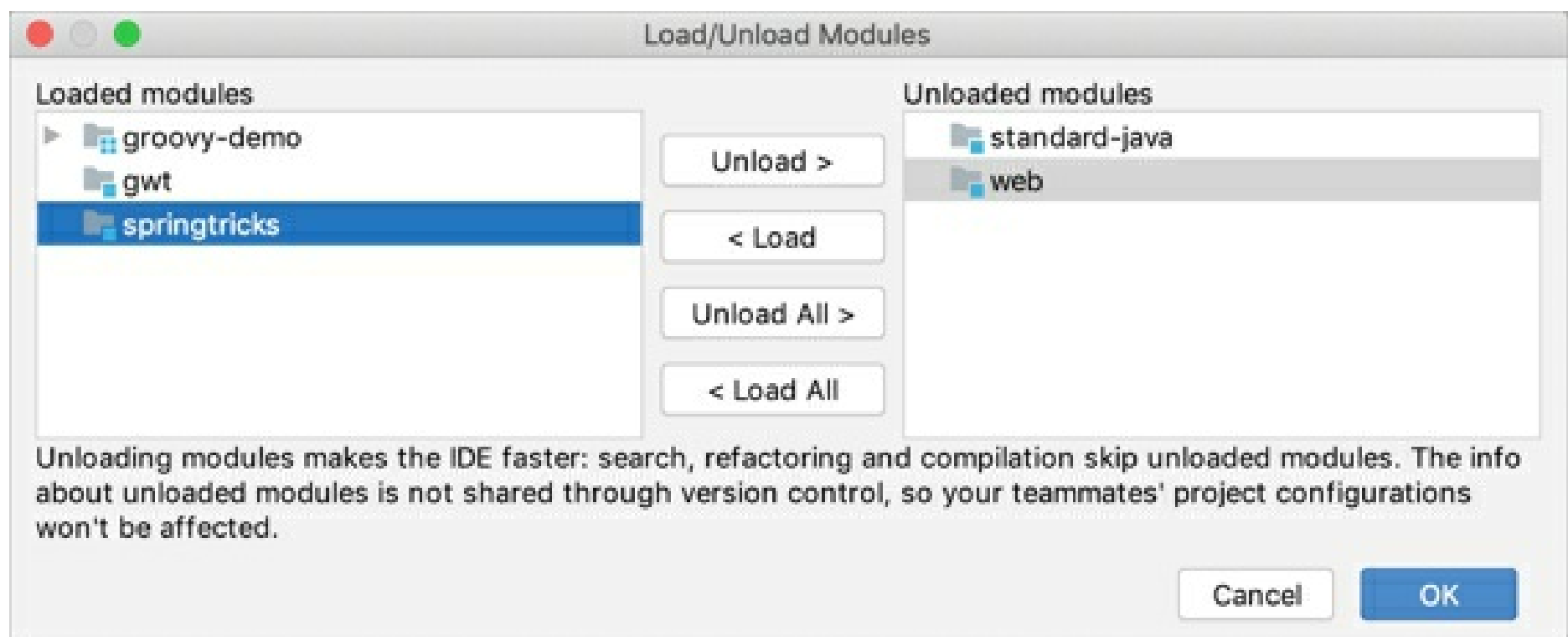
Marking folders as excluded doesn't affect deployment. For information on how to exclude files from deployment, refer to Exclude files and folders from uploading and downloading.

Unload modules

If indexing takes a significant amount of time, then it's likely that your project has more than two modules. Normally, you don't need all of them at same time.

If this is the case, you can temporarily set aside (unload) the modules that you don't need right now. The IDE ignores the unloaded modules when you search through or refactor your code, compile, or index your project.

To unload a module, right-click it in the **Project** tool window and select **Load/Unload Modules**.



Shared indexes

One of the possible ways of reducing the indexing time is by using shared indexes. Unlike the regular indexes that are built locally, shared indexes are generated once and are later reused on another computer whenever they are needed.

For more information on indexing and other ways of reducing the indexing time, refer to [Indexing](#).

IntelliJ IDEA can connect to the dedicated resource to download shared indexes for your JDK and Maven libraries and build shared indexes for your project's code. Whenever IntelliJ IDEA needs to reindex your application, it will use the available shared indexes and will build local indexes for the rest of the project. Normally, this is faster than building local indexes for the entire application from scratch.

Make sure the plugin is installed

To be able to use project shared indexes, the Project Shared Indexes bundled plugin must be enabled in the settings:

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Plugins**.
2. Switch to the **Installed** tab, type `Project Shared Indexes`, and make sure that the checkbox next to it is selected.

Otherwise, select the checkbox to enable the plugin.

3. Apply the changes and close the dialog. Restart the IDE if prompted.

When you launch a project, IntelliJ IDEA processes local and shared indexes together at the same time. This might increase CPU usage on your computer. If you want to avoid this, enable the **Wait for shared indexes** option in **Settings/ Preferences | Tools | Shared Indexes**.

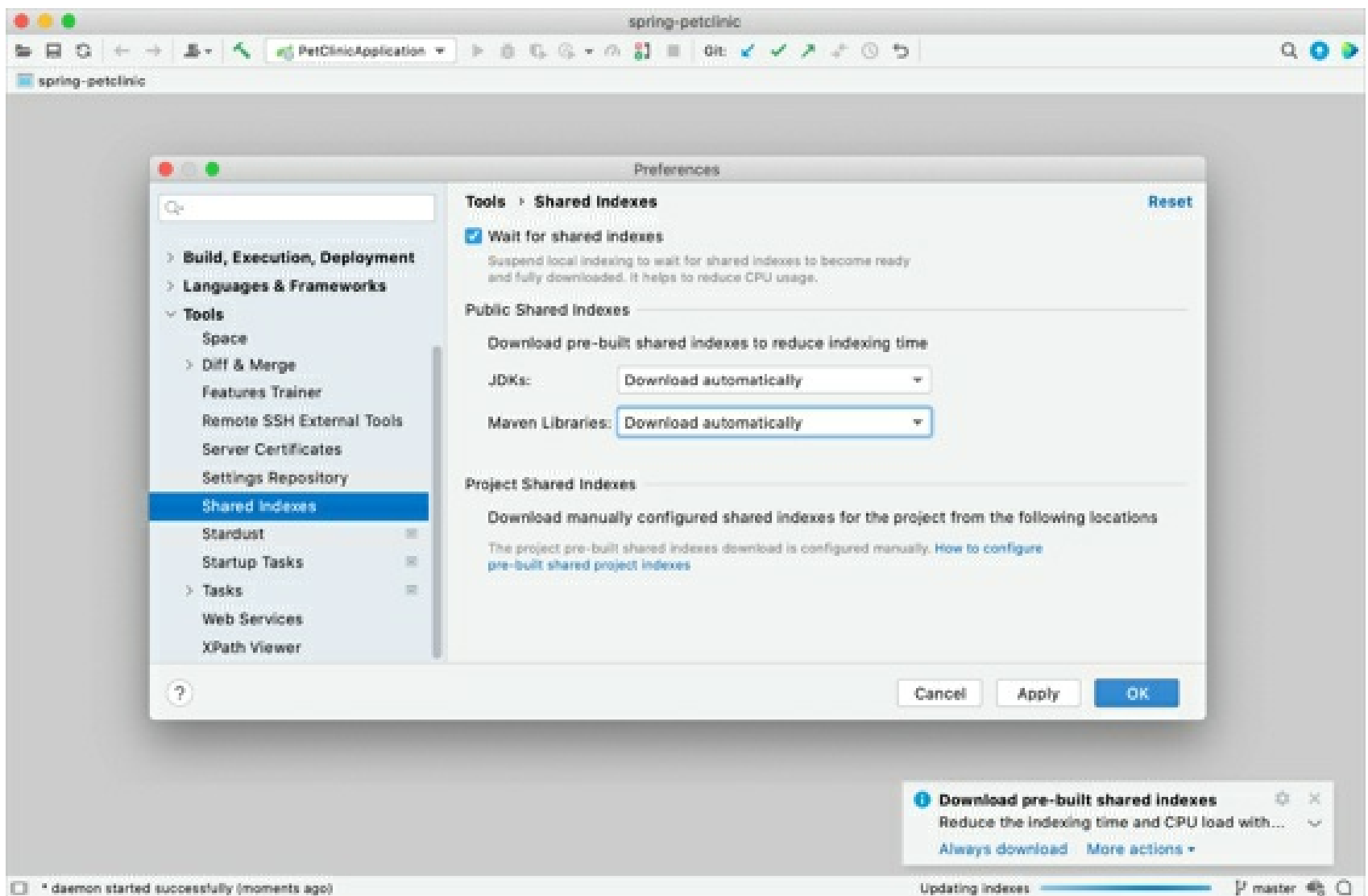
Shared indexes for JDKs and Maven libraries

Indexes for JDKs and Maven libraries are built by JetBrains and stored on the dedicated CDN resource. When you open a project, IntelliJ IDEA shows a notification prompting you to enable the automatic download.

If you miss the notification, you can configure these options in the settings.

1. In the **Settings/ Preferences** dialog, select **Tools | Shared Indexes**.
2. Select the **Download automatically** option for JDKs and Maven libraries to allow IntelliJ IDEA to download the indexes silently whenever they are needed.

Alternatively, select **Ask before download** if you prefer to confirm every download manually.



JDK indexes will be downloaded to **index/shared_indexes** in the IDE system directory. After that, IntelliJ IDEA will be using the suitable indexes whenever they are needed.

Project shared indexes

To be able to use project shared indexes, the Project Shared Indexes bundled plugin must be enabled in **Settings/ Preferences | Plugins**.

Project shared indexes are built for all project sources, libraries, and SDKs. You can generate them on one computer and then apply on another computer. This is the main advantage of shared indexes over ordinary indexes.

Using shared indexes is reasonable for large projects, where indexing might take a lot of time, creating inconveniences for the teams involved. For smaller projects, we recommend other ways of reducing the indexing time.

Before you begin

- To ensure index compatibility, use the same IDE version on the source and the target computer.
- You can have different operating systems on the source and the target computer.

However, in previous IntelliJ IDEA versions, project shared indexes were OS-specific. Refer to the documentation that corresponds to your IDE version by using the version switcher in the top-left corner of this page.

Share project indexes from the command line

In order to share project indexes, install a command line tool and prepare the IDE. By doing so, you're making sure that no caches or other internal files interfere with the information that you are going to export from your project.

When the indexes and related metadata are exported, upload them to your file storage. Other members of your team can then configure their IDEs to automatically connect to that storage and download the indexes whenever they are needed.

Prepare a file storage

Sharing indexes from the command line requires that you have a file storage that will keep the exported project indexes.

- Make sure that you have a file storage ready, and all members of your team have access to

this storage.

Install **cdn-layout-tool**

To be able to export and share project indexes, install **cdn-layout-tool** – a JetBrains tool for working with project shared indexes:

1. Download the latest version of the tool from the Space repository.
2. Once the **.zip** file is downloaded, extract its contents to a meaningful location.

Among the **.zip** file's content, there's a **bin** directory. There, you will find an OS-specific startup script that you will need later for generating metadata.

cdn-layout-tool and all its dependencies are published as Maven artifacts as well.

Prepare the IDE for sharing project indexes

1. Create the following directories or remove the contents from the existing ones:
 - **<temp>/ide-system**
 - **<temp>/ide-config**
 - **<temp>/ide-log**

Note that in different operating systems, the **temp** directory has a different location.

2. Copy the **<IDE home>/bin/idea.properties** file to the **<temp>** directory. For more information on **idea.properties**, refer to Common properties.
3. Rename **idea.properties** to **ide.properties** in **<temp>**.
4. Change the following properties in the **<temp>/ide.properties** file:
 - **idea.system.path=<temp>/ide-system**
 - **idea.config.path=<temp>/ide-config**
 - **idea.log.path=<temp>/ide-log**

If you don't have these properties in the file, add them at the end.

5. Set a temporary environment variable to specify the custom IDE properties file. To do so, run the following command:

Windows

Linux/macOS

```
set IDEA_PROPERTIES=<temp>\ide.properties
```

Copied!

Export project indexes

1. Make sure that the **<temp>/generate-output** directory is empty. If there's no such directory, do not create it as this will be done automatically during the export.
2. Run the IDE with the following command line to export the project indexes to **<temp>/generate-output**.

```
<IDE command-line launcher> dump-shared-index project --output=<temp>/generate-output --tmp=<temp>/temp --project-dir=<path> --project-id=<project-name> --commit=<Git HEAD>
```

Copied!

Windows

macOS

Linux

You can find the executable script for running IntelliJ IDEA in the installation directory under **bin**. To use this executable script as the command-line launcher, add it to your system **PATH** as described in Command-line interface.

Syntax

idea.bat dump-shared-index project --output=<temp>/generate-output --tmp=<temp>/temp --project-dir=<path> --project-id=<project-name> --commit=<Git HEAD>
Copied!

Options

--output=<temp>/generate-output	Output location for the generated shared indexes in the <temp> folder.
--tmp=<temp>/temp	The temp directory used for the internal needs of the IDE.
--project-dir=<path>	The path to the project for which you want to export indexes.
--project-id=<id>	The name of the project.
--commit=<vcs revision>	The VCS revision identifier (for example, Git commit hash) representing the state of the project for which the shared index is being generated. This is needed to lay out the shared indexes of several VCS revisions in the file storage. This identifier is optional. If it is not specified, it will be set to -.
--compression=<compression type>	Compress shared indexes. Possible options: xz, gzip, and plain.

The export might take some time. When the process is finished, the following files are placed to **<temp>/generate-output**:

- **shared-index-project-<name>--<hash>.ijx.xz**
- **shared-index-project-<name>--<hash>.metadata.json**
- **shared-index-project-<name>--<hash>.sha256**

Copy the generated files to a new folder

1. Create a new directory **<temp>/indexes** and download there all the contents of your remote file storage.

You can create the new directory inside the directory that has **<temp>/generate-output** subdirectory. This can be any directory, for example **~/indexes/project/<project name>/<VCS hash>/share**.

2. Copy the generated files (**.ijx.xz**, **.metadata.json**, **.sha256**) from **<temp>/generate-output** to a subdirectory of **<temp>/indexes** the created subdirectory (in our case, **share**).

This can be any folder, for example **<temp>/indexes/project/<project name>/<VCS hash>/<indexes>**.

Generate metadata

- Generate auxiliary files used by the IDE to look up for and download shared indexes. To do so, run **cdn-layout-tool** with the following command line.

<cdn-layout-tool> --indexes-dir=<dir> --url=<file storage root URL>
Copied!

Options

<code><cdn-layout-tool></code>	The path to cdn-layout-tool startup script located in bin in the tool directory.
<code>--indexes-dir=<dir></code>	The directory containing all the files available in the file storage, as well as the newly generated shared indexes residing in a suitable location beneath this directory. The tool will generate the missing files and replace existing ones if necessary. After the tool has finished running, all the contents of this directory must be uploaded to the file storage. This will set the new state of the storage.
<code>--url=<file storage root URL></code>	The base URL of the file storage root directory. cdn-layout-tool will substitute the download URL in all the corresponding .json index lists.

Example

```
/Users/User/Desktop/cdn-layout-tool-0.7.52/bin/cdn-layout-tool --indexes-dir=/var/folders/dr/xxx_x00x0x0_x0xx000xx000000xx/T/indexes --url=https://my-file-storage.com/intellij-indexes
```

Copied!

Upload the index files to your file storage

- Upload all the files from **<temp>/indexes** to the file storage as is.

If there are already uploaded index files in the storage, update them.

Download indexes from the file storage

Once the project indexes are uploaded to the file storage, they can be downloaded and applied on another computer.

1. In the project directory, create a new file **intellij.yaml** and add the following code:
2. `sharedIndex:`
3. `project:`

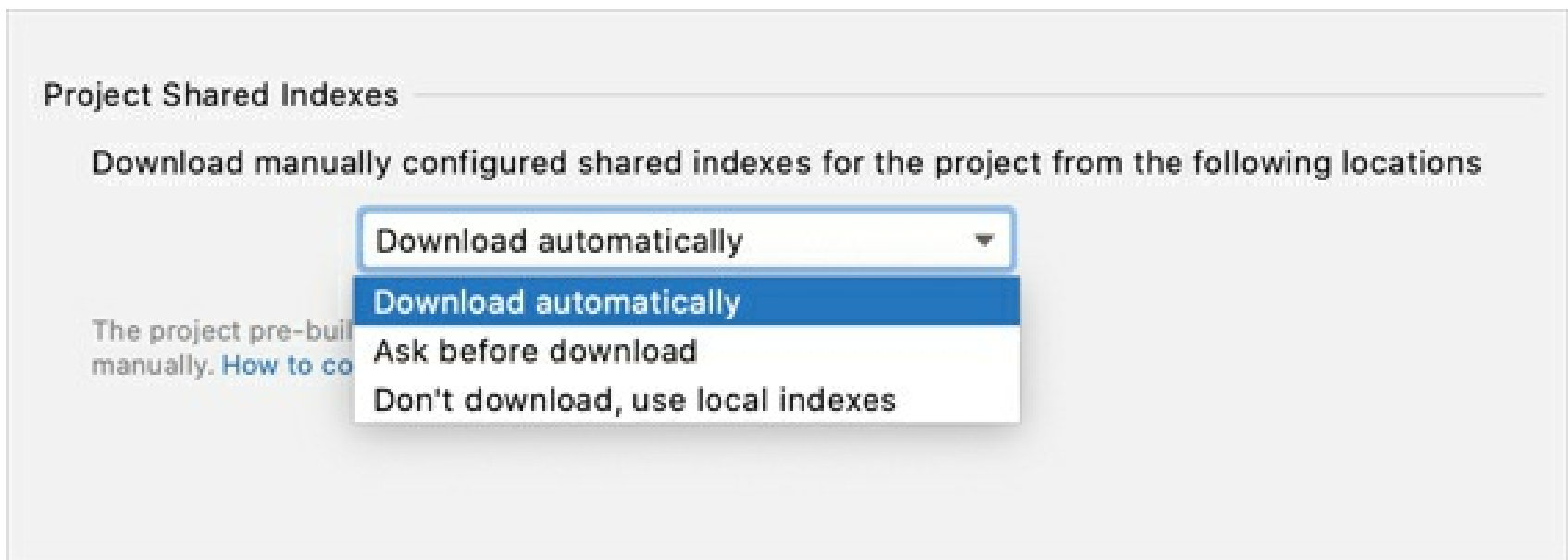
```
- url: <feed URL>
```

Copied!

Replace **<feed URL>** with the URL of your file storage in the following format: **<file storage root URL>/project/<project-id>**.

4. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Tools | Shared Indexes**.
5. In the **Project Shared Indexes** area, select the way you want to download shared indexes from the storage.

Select **Download automatically** to allow IntelliJ IDEA to download the JDK indexes silently whenever they are needed or select **Ask before download** if you prefer to confirm every download manually.



6. Apply the changes and close the dialog.

Project indexes will be downloaded to **index/shared_indexes** in the IDE system directory.

Place the **<project home>/intellij.yaml** file under VCS so that other developers in your team can get access to the shared project indexes.

Remove the downloaded shared indexes

If you don't need the project indexes anymore (for example, if the project is no longer in development), you can remove them all together. In this case, the IDE will remove the regular indexes and all the downloaded shared indexes for the project and the JDK.

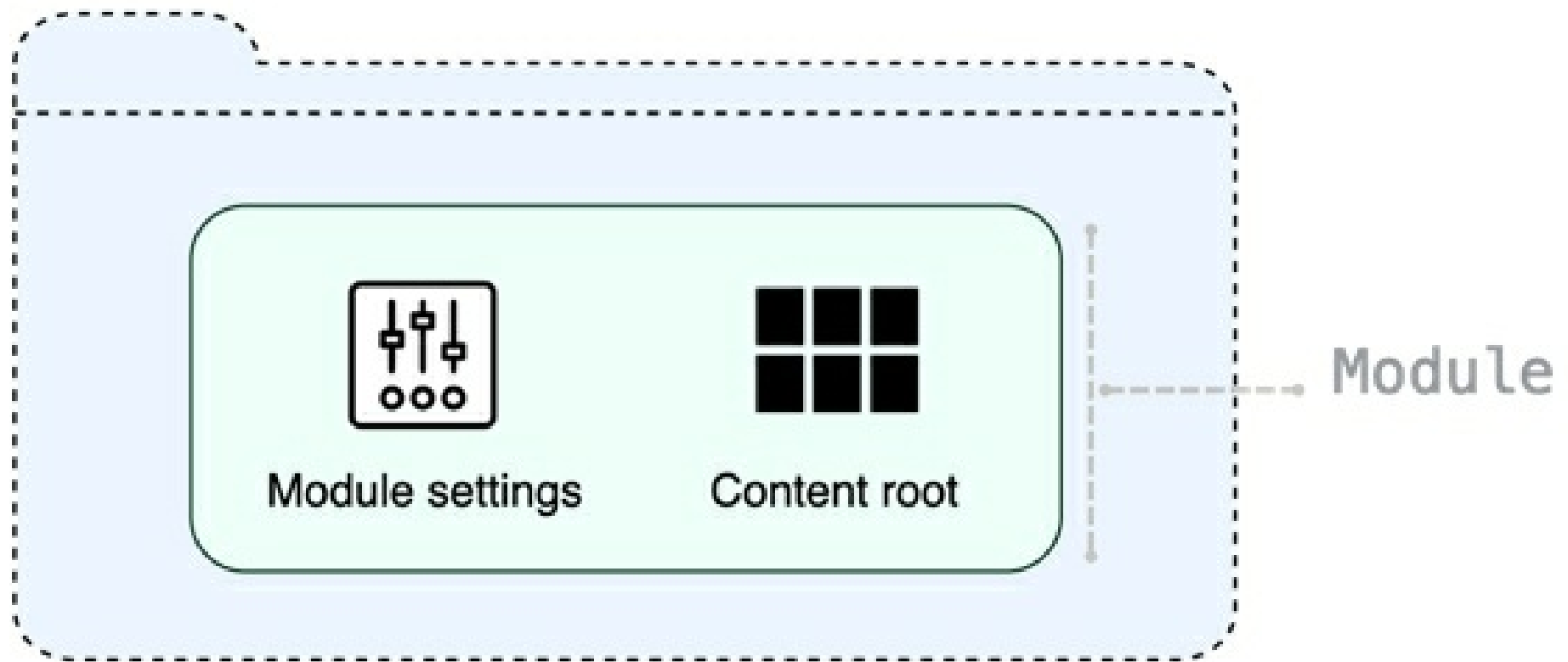
1. From the main menu, select **File | Invalidate Caches**.
2. In the dialog that opens, select **Clear downloaded shared indexes** and click **Invalidate and Restart**.

Modules

In IntelliJ IDEA, a module is an essential part of any project – it's created automatically together with a project. Projects can contain multiple modules – you can add new modules, group them, and unload the modules you don't need at the moment.

Generally, modules consist of one or several content roots and a module file, however, modules can exist without content roots. A content root is a folder where you store your code. Usually, it contains subfolders for source code, unit tests, resource files, and so on. A module file (the **.iml** file) is used for keeping module configuration.

Modules allow you to combine several technologies and frameworks in one application. In IntelliJ IDEA, you can create several modules for a project and each of them can be responsible for its own framework. For more information, refer to [Add frameworks \(facets\)](#).



For more information on how modules are used in projects, refer to [Configure projects](#).

IntelliJ IDEA modules vs Java modules

In version 9, Java introduced the Java Platform Module System. IntelliJ IDEA had already had a concept of modules: every IntelliJ IDEA module built its own classpath. With the introduction of the new Java platform module system, there appeared two systems of modularity: the IntelliJ IDEA modules, and the new Java 9 modules that are configured using **module-info.java**. This documentation section describes IntelliJ IDEA modules.

For more information on Java 9 support in IntelliJ IDEA refer to the [Support for Java 9 Modules in IntelliJ IDEA 2017.1](#) and [Java 9 and IntelliJ IDEA](#) blog posts.

Projects with multiple modules

IntelliJ IDEA allows you to have many modules in one project, and they shouldn't be just Java. You can have one module for a Java application and another module for a Ruby on Rails application or for any other supported technology.

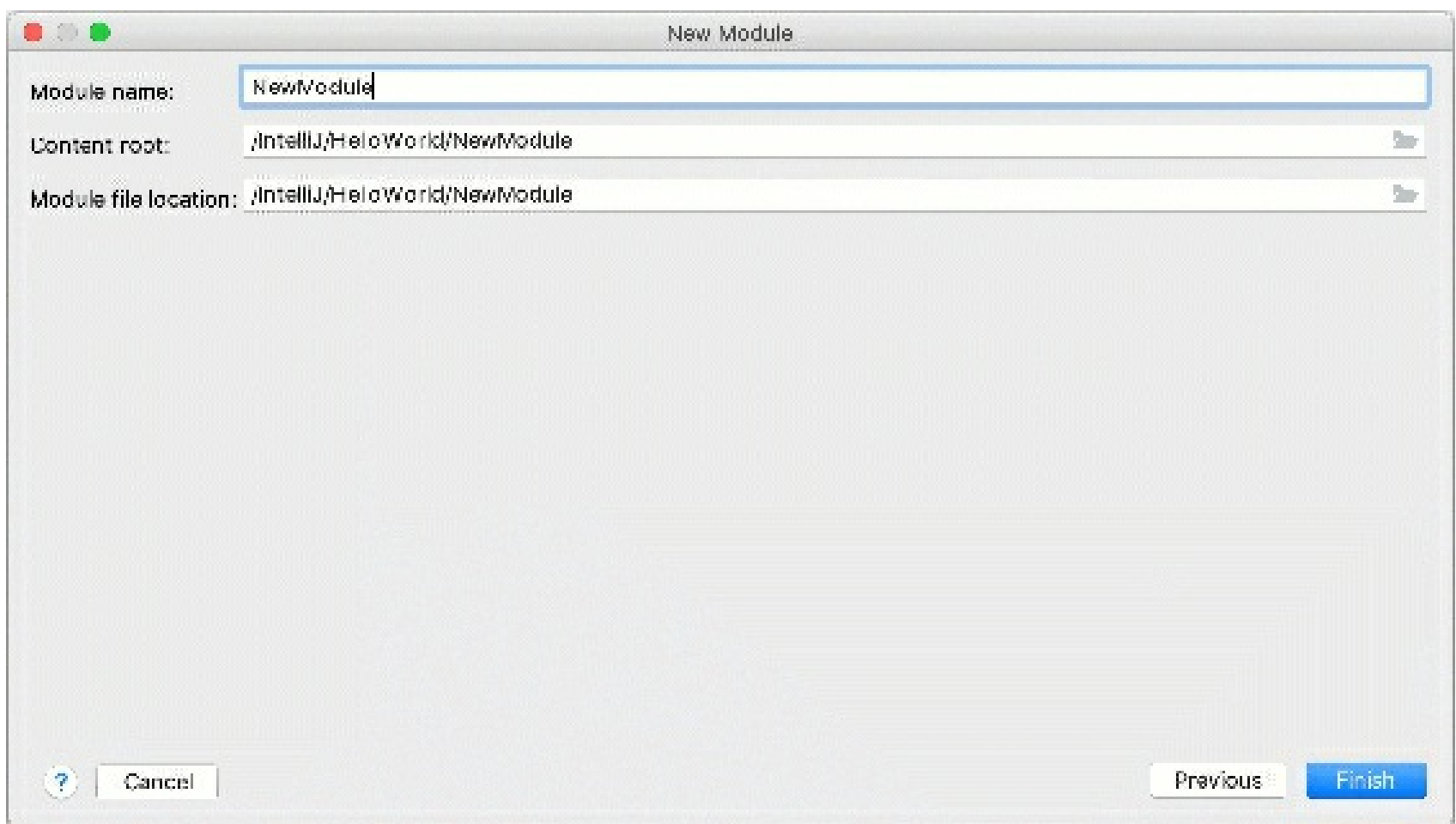
An application that consists of a client side and a server side is a good example a two-module project.

Add a new module to your project

1. Right-click the top-level directory in the **Project** tool window and select **New | Module**. The **New Module** wizard opens.
2. From the list on the left, select a module type.
3. In the right-hand part of the dialog, select an SDK that you want to use from the **Module SDK** list. You can use the project SDK or specify a new one.
4. In the **Additional Libraries and Frameworks** section, select additional assets that you want to use in this module.
5. On the next step, name the module and specify the location of the content root and

the **.iml** file. You can place them within or outside of the project.

6. Click **Finish**.



Gif

Import an existing module

You can import a module to your project by adding the **.iml** file from another project:

1. From the main menu, select **File | New | Module from Existing Sources**.
2. In the dialog that opens, specify the path the **.iml** file of the module that you want to import, and click **Open**.

By doing so, you are attaching another module to the project without physically moving any files. If you don't need the modules to be located in one folder, the module import is finished, and you can start working with the project normally.

If you want the modules in the same folder, in the **Project** tool window, drag the imported module to the top-level directory. In this case, the contents of the imported module will be physically transferred to your project's folder.

For information on how to attach a Maven or Gradle project to your current project, refer to [Link a Gradle project to an IntelliJ IDEA project](#) and [Link and unlink a Maven project](#).

Import a module from existing sources

Group modules

In IntelliJ IDEA, you can logically group modules. If you have a large project with multiple modules, grouping will make it easier to navigate through your project. Module groups can be nested: a group can contain other subgroups.

Create a new module group (deprecated)

In earlier versions (2017.2 and earlier), IntelliJ IDEA used explicit groups for joining modules together. If you've configured manual module groups, you will be able to continue working with them in later versions of the IDE. Alternatively, you can convert module groups and use qualified names instead.

1. In the **Project** tool window (**Alt+1**), select the modules that you want to group.

You can also do so on the **Modules** page of the **Project Structure** dialog (**Ctrl+Alt+Shift+S**).

2. From the context menu, select **Move Module to Group | New Top Level Group**.
3. Name the new group and click **OK**.

The new group is now created and is marked with the  icon.

Select **Outside Any Group** to exclude the selected module from the group, **To This Group** to add the module to the group, or **To New Subgroup** to create a new group in another group.

Convert module groups to qualified names (deprecated)

1. From the main menu, select **File | Convert Module Groups to Qualified Names**.
2. In the next dialog, review the new module names and adjust them if necessary.
3. Apply the changes and close the dialog.

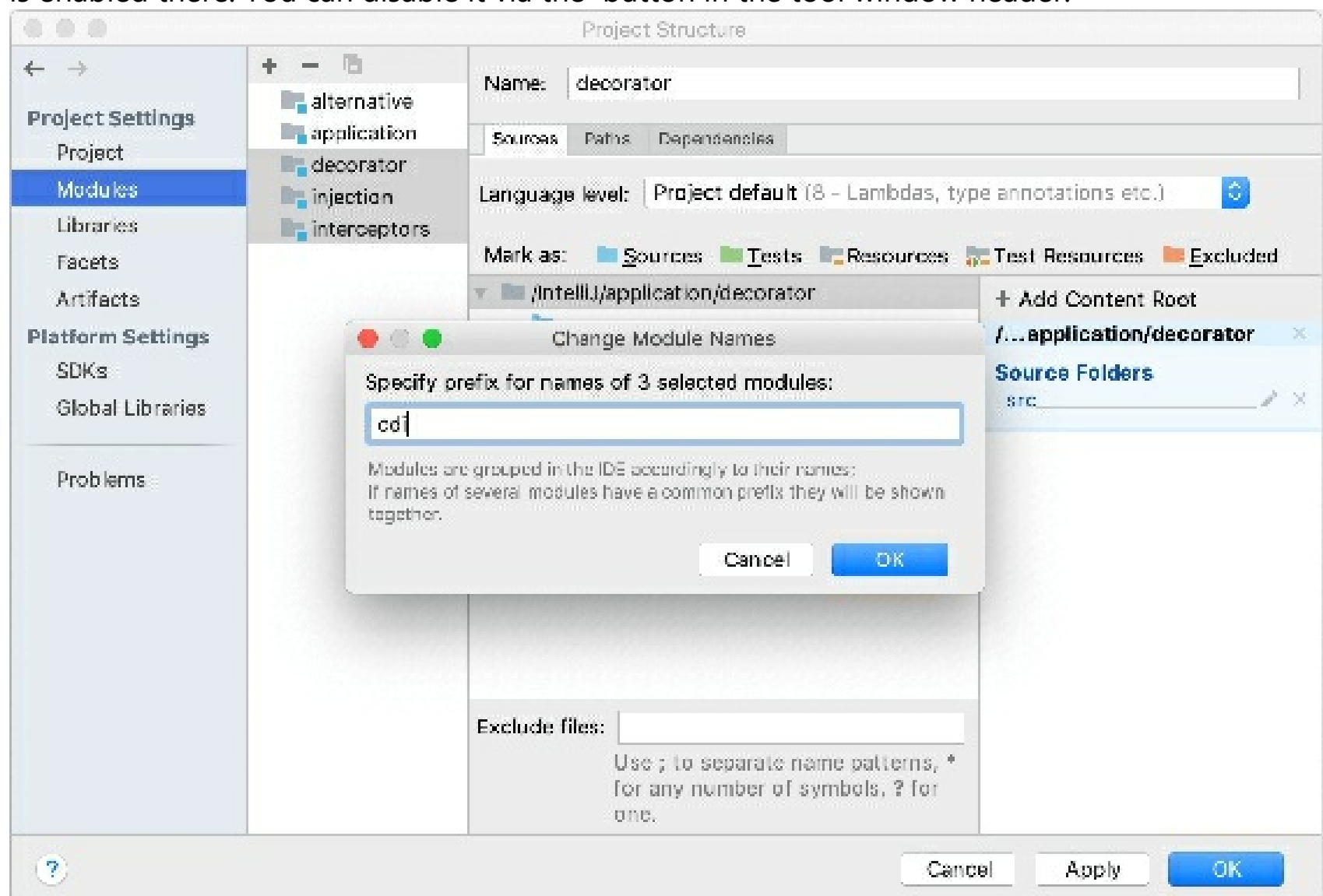
Group modules by fully qualified names

IntelliJ IDEA 2017.3 and later uses fully qualified names to group modules. For example, if you want to group all CDI modules, add the `cdi` prefix to their names.

1. Open the **Project Structure** dialog `Ctrl+Alt+Shift+S` and click **Modules**.
2. Select the modules you want to group, open the context menu, and click **Change Module Names**.
3. Specify a prefix and apply the changes.

To view all modules on the same level in the **Project Structure** dialog, use the **Flatten Modules** context menu option.

Module groups won't be visible in the **Project** tool window (`Alt+1`) if the **Flatten Modules** option is enabled there. You can disable it via the  button in the tool window header.



Module structure settings

Module settings apply only to one module and are stored in the **.iml** file. A module can have an SDK and a language level that are different from those configured for a project, and their own libraries. They can also carry a specific technology or a framework.

Module SDK

An SDK is a collection of tools that you need to develop an application for a specific software framework. To develop Java-based applications, you need a JDK (Java Development Kit).

You can compile a module with an SDK that differs from the project SDK.

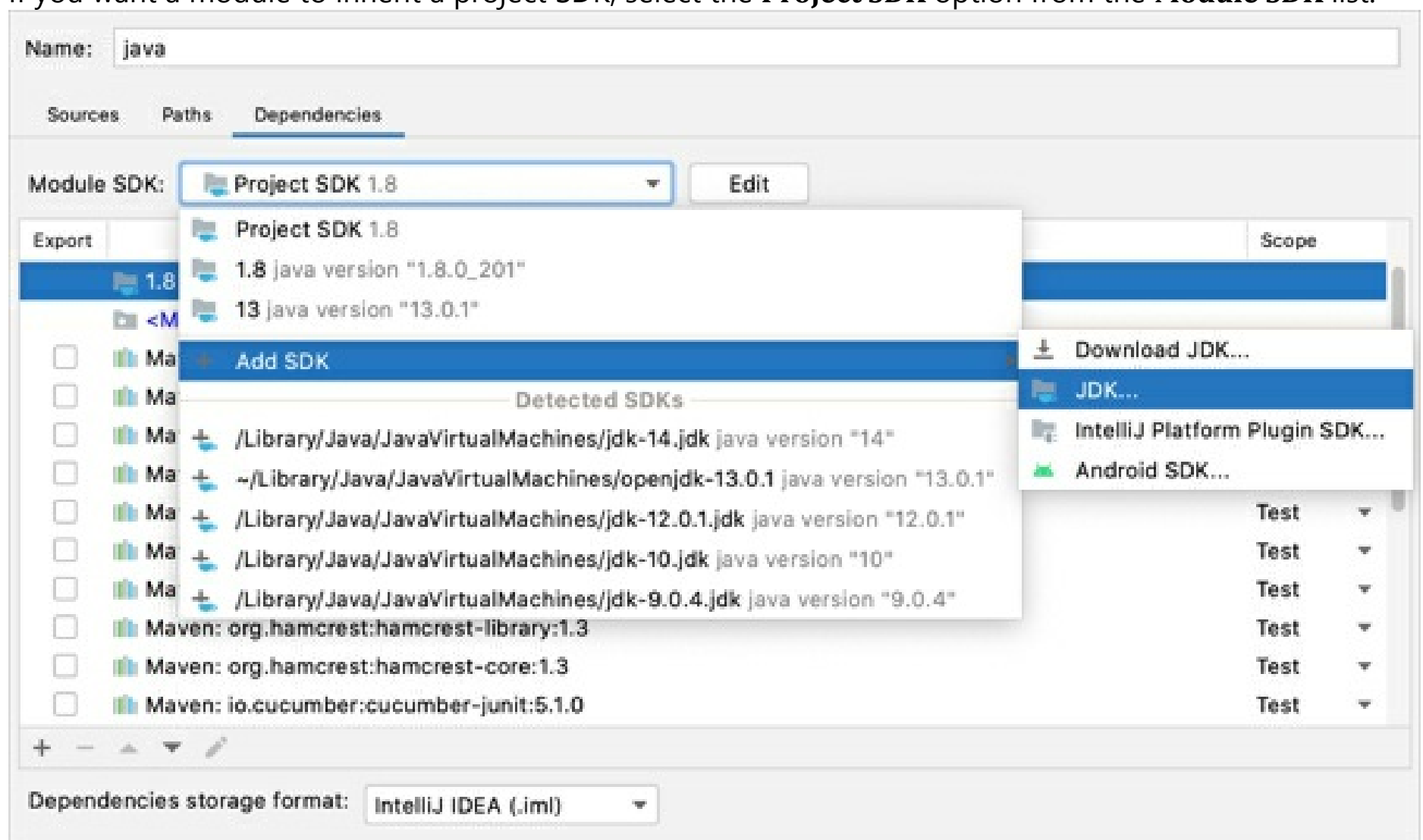
Set up a module SDK

1. From the main menu, select **File | Project Structure | Project Settings | Modules**.
2. Select the module for which you want to set an SDK and click **Dependencies**.
3. If the necessary SDK is already defined in IntelliJ IDEA, select it from the **Module SDK** list.

If the SDK is installed on your computer, but not defined in the IDE, select **Add SDK | 'SDK name'**, and specify the path to the SDK home directory.

Only for JDKs: If you don't have the necessary JDK on your computer, select **Add SDK | Download JDK**. In the next dialog, specify the JDK vendor, version, change the installation path if required, and click **Download**.

If you want a module to inherit a project SDK, select the **Project SDK** option from the **Module SDK** list.



How does IntelliJ IDEA know which JDK to use?

IntelliJ IDEA does the following to determine which JDK to use for compilation if you use different JDKs for modules in your project.

- It checks all JDKs that are used in the project: the JDKs that are defined on both the project and module levels.
- It calculates the latest of these JDKs. This is necessary to make sure all modules can be compiled.
- If the version of the latest JDK configured is lower than 1.6, IntelliJ IDEA will pick the JDK version that is used for running the IDE. This limitation is related to the fact that the

compiler API used by IntelliJ IDEA for building projects is supported starting from JDK 1.6.

- Although a specific version of the compiler will be used (in accordance with the selected JDK version), each separate module will be compiled using the javac's cross-compilation feature against the libraries of the JDK defined for this particular module in the project settings.

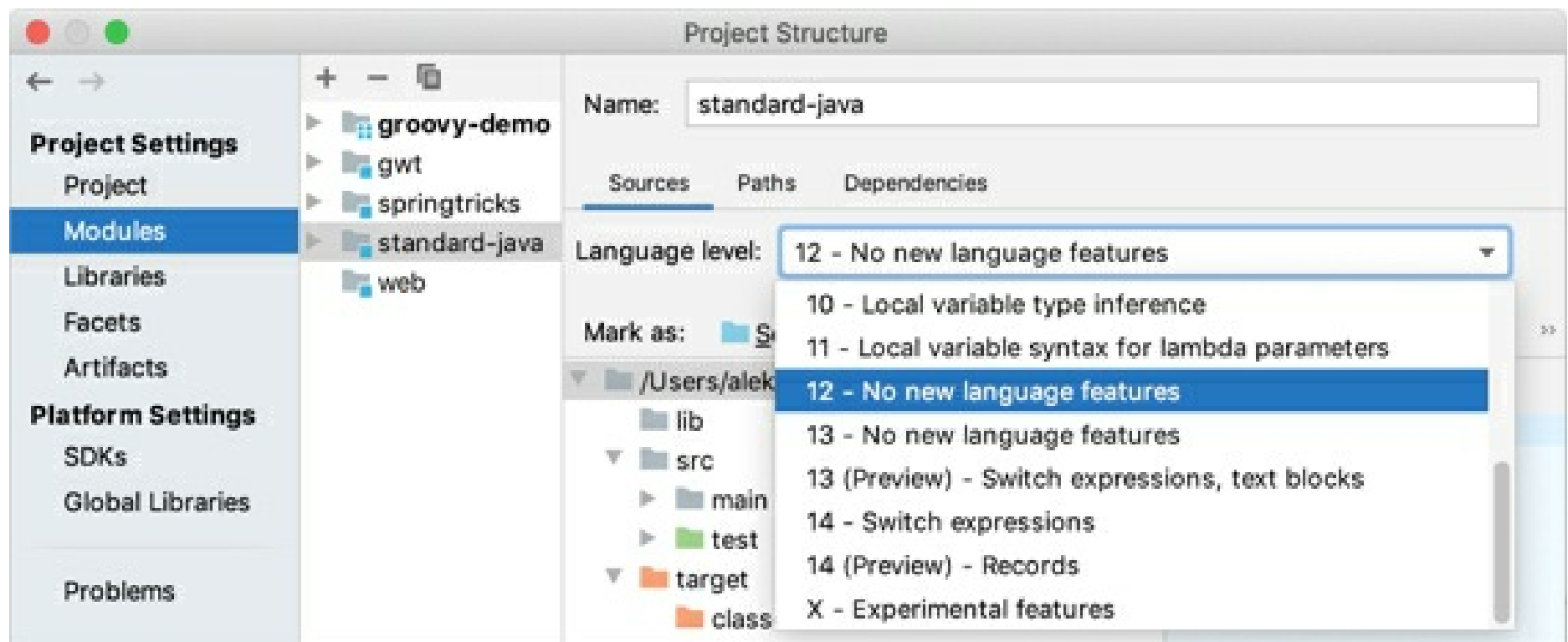
This protects you from a situation when a module is compiled against newer libraries than those for which dependencies are set.

Module language level

Language level defines coding assistance features that the editor provides. To configure a language level for a module:

Configure module language level

1. From the main menu, select **File | Project Structure** Ctrl+Alt+Shift+S.
2. Under **Project Settings**, select **Modules | Sources**.
3. From the **Language level** list, select the necessary option.
4. To use the project language level, select **Project default**.

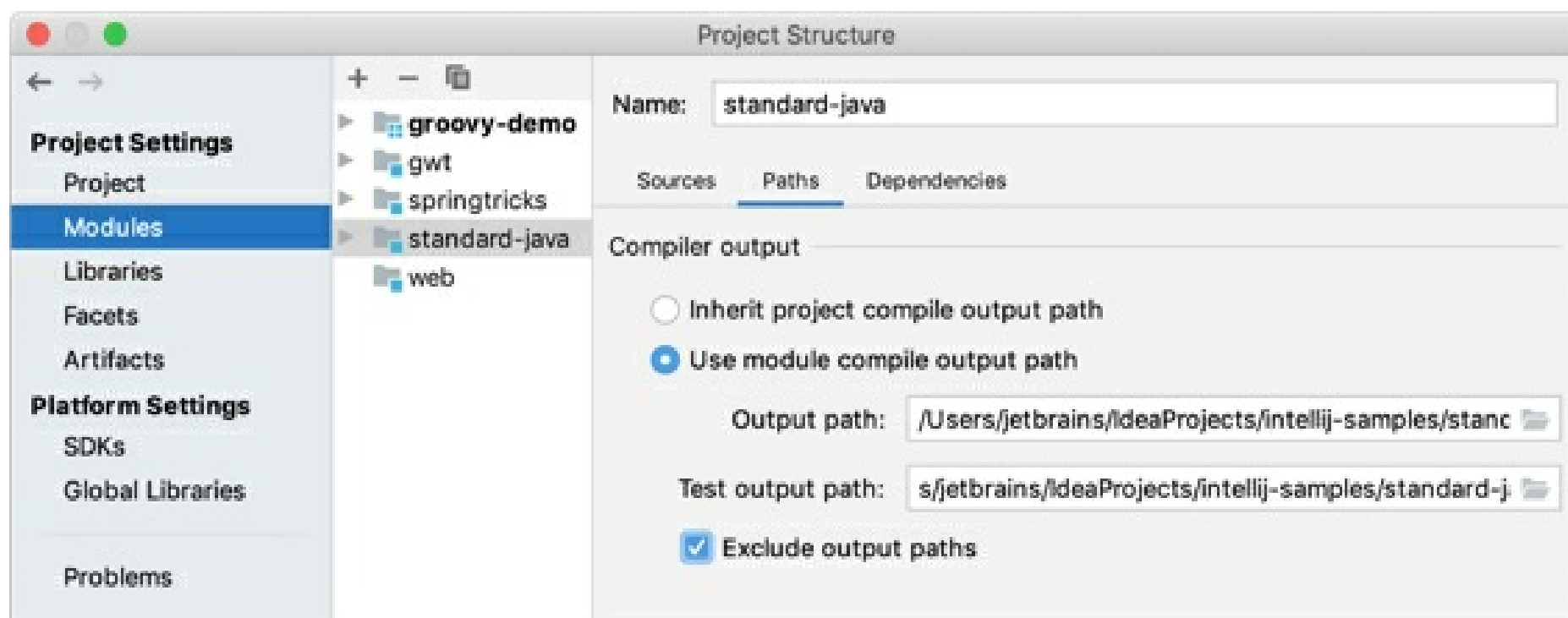


Module compiler output

Compiler output path is the path to the directory in which IntelliJ IDEA stores the compilation results. In this directory, the IDE creates two sub-directories: **output** for production code and **test output** for test sources.

Configure module compiler output

1. From the main menu, select **File | Project Structure** Ctrl+Alt+Shift+S.
2. Under **Project Settings**, select **Modules | Paths**.
3. Change the paths specified in the **Output path** and **Test output path** or select **Inherit project compile output path** to use the paths specified for the project.
4. Select the **Exclude output path** checkbox to exclude the output folders from code completion, navigation, and inspections. This helps increase overall IDE performance.



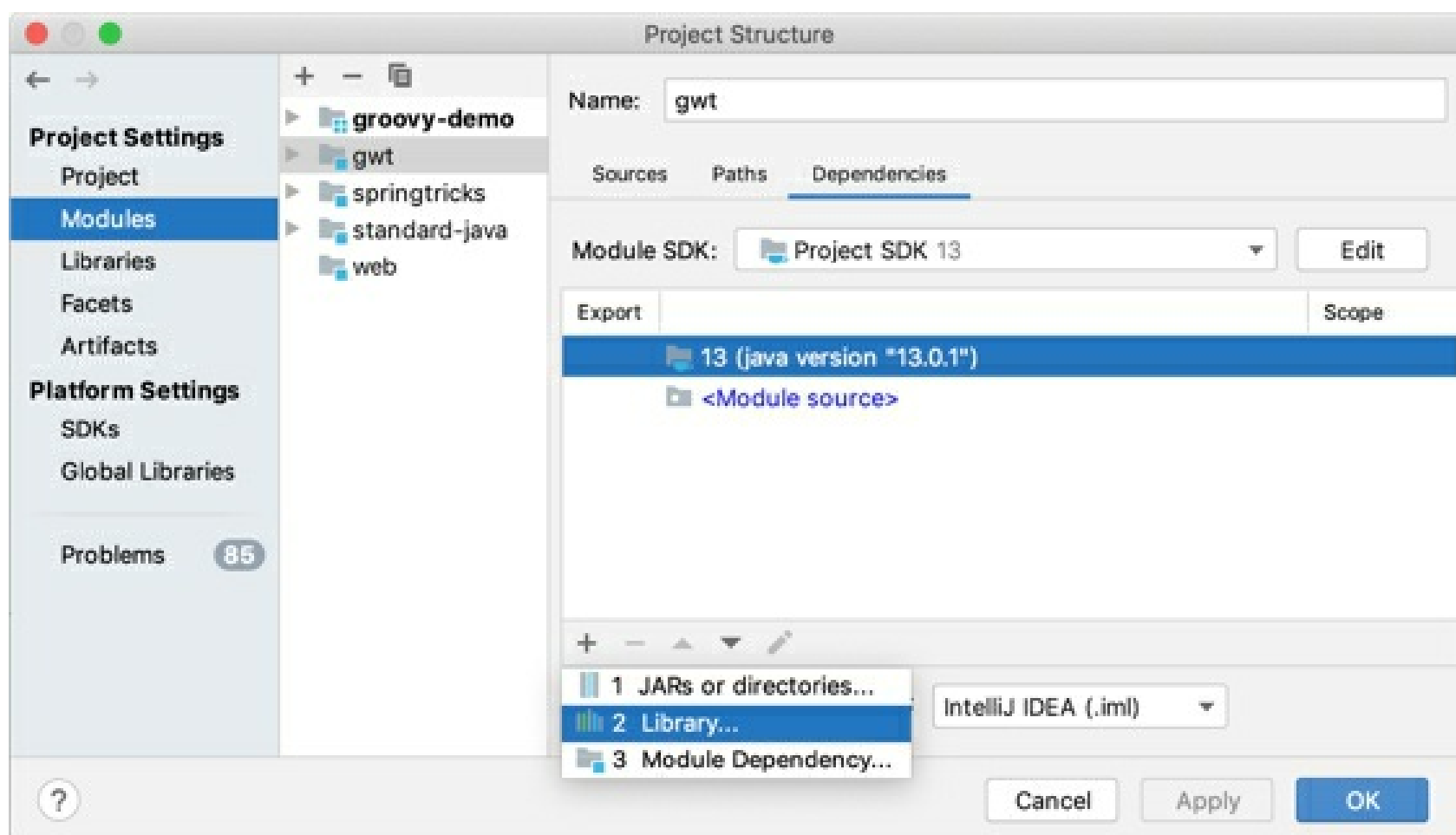
Module libraries

Libraries are a collection of compiled code that you can use when developing applications. You can add libraries on the module level. In this case, only one module can use the code from such libraries.

Add module-level libraries

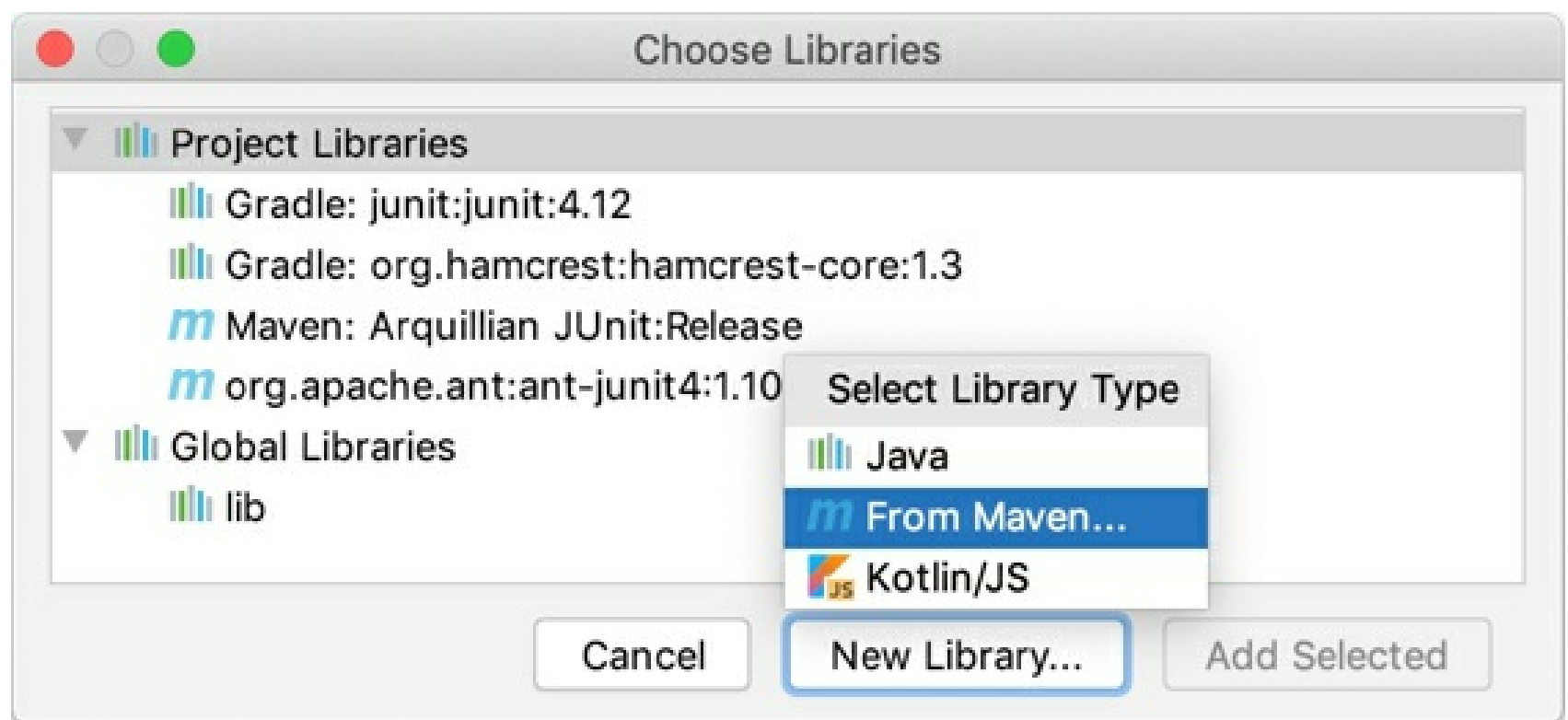
Global and project libraries are not available until you add them to module dependencies.

1. From the main menu, select **File | Project Structure | Project Settings | Modules**.
2. Select the module for which you want to add a library and click **Dependencies**.
3. Click the **+** button and select **Library**.



4. In the dialog that opens, select a project or a global library that you want to add to the module.

Alternatively, click **New Library** and select how do you want to add a new library: you can add a Java and Kotlin libraries from files on your computer, or download a library from Maven.



Content roots

Content in IntelliJ IDEA is a group of files that contain your source code, build scripts, unit tests, and documentation. These files are usually organized in a hierarchy. The top-level folder is called a *content root*.

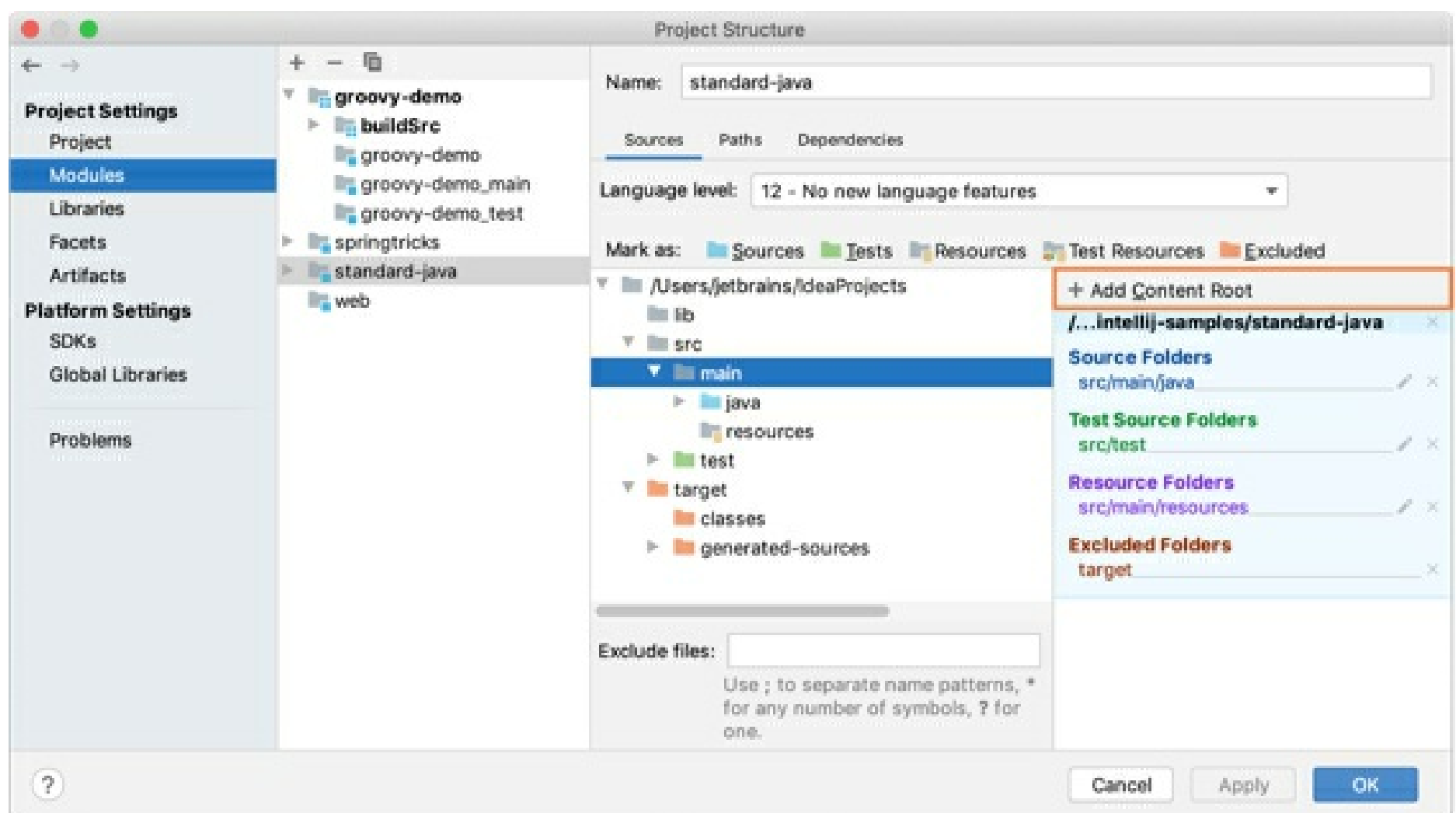
Modules normally have one content root. You can add more content roots. For example, this might be useful if pieces of your code are stored in different locations on your computer.

At the same time, modules can exist without content roots. In this case, you can use them as a collection of dependencies for other modules.

The content root directory in IntelliJ IDEA is marked with the  icon.

Add a new content root

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Project Settings | Modules**.
2. Select the necessary module and then open the **Sources** tab in the right-hand part of the dialog.
3. Click **Add Content Root** and specify the folder that you want to add as a new content root.



To remove a content root, click . IntelliJ IDEA marks the selected root as a regular folder; the folder itself and its contents won't be deleted.

Configure folder structure

Folder categories

Folders within a content root can be assigned to several categories.

- **Sources**

This folder contains production code that should be compiled.

IntelliJ IDEA compiles the code within the Sources folder. That is why, do not place configuration files (the **.idea** folder or its content and the **.iml** file) to this folder. For more information about different types of settings, refer to Project, module, and global settings.

- **Generated Sources**

The IDE considers that files in the Generated Sources folder are generated automatically rather than written manually, and can be regenerated.

- **Test Sources**

These folders keep code related to testing separately from production code. Compilation results for sources and test sources are normally placed into different folders.

- **Generated Test Sources**

The IDE considers that files in this folder are generated automatically rather than written manually, and can be regenerated.

- **Resources**

(Java only) Resource files used in your application: images, configuration XML and properties files, and so on. During the build process, resource files are copied to the output folder as is by default. You can Change the output path for resource files in your project. Similarly to sources, you can specify that your resources are generated. You can also specify which folder within the output folder your resources should be copied to.

- **Test Resources**

These folders are for resource files associated with your test sources.

- **Excluded**

Files in excluded folders are ignored by code completion, navigation and inspection. That is why, when you exclude a folder that you don't need at the moment, you can increase the IDE performance. Normally, compilation output folders are marked as excluded. Apart from excluding the entire folders, you can also exclude specific files. Marking folders as excluded doesn't affect deployment. For information on how to exclude files from deployment, refer to Exclude files and folders from uploading and downloading.

Configure folder categories

1. Right-click a folder in the Project tool window.
2. Select **Mark Directory as** from the context menu.
3. Select the necessary category.

This way, you can assign categories to sub-folders as well.

To restore the previous category of a folder, right-click this folder again, select **Mark Directory as**, and then select **Unmark as <folder category>**. For excluded folders, select **Cancel Exclusion**.

You can also configure folder categories in **Project Structure | Modules | Sources**.

Exclude files and folders

Exclude files

Java files and binaries cannot be excluded.

If you don't need specific files, but you don't want to completely remove them, you can temporarily exclude these files from the project. Excluded files are ignored by code completion, navigation, and inspections.

To exclude a file, you need to mark it as a plain text file. You can always return excluded files to their original state.

1. Right-click the necessary file in the **Project** tool window.
2. Select **Mark as Plain Text** from the menu.

Plain text files are marked with the  icon in the directory tree.

To revert the changes, right-click the file and select **Mark as <file type>** from the menu.

Exclude files and folders by name patterns

In some cases, excluding files or folders one by one is not convenient. For example, this may be inconvenient if your source code files and files that are generated automatically (by a compiler, for instance) are placed in the same directories, and you want to exclude the generated files only. In this case, you can configure one or several name patterns for a specific content root.

If a folder or a filename located inside the selected content root matches one of the patterns, it will be marked as excluded. Objects outside the selected content root won't be affected.

All files within excluded folders will be excluded as well.

1. From the main menu, select **File | Project Structure**, or press `Ctrl+Alt+Shift+S`.
2. Click **Modules** under the **Project Settings** section, and then select a module.

If there are several content roots in this module, select the one that you want to exclude files or folders from.

3. In the **Exclude files** field located at the bottom of the dialog, enter a pattern. For example, enter `*.aj` to exclude AspectJ files.

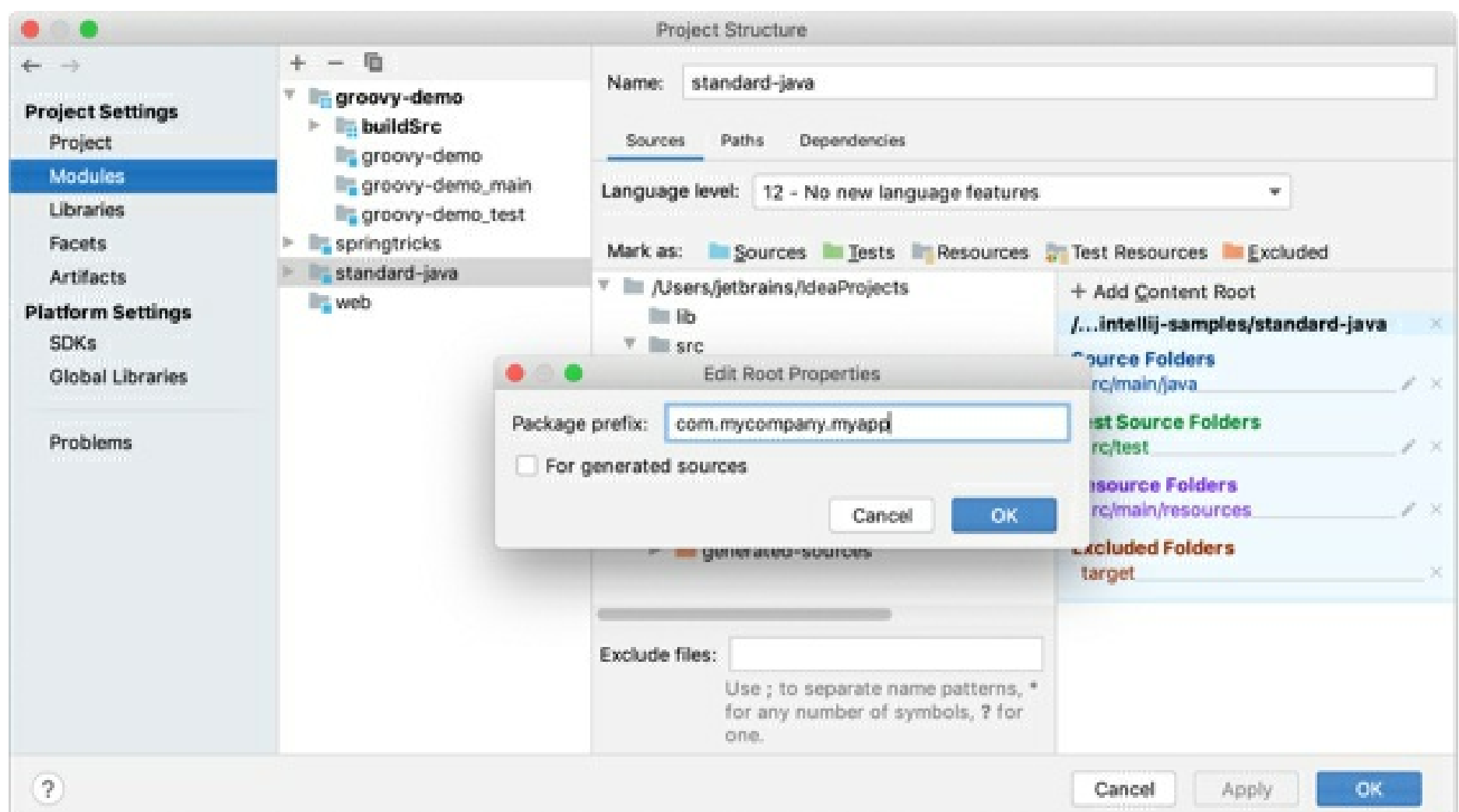
You can configure multiple patterns and separate them with the `;` (semicolon) symbol.

Marking folders as excluded doesn't affect deployment. For information on how to exclude files from deployment, refer to [Exclude files and folders from uploading and downloading](#).

Assign a package prefix to Java sources

In Java, you can assign a package prefix to a folder instead of configuring a folder structure manually. A package prefix can be assigned to source folders, generated source folders, test source folders and generated test source folders.

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules**.
2. Select the necessary module and open the **Sources** tab.
3. In the right-hand pane, click  next to **Source Folders** or **Test Source Folders**.
4. Specify the package prefix and click **OK**.

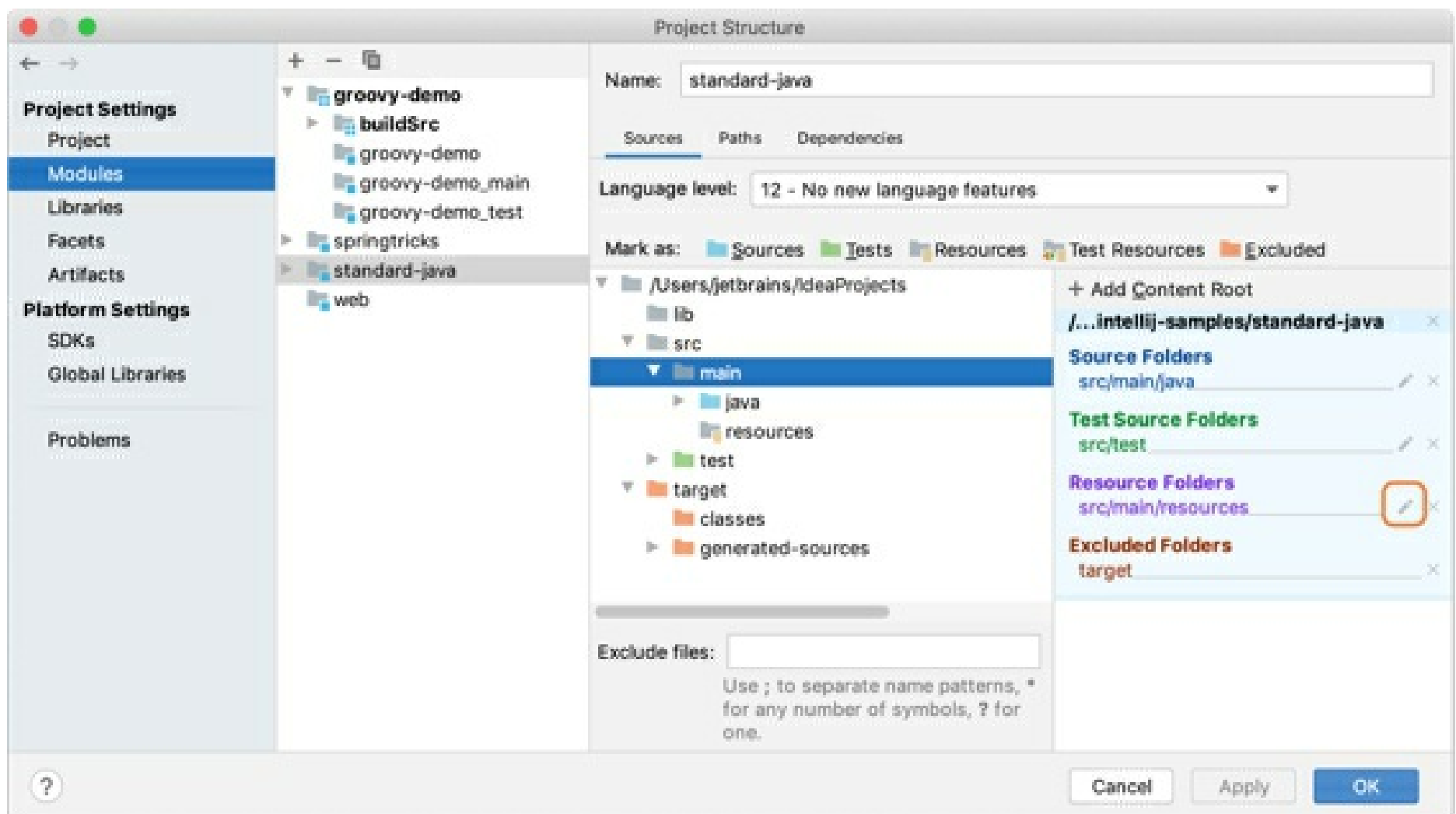


Change the output path for resources

When you're building a project, the resources are copied into the compilation output folder by default. You can specify a different directory within the output folder to place resources.

This information is valid for projects that are built with the native IntelliJ IDEA builder. If you're using a build tool, such as Maven or Gradle, make all changes using the build file.

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules**.
2. Select the necessary module and open the **Sources** tab.
3. In the right-hand pane, under **Resource Folders** or **Test Resource Folders**, click to the right of the necessary folder (folder path).
4. Specify the path relative to the output folder root, and click **OK**.

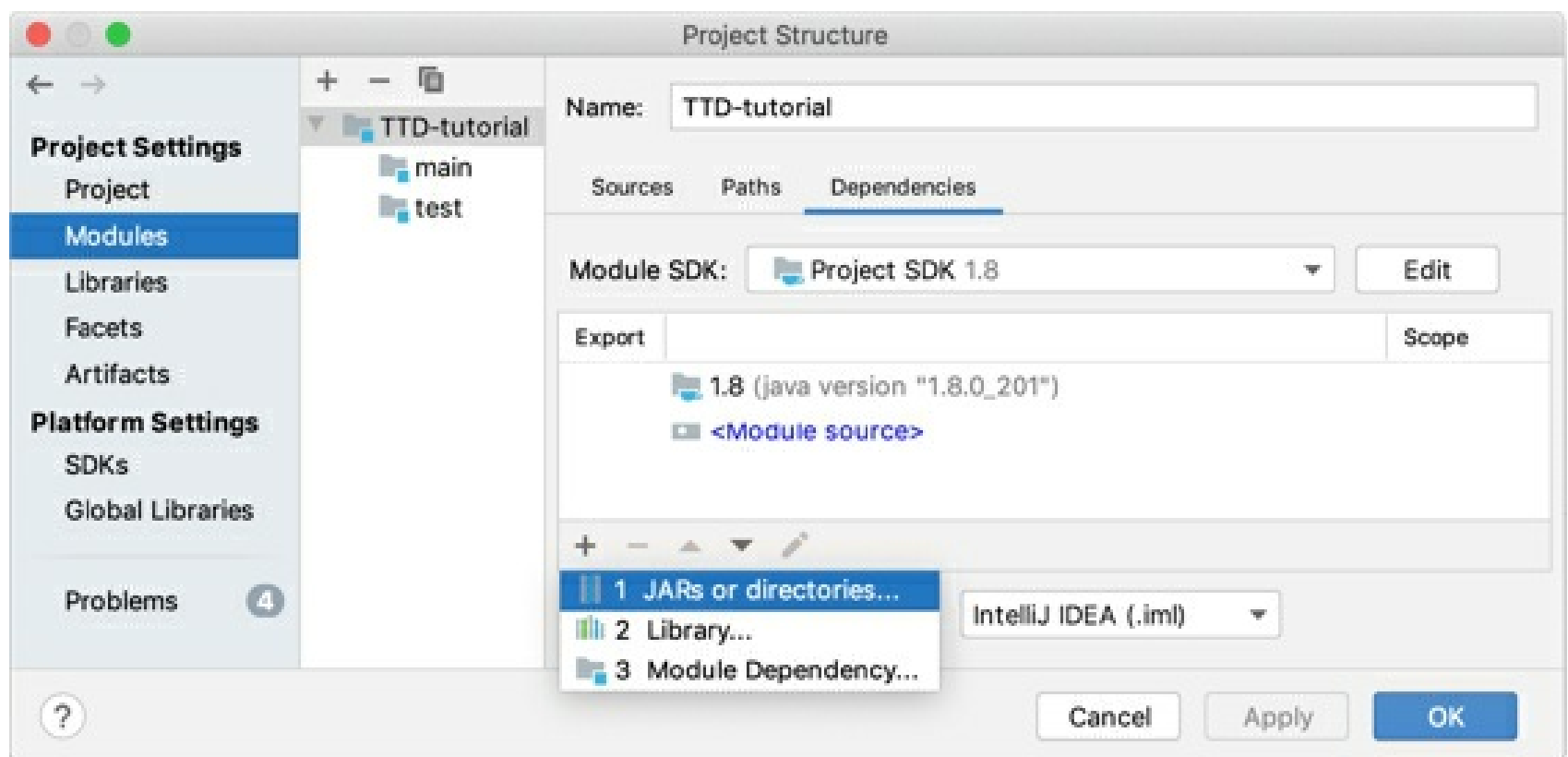


Module dependencies

Modules can depend on SDKs, JAR files (libraries) or other modules within a project. When you compile or run your code, the list of module dependencies is used to form the classpath for the compiler or the JVM.

Add a new dependency

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules | Dependencies**.
2. Click `Alt+Insert` and select a dependency type:
 - **JARs or directories**: select a Java archive or a directory from files on your computer.
 - **Library**: select an existing library or create a new one and then add it to the list of dependencies.
 - **Module Dependency**: select another module in the project.



Remove a dependency

Before removing a dependency, make sure that it is not used in other modules in the project. To do so, select the necessary dependency and press `Alt+F7`. You can also use the **Find Usages** option of the context menu.

- Select the dependency that you want to remove and click or press `Alt+Delete`.

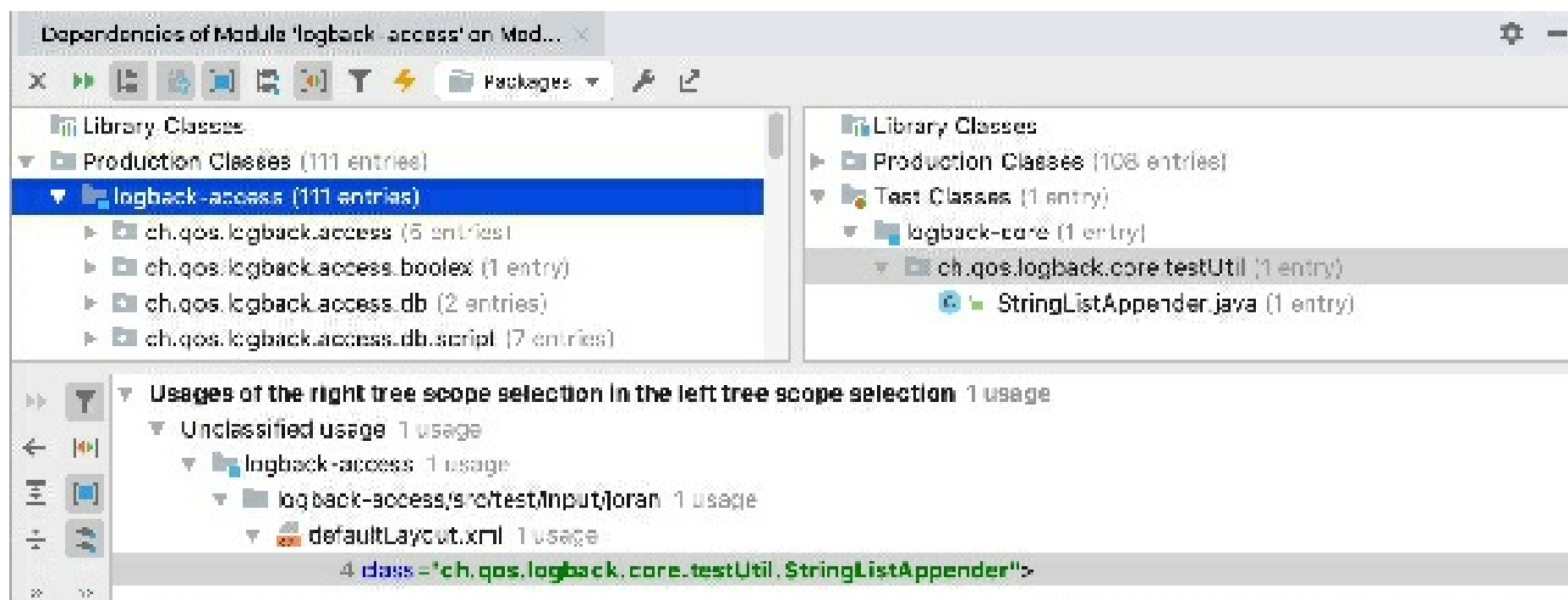
Analyze dependencies

If you want to check whether a dependency still exists in your project, and find its exact usages, you can run dependency analysis:

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules | Dependencies**.
2. Right-click the necessary dependency and select **Analyze This Dependency**.

You can analyze several dependencies one by one without closing the dialog. The result of each analysis will be opened in a separate tab of the **Dependency Viewer** tool window. After you analyze all necessary dependencies, you can close the **Project Structure** dialog and view the results.

If IntelliJ IDEA finds no dependency usages in the project, you will be prompted to remove this dependency.

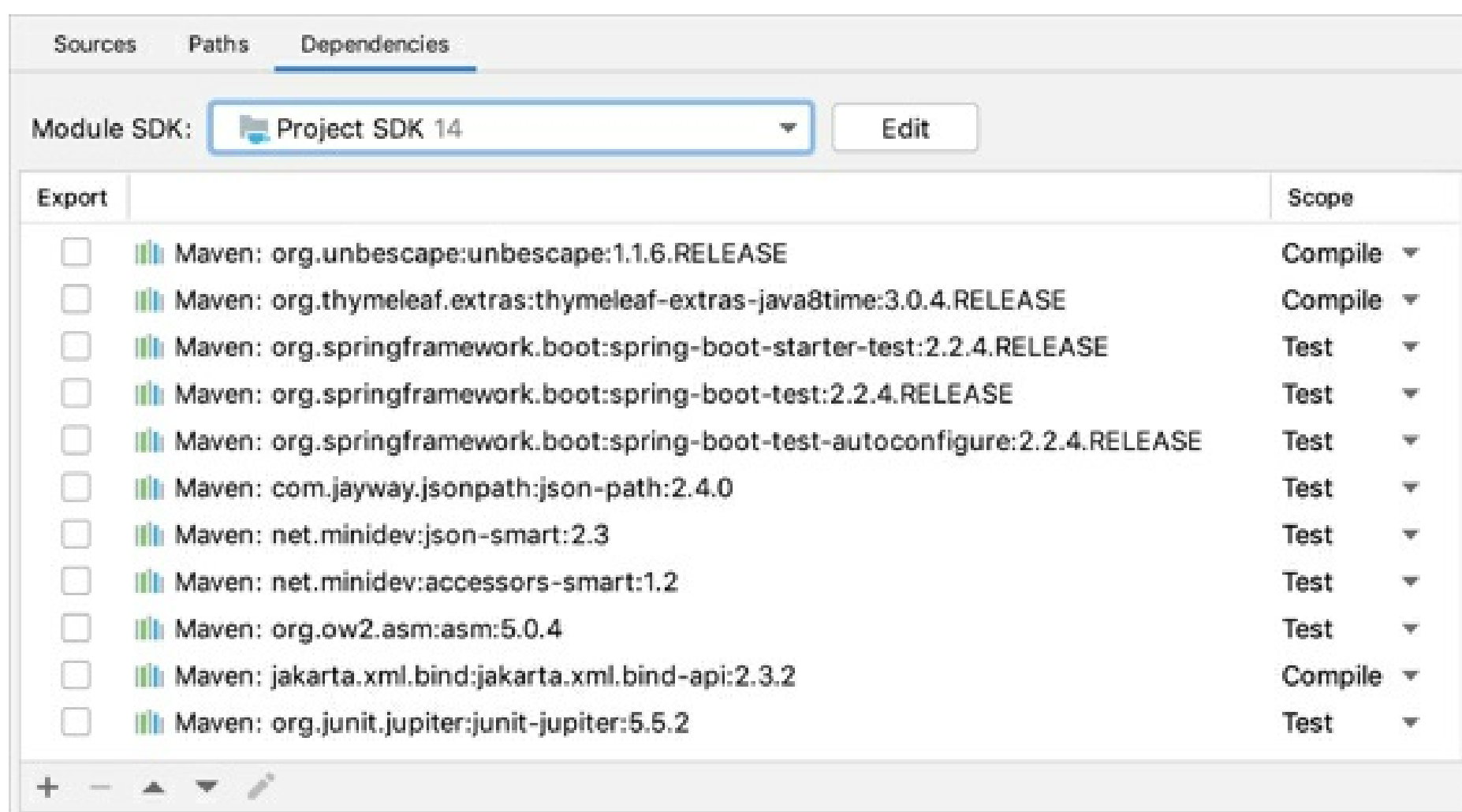


Configure a dependency scope

Specify a dependency scope

Specifying a dependency scope allows you to control at which step of the build the dependency should be used. The classpath may be different when your sources are compiled, your test sources are compiled, your compiled sources are run, your tests are run.

- From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules | Dependencies**.
- Select the necessary scope from the list in the **Scope** column:
 - **Compile**: required to build, test, and run a project (the default scope).
 - **Test**: required to compile and run unit tests.
 - **Runtime**: included in the classpath for your sources and test sources but only at the run phase.
 - **Provided**: used for building and testing a project.
- The **Export** option lets you control the compilation classpath for the modules that depend on this one: the marked items will be included in the compilation classpath of the dependent module.



IntelliJ IDEA processes dependencies for test sources differently from other build tools (for example, Gradle and Maven).

If your module (say, module A) depends on another module (module B), IntelliJ IDEA assumes that the test sources in A depend not only on the sources in B but also on its own test sources. Consequently, the test sources of B are also included in the corresponding classpaths.

The following table summarizes the classpath information for the possible dependency scopes.

Scope	Sources, when compiled	Sources, when run	Tests, when compiled	Tests, when run
Compile	+	+	+	+
Test	-	-	+	+
Runtime	-	+	-	+
Provided	+	-	+	+

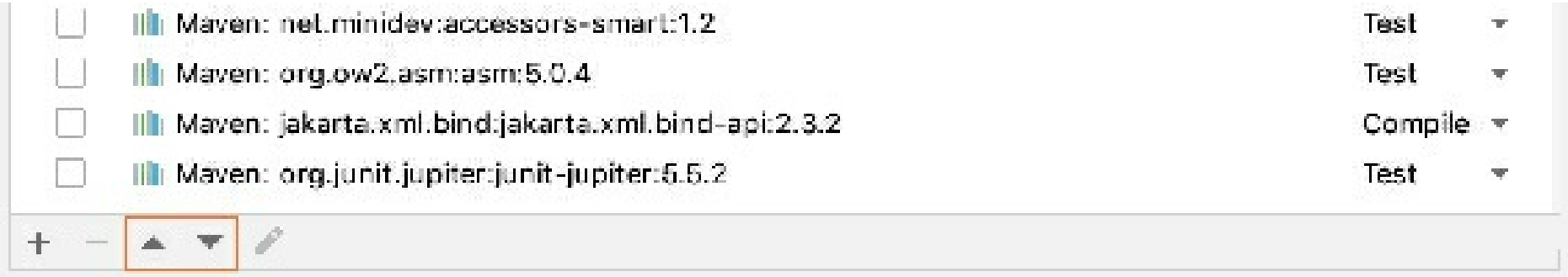
Sort dependencies

The order of dependencies is important as IntelliJ IDEA will process them in the same order as they are specified in the list.

During compilation, the order of dependencies defines the order in which the compiler (javac) looks for classes to resolve the corresponding references. At runtime, this list defines the order in which the JVM searches for the classes.

To sort dependencies, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Modules | Dependencies**

You can sort dependencies by their names and scopes. You can also use `↑` and `↓` to move the items up and down the list.



Add frameworks (facets)

For developing framework-specific applications, IntelliJ IDEA features facets. Facets contain libraries, dependencies, and technologies, and they provide you with additional UI elements for configuring framework-specific settings.

Note that not all facets are available out of the box. To be able to use some of them, you need to install a plugin for the necessary framework first. Refer to JetBrains Plugin Repository for more information on existing plugins.

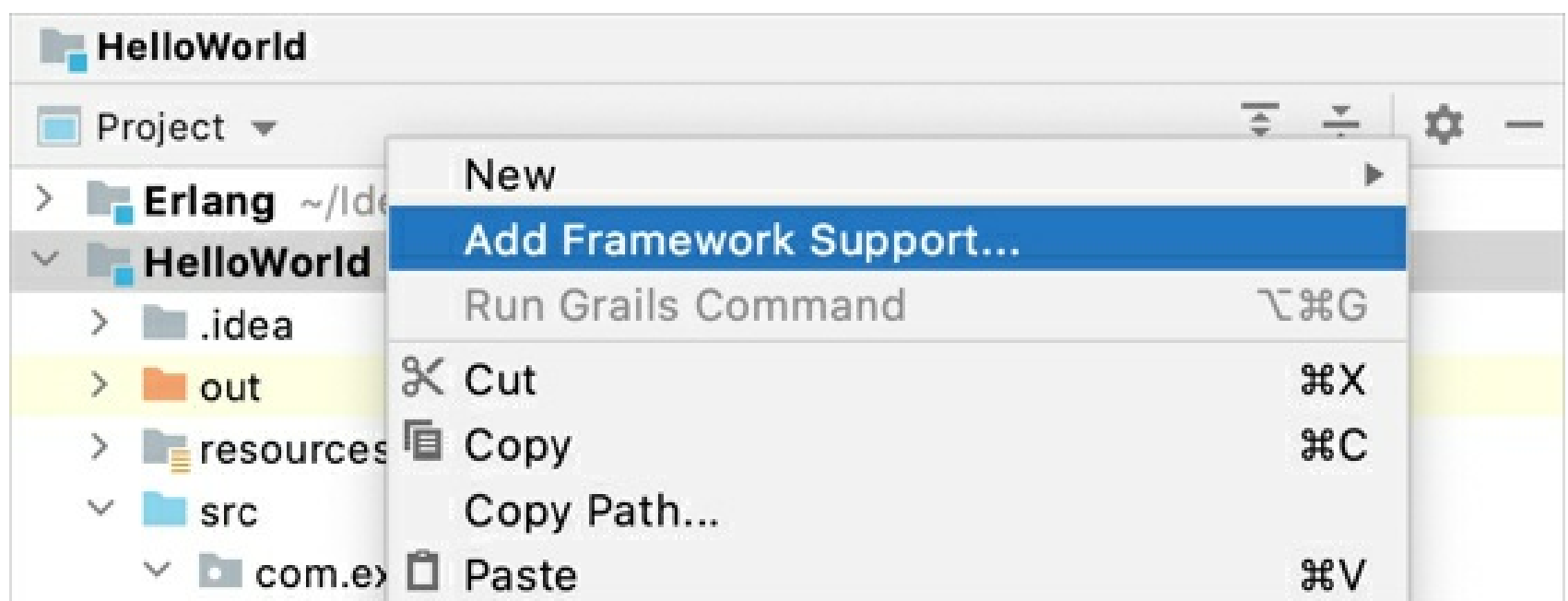
Some frameworks are available only in IntelliJ IDEA Ultimate. Refer to comparison matrix to make sure your version of IntelliJ IDEA supports the necessary frameworks.

IntelliJ IDEA can identify a file or a directory that is typical for a certain framework, and add the necessary facet for you. Once the facet is detected and added, IntelliJ IDEA will inform you about the missing configuration and will suggest the necessary actions.

If a facet is not detected automatically, you can add it manually. You can add more than one facet to a module.

Manually add a facet to a module

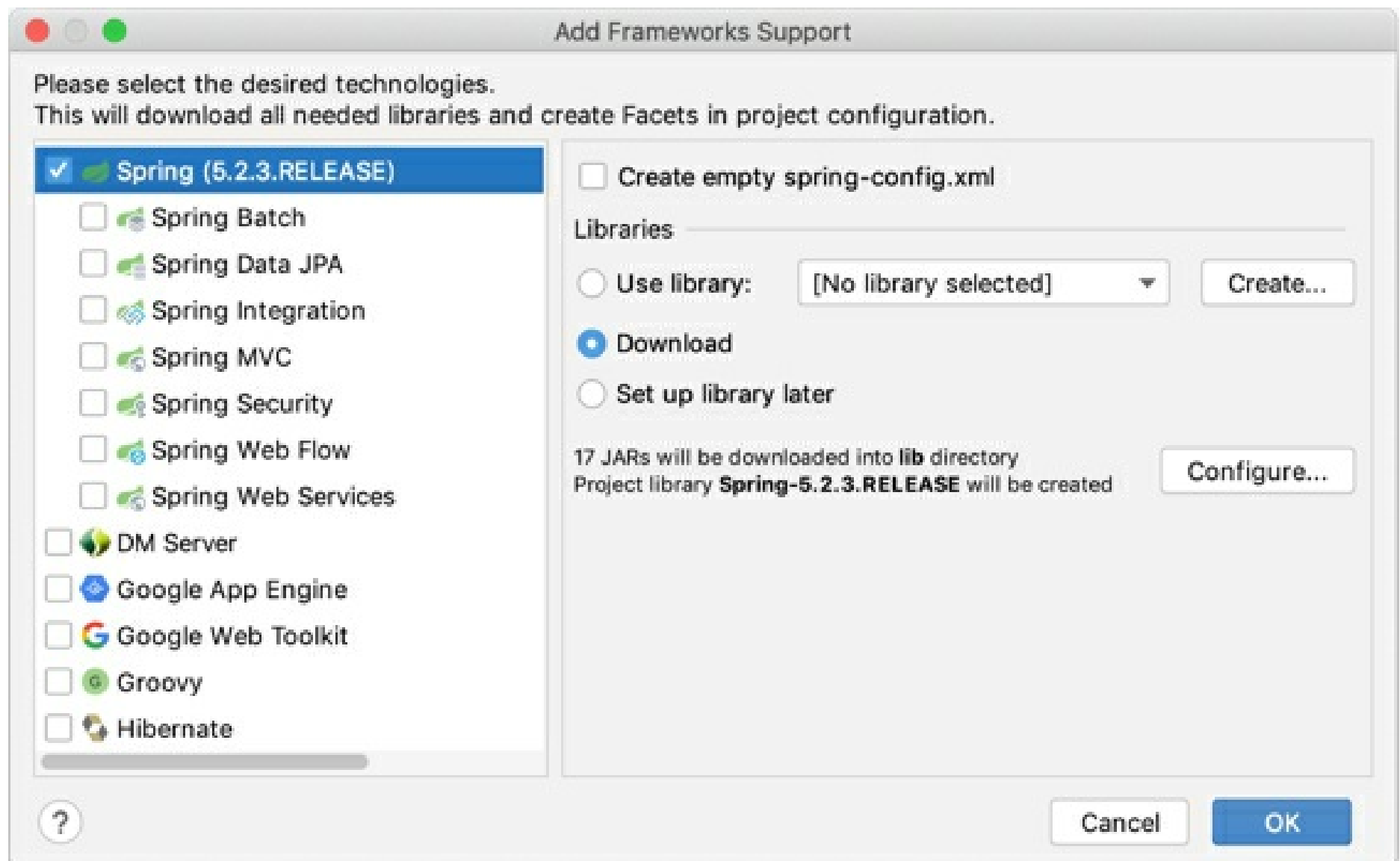
1. In the Project tool window `Alt+1`, right-click the module to which you want to add a facet, and select **Add Framework Support**.



2. Select the necessary framework from the list.

Depending on your choice, you might be prompted to configure additional settings (for example, to configure a library).

3. Apply the changes and close the dialog.



Disable framework auto-detection

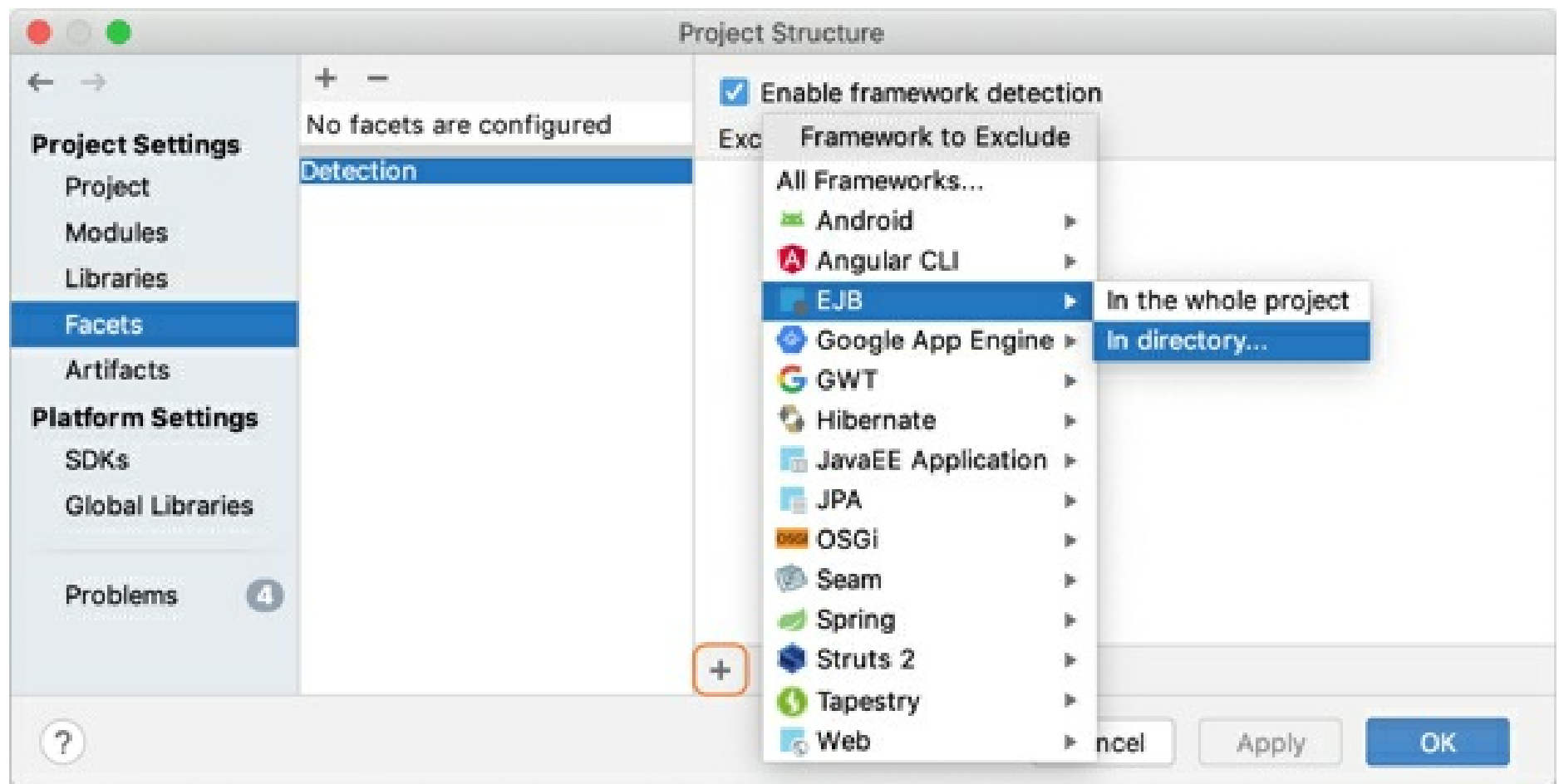
By default, auto-detection is enabled for all the supported frameworks. You can disable framework auto-detection completely, or exclude individual frameworks from auto-detection.

1. From the main menu, select **File | Project Structure** (Ctrl+Alt+Shift+S) and select **Facets**.
2. Select **Detection** and click Alt+Insert.
3. From the **Framework to Exclude** list, select the necessary option.

You can disable auto-detection of a specific framework only in one directory or in the whole project. The list also allows you to disable auto-detection of all frameworks in a specific directory.

4. If you want to disable auto-detection of all frameworks in the whole project, deselect the **Enable framework detection** checkbox.

To re-enable auto-detection everywhere, select the **Enable framework detection** checkbox and remove all entries from the **Exclude from detection** list.



In this dialog, you can also remove a framework or add a new one.

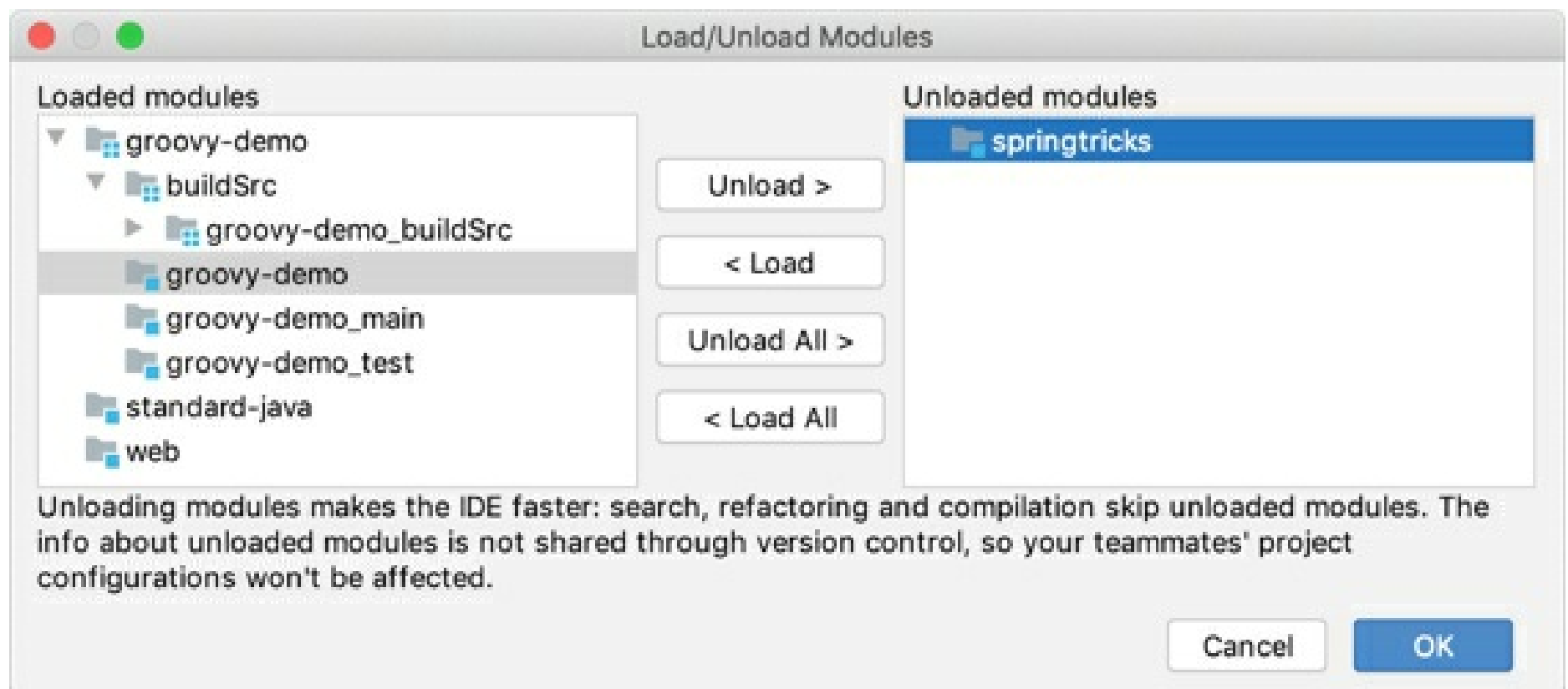
Unload modules

To make IntelliJ IDEA work faster, you can temporary set aside (unload) modules that you don't need at the moment. The IDE ignores the unloaded modules when you search through or refactor your code, or compile your project.

When you unload modules, you do it locally — the information about unloaded modules is not shared through version control.

Manually unload modules

1. In the **Project** tool window, right-click a module, and select **Load/Unload Modules**.
2. You can double-click a module in the dialog to load or unload it, or use the buttons in the middle of the dialog.

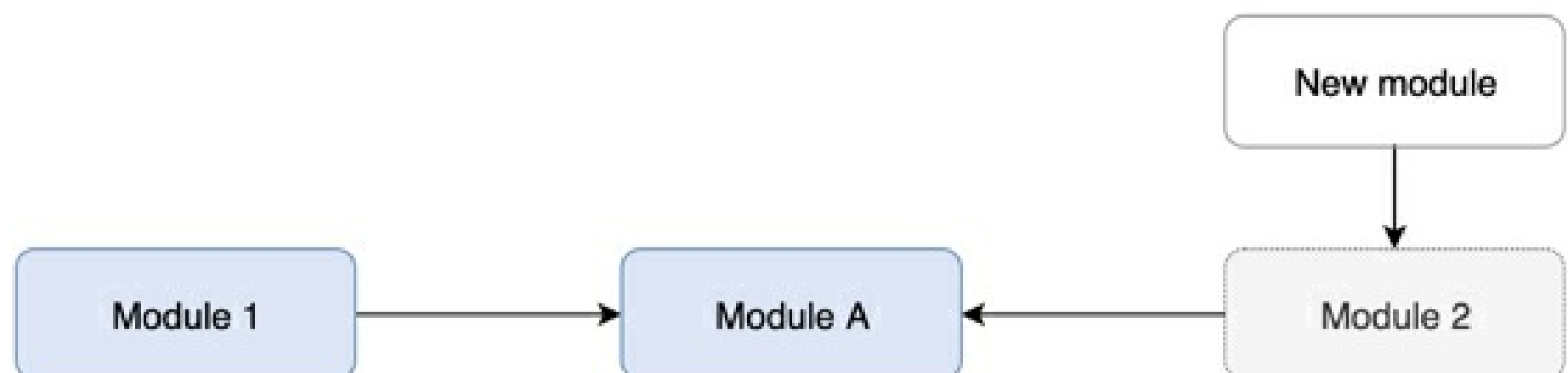


Automatically load and unload new modules

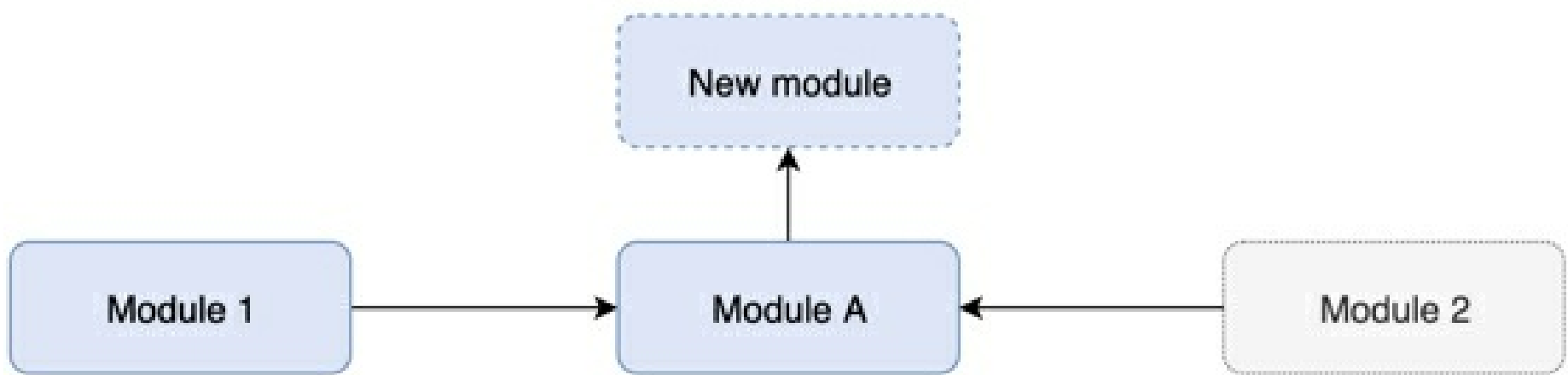
If your teammates add new modules to a project, you will download them to your computer on the project update. After that, the IDE will analyze the dependencies between all modules in the updated project.

If you have unloaded modules, IntelliJ IDEA will load or unload new modules according to the results of the dependency analysis.

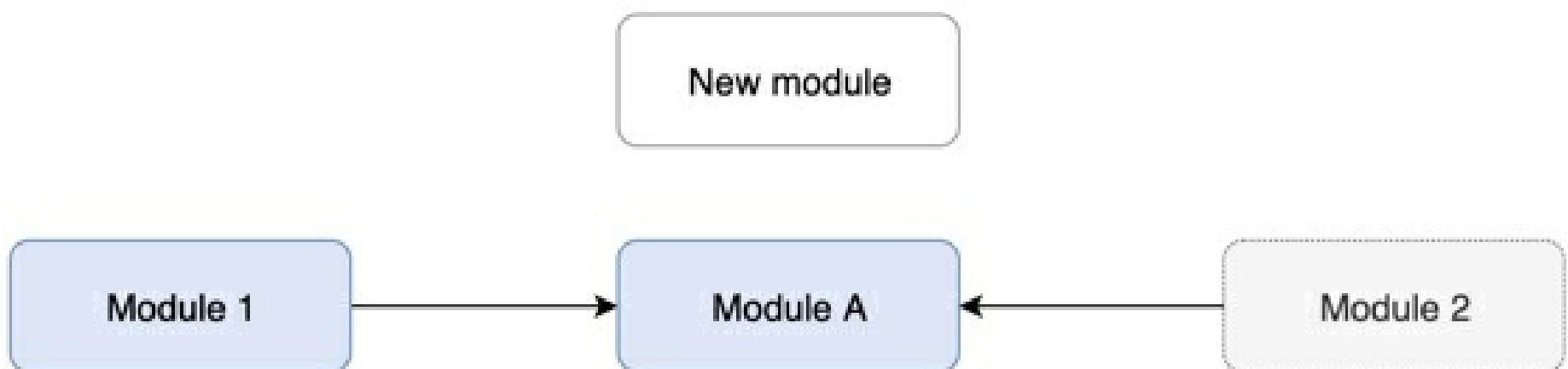
If new modules depend on the existing unloaded modules, the new modules will be marked as unloaded. IntelliJ IDEA will ignore them because otherwise you may face errors when you try to compile them.



If existing loaded modules have direct dependencies on new modules, the new modules will be marked as loaded.



If existing loaded modules have no dependencies on the newly added modules, the new modules will be marked as unloaded. You can manually mark them as loaded as soon as you need them.



Commit changes with unloaded modules

If you have unloaded modules, and you make changes in files that your unloaded modules depend on, compilation of these modules may fail after you load them back.

To avoid compilation failures of unloaded modules, make sure that the **Compile affected unloaded modules** option is selected in the **Commit Changes** dialog.

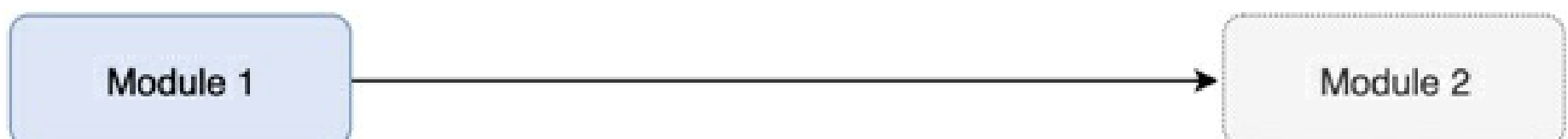
Before committing changed files, IntelliJ IDEA will compile unloaded modules to make sure that the changes don't affect these modules. The IDE will inform you about the detected errors, and will suggest resolving them before the commit.

Troubleshooting

If modules in your project depend on each other, you may face errors when you unload one or more of them.

For example, if Module 1 depends on Module 2, and you unload Module 2, IntelliJ IDEA won't be able to resolve references to classes in Module 2. Moreover, compilation of Module 1 will probably fail.

To avoid such errors, the IDE analyzes dependencies when you load or unload modules. When you load modules, IntelliJ IDEA will suggest loading all dependencies as well. When you unload modules, the IDE will find all dependent modules and will unload them, too.



If you unload Module 1, you may not see any errors in code in Module 2, and you will also be able to compile Module 2. However, you may accidentally break compilation of dependant code in Module 1 by making changes in code in Module 2. Since Module 1 is unloaded, you won't be able to see any errors until you load it back and compile.

If you invoke Find Usages `Alt+F7` or a refactoring on a class, field, or method `Ctrl+Alt+Shift+T`, contained in

Module 2, the result may be incomplete because the contents of Module 1 are not taken into account. IntelliJ IDEA will inform you about that. Moreover, the IDE will compile unloaded modules every time you commit changes, and will check that the changes don't affect unloaded modules. See more information in Committing changes with unloaded modules.



SDKs

A **Software Development Kit**, or an **SDK**, is a collection of tools that you need to develop an application for a specific software framework. For example, to develop applications in Java, you need a Java SDK (JDK). SDKs contain binaries, source code for the binaries, and documentation for the source code. JDK builds also contain annotations.

Generally, SDKs are global. It means that one SDK can be used in multiple projects and modules. After you create a new project and define an SDK for it, you can configure modules in this project to inherit its SDK. You can also specify an SDK for each module individually. For more information, refer to [Change module SDK](#).

Supported SDKs

- Java Development Kit (JDK)
- Kotlin SDK
- Android SDK
- IntelliJ Platform Plugin SDK
- JavaFX SDK
- Grails SDK

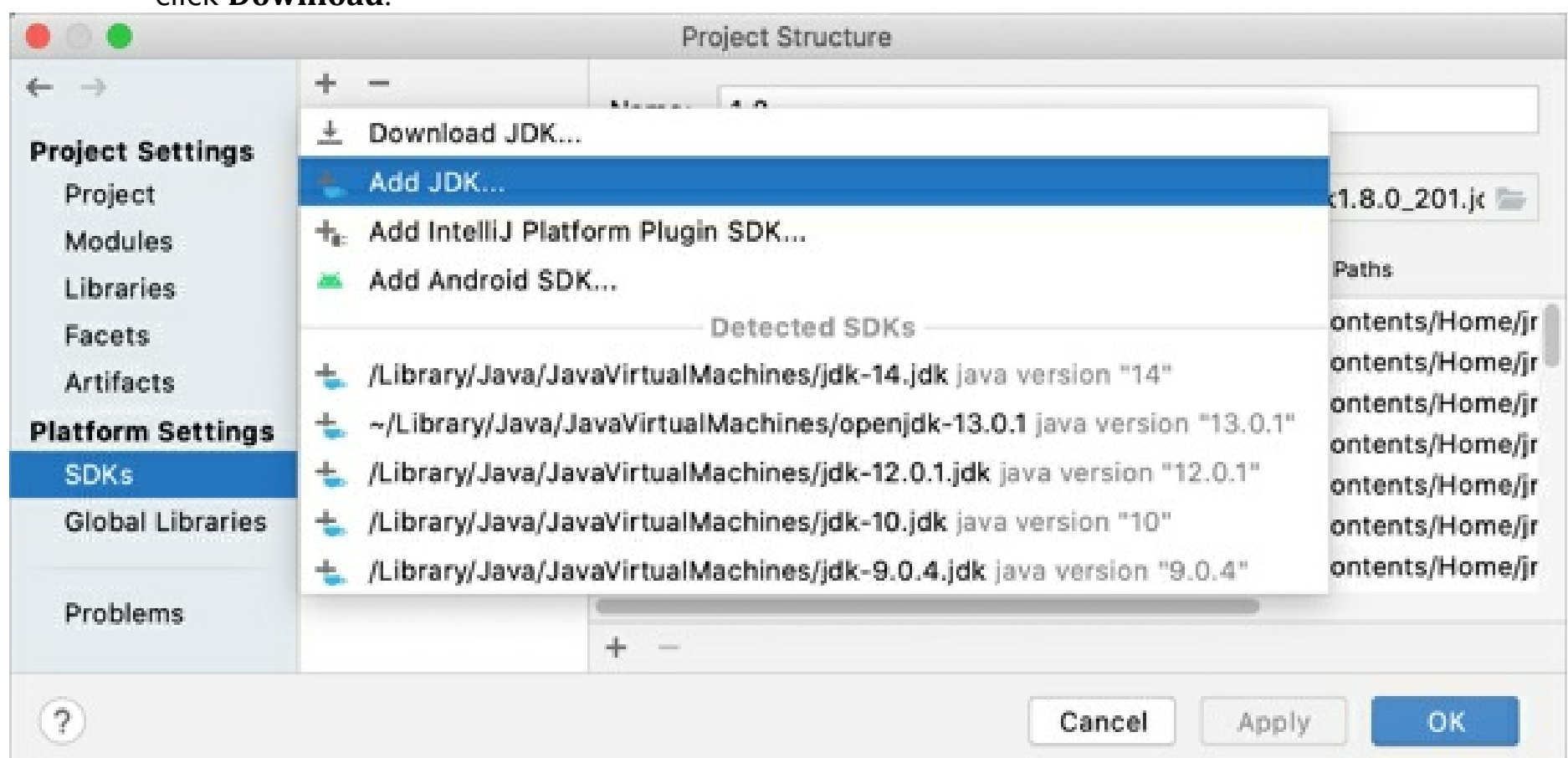
Define an SDK

To *define an SDK* means to let IntelliJ IDEA know in which folder on your computer the necessary SDK version is installed. This folder is called an *SDK home directory*.

Configure global SDKs

1. From the main menu, select **File | Project Structure | Platform Settings | SDKs**.
2. To add an SDK, click **+**, select the necessary SDK and specify its home directory in the dialog that opens.

Only for JDKs: if you don't have the necessary JDK on your computer, select **Download JDK**. In the next dialog, specify the JDK vendor, version, change the installation path if required, and click **Download**.

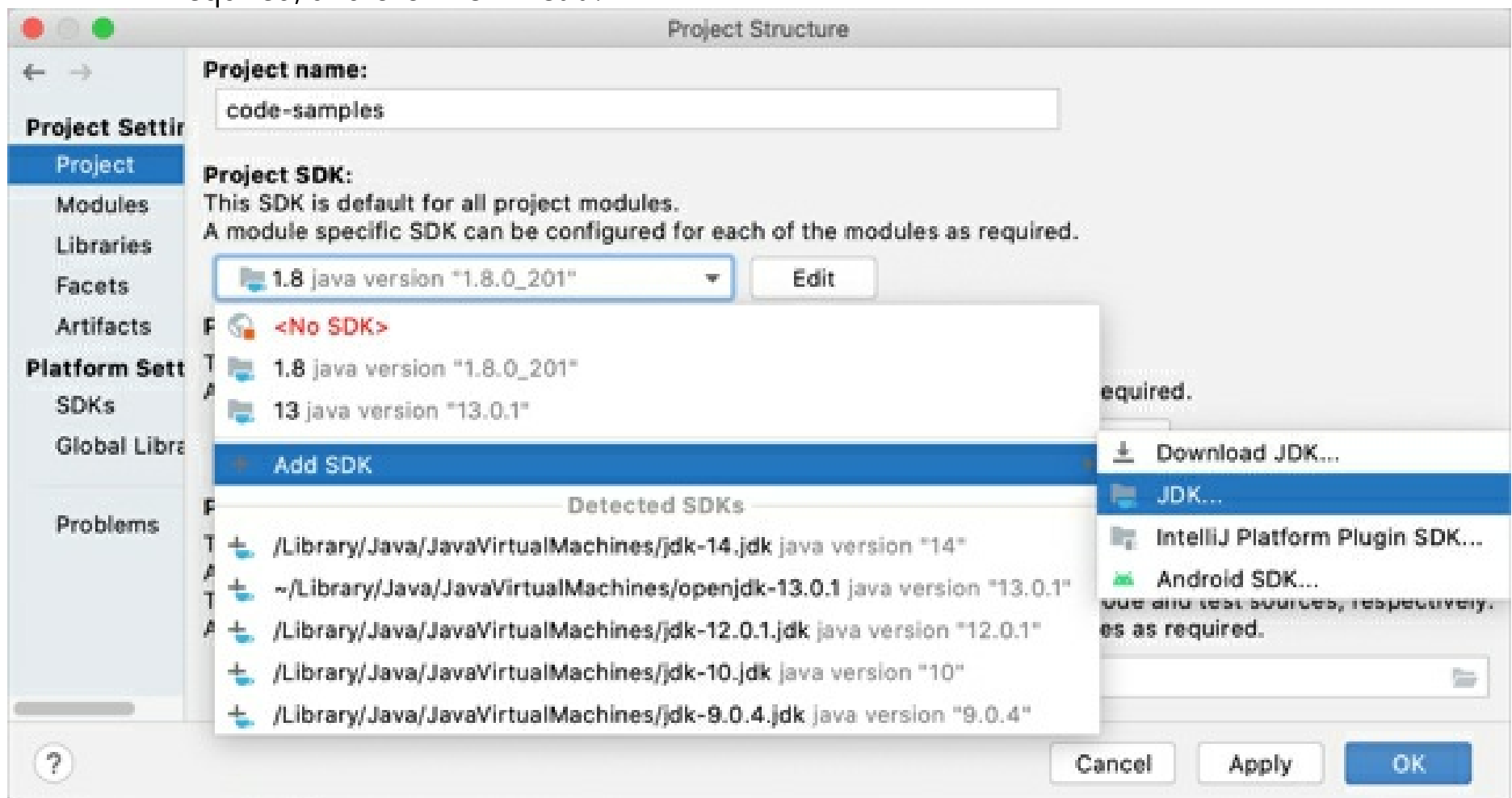


Set up a project SDK

1. From the main menu, select **File | Project Structure | Project Settings | Project**.
2. If the necessary SDK is already defined in IntelliJ IDEA, select it from the **Project SDK** list.

If the SDK is installed on your computer, but not defined in the IDE, select **Add SDK | 'SDK name'**, and specify the path to the SDK home directory.

Only for JDKs: If you don't have the necessary JDK on your computer, select **Add SDK | Download JDK**. In the next dialog, specify the JDK vendor, version, change the installation path if required, and click **Download**.



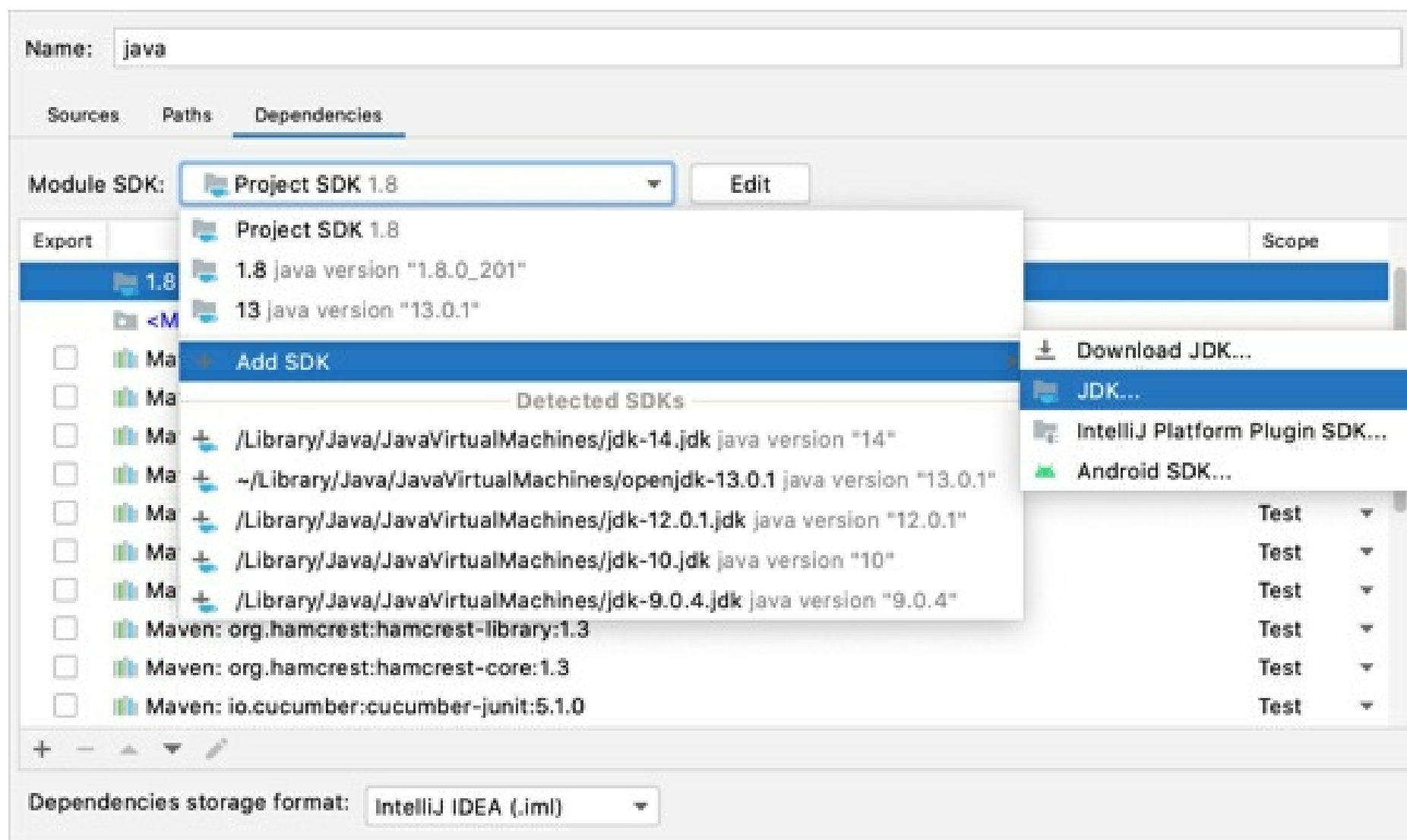
Set up a module SDK

1. From the main menu, select **File | Project Structure | Project Settings | Modules**.
2. Select the module for which you want to set an SDK and click **Dependencies**.
3. If the necessary SDK is already defined in IntelliJ IDEA, select it from the **Module SDK** list.

If the SDK is installed on your computer, but not defined in the IDE, select **Add SDK | 'SDK name'**, and specify the path to the SDK home directory.

Only for JDKs: If you don't have the necessary JDK on your computer, select **Add SDK | Download JDK**. In the next dialog, specify the JDK vendor, version, change the installation path if required, and click **Download**.

If you want a module to inherit a project SDK, select the **Project SDK** option from the **Module SDK** list.



Java Development Kit (JDK)

To develop applications in IntelliJ IDEA, you need a Java SDK (JDK). A JDK is a software package that contains libraries, tools for developing and testing Java applications (development tools), and tools for running applications on the Java platform (Java Runtime Environment — JRE).

The JRE can be obtained separately from the JDK, but it's not suitable for application development, as it doesn't have essential components such as compilers and debuggers.

Important notice

- The bundled JRE is used for running the IDE itself, and it's not sufficient for developing Java applications. Before you start developing in Java, download and install a **standalone JDK** build.
- Due to the changes in the Oracle Java License, you might not have the rights to use Oracle's Java SE for free. We recommend that you use one of the OpenJDK builds to avoid potential compliance failures.

In IntelliJ IDEA, you can download a JDK package right from the IDE, or you can manually download the necessary JDK distribution and define it in the IDE.

For a manual download, use any available distribution that you like, for example:

- Oracle OpenJDK
- AdoptOpenJDK
- Amazon Corretto

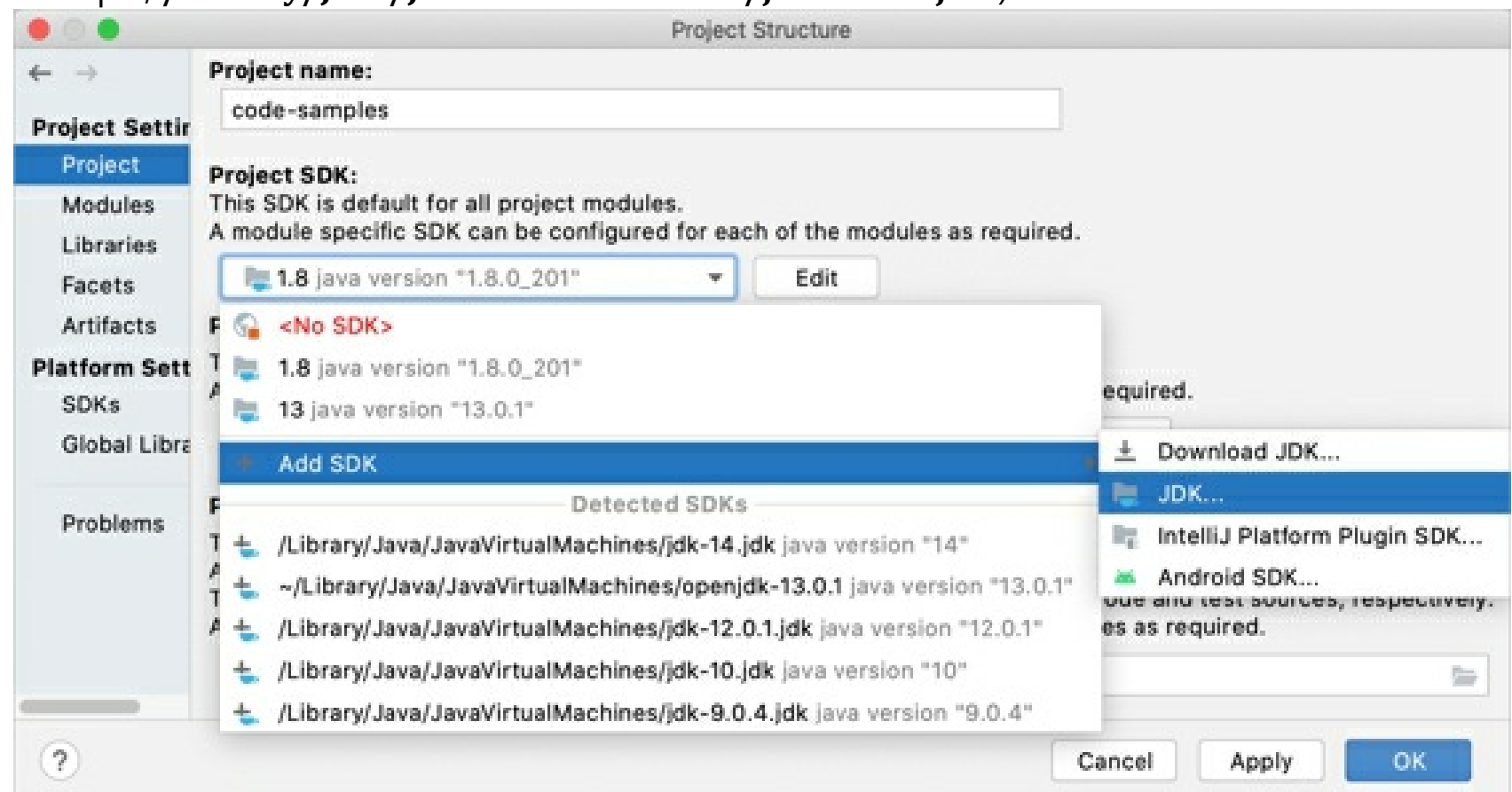
If you don't know which distribution to choose, and you don't have specific requirements that instruct you to use one of the existing distributions, use Oracle OpenJDK.

Set up the project JDK

1. From the main menu, select **File | Project Structure | Project Settings | Project**.
2. If the necessary JDK is already defined in IntelliJ IDEA, select it from the **Project SDK** list.

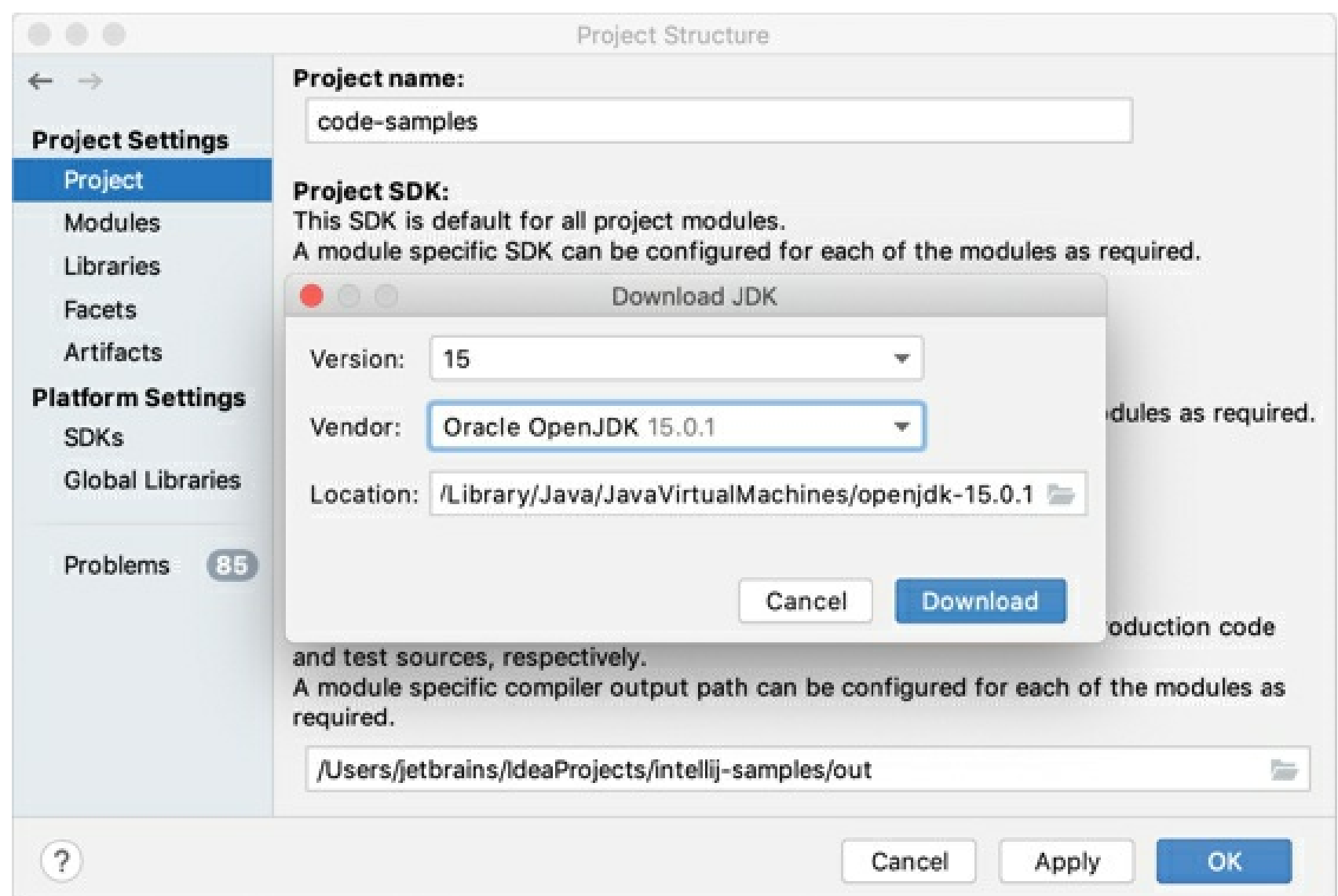
If the JDK is installed on your computer, but not defined in the IDE, select **Add SDK | JDK**, and specify the path to the JDK home directory (for

example, `/Library/Java/JavaVirtualMachines/jdk-12.0.1.jdk`).



If you don't have the necessary JDK on your computer, select **Add SDK | Download JDK**. In the next dialog, specify the JDK vendor, version, change the installation path if required, and click **Download**.

3. Apply the changes and close the dialog.



If you build your project with Maven or Gradle, refer to [Change the JDK version in a Maven project](#) and [Gradle JVM selection](#) respectively for more information on how to work with JDKs.

Configure SDK documentation

You can add SDK documentation to IntelliJ IDEA so that you can get information about symbols and method signatures right from the editor in the Quick documentation popup.

You can also configure external documentation by specifying the path to the reference information online. External documentation opens the necessary information in a browser so that you can navigate to

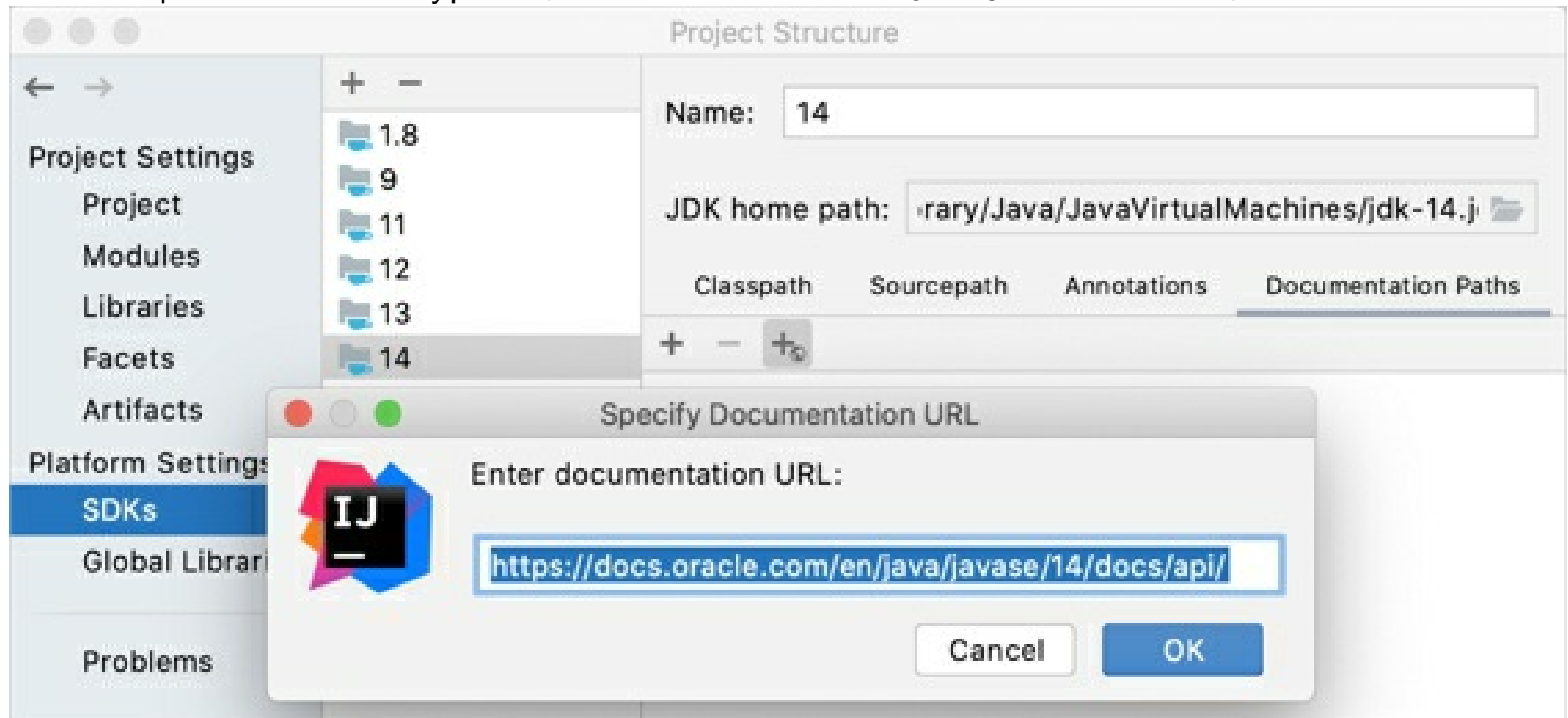
related symbols and keep the information for further reference at the same time.

Specify SDK documentation paths

To view external SDK documentation, configure the documentation URL first.

1. In the **Project Structure** dialog `Ctrl+Alt+Shift+S`, select **SDKs**.
2. Select the necessary SDK version if you have several SDKs configured, and open the **Documentation Path** tab on the right.
3. Click the  icon, enter the external documentation URL, and click **OK**.

For example, for Java 14, type `https://docs.oracle.com/en/java/javase/14/docs/api/`.



4. Apply the changes and close the dialog.

Access SDK documentation offline

If you work offline, you can view external documentation locally.

1. Download the documentation package of the necessary version.

The documentation package is normally distributed in a ZIP archive that you need to unpack once it is downloaded.

For example, you can download the official Java SE Development Kit 14.0.1 Documentation and unzip it.

2. In the **Project Structure** dialog `Ctrl+Alt+Shift+S`, select **SDKs**.
3. Select the necessary JDK version if you have several JDKs configured, and open the **Documentation Path** tab on the right.
4. Click the  icon and specify the directory with the downloaded documentation package (for example, `C:\Users\jetbrains\Desktop\docs\api`).
5. Apply the changes and close the dialog.

Supported Java versions and features

This page lists all Java versions and preview features supported by IntelliJ IDEA for developing applications. For more information about Java releases and features in each release, refer to [Java version history](#).

If you need to run IntelliJ IDEA using another Java runtime, refer to [Change the boot Java runtime of the IDE](#) for instructions.

IntelliJ IDEA 2021.X

IDE version	Java version
IntelliJ IDEA 2021.1	• Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 • Java 12: no new language features • Java 13: no new language features Java 14 standard language features: • Switch expressions Java 15 standard language features: • Text blocks Java 15 preview features: • Sealed types • Records • Patterns • Local enums and interfaces Java 16 standard language features: • Records • Patterns • Local enums and interfaces Java 16 preview features: • Sealed types For more information, refer to the Java 16 and IntelliJ IDEA blog post.

IntelliJ IDEA 2020.X

IDE version	Java version
IntelliJ IDEA 2020.3	• Java 6

- Java 7
- Java 8
- Java 9
- Java 10
- Java 11
- Java 12: no new language features
- Java 13: no new language features

Java 14 standard language features:

- Switch expressions

Java 14 preview features:

- Records
- Patterns
- Text blocks

Java 15 standard language features:

- Text blocks

Java 15 preview features:

- Sealed types
- Records
- Patterns
- Local enums and interfaces

For more information, refer to the Java 15 and IntelliJ IDEA blog post.

IntelliJ IDEA 2020.2

- Java 6
- Java 7
- Java 8
- Java 9
- Java 10
- Java 11
- Java 12: no new language features
- Java 13: no new language features

Java 14 standard language features:

- Switch expressions

Java 14 preview features:

- Records
- Patterns
- Text blocks

Java 15 standard language features:

- Text blocks

	Java 15 preview features: • Sealed types • Records • Patterns • Local enums and interfaces For more information, refer to the Java 15 and IntelliJ IDEA blog post.
IntelliJ IDEA 2020.1	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 • Java 12: no new language features • Java 13: no new language features Java 13 preview features: • Switch expressions • Text blocks Java 14 standard language features: • Switch expressions Java 14 preview features: • Records • Patterns For more information, refer to the Java 14 and IntelliJ IDEA blog post.

IntelliJ IDEA 2019.X

IDE version	Java version
IntelliJ IDEA 2019.3	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11

	• Java 12: no new language features • Java 13: no new language features Java 13 preview features: • Switch expressions • Text blocks For more information on Java 13 support in IntelliJ IDEA 2019.3, refer to the Java 13 and IntelliJ IDEA blog post.
IntelliJ IDEA 2019.2	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 • Java 12: no new language features • Java 13: no new language features Java 13 preview features: • Switch expressions • Text blocks For more information, refer to the Support for Java 13 Preview Features in IntelliJ IDEA 2019.2 blog post.
IntelliJ IDEA 2019.1	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 • Java 12: no new language features • Java 12 Switch expressions (preview feature) For more information, refer to: • Live Webinar: Java 12 in IntelliJ IDEA • Java 12 and IntelliJ IDEA

IntelliJ IDEA 2018.X

IDE version	Java version
-------------	--------------

IntelliJ IDEA 2018.3	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 • Java 12: no new language features
IntelliJ IDEA 2018.2	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 • Java 11 For more information about, refer to: • Java 11 and IntelliJ IDEA • Java 11 in IntelliJ IDEA 2018.2 • Using Java 11 In Production: Important Things To Know
IntelliJ IDEA 2018.1	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 • Java 10 For more information, refer to: • Webinar: IntelliJ IDEA and Java 10 • Advanced Support for Java 9 Modules in IntelliJ IDEA 2018.1

IntelliJ IDEA 2017.X

IDE version	Java version

IntelliJ IDEA 2017.3	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 For more information, refer to the Java 9 and IntelliJ IDEA.
IntelliJ IDEA 2017.2	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 For more information, refer to the Support for Java 9 in IntelliJ IDEA 2017.2.
IntelliJ IDEA 2017.1	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9 For more information, refer to the Support for Java 9 in IntelliJ IDEA 2017.1.

IntelliJ IDEA 2016.X

IDE version	Java version
IntelliJ IDEA 2016.3 IntelliJ IDEA 2016.2 IntelliJ IDEA 2016.1	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9

IntelliJ IDEA 15

IDE version	Java version
IntelliJ IDEA 15	• Java 1.3

	• Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8 • Java 9
--	---

IntelliJ IDEA 14.X

IDE version	Java version
IntelliJ IDEA 14.1 IntelliJ IDEA 14.0	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8

IntelliJ IDEA 13

IDE version	Java version
IntelliJ IDEA 13	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8

IntelliJ IDEA 12

IDE version	Java version
IntelliJ IDEA 12	• Java 1.3 • Java 1.4 • Java 5 • Java 6 • Java 7 • Java 8

IntelliJ IDEA 11.X

IDE version	Java version

IntelliJ IDEA 11.1

IntelliJ IDEA 11.0

- Java 1.3
- Java 1.4
- Java 5
- Java 6
- Java 7
- Java 8

Libraries

A library is a collection of compiled code that you can add to your project. In IntelliJ IDEA, libraries can be defined at three levels: *global* (available for many projects), *project* (available for all modules within a project), and *module* (available for one module).

A Java library can include class files, archives and directories with class files as well as directories with Java native libraries **.dll**, **.so**, or **.jnlib**.

This information is valid for projects that are built with the native IntelliJ IDEA builder. If you're using a build tool, such as Maven or Gradle, make all changes using the build file.

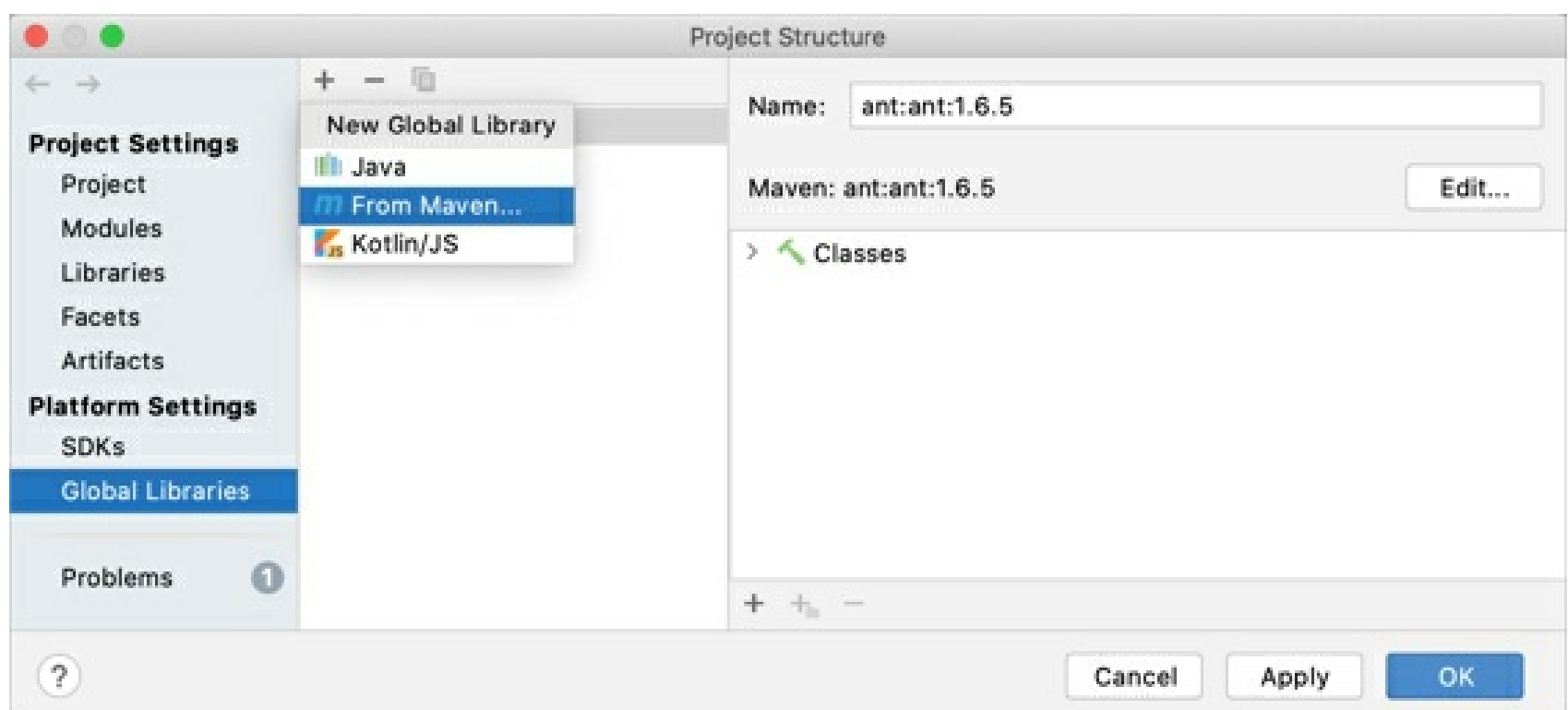
Define a library

After you define a library and add it to module dependencies, the IDE will be supplying its contents to you as you write your code. IntelliJ IDEA will also use the code from the libraries to build and deploy your application.

You can also create a new library from the JAR files located within a project content root. Select these files in the **Project** tool window, and then select **Add as Library** from the context menu.

Define a global library

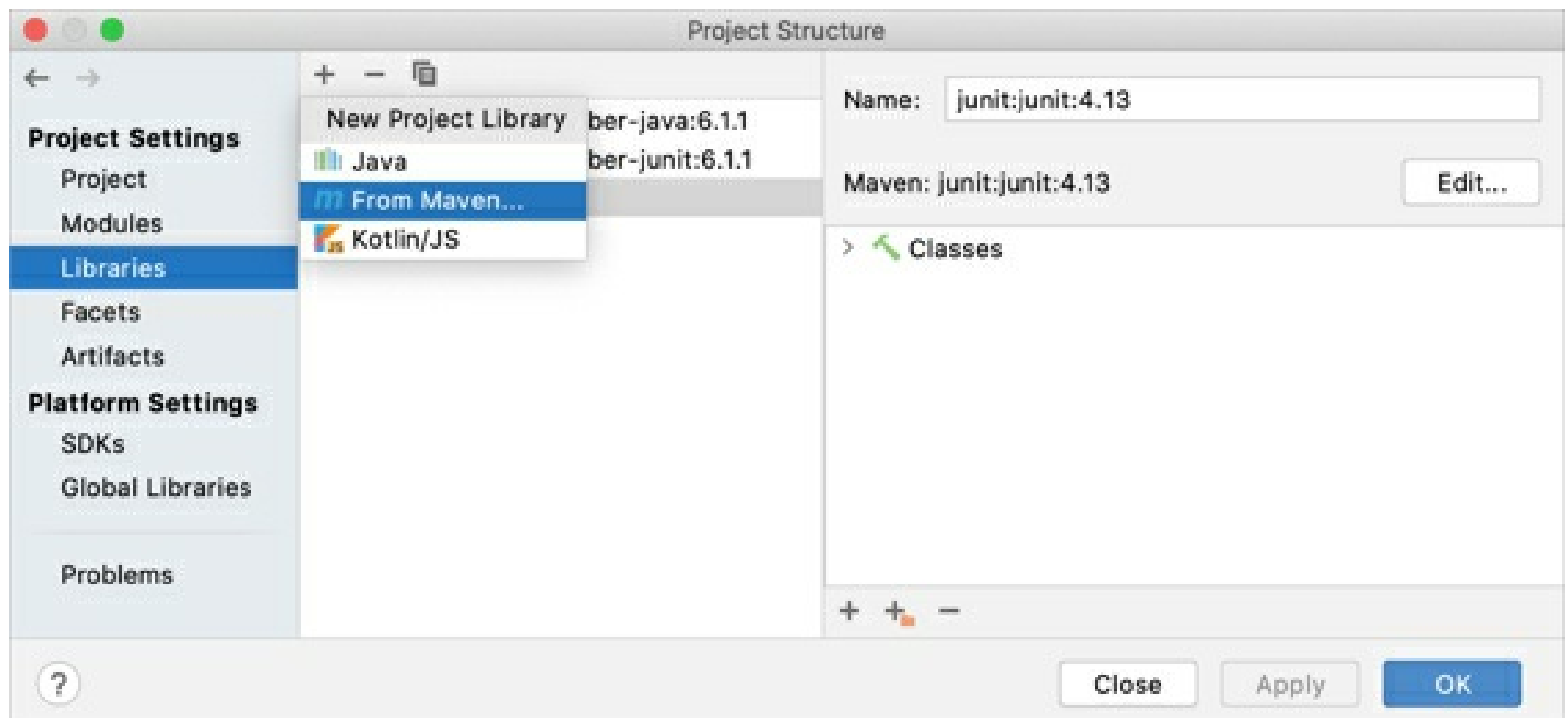
1. From the main menu, select **File | Project Structure** Ctrl+Alt+Shift+S.
2. Under **Platform Settings**, select **Global Libraries**.
3. Click  and select one of the following:
 - Select **Java** or **Kotlin/JS** to add a library from the files located on your computer.
 - Select **From Maven** to download a library from Maven.



References to global libraries are stored in the IDE configuration directory in **options | applicationLibraries.xml**.

Define a project library

1. From the main menu, select **File | Project Structure** Ctrl+Alt+Shift+S.
2. Under **Project Settings**, select **Libraries**.
3. Click  and select one of the following:
 - Select **Java** or **Kotlin/JS** to add a library from the files located on your computer.
 - Select **From Maven** to download a library from Maven.

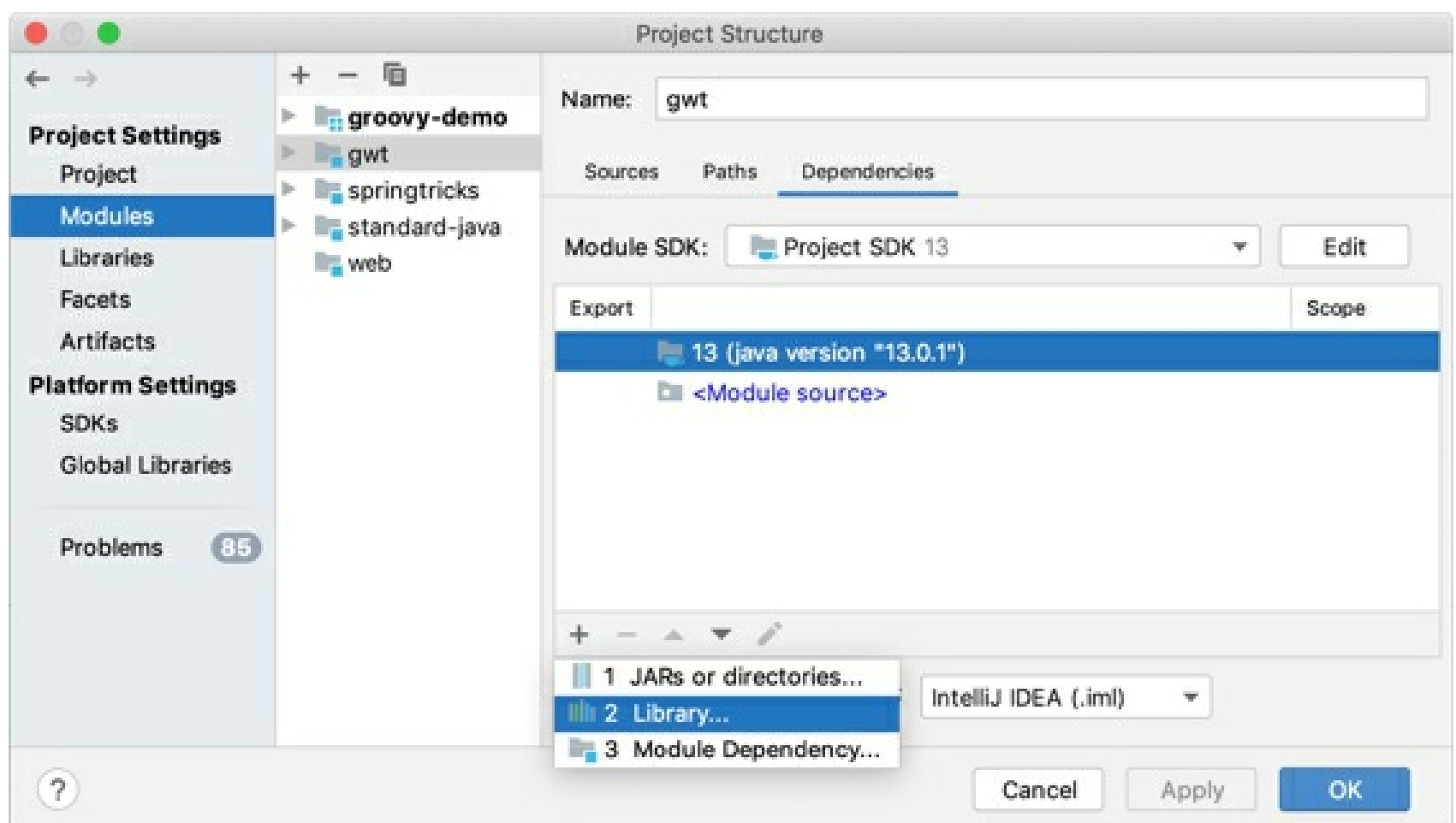


References to project libraries are stored together with the project in the **.idea** folder in **libraries**.

Add a library to module dependencies

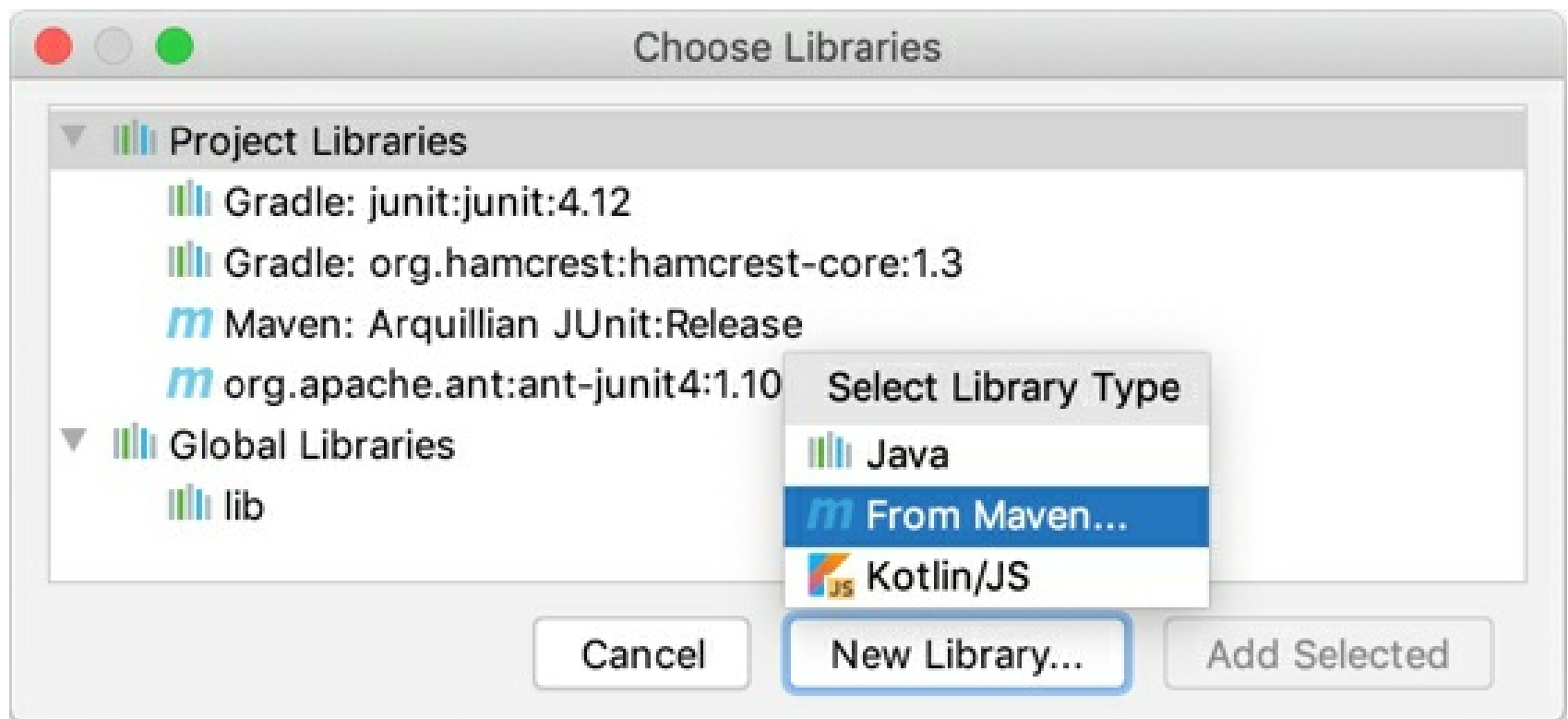
Global and project libraries are not available until you add them to module dependencies.

1. From the main menu, select **File | Project Structure | Project Settings | Modules**.
2. Select the module for which you want to add a library and click **Dependencies**.
3. Click the **+** button and select **Library**.



4. In the dialog that opens, select a project or a global library that you want to add to the module.

Alternatively, click **New Library** and select how do you want to add a new library: you can add a Java and Kotlin libraries from files on your computer, or download a library from Maven.

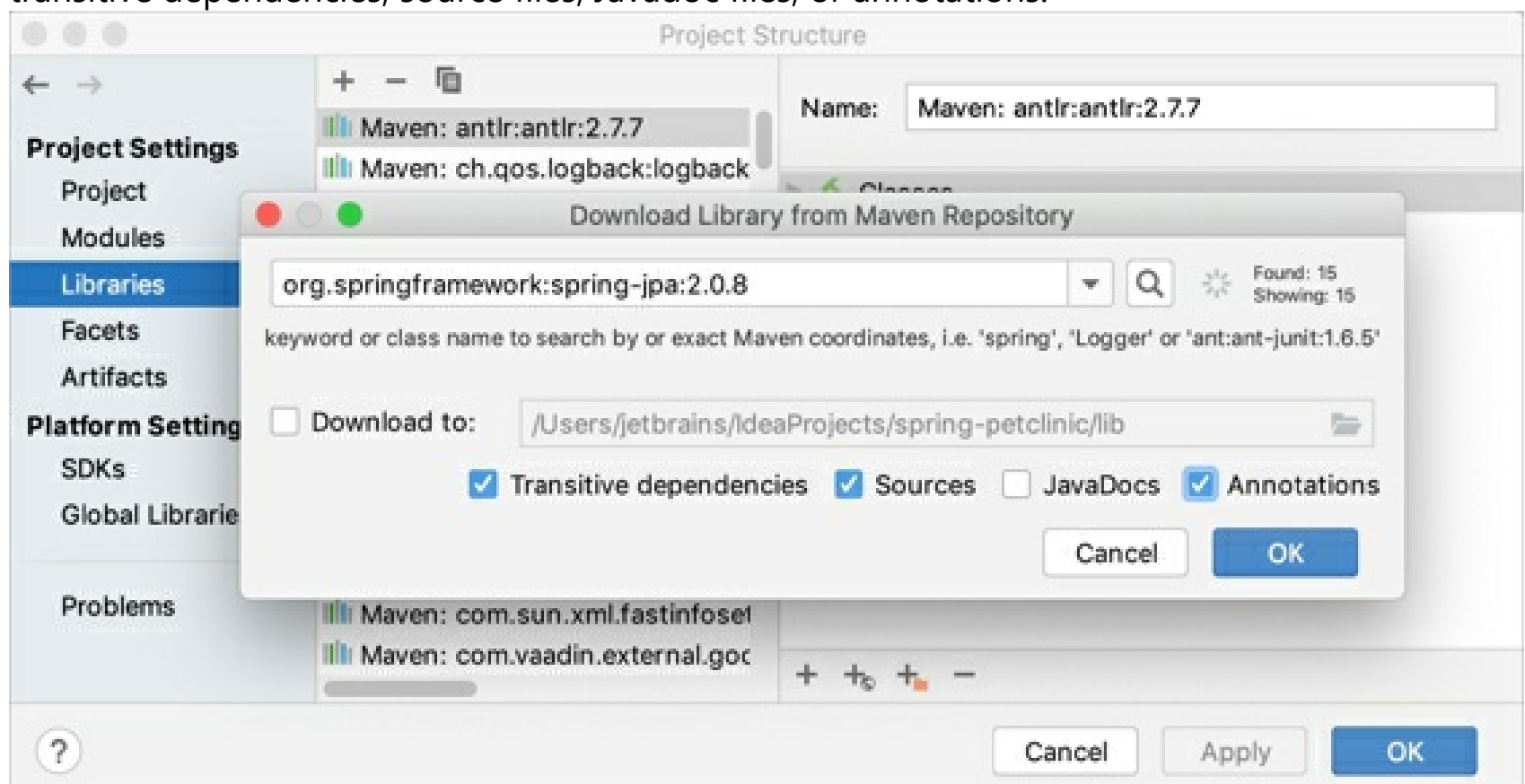


References to module libraries are stored in the module **.iml** file. This file is used for keeping module configuration. For more information on module files, refer to Modules.

Download a library from Maven

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Libraries**.
2. Click **+** and select **From Maven**.
3. In the next dialog, specify the library artifact (for example, `org.jetbrains:annotations:16.0.2`). If you don't know its exact name, enter the key words and click **Search**.

You can also specify another library location, and select whether you want to download transitive dependencies, source files, Javadoc files, or annotations.



IntelliJ IDEA will download the library from Maven or Nexus public repositories. You can also configure a custom remote repository.

Include specific transitive dependencies

If you want to include only specific transitive dependencies, you can use the library properties editor.

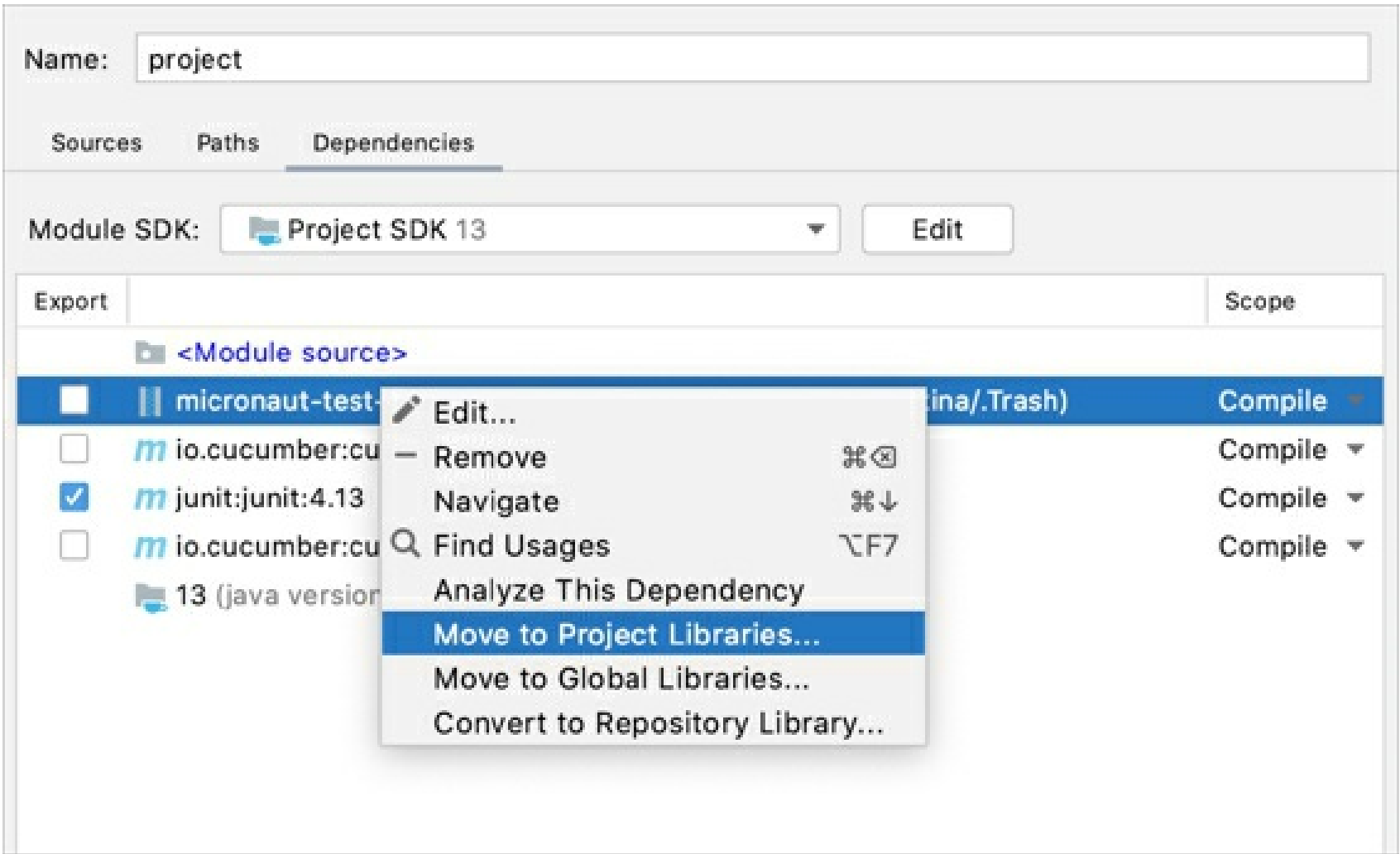
1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S`, and go to **Modules | Dependencies**.
2. Select the necessary Maven library and click **+**.
3. In the next dialog, click **Edit**, and then click **Configure** next to the **Include transitive dependencies** option.
4. Select the dependencies you want to include in the library and click **OK**.

Change the library level

Move a library to a higher level

In IntelliJ IDEA, you can move a project or a module library to a higher level. This is helpful if you want to extend its scope of usage. For example, use this procedure if you want to use a module library across the project or the entire IDE.

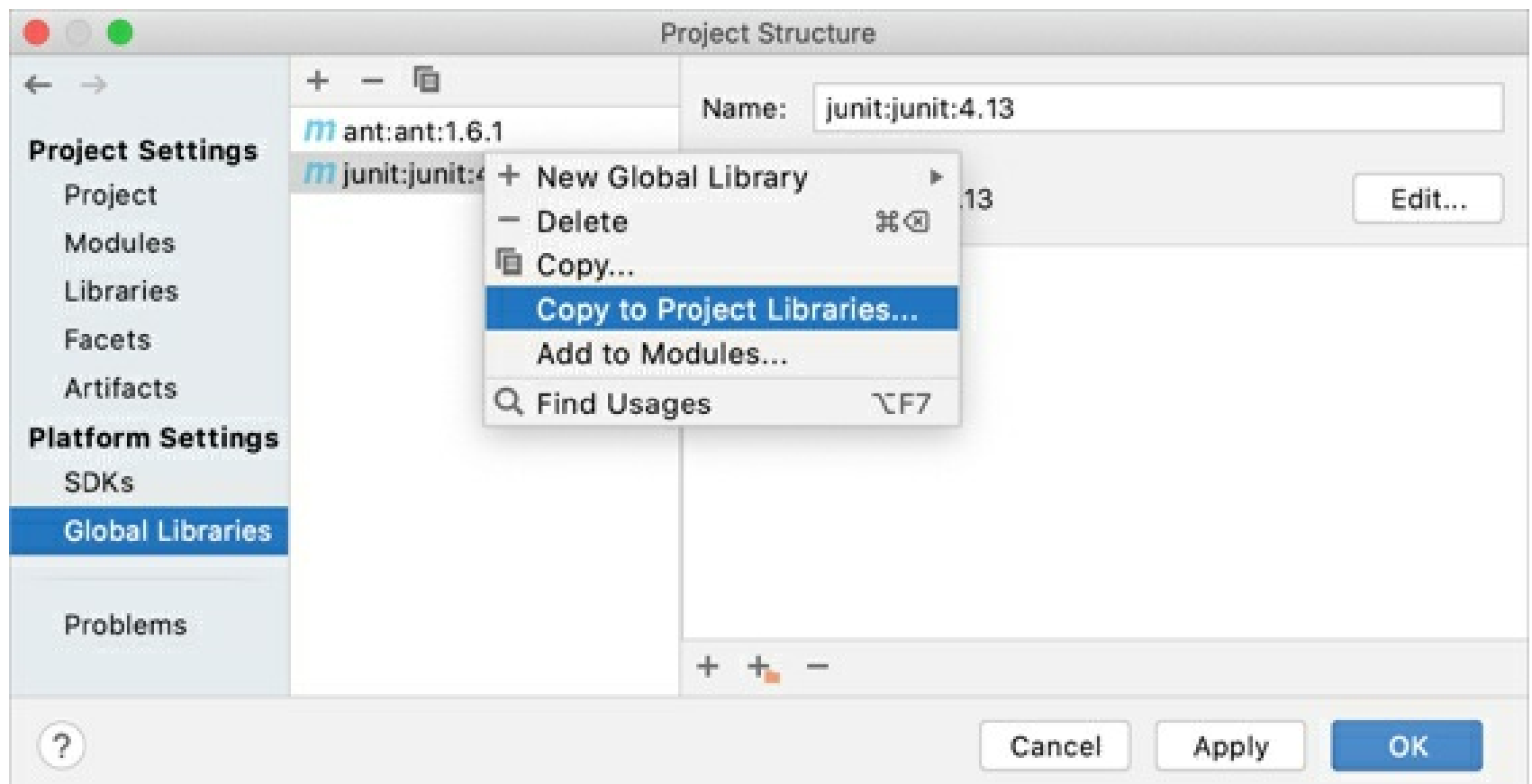
- 1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and locate the library that you want to change.
- 2. Right-click the necessary library and select **Move to Project Libraries** or **Move to Global Libraries**.



Copy a library to a lower level

You can create a copy of a library on a lower level. For example, use this procedure if you want to add more classes to a project library, but you want to use them in one module only.

- 1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and locate the library that you want to change.
- 2. Right-click the necessary library and select **Copy to Project Libraries** or **Add to Modules**.



Exclude library items

IntelliJ IDEA allows you to temporarily exclude library items in order to increase IDE performance. You can exclude folders, archives (for example, JARs) and folders within archives.

Classes from excluded packages won't be shown in code completion suggestion lists, references to such classes will be shown in the editor as unresolved, and so on. However, when you compile or run your code, a library will still be used as a whole, irrespective of whether there are excluded items in that library or not.

Exclude items from a project or a global library

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Libraries**.
2. Click  and select the library items that you want to exclude.

Exclude items from a module library

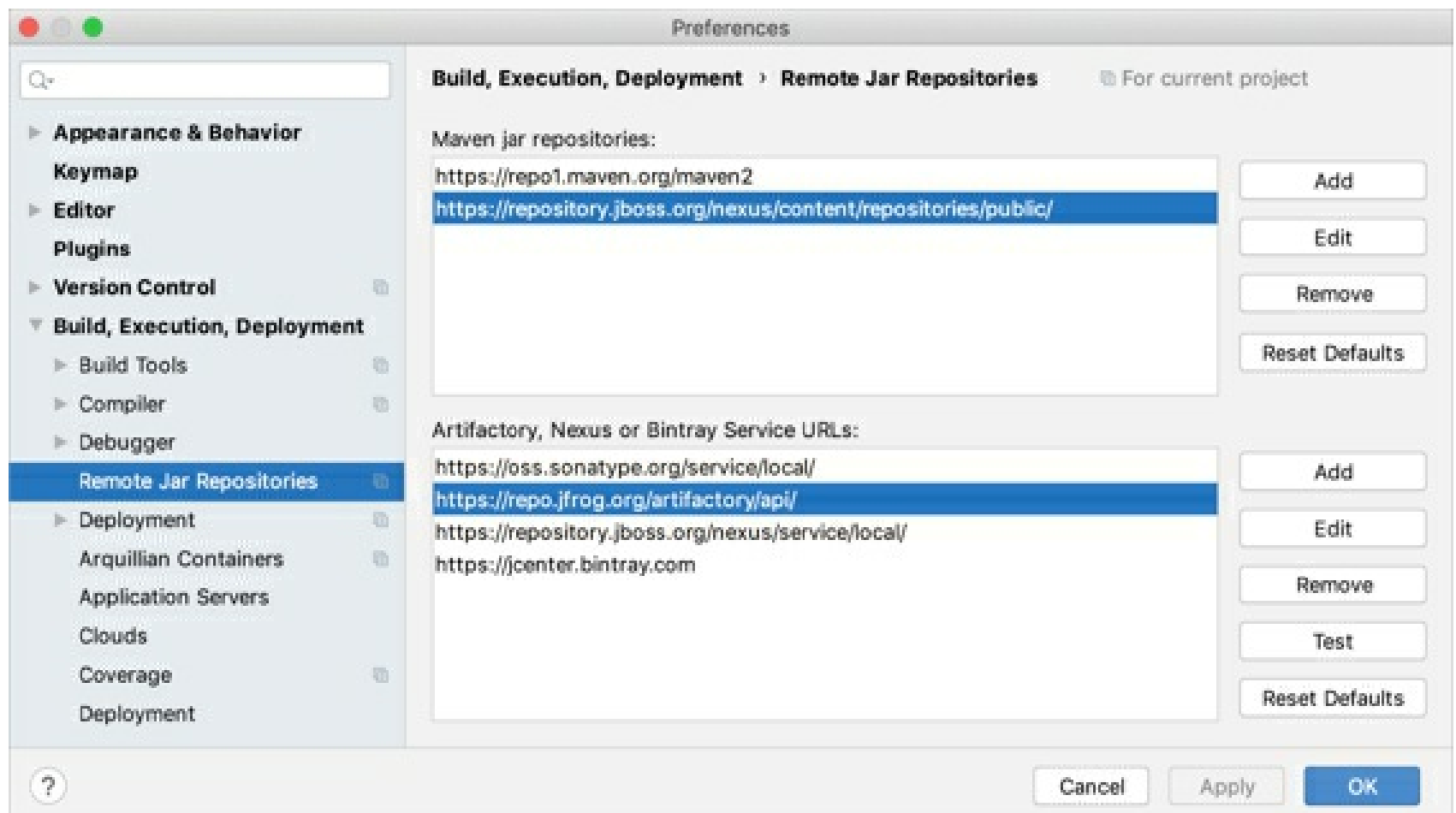
1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and go to **Modules | Dependencies**.
2. Click  and select the library items that you want to exclude.

Excluded items will be marked with the  icon. To return library items to their original state, remove the excluded items.

Configure a custom remote repository

You can view the full list of remote repositories and add a custom repository in the settings. Note that IntelliJ IDEA can load libraries from Maven even if you don't use Maven as a build tool for your project.

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Build, Execution, Deployment | Remote Jar Repositories**.
2. Click **Add** in the corresponding dialog section, and specify the repository URL.



Configure library documentation

You can add library documentation to IntelliJ IDEA so that you can get information about symbols and method signatures right from the editor in the Quick documentation popup.

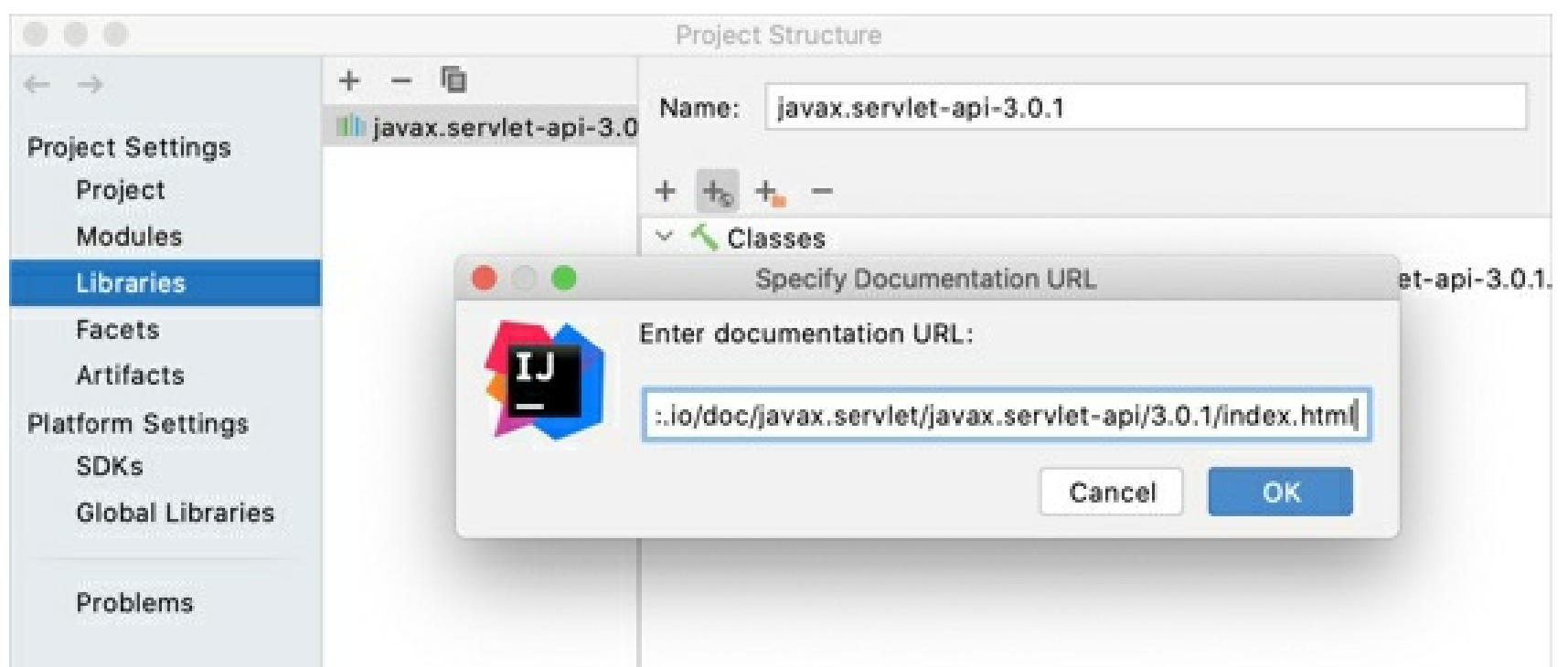
You can also configure external documentation by specifying the path to the reference information online. External documentation opens the necessary information in a browser so that you can navigate to related symbols and keep the information for further reference at the same time.

If you're downloading a library from Maven, select the **JavaDoc** checkbox to get the library documentation together with the library code.

Specify library documentation paths

To view external library documentation, configure the documentation URL first.

1. In the **Project Structure** dialog `Ctrl+Alt+Shift+S`, select **Libraries**.
2. Select the necessary library, click the  icon and enter the external documentation URL.



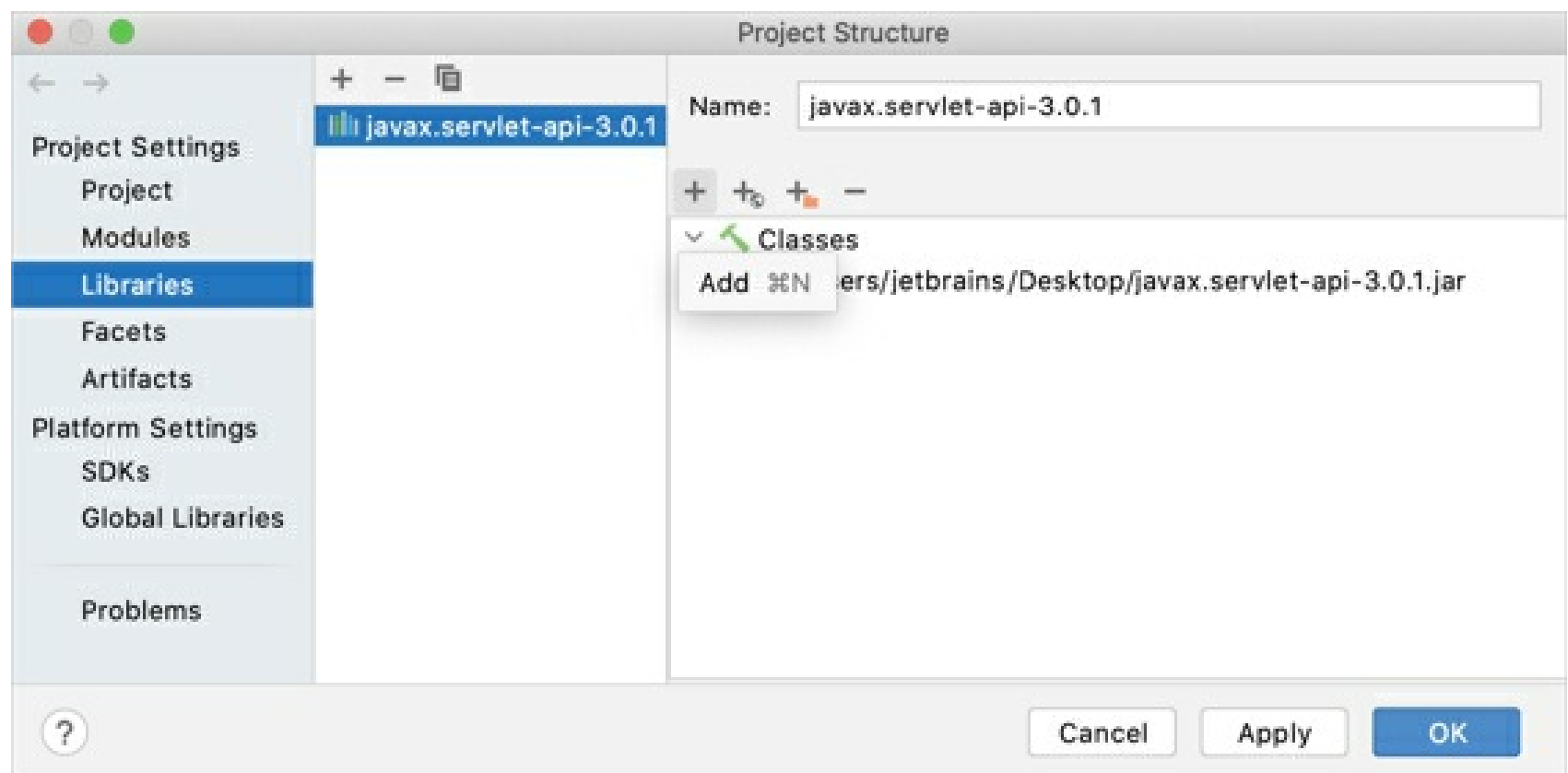
3. Apply the changes and close the dialog.

Add library documentation to your project

You can add downloaded documentation to your project to be able to access it offline.

1. From the main menu, select **File | Project Structure** `Ctrl+Alt+Shift+S` and click **Libraries**.
2. Select the library for which you want to add the documentation and click **+** in the right

section of the dialog.



3. In the dialog that opens, select the file with the documentation and click **Open**.
4. Apply the changes and close the dialog.

When the documentation is configured, you can open it in the editor.

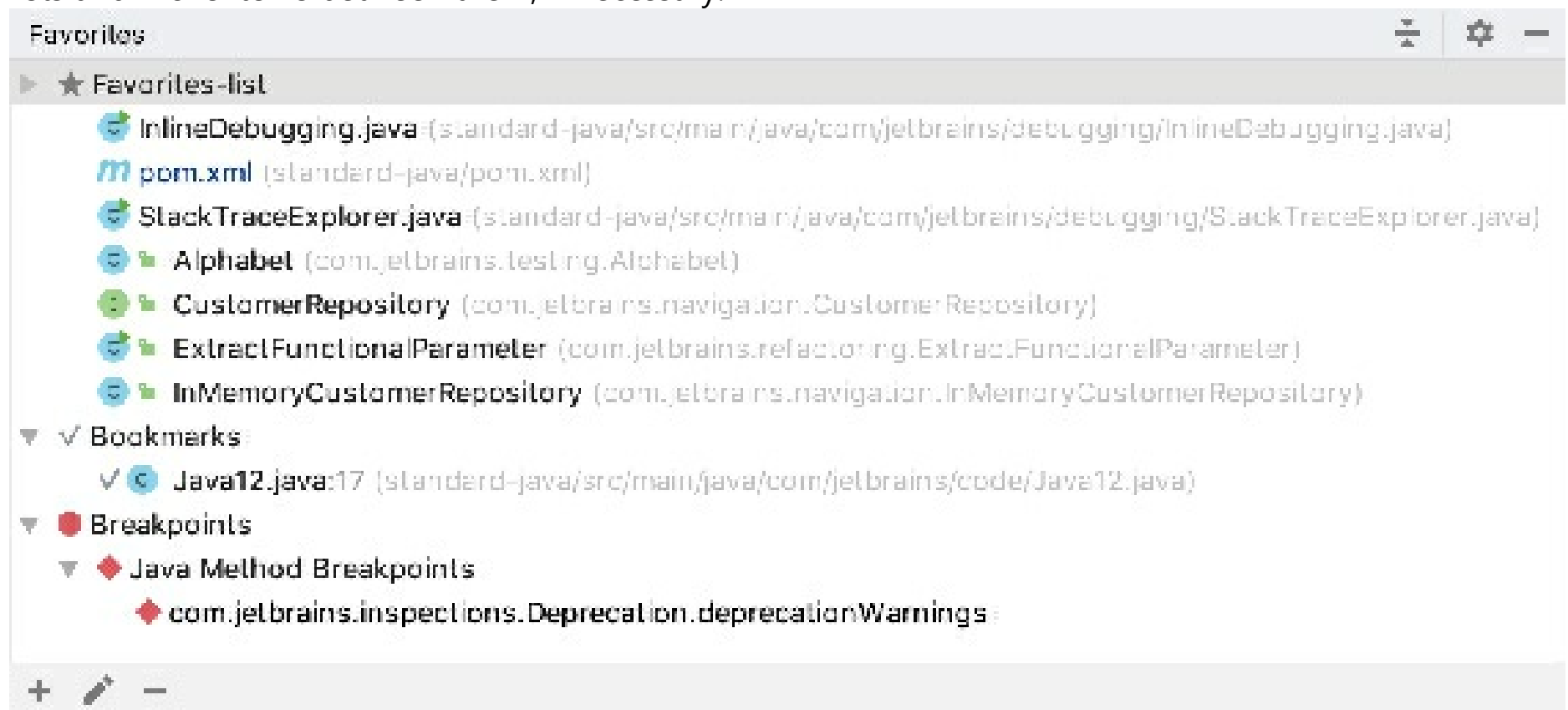
Favorites

When your project comprises thousands of files, browsing them can be tedious. Normally, there's a number of files or folders that you need more often than the rest of the project. To quickly access such files, add them to the **Favorites** list.

The list can include project elements (files, folders, packages, instance and class members), bookmarks, and breakpoints.

IntelliJ IDEA adds breakpoints and bookmarks to the **Favorites** list automatically.

There's always one pre-defined empty list that has the same name as the project. You can create more lists and move items between them, if necessary.



Whenever you need to access your favorite items, press **Alt+2** or choose **View | Tool Windows | Favorites** from the menu.

To open one or more favorite items, select them in the list and press **F4**.

Add one or multiple files

1. Open the file in the editor and press **Alt+Shift+F**.
2. Select the existing list to which you want to add the file, or click **Add to New Favorites List** to add it to the new list.

You can quickly add all files opened in the editor to the list of favorite items. Right-click any tab and select **Add All to Favorites**.

Add folders and packages

1. Right-click the necessary folder or package in the **Project** or **Find** tool window and select **Add to Favorites**.
2. Select the existing list to which you want to add the item, or click **Add to New Favorites List** to add it to the new list.

Add instance and class members

1. Position the caret at the string with the necessary class or instance member and select **File | Add to Favorites** from the main menu.
2. Select the existing list to which you want to add the item, or click **Add to New Favorites List** to add it to the new list.

Add non-project files to the list of favorites

1. From the main menu, select **View | Tool Windows | Favorites** Alt+2 to open the **Favorites** tool window.
2. Drag the file or folder from **Explorer** or **Finder** to the desired list.

Move an item to another list

1. From the main menu, select **View | Tool Windows | Favorites** Alt+2 to open the **Favorites** tool window.
2. Select the necessary items and drag them to another list.

Remove items

1. From the main menu, select **View | Tool Windows | Favorites** Alt+2 to open the **Favorites** tool window.
2. Select the items that you want to remove from favorites.
3. Click on the toolbar or press Delete.

Scopes and file colors

A *scope* is a group of files, packages, and folders in a project. You can use scopes to visually distinguish project items in different IDE views and to limit the range of specific operations.

Scopes are designed to logically organize files in your project: test sources can go to the test-related scope, and production code can be associated with the scope of production files. These logical chunks make your project easier to manage. For example, running test-related inspections only in test classes takes less than if you run them in all files in your application.

IntelliJ IDEA comes with a set of predefined scopes, but you can also create custom scopes. There, you can include any files and folders. For example, a custom scope can include only those files in the project for which you are responsible.

In IntelliJ IDEA, scopes are used in code inspections, some refactorings, search, in copyright settings, in various features for code analysis, and so on.

There are 2 types of scopes: **local** and **shared**.

- **Local scopes** are stored in the IDE configuration directory, that is why they are not shared through VCS and are not available to other members of your team.
- **Shared scopes** are added to a VCS so that people who work on a project can use the same scopes. These scopes are stored together with the project in the **scopes** folder under **.idea**. Each scope is saved as a file with the **.xml** extension (for example: **MyProject/.idea/scopes/shared-scope.xml**).

Using shared scopes makes sense if your project is under version control. If you don't use a VCS, local scopes will be sufficient to cover your needs.

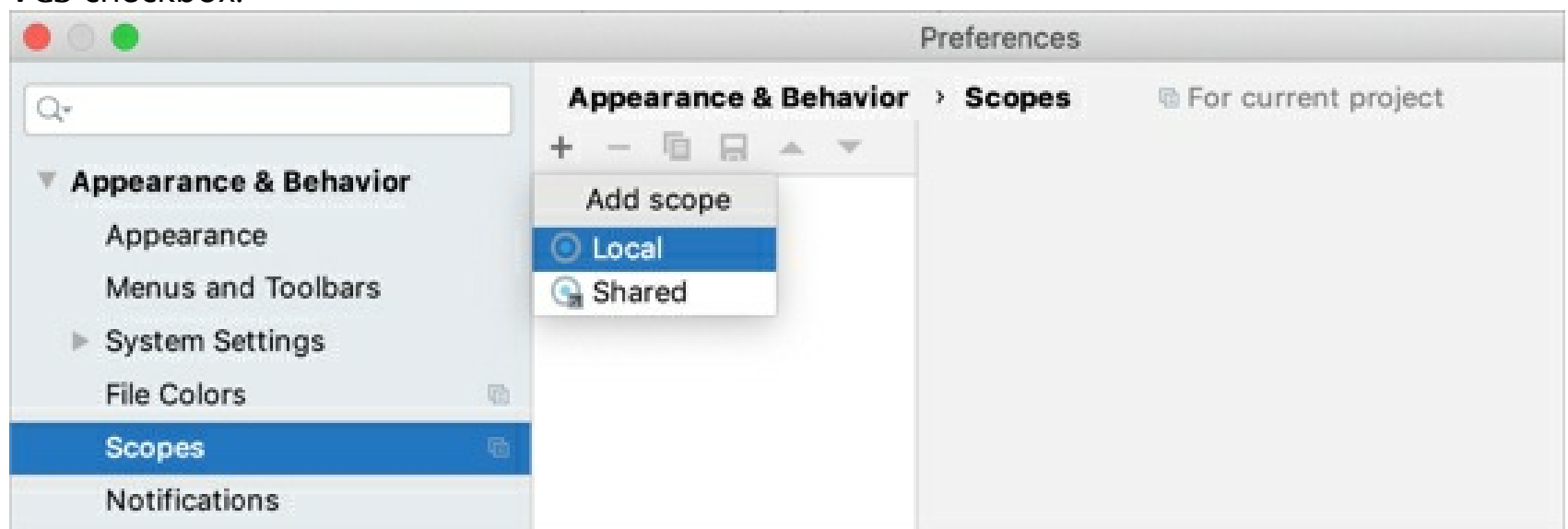
This page describes how to configure scopes and file colors for a single project. If you want to configure these settings for the current project and for all newly created projects (globally), go to **File | New Projects Settings | Settings/Preferences for New Projects**.

Define a new scope

In IntelliJ IDEA, there's a set of predefined scopes, but you can also define your own scopes.

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | Scopes**.
2. Click  and select what kind of scope you want to define: local or shared.

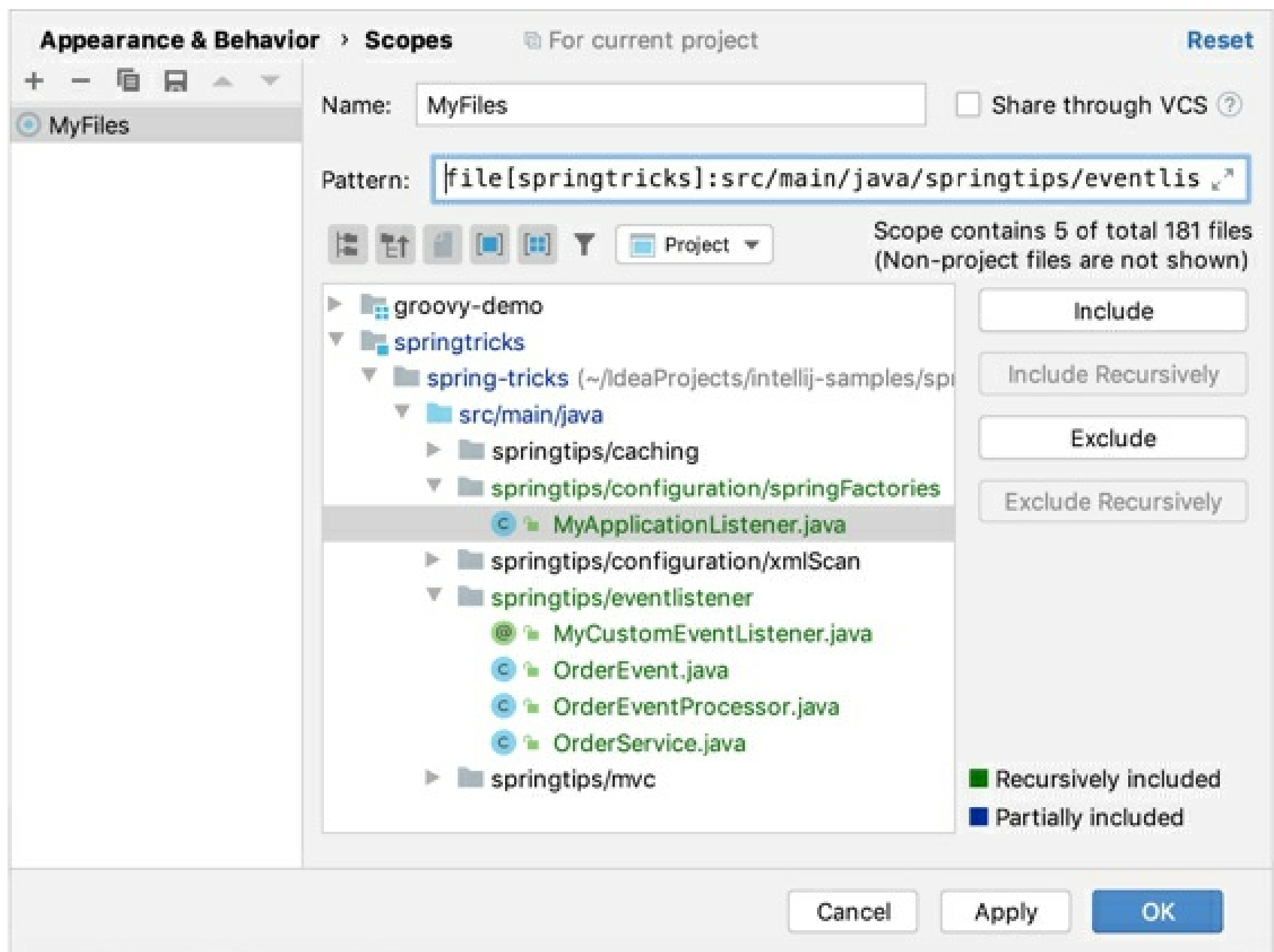
You can change the state of the selected scope (local or shared) later using the **Share through VCS** checkbox.



3. In the dialog that opens, name the new scope and click **OK**.
4. Add files to the new scope. Select the necessary items in the project tree and click one of the options located on the right from the tree:
 - **Include:** include the selected items. If you are including a folder, this action adds only the files located inside this folder. All nested subfolders and their contents

will not be included.

- **Include Recursively**: include the selected folder together with the nested subfolders and their contents.
- **Exclude**: exclude the selected items from the scope. If you are excluding a folder, this action removes only the files located inside this folder. All nested subfolders and their contents will remain in the scope.
- **Exclude Recursively**: exclude the selected folder together with the nested subfolders and their contents.



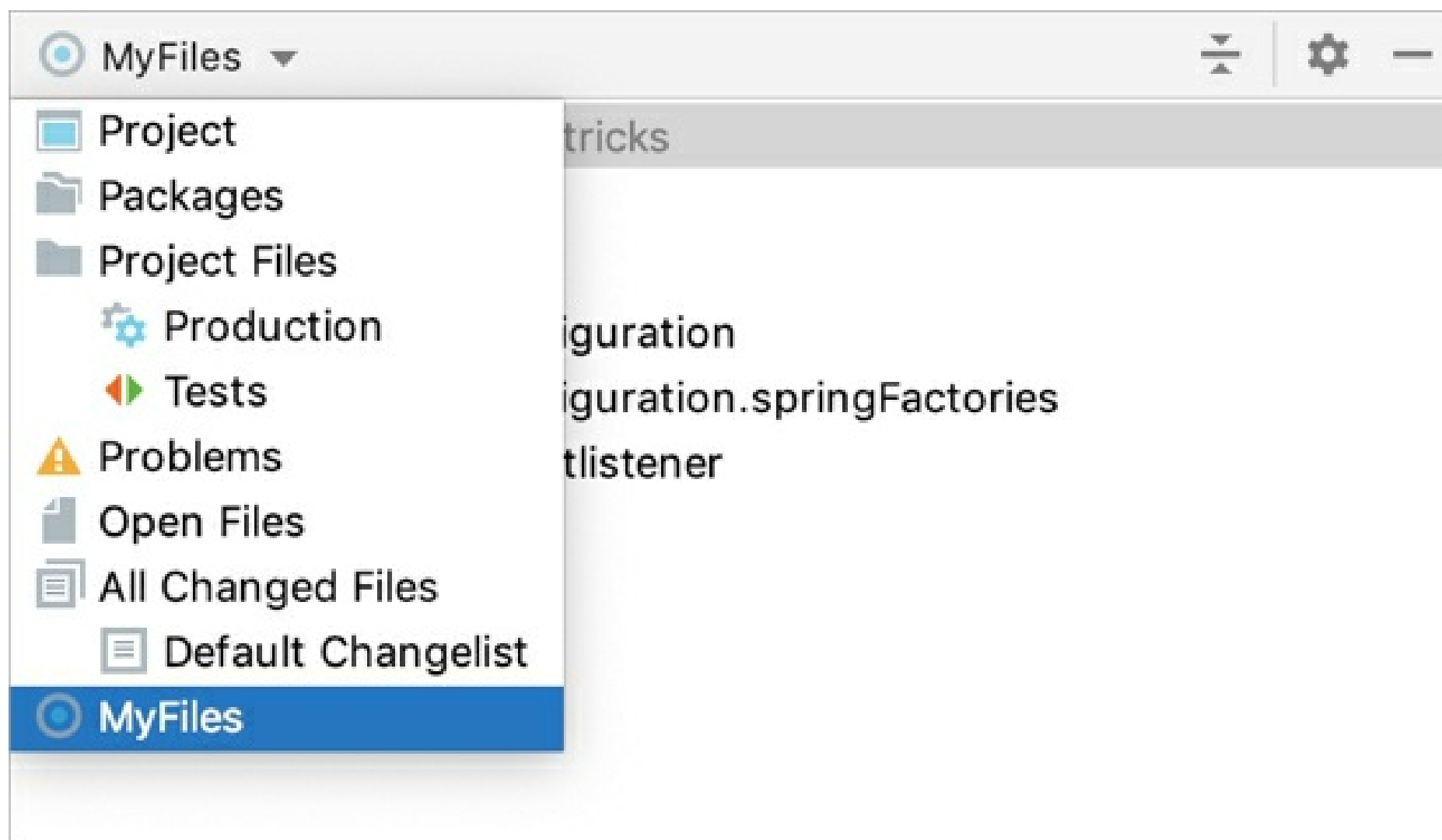
As you add files to the scope, IntelliJ IDEA creates an expression and displays it in the **Pattern** field. Instead of using the buttons, you can also type a pattern in the **Pattern** field manually using the scope language syntax reference.

5. Apply the changes and close the dialog.

When you add items to the scope, their names change the color accordingly:

- ■ Green: folders and files included in the scope.
- ■ Blue: folders that contain both excluded and included files and folders.
- ■ Black: files and folders whose names are written in black are excluded from the scope.

After you create a custom scope, you can find it in the Project tool window and in all dialogs that allow you to limit the number of files to which you want to apply an action.



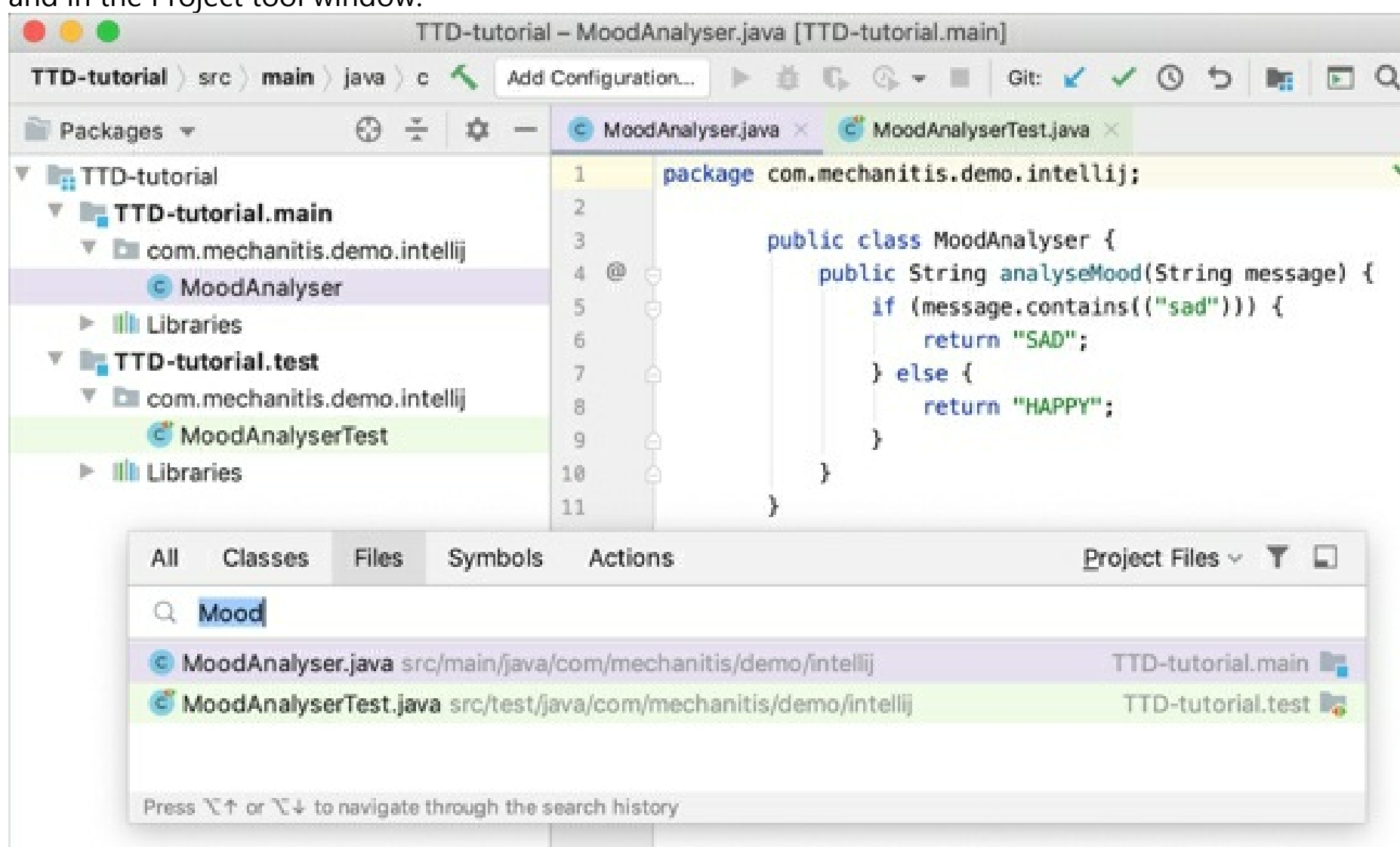
Predefined scopes

IntelliJ IDEA provides a set of predefined scopes. The IDE adds files to these scopes automatically based on the information about them. Note that these scopes cannot be modified.

List of predefined scopes

Associate scopes with colors

Files that belong to different scopes can be highlighted in different colors in search results, in editor tabs, and in the Project tool window.



To each scope, you can assign its own color. For example, you can assign a color to the **Open Files** scope and configure the IDE to show this color in the **Project** tool window. In this case, the files that you are

currently working with in the editor will be colored in the project tree. This makes project navigation faster and simpler. Note that file colors work only in association with scopes. Similarly to scopes, color associations can be **local** and **shared**.

- **Local colors** are only visible to you and are not shared through VCS.
- **Shared colors** are placed under version control so that people who work on a project can use the same color associations. They are stored in the project folder under **.idea** in the **fileColors.xml** file (for example: **MyProject/.idea/fileColors.xml**).

Create a new color association

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | File Colors**.
2. Make sure that the **Enable File Colors** checkbox is selected, and then choose where you want to use the colors: select **Use in Editor Tabs** or **Use in Project View**.

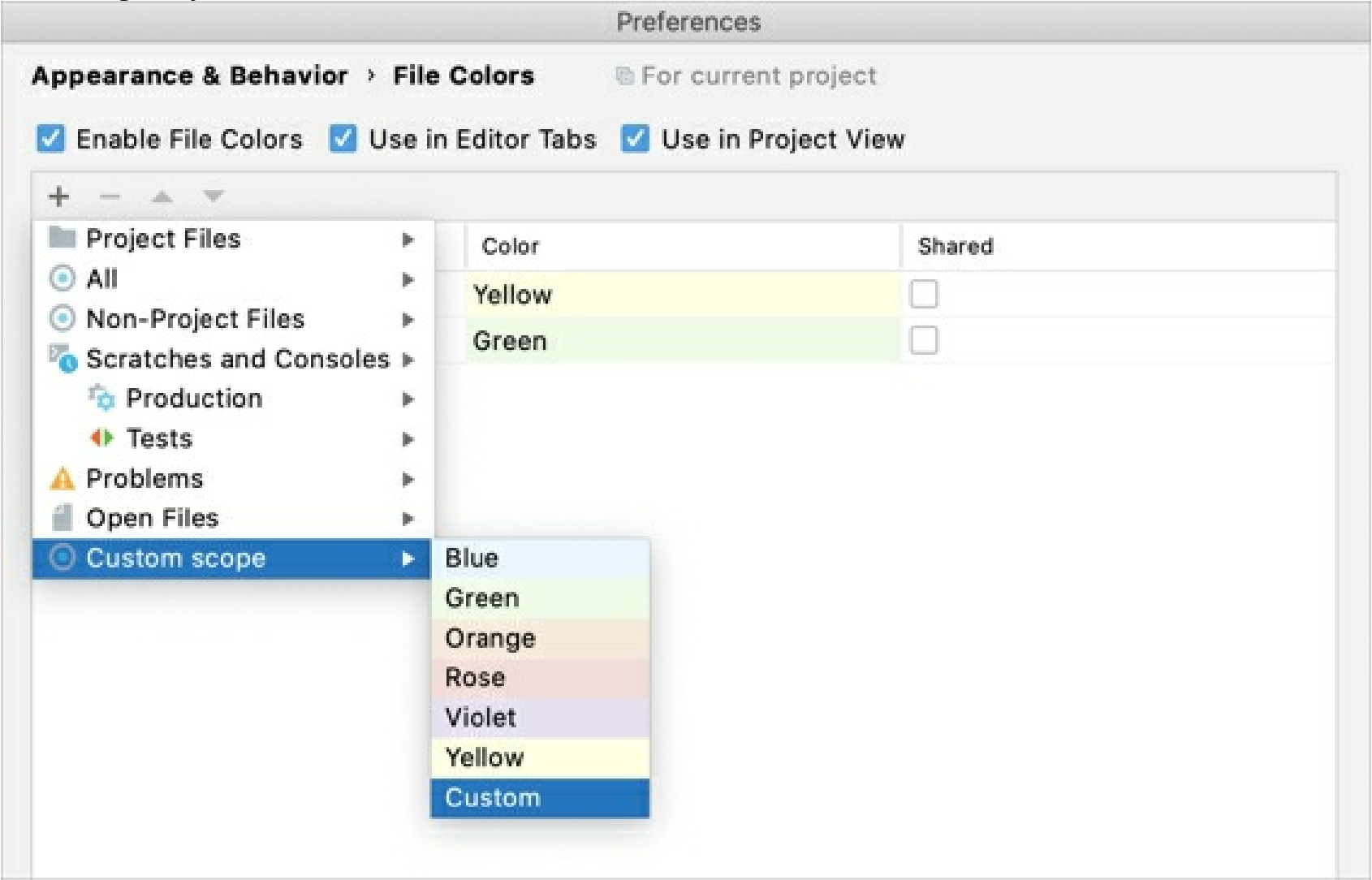
If you select the **Use in Project View** checkbox, you will see colors in the Project tool window and in search results (for example, in the **Find in Files** dialog **Ctrl+Shift+F**).

3. Click  and select the scope for which you want to configure a color.

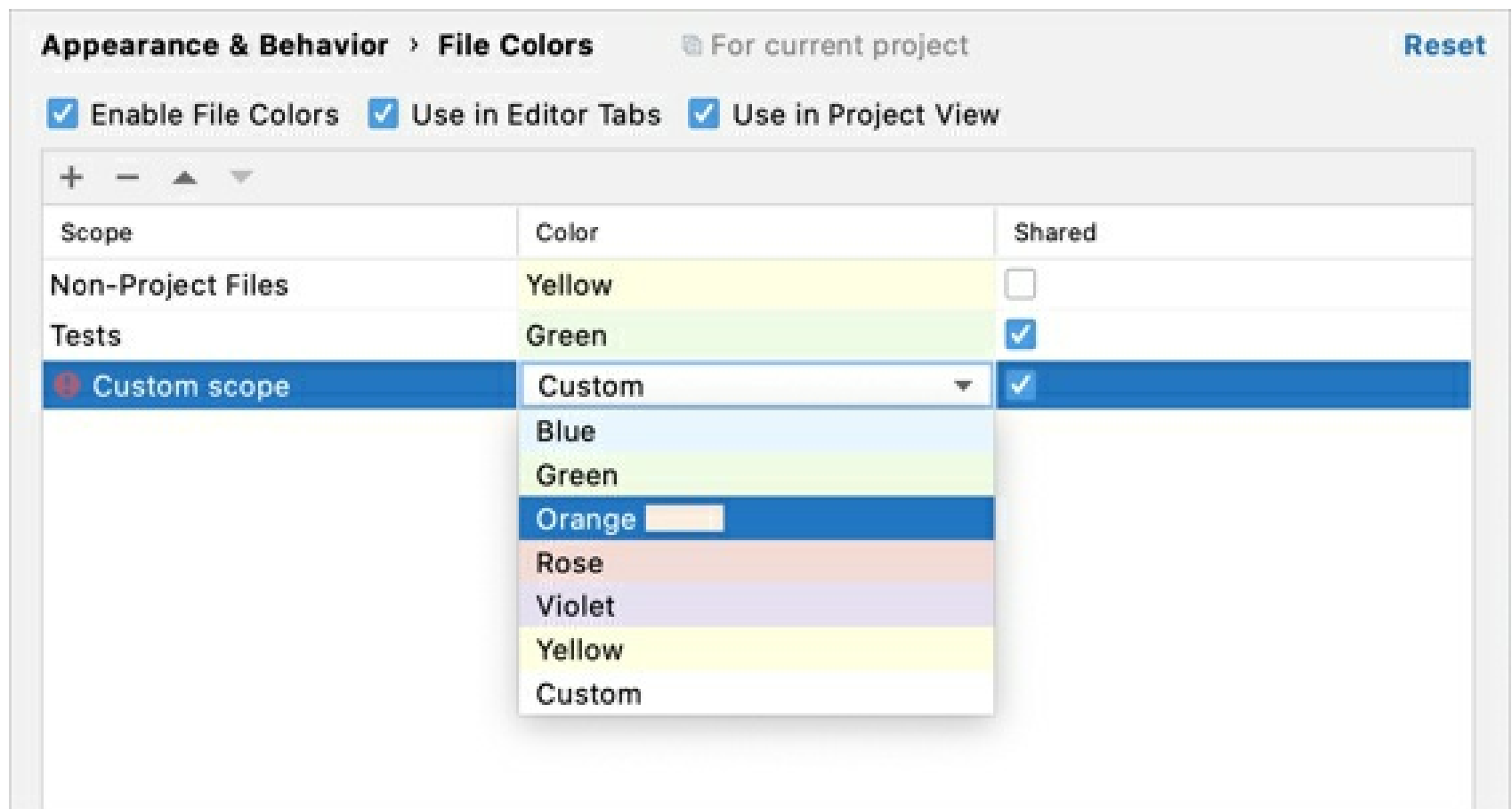
You can select one of the pre-defined scopes or use a custom scope.

4. Click the arrow  next to the necessary scope and select a color from the list that opens.

To configure your own color, click **Custom**.



5. To edit a color, click the cell that corresponds to the necessary scope in the **Color** column and select a new color from the list.
6. To share a color through a VCS, select the checkbox on the corresponding line in the **Shared** column. If the checkbox is cleared, the color will be used locally.



7. Apply changes and close the dialog.

If a file is included in several scopes, the order of the scopes becomes important: IntelliJ IDEA processes the scopes from the top to the bottom starting from local scopes. It means that the IDE will apply the color of the last scope in the list to such a file.

You can change the order of the scopes if you want IntelliJ IDEA to process color associations in a different order.

Change the order of scopes

1. Press **Ctrl+Alt+S** to open IDE settings and select **Appearance & Behavior | Scopes**.
2. Select the scope that you want to move and click **(Alt+Up)** or **(Alt+Down)**.
3. Apply the changes and close the dialog.

Scope language syntax reference

You can use the *scopes language* to specify project scopes: sets of files, directories, and subdirectories.

Sets of classes

- Single class is defined by a class name, for example: `com.intellij.openapi.MyClass`
- Set of all classes in a package, not recusing into subpackages, is defined by an asterisk after dot, for example: `com.intellij.openapi.*`
- Set of all classes in a package including contents of subpackages, is defined by an asterisk after double dot, for example `com.intellij.openapi..*`

Sets of files

- To add a single file, use a filename (for example, `MyDir/MyFile.txt`)
- To add all the files in a directory without subdirectories, use an asterisk after a slash (for example: `file:src/main/myDir/*`)
- To add all the files in a directory with subdirectories, use an asterisk after a double slash (for example, `file:src/main/myDir/**`)

Modifiers

Location modifiers

Location modifiers help you specify location of the necessary file:

- `src:` – for source files
- `lib:` – for library classes
- `test:` – for test code

For example, the `src:com.intellij.openapi.*` pattern places in a scope all classes under the source root in the `com.intellij.openapi` package, excluding subpackages. The default location is the module root.

Module modifiers

Module modifiers help you narrow down the scope by specifying the name of the related module:

- `src[module name]:<E>`
- `lib[module name]:<E>`
- `test[module name]:<E>`

For example, the `src[MyModule]:com.intellij.openapi.*` pattern places in a scope all classes under the source folders related to the `MyModule` module in the `com.intellij.openapi` package, excluding subpackages.

Logical operators

When you define scopes, you can use logical operators:

`&&` for AND

`||` for OR

`!` for NOT

Copied!

Also, you can use parentheses to join logical operators into groups. For example, the following scope includes either `<a>` and `<c>`, or `<b>` and `<c>`:

`(<a>||<b>)&&<c>`

Copied!

Another example

`file[*web*]:src/main/java/**`

Copied!

denotes a scope of all modules whose name contains `web`, and all the files recursively in the directory **src/main/java**.

Create a new scope from existing scopes

You can make a new scope from several existing scopes. In this case, you can reference the existing scopes by using `$ $MyScope`.

For example, the `$Scope1||$Scope2` pattern places in a scope all files from `scope1` and `scope2`.

Examples

- `file[MyMod]:src/main/java/com/example/my_package/**`- include in a project all the files from module "MyMod", located in the specified directory and all subdirectories.
 - `src[MyMod]:com.example.my_package.*`- recursively include all classes in a package in the source directories of the module.
 - `lib:com.company.*||com.company.*`- recursively include all classes in a package from both project and libraries.
 - `test:com.company.*`- include all test classes in a package, but not in subpackages.
 - `[MyMod]:com.company.util.*`- include all classes and test classes in the package of the specified module.
 - `file:*.js||file:*.coffee`- include all JavaScript and CoffeeScript files.
 - `file:*js&&!file:*.min.*`- include all JavaScript files except those that were generated through *minification*, which is indicated by the `min` extension.
 - `!file:*/.npm/**`- exclude all **.npm** folders.
-
- Scope language syntax reference
 - Sets of classes
 - Sets of files
 - Modifiers
 - Logical operators
 - Create a new scope from existing scopes
 - Examples

Invalidate caches

IntelliJ IDEA caches a great number of files for all projects that you have even worked with in this IDE version, therefore the system cache may become overloaded. Sometimes the caches will never be needed again, for example, if you work with frequent short-term projects.

When you invalidate the cache, IntelliJ IDEA removes the cache files for all project even run in the current version of the IDE. The files will be recreated the next time you open these projects. The IDE also rebuilds the projects if they are built with the native IntelliJ IDEA builder.

Note the following before you proceed:

- The caches will not be deleted until you restart IntelliJ IDEA.
- Opening and closing a project without invalidating the cache does not result in deleting any files.
- Local History is not deleted when you invalidate the cache unless you explicitly enable this option in the **Invalidate Caches** dialog. However, mind that Local History has a retention period of 5 working days by default.

Clear the system cache

1. From the main menu, select **File | Invalidate Caches**.
2. In the **Invalidate Caches** dialog, you can select additional actions that the IDE will perform while removing the cache files:

- **Clear file system cache and Local History:** remove the virtual file system cache together with the information stored in Local History.

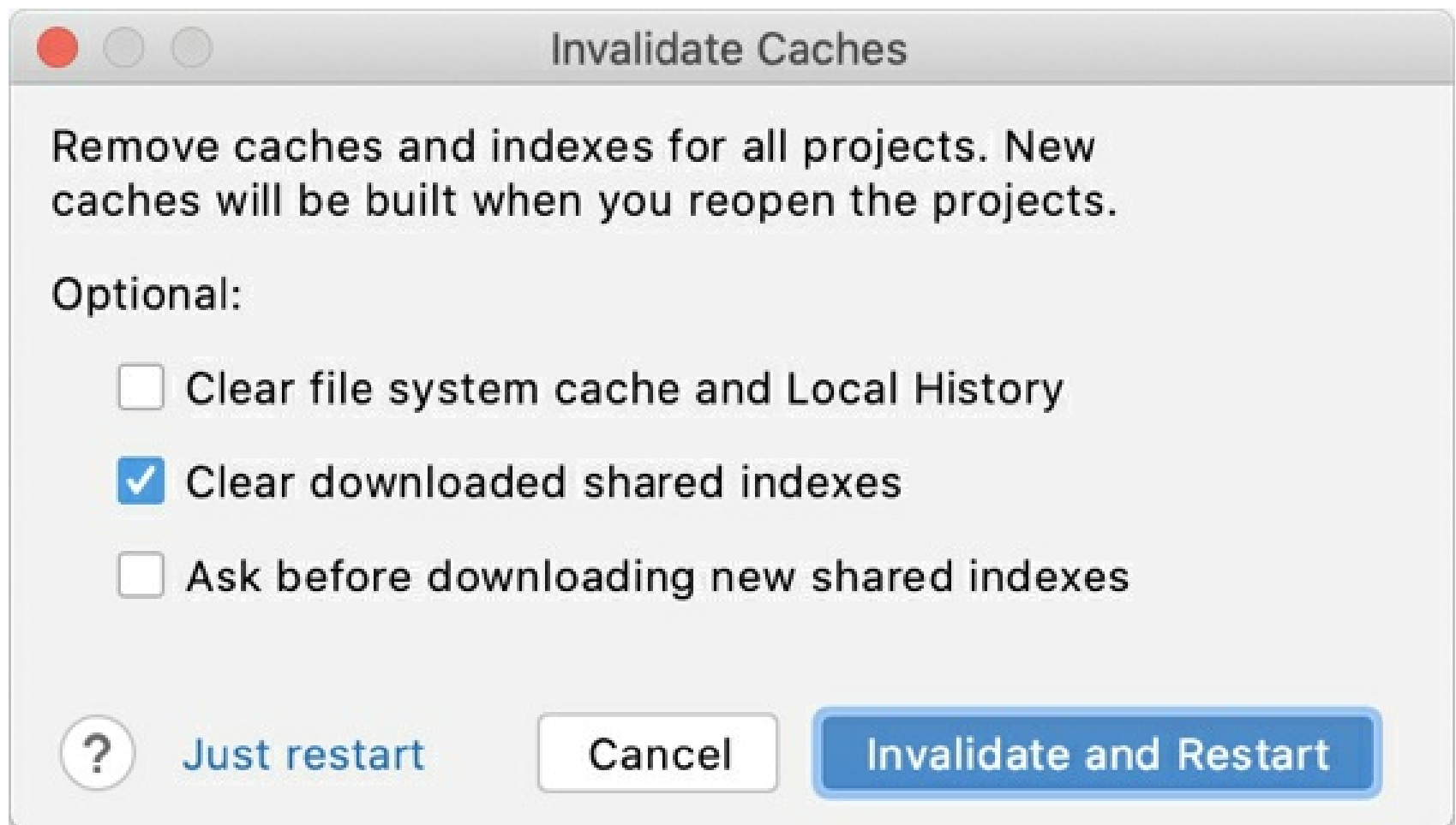
This action might be helpful for troubleshooting purposes when the usual cache invalidation is not enough to solve the problem.

- **Ask before downloading new shared indexes:** show a notification prompting you to download new shared indexes as they become available.

Enabling this option also updates your settings for shared project indexes in **Settings/ Preferences | Tools | Shared Indexes**.

- **Clear downloaded shared indexes:** remove the downloaded shared index files.

3. Click **Invalidate and Restart**.



If you click **Just restart**, cache files won't be deleted, and the selected optional actions won't be applied.

We recommend that you restart the IDE via **Find Action**: press `Ctrl+Shift+A` and type `Restart IDE`.

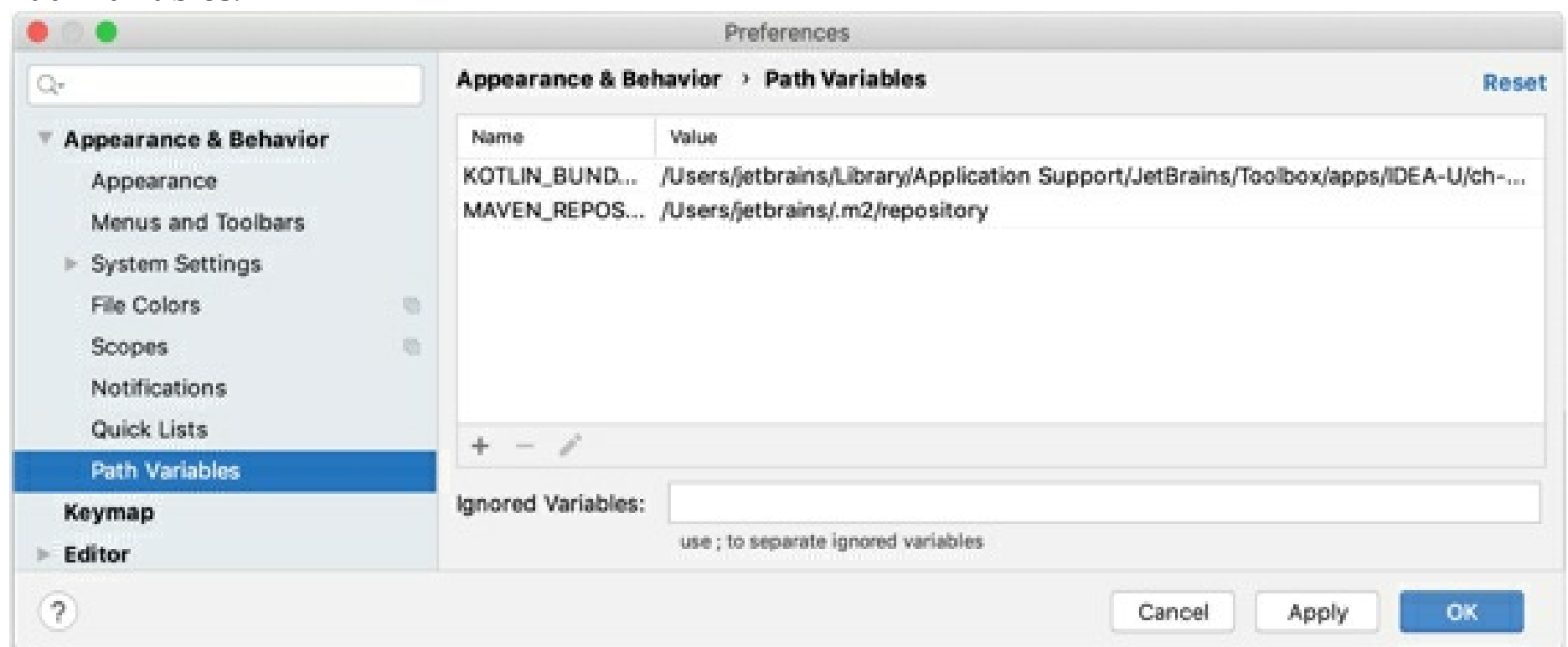
Path variables

Path variables are placeholders that stand for absolute paths to resources that are linked to your project but are stored outside it. If you are working in a team, these absolute paths on the computers of your teammates may differ. With path variables, you can flexibly share your code so all the references to the linked resources are resolved properly as the path variables accept the values according to the configuration on each specific computer.

In IntelliJ IDEA, there are some pre-defined variables:

- `$USER_HOME$`: stands for your home directory.
- `$PROJECT_DIR$`: stands for the directory where your project is stored.
- `$MODULE_DIR$`: stands for the directory that keeps module configuration files **IML**.

To configure path variables, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, select **Appearance & Behavior | Path Variables**.



Create a new path variable

For example, you have a third-party library that is not stored in your project directory. If you want to make sure that the path is correct on your teammates' computers after they update the project from VCS, you can create a new variable.

1. Press `Ctrl+Alt+S` to open IDE settings and select **Appearance & Behavior | Path Variables**.
2. Click **+** and enter the name of the new variable (for example, `PATH_TO_LIB`) and its value that points to the library location on your disk.
3. Share the **IML** file through your version control system.
4. After your teammates update their projects from VCS, they will change the `PATH_TO_LIB` variable value so that it points to the library location on their computers.

Ignored path variables

When you open a project, IntelliJ IDEA checks if there are any unresolved path variables. If the IDE detects any, it will ask you to define values for them. If for some reason you don't want to do that (for example, if you are not going to use files or directories with the unresolved path variables), you can add them to the list of ignored variables.

You can also use the list of ignored variables if, for example, a program argument passed to the JVM in a run/debug configuration has the same format as an internal (`$SOME_STRING$`) path variable. In this case, you can add this parameter to the list of ignored variables in order to avoid confusion. Enter `SOME_STRING` to the **Ignored Variables** field in the **Path Variables** dialog.

Resource files

Resources include properties files, images, DTDs, and XML files. These files are located in the classpath of your application, and are usually loaded from the classpath using the following methods:

- `ResourceBundle.getBundle()` for properties files and resource bundles
- `getResourceAsStream()` for icons and other files

For more information about these methods, refer to [Location-Independent Access to Resources](#).

When building an application, IntelliJ IDEA copies all resources into the output directory, preserving the directory structure of the resources relative to the source path. The following file types are recognized as resources by default:

- **.dtd**
- **.jpeg**
- **.properties**
- **.gif**
- **.jpg**
- **.tld**
- **.html**
- **.png**
- **.xml**

The pattern of recognized resource files can be configured as a regular expression in the **Compiler** dialog (**Settings/Preferences** `Ctrl+Alt+S` | **Build, Execution, Deployment** | **Compiler**). Using the **Resource Patterns** field, you can add your own file extensions and create custom list of resources.

Write and edit source code

When you work with code, IntelliJ IDEA ensures that your work is stress-free. It offers various shortcuts and features to help you add, select, copy, move, edit, fold, find occurrences, and save code.

For navigation inside the editor, refer to Editor basics.

Find action

- If you do not remember a shortcut for the action you want to use, press `Ctrl+Shift+A` to find any action by name.



You can use the same dialog to find classes, files, or symbols. For more information, refer to Searching Everywhere.

Add a new class, file, package, or scratch file

- In the editor, press `Ctrl+Alt+Insert` to add a class, file, or package.

If the focus is inside the **Project** tool window and you want to add a new element, press `Alt+Insert`.

- To create a new *Scratch* file, press `Ctrl+Alt+Shift+Insert`.

IntelliJ IDEA creates a temporary file that you can run and debug. For more information, refer to Scratch files.

Toggle read-only attribute of a file

If a file is read-only, it is marked with the closed lock icon in the status bar, in its editor tab, or in the Project tool window. If a file is writable, it is marked with the open lock icon in the Status bar.

1. Open file in the editor or select it in the Project tool window.
2. Do one of the following:
 - From the main menu, select **File | File Properties | Make File Read-only** or **File | File Properties | Make File Writable**.
 - Click the lock icon in the status bar.

If a read-only status is set by a version control system, it's suggested that you use IntelliJ IDEA version control integration features. For more information, see Version control.

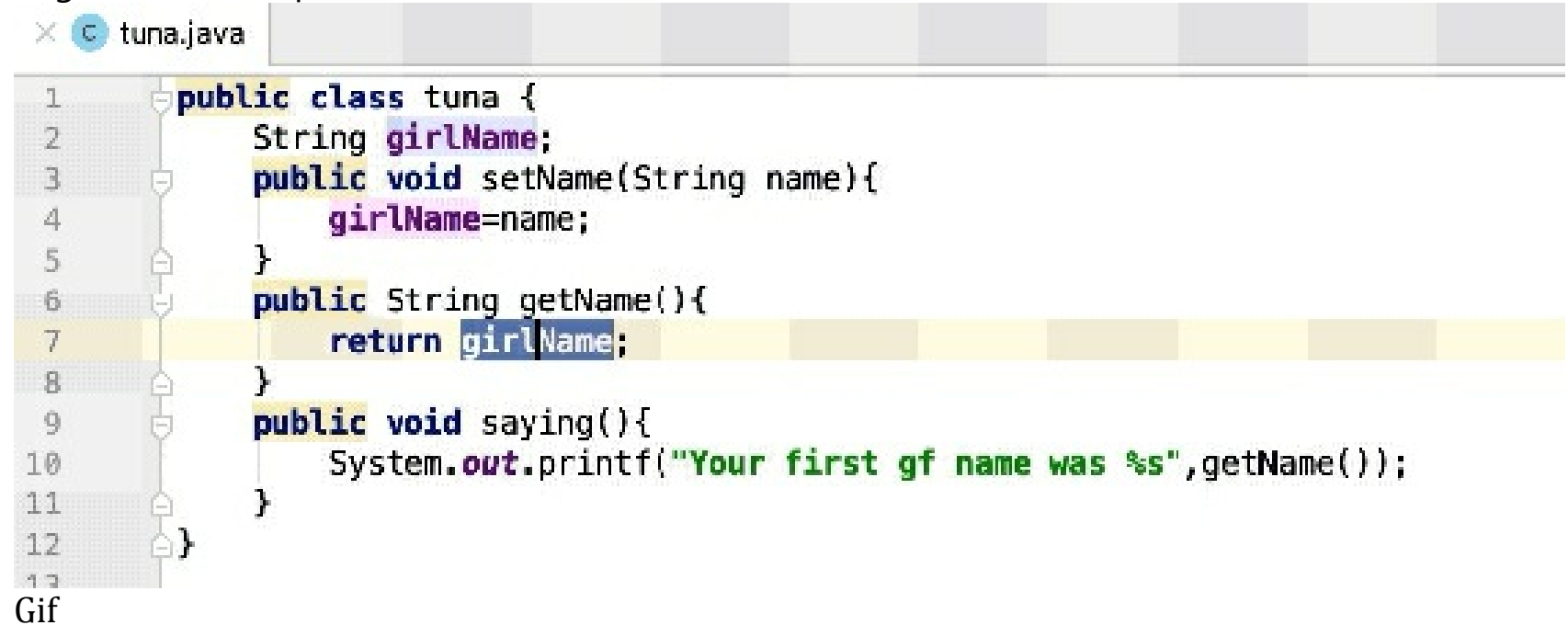
Select code constructs

- In the editor, place the caret at the item you want to select and press `Ctrl+W` / `Ctrl+Shift+W` to extend or shrink your selection.

For example, in a plain text file, the selection starts within the whole word then extends to the sentence, paragraph, and so on.

In a Java file, if you start by selecting an argument in a method call, it will extend to all

arguments, then to the whole method, then to the expression containing this method, then to a larger block of expressions, and so on.



- If you need just to highlight your braces, place the caret immediately after the block closing brace/bracket or before the block opening brace/bracket.

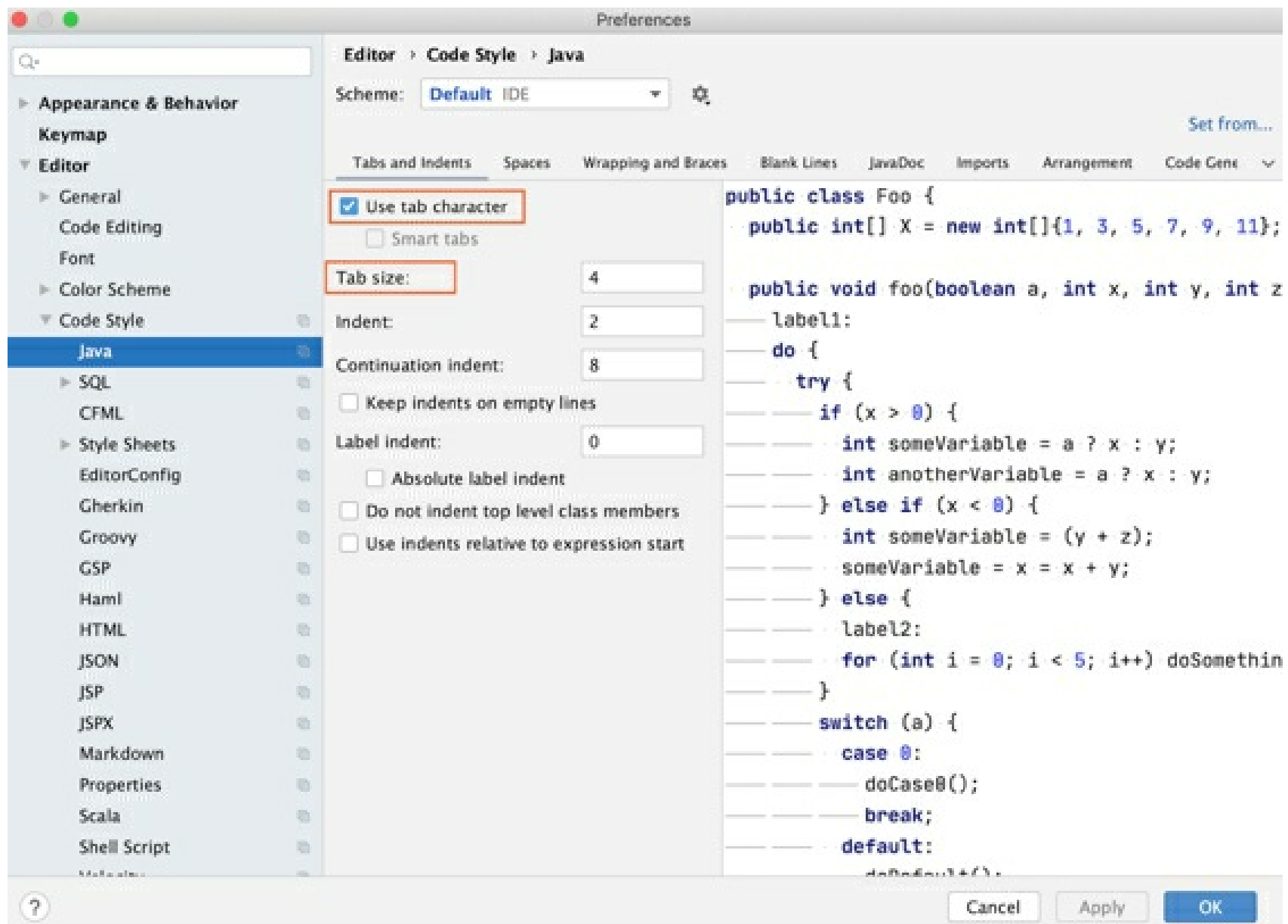
Select code according to capitalization

1. In the **Settings/Preferences** dialog **Ctrl+Alt+S**, go to **Editor | General | Smart Keys**.
2. Select the **Use "CamelHumps" words** checkbox.

If you want to use double-click when selecting according to capitalization, make sure that the **Honor CamelHumps words...** checkbox is selected on the **Editor | General** page of the **Settings/Preferences** dialog **Ctrl+Alt+S**.

Configure tabs and indents

1. In the **Settings/Preferences** dialog **Ctrl+Alt+S**, go to **Editor | Code Style**.
2. Select a language for which you want to configure the indentation.
3. From the options on the right, on the **Tabs and Indents**, select the **Use tab character** for the editor to use tabs when you press **Tab**, indent, or reformat code. You can also configure the tab size if you need. If you don't select this option, IntelliJ IDEA will use spaces.



Copy and paste code

You can use the standard shortcuts to copy `Ctrl+C` and paste `Ctrl+V` any selected code fragment. If nothing is selected, IntelliJ IDEA automatically copies as is the whole line where the caret is located.

By default, when you paste anything in the editor, IntelliJ IDEA performs "smart" paste, for example, pasting multiple lines in comments will automatically add the appropriate markers to the lines you are pasting. If you need to paste just plain text, press `Ctrl+Alt+Shift+V`.

- Place the caret at a line or a symbol, right-click to open the context menu, select **Copy/Paste Special | Copy Reference**. When you select the **Copy Reference** (`Ctrl+Alt+Shift+C`) option, IntelliJ IDEA creates a reference string that includes the line number of the selected line or symbol. You can press `Ctrl+V` to paste the copied reference anywhere.
- IntelliJ IDEA keeps track of everything you copy to the clipboard. To paste from history, in the editor, from the context menu, select **Copy/Paste Special | Paste from History** (`Ctrl+Shift+V`). In the dialog that opens, select your entry and click **Paste**.

The default number of items stored in the clipboard history is 100.

- When you copy and paste code to the editor, IntelliJ IDEA displays the hidden (special) characters represented by their Unicode name abbreviation.

```
#db.default.url="jdbc:mysql://ZWSPlocalhost/play-fullcalendar"
db.default.url="jdbc:mysql://localhost/play-fullcalendar"
```

Lines of code

IntelliJ IDEA offers several useful shortcuts for manipulating code lines.

If you need to undo or redo your changes, press `Ctrl+Z`/ `Ctrl+Shift+Z` respectively.

- To add a line after the current one, press `Shift+Enter`. IntelliJ IDEA moves the caret to the next

line.

- To add a line before the current one, press `Ctrl+Alt+Enter`. IntelliJ IDEA moves the caret to the previous line.
- To duplicate a line, press `Ctrl+D`.
- To sort lines alphabetically in the whole file or in a code selection, from the main menu, select **Edit | Sort Lines** or **Edit | Reverse Lines**. These actions might be helpful when you work with property files, data sets, text files, log files, and so on. If you need to assign shortcuts to those actions, refer to [Configure keyboard shortcuts](#) for more information.
- To delete a line, place the caret at the line you need and press `Ctrl+Y`.

Note that when you install IntelliJ IDEA with Windows default keymap for the first time, a dialog appears offering you to map this shortcut to either the **Redo** or **Delete Line** action.

To adjust your keymap after the installation, refer to [Choose the right keymap](#).

- To join lines, place the caret at the line to which you want to join the other lines and press `Ctrl+Shift+J`. Keep pressing the keys until all the needed elements are joined.

You can also join string literals, a field or variable declaration, and a statement. Note that IntelliJ IDEA checks the code style settings and eliminates unwanted spaces and redundant characters.

- To split string literals into two parts, press `Enter`.

IntelliJ IDEA splits the string and provides the correct syntax. You can also use the **Break string on '\n'** intention to split string literals. Press `Alt+Enter` or click  to select this intention.

- To comment a line of code, place the caret at the appropriate line and press `Ctrl+/.`
- To move a line up or down, press `Alt+Shift+Up` or `Alt+Shift+Down` respectively.
- To move (swap) a code element to the left or to the right, place the caret at it, or select it and press `Ctrl+Alt+Shift+Left` for left or `Ctrl+Alt+Shift+Right` for right.

For example, for Java you can use these actions for method invocation or method declaration arguments, enum constants, array initializer expressions. For XML or HTML, use these actions for tag attributes.



Gif

Code statements

Move statements

- In the editor, place the caret at the needed statement and press `Ctrl+Shift+Up` to move a statement up or `Ctrl+Shift+Down` to move a statement down. IntelliJ IDEA moves the selected statement performing a syntax check.

If moving of the statement is not allowed in the current context, the actions will be disabled.

Complete current statement

- In the editor, press `Ctrl+Shift+Enter` or from the main menu select **Code | Complete Current Statement**. IntelliJ IDEA inserts the required trailing comma automatically in structs, slices,

and other composite literals. The caret is moved to the position where you can start typing the next statement.

Unwrap or remove statement

1. Place the caret at the expression you want to remove or unwrap.
2. Press Ctrl+Shift+Delete.

IntelliJ IDEA shows a popup with all actions available in the current context. To make it easier to distinguish between statements to be extracted and statements to be removed, IntelliJ IDEA uses different background colors.

```
public class IfElseDemo {  
    public static void main (String[] args){  
        int testscore = 76;  
        String grade;  
  
        if (testscore >= 90) {  
            grade = "A";  
        }  
        else if (testscore >= 80) {  
            grade = "B";  
        }  
        else if (testscore <= 70){  
            grade = "C";  
        }  
        else if (testscore >= 60){  
            grade = "D";  
        }  
        else {  
            grade = "F";  
        }  
  
        System.out.println("Grade = " + grade);  
    }  
}
```

Choose the statement to unwrap/remove
Unwrap 'if...'

3. Select an action and press Enter.

```

public class IfElseDemo {
    public static void main (String[] args){
        int testscore = 76;
        String grade;

        grade = "A";
        System.out.println("Grade = " + grade);
    }
}

```

Code fragments

- Move and copy code fragments by dragging them in the editor.
 - To move a code fragment, select it and drag the selection to the target location.
 - To copy a code selection, keeping `Ctrl` pressed, drag it to the target location.

The copy action might not be available in macOS since it can conflict with global OS shortcuts.

- The drag functionality is enabled by default. To disable it, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General** and clear the **Enable Drag'n'Drop functionality in editor** checkbox in the **Mouse** section.
- To toggle between the upper and lower case for the selected code fragment, press `Ctrl+Shift+U`.

Note that when you apply the toggle case action to the *CamelCase* name format, IntelliJ IDEA converts the name to the lower case.

- To comment or uncomment a code fragment, select it and press `Ctrl+Shift+/.`

To configure settings for where the generated line or block comments should be placed in Java, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Code Style | Java** and on the **Code Generation** tab use options in the **Comment Code** section.

Code folding

Folded code fragments are shown as shaded ellipses (`...`). If a folded code fragment contains errors, IntelliJ IDEA highlights the fragment in red.


```

1  public class tuna {
2
3      String girlName;
4
5      public void setName(String name) {
6          girlName = name;
7      }
8
9      public String getName() {
10         return girlName;
11     }
12
13     public void saying() {
14         System.out.printf("Your girl's name is %s", getName());
15     }
16
17 }

```

Gif

To configure the default code folding behavior, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Code Folding**.

If IntelliJ IDEA changes code in the folded fragment during the code reformatting, the code fragment will be automatically expanded.

Expand or collapse code elements

- To fold or unfold a code fragment, press `Ctrl+NumPad -/ Ctrl+NumPad +`. IntelliJ IDEA folds or unfolds the current code fragment, for example, a single method.
- To collapse or expand all code fragments, press `Ctrl+Shift+NumPad -/ Ctrl+Shift+NumPad +`.

IntelliJ IDEA collapses or expands all fragments within the selection, or, if nothing is selected, all fragments in the current file, for example, all methods in a file.

- To collapse or expand code recursively, press `Ctrl+Alt+NumPad -/ Ctrl+Alt+NumPad +`. IntelliJ IDEA collapses or expands the current fragment and all its subordinate regions within that fragment.
- To fold blocks of code, press `Ctrl+Shift+.`. This action collapses the code fragment between the matched pair of curly braces {}, creates a *custom folding region* for that fragment, and makes it "foldable".
- To collapse or expand doc comments in the current file, in the main menu select **Code | Folding | Expand doc comments/Collapse doc comments**.
- To collapse or expand a custom code fragment, select it and press `Ctrl+.`

You can fold or unfold any manually selected regions in code.

Fold or unfold nested fragments

- To expand the current fragment and all the nested fragments, press `Ctrl+NumPad *, 1`. You can expand the current fragment up to the specified nesting level (from 1 to 5).
- To expand all the collapsed fragments in the file, press `Ctrl+Shift+NumPad *, 1`. You can expand the collapsed fragments up to the specified nesting level (from 1 to 5).

Use the Surround With action

You can collapse or expand code using the **Surround With** action.

1. In the editor, select a code fragment and press `Ctrl+Alt+T`.
2. From the popup menu, select **<editor-fold...> Comments** or **region...endregion**

Comments.

3. Optionally, specify a description under which the collapsed fragment will be hidden.
4. To collapse or expand the created region, press `Ctrl+.`
5. To navigate to the created custom region, press `Ctrl+Alt+.`

Disable code folding outline

You can disable the code folding outline that appears on the gutter.

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Code Folding**.
2. Clear the **Show code folding outline** checkbox.

Autosave

IntelliJ IDEA automatically saves changes that you make in your files. Saving is triggered by various events, such as compiling, running, debugging, performing version control operations, closing a file or a project, or quitting the IDE. Saving files can be also triggered by third-party plugins.

Most of the events that trigger auto-save are predefined and cannot be configured, but you can be sure that changes will not be lost and you can find all of them in local history.

To force saving all unsaved files, press `Ctrl+S` or choose **File | Save All** from the menu.

Configure autosave behavior

1. Press `Ctrl+Alt+S` to open IDE settings and select **Appearance and Behavior | System Settings**.
2. Under **Autosave**, configure the following options:
 - **Save files when switching to a different application**
 - **Save files if the IDE is idle for N seconds**

If you use version control integration, names of all modified files will be marked with a dedicated color on the file tab. But you can also mark unsaved files with an asterisk (*) on the file tab.

Mark files with unsaved changes

1. Press `Ctrl+Alt+S` to open IDE settings and select **Editor | General | Editor Tabs**.
2. Select the **Mark modified (*)** checkbox.

Revert changes

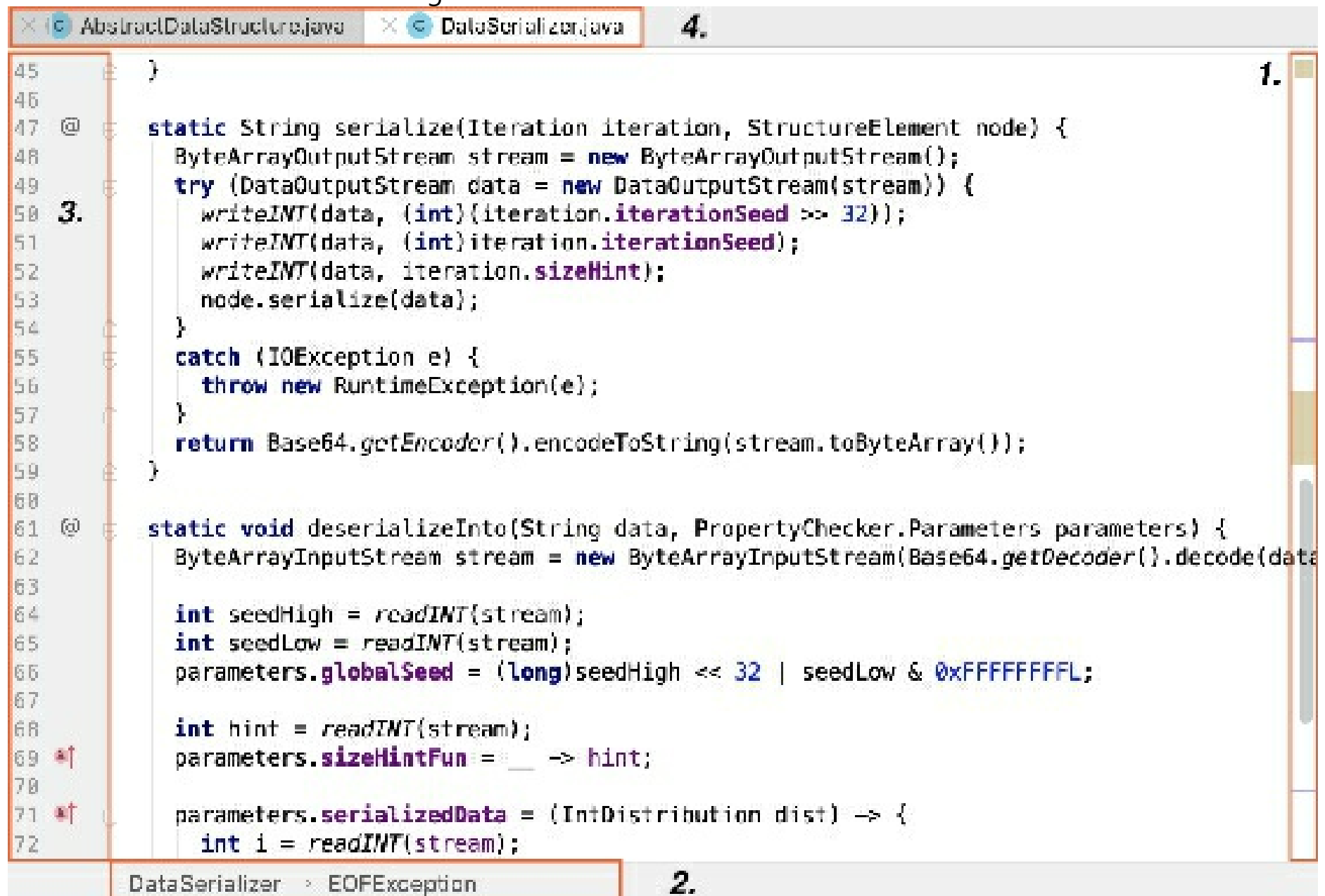
- For most recent changes, including refactorings, press `Ctrl+Z` or choose **Edit | Undo** from the menu.
- For a more detailed view of past changes, use Local History.
- For the most robust tracking of all changes, use a version control system.

Editor basics

The IntelliJ IDEA editor is the main part of the IDE that you use to create, read and modify code.

For information about adding and editing code, refer to [Write and edit source code](#).

The editor consists of the following areas:



1. The scrollbar shows errors and warnings in the current file.
2. Breadcrumbs help you navigate inside the code in the current file.
3. The gutter shows line numbers and annotations.
4. Tabs show the names of the currently opened files.

Navigation

You can use various shortcuts to switch between the editor and different tool windows, change the editor size, switch focus, or return to the original layout.

Maximize editor pane

- In the editor, press `Ctrl+Shift+F12`. IntelliJ IDEA hides all windows except the active editor.

You can maximize a split screen as well. In this case the active screen is maximized and other screens are moved aside.

Switch the focus from a window to the editor

- Press `Escape`. IntelliJ IDEA moves the focus from any window to the active editor.

Return to the editor from the command-line terminal

- Press `Alt+F12`. IntelliJ IDEA closes the terminal window.
- If you need to keep the terminal window open when you switch back to the active editor, press `Ctrl+Tab`.

Return to the default layout

- Press `Shift+F12`.
- To save the current layout as the default, from the main menu select **Window | Store Current Layout as Default**. You can use the same shortcut `Shift+F12` to restore the saved layout.

Jump to the last active window

- Press `F12`.

Use the switcher for navigation

1. To jump between the opened files and tool windows with the switcher, press `Ctrl+Tab`.
2. Keep `Ctrl` pressed to leave the switcher popup open.
3. Press `Tab` to move between elements. Press `Backspace` to remove the selected file from the list and close it in the editor.

Change the IDE appearance

You can switch between schemes, keymaps, or viewing modes.

1. Press `Ctrl+``.
2. In the **Switch** menu, select the option you need and press `Enter`. Use the same shortcut `Ctrl+`` to undo your changes.

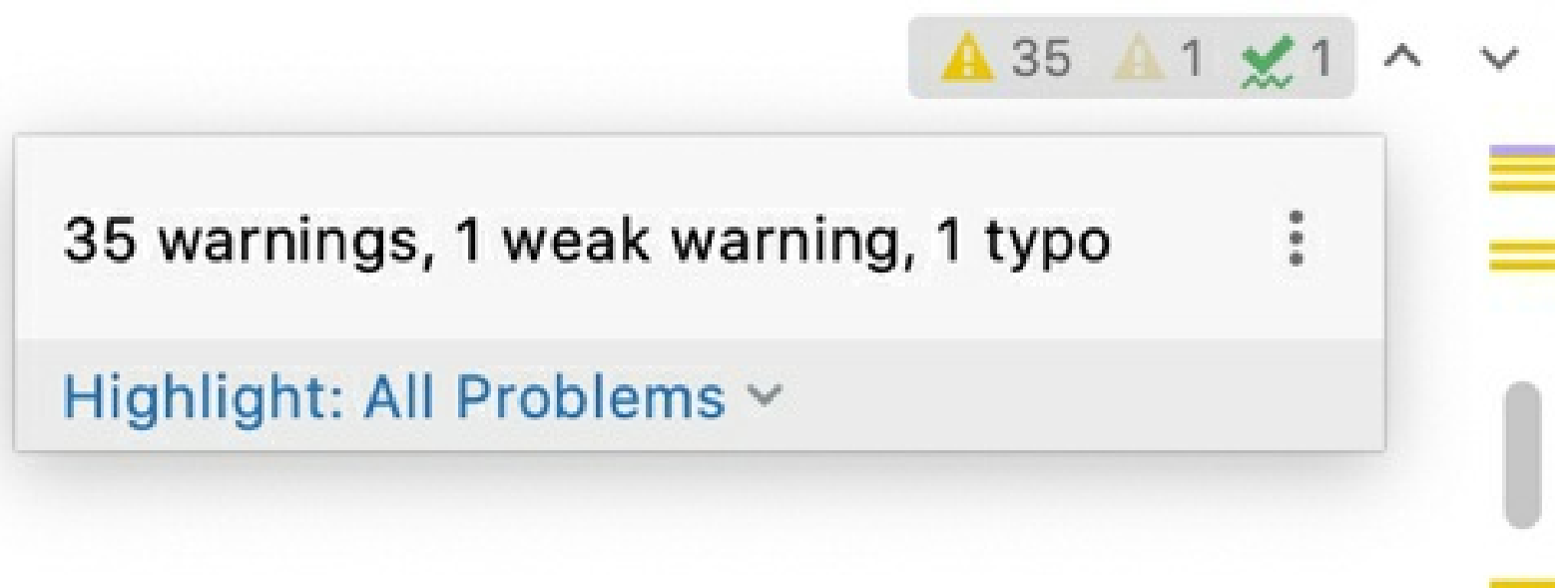
You can also find and adjust the color scheme settings including the high contrast color scheme for people with eyesight deficiency on the **Editor | Color Scheme** page and the keymap settings on the **Keymap** page of the **Settings/Preferences** dialog `Ctrl+Alt+S`.

Jump to the navigation bar

- Press `Alt + Home`.

Scrollbar

When you work with code in the editor, IntelliJ IDEA displays code analysis results that include errors and warnings on the scrollbar. You can check whether your code has issues and quickly navigate to them. The top of the scrollbar has the **Inspections** widget that gives you a brief summary of the code problems. Click the widget get more information on each detected problem in the **Problems** tool window.



For more information, refer to [Current file](#).

The stripes on the scrollbar indicate places where IntelliJ IDEA found a problem. Hover over a stripe to see a tooltip describing the problem or click the stripe for a quick navigation.

It is normal to see many stripes while your are working on a file. Many of these errors, warnings, and

suggestions are eventually resolved as you complete the code. Should any errors remain when you feel your code is complete we recommend that you explore and resolve them before compiling your project. The different colors of stripes indicate severity of the problems from an error marked in red to a **TODO** comment marked in blue, but you can change the displayed colors if you need. For more information, refer to [Change inspection severity](#).

Editor tabs

You can close, hide, and detach editor tabs. Every time you open a file for editing, a tab with its name is added next to the active editor tab.

From the main menu, select **Window | Editor Tabs** to see what additional actions you can perform with the editor tabs. For example, **Close Tabs to the Left** or **Close Tabs to the Right**.

You can use the tab's context menu for the same purpose.

To configure the settings for editor tabs, use the **Editor | General | Editor Tabs** page of the **Settings/Preferences** dialog `Ctrl+Alt+S`. Alternatively, right-click a tab and select **Configure Editor Tabs** from the list of options.

Open or close tabs

- To close all opened tabs, select **Window | Editor Tabs | Close All Tabs** from the main menu.
- To close all inactive tabs, press `Alt` and click on the active tab. In this case, only the active tab stays open.
- To close all inactive tabs except the active one and the pinned tabs, right-click any tab and select **Close Other Tabs**.
- To close only the active tab, press `Ctrl+F4`. You can also click the mouse's wheel button anywhere on a tab to close it.
- To reopen the closed tab, right-click any tab, and from the context menu, select **Reopen Closed Tab**.
- To open a new tab at the end of the already opened one, select the **Open new tabs at the end** in the tab settings.

Copy path or filename

1. Right-click the tab.
2. From the list that opens, select **Copy Path**.
3. From the list that opens, select your copy option.

Copy	
1. Absolute Path	/Users/jetbrains/micronaut-petclinic/src/main/java...
2. File Name	PetClinicApplication
3. Path From Content Root	src/main/java/com/example/micronaut/p...
4. Path From Source Root	com/example/micronaut/petclinic/PetClini...
5. Path From Repository Root	src/main/java/com/example/micronau...
6. Toolbox URL	jetbrains://idea/navigate/reference?project=microna...

IntelliJ IDEA copies the item to the clipboard and you can paste it (`Ctrl+V`) wherever you need.

Move, remove, or sort tabs

- To move or remove the icon on a tab, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Editor Tabs** and select the appropriate option in the **Close button position** field.
- To place the editor tabs in a different part of the editor frame or hide the tabs, right-click a tab and select **Configure Editor Tabs** to open the **Editor Tabs** settings. In the *Appearance* section, in the **Tab placement** list, select the appropriate option.

To access the **Editor Tabs** settings when all tabs are hidden, select **Window | Editor Tabs | Configure Editor Tabs** from the main menu.

- To sort the editor tabs alphabetically, right-click a tab and select **Configure Editor Tabs** to open the **Editor Tabs** settings. In the *Tab order* section, select **Sort tabs alphabetically**.

Pin or unpin a tab

You can pin an active tab in the editor so that it will stay open when the tab limit is reached or when you use the **Close Other Tabs** command.

- To pin or unpin an active tab, right-click it and select **Pin Tab** or **Unpin Tab** from the context menu.
- To close all tabs, but the pinned ones, right-click any tab and select **Close All but Pinned**.
- To assign a keyboard shortcut for the **Pin Tab** action, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Keymap**, find the **Pin Active Tab** action, right-click it, select **Add Keyboard Shortcut**, and press the key combination you want to use.

Detach a tab

When you detach a tab, the tab opens in a separated window and the window becomes reserved for the detached tab. If you try to detach another tab from the main frame, it will be opened in the new window.

- To detach an active tab, press `Shift+F4`.
- Drag the tab you need outside of the main window and drag the tab back to attach it.
- You can also use `Alt+mouse` for the same action.
- In the **Project** tool window, select a file you want to detach and press `Shift+Enter`.

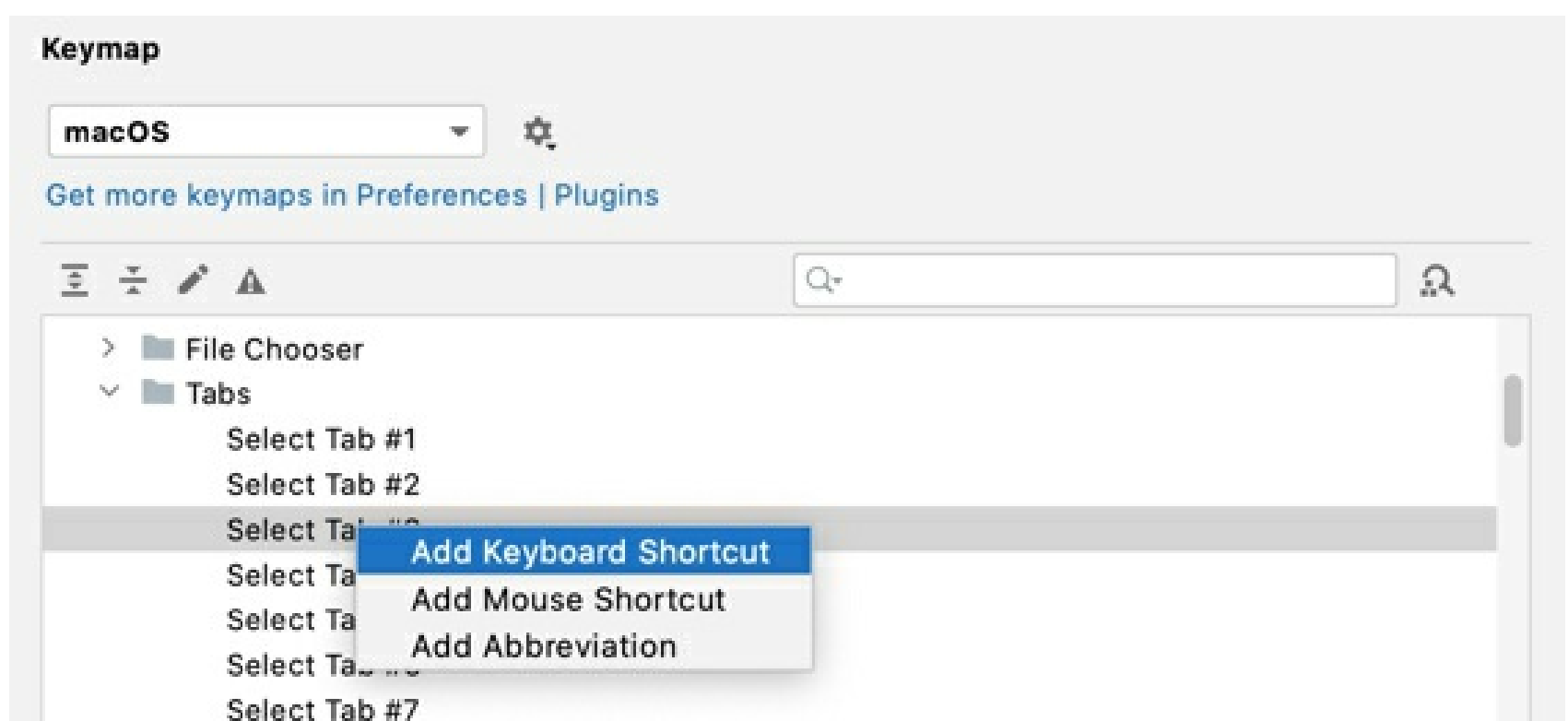
Switch between tabs

- To move between tabs, press `Alt+Right` OR `Alt+Left`.
- You can also switch between recently viewed tabs or files.

In the editor, press `Ctrl+Tab`. Keep pressing `Ctrl` for the **Switcher** window to stay open. Use `Tab` to switch between tabs and other files.

Assign a shortcut for the opened tab

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Keymap**.
2. In the list of directories, click the **Other** directory and from the list of tabs, select the one for which you need to add a shortcut. The limit of tabs to which you can assign shortcuts is 9.



Change the default tab limit

IntelliJ IDEA limits number of tabs that you can open in the editor simultaneously (the default tab limit is 10).

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Editor Tabs**.
2. In the **Tab closing policy** section, adjust the settings according to your preferences and click **OK**.

If the tab limit equals to 1, the tabs in the editor will be disabled. If you want the editor to never close the tabs, type some unreachable number.

Open files in the preview tab

The preview tab allows you to view files in a single tab one by one without opening each file in a new tab. This is helpful if you need to look through several files without exceeding the tab limit.

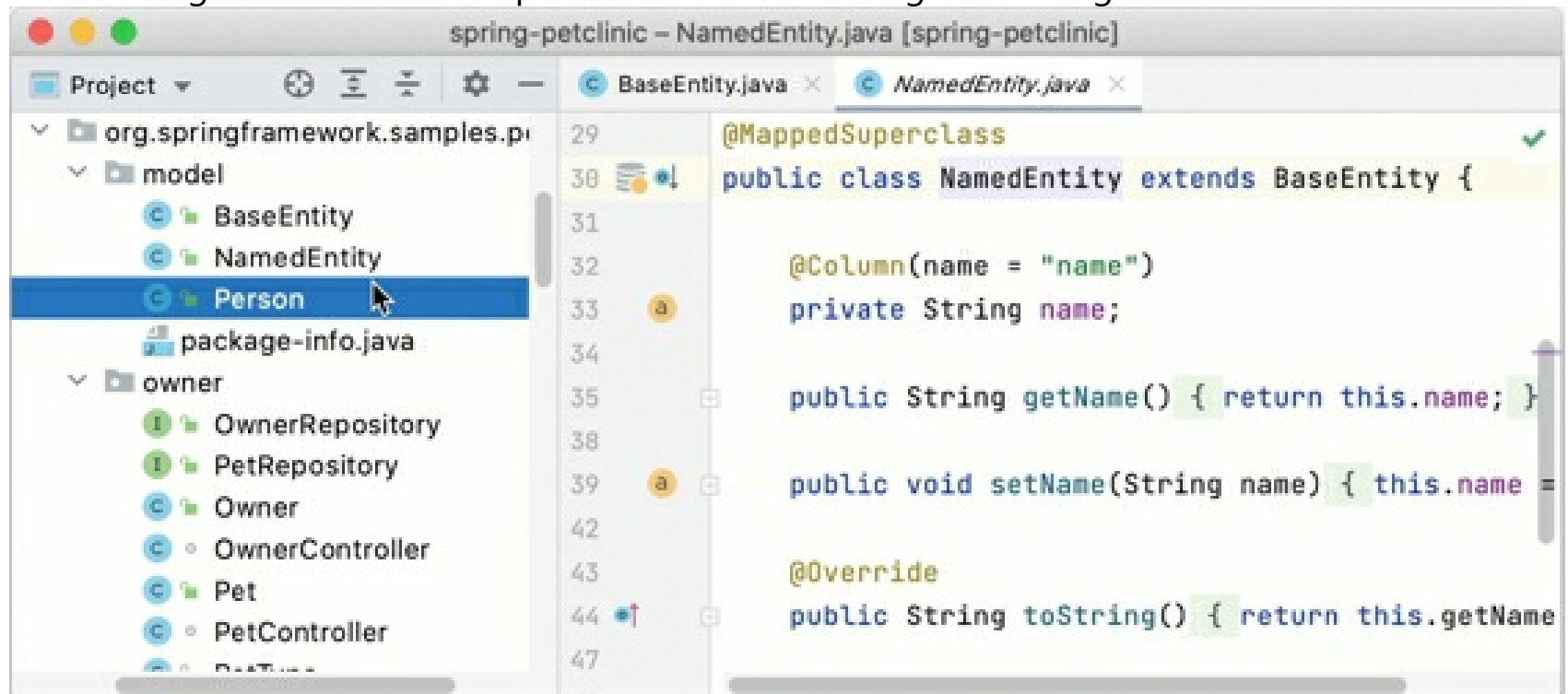
1. In the **Project** tool window `Alt+1`, click  and select the **Enable Preview Tab** option.

You can also enable the preview tab in **Settings/Preferences | General | Editor Tabs | Opening Policy**.

2. In the **Project** tool window, select a file that is not already open in any other tab.

The name of the file is written in *italic* to indicate the preview mode. Any other file that you select will replace the previous one in the preview tab.

Start editing the file to exit the preview mode and change it to a regular tab.



Gif

Note that when the preview tab is enabled, the **Open Files with Single Click** option is ignored. Double-click a file to open it in a regular tab.

Hide editor tabs if there is no more space

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Editor Tabs**.
2. Select the **Hide tabs if there is no space** option. Extra tabs will be placed in the list located in the upper right part of the editor.

Change the font size in tabs

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Appearance & Behavior | Appearance**.
2. In the **Size** field, specify the font size and click **OK** to save the changes.

Keep in mind that the font size will change not only for tabs, but for tool windows as well.

Split screen

IntelliJ IDEA offers various actions that you can invoke from main or context menu, editor, or the project tool window to split the editor screen.

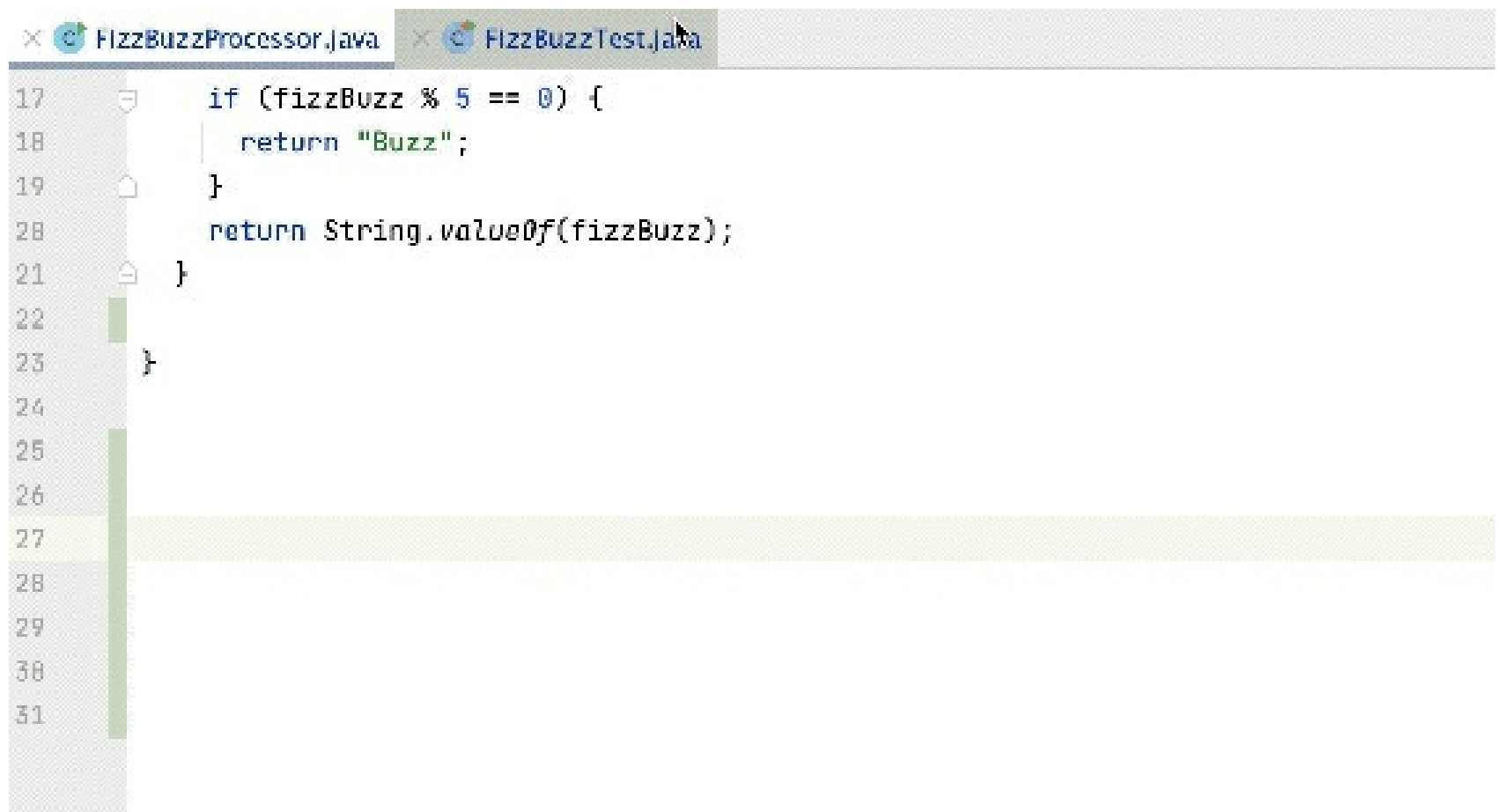
- In the editor, right-click the desired editor tab and select how you want to split the editor

window (**Split Right** or **Split Down**). IntelliJ IDEA creates a split view of the editor and places it according to your selection.

- As an alternative, from the main menu, select **Window | Editor Tabs** and the **Split and Move Right** or **Split and Move Down** option.

If necessary, you can assign keyboard shortcuts for these actions. To do this, in the **Settings/Preferences** dialog **Ctrl+Alt+S**, go to **Keymap**, find the **Split Right** or **Split Down** action, right-click it, select **Add Keyboard Shortcut**, and press the key combination you want to use. You can do the same for the **Split and Move Right** or **Split and Move Down** action.

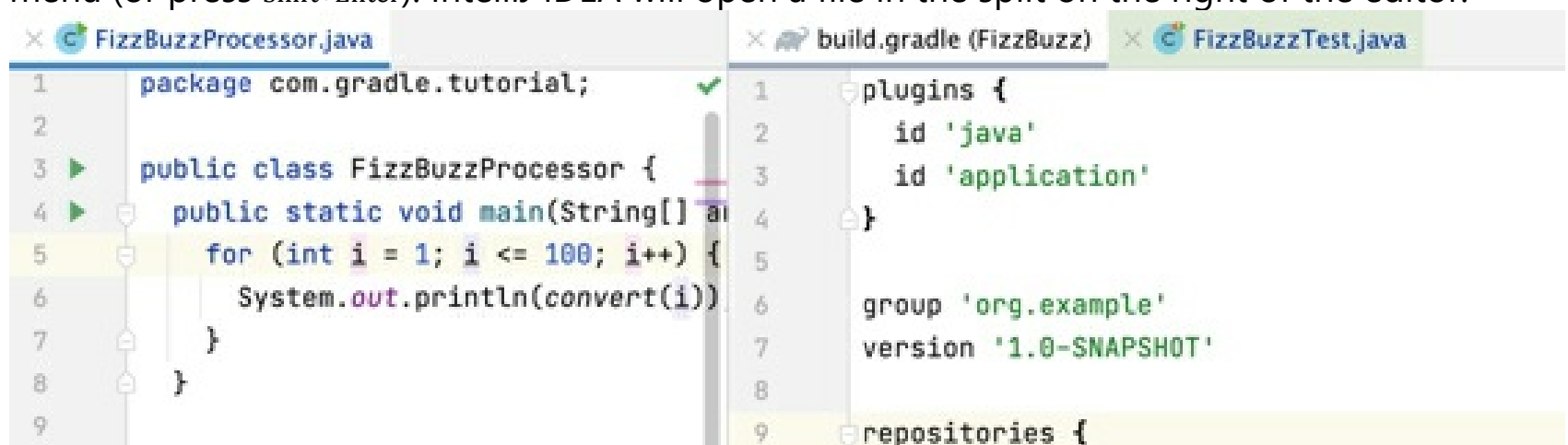
- You can drag a tab vertically or horizontally in order to split the editor, and drag the tab back to unsplit the screen.



Gif

- You can open a file in the editor in the right split.

In the **Project** tool window, right-click a file and select **Open in Right Split** from the context menu (or press **Shift+Enter**). IntelliJ IDEA will open a file in the split on the right of the editor.



If there are two splits and focus is in the left split, the file will be opened in the existing right split. If the focus is in the right split, the file will be opened in the next right split.

- You can move files between split screens. Right-click the needed file tab in the editor and from the context menu select **Move To Opposite Group** or **Open In Opposite Group**.
- You can maximize the active split screen. Place the cursor in the screen you want to maximize and press **Ctrl+Shift+F12**.

In this case, IntelliJ IDEA hides all tool windows and moves other split screens aside. An alternative, you can double-click the active split screen to maximize it.

- To unsplit the screen, from the context menu, select **Unsplit** or **Unsplit All** to unsplit all the split frames.

Move the split screen

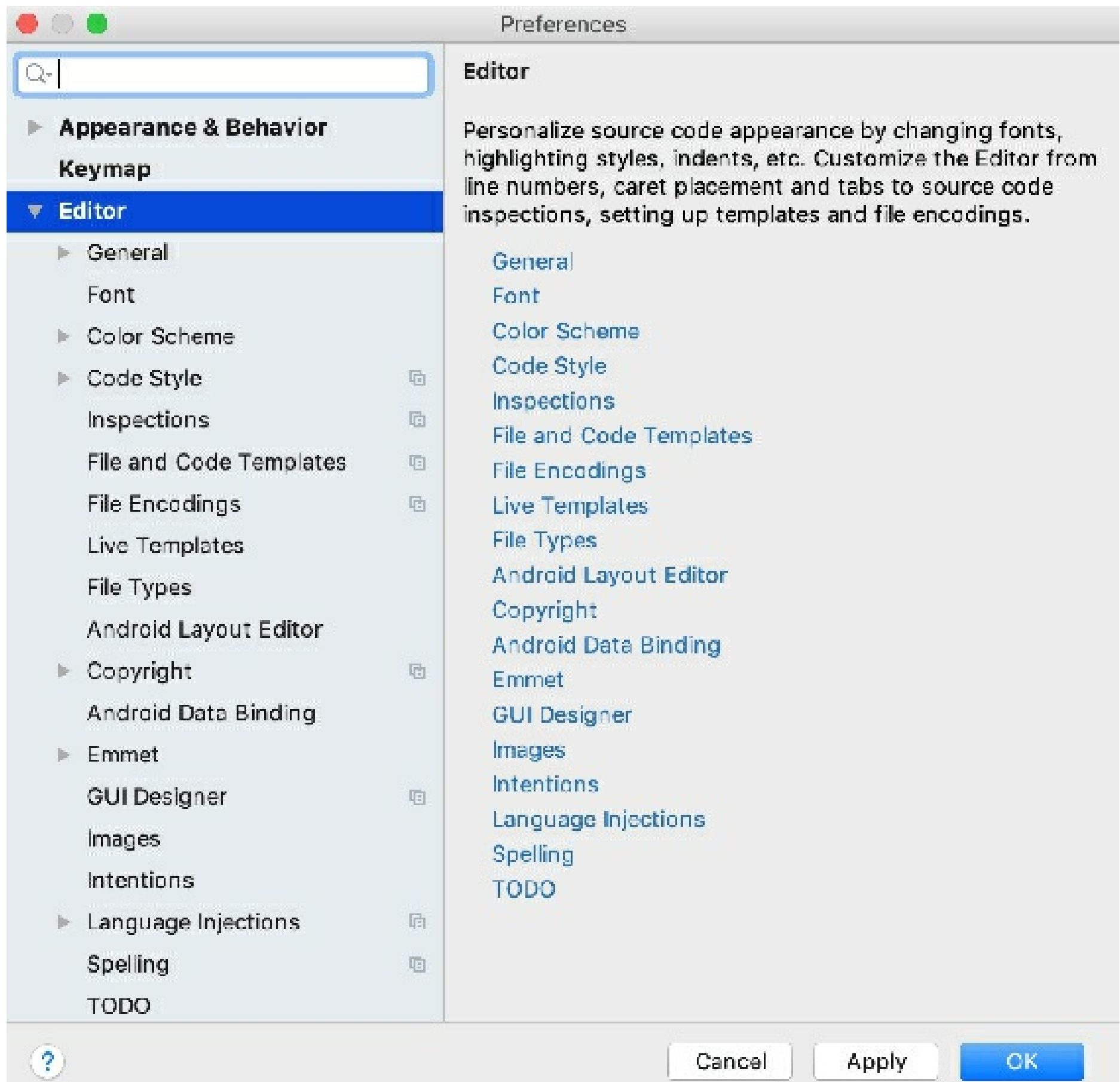
1. Place the caret inside the desired split frame.
2. From the main menu, select **Window | Editor Tabs**.
3. From the list of options, select one of the following options:
 - Stretch Editor to Top
 - Stretch Editor to Left
 - Stretch Editor to Bottom
 - Stretch Editor to Right

You can assign a shortcut to each option and use a keyboard to stretch the split frame.

To move between the split frames which you've created, from the main menu, select **Window | Editor Tabs**. From the list of options select **Goto Next Splitter** N/A or **Goto Previous Splitter** N/A respectively.

Useful editor configurations

You can use the **Settings/Preferences** dialog Ctrl+Alt+S to customize the editor's behavior.



Check the following popular configurations:

Configure code formatting

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Code Style**.
2. From the list of languages select the appropriate one and on the language page, configure settings for tabs and indents, spaces, wrapping and braces, hard and soft margins, and so on.

Configure fonts, size, and font ligatures

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Font**.

If you, for example, have previously saved **Color Scheme Font** settings, the main settings become overridden. The link with the appropriate notification will appear on the **Font** page.

Change the font size in the editor

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General (Mouse Control section)**.
2. Select the **Change font size with Ctrl+Mouse Wheel** option.

3. Return to the editor, press and hold `Ctrl`, and using the mouse wheel, adjust the font size.

You can configure the editor size on the **Font** page of the editor settings.

Configure the color scheme settings for different languages and frameworks

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | Color Scheme**.
2. Open the **Color Scheme** node and select the needed language or framework. You can also select the **General** option from the node's list to configure the color scheme settings for general items such as code, editor, errors and warnings, popups and hints, search results, and so on.

Configure code completion options

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Code Completion**. Here you can configure the case sensitive completion, auto-display options, code sorting, and so on.

Configure the caret placement

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General**. In the **Virtual Space** section, you can configure the caret placement options.

Select the **Allow placement of caret after end of line** option to place the caret at the next line in the same position as the end of the previous line. If this option is cleared, the caret at the next line is placed at the end of the actual line.

Select the **Allow placement of caret inside tabs** option to help you move the caret up or down inside the file while keeping it in the same position.

Configure the behavior of trailing spaces on save

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General**. In the **Other** section, you can configure options for trailing spaces.

For example, when you save your code either manually or automatically and want to preserve trailing spaces on the caret line regardless of what option is selected in the **Strip trailing spaces on save** list, select the **Always keep trailing spaces on caret line** option.

Configure the editor appearance options

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Appearance**.

For example, you can configure showing the hard wrap guide, or showing parameter hints.

Manage the appearance for long lines

1. In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General**.
2. In the **Soft Wraps** section, specify the appropriate options.

For example, you can specify file types to which you want to apply soft wraps. It might be helpful when you write documentation in markdown files.

Configure smart keys

You can configure a certain behavior for different basic editor actions depending on the language you use.

- In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Smart keys**.

For example, for Java, SQL or Python, you can select the Jump outside closing bracket/quote with Tab option to enable navigation outside the closing brackets or quotes with `Tab` when you type your code.

Multiple carets and selection ranges

When typing, copying, or pasting, you can toggle multiple carets so that your actions apply in several places simultaneously. Advanced editor actions, such as code completion and live templates are supported as well and will apply to each caret.

The most recently added caret is considered **primary**. Highlighting of the current editor line, completion lists, and other visual assistance features will apply to the primary caret. This caret will also remain when you turn off multiple carets.

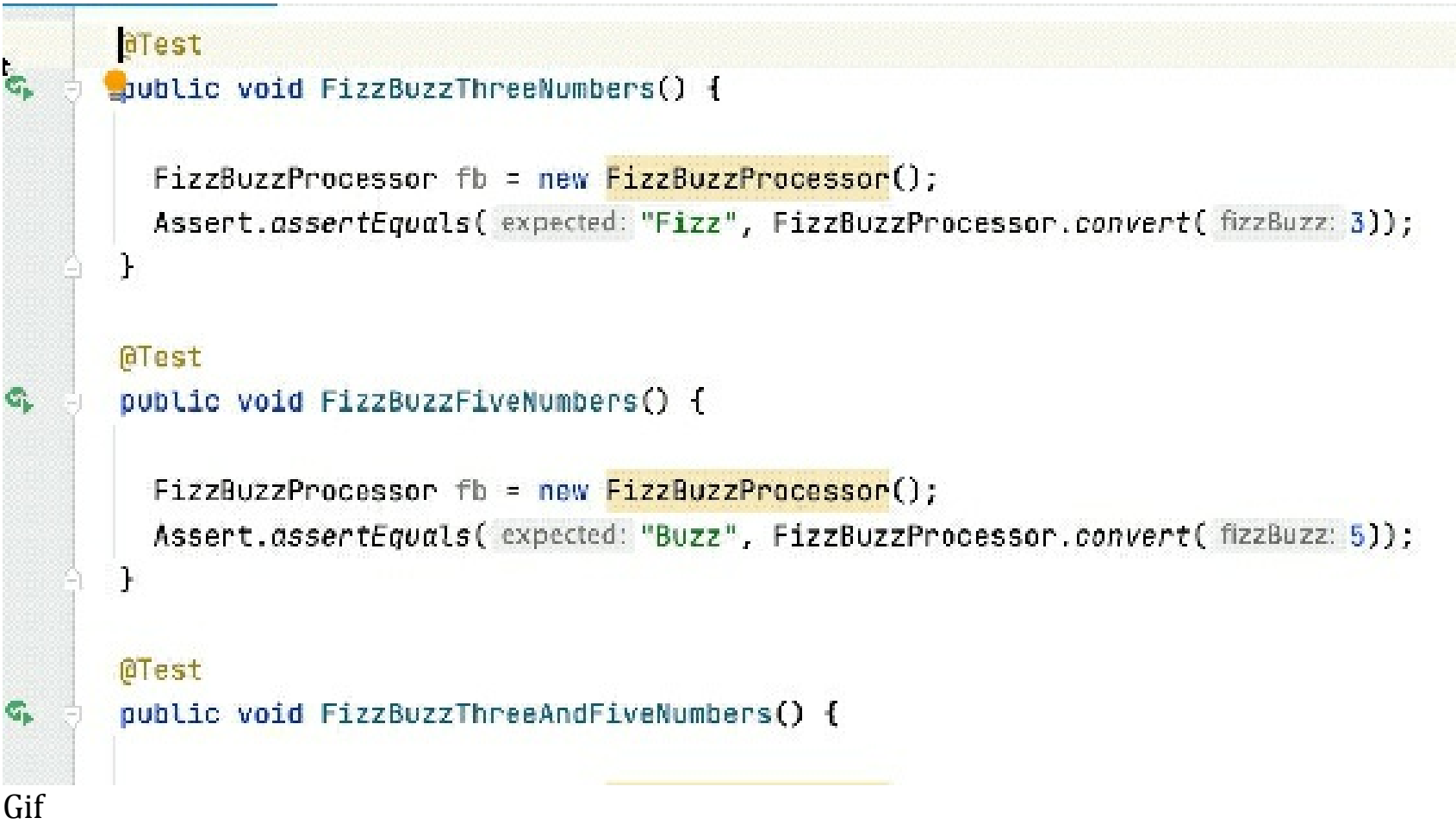
Add and remove carets

You can add carets in two different ways:

To existing characters	Using virtual spaces
If there is no character, tab, or whitespace at the position where you want to add a new caret, the new caret will be added to the last character position in the target line.	This way you can add new carets anywhere after the last character in any line. As soon as you start typing at a position beyond the end of the line, the necessary number of spaces will be added between the end of the line and the beginning of your input. You can enable virtual spaces on the Editor General page of the Settings/Preferences <code>Ctrl+Alt+S</code> and they are also enabled in the column selection mode.

Add or remove carets at selected locations using mouse

- `Alt+Shift+Click` at the target location to add another caret.



- `Alt+Shift+Click` at one of the multiple carets to remove it. The last caret will not be removed.

Add carets above or below the current caret using keyboard

- Press `Ctrl` twice, and then without releasing it, press up or down arrow keys.

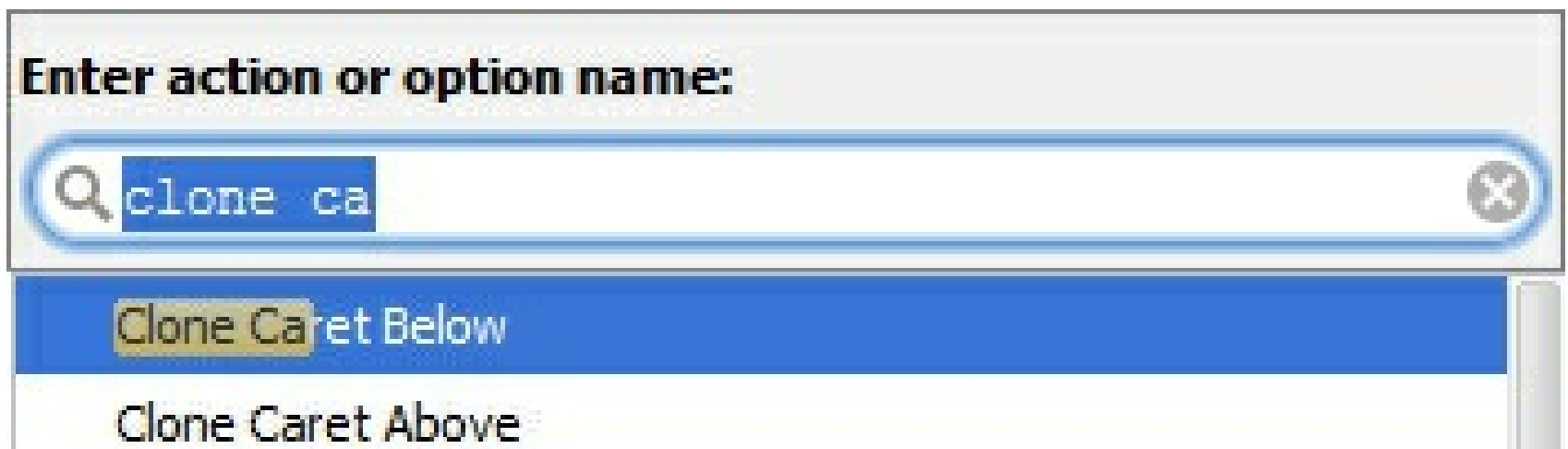
If virtual spaces are enabled, new carets will be added exactly above or below the current caret position. Otherwise, in lines, which are shorter than the current offset, carets will added at line

ends.

```
public static String convert(int fizzBuzz) {  
    if (fizzBuzz % 15 == 0) {  
        return "FizzBuzz";  
    }  
    if (fizzBuzz % 3 == 0) {  
        return "Fizz";  
    }  
    if (fizzBuzz % 5 == 0) {  
        return "Buzz";  
    }  
    return String.valueOf(fizzBuzz);  
}
```

Gif

- Enable the column selection mode (press Alt+Shift+Insert) and then press Shift+Up/ Shift+Down.
- Press Ctrl+Shift+A, type *Clone caret*, and choose the desired action from the suggestion list:



Note that by default these actions are not associated with keyboard shortcuts. You can assign your custom shortcuts to these actions as described in configuring keyboard shortcuts.

Add carets at each line of the current document

- Press Ctrl+Home to set the caret at the beginning of the first line, enable the column selection mode (press Alt+Shift+Insert), and then press Ctrl+Shift+End.

Add carets to the end of each line in the selected block

- Select a code block in the editor and then press Alt+Shift+G.

Remove multiple carets

- Press Esc to delete all existing carets, except the one that was added last.
- Alt+Shift+Click at one of the multiple carets to remove it. The last caret will not be removed.

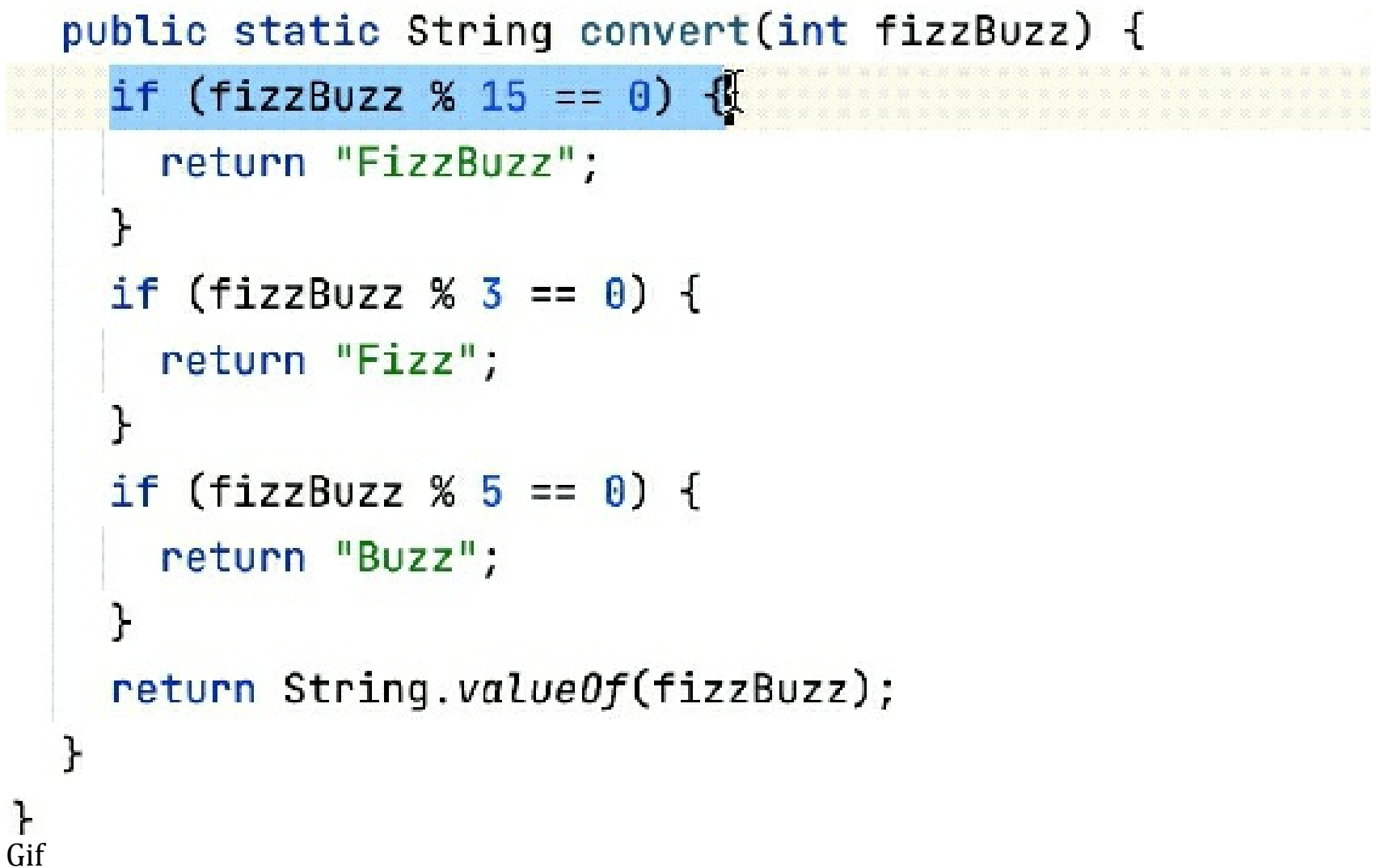
Select multiple non-contiguous ranges

When you select multiple text ranges (non-contiguous selection), note the following:

- Each selection range is associated with its own caret, so you can start typing to replace all selected ranges with your input, or you can press **Left Arrow** Or **Right Arrow** to remove the selection ranges but keep multiple carets at beginnings/ends of the ranges.
- As soon as selection ranges overlap, they are merged into a single selection range with a single caret.
- Selection works independently of the code structure, that is selection ranges can include any characters, identifiers, words in string literals, comments, or their parts. So you have to be careful when changing the selected ranges as they may include different identifiers or their parts.

Select multiple words or text ranges

- While **Alt+Shift+Click** will add a new caret, double-clicking words or dragging the mouse over text ranges (keeping the same keys pressed) will add new carets with the corresponding selections.



```
public static String convert(int fizzBuzz) {
    if (fizzBuzz % 15 == 0) {
        return "FizzBuzz";
    }
    if (fizzBuzz % 3 == 0) {
        return "Fizz";
    }
    if (fizzBuzz % 5 == 0) {
        return "Buzz";
    }
    return String.valueOf(fizzBuzz);
}
}
Gif
```

Select multiple occurrences of a word or a text range

1. If you want to select words, set your caret at an occurrence of the desired word. Otherwise, select the desired range with the mouse or with keyboard shortcuts.
2. Do one of the following:
 - Successively press **Alt+J** to find and select the next occurrence of case-sensitively matching word or text range.

```

public static String convert(int fizzBuzz) {
    if (fizzBuzz % 15 == 0) {
        return "FizzBuzz";
    }
    if (fizzBuzz % 3 == 0) {
        return "Fizz";
    }
    if (fizzBuzz % 5 == 0) {
        return "Buzz";
    }
    return String.valueOf(fizzBuzz);
}
}

```

Gif

- Press Ctrl+Alt+Shift+J to select all case-sensitively matching words or text ranges in the document.

3. To remove selection from the last selected occurrence, press Alt+Shift+J.
4. After the second or any consecutive selection was added with Alt+J, you can skip it and select the next occurrence with F3. To return the selection to the lastly skipped occurrence, press Shift+F3.

```

System.out.println("Year : " + cal.get(Calendar.YEAR));
System.out.println("Month : " + cal.get(Calendar.MONTH));
System.out.println("Day of Month : " + cal.get(Calendar.DAY_OF_MONTH));
System.out.println("Day of Week : " + cal.get(Calendar.DAY_OF_WEEK));
System.out.println("Day of Year : " + cal.get(Calendar.DAY_OF_YEAR));
System.out.println("Week of Year : " + cal.get(Calendar.WEEK_OF_YEAR));
System.out.println("Week of Month : " + cal.get(Calendar.WEEK_OF_MONTH));
System.out.println("Day of the Week in Month : " + cal.get(Calendar.DAY_OF_WEEK_IN_MONTH));
System.out.println("Hour : " + cal.get(Calendar.HOUR));
System.out.println("AM PM : " + cal.get(Calendar.AM_PM));
System.out.println("Hour of the Day : " + cal.get(Calendar.HOUR_OF_DAY));
System.out.println("Minute : " + cal.get(Calendar.MINUTE));

```

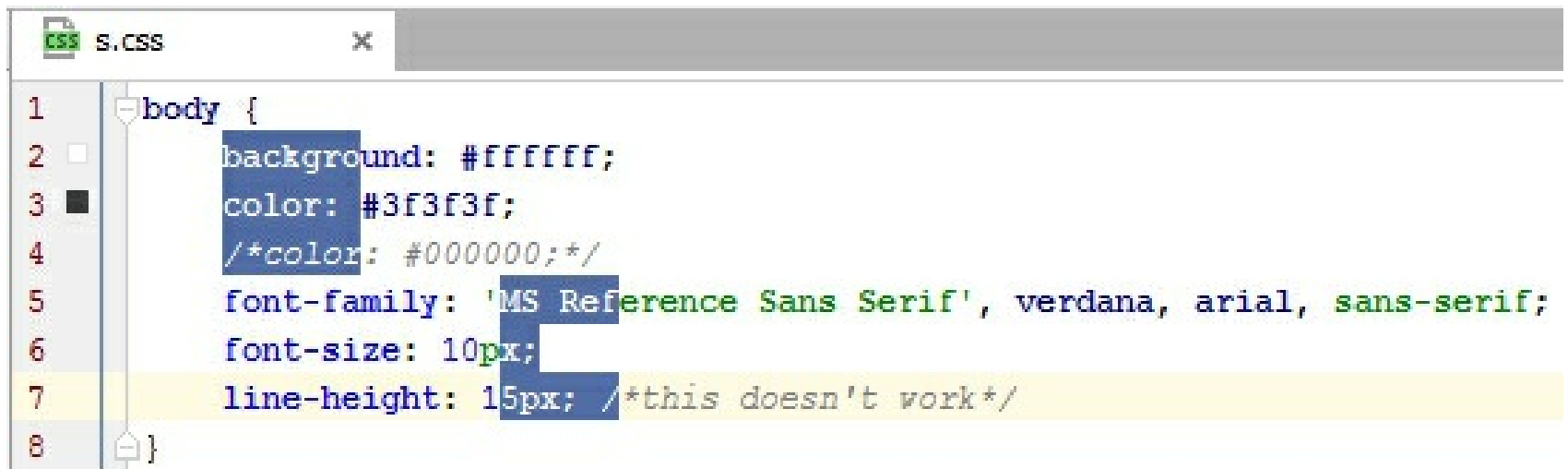
Gif

Find and select multiple occurrences of a string

1. Press Ctrl+F or choose **Edit | Find | Find** from the main menu. The search pane appears at the top of the active editor.
2. Enter the string that you want to find and select. To the right of the search string, you will see the number of occurrences in the current document.
3. Optionally, restrict your search by case Alt+C or to match only whole words Alt+W.
4. Press Ctrl+Alt+Shift+J or click **Select All Occurrences** on the toolbar.

Use mouse to select rectangular fragments of text in normal selection mode

1. Make sure that the column selection mode is *disabled*.
2. To select ranges as a single rectangle, do one of the following:
 - Set the caret at one corner of the rectangle, and then Alt+Shift+Middle-Click at the diagonally opposite corner.
 - Alt+Click and drag the mouse to make the selection.
3. To select ranges as multiple rectangular selections, Ctrl+Alt+Shift+Click and drag the mouse over the desired parts of code.
4. As a result, you will have multiple selection ranges in each affected document line. On lines that are shorter than the rectangle, the selection will only span to the last character.



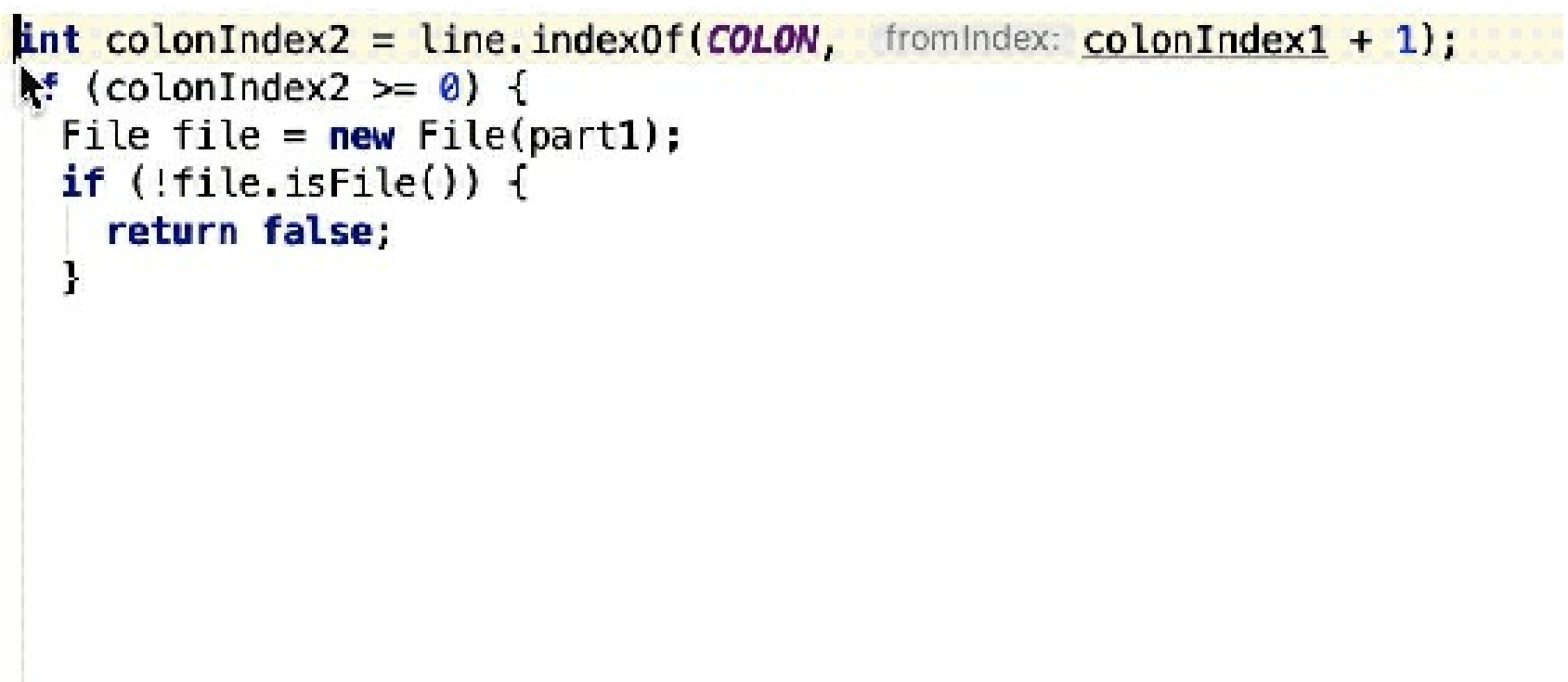
Column selection mode

In the *column selection mode*, keyboard navigation and selection shortcuts in the current document work differently to simplify adding multiple carets and making rectangular selections:

- You will be able to set your caret anywhere after the last character in any line. As soon as you start typing at a position beyond the end of the line, the necessary number of spaces will be added between the end of the line and the beginning of your input.

You can also enable this in the normal selection mode — select **Allow placement of caret after end of line** on the **Editor | General** page of the **Settings/Preferences** Ctrl+Alt+S.

- Pressing Shift+Up/ Shift+Down or dragging the mouse up and down will add new carets above/below the current one instead of making a continuous selection.



Gif

- The enabled column selection mode only affects the current editor tab. If you close or reopen the tab, it will switch back to the normal mode.

If the column selection mode is enabled for the current document, you will see the **Column** indicator on the status bar.

Toggle between normal and column selection modes

- Press `Alt+Shift+Insert`.
- From the main menu, choose **Edit | Column Selection Mode**.
- From the context menu of the editor, choose **Column Selection Mode**.

Copy and paste with multiple carets

When text ranges selected with multiple carets are copied `Ctrl+C` or cut `Ctrl+X`, selections for each caret are placed to the clipboard as separate lines, even if the original selections were on the same line.

If the column selection mode was enabled, the selection could also include empty spaces after ends of lines. These will be replaced with whitespaces in the clipboard if you copy the selection.

When you paste any multi-line content from the clipboard, you can add multiple carets for each line in desired places, and then press `Ctrl+V` to paste each line at its own caret.

LightEdit mode

When you need to edit just one file without creating or loading the whole project in IntelliJ IDEA, you can use the **LightEdit** mode.

Keep in mind that the LightEdit mode works as a text-like editor, and it doesn't support the usual IDE editor features such as code completion, or code navigation. However, you can navigate to a specific line of code (Ctrl+G), fold or unfold parts of code, check, and change file encoding.



Open a file in the LightEdit mode

You can use several ways to open a file in the LightEdit mode.

When you open a file from the welcome screen or from the local file system, it opens in a temporary project in the IDE not in the LightEdit mode, and you can use all the available IntelliJ IDEA features.

Open a file from the command line

- Depending on your OS, open the file from the command line:

Windows

macOS

Linux

```
idea.bat -e myfile.txt
```

Copied!

You can find the executable script for running IntelliJ IDEA in the installation directory under **bin**. To use this executable script as the command-line launcher, add it to your system PATH as described in Command-line interface.

Open a file inside the opened project

You can open a non-project file in the already opened project.

- If you don't have the IntelliJ IDEA launcher configured, you need to configure it before you can open a file.
- Depending on your OS, open the file adding the **-e** command before the name of your file.

For example, for macOS the syntax would look as follows:

```
idea -e myfile.txt
```

Copied!

Open and edit a file with the wait switch

You can interrupt a process in the command line and put terminal on hold until you done editing a file in the LightEdit mode. For example, when you work in the command line and run a commit process to Git, you can pause the terminal and use a text editor in the LightEdit mode to quickly write a commit

message.

If you have a Toolbox installation of IntelliJ IDEA, ensure that you have installed IntelliJ IDEA from the Toolbox 1.9 version. If you have a stand-alone installation of IntelliJ IDEA, create a command-line launcher (**Tools | Create Command-line Launcher**) first.

1. Depending on your OS, open the file adding the `-e` and `-w` commands before the name of your file.

Windows

macOS

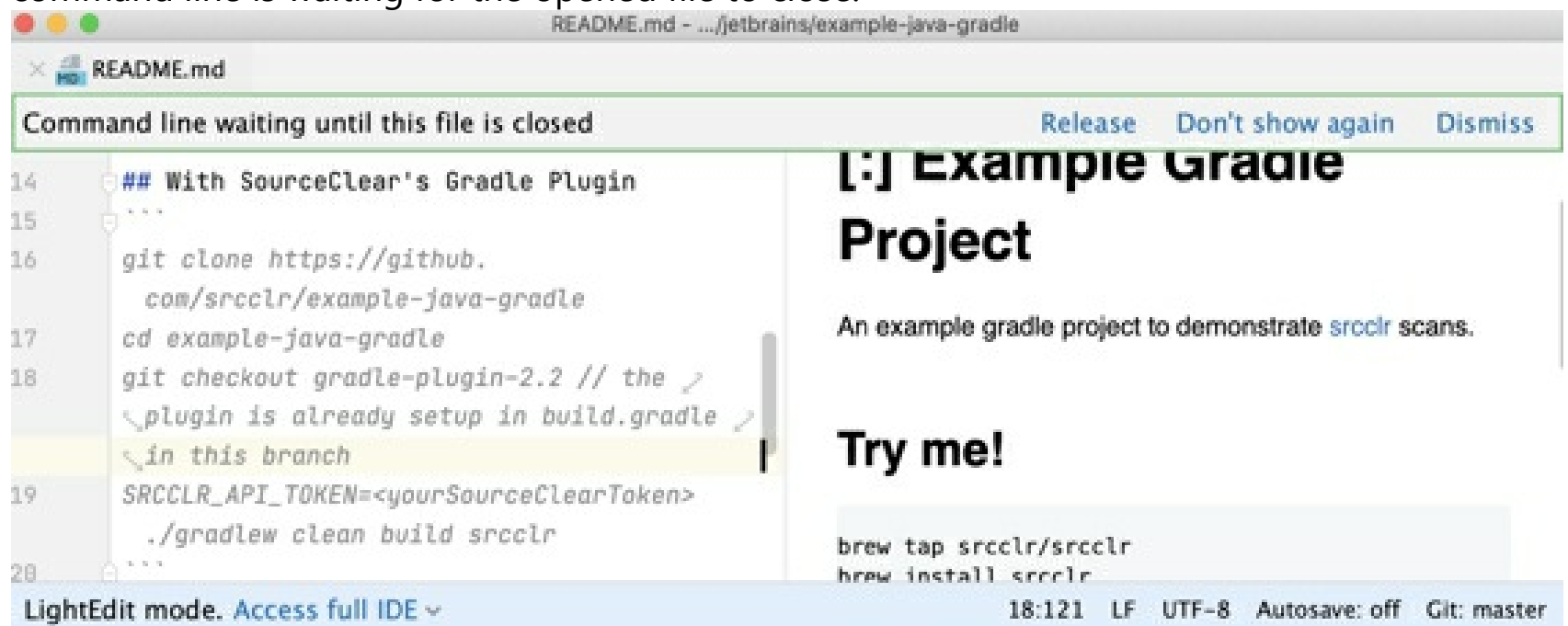
Linux

`idea.bat -e -w myfile.txt`

Copied!

You can find the executable script for running IntelliJ IDEA in the installation directory under **bin**. To use this executable script as the command-line launcher, add it to your system PATH as described in Command-line interface.

IntelliJ IDEA opens the file in the LightEdit mode, and displays a notification indicating that the command line is waiting for the opened file to close.



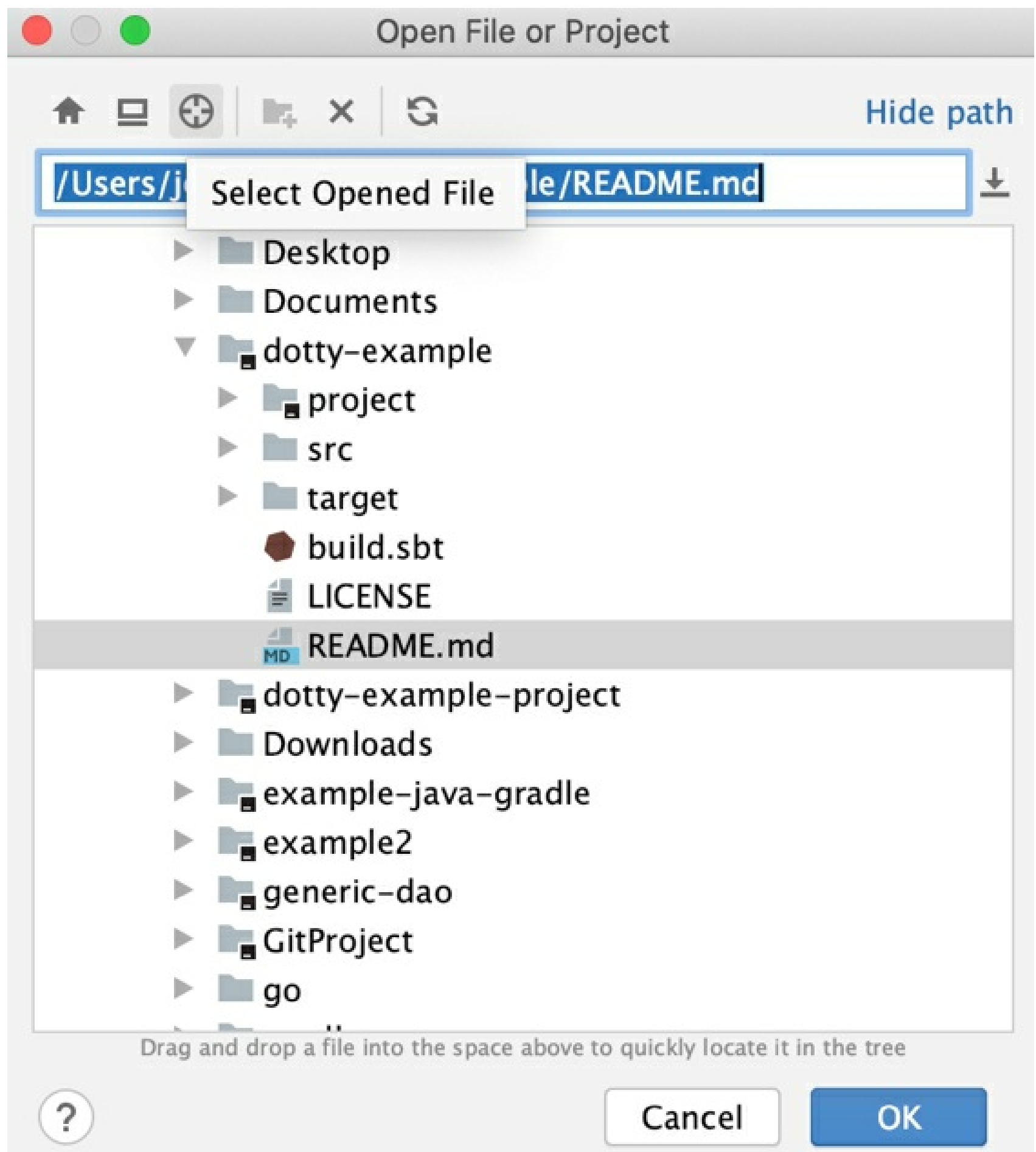
2. Click one of the notification's options or close the file to release the command line.

Work with code in the LightEdit mode

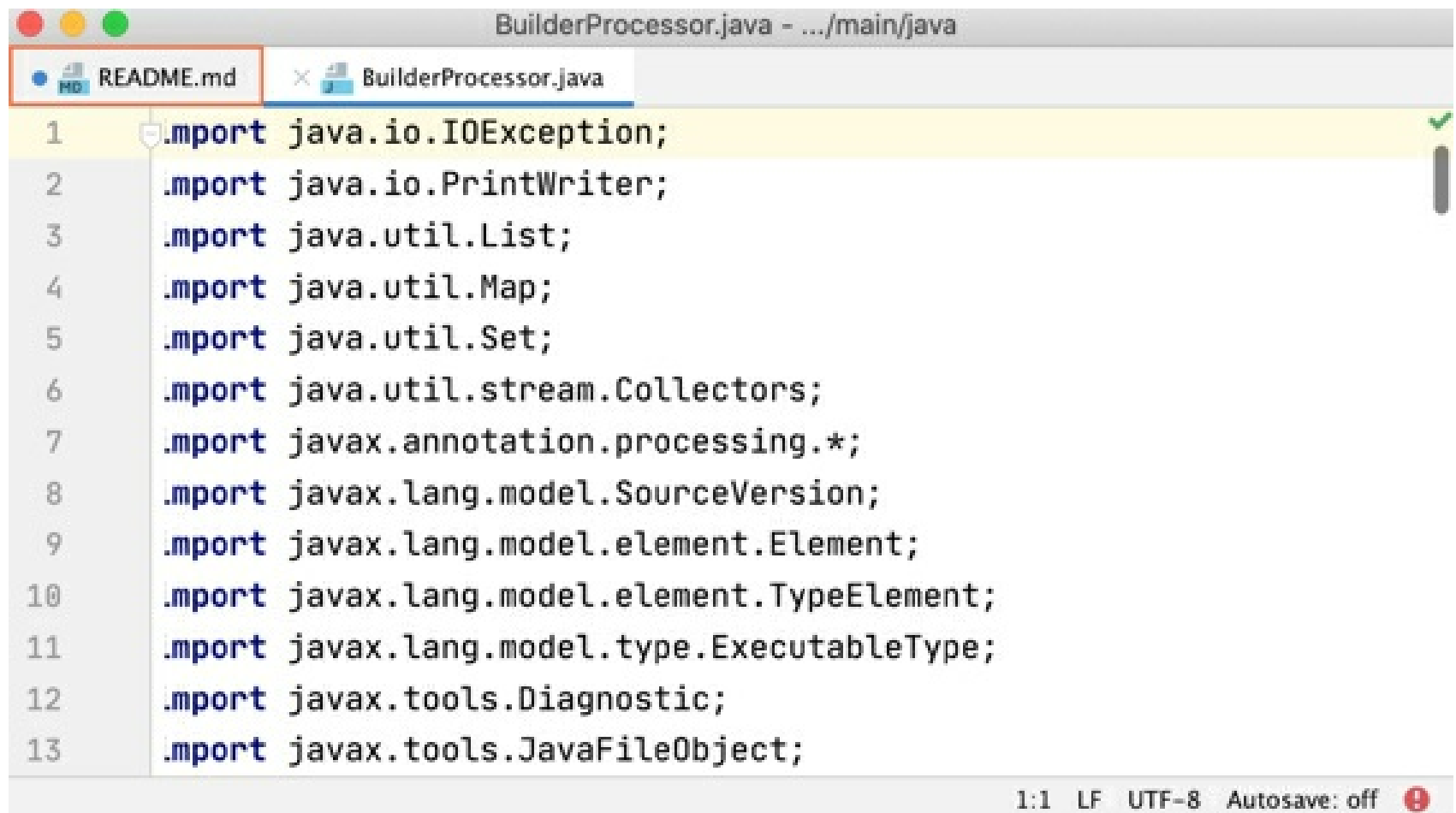
Even though the LightEdit mode doesn't support all of the IntelliJ IDEA editor coding assistance, you can still use basic editing features and basic menu options.

- Use the main menu to open recent files, show the line numbers, whitespaces, extend the code selection, and so on.

When you select **File | Open**, IntelliJ IDEA opens the **Open File or Project** dialog where you can quickly navigate to the opened file in the project's root directory. Click the **Select opened file** icon on the toolbar. Note that for macOS, the native dialog will open.



When you edit a file, the blue color indication on the tab shows that the file content was changed.

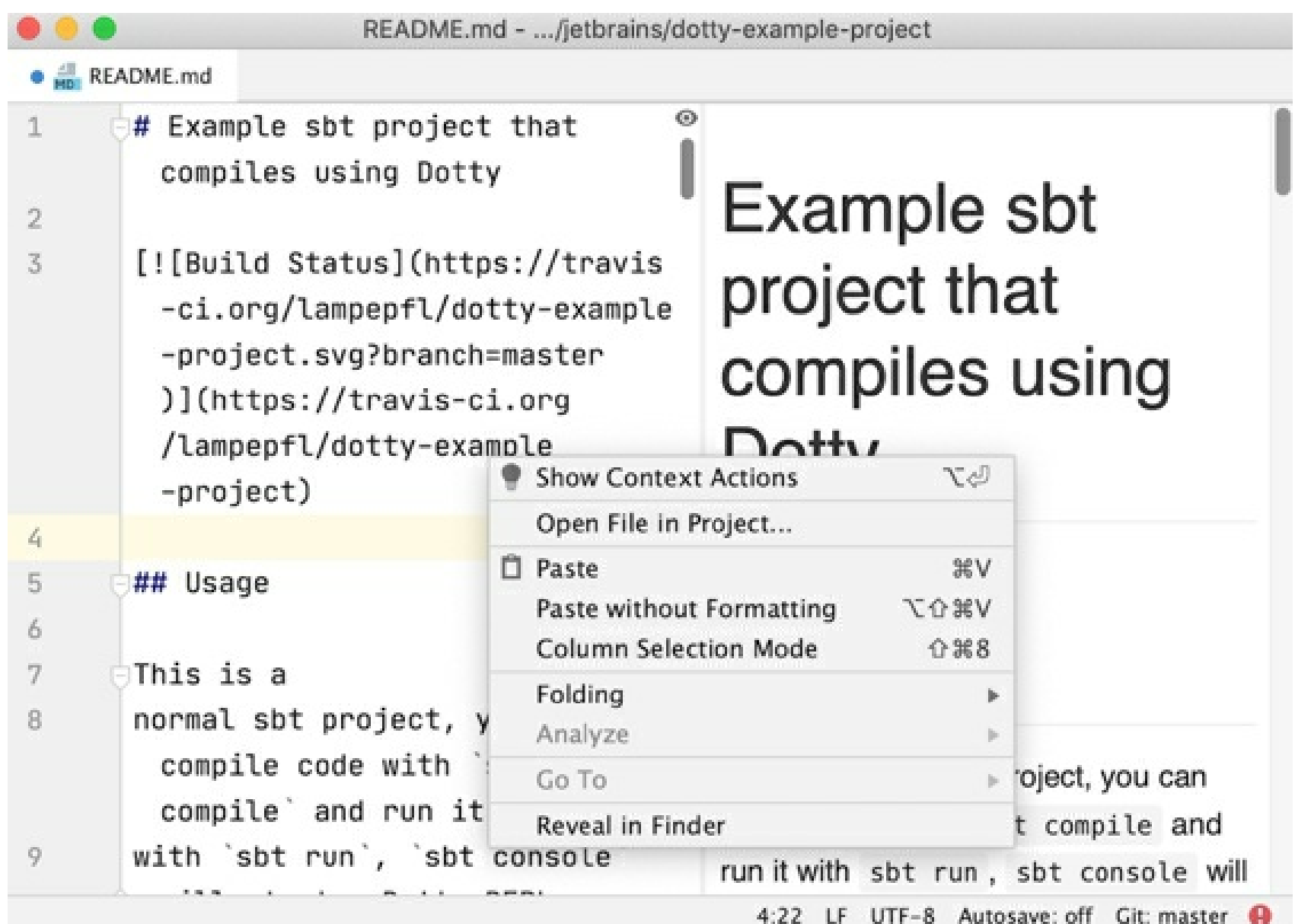


BuilderProcessor.java - .../main/java

```
1  import java.io.IOException;
2  import java.io.PrintWriter;
3  import java.util.List;
4  import java.util.Map;
5  import java.util.Set;
6  import java.util.stream.Collectors;
7  import javax.annotation.processing.*;
8  import javax.lang.model.SourceVersion;
9  import javax.lang.model.element.Element;
10 import javax.lang.model.element.TypeElement;
11 import javax.lang.model.type.ExecutableType;
12 import javax.tools.Diagnostic;
13 import javax.tools.JavaFileObject;
```

1:1 LF UTF-8 Autosave: off

- When external changes are made to the file you are working on, you can update it by selecting **File | Reload from Disk** from the main menu.
- Use the context menu for pasting or folding your code as well as switching to column selection mode.



README.md - .../jetbrains/dotty-example-project

```
1  # Example sbt project that
2    compiles using Dotty
3
4  [![Build Status](https://travis
5    -ci.org/lampepfl/dotty-example
6    -project.svg?branch=master
7    )](https://travis-ci.org
8    /lampepfl/dotty-example
9    -project)
```

Usage

This is a normal sbt project, you can compile code with `sbt compile` and run it with `sbt run`, `sbt console`

Example sbt project that compiles using Dotty

Context menu actions:

- Show Context Actions
- Open File in Project...
- Paste
- Paste without Formatting
- Column Selection Mode
- Folding
- Analyze
- Go To
- Reveal in Finder

4:22 LF UTF-8 Autosave: off Git: master

- Use the status bar to go to the line you need, check the VCS, or toggle the Autosave mode.

Turn on autosave

- Click **Autosave: off** on the Status bar and select the **Save changes automatically** in the

popup that opens.

Exit the LightEdit mode

You can quit the LightEdit mode and switch from editing a single file to working on the entire project. To do so, use one of the following ways:

Status bar

On the status bar of the LightEdit mode, click **Access full IDE** and from the list of options, select how you want to proceed such as open the current file in the project, open the recent project, or open the new one.



Find action window

Press **Alt+Enter** and select **Open file in project**.

Main menu

Select **File | Open File in Project** from the main menu.

Source code navigation

You can quickly navigate through code in the editor using different actions and popups. For the detailed information on navigating between the editor and tool windows, check the editor basics.

Navigate with the caret

- To navigate backwards, press `Ctrl+Alt+Left`. To navigate forward, press `Ctrl+Alt+Right`.
- To navigate to the last edited location, press `Ctrl+Shift+Backspace`.
- To find the current caret location in the editor, press `Ctrl+M`. This action might be helpful if you do not want to scroll through a large file.

However, you can press the `Up` and `Down` arrow keys to achieve the same result.

- To highlight a word at the caret you are trying to locate, select **Edit | Find | Next Occurrence of the Word at Caret** from the main menu.
- To see on what element the caret is currently positioned, press `Alt+Q`.
- To move caret between matching code block braces, press `Ctrl+Shift+M`.
- To navigate between code blocks, press `Ctrl+[` or `Ctrl+]`.

Move the caret

You can use different actions to move the caret through code. You can also configure where the caret should stop when moved by words and on line breaks.

- To move the caret to the next word or the previous word, press `Ctrl+Right` Or `Ctrl+Left`.

By default, IntelliJ IDEA moves the caret to the end of the current word.

When you move the caret to the previous word, the caret is placed in the beginning of the current word. You can configure the position of the caret when you use these actions.

In the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General**. In the Caret Movement section, use the **When moving by words** and **Upon line break** options to configure the caret's behavior.

- To move the caret forward to the next paragraph or backward to the previous one, press `Ctrl+Shift+A` and search for the **Move Caret Forward a Paragraph** or **Move Caret Backward a Paragraph** action.

You can also select a text and then move the caret forward or backward to a paragraph.

Press `Ctrl+Shift+A` and search for the **Move Caret Forward a Paragraph with Selection** or **Move Caret Backward a Paragraph with Selection** action.

If you need, you can assign shortcuts to these actions. Refer to Configure keyboard shortcuts for details.

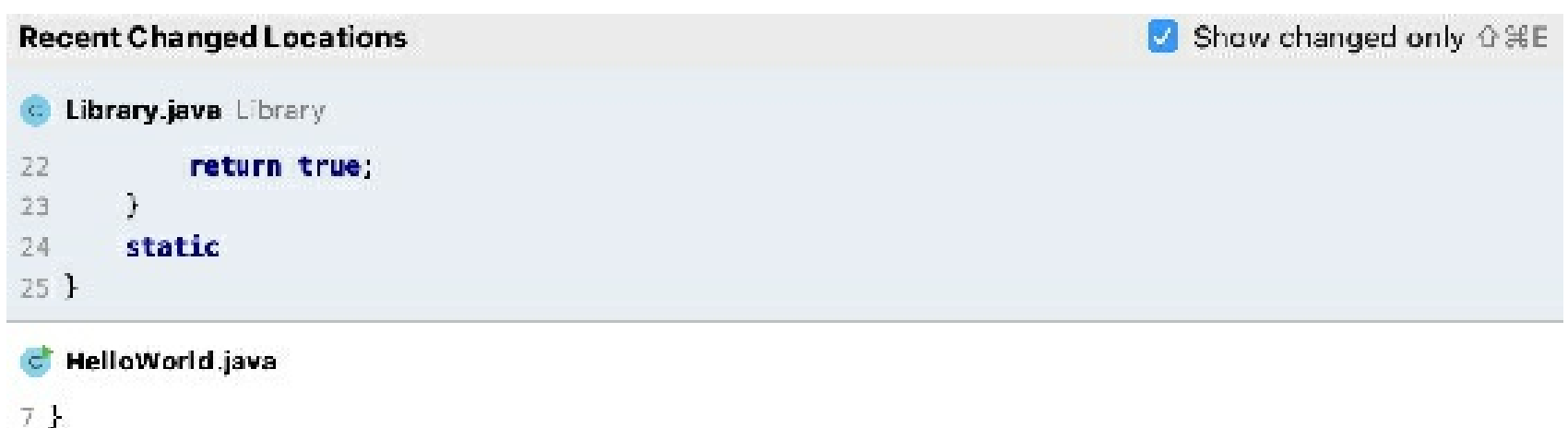
Find recent locations

You can also check your recently viewed or changed code using the **Recent Locations** popup.

- To open the **Recent Locations** popup, press `Ctrl+Shift+E`. The list starts with the latest visited location at the top and contains code snippets.



- While in the popup, use the same shortcut or select the **Show changed only** checkbox to see only the locations with changed code.



- To search for a code snippet, in the **Recent Locations** popup, start typing your search query. You can search by the code text, filename, or breadcrumbs.



- To delete a location entry from the search results, press either `Delete` or `Backspace`.

Keep in mind that the deleted location is also removed from the list of entries that you access with the `Ctrl+Alt+Left` shortcut.

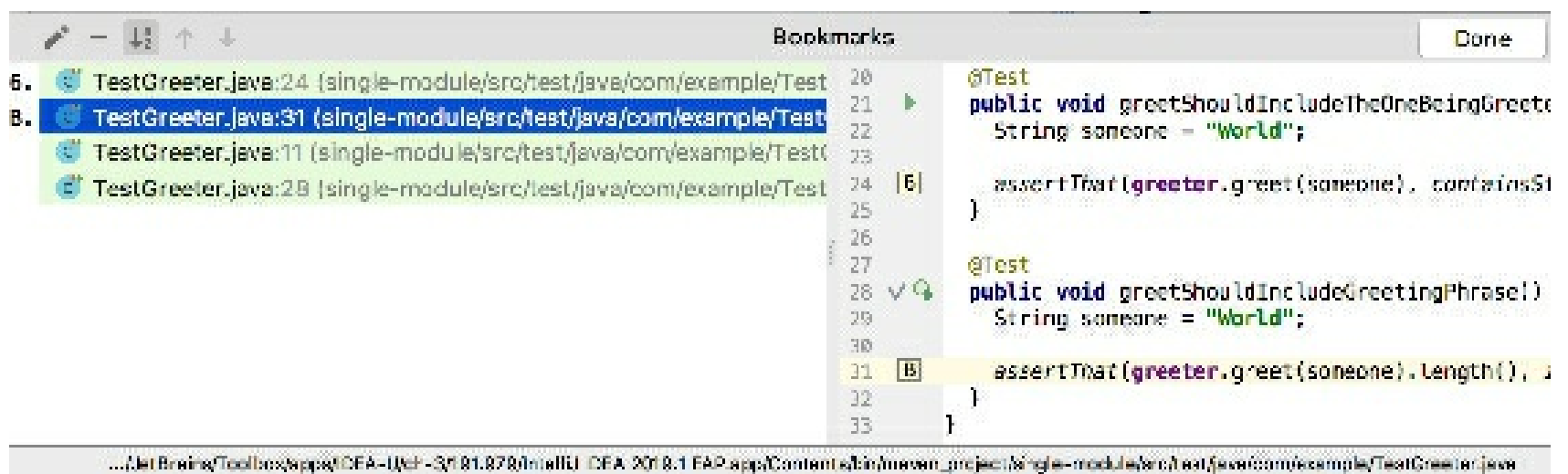
Use bookmarks for navigation

- To create an anonymous bookmark, place the caret at the needed code line and press `F11`.
- To create a bookmark with mnemonics, place the caret at the needed code line,

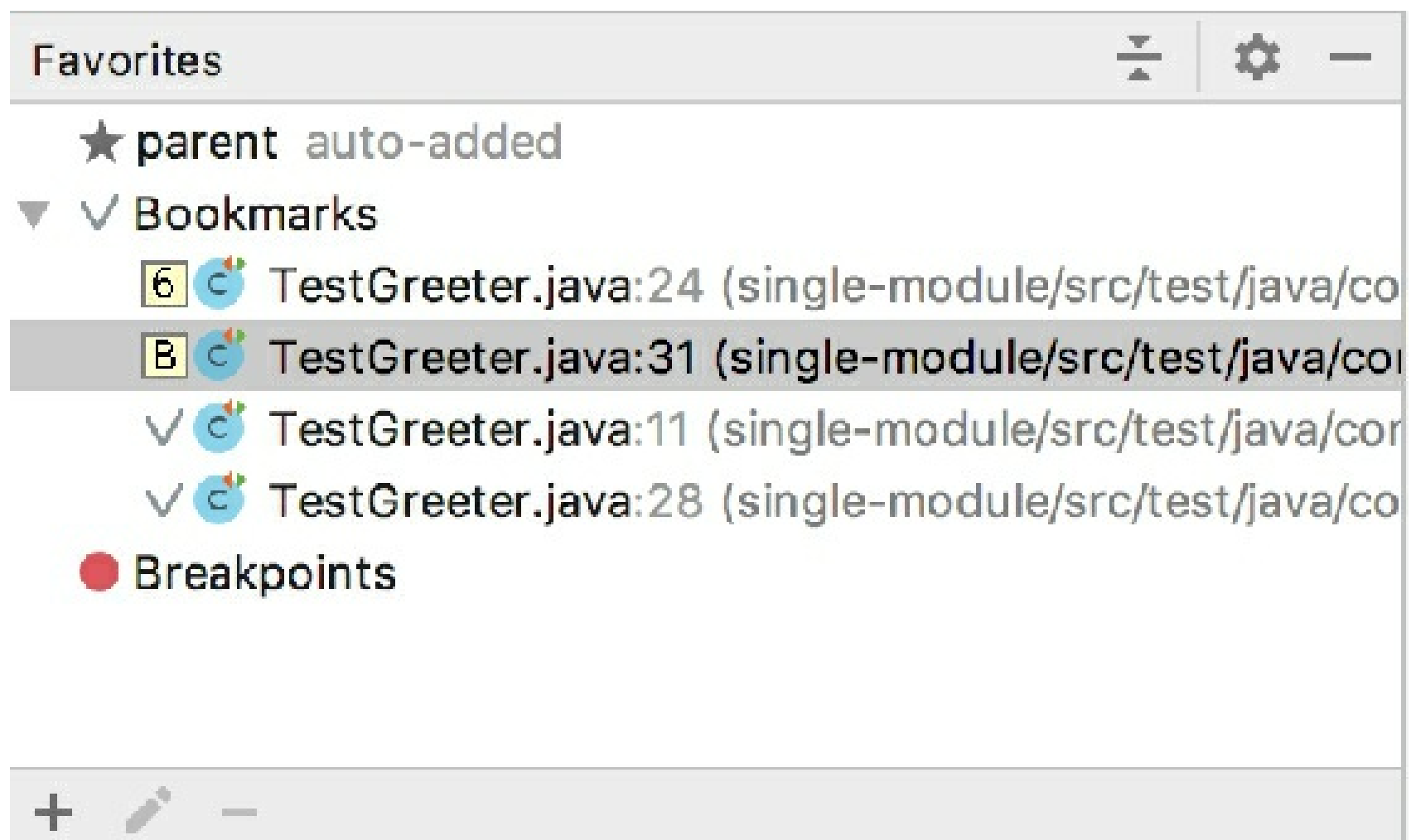
press Ctrl+F11 and select a number or a letter for the mnemonics.



- To show the next or the previous bookmark, in the main menu select **Navigate | Bookmarks | Next Bookmark** or **Navigate | Bookmarks | Previous Bookmark**.
- To open the **Bookmarks** dialog, press Shift+F11. You can use this dialog to manage bookmarks, for example, delete, sort bookmarks, or supply them with a brief description.



- To navigate to an existing bookmark with letter mnemonics, press Shift+F11 and then press a letter you need. IntelliJ IDEA returns you to the editor and to the corresponding bookmark.
- To navigate to an existing bookmark with number mnemonics, press Ctrl and the bookmark's number.
- Every created bookmark is reflected in the Favorites Alt+2 (**View | Tool Windows | Favourites**) tool window which you can also use for navigation to your bookmarks.



Navigate between changes

If you edit a file that is under version control, IntelliJ IDEA provides several ways to move back and forth with the updates. In particular, you can use the navigation commands, keyboard shortcuts, and the change markers.

- Press `Ctrl+Alt+Shift+Down` / `Ctrl+Alt+Shift+Up`.
- From the main menu, choose **Navigate | Next / Previous Change**.
- Click a change marker, and then click `or` .

To navigate to the place of your last edit, press `Ctrl+Shift+Backspace` or select **Navigate | Last Edit Location** from the main menu.

See recent changes

You can use the **Recent Changes** popup to see a list of files that were changed either locally or externally in your project. If necessary, you can revert those changes.

1. From the main menu, select **View | Recent Changes** `Alt+Shift+C`.

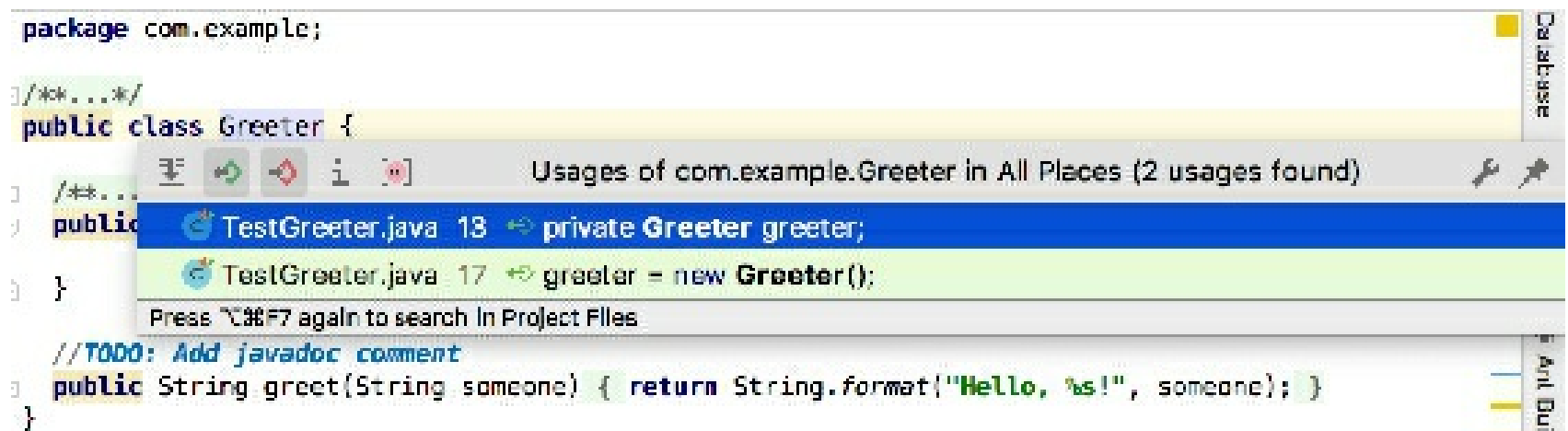
Recent Changes	
Creating class com.destinationpackage.MainActivity	2019-07-24 20:17
External change	2019-07-24 20:17
Groovy Class	2019-07-24 19:52
External change	2019-07-24 19:49
External change	2019-07-24 19:48
External change	2019-07-24 18:40
Un-inject language/reference	2019-07-24 18:14
Create File From Template	2019-07-24 18:09
External change	2019-07-24 16:47
Scala Class	2019-07-24 16:42
External change	2019-07-23 10:41
External change	2019-07-22 21:40
Remove orphan modules after import	2019-07-22 21:31
Remove orphan modules after import	2019-07-22 21:31
Renaming directory blah to module2	2019-07-22 21:31

2. In the **Recent Changes** popup, select a file you need and press `Enter` to open it in a separate dialog where you can check what was changed and revert those changes if necessary.

You can navigate to the initial declaration of a symbol and symbol's type from its usage.

Go to declaration and its type

- Place the caret at the desired symbol and press `Ctrl+B`.

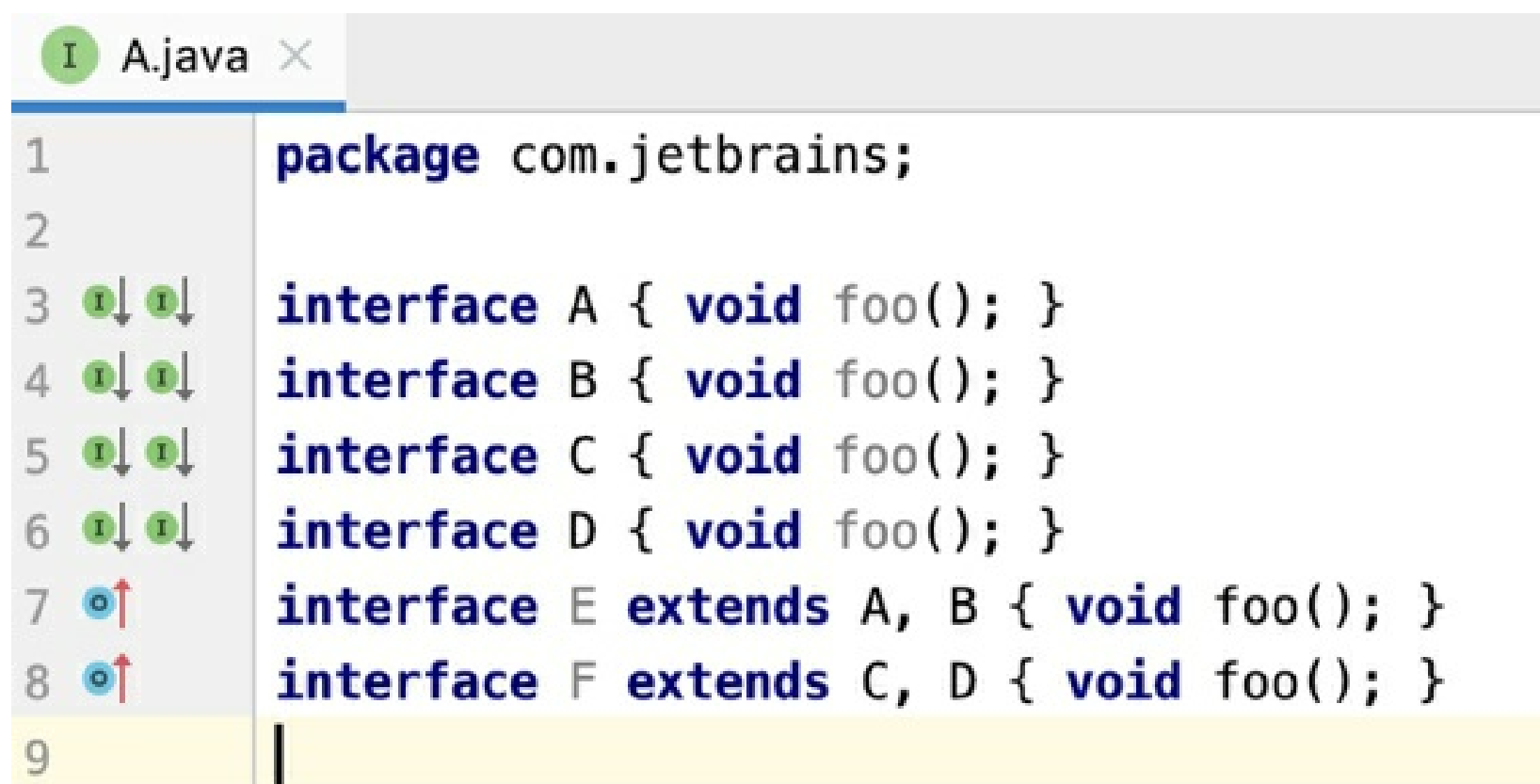


- For a type declaration, press `Ctrl+Shift+B`.

Go to implementation

You can keep track of class implementations and overriding methods either using the gutter icons in the editor or pressing the appropriate shortcuts.

- Click one of the `/`, `/` gutter icons located in the editor and select the desired ascendant or descendant class from the list.



- To navigate to the super method, press `Ctrl+U`.
- To navigate to the implementation, press `Ctrl+Alt+B`.

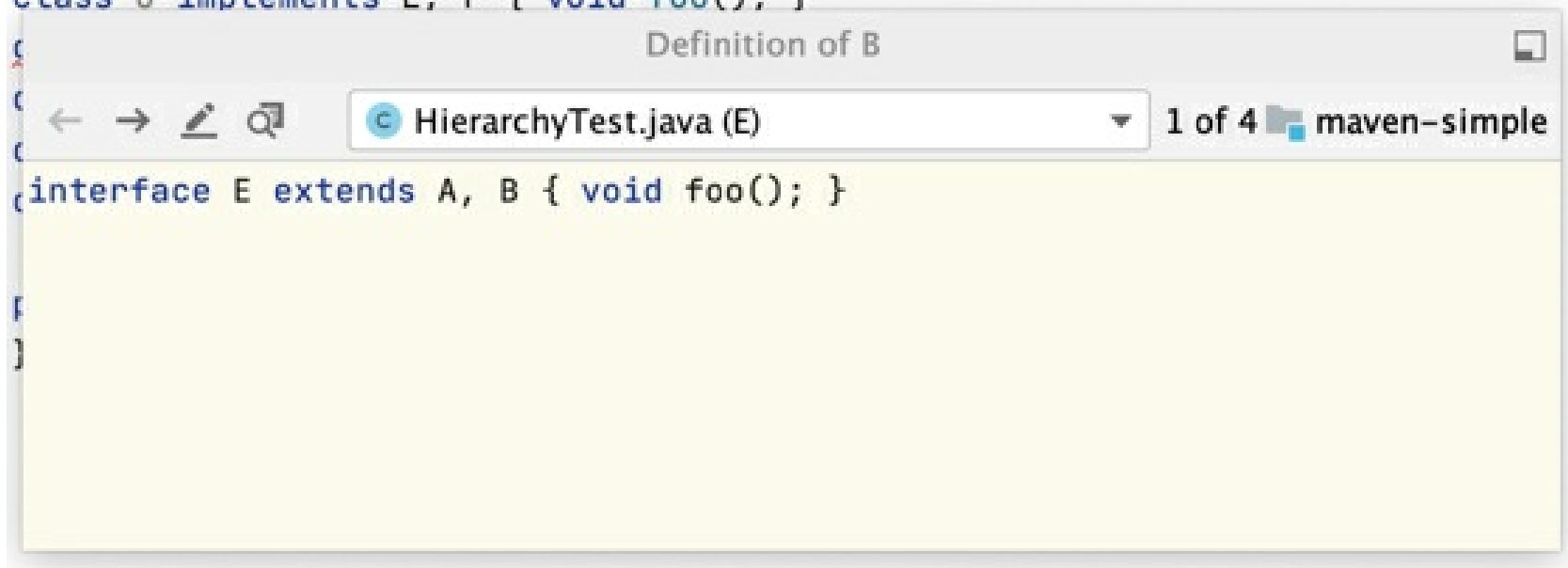
Show siblings

You can view the implementations of methods in the neighboring classes in a separate popup.

1. In the editor, place the caret at the method's name.
2. From the main menu, select **View | Show Siblings**.

IntelliJ IDEA opens a popup where you can browse through the implementations, navigate to source, edit code, and open the list in the **Find** tool window.

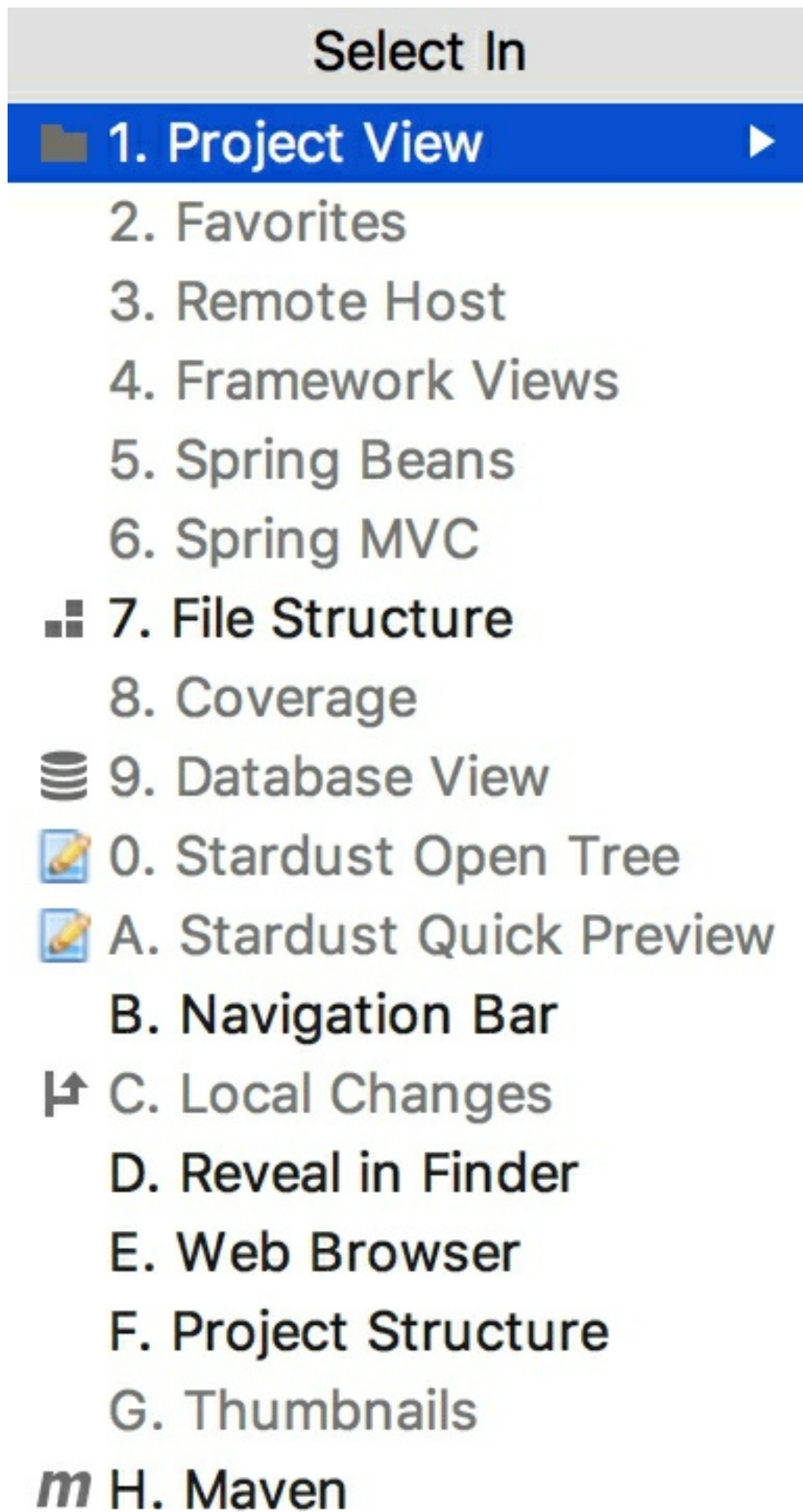
```
interface A { void foo(); }
interface B { void foo(); }
interface C { void foo(); }
interface D { void foo(); }
interface E extends A, B { void foo(); }
interface F extends C, D { void foo(); }
class G implements E, F { void foo(); }
```



Navigate with the Select In popup

You can automatically locate a class in the **Project** tool window.

1. If the class is opened in the editor, press `Alt+F1` to open the **Select In** popup.

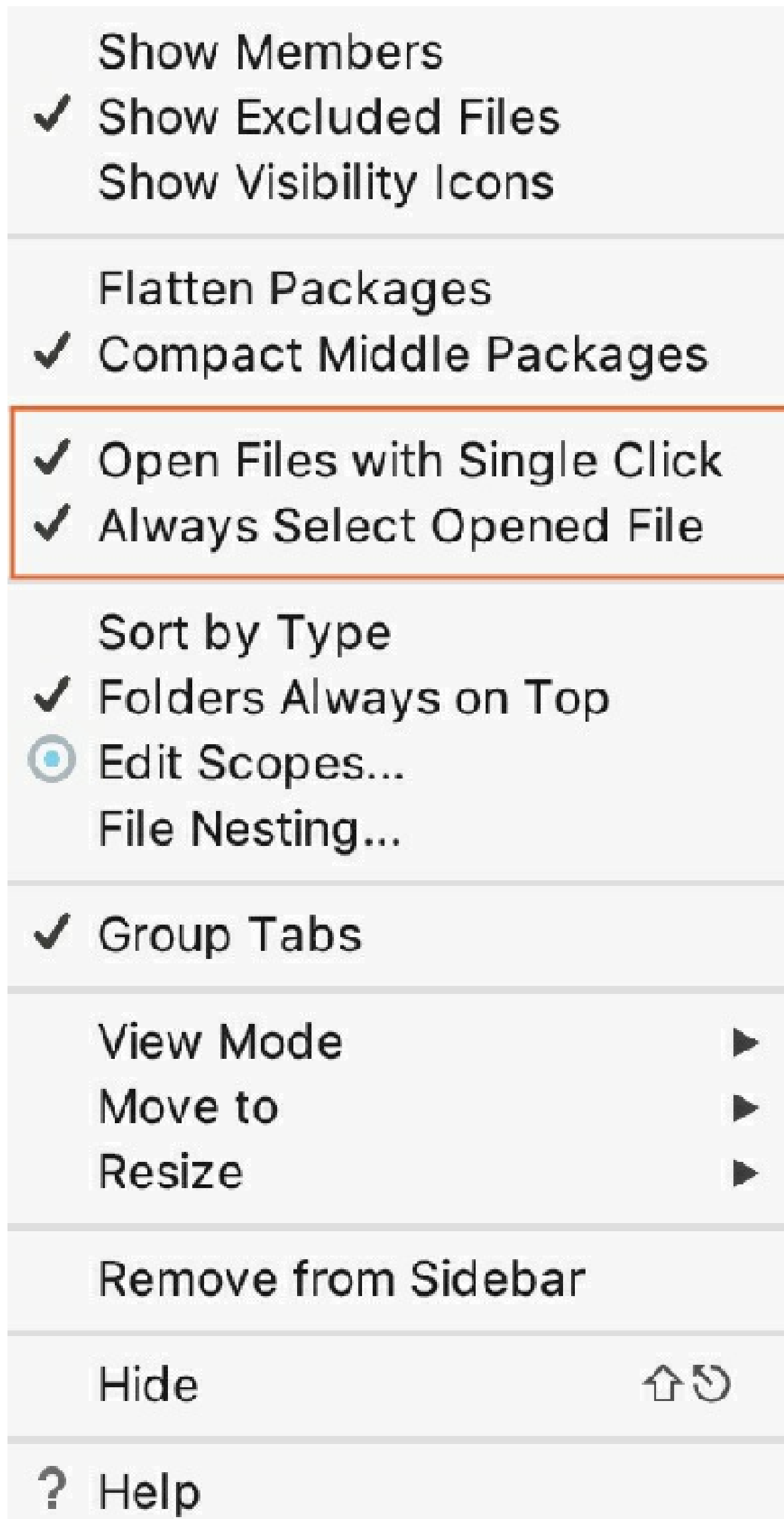


2. In the popup, select **Project View** and press Enter. IntelliJ IDEA locates your target in the **Project** tool window.

Locate a file in the Project tool window

You can use the **Open Files with Single Click** (previously called *Autoscroll to Source*) and **Always Select Opened Files** (previously called *Autoscroll from Source*) actions to locate your file in the **Project** tool window.

1. In the **Project** tool window, right-click the **Project** toolbar and from the context menu select **Always Select Opened File**. After that IntelliJ IDEA will track the file that is currently opened in the active editor tab and locate it in the **Project** tool window automatically.



2. You can also select the **Open Files with Single Click** option. In this case, when you click a file in the **Project** view, IntelliJ IDEA will automatically open it in the editor.

Navigate between errors or warnings

- To jump to the next or previous found issue in your code, press F2 or Shift+F2 respectively.

Alternatively, from the main menu, select **Navigate | Next / Previous Highlighted Error**.

IntelliJ IDEA places the caret immediately before the code issue.

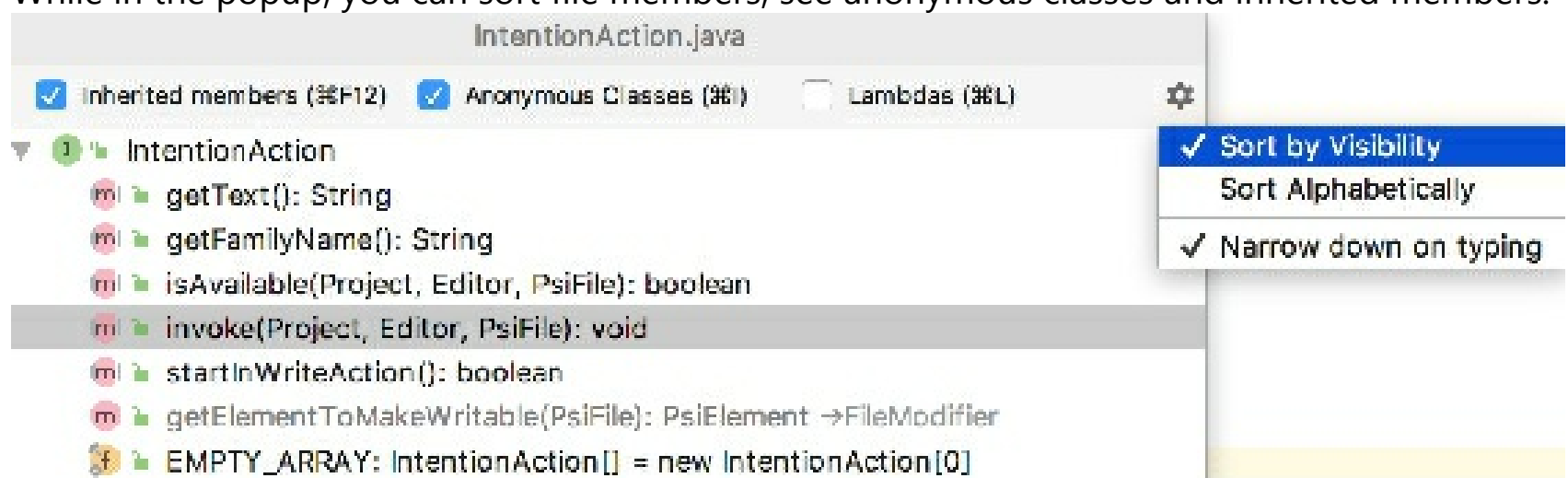
- Configure the way IntelliJ IDEA navigates between code issues: it can either jump between all code issues or skip minor issues and only navigate between detected errors. Right-click the code analysis marker in the scroll bar area and choose one of the available navigation modes from the context menu:
 - To have IntelliJ IDEA skip warnings, infos, and other minor issues, choose **Go to high priority problems only**.
 - To have IntelliJ IDEA jump between all detected code issues, choose **Go to next problem**.

Locate a code element with the Structure view popup

You can use the structure view popup to locate a code element in the file you are working on.

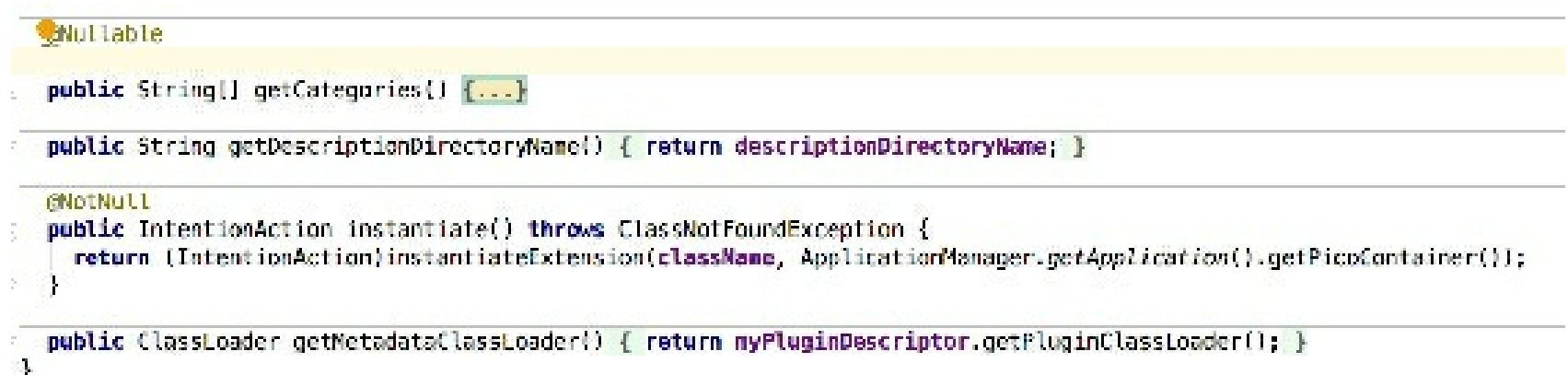
1. To open the structure view popup, press `Ctrl+F12`.
2. In the popup, locate an item you need. You can start typing a name of the element for IntelliJ IDEA to narrow down the search. Press `Enter` to return to the editor and the corresponding element.

While in the popup, you can sort file members, see anonymous classes and inherited members.



Browse through methods

- Press `Alt+Down` Or `Alt+Up`.
- To visually separate methods in code, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Appearance** and select the **Show method separators** option.



- To open the **Structure** tool window, press `Alt+7`.

Use the Lens mode

The lens mode lets you preview your code without actually scrolling to it. The mode is available in the editor by default when you hover your mouse over the scrollbar. It is especially useful when you hover over a warning or an error message.

- To disable the lens mode, right-click the code analysis marker located on the right side of the editor and in the context menu clear the **Show code lens on the scrollbar hover** checkbox.
- As an alternative, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Appearance** and clear the **Show code lens on the scrollbar hover** checkbox.

Use breadcrumbs for navigation

You can navigate through the source code with breadcrumbs that show names of classes, variables, functions, methods, and tags in the currently opened file. By default, breadcrumbs are enabled and displayed at the bottom of the editor.

- To change the location of breadcrumbs, right-click a breadcrumb, in the context menu select **Breadcrumbs** and the location preference.
- To edit the breadcrumbs settings, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Breadcrumbs**.

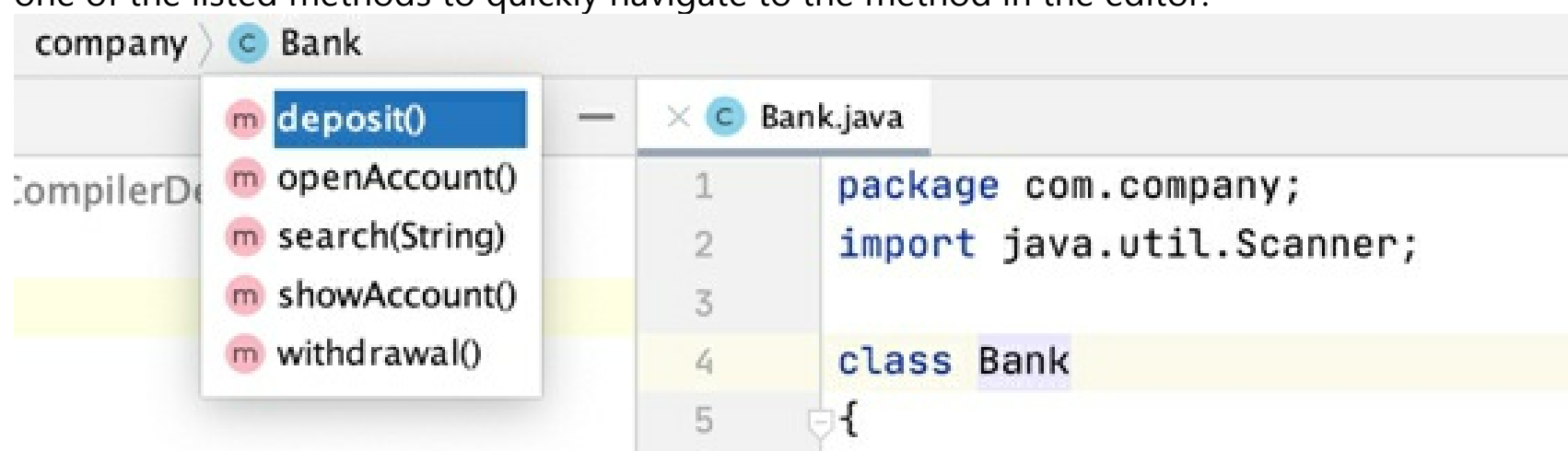
Clear the **Show breadcrumbs** option to hide breadcrumbs in the editor.

Navigate to a file with the Navigation bar

Use the Navigation bar as a handy tool to find your way across the project.

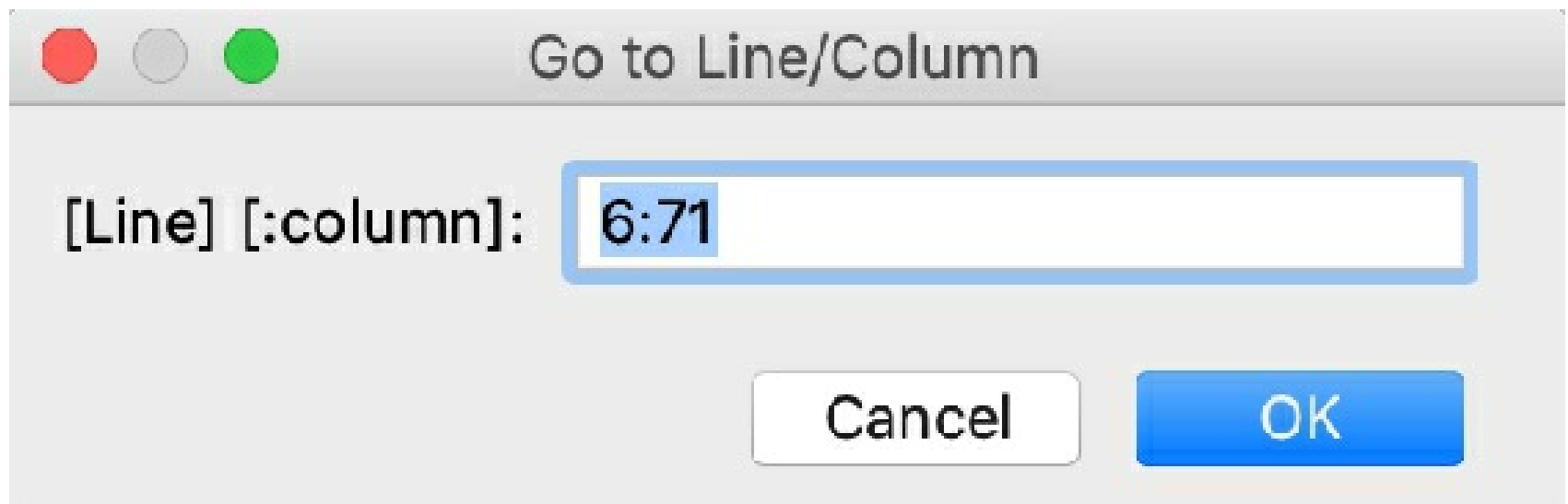
1. Press `Alt+Home` to activate the Navigation bar.
2. Use the arrow keys or the mouse pointer to locate the desired file.
3. Double-click the selected file, or press `Enter` to open it in the editor.

You can click a Java class or an interface in the navigation bar to see the list of methods. Click one of the listed methods to quickly navigate to the method in the editor.



Find a line or a column

1. In the editor, press `Ctrl+G`.
2. In the **Go to Line/Column** dialog, specify a line or column number, or both, separating them with `:` and click **OK**.

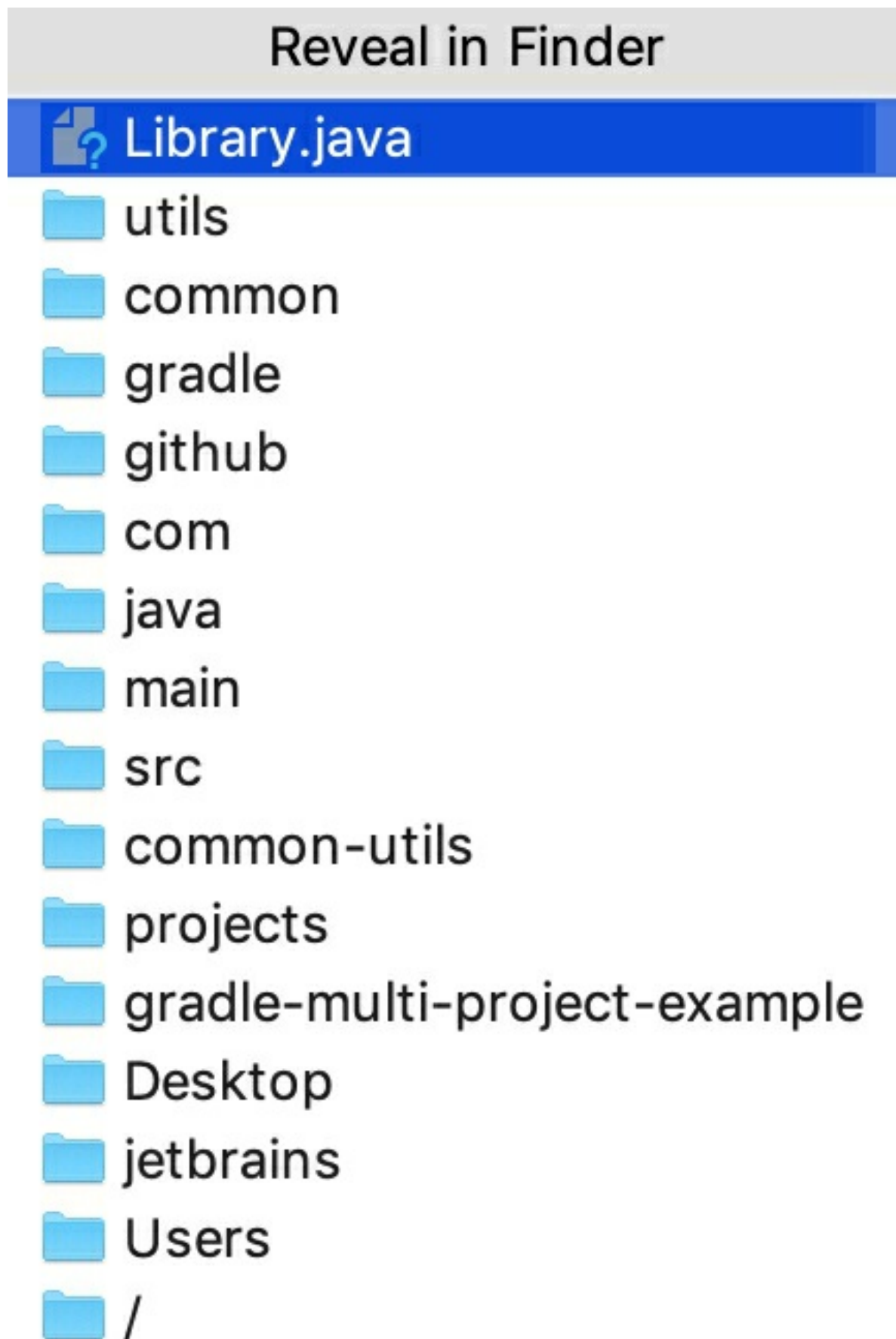


3. If you don't want to see the line numbers in the editor, in the **Settings/Preferences** dialog `Ctrl+Alt+S`, go to **Editor | General | Appearance** and clear the **Show line numbers** checkbox.

For quicker access, assign a shortcut to the **Show line numbers** action.

Find a file path

1. In the editor, press `Ctrl+Alt+F12` or in the context menu, select **Open in | Finder**.
2. In the **Reveal in Finder** popup, select a file or a directory to open in a path finder and press `Enter`.



Find recent files

You can search for the recent and recently edited files with the **Recent Files** popup.

- To open the **Recent Files** popup with the list of recent files, press **Ctrl+E**.

Recent Files

Build

Project

Favorites

TODO

Structure

Version Control

Ant Build

Bean Validation

Database

Event Log

Gradle

Java Enterprise

Learn

Maven

Spring

Terminal

Web

* HelloWorld.java

* gradle-multi-project-example

* LibraryTest.java

- To see only the recently edited files, press `Ctrl+E` again or select the **Show changed only** checkbox.
- To search for items in the popup, use the *Speed Search* functionality. Just start typing a search query, and the **Search for** field appears. IntelliJ IDEA displays the results based on your search query, the list shrinks as you type.

Search for: java

Java Enterprise

HelloWorld.java

LibraryTest.java

Source code hierarchy

With IntelliJ IDEA, you can examine the hierarchy of classes, methods, and calls and explore the structure of source files.

Build hierarchies

- *Type* hierarchies show parent and child classes of a class.
- *Method* hierarchies show subclasses where the method overrides the selected one as well as superclasses or interfaces where the selected method gets overridden.

In the hierarchy tree, IntelliJ IDEA displays  to indicate subclasses that are not abstract but don't have the method defined in them. When a method is not defined in a class, but is defined in the superclass, IntelliJ IDEA displays .

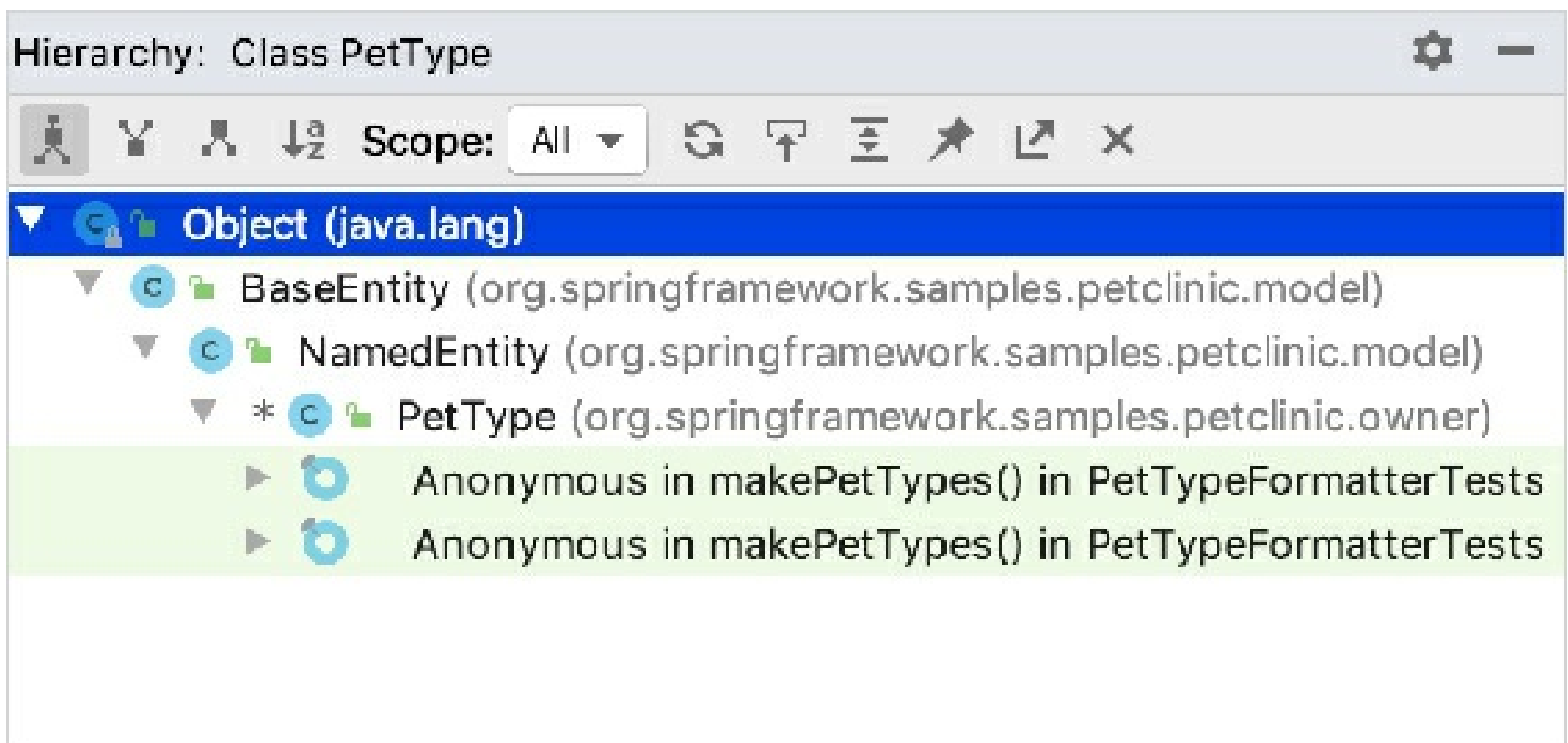
- *Call* hierarchies show callers (supertypes) or callees (subtypes) of a method.

If you invoke the call hierarchy on a field, it will show you the list of the methods where the selected field is used.

When built, a hierarchy can be immediately viewed and examined in the **Hierarchy** tool window. By default, every new built hierarchy overwrites the contents of the current tab. You can retain the current tab and have the next hierarchy built in a new one.

Build a type hierarchy

1. Select the desired class in the **Project** tool window or open it in the editor.
2. From the main menu, select **Navigate | Type Hierarchy** or just press `Ctrl+H`.



Build a method hierarchy

1. Open the file in the editor and place the caret at the declaration of the desired method.

Alternatively, select the desired method in the **Project** tool window.

2. From the main menu, select **Navigate | Method Hierarchy** or press `Ctrl+Shift+H`.

Build a call hierarchy

1. Open the file in the editor and place the caret at the declaration or usage of the desired method or a field.

Alternatively, select the desired method or the field in the **Project** tool window.

2. From the main menu, select **Navigate | Call Hierarchy** or press `Ctrl+Alt+H`.

Retain a hierarchy tab

- In the **Hierarchy** tool window, click the **Pin Tab** button on the toolbar.

View hierarchies

Open the Hierarchy tool window

1. Make sure, you have already built hierarchies to show, see Building hierarchies above.
2. Select **View | Tool Windows | Hierarchy** from the main menu.

Navigate between the tabs

- Click the currently displayed tab, and select the next one to display from the list.

Toggle between views

- With IntelliJ IDEA, you can build and explore ascending or descending hierarchies, that is, callee or caller methods, parent or children classes, and so on.

Click  or  to show caller methods or callee methods respectively.

Hierarchy tool window buttons

Item	Description	Available In
	Shows both the parent and child classes of the selected class, which is marked with an arrow in the tree of results. For interfaces, this button is disabled.	Class hierarchies
	Depending on the hierarchy type: • Class hierarchies: shows the hierarchy of each supertype of the current class. • Call hierarchies: shows the callers of the selected method.	Class hierarchies Call hierarchies
	Depending on the hierarchy type: • Class hierarchies: shows all classes that extend the selected class. • Call hierarchies: shows the callees of the selected method.	Class hierarchies Call hierarchies
	Sorts all elements within a tree alphabetically.	All hierarchies
Scope	Use this list to limit the scope of the current hierarchy: • Project: traces usages of the method across the project. • Test: traces usages of the method across the test classes.	Call hierarchies

	• All: traces usages of the method across the project and the libraries. • This class: limits the scope to the current class. In addition to the preconfigured scopes, you can define your own one. To define a scope, select Configure from the list and define the required scope in the Scopes dialog.	
In a method hierarchy, the tree views of the following classes are available: • : the method is defined. • : the method is defined only in the superclass. • : the method must be defined because the class is not abstract.		
	Shows all updated classes or class structures.	All hierarchies
	Moves to a file and a section in a source code that corresponds to the selected node in the hierarchy tree.	All hierarchies
	Expands all nodes.	All hierarchies
	Locks the current tab from closing and reusing. Results of the next command are displayed in a new tab.	All hierarchies
	Exports a hierarchy into a text file. You can specify a location for this file.	All hierarchies
	Closes the tool window.	All hierarchies

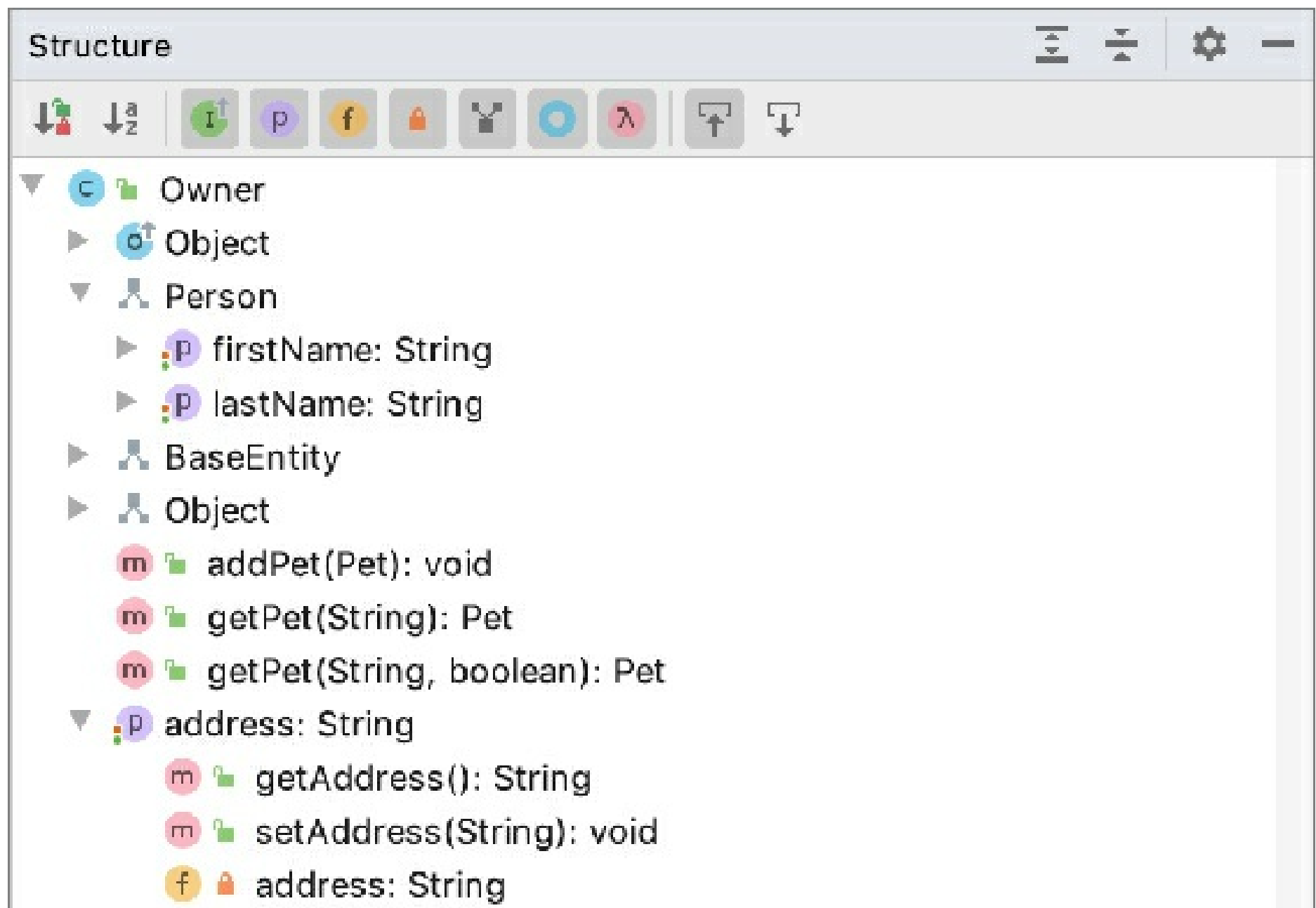
Source file structure

The **Structure** tool window: **View | Tool Windows | Structure** or **Alt+7**

The **Structure** popup: **Navigate | File Structure** or **Ctrl+F12**

By default, IntelliJ IDEA shows all classes, methods, and other elements of the current file. To toggle the elements you want to show, click the corresponding buttons on the Structure tool window toolbar. For example:

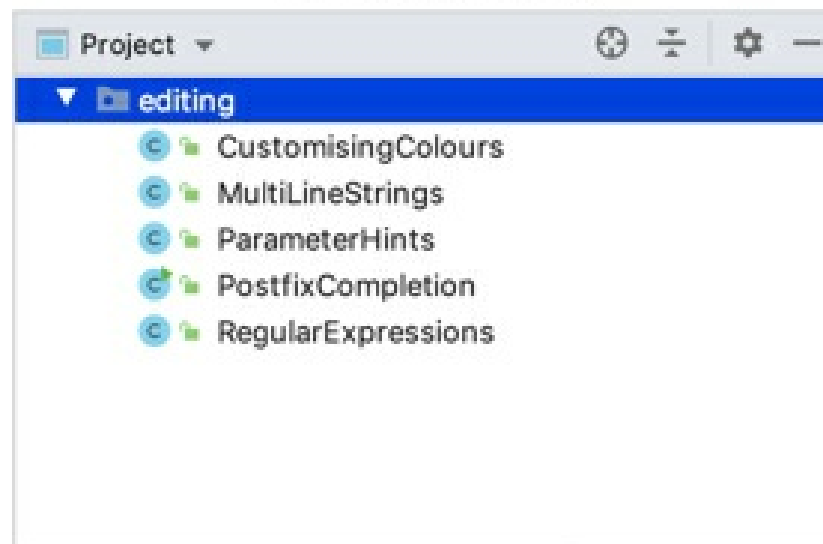
- Click to show class fields.
- Click to show inherited members.
- Click to show lambdas.



Show class members in the Project tool window

- Right-click the **Project** tool window title bar and select **Show members** from the context menu.

Show Members: OFF



Show Members: ON

